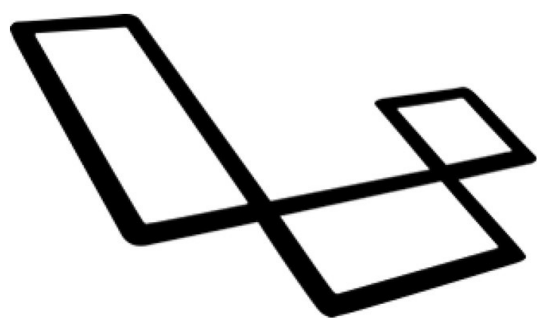




Laravel

7.x

DOCUMENTATION

Laravel Documentation - 7.x

<https://laravel.com/docs/>

eBook compiled from the source

<https://github.com/laravel/docs/>

Generated with raravel-2026

Get the latest version at <https://github.com/driade/laravel-book>

Date: Friday, 15-May-26 01:00:14 UTC

Contents

Prologue

[Release Notes](#)
[Upgrade Guide](#)
[Contribution Guide](#)

Getting Started

[Installation](#)
[Configuration](#)
[Directory Structure](#)
[Homestead](#)
[Valet](#)
[Deployment](#)

Architecture Concepts

[Request Lifecycle](#)
[Service Container](#)
[Service Providers](#)
[Facades](#)
[Contracts](#)

The Basics

[Routing](#)
[Middleware](#)
[CSRF Protection](#)
[Controllers](#)
[Requests](#)
[Responses](#)
[Views](#)
[URL Generation](#)
[Session](#)
[Validation](#)
[Error Handling](#)
[Logging](#)

Frontend

[Blade Templates](#)
[Localization](#)
[Frontend Scaffolding](#)
[Compiling Assets](#)

Security

[Authentication](#)
[Authorization](#)
[Email Verification](#)
[Encryption](#)
[Hashing](#)
[Password Reset](#)

Digging Deeper

[Artisan Console](#)
[Broadcasting](#)
[Cache](#)

[Collections](#)
[Events](#)
[File Storage](#)
[Helpers](#)
[HTTP Client](#)
[Mail](#)
[Notifications](#)
[Package Development](#)
[Queues](#)
[Task Scheduling](#)

Database

[Getting Started](#)
[Query Builder](#)
[Pagination](#)
[Migrations](#)
[Seeding](#)
[Redis](#)

Eloquent ORM

[Getting Started](#)
[Relationships](#)
[Collections](#)
[Mutators](#)
[API Resources](#)
[Serialization](#)

Testing

[Getting Started](#)
[HTTP Tests](#)
[Console Tests](#)
[Browser Tests](#)
[Database](#)
[Mocking](#)

Official Packages

[Cashier \(Stripe\)](#)
[Cashier \(Paddle\)](#)
[Dusk](#)
[Envoy](#)
[Horizon](#)
[Passport](#)
[Sanctum](#)
[Scout](#)
[Socialite](#)
[Telescope](#)

Prologue

Release Notes

Release Notes

- [Versioning Scheme](#)
- [Support Policy](#)
- [Laravel 7](#)

Versioning Scheme

Laravel and its other first-party packages follow [Semantic Versioning](#). Major framework releases are released every six months (~February and ~August), while minor and patch releases may be released as often as every week. Minor and patch releases should **never** contain breaking changes.

When referencing the Laravel framework or its components from your application or package, you should always use a version constraint such as `^7.0`, since major releases of Laravel do include breaking changes. However, we strive to always ensure you may update to a new major release in one day or less.

Support Policy

For LTS releases, such as Laravel 6, bug fixes are provided for 2 years and security fixes are provided for 3 years. These releases provide the longest window of support and maintenance. For general releases, bug fixes are provided for 6 months and security fixes are provided for 1 year. For all additional libraries, including Lumen, only the latest release receives bug fixes. In addition, please review the database versions [supported by Laravel](#).

Version	Release	Bug Fixes Until	Security Fixes Until
6 (LTS)	September 3rd, 2019	September 3rd, 2021	September 3rd, 2022
7	March 3rd, 2020	September 10th, 2020	March 3rd, 2021
8	September 8th, 2020	March 8th, 2021	September 8th, 2021

Laravel 7

Laravel 7 continues the improvements made in Laravel 6.x by introducing Laravel Sanctum, routing speed improvements, custom Eloquent casts, Blade component tags, fluent string operations, a developer focused HTTP client, first-party CORS support, improved scoping for route model binding, stub customization, database queue improvements, multiple mail drivers, query-time casts, a new `artisan test` command, and a variety of other bug fixes and usability improvements.

Laravel Sanctum

Laravel Sanctum was built by [Taylor Otwell](#).

Laravel Sanctum provides a featherweight authentication system for SPAs (single page applications), mobile applications, and simple, token based APIs. Sanctum allows each user of your application to generate multiple API tokens for their account. These tokens may be granted abilities / scopes which specify which actions the tokens are allowed to perform.

For more information on Laravel Sanctum, consult the [Sanctum documentation](#).

Custom Eloquent Casts

Custom Eloquent casts was contributed by [Taylor Otwell](#).

Laravel has a variety of built-in, helpful cast types; however, you may occasionally need to define your own

cast types. You may now accomplish this by defining a class that implements the `castsAttributes` interface.

Classes that implement this interface must define a `get` and `set` methods. The `get` method is responsible for transforming a raw value from the database into a cast value, while the `set` method should transform a cast value into a raw value that can be stored in the database. As an example, we will re-implement the built-in `json` cast type as a custom cast type:

```
<?php

namespace App\Casts;

use Illuminate\Contracts\Database\Eloquent\CastsAttributes;

class Json implements CastsAttributes
{
    /**
     * Cast the given value.
     *
     * @param \Illuminate\Database\Eloquent\Model $model
     * @param string $key
     * @param mixed $value
     * @param array $attributes
     * @return array
     */
    public function get($model, $key, $value, $attributes)
    {
        return json_decode($value, true);
    }

    /**
     * Prepare the given value for storage.
     *
     * @param \Illuminate\Database\Eloquent\Model $model
     * @param string $key
     * @param array $value
     * @param array $attributes
     * @return string
     */
    public function set($model, $key, $value, $attributes)
    {
        return json_encode($value);
    }
}
```

Once you have defined a custom cast type, you may attach it to a model attribute using its class name:

```
<?php

namespace App;

use App\Casts\Json;
use Illuminate\Database\Eloquent\Model;

class User extends Model
{
    /**
     * The attributes that should be cast to native types.
     *
     * @var array
     */
    protected $casts = [
        'options' => Json::class,
    ];
}
```

To learn how to write custom Eloquent casts, including custom casts that cast to value objects, please consult the [Eloquent documentation](#).

Blade Component Tags & Improvements

Blade component tags were contributed by [Spatie](#), [Marcel Pociot](#), [Caleb Porzio](#), [Dries Vints](#), and [Taylor Otwell](#).

TIP Blade components have been overhauled to allow tag based rendering, attribute management, component classes, inline view components, and more. Since the overhaul of Blade components is so

extensive, please consult the [full Blade component documentation](#) to learn about this feature.

In summary, a component may now have an associated class which specifies the data it accepts. All public properties and methods defined on the component class will automatically be made available to the component view. Any additional HTML attributes specified on the component may be managed using the automatically included `$attributes` variable, which is an attribute bag instance.

In this example, we will assume that an `App\View\Components\Alert` component has been defined like so:

```
<?php

namespace App\View\Components;

use Illuminate\View\Component;

class Alert extends Component
{
    /**
     * The alert type.
     *
     * @var string
     */
    public $type;

    /**
     * Create the component instance.
     *
     * @param string $type
     * @return void
     */
    public function __construct($type)
    {
        $this->type = $type;
    }

    /**
     * Get the class for the given alert type.
     *
     * @return string
     */
    public function classForType()
    {
        return $this->type == 'danger' ? 'alert-danger' : 'alert-warning';
    }

    /**
     * Get the view / contents that represent the component.
     *
     * @return \Illuminate\View\View|string
     */
    public function render()
    {
        return view('components.alert');
    }
}
```

And, assuming the component's Blade template has been defined like so:

```
<!-- /resources/views/components/alert.blade.php -->

<div class="alert {{ $classForType }}" {{ $attributes }}>
    {{ $heading }}

    {{ $slot }}
</div>
```

The component may be rendered in another Blade view using the component's tag:

```
<x-alert type="error" class="mb-4">
    <x-slot name="heading">
        Alert content...
    </x-slot>

    Default slot content...
</x-alert>
```

As mentioned, this is just a very small sample of the functionality of the Blade component overhaul in Laravel 7 and does not demonstrate anonymous components, inline view components, and a variety of other features. Please consult the [full Blade component documentation](#) to learn about this feature.

NOTE The previous `@component` syntax for Blade components has not and will not be removed.

HTTP Client

The HTTP client is a wrapper of Guzzle and was contributed by [Adam Wathan](#), [Jason McCreary](#), and [Taylor Otwell](#).

Laravel now provides an expressive, minimal API around the [Guzzle HTTP client](#), allowing you to quickly make outgoing HTTP requests to communicate with other web applications. Laravel's wrapper around Guzzle is focused on its most common use cases and a wonderful developer experience. For example, the client makes it a breeze to POST and interface with JSON data:

```
use Illuminate\Support\Facades\Http;

$response = Http::withHeaders([
    'X-First' => 'foo',
    'X-Second' => 'bar'
])->post('http://test.com/users', [
    'name' => 'Taylor',
]);

return $response['id'];
```

In addition, the HTTP client provides fantastic, ergonomic testing functionality:

```
Http::fake([
    // Stub a JSON response for GitHub endpoints...
    'github.com/*' => Http::response(['foo' => 'bar'], 200, ['Headers']),

    // Stub a string response for Google endpoints...
    'google.com/*' => Http::response('Hello World', 200, ['Headers']),

    // Stub a series of responses for Facebook endpoints...
    'facebook.com/*' => Http::sequence()
        ->push('Hello World', 200)
        ->push(['foo' => 'bar'], 200)
        ->pushStatus(404),
]);
```

To learn more about all of the features of the HTTP client, please consult the [HTTP client documentation](#).

Fluent String Operations

Fluent string operations were contributed by [Taylor Otwell](#).

You are likely familiar with Laravel's existing `Illuminate\Support\Str` class, which provides a variety of helpful string manipulation functions. Laravel 7 now offers a more object-oriented, fluent string manipulation library built on top of these functions. You may create a fluent `Illuminate\Support\Stringable` object using the `Str::of` method. A variety of methods may then be chained onto the object to manipulate the string:

```
return (string) Str::of(' Laravel Framework 6.x ')
    ->trim()
    ->replace('6.x', '7.x')
    ->slug();
```

For more information on the methods available via fluent string manipulation, please consult its [full documentation](#).

Route Model Binding Improvements

Route model binding improvements were contributed by [Taylor Otwell](#).

Key Customization

Sometimes you may wish to resolve Eloquent models using a column other than `id`. To do so, Laravel 7 allows you to specify the column in the route parameter definition:

```
Route::get('api/posts/{post:slug}', function (App\Post $post) {  
    return $post;  
});
```

Automatic Scoping

Sometimes, when implicitly binding multiple Eloquent models in a single route definition, you may wish to scope the second Eloquent model such that it must be a child of the first Eloquent model. For example, consider this situation that retrieves a blog post by slug for a specific user:

```
use App\Post;  
use App\User;  
  
Route::get('api/users/{user}/posts/{post:slug}', function (User $user, Post $post) {  
    return $post;  
});
```

When using a custom keyed implicit binding as a nested route parameter, Laravel 7 will automatically scope the query to retrieve the nested model by its parent using conventions to guess the relationship name on the parent. In this case, it will be assumed that the `user` model has a relationship named `posts` (the plural of the route parameter name) which can be used to retrieve the `Post` model.

For more information on route model binding, please consult the [routing documentation](#).

Multiple Mail Drivers

Multiple mail driver support was contributed by [Taylor Otwell](#).

Laravel 7 allows the configuration of multiple "mailers" for a single application. Each mailer configured within the `mail` configuration file may have its own options and even its own unique "transport", allowing your application to use different email services to send certain email messages. For example, your application might use Postmark to send transactional mail while using Amazon SES to send bulk mail.

By default, Laravel will use the mailer configured as the `default` mailer in your `mail` configuration file. However, you may use the `mailer` method to send a message using a specific mailer configuration:

```
Mail::mailer('postmark')  
    ->to($request->user())  
    ->send(new OrderShipped($order));
```

Route Caching Speed Improvements

The route caching speed improvements were contributed by upstream [Symfony](#) contributors and [Dries Vints](#).

Laravel 7 includes a new method of matching compiled, cached routes that have been cached using the `route:cache` Artisan command. On large applications (for example, applications with 800 or more routes), these improvements can result in a **2x speed improvement** in requests per second on a simple "Hello World" benchmark. No changes to your application are required.

CORS Support

CORS support was contributed by [Barry vd. Heuvel](#).

Laravel 7 includes first-party support for configuring Cross-Origin Resource Sharing (CORS) OPTIONS request responses by integrating the popular Laravel CORS package written by Barry vd. Heuvel. A new `cors` configuration is included in the [default Laravel application skeleton](#).

For more information on CORS support in Laravel 7.x, please consult the [CORS documentation](#).

Query Time Casts

Query time casting was contributed by [Matt Barlow](#).

Sometimes you may need to apply casts while executing a query, such as when selecting a raw value from a table. For example, consider the following query:

```
use App\Post;
use App\User;

$users = User::select([
    'users.*',
    'last_posted_at' => Post::selectRaw('MAX(created_at)')
    ->whereColumn('user_id', 'users.id')
])->get();
```

The `last_posted_at` attribute on the results of this query will be a raw string. It would be convenient if we could apply a date cast to this attribute when executing the query. To accomplish this, we may use the `withCasts` method provided by Laravel 7:

```
$users = User::select([
    'users.*',
    'last_posted_at' => Post::selectRaw('MAX(created_at)')
    ->whereColumn('user_id', 'users.id')
])->withCasts([
    'last_posted_at' => 'date'
])->get();
```

MySQL 8+ Database Queue Improvements

MySQL database queue improvements were contributed by [Mohamed Said](#).

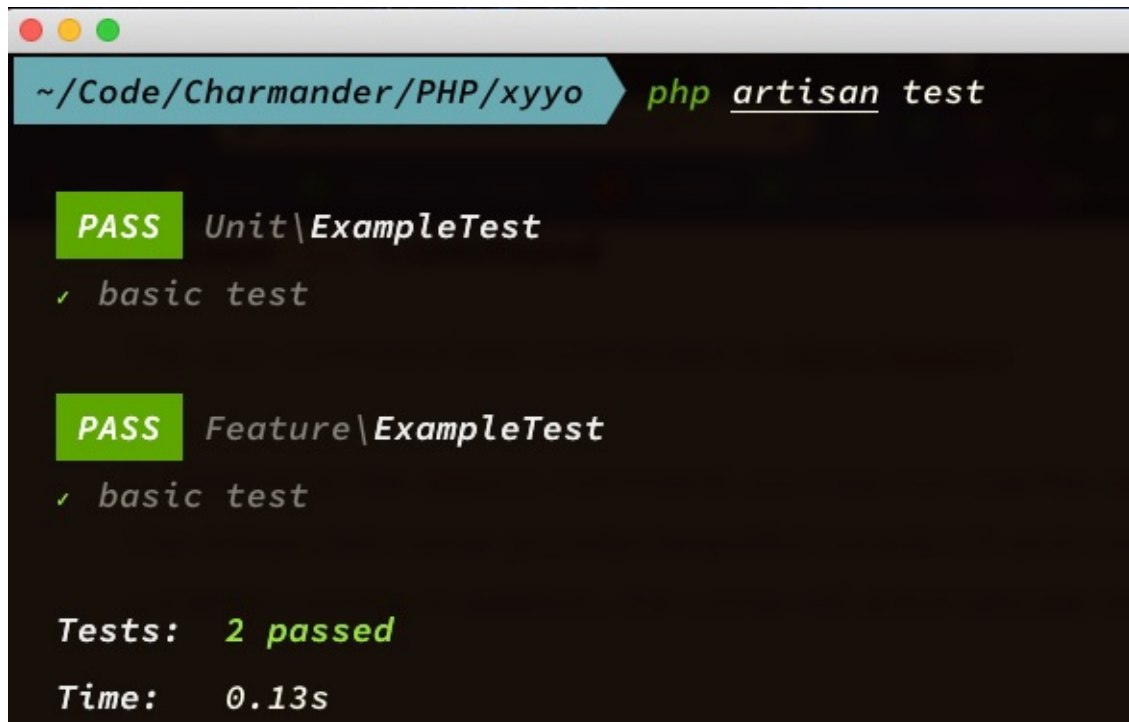
In previous releases of Laravel, the database queue was not considered robust enough for production usage, due to deadlocks. However, Laravel 7 provides improvements to applications using MySQL 8+ as their database backed queue. By using the `FOR UPDATE SKIP LOCKED` clause and other SQL enhancements, the database driver may now safely be used in higher volume production applications.

Artisan test Command

The test command was contributed by [Nuno Maduro](#).

In addition to the `phpunit` command, you may now use the `test` Artisan command to run your tests. The Artisan test runner provides beautiful console UX and more information regarding the test that is currently running. In addition, the runner will automatically stop on the first test failure:

```
php artisan test
```

A terminal window with a dark background and light green text. The window title bar shows three colored circles (red, yellow, green). The terminal prompt is `~/Code/Charmander/PHP/xyyo` followed by a blue arrow pointing right. The command `php artisan test` is entered. The output shows two test groups, both passing. The first group is `Unit\ExampleTest` with a sub-test `basic test`. The second group is `Feature\ExampleTest` with a sub-test `basic test`. At the bottom, it summarizes `Tests: 2 passed` and `Time: 0.13s`.

```
~/Code/Charmander/PHP/xyyo ➤ php artisan test

PASS Unit\ExampleTest
✓ basic test

PASS Feature\ExampleTest
✓ basic test

Tests: 2 passed
Time: 0.13s
```

Any arguments that can be passed to the `phpunit` command may also be passed to the Artisan `test` command:

```
php artisan test --group=feature
```

Markdown Mail Template Improvements

Markdown mail template improvements were contributed by [Taylor Otwell](#).

The default Markdown mail template has received a fresh, more modern design based on the Tailwind CSS color palette. Of course, this template can be published and customized according to your application's needs:



Introduction

The body of your message.

Button Text

Thanks,
Laravel

© 2020 Laravel. All rights reserved.

For more information on Markdown mail, please consult the [mail documentation](#).

Stub Customization

Stub customization was contributed by [Taylor Otwell](#).

The Artisan console's `make` commands are used to create a variety of classes, such as controllers, jobs, migrations, and tests. These classes are generated using "stub" files that are populated with values based on your input. However, you may sometimes wish to make small changes to files generated by Artisan. To accomplish this, Laravel 7 provides the `stub:publish` command to publish the most common stubs for customization:

```
php artisan stub:publish
```

The published stubs will be located within a `stubs` directory in the root of your application. Any changes you make to these stubs will be reflected when you generate their corresponding classes using Artisan `make` commands.

Queue `maxExceptions` Configuration

The `maxExceptions` property was contributed by [Mohamed Said](#).

Sometimes you may wish to specify that a job may be attempted many times, but should fail if the retries are triggered by a given number of exceptions. In Laravel 7, you may define a `maxExceptions` property on your job class:

```
<?php

namespace App\Jobs;

class ProcessPodcast implements ShouldQueue
{
```

```
/**
 * The number of times the job may be attempted.
 *
 * @var int
 */
public $tries = 25;

/**
 * The maximum number of exceptions to allow before failing.
 *
 * @var int
 */
public $maxExceptions = 3;

/**
 * Execute the job.
 *
 * @return void
 */
public function handle()
{
    Redis::throttle('key')->allow(10)->every(60)->then(function () {
        // Lock obtained, process the podcast...
    }, function () {
        // Unable to obtain lock...
        return $this->release(10);
    });
}
```

In this example, the job is released for ten seconds if the application is unable to obtain a Redis lock and will continue to be retried up to 25 times. However, the job will fail if three unhandled exceptions are thrown by the job.

Prologue

Upgrade Guide

Upgrade Guide

- [Upgrading To 7.0 From 6.x](#)

High Impact Changes

- [Authentication Scaffolding](#)
- [Date Serialization](#)
- [Symfony 5 Related Upgrades](#)

Medium Impact Changes

- [Blade Components & "Blade X"](#)
- [CORS Support](#)
- [Factory Types](#)
- [Markdown Mail Template Updates](#)
- [The `blade::component` Method](#)
- [The `assertSee` Assertion](#)
- [The different Validation Rule](#)
- [Unique Route Names](#)

Upgrading To 7.0 From 6.x

Estimated Upgrade Time: 15 Minutes

NOTE We attempt to document every possible breaking change. Since some of these breaking changes are in obscure parts of the framework only a portion of these changes may actually affect your application.

Symfony 5 Required

Likelihood Of Impact: High

Laravel 7 upgraded its underlying Symfony components to the 5.x series, which is now also the new minimum compatible version.

PHP 7.2.5 Required

Likelihood Of Impact: Low

The new minimum PHP version is now 7.2.5.

Updating Dependencies

Update the following dependencies in your `composer.json` file:

- `laravel/framework` to `^7.0`
- `nunomaduro/collision` to `^4.1`
- `phpunit/phpunit` to `^8.5`
- `laravel/tinker` to `^2.0`
- `facade/ignition` to `^2.0`

The following first-party packages have new major releases to support Laravel 7. If there are any, read through their individual upgrade guides before upgrading:

- [Browser Kit Testing v6.0](#)
- [Envoy v2.0](#)
- [Horizon v4.0](#)
- [Nova v3.0](#)
- [Passport v9.0](#)
- [Scout v8.0](#)
- [Telescope v3.0](#)
- [Tinker v2.0](#)
- UI v2.0 (No changes necessary)

Finally, examine any other third-party packages consumed by your application and verify you are using the proper version for Laravel 7 support.

Symfony 5 Related Upgrades

Likelihood Of Impact: High

Laravel 7 utilizes the 5.x series of the Symfony components. Some minor changes to your application are required to accommodate this upgrade.

First, the `report`, `shouldReport`, and `renderForConsole` methods of your application's `App\Exceptions\Handler` class should accept instances of the `Throwable` interface instead of `Exception` instances:

```
use Throwable;

public function report(Throwable $exception);
public function shouldReport(Throwable $exception);
public function render($request, Throwable $exception);
public function renderForConsole($output, Throwable $exception);
```

Additionally, any `failed` methods implemented on queued jobs should accept instances of `Throwable` instead of `Exception` instances.

Next, please update your session configuration file's `secure` option to have a fallback value of `null`:

```
'secure' => env('SESSION_SECURE_COOKIE', null),
```

Symfony Console, which is the underlying component that powers Artisan, expects all commands to return an integer. Therefore, you should ensure that any of your commands which return a value are returning integers:

```
public function handle()
{
    // Before...
    return true;

    // After...
    return 0;
}
```

Authentication

Scaffolding

Likelihood Of Impact: High

All authentication scaffolding has been [moved](#) to the `laravel/ui` repository. If you are using Laravel's authentication scaffolding, you should install the `^2.0` release of this package and the package should be installed in all environments. If you were previously including this package in the `require-dev` portion of your application's `composer.json` file, you should move it to the `require` section:

```
composer require laravel/ui "^2.0"
```

The `TokenRepositoryInterface`

Likelihood Of Impact: Low

A `recentlyCreatedToken` method has been added to the `Illuminate\Auth\Passwords\TokenRepositoryInterface` interface. If you are writing a custom implementation of this interface, you should add this method to your implementation.

Blade

The `component` Method

Likelihood Of Impact: Medium

The `Blade::component` method has been renamed to `Blade::aliasComponent`. Please update your calls to this method accordingly.

Blade Components & "Blade X"

Likelihood Of Impact: Medium

Laravel 7 includes first-party support for Blade "tag components". If you wish to disable Blade's built-in tag component functionality, you may call the `withoutComponentTags` method from the `boot` method of your `AppServiceProvider`:

```
use Illuminate\Support\Facades\Blade;

Blade::withoutComponentTags();
```

Eloquent

The `addHidden` / `addVisible` Methods

Likelihood Of Impact: Low

The undocumented `addHidden` and `addVisible` methods have been removed. Instead, please use the `makeHidden` and `makeVisible` methods.

The `booting` / `booted` Methods

Likelihood Of Impact: Low

The `booting` and `booted` methods have been added to Eloquent to provide a place to conveniently define any logic that should execute during the model "boot" process. If you already have model methods with these names, you will need to rename your methods so they do not conflict with the newly added methods.

Date Serialization

Likelihood Of Impact: High

Laravel 7 uses a new date serialization format when using the `toArray` or `toJson` method on Eloquent models. To format dates for serialization, the framework now uses Carbon's `toJSON` method, which produces an ISO-8601 compatible date including timezone information and fractional seconds. In addition, this change provides better support and integration with client-side date parsing libraries.

Previously, dates would be serialized to a format like the following: `2019-12-02 20:01:00`. Dates serialized using the new format will appear like: `2019-12-02T20:01:00.283041Z`. Please note that ISO-8601 dates are always expressed in UTC.

If you would like to keep using the previous behavior you can override the `serializeDate` method on your model:

```
use DateTimeInterface;

/**
```



```
* Prepare a date for array / JSON serialization.
*
* @param \DateTimeInterface $date
* @return string
*/
protected function serializeDate(DateTimeInterface $date)
{
    return $date->format('Y-m-d H:i:s');
}
```

TIP This change only affects serialization of models and model collections to arrays and JSON. This change has no effect on how dates are stored in your database.

Factory Types

Likelihood Of Impact: Medium

Laravel 7 removes the "factory types" feature. This feature has been undocumented since October 2016. If you are still using this feature, you should upgrade to [factory states](#), which provide more flexibility.

The `getOriginal` Method

Likelihood Of Impact: Low

The `$model->getOriginal()` method will now respect any casts and mutators defined on the model. Previously, this method returned the uncast, raw attributes. If you would like to continue retrieving the raw, uncast values, you may use the `getRawOriginal` method instead.

Route Binding

Likelihood Of Impact: Low

The `resolveRouteBinding` method of the `Illuminate\Contracts\Routing\UrlRoutable` interface now accepts a `$field` argument. If you were implementing this interface by hand, you should update your implementation.

In addition, the `resolveRouteBinding` method of the `Illuminate\Database\Eloquent\Model` class also now accepts a `$field` parameter. If you were overriding this method, you should update your method to accept this argument.

Finally, the `resolveRouteBinding` method of the `Illuminate\Http\Resources\DelegatesToResources` trait also now accepts a `$field` parameter. If you were overriding this method, you should update your method to accept this argument.

HTTP

PSR-7 Compatibility

Likelihood Of Impact: Low

The Zend Diactoros library for generating PSR-7 responses has been deprecated. If you are using this package for PSR-7 compatibility, please install the `nyholm/psr7` Composer package instead. In addition, please install the `^2.0` release of the `symfony/psr-http-message-bridge` Composer package.

Mail

Configuration File Changes

Likelihood Of Impact: Optional

In order to support multiple mailers, the default `mail` configuration file has changed in Laravel 7.x to include an array of `mailers`. However, in order to preserve backwards compatibility, the Laravel 6.x format of this configuration file is still supported. So, no changes are **required** when upgrading to Laravel 7.x; however, you

may wish to [examine the new mail configuration file](#) structure and update your file to reflect the changes.

In addition, the `MAIL_DRIVER` environment variable has been renamed to `MAIL_MAILER`.

Markdown Mail Template Updates

Likelihood Of Impact: Medium

The default Markdown mail templates have been refreshed with a more professional and appealing design. In addition, the undocumented `promotion` Markdown mail component has been removed.

Because indentation has special meaning within Markdown, Markdown mail templates expect unindented HTML. If you've previously published Laravel's default mail templates, you'll need to re-publish your mail templates or manually unindent them:

```
php artisan vendor:publish --tag=laravel-mail --force
```

Swift Mailer Bindings

Likelihood Of Impact: Low

Laravel 7.x doesn't provide `swift.mailer` and `swift.transport` container bindings. You may now access these objects through the `mailer` binding:

```
$swiftMailer = app('mailer')->getSwiftMailer();  
$swiftTransport = $swiftMailer->getTransport();
```

Resources

The `Illuminate\Http\Resources\Json\Resource` Class

Likelihood Of Impact: Low

The deprecated `Illuminate\Http\Resources\Json\Resource` class has been removed. Your resources should extend the `Illuminate\Http\Resources\Json\JsonResource` class instead.

Routing

The Router `getRoutes` Method

Likelihood Of Impact: Low

The router's `getRoutes` method now returns an instance of `Illuminate\Routing\RouteCollectionInterface` instead of `Illuminate\Routing\RouteCollection`.

Unique Route Names

Likelihood Of Impact: Medium

Even though never officially documented, previous Laravel releases allow you to define two different routes with the same name. In Laravel 7 this is no longer possible and you should always provide unique names for your routes. Routes with duplicate names can cause unexpected behavior in multiple areas of the framework.

CORS Support

Likelihood Of Impact: Medium

Cross-Origin Resource Sharing (CORS) support is now integrated by default. If you are using any third-party CORS libraries you are now advised to use the [new cors configuration file](#).

Next, install the underlying CORS library as a dependency of your application:

```
composer require fruitcake/laravel-cors
```

Finally, add the `\Fruitcake\Cors\HandleCors::class` middleware to your `App\Http\Kernel` global middleware list.

Session

The array Session Driver

Likelihood Of Impact: Low

The array session driver data is now persistent for the current request. Previously, data stored in the array session could not be retrieved even during the current request.

Testing

The assertSee Assertion

Likelihood Of Impact: Medium

The `assertSee`, `assertDontSee`, `assertSeeText`, `assertDontSeeText`, `assertSeeInOrder` and `assertSeeTextInOrder` assertions on the `TestResponse` class will now automatically escape values. If you are manually escaping any values passed to these assertions you should no longer do so. If you need to assert unescaped values, you may pass `false` as the second argument to the method.

The TestResponse Class

Likelihood Of Impact: Low

The `Illuminate\Foundation\Testing\TestResponse` class has been renamed to `Illuminate\Testing\TestResponse`. If you're extending this class, make sure to update the namespace.

The Assert Class

Likelihood Of Impact: Low

The `Illuminate\Foundation\Testing\Assert` class has been renamed to `Illuminate\Testing\Assert`. If you're using this class, make sure to update the namespace.

Validation

The different Rule

Likelihood Of Impact: Medium

The `different` rule will now fail if one of the specified parameters is missing from the request.

Miscellaneous

We also encourage you to view the changes in the `laravel/laravel` [GitHub repository](#). While many of these changes are not required, you may wish to keep these files in sync with your application. Some of these changes will be covered in this upgrade guide, but others, such as changes to configuration files or comments, will not be. You can easily view the changes with the [GitHub comparison tool](#) and choose which updates are important to you.

Prologue

Contribution Guide

Contribution Guide

- [Bug Reports](#)
- [Support Questions](#)
- [Core Development Discussion](#)
- [Which Branch?](#)
- [Compiled Assets](#)
- [Security Vulnerabilities](#)
- [Coding Style](#)
 - [PHPDoc](#)
 - [StyleCI](#)
- [Code of Conduct](#)

Bug Reports

To encourage active collaboration, Laravel strongly encourages pull requests, not just bug reports. "Bug reports" may also be sent in the form of a pull request containing a failing test.

However, if you file a bug report, your issue should contain a title and a clear description of the issue. You should also include as much relevant information as possible and a code sample that demonstrates the issue. The goal of a bug report is to make it easy for yourself - and others - to replicate the bug and develop a fix.

Remember, bug reports are created in the hope that others with the same problem will be able to collaborate with you on solving it. Do not expect that the bug report will automatically see any activity or that others will jump to fix it. Creating a bug report serves to help yourself and others start on the path of fixing the problem. If you want to chip in, you can help out by fixing [any bugs listed in our issue trackers](#).

The Laravel source code is managed on GitHub, and there are repositories for each of the Laravel projects:

- [Laravel Application](#)
- [Laravel Art](#)
- [Laravel Documentation](#)
- [Laravel Dusk](#)
- [Laravel Cashier Stripe](#)
- [Laravel Cashier Paddle](#)
- [Laravel Echo](#)
- [Laravel Envoy](#)
- [Laravel Framework](#)
- [Laravel Homestead](#)
- [Laravel Homestead Build Scripts](#)
- [Laravel Horizon](#)
- [Laravel Passport](#)
- [Laravel Sanctum](#)
- [Laravel Scout](#)
- [Laravel Socialite](#)
- [Laravel Telescope](#)
- [Laravel Website](#)
- [Laravel UI](#)

Support Questions

Laravel's GitHub issue trackers are not intended to provide Laravel help or support. Instead, use one of the following channels:

- [GitHub Discussions](#)

- [Laracasts Forums](#)
- [Laravel.io Forums](#)
- [StackOverflow](#)
- [Discord](#)
- [Larachat](#)
- [IRC](#)

Core Development Discussion

You may propose new features or improvements of existing Laravel behavior in the Laravel Ideas [issue board](#). If you propose a new feature, please be willing to implement at least some of the code that would be needed to complete the feature.

Informal discussion regarding bugs, new features, and implementation of existing features takes place in the `#internals` channel of the [Laravel Discord server](#). Taylor Otwell, the maintainer of Laravel, is typically present in the channel on weekdays from 8am-5pm (UTC-06:00 or America/Chicago), and sporadically present in the channel at other times.

Which Branch?

All bug fixes should be sent to the latest stable branch or to the [current LTS branch](#). Bug fixes should **never** be sent to the `master` branch unless they fix features that exist only in the upcoming release.

Minor features that are **fully backward compatible** with the current release may be sent to the latest stable branch.

Major new features should always be sent to the `master` branch, which contains the upcoming release.

If you are unsure if your feature qualifies as a major or minor, please ask Taylor Otwell in the `#internals` channel of the [Laravel Discord server](#).

Compiled Assets

If you are submitting a change that will affect a compiled file, such as most of the files in `resources/sass` or `resources/js` of the `laravel/laravel` repository, do not commit the compiled files. Due to their large size, they cannot realistically be reviewed by a maintainer. This could be exploited as a way to inject malicious code into Laravel. In order to defensively prevent this, all compiled files will be generated and committed by Laravel maintainers.

Security Vulnerabilities

If you discover a security vulnerability within Laravel, please send an email to Taylor Otwell at taylor@laravel.com. All security vulnerabilities will be promptly addressed.

Coding Style

Laravel follows the [PSR-2](#) coding standard and the [PSR-4](#) autoloading standard.

PHPDoc

Below is an example of a valid Laravel documentation block. Note that the `@param` attribute is followed by two spaces, the argument type, two more spaces, and finally the variable name:

```
/**
 * Register a binding with the container.
 *
 * @param string|array $abstract
 * @param \Closure|string|null $concrete
 * @param bool $shared
```

```
* @return void
*
* @throws \Exception
*/
public function bind($abstract, $concrete = null, $shared = false)
{
    //
}
```

StyleCI

Don't worry if your code styling isn't perfect! [StyleCI](#) will automatically merge any style fixes into the Laravel repository after pull requests are merged. This allows us to focus on the content of the contribution and not the code style.

Code of Conduct

The Laravel code of conduct is derived from the Ruby code of conduct. Any violations of the code of conduct may be reported to Taylor Otwell (taylor@laravel.com):

- Participants will be tolerant of opposing views.
- Participants must ensure that their language and actions are free of personal attacks and disparaging personal remarks.
- When interpreting the words and actions of others, participants should always assume good intentions.
- Behavior that can be reasonably considered harassment will not be tolerated.

Getting Started

Installation

Installation

- [Installation](#)
 - [Server Requirements](#)
 - [Installing Laravel](#)
 - [Configuration](#)
- [Web Server Configuration](#)
 - [Directory Configuration](#)
 - [Pretty URLs](#)

Installation

Server Requirements

The Laravel framework has a few system requirements. All of these requirements are satisfied by the [Laravel Homestead](#) virtual machine, so it's highly recommended that you use Homestead as your local Laravel development environment.

However, if you are not using Homestead, you will need to make sure your server meets the following requirements:

- PHP >= 7.2.5
- BCMath PHP Extension
- Ctype PHP Extension
- Fileinfo PHP extension
- JSON PHP Extension
- Mbstring PHP Extension
- OpenSSL PHP Extension
- PDO PHP Extension
- Tokenizer PHP Extension
- XML PHP Extension

Installing Laravel

Laravel utilizes [Composer](#) to manage its dependencies. So, before using Laravel, make sure you have Composer installed on your machine.

Via Laravel Installer

First, download the Laravel installer using Composer:

```
composer global require laravel/installer
```

Make sure to place Composer's system-wide vendor bin directory in your \$PATH so the laravel executable can be located by your system. This directory exists in different locations based on your operating system; however, some common locations include:

- macOS: \$HOME/.composer/vendor/bin
- Windows: %USERPROFILE%\AppData\Roaming\Composer\vendor\bin
- GNU / Linux Distributions: \$HOME/.config/composer/vendor/bin OR \$HOME/.composer/vendor/bin

You could also find the composer's global installation path by running `composer global about` and looking up from the first line.

Once installed, the `laravel new` command will create a fresh Laravel installation in the directory you specify.

For instance, `laravel new blog` will create a directory named `blog` containing a fresh Laravel installation with all of Laravel's dependencies already installed:

```
laravel new blog
```

Via Composer Create-Project

Alternatively, you may also install Laravel by issuing the Composer `create-project` command in your terminal:

```
composer create-project --prefer-dist laravel/laravel:^7.0 blog
```

Local Development Server

If you have PHP installed locally and you would like to use PHP's built-in development server to serve your application, you may use the `serve` Artisan command. This command will start a development server at

```
http://localhost:8000:
```

```
php artisan serve
```

More robust local development options are available via [Homestead](#) and [Valet](#).

Configuration

Public Directory

After installing Laravel, you should configure your web server's document / web root to be the `public` directory. The `index.php` in this directory serves as the front controller for all HTTP requests entering your application.

Configuration Files

All of the configuration files for the Laravel framework are stored in the `config` directory. Each option is documented, so feel free to look through the files and get familiar with the options available to you.

Directory Permissions

After installing Laravel, you may need to configure some permissions. Directories within the `storage` and the `bootstrap/cache` directories should be writable by your web server or Laravel will not run. If you are using the [Homestead](#) virtual machine, these permissions should already be set.

Application Key

The next thing you should do after installing Laravel is set your application key to a random string. If you installed Laravel via Composer or the Laravel installer, this key has already been set for you by the `php artisan key:generate` command.

Typically, this string should be 32 characters long. The key can be set in the `.env` environment file. If you have not copied the `.env.example` file to a new file named `.env`, you should do that now. **If the application key is not set, your user sessions and other encrypted data will not be secure!**

Additional Configuration

Laravel needs almost no other configuration out of the box. You are free to get started developing! However, you may wish to review the `config/app.php` file and its documentation. It contains several options such as `timezone` and `locale` that you may wish to change according to your application.

You may also want to configure a few additional components of Laravel, such as:

- [Cache](#)
- [Database](#)
- [Session](#)

Web Server Configuration

Directory Configuration

Laravel should always be served out of the root of the "web directory" configured for your web server. You should not attempt to serve a Laravel application out of a subdirectory of the "web directory". Attempting to do so could expose sensitive files present within your application.

Pretty URLs

Apache

Laravel includes a `public/.htaccess` file that is used to provide URLs without the `index.php` front controller in the path. Before serving Laravel with Apache, be sure to enable the `mod_rewrite` module so the `.htaccess` file will be honored by the server.

If the `.htaccess` file that ships with Laravel does not work with your Apache installation, try this alternative:

```
Options +FollowSymLinks -Indexes
RewriteEngine On

RewriteCond %{HTTP:Authorization} .
RewriteRule .* - [E=HTTP_AUTHORIZATION:%{HTTP:Authorization}]

RewriteCond %{REQUEST_FILENAME} !-d
RewriteCond %{REQUEST_FILENAME} !-f
RewriteRule ^ index.php [L]
```

Nginx

If you are using Nginx, the following directive in your site configuration will direct all requests to the `index.php` front controller:

```
location / {
    try_files $uri $uri/ /index.php?$query_string;
}
```

When using [Homestead](#) or [Valet](#), pretty URLs will be automatically configured.

Getting Started

Configuration

Configuration

- [Introduction](#)
- [Environment Configuration](#)
 - [Environment Variable Types](#)
 - [Retrieving Environment Configuration](#)
 - [Determining The Current Environment](#)
 - [Hiding Environment Variables From Debug Pages](#)
- [Accessing Configuration Values](#)
- [Configuration Caching](#)
- [Maintenance Mode](#)

Introduction

All of the configuration files for the Laravel framework are stored in the `config` directory. Each option is documented, so feel free to look through the files and get familiar with the options available to you.

Environment Configuration

It is often helpful to have different configuration values based on the environment where the application is running. For example, you may wish to use a different cache driver locally than you do on your production server.

To make this a cinch, Laravel utilizes the [DotEnv](#) PHP library by Vance Lucas. In a fresh Laravel installation, the root directory of your application will contain a `.env.example` file. If you install Laravel via Composer, this file will automatically be copied to `.env`. Otherwise, you should copy the file manually.

Your `.env` file should not be committed to your application's source control, since each developer / server using your application could require a different environment configuration. Furthermore, this would be a security risk in the event an intruder gains access to your source control repository, since any sensitive credentials would get exposed.

If you are developing with a team, you may wish to continue including a `.env.example` file with your application. By putting placeholder values in the example configuration file, other developers on your team can clearly see which environment variables are needed to run your application. You may also create a `.env.testing` file. This file will override the `.env` file when running PHPUnit tests or executing Artisan commands with the `-env=testing` option.

TIP Any variable in your `.env` file can be overridden by external environment variables such as server-level or system-level environment variables.

Environment Variable Types

All variables in your `.env` files are parsed as strings, so some reserved values have been created to allow you to return a wider range of types from the `env()` function:

<code>.env</code> Value	<code>env()</code> Value
<code>true</code>	(bool) true
<code>(true)</code>	(bool) true
<code>false</code>	(bool) false
<code>(false)</code>	(bool) false
<code>empty</code>	(string) ""
<code>(empty)</code>	(string) ""
<code>null</code>	(null) null

```
(null)      (null) null
```

If you need to define an environment variable with a value that contains spaces, you may do so by enclosing the value in double quotes.

```
APP_NAME="My Application"
```

Retrieving Environment Configuration

All of the variables listed in this file will be loaded into the `$_ENV` PHP super-global when your application receives a request. However, you may use the `env` helper to retrieve values from these variables in your configuration files. In fact, if you review the Laravel configuration files, you will notice several of the options already using this helper:

```
'debug' => env('APP_DEBUG', false),
```

The second value passed to the `env` function is the "default value". This value will be used if no environment variable exists for the given key.

Determining The Current Environment

The current application environment is determined via the `APP_ENV` variable from your `.env` file. You may access this value via the `environment` method on the App [facade](#):

```
$environment = App::environment();
```

You may also pass arguments to the `environment` method to check if the environment matches a given value. The method will return `true` if the environment matches any of the given values:

```
if (App::environment('local')) {
    // The environment is local
}

if (App::environment(['local', 'staging'])) {
    // The environment is either local OR staging...
}
```

TIP The current application environment detection can be overridden by a server-level `APP_ENV` environment variable. This can be useful when you need to share the same application for different environment configurations, so you can set up a given host to match a given environment in your server's configurations.

Hiding Environment Variables From Debug Pages

When an exception is uncaught and the `APP_DEBUG` environment variable is `true`, the debug page will show all environment variables and their contents. In some cases you may want to obscure certain variables. You may do this by updating the `debug_hide` option in your `config/app.php` configuration file.

Some variables are available in both the environment variables and the server / request data. Therefore, you may need to hide them for both `$_ENV` and `$_SERVER`:

```
return [
    // ...

    'debug_hide' => [
        '_ENV' => [
            'APP_KEY',
            'DB_PASSWORD',
        ],

        '_SERVER' => [
            'APP_KEY',
            'DB_PASSWORD',
        ],

        '_POST' => [
```

```
        'password',  
    ],  
];
```

Accessing Configuration Values

You may easily access your configuration values using the global `config` helper function from anywhere in your application. The configuration values may be accessed using "dot" syntax, which includes the name of the file and option you wish to access. A default value may also be specified and will be returned if the configuration option does not exist:

```
$value = config('app.timezone');  
  
// Retrieve a default value if the configuration value does not exist...  
$value = config('app.timezone', 'Asia/Seoul');
```

To set configuration values at runtime, pass an array to the `config` helper:

```
config(['app.timezone' => 'America/Chicago']);
```

Configuration Caching

To give your application a speed boost, you should cache all of your configuration files into a single file using the `config:cache` Artisan command. This will combine all of the configuration options for your application into a single file which will be loaded quickly by the framework.

You should typically run the `php artisan config:cache` command as part of your production deployment routine. The command should not be run during local development as configuration options will frequently need to be changed during the course of your application's development.

NOTE If you execute the `config:cache` command during your deployment process, you should be sure that you are only calling the `env` function from within your configuration files. Once the configuration has been cached, the `.env` file will not be loaded and all calls to the `env` function will return `null`.

Maintenance Mode

When your application is in maintenance mode, a custom view will be displayed for all requests into your application. This makes it easy to "disable" your application while it is updating or when you are performing maintenance. A maintenance mode check is included in the default middleware stack for your application. If the application is in maintenance mode, a `MaintenanceModeException` will be thrown with a status code of 503.

To enable maintenance mode, execute the `down` Artisan command:

```
php artisan down
```

You may also provide `message` and `retry` options to the `down` command. The `message` value may be used to display or log a custom message, while the `retry` value will be set as the `Retry-After` HTTP header's value:

```
php artisan down --message="Upgrading Database" --retry=60
```

Even while in maintenance mode, specific IP addresses or networks may be allowed to access the application using the command's `allow` option:

```
php artisan down --allow=127.0.0.1 --allow=192.168.0.0/16
```

To disable maintenance mode, use the `up` command:

```
php artisan up
```

TIP You may customize the default maintenance mode template by defining your own template at `resources/views/errors/503.blade.php`.

Maintenance Mode & Queues

While your application is in maintenance mode, no [queued jobs](#) will be handled. The jobs will continue to be handled as normal once the application is out of maintenance mode.

Alternatives To Maintenance Mode

Since maintenance mode requires your application to have several seconds of downtime, consider alternatives like [Envoyer](#) to accomplish zero-downtime deployment with Laravel.

Getting Started

Directory Structure

Directory Structure

- [Introduction](#)
- [The Root Directory](#)
 - [The app Directory](#)
 - [The bootstrap Directory](#)
 - [The config Directory](#)
 - [The database Directory](#)
 - [The public Directory](#)
 - [The resources Directory](#)
 - [The routes Directory](#)
 - [The storage Directory](#)
 - [The tests Directory](#)
 - [The vendor Directory](#)
- [The App Directory](#)
 - [The Broadcasting Directory](#)
 - [The Console Directory](#)
 - [The Events Directory](#)
 - [The Exceptions Directory](#)
 - [The Http Directory](#)
 - [The Jobs Directory](#)
 - [The Listeners Directory](#)
 - [The Mail Directory](#)
 - [The Notifications Directory](#)
 - [The Policies Directory](#)
 - [The Providers Directory](#)
 - [The Rules Directory](#)

Introduction

The default Laravel application structure is intended to provide a great starting point for both large and small applications. But you are free to organize your application however you like. Laravel imposes almost no restrictions on where any given class is located - as long as Composer can autoload the class.

Where Is The Models Directory?

When getting started with Laravel, many developers are confused by the lack of a `models` directory. However, the lack of such a directory is intentional. We find the word "models" ambiguous since it means many different things to many different people. Some developers refer to an application's "model" as the totality of all of its business logic, while others refer to "models" as classes that interact with a relational database.

For this reason, we choose to place Eloquent models in the `app` directory by default, and allow the developer to place them somewhere else if they choose.

The Root Directory

The App Directory

The `app` directory contains the core code of your application. We'll explore this directory in more detail soon; however, almost all of the classes in your application will be in this directory.

The Bootstrap Directory

The `bootstrap` directory contains the `app.php` file which bootstraps the framework. This directory also houses a

cache directory which contains framework generated files for performance optimization such as the route and services cache files.

The Config Directory

The `config` directory, as the name implies, contains all of your application's configuration files. It's a great idea to read through all of these files and familiarize yourself with all of the options available to you.

The Database Directory

The `database` directory contains your database migrations, model factories, and seeds. If you wish, you may also use this directory to hold an SQLite database.

The Public Directory

The `public` directory contains the `index.php` file, which is the entry point for all requests entering your application and configures autoloading. This directory also houses your assets such as images, JavaScript, and CSS.

The Resources Directory

The `resources` directory contains your views as well as your raw, un-compiled assets such as LESS, SASS, or JavaScript. This directory also houses all of your language files.

The Routes Directory

The `routes` directory contains all of the route definitions for your application. By default, several route files are included with Laravel: `web.php`, `api.php`, `console.php` and `channels.php`.

The `web.php` file contains routes that the `RouteServiceProvider` places in the `web` middleware group, which provides session state, CSRF protection, and cookie encryption. If your application does not offer a stateless, RESTful API, all of your routes will most likely be defined in the `web.php` file.

The `api.php` file contains routes that the `RouteServiceProvider` places in the `api` middleware group, which provides rate limiting. These routes are intended to be stateless, so requests entering the application through these routes are intended to be authenticated via tokens and will not have access to session state.

The `console.php` file is where you may define all of your Closure based console commands. Each Closure is bound to a command instance allowing a simple approach to interacting with each command's IO methods. Even though this file does not define HTTP routes, it defines console based entry points (routes) into your application.

The `channels.php` file is where you may register all of the event broadcasting channels that your application supports.

The Storage Directory

The `storage` directory contains your compiled Blade templates, file based sessions, file caches, and other files generated by the framework. This directory is segregated into `app`, `framework`, and `logs` directories. The `app` directory may be used to store any files generated by your application. The `framework` directory is used to store framework generated files and caches. Finally, the `logs` directory contains your application's log files.

The `storage/app/public` directory may be used to store user-generated files, such as profile avatars, that should be publicly accessible. You should create a symbolic link at `public/storage` which points to this directory. You may create the link using the `php artisan storage:link` command.

The Tests Directory

The `tests` directory contains your automated tests. An example [PHPUnit](#) test is provided out of the box. Each

test class should be suffixed with the word `test`. You may run your tests using the `phpunit` or `php vendor/bin/phpunit` commands.

The Vendor Directory

The vendor directory contains your [Composer](#) dependencies.

The App Directory

The majority of your application is housed in the app directory. By default, this directory is namespaced under `App` and is autoloaded by Composer using the [PSR-4 autoloading standard](#).

The app directory contains a variety of additional directories such as `Console`, `Http`, and `Providers`. Think of the `Console` and `Http` directories as providing an API into the core of your application. The HTTP protocol and CLI are both mechanisms to interact with your application, but do not actually contain application logic. In other words, they are two ways of issuing commands to your application. The `Console` directory contains all of your Artisan commands, while the `Http` directory contains your controllers, middleware, and requests.

A variety of other directories will be generated inside the app directory as you use the `make` Artisan commands to generate classes. So, for example, the `app/Jobs` directory will not exist until you execute the `make:job` Artisan command to generate a job class.

TIP Many of the classes in the app directory can be generated by Artisan via commands. To review the available commands, run the `php artisan list make` command in your terminal.

The Broadcasting Directory

The `Broadcasting` directory contains all of the broadcast channel classes for your application. These classes are generated using the `make:channel` command. This directory does not exist by default, but will be created for you when you create your first channel. To learn more about channels, check out the documentation on [event broadcasting](#).

The Console Directory

The `Console` directory contains all of the custom Artisan commands for your application. These commands may be generated using the `make:command` command. This directory also houses your console kernel, which is where your custom Artisan commands are registered and your [scheduled tasks](#) are defined.

The Events Directory

This directory does not exist by default, but will be created for you by the `event:generate` and `make:event` Artisan commands. The `Events` directory houses [event classes](#). Events may be used to alert other parts of your application that a given action has occurred, providing a great deal of flexibility and decoupling.

The Exceptions Directory

The `Exceptions` directory contains your application's exception handler and is also a good place to place any exceptions thrown by your application. If you would like to customize how your exceptions are logged or rendered, you should modify the `Handler` class in this directory.

The Http Directory

The `Http` directory contains your controllers, middleware, and form requests. Almost all of the logic to handle requests entering your application will be placed in this directory.

The Jobs Directory

This directory does not exist by default, but will be created for you if you execute the `make:job` Artisan

command. The `Jobs` directory houses the [queueable jobs](#) for your application. Jobs may be queued by your application or run synchronously within the current request lifecycle. Jobs that run synchronously during the current request are sometimes referred to as "commands" since they are an implementation of the [command pattern](#).

The Listeners Directory

This directory does not exist by default, but will be created for you if you execute the `event:generate` or `make:listener` Artisan commands. The `Listeners` directory contains the classes that handle your [events](#). Event listeners receive an event instance and perform logic in response to the event being fired. For example, a `UserRegistered` event might be handled by a `SendWelcomeEmail` listener.

The Mail Directory

This directory does not exist by default, but will be created for you if you execute the `make:mail` Artisan command. The `Mail` directory contains all of your classes that represent emails sent by your application. Mail objects allow you to encapsulate all of the logic of building an email in a single, simple class that may be sent using the `Mail::send` method.

The Notifications Directory

This directory does not exist by default, but will be created for you if you execute the `make:notification` Artisan command. The `Notifications` directory contains all of the "transactional" notifications that are sent by your application, such as simple notifications about events that happen within your application. Laravel's notification features abstracts sending notifications over a variety of drivers such as email, Slack, SMS, or stored in a database.

The Policies Directory

This directory does not exist by default, but will be created for you if you execute the `make:policy` Artisan command. The `Policies` directory contains the authorization policy classes for your application. Policies are used to determine if a user can perform a given action against a resource. For more information, check out the [authorization documentation](#).

The Providers Directory

The `Providers` directory contains all of the [service providers](#) for your application. Service providers bootstrap your application by binding services in the service container, registering events, or performing any other tasks to prepare your application for incoming requests.

In a fresh Laravel application, this directory will already contain several providers. You are free to add your own providers to this directory as needed.

The Rules Directory

This directory does not exist by default, but will be created for you if you execute the `make:rule` Artisan command. The `Rules` directory contains the custom validation rule objects for your application. Rules are used to encapsulate complicated validation logic in a simple object. For more information, check out the [validation documentation](#).

Getting Started

Homestead

Laravel Homestead

- [Introduction](#)
- [Installation & Setup](#)
 - [First Steps](#)
 - [Configuring Homestead](#)
 - [Launching The Vagrant Box](#)
 - [Per Project Installation](#)
 - [Installing Optional Features](#)
 - [Aliases](#)
- [Daily Usage](#)
 - [Accessing Homestead Globally](#)
 - [Connecting Via SSH](#)
 - [Connecting To Databases](#)
 - [Database Backups](#)
 - [Database Snapshots](#)
 - [Adding Additional Sites](#)
 - [Environment Variables](#)
 - [Wildcard SSL](#)
 - [Configuring Cron Schedules](#)
 - [Configuring Mailhog](#)
 - [Configuring Minio](#)
 - [Ports](#)
 - [Sharing Your Environment](#)
 - [Multiple PHP Versions](#)
 - [Web Servers](#)
 - [Mail](#)
- [Debugging & Profiling](#)
 - [Debugging Web Requests With Xdebug](#)
 - [Debugging CLI Applications](#)
 - [Profiling Applications with Blackfire](#)
- [Network Interfaces](#)
- [Extending Homestead](#)
- [Updating Homestead](#)
- [Provider Specific Settings](#)
 - [VirtualBox](#)

Introduction

Laravel strives to make the entire PHP development experience delightful, including your local development environment. [Vagrant](#) provides a simple, elegant way to manage and provision Virtual Machines.

Laravel Homestead is an official, pre-packaged Vagrant box that provides you a wonderful development environment without requiring you to install PHP, a web server, and any other server software on your local machine. No more worrying about messing up your operating system! Vagrant boxes are completely disposable. If something goes wrong, you can destroy and re-create the box in minutes!

Homestead runs on any Windows, Mac, or Linux system, and includes Nginx, PHP, MySQL, PostgreSQL, Redis, Memcached, Node, and all of the other goodies you need to develop amazing Laravel applications.

NOTE If you are using Windows, you may need to enable hardware virtualization (VT-x). It can usually be enabled via your BIOS. If you are using Hyper-V on a UEFI system you may additionally need to disable Hyper-V in order to access VT-x.

Included Software

Ubuntu 18.04	Imm for MySQL or	Memcached
Git	MariaDB database	Beanstalkd
PHP 7.4	snapshots	Mailhog
PHP 7.3	Sqlite3	avahi
PHP 7.2	PostgreSQL (9.6, 10, 11,	ngrok
PHP 7.1	12)	Xdebug
PHP 7.0	Composer	XHProf / Tideways /
PHP 5.6	Node (With Yarn, Bower,	XHGui
Nginx	Grunt, and Gulp)	wp-cli
MySQL	Redis	

Optional Software

Apache	Gearman	Oh My Zsh
Blackfire	Go	Open Resty
Cassandra	Grafana	PM2
Chronograf	InfluxDB	Python
CouchDB	MariaDB	RabbitMQ
Crystal & Lucky	MinIO	Solr
Framework	MongoDB	Webdriver & Laravel
Docker	MySQL 8	Dusk Utilities
Elasticsearch	Neo4j	

Installation & Setup

First Steps

Before launching your Homestead environment, you must install [VirtualBox 6.x](#), [VMWare](#), [Parallels](#) or [Hyper-V](#) as well as [Vagrant](#). All of these software packages provide easy-to-use visual installers for all popular operating systems.

To use the VMware provider, you will need to purchase both VMware Fusion / Workstation and the [VMware Vagrant plug-in](#). Though it is not free, VMware can provide faster shared folder performance out of the box.

To use the Parallels provider, you will need to install [Parallels Vagrant plug-in](#). It is free of charge.

Because of [Vagrant limitations](#), The Hyper-V provider ignores all networking settings.

Installing The Homestead Vagrant Box

Once VirtualBox / VMware and Vagrant have been installed, you should add the `laravel/homestead` box to your Vagrant installation using the following command in your terminal. It will take a few minutes to download the box, depending on your Internet connection speed:

```
vagrant box add laravel/homestead
```

If this command fails, make sure your Vagrant installation is up to date.

NOTE Homestead periodically issues "alpha" / "beta" boxes for testing, which may interfere with the `vagrant box add` command. If you are having issues running `vagrant box add`, you may run the `vagrant up` command and the correct box will be downloaded when Vagrant attempts to start the virtual machine.

Installing Homestead

You may install Homestead by cloning the repository onto your host machine. Consider cloning the repository into a `Homestead` folder within your "home" directory, as the Homestead box will serve as the host to all of your Laravel projects:

```
git clone https://github.com/laravel/homestead.git ~/Homestead
```

You should check out a tagged version of Homestead since the `master` branch may not always be stable. You can find the latest stable version on the [GitHub Release Page](#). Alternatively, you may checkout the `release` branch which always contains the latest stable release:

```
cd ~/Homestead
git checkout release
```

Once you have cloned the Homestead repository, run the `bash init.sh` command from the Homestead directory to create the `Homestead.yaml` configuration file. The `Homestead.yaml` file will be placed in the Homestead directory:

```
// Mac / Linux...
bash init.sh

// Windows...
init.bat
```

Configuring Homestead

Setting Your Provider

The `provider` key in your `Homestead.yaml` file indicates which Vagrant provider should be used: `virtualbox`, `vmware_fusion`, `vmware_workstation`, `parallels` or `hyperv`. You may set this to the provider you prefer:

```
provider: virtualbox
```

Configuring Shared Folders

The `folders` property of the `Homestead.yaml` file lists all of the folders you wish to share with your Homestead environment. As files within these folders are changed, they will be kept in sync between your local machine and the Homestead environment. You may configure as many shared folders as necessary:

```
folders:
  - map: ~/code/project1
    to: /home/vagrant/project1
```

NOTE Windows users should not use the `~/` path syntax and instead should use the full path to their project, such as `C:\Users\User\Code\project1`.

You should always map individual projects to their own folder mapping instead of mapping your entire `~/code` folder. When you map a folder the virtual machine must keep track of all disk IO for every file in the folder. This leads to performance issues if you have a large number of files in a folder.

```
folders:
  - map: ~/code/project1
    to: /home/vagrant/project1

  - map: ~/code/project2
    to: /home/vagrant/project2
```

NOTE You should never mount `.` (the current directory) when using Homestead. This causes Vagrant to not map the current folder to `/vagrant` and will break optional features and cause unexpected results while provisioning.

To enable [NFS](#), you only need to add a simple flag to your synced folder configuration:

```
folders:
  - map: ~/code/project1
    to: /home/vagrant/project1
    type: "nfs"
```

NOTE When using NFS on Windows, you should consider installing the [vagrant-winnfsd](#) plug-in. This plug-in will maintain the correct user / group permissions for files and directories within the Homestead box.

You may also pass any options supported by Vagrant's [Synced Folders](#) by listing them under the `options` key:

```
folders:
  - map: ~/code/project1
    to: /home/vagrant/project1
    type: "rsync"
    options:
      rsync__args: ["--verbose", "--archive", "--delete", "-zz"]
      rsync__exclude: ["node_modules"]
```

Configuring Nginx Sites

Not familiar with Nginx? No problem. The `sites` property allows you to easily map a "domain" to a folder on your Homestead environment. A sample site configuration is included in the `Homestead.yaml` file. Again, you may add as many sites to your Homestead environment as necessary. Homestead can serve as a convenient, virtualized environment for every Laravel project you are working on:

```
sites:
  - map: homestead.test
    to: /home/vagrant/project1/public
```

If you change the `sites` property after provisioning the Homestead box, you should re-run `vagrant reload --provision` to update the Nginx configuration on the virtual machine.

NOTE Homestead scripts are built to be as idempotent as possible. However, if you are experiencing issues while provisioning you should destroy and rebuild the machine via `vagrant destroy && vagrant up`.

Enable / Disable Services

Homestead starts several services by default; however, you may customize which services are enabled or disabled during provisioning. For example, you may enable PostgreSQL and disable MySQL:

```
services:
  - enabled:
    - "postgresql@12-main"
  - disabled:
    - "mysql"
```

The specified services will be started or stopped based on their order in the `enabled` and `disabled` directives.

Hostname Resolution

Homestead publishes hostnames over `mDNS` for automatic host resolution. If you set `hostname: homestead` in your `Homestead.yaml` file, the host will be available at `homestead.local`. MacOS, iOS, and Linux desktop distributions include `mDNS` support by default. Windows requires installing [Bonjour Print Services for Windows](#).

Using automatic hostnames works best for "per project" installations of Homestead. If you host multiple sites on a single Homestead instance, you may add the "domains" for your web sites to the `hosts` file on your machine. The `hosts` file will redirect requests for your Homestead sites into your Homestead machine. On Mac and Linux, this file is located at `/etc/hosts`. On Windows, it is located at `C:\Windows\System32\drivers\etc\hosts`. The lines you add to this file will look like the following:

```
192.168.10.10 homestead.test
```

Make sure the IP address listed is the one set in your `Homestead.yaml` file. Once you have added the domain to

your `hosts` file and launched the Vagrant box you will be able to access the site via your web browser:

`http://homestead.test`

Launching The Vagrant Box

Once you have edited the `Homestead.yaml` to your liking, run the `vagrant up` command from your Homestead directory. Vagrant will boot the virtual machine and automatically configure your shared folders and Nginx sites.

To destroy the machine, you may use the `vagrant destroy --force` command.

Per Project Installation

Instead of installing Homestead globally and sharing the same Homestead box across all of your projects, you may instead configure a Homestead instance for each project you manage. Installing Homestead per project may be beneficial if you wish to ship a `vagrantfile` with your project, allowing others working on the project to `vagrant up`.

To install Homestead directly into your project, require it using Composer:

```
composer require laravel/homestead --dev
```

Once Homestead has been installed, use the `make` command to generate the `vagrantfile` and `Homestead.yaml` file in your project root. The `make` command will automatically configure the `sites` and `folders` directives in the `Homestead.yaml` file.

Mac / Linux:

```
php vendor/bin/homestead make
```

Windows:

```
vendor\bin\homestead make
```

Next, run the `vagrant up` command in your terminal and access your project at `http://homestead.test` in your browser. Remember, you will still need to add an `/etc/hosts` file entry for `homestead.test` or the domain of your choice if you are not using automatic [hostname resolution](#).

Installing Optional Features

Optional software is installed using the "features" setting in your Homestead configuration file. Most features can be enabled or disabled with a boolean value, while some features allow multiple configuration options:

```
features:
  - blackfire:
      server_id: "server_id"
      server_token: "server_value"
      client_id: "client_id"
      client_token: "client_value"
  - cassandra: true
  - chronograf: true
  - couchdb: true
  - crystal: true
  - docker: true
  - elasticsearch:
      version: 7.9.0
  - gearman: true
  - golang: true
  - grafana: true
  - influxdb: true
  - mariadb: true
  - minio: true
  - mongodb: true
  - mysql8: true
  - neo4j: true
  - ohmyzsh: true
  - openresty: true
```

```
- pm2: true
- python: true
- rabbitmq: true
- solr: true
- webdriver: true
```

MariaDB

Enabling MariaDB will remove MySQL and install MariaDB. MariaDB serves as a drop-in replacement for MySQL, so you should still use the `mysql` database driver in your application's database configuration.

MongoDB

The default MongoDB installation will set the database username to `homestead` and the corresponding password to `secret`.

Elasticsearch

You may specify a supported version of Elasticsearch, which may be a major version or an exact version number (major.minor.patch). The default installation will create a cluster named 'homestead'. You should never give Elasticsearch more than half of the operating system's memory, so make sure your Homestead machine has at least twice the Elasticsearch allocation.

TIP Check out the [Elasticsearch documentation](#) to learn how to customize your configuration.

Neo4j

The default Neo4j installation will set the database username to `homestead` and corresponding password to `secret`. To access the Neo4j browser, visit `http://homestead.test:7474` via your web browser. The ports 7687 (Bolt), 7474 (HTTP), and 7473 (HTTPS) are ready to serve requests from the Neo4j client.

Aliases

You may add Bash aliases to your Homestead machine by modifying the `aliases` file within your Homestead directory:

```
alias c='clear'
alias ..='cd ..'
```

After you have updated the `aliases` file, you should re-provision the Homestead machine using the `vagrant reload --provision` command. This will ensure that your new aliases are available on the machine.

Daily Usage

Accessing Homestead Globally

Sometimes you may want to `vagrant up` your Homestead machine from anywhere on your filesystem. You can do this on Mac / Linux systems by adding a Bash function to your Bash profile. On Windows, you may accomplish this by adding a "batch" file to your `PATH`. These scripts will allow you to run any Vagrant command from anywhere on your system and will automatically point that command to your Homestead installation:

Mac / Linux

```
function homestead() {
    ( cd ~/Homestead && vagrant $* )
}
```

Make sure to tweak the `~/Homestead` path in the function to the location of your actual Homestead installation. Once the function is installed, you may run commands like `homestead up` or `homestead ssh` from anywhere on your system.

Windows

Create a `homestead.bat` batch file anywhere on your machine with the following contents:

```
@echo off

set cwd=%cd%
set homesteadVagrant=C:\Homestead

cd /d %homesteadVagrant% && vagrant %*
cd /d %cwd%

set cwd=
set homesteadVagrant=
```

Make sure to tweak the example `c:\Homestead` path in the script to the actual location of your Homestead installation. After creating the file, add the file location to your `PATH`. You may then run commands like `homestead up` or `homestead ssh` from anywhere on your system.

Connecting Via SSH

You can SSH into your virtual machine by issuing the `vagrant ssh` terminal command from your Homestead directory.

But, since you will probably need to SSH into your Homestead machine frequently, consider adding the "function" described above to your host machine to quickly SSH into the Homestead box.

Connecting To Databases

A homestead database is configured for both MySQL and PostgreSQL out of the box. To connect to your MySQL or PostgreSQL database from your host machine's database client, you should connect to `127.0.0.1` and port `33060` (MySQL) or `54320` (PostgreSQL). The username and password for both databases is `homestead / secret`.

NOTE You should only use these non-standard ports when connecting to the databases from your host machine. You will use the default `3306` and `5432` ports in your Laravel database configuration file since Laravel is running *within* the virtual machine.

Database Backups

Homestead can automatically backup your database when your Vagrant box is destroyed. To utilize this feature, you must be using Vagrant 2.1.0 or greater. Or, if you are using an older version of Vagrant, you must install the `vagrant-triggers` plug-in. To enable automatic database backups, add the following line to your `Homestead.yaml` file:

```
backup: true
```

Once configured, Homestead will export your databases to `mysql_backup` and `postgres_backup` directories when the `vagrant destroy` command is executed. These directories can be found in the folder where you cloned Homestead or in the root of your project if you are using the [per project installation](#) method.

Database Snapshots

Homestead supports freezing the state of MySQL and MariaDB databases and branching between them using [Logical MySQL Manager](#). For example, imagine working on a site with a multi-gigabyte database. You can import the database and take a snapshot. After doing some work and creating some test content locally, you may quickly restore back to the original state.

Under the hood, LMM uses LVM's thin snapshot functionality with copy-on-write support. In practice, this means that changing a single row in a table will only cause the changes you made to be written to disk, saving significant time and disk space during restores.

Since `lmm` interacts with LVM, it must be run as `root`. To see all available commands, run `sudo lmm` inside your

Vagrant box. A common workflow looks like the following:

1. Import a database into the default `master lmm` branch.
2. Save a snapshot of the unchanged database using `sudo lmm branch prod-YYYY-MM-DD`.
3. Modify the database.
4. Run `sudo lmm merge prod-YYYY-MM-DD` to undo all changes.
5. Run `sudo lmm delete <branch>` to delete unneeded branches.

Adding Additional Sites

Once your Homestead environment is provisioned and running, you may want to add additional Nginx sites for your Laravel applications. You can run as many Laravel installations as you wish on a single Homestead environment. To add an additional site, add the site to your `Homestead.yaml` file:

```
sites:
- map: homestead.test
  to: /home/vagrant/project1/public
- map: another.test
  to: /home/vagrant/project2/public
```

If Vagrant is not automatically managing your "hosts" file, you may need to add the new site to that file as well:

```
192.168.10.10 homestead.test
192.168.10.10 another.test
```

Once the site has been added, run the `vagrant reload --provision` command from your Homestead directory.

Site Types

Homestead supports several types of sites which allow you to easily run projects that are not based on Laravel. For example, we may easily add a Symfony application to Homestead using the `symfony2` site type:

```
sites:
- map: symfony2.test
  to: /home/vagrant/my-symfony-project/web
  type: "symfony2"
```

The available site types are: `apache`, `apigility`, `expressive`, `laravel` (the default), `proxy`, `silverstripe`, `statamic`, `symfony2`, `symfony4`, and `zf`.

Site Parameters

You may add additional Nginx `fastcgi_param` values to your site via the `params` site directive. For example, we'll add a `FOO` parameter with a value of `BAR`:

```
sites:
- map: homestead.test
  to: /home/vagrant/project1/public
  params:
    - key: FOO
      value: BAR
```

Environment Variables

You can set global environment variables by adding them to your `Homestead.yaml` file:

```
variables:
- key: APP_ENV
  value: local
- key: FOO
  value: bar
```

After updating the `Homestead.yaml`, be sure to re-provision the machine by running `vagrant reload --provision`. This will update the PHP-FPM configuration for all of the installed PHP versions and also update the environment for the `vagrant` user.

Wildcard SSL

Homestead configures a self-signed SSL certificate for each site defined in the `sites:` section of your `Homestead.yaml` file. If you would like to generate a wildcard SSL certificate for a site you may add a `wildcard` option to that site's configuration. By default, the site will use the wildcard certificate *instead* of the specific domain certificate:

```
- map: foo.domain.test
  to: /home/vagrant/domain
  wildcard: "yes"
```

If the `use_wildcard` option is set to `no`, the wildcard certificate will be generated but will not be used:

```
- map: foo.domain.test
  to: /home/vagrant/domain
  wildcard: "yes"
  use_wildcard: "no"
```

Configuring Cron Schedules

Laravel provides a convenient way to [schedule Cron jobs](#) by scheduling a single `schedule:run` Artisan command to be run every minute. The `schedule:run` command will examine the job schedule defined in your `App\Console\Kernel` class to determine which jobs should be run.

If you would like the `schedule:run` command to be run for a Homestead site, you may set the `schedule` option to `true` when defining the site:

```
sites:
- map: homestead.test
  to: /home/vagrant/project1/public
  schedule: true
```

The Cron job for the site will be defined in the `/etc/cron.d` folder of the virtual machine.

Configuring Mailhog

Mailhog allows you to easily catch your outgoing email and examine it without actually sending the mail to its recipients. To get started, update your `.env` file to use the following mail settings:

```
MAIL_MAILER=smtp
MAIL_HOST=localhost
MAIL_PORT=1025
MAIL_USERNAME=null
MAIL_PASSWORD=null
MAIL_ENCRYPTION=null
```

Once Mailhog has been configured, you may access the Mailhog dashboard at `http://localhost:8025`.

Configuring Minio

Minio is an open source object storage server with an Amazon S3 compatible API. To install Minio, update your `Homestead.yaml` file with the following configuration option in the [features](#) section:

```
minio: true
```

By default, Minio is available on port 9600. You may access the Minio control panel by visiting `http://localhost:9600/`. The default access key is `homestead`, while the default secret key is `secretkey`. When accessing Minio, you should always use region `us-east-1`.

In order to use Minio you will need to adjust the S3 disk configuration in your `config/filesystems.php` configuration file. You will need to add the `use_path_style_endpoint` option to the disk configuration, as well as change the `url` key to `endpoint`:

```
's3' => [
    'driver' => 's3',
    'key' => env('AWS_ACCESS_KEY_ID'),
```

```
'secret' => env('AWS_SECRET_ACCESS_KEY'),
'region' => env('AWS_DEFAULT_REGION'),
'bucket' => env('AWS_BUCKET'),
'endpoint' => env('AWS_URL'),
'use_path_style_endpoint' => true,
]
```

Finally, ensure your `.env` file has the following options:

```
AWS_ACCESS_KEY_ID=homestead
AWS_SECRET_ACCESS_KEY=secretkey
AWS_DEFAULT_REGION=us-east-1
AWS_URL=http://localhost:9600
```

To provision buckets, add a `buckets` directive to your Homestead configuration file:

```
buckets:
- name: your-bucket
  policy: public
- name: your-private-bucket
  policy: none
```

Supported `policy` values include: `none`, `download`, `upload`, and `public`.

Ports

By default, the following ports are forwarded to your Homestead environment:

- **SSH:** 2222 → Forwards To 22
- **ngrok UI:** 4040 → Forwards To 4040
- **HTTP:** 8000 → Forwards To 80
- **HTTPS:** 44300 → Forwards To 443
- **MySQL:** 33060 → Forwards To 3306
- **PostgreSQL:** 54320 → Forwards To 5432
- **MongoDB:** 27017 → Forwards To 27017
- **Mailhog:** 8025 → Forwards To 8025
- **Minio:** 9600 → Forwards To 9600

Forwarding Additional Ports

If you wish, you may forward additional ports to the Vagrant box, as well as specify their protocol:

```
ports:
- send: 50000
  to: 5000
- send: 7777
  to: 777
  protocol: udp
```

Sharing Your Environment

Sometimes you may wish to share what you're currently working on with coworkers or a client. Vagrant has a built-in way to support this via `vagrant share`; however, this will not work if you have multiple sites configured in your `Homestead.yaml` file.

To solve this problem, Homestead includes its own `share` command. To get started, SSH into your Homestead machine via `vagrant ssh` and run `share homestead.test`. This will share the `homestead.test` site from your `Homestead.yaml` configuration file. You may substitute any of your other configured sites for `homestead.test`:

```
share homestead.test
```

After running the command, you will see an Ngrok screen appear which contains the activity log and the publicly accessible URLs for the shared site. If you would like to specify a custom region, subdomain, or other Ngrok runtime option, you may add them to your `share` command:

```
share homestead.test -region=eu -subdomain=laravel
```

NOTE Remember, Vagrant is inherently insecure and you are exposing your virtual machine to the Internet when running the `share` command.

Multiple PHP Versions

Homestead 6 introduced support for multiple versions of PHP on the same virtual machine. You may specify which version of PHP to use for a given site within your `Homestead.yaml` file. The available PHP versions are: "5.6", "7.0", "7.1", "7.2", "7.3", and "7.4" (the default):

```
sites:
  - map: homestead.test
    to: /home/vagrant/project1/public
    php: "7.1"
```

In addition, you may use any of the supported PHP versions via the CLI:

```
php5.6 artisan list
php7.0 artisan list
php7.1 artisan list
php7.2 artisan list
php7.3 artisan list
php7.4 artisan list
```

You may also update the default CLI version by issuing the following commands from within your Homestead virtual machine:

```
php56
php70
php71
php72
php73
php74
```

Web Servers

Homestead uses the Nginx web server by default. However, it can install Apache if `apache` is specified as a site type. While both web servers can be installed at the same time, they cannot both be *running* at the same time. The `flip` shell command is available to ease the process of switching between web servers. The `flip` command automatically determines which web server is running, shuts it off, and then starts the other server. To use this command, SSH into your Homestead machine and run the command in your terminal:

```
flip
```

Mail

Homestead includes the Postfix mail transfer agent, which is listening on port `1025` by default. So, you may instruct your application to use the `smtp` mail driver on `localhost` port `1025`. Then, all sent mail will be handled by Postfix and caught by Mailhog. To view your sent emails, open <http://localhost:8025> in your web browser.

Debugging & Profiling

Debugging Web Requests With Xdebug

Homestead includes support for step debugging using [Xdebug](#). For example, you can load a web page from a browser, and PHP will connect to your IDE to allow inspection and modification of the running code.

By default Xdebug is already running and ready to accept connections. If you need to enable Xdebug on the CLI run the `sudo phpenmod xdebug` command within your Vagrant box. Next, follow your IDE's instructions to enable debugging. Finally, configure your browser to trigger Xdebug with an extension or [bookmarklet](#).

NOTE Xdebug causes PHP to run significantly slower. To disable Xdebug, run `sudo phpdismod xdebug` within your Vagrant box and restart the FPM service.

Debugging CLI Applications

To debug a PHP CLI application, use the `xphp` shell alias inside your Vagrant box:

```
xphp path/to/script
```

Autostarting Xdebug

When debugging functional tests that make requests to the web server, it is easier to autostart debugging rather than modifying tests to pass through a custom header or cookie to trigger debugging. To force Xdebug to start automatically, modify `/etc/php/7.x/fpm/conf.d/20-xdebug.ini` inside your Vagrant box and add the following configuration:

```
; If Homestead.yaml contains a different subnet for the IP address, this address may be different...
xdebug.remote_host = 192.168.10.1
xdebug.remote_autostart = 1
```

Profiling Applications with Blackfire

[Blackfire](#) is a SaaS service for profiling web requests and CLI applications and writing performance assertions. It offers an interactive user interface which displays profile data in call-graphs and timelines. It is built for use in development, staging, and production, with no overhead for end users. It provides performance, quality, and security checks on code and `php.ini` configuration settings.

The [Blackfire Player](#) is an open-source Web Crawling, Web Testing and Web Scraping application which can work jointly with Blackfire in order to script profiling scenarios.

To enable Blackfire, use the "features" setting in your Homestead configuration file:

```
features:
  - blackfire:
      server_id: "server_id"
      server_token: "server_value"
      client_id: "client_id"
      client_token: "client_value"
```

Blackfire server credentials and client credentials [require a user account](#). Blackfire offers various options to profile an application, including a CLI tool and browser extension. Please [review the Blackfire documentation for more details](#).

Profiling PHP Performance Using XHGui

[XHGui](#) is a user interface for exploring the performance of your PHP applications. To enable XHGui, add `xhgui: 'true'` to your site configuration:

```
sites:
  -
    map: your-site.test
    to: /home/vagrant/your-site/public
    type: "apache"
    xhgui: 'true'
```

If the site already exists, make sure to run `vagrant provision` after updating your configuration.

To profile a web request, add `xhgui=on` as a query parameter to a request. XHGui will automatically attach a cookie to the response so that subsequent requests do not need the query string value. You may view your application profile results by browsing to `http://your-site.test/xhgui`.

To profile a CLI request using XHGui, prefix the command with `XHGUI=on`:

```
XHGUI=on path/to/script
```

CLI profile results may be viewed in the same way as web profile results.

Note that the act of profiling slows down script execution, and absolute times may be as much as twice as real-

world requests. Therefore, always compare percentage improvements and not absolute numbers. Also, be aware the execution time includes any time spent paused in a debugger.

Since performance profiles take up significant disk space, they are deleted automatically after a few days.

Network Interfaces

The `networks` property of the `Homestead.yaml` configures network interfaces for your Homestead environment. You may configure as many interfaces as necessary:

```
networks:
  - type: "private_network"
    ip: "192.168.10.20"
```

To enable a [bridged](#) interface, configure a `bridge` setting and change the network type to `public_network`:

```
networks:
  - type: "public_network"
    ip: "192.168.10.20"
    bridge: "en1: Wi-Fi (AirPort)"
```

To enable [DHCP](#), just remove the `ip` option from your configuration:

```
networks:
  - type: "public_network"
    bridge: "en1: Wi-Fi (AirPort)"
```

Extending Homestead

You may extend Homestead using the `after.sh` script in the root of your Homestead directory. Within this file, you may add any shell commands that are necessary to properly configure and customize your virtual machine.

When customizing Homestead, Ubuntu may ask you if you would like to keep a package's original configuration or overwrite it with a new configuration file. To avoid this, you should use the following command when installing packages to avoid overwriting any configuration previously written by Homestead:

```
sudo apt-get -y \
  -o Dpkg::Options::="--force-confdef" \
  -o Dpkg::Options::="--force-confold" \
  install your-package
```

User Customizations

When using Homestead in a team setting, you may want to tweak Homestead to better fit your personal development style. You may create a `user-customizations.sh` file in the root of your Homestead directory (The same directory containing your `Homestead.yaml`). Within this file, you may make any customization you would like; however, the `user-customizations.sh` should not be version controlled.

Updating Homestead

Before you begin updating Homestead ensure you have removed your current virtual machine by running the following command in your Homestead directory:

```
vagrant destroy
```

Next, you need to update the Homestead source code. If you cloned the repository you can run the following commands at the location you originally cloned the repository:

```
git fetch
```

```
git pull origin release
```

These commands pull the latest Homestead code from the GitHub repository, fetches the latest tags, and then checks out the latest tagged release. You can find the latest stable release version on the [GitHub releases page](#).

If you have installed Homestead via your project's `composer.json` file, you should ensure your `composer.json` file contains `"laravel/homestead": "^11"` and update your dependencies:

```
composer update
```

Then, you should update the Vagrant box using the `vagrant box update` command:

```
vagrant box update
```

Next, you should run the `bash init.sh` command from the Homestead directory in order to update some additional configuration files. You will be asked whether you wish to overwrite your existing `Homestead.yaml`, `after.sh`, and `aliases` files:

```
// Mac / Linux...
bash init.sh
```

```
// Windows...
init.bat
```

Finally, you will need to regenerate your Homestead box to utilize the latest Vagrant installation:

```
vagrant up
```

Provider Specific Settings

VirtualBox

`natdnshostresolver`

By default, Homestead configures the `natdnshostresolver` setting to `on`. This allows Homestead to use your host operating system's DNS settings. If you would like to override this behavior, add the following lines to your `Homestead.yaml` file:

```
provider: virtualbox
natdnshostresolver: 'off'
```

Symbolic Links On Windows

If symbolic links are not working properly on your Windows machine, you may need to add the following block to your `vagrantfile`:

```
config.vm.provider "virtualbox" do |v|
  v.customize ["setextradata", :id, "VBoxInternal2/SharedFoldersEnableSymlinksCreate/v-root", "1"]
end
```

Getting Started

Valet

Laravel Valet

- [Introduction](#)
 - [Valet Or Homestead](#)
- [Installation](#)
 - [Upgrading](#)
- [Serving Sites](#)
 - [The "Park" Command](#)
 - [The "Link" Command](#)
 - [Securing Sites With TLS](#)
 - [Serving a Default Site](#)
- [Sharing Sites](#)
- [Site Specific Environment Variables](#)
- [Proxying Services](#)
- [Custom Valet Drivers](#)
 - [Local Drivers](#)
- [Other Valet Commands](#)
- [Valet Directories & Files](#)

Introduction

Valet is a Laravel development environment for Mac minimalists. No Vagrant, no `/etc/hosts` file. You can even share your sites publicly using local tunnels. *Yeah, we like it too.*

Laravel Valet configures your Mac to always run [Nginx](#) in the background when your machine starts. Then, using [DnsMasq](#), Valet proxies all requests on the `*.test` domain to point to sites installed on your local machine.

In other words, a blazing fast Laravel development environment that uses roughly 7 MB of RAM. Valet isn't a complete replacement for Vagrant or Homestead, but provides a great alternative if you want flexible basics, prefer extreme speed, or are working on a machine with a limited amount of RAM.

Out of the box, Valet support includes, but is not limited to:

Laravel	ExpressionEngine	Slim
Lumen	Jigsaw	Statamic
Bedrock	Joomla	Static HTML
CakePHP 3	Katana	Symfony
Concrete5	Kirby	WordPress
Contao	Magento	Zend
Craft	OctoberCMS	
Drupal	Sculpin	

However, you may extend Valet with your own [custom drivers](#).

Valet Or Homestead

As you may know, Laravel offers [Homestead](#), another local Laravel development environment. Homestead and Valet differ in regards to their intended audience and their approach to local development. Homestead offers an entire Ubuntu virtual machine with automated Nginx configuration. Homestead is a wonderful choice if you

want a fully virtualized Linux development environment or are on Windows / Linux.

Valet only supports Mac, and requires you to install PHP and a database server directly onto your local machine. This is easily achieved by using [Homebrew](#) with commands like `brew install php` and `brew install mysql`. Valet provides a blazing fast local development environment with minimal resource consumption, so it's great for developers who only require PHP / MySQL and do not need a fully virtualized development environment.

Both Valet and Homestead are great choices for configuring your Laravel development environment. Which one you choose will depend on your personal taste and your team's needs.

Installation

Valet requires macOS and [Homebrew](#). Before installation, you should make sure that no other programs such as Apache or Nginx are binding to your local machine's port 80.

- Install or update [Homebrew](#) to the latest version using `brew update`.
- Install PHP 7.4 using Homebrew via `brew install php`.
- Install [Composer](#).
- Install Valet with Composer via `composer global require laravel/valet`. Make sure the `~/.composer/vendor/bin` directory is in your system's "PATH".
- Run the `valet install` command. This will configure and install Valet and DnsMasq, and register Valet's daemon to launch when your system starts.

Once Valet is installed, try pinging any `*.test` domain on your terminal using a command such as `ping foobar.test`. If Valet is installed correctly you should see this domain responding on `127.0.0.1`.

Valet will automatically start its daemon each time your machine boots. There is no need to run `valet start` or `valet install` ever again once the initial Valet installation is complete.

Using Another Domain

By default, Valet serves your projects using the `.test` TLD. If you'd like to use another domain, you can do so using the `valet tld tld-name` command.

For example, if you'd like to use `.app` instead of `.test`, run `valet tld app` and Valet will start serving your projects at `*.app` automatically.

Database

If you need a database, try MySQL by running `brew install mysql@5.7` on your command line. Once MySQL has been installed, you may start it using the `brew services start mysql@5.7` command. You can then connect to the database at `127.0.0.1` using the `root` username and an empty string for the password.

PHP Versions

Valet allows you to switch PHP versions using the `valet use php@version` command. Valet will install the specified PHP version via Brew if it is not already installed:

```
valet use php@7.2
```

```
valet use php
```

NOTE Valet only serves one PHP version at a time, even if you have multiple PHP versions installed.

Resetting Your Installation

If you are having trouble getting your Valet installation to run properly, executing the `composer global update` command followed by `valet install` will reset your installation and can solve a variety of problems. In rare cases it may be necessary to "hard reset" Valet by executing `valet uninstall --force` followed by `valet install`.

Upgrading

You may update your Valet installation using the `composer global update` command in your terminal. After upgrading, it is good practice to run the `valet install` command so Valet can make additional upgrades to your configuration files if necessary.

Serving Sites

Once Valet is installed, you're ready to start serving sites. Valet provides two commands to help you serve your Laravel sites: `park` and `link`.

The `park` Command

- Create a new directory on your Mac by running something like `mkdir ~/Sites`. Next, `cd ~/Sites` and run `valet park`. This command will register your current working directory as a path that Valet should search for sites.
- Next, create a new Laravel site within this directory: `laravel new blog`.
- Open `http://blog.test` in your browser.

That's all there is to it. Now, any Laravel project you create within your "parked" directory will automatically be served using the `http://folder-name.test` convention.

The `link` Command

The `link` command may also be used to serve your Laravel sites. This command is useful if you want to serve a single site in a directory and not the entire directory.

- To use the command, navigate to one of your projects and run `valet link app-name` in your terminal. Valet will create a symbolic link in `~/.config/valet/Sites` which points to your current working directory.
- After running the `link` command, you can access the site in your browser at `http://app-name.test`.

To see a listing of all of your linked directories, run the `valet links` command. You may use `valet unlink app-name` to destroy the symbolic link.

TIP You can use `valet link` to serve the same project from multiple (sub)domains. To add a subdomain or another domain to your project run `valet link subdomain.app-name` from the project folder.

Securing Sites With TLS

By default, Valet serves sites over plain HTTP. However, if you would like to serve a site over encrypted TLS using HTTP/2, use the `secure` command. For example, if your site is being served by Valet on the `laravel.test` domain, you should run the following command to secure it:

```
valet secure laravel
```

To "unsecure" a site and revert back to serving its traffic over plain HTTP, use the `unsecure` command. Like the `secure` command, this command accepts the host name that you wish to unsecure:

```
valet unsecure laravel
```

Serving A Default Site

Sometimes, you may wish to configure Valet to serve a "default" site instead of a 404 when visiting an unknown test domain. To accomplish this, you may add a `default` option to your `~/.config/valet/config.json` configuration file containing the path to the site that should serve as your default site:

```
"default": "/Users/Sally/Sites/foo",
```

Sharing Sites

Valet even includes a command to share your local sites with the world, providing an easy way to test your site on mobile devices or share it with team members and clients. No additional software installation is required once Valet is installed.

Sharing Sites Via Ngrok

To share a site, navigate to the site's directory in your terminal and run the `valet share` command. A publicly accessible URL will be inserted into your clipboard and is ready to paste directly into your browser or share with your team.

To stop sharing your site, hit `control + c` to cancel the process.

TIP You may pass additional parameters to the share command, such as `valet share --region=eu`. For more information, consult the [ngrok documentation](#).

Sharing Sites Via Expose

If you have [Expose](#) installed, you can share your site by navigating to the site's directory in your terminal and running the `expose` command. Consult the expose documentation for additional command-line parameters it supports. After sharing the site, Expose will display the sharable URL that you may use on your other devices or amongst team members.

To stop sharing your site, hit `control + c` to cancel the process.

Sharing Sites On Your Local Network

Valet restricts incoming traffic to the internal `127.0.0.1` interface by default. This way your development machine isn't exposed to security risks from the Internet.

If you wish to allow other devices on your local network to access the Valet sites on your machine via your machine's IP address (eg: `192.168.1.10/app-name.test`), you will need to manually edit the appropriate Nginx configuration file for that site to remove the restriction on the `listen` directive by removing the `127.0.0.1:` prefix on the directive for ports 80 and 443.

If you have not run `valet secure` on the project, you can open up network access for all non-HTTPS sites by editing the `/usr/local/etc/nginx/valet/valet.conf` file. However, if you're serving the project site over HTTPS (you have run `valet secure` for the site) then you should edit the `~/config/valet/Nginx/app-name.test` file.

Once you have updated your Nginx configuration, run the `valet restart` command to apply the configuration changes.

Site Specific Environment Variables

Some applications using other frameworks may depend on server environment variables but do not provide a way for those variables to be configured within your project. Valet allows you to configure site specific environment variables by adding a `.valet-env.php` file within the root of your project. These variables will be added to the `$_SERVER` global array:

```
<?php

// Set $_SERVER['key'] to "value" for the foo.test site...
return [
    'foo' => [
        'key' => 'value',
    ],
];

// Set $_SERVER['key'] to "value" for all sites...
return [
    '*' => [
        'key' => 'value',
    ],
];
```

Proxying Services

Sometimes you may wish to proxy a Valet domain to another service on your local machine. For example, you may occasionally need to run Valet while also running a separate site in Docker; however, Valet and Docker can't both bind to port 80 at the same time.

To solve this, you may use the `proxy` command to generate a proxy. For example, you may proxy all traffic from `http://elasticsearch.test` to `http://127.0.0.1:9200`:

```
valet proxy elasticsearch http://127.0.0.1:9200
```

You may remove a proxy using the `unproxy` command:

```
valet unproxy elasticsearch
```

You may use the `proxies` command to list all site configuration that are proxied:

```
valet proxies
```

Custom Valet Drivers

You can write your own Valet "driver" to serve PHP applications running on another framework or CMS that is not natively supported by Valet. When you install Valet, a `~/.config/valet/Drivers` directory is created which contains a `SampleValetDriver.php` file. This file contains a sample driver implementation to demonstrate how to write a custom driver. Writing a driver only requires you to implement three methods: `serves`, `isStaticFile`, and `frontControllerPath`.

All three methods receive the `$sitePath`, `$siteName`, and `$uri` values as their arguments. The `$sitePath` is the fully qualified path to the site being served on your machine, such as `/Users/Lisa/Sites/my-project`. The `$siteName` is the "host" / "site name" portion of the domain (`my-project`). The `$uri` is the incoming request URI (`/foo/bar`).

Once you have completed your custom Valet driver, place it in the `~/.config/valet/Drivers` directory using the `FrameworkValetDriver.php` naming convention. For example, if you are writing a custom valet driver for WordPress, your file name should be `WordPressValetDriver.php`.

Let's take a look at a sample implementation of each method your custom Valet driver should implement.

The `serves` Method

The `serves` method should return `true` if your driver should handle the incoming request. Otherwise, the method should return `false`. So, within this method you should attempt to determine if the given `$sitePath` contains a project of the type you are trying to serve.

For example, let's pretend we are writing a `WordPressValetDriver`. Our `serves` method might look something like this:

```
/**
 * Determine if the driver serves the request.
 *
 * @param string $sitePath
 * @param string $siteName
 * @param string $uri
 * @return bool
 */
public function serves($sitePath, $siteName, $uri)
{
    return is_dir($sitePath.'/wp-admin');
}
```

The `isStaticFile` Method

The `isStaticFile` should determine if the incoming request is for a file that is "static", such as an image or a stylesheet. If the file is static, the method should return the fully qualified path to the static file on disk. If the

incoming request is not for a static file, the method should return false:

```
/**
 * Determine if the incoming request is for a static file.
 *
 * @param string $sitePath
 * @param string $siteName
 * @param string $uri
 * @return string|false
 */
public function isStaticFile($sitePath, $siteName, $uri)
{
    if (file_exists($staticFilePath = $sitePath.'/public/'.$uri)) {
        return $staticFilePath;
    }

    return false;
}
```

NOTE The `isStaticFile` method will only be called if the `serves` method returns true for the incoming request and the request URI is not `/`.

The `frontControllerPath` Method

The `frontControllerPath` method should return the fully qualified path to your application's "front controller", which is typically your "index.php" file or equivalent:

```
/**
 * Get the fully resolved path to the application's front controller.
 *
 * @param string $sitePath
 * @param string $siteName
 * @param string $uri
 * @return string
 */
public function frontControllerPath($sitePath, $siteName, $uri)
{
    return $sitePath.'/public/index.php';
}
```

Local Drivers

If you would like to define a custom Valet driver for a single application, create a `LocalValetDriver.php` in the application's root directory. Your custom driver may extend the base `ValetDriver` class or extend an existing application specific driver such as the `LaravelValetDriver`:

```
class LocalValetDriver extends LaravelValetDriver
{
    /**
     * Determine if the driver serves the request.
     *
     * @param string $sitePath
     * @param string $siteName
     * @param string $uri
     * @return bool
     */
    public function serves($sitePath, $siteName, $uri)
    {
        return true;
    }

    /**
     * Get the fully resolved path to the application's front controller.
     *
     * @param string $sitePath
     * @param string $siteName
     * @param string $uri
     * @return string
     */
    public function frontControllerPath($sitePath, $siteName, $uri)
    {
        return $sitePath.'/public_html/index.php';
    }
}
```

Other Valet Commands

Command	Description
<code>valet forget</code>	Run this command from a "parked" directory to remove it from the parked directory list.
<code>valet log</code>	View a list of logs which are written by Valet's services.
<code>valet paths</code>	View all of your "parked" paths.
<code>valet restart</code>	Restart the Valet daemon.
<code>valet start</code>	Start the Valet daemon.
<code>valet stop</code>	Stop the Valet daemon.
<code>valet trust</code>	Add sudoers files for Brew and Valet to allow Valet commands to be run without prompting for passwords.
<code>valet uninstall</code>	Uninstall Valet: Shows instructions for manual uninstall; or pass the <code>--force</code> parameter to aggressively delete all of Valet.

Valet Directories & Files

You may find the following directory and file information helpful while troubleshooting issues with your Valet environment:

File / Path	Description
<code>~/.config/valet/</code>	Contains all of Valet's configuration. You may wish to maintain a backup of this folder.
<code>~/.config/valet/dnsmasq.d/</code>	Contains DNSMasq's configuration.
<code>~/.config/valet/Drivers/</code>	Contains custom Valet drivers.
<code>~/.config/valet/Extensions/</code>	Contains custom Valet extensions / commands.
<code>~/.config/valet/Nginx/</code>	Contains all Valet generated Nginx site configurations. These files are rebuilt when running the <code>install</code> , <code>secure</code> , and <code>tld</code> commands.
<code>~/.config/valet/Sites/</code>	Contains all symbolic links for linked projects.
<code>~/.config/valet/config.json</code>	Valet's master configuration file
<code>~/.config/valet/valet.sock</code>	The PHP-FPM socket used by Valet's Nginx configuration. This will only exist if PHP is running properly.
<code>~/.config/valet/Log/fpm-php.www.log</code>	User log for PHP errors.
<code>~/.config/valet/Log/nginx-error.log</code>	User log for Nginx errors.
<code>/usr/local/var/log/php-fpm.log</code>	System log for PHP-FPM errors.
<code>/usr/local/var/log/nginx</code>	Contains Nginx access and error logs.
<code>/usr/local/etc/php/X.X/conf.d</code>	Contains <code>*.ini</code> files for various PHP configuration settings.
<code>/usr/local/etc/php/X.X/php-fpm.d/valet-fpm.conf</code>	PHP-FPM pool configuration file.
<code>~/.composer/vendor/laravel/valet/cli/stubs/secure.valet.conf</code>	The default Nginx configuration used for building site certificates.

Getting Started

Deployment

Deployment

- [Introduction](#)
- [Server Configuration](#)
 - [Nginx](#)
- [Optimization](#)
 - [Autoloader Optimization](#)
 - [Optimizing Configuration Loading](#)
 - [Optimizing Route Loading](#)
 - [Optimizing View Loading](#)
- [Deploying With Forge / Vapor](#)

Introduction

When you're ready to deploy your Laravel application to production, there are some important things you can do to make sure your application is running as efficiently as possible. In this document, we'll cover some great starting points for making sure your Laravel application is deployed properly.

Server Configuration

Nginx

If you are deploying your application to a server that is running Nginx, you may use the following configuration file as a starting point for configuring your web server. Most likely, this file will need to be customized depending on your server's configuration. If you would like assistance in managing your server, consider using a service such as [Laravel Forge](#):

```
server {
    listen 80;
    server_name example.com;
    root /srv/example.com/public;

    add_header X-Frame-Options "SAMEORIGIN";
    add_header X-XSS-Protection "1; mode=block";
    add_header X-Content-Type-Options "nosniff";

    index index.php;

    charset utf-8;

    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }

    location = /favicon.ico { access_log off; log_not_found off; }
    location = /robots.txt  { access_log off; log_not_found off; }

    error_page 404 /index.php;

    location ~ \.php$ {
        fastcgi_pass unix:/var/run/php/php7.4-fpm.sock;
        fastcgi_param SCRIPT_FILENAME $realpath_root$fastcgi_script_name;
        include fastcgi_params;
    }

    location ~ /\.(!well-known).* {
        deny all;
    }
}
```

Optimization

Autoloader Optimization

When deploying to production, make sure that you are optimizing Composer's class autoloader map so Composer can quickly find the proper file to load for a given class:

```
composer install --optimize-autoloader --no-dev
```

TIP In addition to optimizing the autoloader, you should always be sure to include a `composer.lock` file in your project's source control repository. Your project's dependencies can be installed much faster when a `composer.lock` file is present.

Optimizing Configuration Loading

When deploying your application to production, you should make sure that you run the `config:cache` Artisan command during your deployment process:

```
php artisan config:cache
```

This command will combine all of Laravel's configuration files into a single, cached file, which greatly reduces the number of trips the framework must make to the filesystem when loading your configuration values.

NOTE If you execute the `config:cache` command during your deployment process, you should be sure that you are only calling the `env` function from within your configuration files. Once the configuration has been cached, the `.env` file will not be loaded and all calls to the `env` function will return `null`.

Optimizing Route Loading

If you are building a large application with many routes, you should make sure that you are running the `route:cache` Artisan command during your deployment process:

```
php artisan route:cache
```

This command reduces all of your route registrations into a single method call within a cached file, improving the performance of route registration when registering hundreds of routes.

NOTE Since this feature uses PHP serialization, you may only cache the routes for applications that exclusively use controller based routes. PHP is not able to serialize Closures.

Optimizing View Loading

When deploying your application to production, you should make sure that you run the `view:cache` Artisan command during your deployment process:

```
php artisan view:cache
```

This command precompiles all your Blade views so they are not compiled on demand, improving the performance of each request that returns a view.

Deploying With Forge / Vapor

If you aren't quite ready to manage your own server configuration or aren't comfortable configuring all of the various services needed to run a robust Laravel application, [Laravel Forge](#) is a wonderful alternative.

Laravel Forge can create servers on various infrastructure providers such as DigitalOcean, Linode, AWS, and more. In addition, Forge installs and manages all of the tools needed to build robust Laravel applications, such as Nginx, MySQL, Redis, Memcached, Beanstalk, and more.

Laravel Vapor

If you would like a totally serverless, auto-scaling deployment platform tuned for Laravel, check out [Laravel Vapor](#). Laravel Vapor is a serverless deployment platform for Laravel, powered by AWS. Launch your Laravel

infrastructure on Vapor and fall in love with the scalable simplicity of serverless. Laravel Vapor is fine-tuned by Laravel's creators to work seamlessly with the framework so you can keep writing your Laravel applications exactly like you're used to.

Architecture Concepts

Request Lifecycle

Request Lifecycle

- [Introduction](#)
- [Lifecycle Overview](#)
- [Focus On Service Providers](#)

Introduction

When using any tool in the "real world", you feel more confident if you understand how that tool works. Application development is no different. When you understand how your development tools function, you feel more comfortable and confident using them.

The goal of this document is to give you a good, high-level overview of how the Laravel framework works. By getting to know the overall framework better, everything feels less "magical" and you will be more confident building your applications. If you don't understand all of the terms right away, don't lose heart! Just try to get a basic grasp of what is going on, and your knowledge will grow as you explore other sections of the documentation.

Lifecycle Overview

First Things

The entry point for all requests to a Laravel application is the `public/index.php` file. All requests are directed to this file by your web server (Apache / Nginx) configuration. The `index.php` file doesn't contain much code. Rather, it is a starting point for loading the rest of the framework.

The `index.php` file loads the Composer generated autoloader definition, and then retrieves an instance of the Laravel application from `bootstrap/app.php` script. The first action taken by Laravel itself is to create an instance of the application / [service container](#).

HTTP / Console Kernels

Next, the incoming request is sent to either the HTTP kernel or the console kernel, depending on the type of request that is entering the application. These two kernels serve as the central location that all requests flow through. For now, let's just focus on the HTTP kernel, which is located in `app/Http/Kernel.php`.

The HTTP kernel extends the `Illuminate\Foundation\Http\Kernel` class, which defines an array of bootstrappers that will be run before the request is executed. These bootstrappers configure error handling, configure logging, [detect the application environment](#), and perform other tasks that need to be done before the request is actually handled.

The HTTP kernel also defines a list of HTTP [middleware](#) that all requests must pass through before being handled by the application. These middleware handle reading and writing the [HTTP session](#), determining if the application is in maintenance mode, [verifying the CSRF token](#), and more.

The method signature for the HTTP kernel's `handle` method is quite simple: receive a `Request` and return a `Response`. Think of the Kernel as being a big black box that represents your entire application. Feed it HTTP requests and it will return HTTP responses.

Service Providers

One of the most important Kernel bootstrapping actions is loading the [service providers](#) for your application. All of the service providers for the application are configured in the `config/app.php` configuration file's `providers` array. First, the `register` method will be called on all providers, then, once all providers have been

registered, the `boot` method will be called.

Service providers are responsible for bootstrapping all of the framework's various components, such as the database, queue, validation, and routing components. Since they bootstrap and configure every feature offered by the framework, service providers are the most important aspect of the entire Laravel bootstrap process.

Dispatch Request

Once the application has been bootstrapped and all service providers have been registered, the `Request` will be handed off to the router for dispatching. The router will dispatch the request to a route or controller, as well as run any route specific middleware.

Focus On Service Providers

Service providers are truly the key to bootstrapping a Laravel application. The application instance is created, the service providers are registered, and the request is handed to the bootstrapped application. It's really that simple!

Having a firm grasp of how a Laravel application is built and bootstrapped via service providers is very valuable. Your application's default service providers are stored in the `app/Providers` directory.

By default, the `AppServiceProvider` is fairly empty. This provider is a great place to add your application's own bootstrapping and service container bindings. For large applications, you may wish to create several service providers, each with a more granular type of bootstrapping.

Architecture Concepts

Service Container

Service Container

- [Introduction](#)
- [Binding](#)
 - [Binding Basics](#)
 - [Binding Interfaces To Implementations](#)
 - [Contextual Binding](#)
 - [Binding Primitives](#)
 - [Binding Typed Variadics](#)
 - [Tagging](#)
 - [Extending Bindings](#)
- [Resolving](#)
 - [The Make Method](#)
 - [Automatic Injection](#)
- [Container Events](#)
- [PSR-11](#)

Introduction

The Laravel service container is a powerful tool for managing class dependencies and performing dependency injection. Dependency injection is a fancy phrase that essentially means this: class dependencies are "injected" into the class via the constructor or, in some cases, "setter" methods.

Let's look at a simple example:

```
<?php

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;
use App\Repositories\UserRepository;
use App\User;

class UserController extends Controller
{
    /**
     * The user repository implementation.
     *
     * @var UserRepository
     */
    protected $users;

    /**
     * Create a new controller instance.
     *
     * @param UserRepository $users
     * @return void
     */
    public function __construct(UserRepository $users)
    {
        $this->users = $users;
    }

    /**
     * Show the profile for the given user.
     *
     * @param int $id
     * @return Response
     */
    public function show($id)
    {
        $user = $this->users->find($id);

        return view('user.profile', ['user' => $user]);
    }
}
```

```
}
```

In this example, the `UserController` needs to retrieve users from a data source. So, we will **inject** a service that is able to retrieve users. In this context, our `UserRepository` most likely uses [Eloquent](#) to retrieve user information from the database. However, since the repository is injected, we are able to easily swap it out with another implementation. We are also able to easily "mock", or create a dummy implementation of the `UserRepository` when testing our application.

A deep understanding of the Laravel service container is essential to building a powerful, large application, as well as for contributing to the Laravel core itself.

Binding

Binding Basics

Almost all of your service container bindings will be registered within [service providers](#), so most of these examples will demonstrate using the container in that context.

TIP There is no need to bind classes into the container if they do not depend on any interfaces. The container does not need to be instructed on how to build these objects, since it can automatically resolve these objects using reflection.

Simple Bindings

Within a service provider, you always have access to the container via the `$this->app` property. We can register a binding using the `bind` method, passing the class or interface name that we wish to register along with a closure that returns an instance of the class:

```
$this->app->bind('HelpSpot\API', function ($app) {  
    return new \HelpSpot\API($app->make('HttpClient'));  
});
```

Note that we receive the container itself as an argument to the resolver. We can then use the container to resolve sub-dependencies of the object we are building.

Binding A Singleton

The `singleton` method binds a class or interface into the container that should only be resolved one time. Once a singleton binding is resolved, the same object instance will be returned on subsequent calls into the container:

```
$this->app->singleton('HelpSpot\API', function ($app) {  
    return new \HelpSpot\API($app->make('HttpClient'));  
});
```

Binding Instances

You may also bind an existing object instance into the container using the `instance` method. The given instance will always be returned on subsequent calls into the container:

```
$api = new \HelpSpot\API(new HttpClient);  
$this->app->instance('HelpSpot\API', $api);
```

Binding Interfaces To Implementations

A very powerful feature of the service container is its ability to bind an interface to a given implementation. For example, let's assume we have an `EventPusher` interface and a `RedisEventPusher` implementation. Once we have coded our `RedisEventPusher` implementation of this interface, we can register it with the service container like so:

```
$this->app->bind(  
    'App\Contracts\EventPusher',  
    'App\Services\RedisEventPusher'
```

```
);
```

This statement tells the container that it should inject the `RedisEventPusher` when a class needs an implementation of `EventPusher`. Now we can type-hint the `EventPusher` interface in a constructor, or any other location where dependencies are injected by the service container:

```
use App\Contracts\EventPusher;

/**
 * Create a new class instance.
 *
 * @param EventPusher $pusher
 * @return void
 */
public function __construct(EventPusher $pusher)
{
    $this->pusher = $pusher;
}
```

Contextual Binding

Sometimes you may have two classes that utilize the same interface, but you wish to inject different implementations into each class. For example, two controllers may depend on different implementations of the `Illuminate\Contracts\Filesystem\Filesystem` [contract](#). Laravel provides a simple, fluent interface for defining this behavior:

```
use App\Http\Controllers\PhotoController;
use App\Http\Controllers\UploadController;
use App\Http\Controllers\VideoController;
use Illuminate\Contracts\Filesystem\Filesystem;
use Illuminate\Support\Facades\Storage;

$this->app->when(PhotoController::class)
    ->needs(Filesystem::class)
    ->give(function () {
        return Storage::disk('local');
    });

$this->app->when([VideoController::class, UploadController::class])
    ->needs(Filesystem::class)
    ->give(function () {
        return Storage::disk('s3');
    });
```

Binding Primitives

Sometimes you may have a class that receives some injected classes, but also needs an injected primitive value such as an integer. You may easily use contextual binding to inject any value your class may need:

```
$this->app->when('App\Http\Controllers\UserController')
    ->needs('$variableName')
    ->give($value);
```

Sometimes a class may depend on an array of tagged instances. Using the `giveTagged` method, you may easily inject all of the container bindings with that tag:

```
$this->app->when(ReportAggregator::class)
    ->needs('$reports')
    ->giveTagged('reports');
```

Binding Typed Variadics

Occasionally you may have a class that receives an array of typed objects using a variadic constructor argument:

```
class Firewall
{
    protected $logger;
    protected $filters;

    public function __construct(Logger $logger, Filter ...$filters)
```

```
    {
        $this->logger = $logger;
        $this->filters = $filters;
    }
}
```

Using contextual binding, you may resolve this dependency by providing the `give` method with a Closure that returns an array of resolved `Filter` instances:

```
$this->app->when(Firewall::class)
    ->needs(Filter::class)
    ->give(function ($app) {
        return [
            $app->make(NullFilter::class),
            $app->make(ProfanityFilter::class),
            $app->make(TooLongFilter::class),
        ];
    });
```

For convenience, you may also just provide an array of class names to be resolved by the container whenever `Firewall` needs `Filter` instances:

```
$this->app->when(Firewall::class)
    ->needs(Filter::class)
    ->give([
        NullFilter::class,
        ProfanityFilter::class,
        TooLongFilter::class,
    ]);
```

Variadic Tag Dependencies

Sometimes a class may have a variadic dependency that is type-hinted as a given class (`Report ...$reports`). Using the `needs` and `giveTagged` methods, you may easily inject all of the container bindings with that tag for the given dependency:

```
$this->app->when(ReportAggregator::class)
    ->needs(Report::class)
    ->giveTagged('reports');
```

Tagging

Occasionally, you may need to resolve all of a certain "category" of binding. For example, perhaps you are building a report aggregator that receives an array of many different `Report` interface implementations. After registering the `Report` implementations, you can assign them a tag using the `tag` method:

```
$this->app->bind('SpeedReport', function () {
    //
});

$this->app->bind('MemoryReport', function () {
    //
});

$this->app->tag(['SpeedReport', 'MemoryReport'], 'reports');
```

Once the services have been tagged, you may easily resolve them all via the `tagged` method:

```
$this->app->bind('ReportAggregator', function ($app) {
    return new ReportAggregator($app->tagged('reports'));
});
```

Extending Bindings

The `extend` method allows the modification of resolved services. For example, when a service is resolved, you may run additional code to decorate or configure the service. The `extend` method accepts a Closure, which should return the modified service, as its only argument. The Closure receives the service being resolved and the container instance:

```
$this->app->extend(Service::class, function ($service, $app) {
```

```
        return new DecoratedService($service);
    });
```

Resolving

The make Method

You may use the `make` method to resolve a class instance out of the container. The `make` method accepts the name of the class or interface you wish to resolve:

```
$api = $this->app->make('HelpSpot\API');
```

If you are in a location of your code that does not have access to the `$app` variable, you may use the global `resolve` helper:

```
$api = resolve('HelpSpot\API');
```

If some of your class' dependencies are not resolvable via the container, you may inject them by passing them as an associative array into the `makeWith` method:

```
$api = $this->app->makeWith('HelpSpot\API', ['id' => 1]);
```

Automatic Injection

Alternatively, and importantly, you may "type-hint" the dependency in the constructor of a class that is resolved by the container, including [controllers](#), [event listeners](#), [middleware](#), and more. Additionally, you may type-hint dependencies in the `handle` method of [queued jobs](#). In practice, this is how most of your objects should be resolved by the container.

For example, you may type-hint a repository defined by your application in a controller's constructor. The repository will automatically be resolved and injected into the class:

```
<?php

namespace App\Http\Controllers;

use App\Users\Repository as UserRepository;

class UserController extends Controller
{
    /**
     * The user repository instance.
     */
    protected $users;

    /**
     * Create a new controller instance.
     *
     * @param UserRepository $users
     * @return void
     */
    public function __construct(UserRepository $users)
    {
        $this->users = $users;
    }

    /**
     * Show the user with the given ID.
     *
     * @param int $id
     * @return Response
     */
    public function show($id)
    {
        //
    }
}
```

Container Events

The service container fires an event each time it resolves an object. You may listen to this event using the `resolving` method:

```
$this->app->resolving(function ($object, $app) {  
    // Called when container resolves object of any type...  
});  
  
$this->app->resolving(\HelpSpot\API::class, function ($api, $app) {  
    // Called when container resolves objects of type "HelpSpot\API"...  
});
```

As you can see, the object being resolved will be passed to the callback, allowing you to set any additional properties on the object before it is given to its consumer.

PSR-11

Laravel's service container implements the [PSR-11](#) interface. Therefore, you may type-hint the PSR-11 container interface to obtain an instance of the Laravel container:

```
use Psr\Container\ContainerInterface;  
  
Route::get('/', function (ContainerInterface $container) {  
    $service = $container->get('Service');  
  
    //  
});
```

An exception is thrown if the given identifier can't be resolved. The exception will be an instance of `Psr\Container\NotFoundExceptionInterface` if the identifier was never bound. If the identifier was bound but was unable to be resolved, an instance of `Psr\Container\ContainerExceptionInterface` will be thrown.

Architecture Concepts

Service Providers

Service Providers

- [Introduction](#)
- [Writing Service Providers](#)
 - [The Register Method](#)
 - [The Boot Method](#)
- [Registering Providers](#)
- [Deferred Providers](#)

Introduction

Service providers are the central place of all Laravel application bootstrapping. Your own application, as well as all of Laravel's core services are bootstrapped via service providers.

But, what do we mean by "bootstrapped"? In general, we mean **registering** things, including registering service container bindings, event listeners, middleware, and even routes. Service providers are the central place to configure your application.

If you open the `config/app.php` file included with Laravel, you will see a `providers` array. These are all of the service provider classes that will be loaded for your application. Note that many of these are "deferred" providers, meaning they will not be loaded on every request, but only when the services they provide are actually needed.

In this overview you will learn how to write your own service providers and register them with your Laravel application.

Writing Service Providers

All service providers extend the `Illuminate\Support\ServiceProvider` class. Most service providers contain a `register` and a `boot` method. Within the `register` method, you should **only bind things into the [service container](#)**. You should never attempt to register any event listeners, routes, or any other piece of functionality within the `register` method.

The Artisan CLI can generate a new provider via the `make:provider` command:

```
php artisan make:provider RiakServiceProvider
```

The Register Method

As mentioned previously, within the `register` method, you should only bind things into the [service container](#). You should never attempt to register any event listeners, routes, or any other piece of functionality within the `register` method. Otherwise, you may accidentally use a service that is provided by a service provider which has not loaded yet.

Let's take a look at a basic service provider. Within any of your service provider methods, you always have access to the `$app` property which provides access to the service container:

```
<?php

namespace App\Providers;

use Illuminate\Support\ServiceProvider;
use Riak\Connection;

class RiakServiceProvider extends ServiceProvider
{
    /**
     * Register any application services.
     */
}
```

```
*
* @return void
*/
public function register()
{
    $this->app->singleton(Connection::class, function ($app) {
        return new Connection(config('riak'));
    });
}
```

This service provider only defines a `register` method, and uses that method to define an implementation of `Riak\Connection` in the service container. If you don't understand how the service container works, check out [its documentation](#).

The bindings And singletons Properties

If your service provider registers many simple bindings, you may wish to use the `bindings` and `singletons` properties instead of manually registering each container binding. When the service provider is loaded by the framework, it will automatically check for these properties and register their bindings:

```
<?php

namespace App\Providers;

use App\Contracts\DowntimeNotifier;
use App\Contracts\ServerProvider;
use App\Services\DigitalOceanServerProvider;
use App\Services\PingdomDowntimeNotifier;
use App\Services\ServerToolsProvider;
use Illuminate\Support\ServiceProvider;

class AppServiceProvider extends ServiceProvider
{
    /**
     * All of the container bindings that should be registered.
     *
     * @var array
     */
    public $bindings = [
        ServerProvider::class => DigitalOceanServerProvider::class,
    ];

    /**
     * All of the container singletons that should be registered.
     *
     * @var array
     */
    public $singletons = [
        DowntimeNotifier::class => PingdomDowntimeNotifier::class,
        ServerProvider::class => ServerToolsProvider::class,
    ];
}
```

The Boot Method

So, what if we need to register a [view composer](#) within our service provider? This should be done within the `boot` method. **This method is called after all other service providers have been registered**, meaning you have access to all other services that have been registered by the framework:

```
<?php

namespace App\Providers;

use Illuminate\Support\ServiceProvider;

class ComposerServiceProvider extends ServiceProvider
{
    /**
     * Bootstrap any application services.
     *
     * @return void
     */
    public function boot()
```

```
        {
            view()->composer('view', function () {
                //
            });
        }
    }
}
```

Boot Method Dependency Injection

You may type-hint dependencies for your service provider's boot method. The [service container](#) will automatically inject any dependencies you need:

```
use Illuminate\Contracts\Routing\ResponseFactory;

public function boot(ResponseFactory $response)
{
    $response->macro('caps', function ($value) {
        //
    });
}
```

Registering Providers

All service providers are registered in the `config/app.php` configuration file. This file contains a `providers` array where you can list the class names of your service providers. By default, a set of Laravel core service providers are listed in this array. These providers bootstrap the core Laravel components, such as the mailer, queue, cache, and others.

To register your provider, add it to the array:

```
'providers' => [
    // Other Service Providers

    App\Providers\ComposerServiceProvider::class,
],
```

Deferred Providers

If your provider is **only** registering bindings in the [service container](#), you may choose to defer its registration until one of the registered bindings is actually needed. Deferring the loading of such a provider will improve the performance of your application, since it is not loaded from the filesystem on every request.

Laravel compiles and stores a list of all of the services supplied by deferred service providers, along with the name of its service provider class. Then, only when you attempt to resolve one of these services does Laravel load the service provider.

To defer the loading of a provider, implement the `\Illuminate\Contracts\Support\DeferrableProvider` interface and define a `provides` method. The `provides` method should return the service container bindings registered by the provider:

```
<?php

namespace App\Providers;

use Illuminate\Contracts\Support\DeferrableProvider;
use Illuminate\Support\ServiceProvider;
use Riak\Connection;

class RiakServiceProvider extends ServiceProvider implements DeferrableProvider
{
    /**
     * Register any application services.
     *
     * @return void
     */
    public function register()
    {
        $this->app->singleton(Connection::class, function ($app) {
            return new Connection($app['config']['riak']);
        });
    }
}
```

```
        });
    }

    /**
     * Get the services provided by the provider.
     *
     * @return array
     */
    public function provides()
    {
        return [Connection::class];
    }
}
```

Architecture Concepts

Facades

Facades

- [Introduction](#)
- [When To Use Facades](#)
 - [Facades Vs. Dependency Injection](#)
 - [Facades Vs. Helper Functions](#)
- [How Facades Work](#)
- [Real-Time Facades](#)
- [Facade Class Reference](#)

Introduction

Facades provide a "static" interface to classes that are available in the application's [service container](#). Laravel ships with many facades which provide access to almost all of Laravel's features. Laravel facades serve as "static proxies" to underlying classes in the service container, providing the benefit of a terse, expressive syntax while maintaining more testability and flexibility than traditional static methods.

All of Laravel's facades are defined in the `Illuminate\Support\Facades` namespace. So, we can easily access a facade like so:

```
use Illuminate\Support\Facades\Cache;

Route::get('/cache', function () {
    return Cache::get('key');
});
```

Throughout the Laravel documentation, many of the examples will use facades to demonstrate various features of the framework.

When To Use Facades

Facades have many benefits. They provide a terse, memorable syntax that allows you to use Laravel's features without remembering long class names that must be injected or configured manually. Furthermore, because of their unique usage of PHP's dynamic methods, they are easy to test.

However, some care must be taken when using facades. The primary danger of facades is class scope creep. Since facades are so easy to use and do not require injection, it can be easy to let your classes continue to grow and use many facades in a single class. Using dependency injection, this potential is mitigated by the visual feedback a large constructor gives you that your class is growing too large. So, when using facades, pay special attention to the size of your class so that its scope of responsibility stays narrow.

TIP When building a third-party package that interacts with Laravel, it's better to inject [Laravel contracts](#) instead of using facades. Since packages are built outside of Laravel itself, you will not have access to Laravel's facade testing helpers.

Facades Vs. Dependency Injection

One of the primary benefits of dependency injection is the ability to swap implementations of the injected class. This is useful during testing since you can inject a mock or stub and assert that various methods were called on the stub.

Typically, it would not be possible to mock or stub a truly static class method. However, since facades use dynamic methods to proxy method calls to objects resolved from the service container, we actually can test facades just as we would test an injected class instance. For example, given the following route:

```
use Illuminate\Support\Facades\Cache;
```

```
Route::get('/cache', function () {  
    return Cache::get('key');  
});
```

We can write the following test to verify that the `cache::get` method was called with the argument we expected:

```
use Illuminate\Support\Facades\Cache;  
  
/**  
 * A basic functional test example.  
 *  
 * @return void  
 */  
public function testBasicExample()  
{  
    Cache::shouldReceive('get')  
        ->with('key')  
        ->andReturn('value');  
  
    $this->visit('/cache')  
        ->see('value');  
}
```

Facades Vs. Helper Functions

In addition to facades, Laravel includes a variety of "helper" functions which can perform common tasks like generating views, firing events, dispatching jobs, or sending HTTP responses. Many of these helper functions perform the same function as a corresponding facade. For example, this facade call and helper call are equivalent:

```
return View::make('profile');  
  
return view('profile');
```

There is absolutely no practical difference between facades and helper functions. When using helper functions, you may still test them exactly as you would the corresponding facade. For example, given the following route:

```
Route::get('/cache', function () {  
    return cache('key');  
});
```

Under the hood, the `cache` helper is going to call the `get` method on the class underlying the `cache` facade. So, even though we are using the helper function, we can write the following test to verify that the method was called with the argument we expected:

```
use Illuminate\Support\Facades\Cache;  
  
/**  
 * A basic functional test example.  
 *  
 * @return void  
 */  
public function testBasicExample()  
{  
    Cache::shouldReceive('get')  
        ->with('key')  
        ->andReturn('value');  
  
    $this->visit('/cache')  
        ->see('value');  
}
```

How Facades Work

In a Laravel application, a facade is a class that provides access to an object from the container. The machinery that makes this work is in the `Facade` class. Laravel's facades, and any custom facades you create, will extend the base `Illuminate\Support\Facades\Facade` class.

The `Facade` base class makes use of the `__callStatic()` magic-method to defer calls from your facade to an object resolved from the container. In the example below, a call is made to the Laravel cache system. By glancing at this code, one might assume that the static method `get` is being called on the `cache` class:

```
<?php

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;
use Illuminate\Support\Facades\Cache;

class UserController extends Controller
{
    /**
     * Show the profile for the given user.
     *
     * @param int $id
     * @return Response
     */
    public function showProfile($id)
    {
        $user = Cache::get('user:'.$id);

        return view('profile', ['user' => $user]);
    }
}
```

Notice that near the top of the file we are "importing" the cache facade. This facade serves as a proxy to accessing the underlying implementation of the `Illuminate\Contracts\Cache\Factory` interface. Any calls we make using the facade will be passed to the underlying instance of Laravel's cache service.

If we look at that `Illuminate\Support\Facades\Cache` class, you'll see that there is no static method `get`:

```
class Cache extends Facade
{
    /**
     * Get the registered name of the component.
     *
     * @return string
     */
    protected static function getFacadeAccessor() { return 'cache'; }
}
```

Instead, the cache facade extends the base `Facade` class and defines the method `getFacadeAccessor()`. This method's job is to return the name of a service container binding. When a user references any static method on the cache facade, Laravel resolves the cache binding from the [service container](#) and runs the requested method (in this case, `get`) against that object.

Real-Time Facades

Using real-time facades, you may treat any class in your application as if it were a facade. To illustrate how this can be used, let's examine an alternative. For example, let's assume our `Podcast` model has a `publish` method. However, in order to publish the podcast, we need to inject a `Publisher` instance:

```
<?php

namespace App;

use App\Contracts\Publisher;
use Illuminate\Database\Eloquent\Model;

class Podcast extends Model
{
    /**
     * Publish the podcast.
     *
     * @param Publisher $publisher
     * @return void
     */
    public function publish(Publisher $publisher)
    {
        $this->update(['publishing' => now()]);

        $publisher->publish($this);
    }
}
```

Injecting a publisher implementation into the method allows us to easily test the method in isolation since we

can mock the injected publisher. However, it requires us to always pass a publisher instance each time we call the `publish` method. Using real-time facades, we can maintain the same testability while not being required to explicitly pass a `Publisher` instance. To generate a real-time facade, prefix the namespace of the imported class with `Facades`:

```
<?php

namespace App;

use Facades\App\Contracts\Publisher;
use Illuminate\Database\Eloquent\Model;

class Podcast extends Model
{
    /**
     * Publish the podcast.
     *
     * @return void
     */
    public function publish()
    {
        $this->update(['publishing' => now()]);

        Publisher::publish($this);
    }
}
```

When the real-time facade is used, the publisher implementation will be resolved out of the service container using the portion of the interface or class name that appears after the `Facades` prefix. When testing, we can use Laravel's built-in facade testing helpers to mock this method call:

```
<?php

namespace Tests\Feature;

use App\Podcast;
use Facades\App\Contracts\Publisher;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Tests\TestCase;

class PodcastTest extends TestCase
{
    use RefreshDatabase;

    /**
     * A test example.
     *
     * @return void
     */
    public function test_podcast_can_be_published()
    {
        $podcast = factory(Podcast::class)->create();

        Publisher::shouldReceive('publish')->once()->with($podcast);

        $podcast->publish();
    }
}
```

Facade Class Reference

Below you will find every facade and its underlying class. This is a useful tool for quickly digging into the API documentation for a given facade root. The [service container binding](#) key is also included where applicable.

Facade	Class	Service Container Binding
App	Illuminate\Foundation\Application	app
Artisan	Illuminate\Contracts\Console\Kernel	artisan
Auth	Illuminate\Auth\AuthManager	auth
Auth (Instance)	Illuminate\Contracts\Auth\Guard	auth.driver
Blade	Illuminate\View\Compilers\BladeCompiler	blade.compiler
Broadcast	Illuminate\Contracts\Broadcasting\Factory	

Broadcast (Instance)	Illuminate\Contracts/Broadcasting/Broadcaster	
Bus	Illuminate\Contracts/Bus/Dispatcher	
Cache	Illuminate\Cache/CacheManager	cache
Cache (Instance)	Illuminate\Cache/Repository	cache.store
Config	Illuminate\Config/Repository	config
Cookie	Illuminate/Cookie/CookieJar	cookie
Crypt	Illuminate/Encryption/Encrypter	encrypter
DB	Illuminate/Database/DatabaseManager	db
DB (Instance)	Illuminate/Database/Connection	db.connection
Event	Illuminate/Events/Dispatcher	events
File	Illuminate/Filesystem/Filesystem	files
Gate	Illuminate/Contracts/Auth/Access/Gate	
Hash	Illuminate/Contracts/Hashing/Hasher	hash
Http	Illuminate/Http/Client/Factory	
Lang	Illuminate/Translation/Translator	translator
Log	Illuminate/Log/LogManager	log
Mail	Illuminate/Mail/Mailer	mailer
Notification	Illuminate/Notifications/ChannelManager	
Password	Illuminate/Auth/Passwords/PasswordBrokerManager	auth.password
Password (Instance)	Illuminate/Auth/Passwords/PasswordBroker	auth.password.broker
Queue	Illuminate/Queue/QueueManager	queue
Queue (Instance)	Illuminate/Contracts/Queue/Queue	queue.connection
Queue (Base Class)	Illuminate/Queue/Queue	
Redirect	Illuminate/Routing/Redirector	redirect
Redis	Illuminate/Redis/RedisManager	redis
Redis (Instance)	Illuminate/Redis/Connections/Connection	redis.connection
Request	Illuminate/Http/Request	request
Response	Illuminate/Contracts/Routing/ResponseFactory	
Response (Instance)	Illuminate/Http/Response	
Route	Illuminate/Routing/Router	router
Schema	Illuminate/Database/Schema/Builder	
Session	Illuminate/Session/SessionManager	session
Session (Instance)	Illuminate/Session/Store	session.store
Storage	Illuminate/Filesystem/FilesystemManager	filesystem
Storage (Instance)	Illuminate/Contracts/Filesystem/Filesystem	filesystem.disk
URL	Illuminate/Routing/UrlGenerator	url
Validator	Illuminate/Validation/Factory	validator
Validator (Instance)	Illuminate/Validation/Validator	
View	Illuminate/View/Factory	view
View (Instance)	Illuminate/View/View	

Architecture Concepts

Contracts

Contracts

- [Introduction](#)
 - [Contracts Vs. Facades](#)
- [When To Use Contracts](#)
 - [Loose Coupling](#)
 - [Simplicity](#)
- [How To Use Contracts](#)
- [Contract Reference](#)

Introduction

Laravel's Contracts are a set of interfaces that define the core services provided by the framework. For example, a `Illuminate\Contracts\Queue\Queue` contract defines the methods needed for queueing jobs, while the `Illuminate\Contracts\Mail\Mailer` contract defines the methods needed for sending e-mail.

Each contract has a corresponding implementation provided by the framework. For example, Laravel provides a queue implementation with a variety of drivers, and a mailer implementation that is powered by [SwiftMailer](#).

All of the Laravel contracts live in [their own GitHub repository](#). This provides a quick reference point for all available contracts, as well as a single, decoupled package that may be utilized by package developers.

Contracts Vs. Facades

Laravel's [facades](#) and helper functions provide a simple way of utilizing Laravel's services without needing to type-hint and resolve contracts out of the service container. In most cases, each facade has an equivalent contract.

Unlike facades, which do not require you to require them in your class' constructor, contracts allow you to define explicit dependencies for your classes. Some developers prefer to explicitly define their dependencies in this way and therefore prefer to use contracts, while other developers enjoy the convenience of facades.

TIP Most applications will be fine regardless of whether you prefer facades or contracts. However, if you are building a package, you should strongly consider using contracts since they will be easier to test in a package context.

When To Use Contracts

As discussed elsewhere, much of the decision to use contracts or facades will come down to personal taste and the tastes of your development team. Both contracts and facades can be used to create robust, well-tested Laravel applications. As long as you are keeping your class' responsibilities focused, you will notice very few practical differences between using contracts and facades.

However, you may still have several questions regarding contracts. For example, why use interfaces at all? Isn't using interfaces more complicated? Let's distill the reasons for using interfaces to the following headings: loose coupling and simplicity.

Loose Coupling

First, let's review some code that is tightly coupled to a cache implementation. Consider the following:

```
<?php
namespace App\Orders;

class Repository
```

```
{
    /**
     * The cache instance.
     */
    protected $cache;

    /**
     * Create a new repository instance.
     *
     * @param \SomePackage\Cache\Memcached $cache
     * @return void
     */
    public function __construct(\SomePackage\Cache\Memcached $cache)
    {
        $this->cache = $cache;
    }

    /**
     * Retrieve an Order by ID.
     *
     * @param int $id
     * @return Order
     */
    public function find($id)
    {
        if ($this->cache->has($id)) {
            //
        }
    }
}
```

In this class, the code is tightly coupled to a given cache implementation. It is tightly coupled because we are depending on a concrete Cache class from a package vendor. If the API of that package changes our code must change as well.

Likewise, if we want to replace our underlying cache technology (Memcached) with another technology (Redis), we again will have to modify our repository. Our repository should not have so much knowledge regarding who is providing them data or how they are providing it.

Instead of this approach, we can improve our code by depending on a simple, vendor agnostic interface:

```
<?php

namespace App\Orders;

use Illuminate\Contracts\Cache\Repository as Cache;

class Repository
{
    /**
     * The cache instance.
     */
    protected $cache;

    /**
     * Create a new repository instance.
     *
     * @param Cache $cache
     * @return void
     */
    public function __construct(Cache $cache)
    {
        $this->cache = $cache;
    }
}
```

Now the code is not coupled to any specific vendor, or even Laravel. Since the contracts package contains no implementation and no dependencies, you may easily write an alternative implementation of any given contract, allowing you to replace your cache implementation without modifying any of your cache consuming code.

Simplicity

When all of Laravel's services are neatly defined within simple interfaces, it is very easy to determine the

functionality offered by a given service. **The contracts serve as succinct documentation to the framework's features.**

In addition, when you depend on simple interfaces, your code is easier to understand and maintain. Rather than tracking down which methods are available to you within a large, complicated class, you can refer to a simple, clean interface.

How To Use Contracts

So, how do you get an implementation of a contract? It's actually quite simple.

Many types of classes in Laravel are resolved through the [service container](#), including controllers, event listeners, middleware, queued jobs, and even route Closures. So, to get an implementation of a contract, you can just "type-hint" the interface in the constructor of the class being resolved.

For example, take a look at this event listener:

```
<?php

namespace App\Listeners;

use App\Events\OrderWasPlaced;
use App\User;
use Illuminate\Contracts\Redis\Factory;

class CacheOrderInformation
{
    /**
     * The Redis factory implementation.
     */
    protected $redis;

    /**
     * Create a new event handler instance.
     *
     * @param Factory $redis
     * @return void
     */
    public function __construct(Factory $redis)
    {
        $this->redis = $redis;
    }

    /**
     * Handle the event.
     *
     * @param OrderWasPlaced $event
     * @return void
     */
    public function handle(OrderWasPlaced $event)
    {
        //
    }
}
```

When the event listener is resolved, the service container will read the type-hints on the constructor of the class, and inject the appropriate value. To learn more about registering things in the service container, check out [its documentation](#).

Contract Reference

This table provides a quick reference to all of the Laravel contracts and their equivalent facades:

Contract	References Facade
Illuminate\Contracts\Auth\Access\Authorizable	Gate
Illuminate\Contracts\Auth\Access\Gate	
Illuminate\Contracts\Auth\Authenticatable	
Illuminate\Contracts\Auth\CanResetPassword	

Illuminate\Contracts\Auth\Factory	Auth
Illuminate\Contracts\Auth/Guard	Auth::guard()
Illuminate\Contracts\Auth/PasswordBroker	Password::broker()
Illuminate\Contracts/Auth/PasswordBrokerFactory	Password
Illuminate\Contracts/Auth/StatefulGuard	
Illuminate\Contracts/Auth/SupportsBasicAuth	
Illuminate\Contracts/Auth/UserProvider	
Illuminate\Contracts/Bus/Dispatcher	Bus
Illuminate\Contracts/Bus/QueueingDispatcher	Bus::dispatchToQueue()
Illuminate\Contracts/Broadcasting/Factory	Broadcast
Illuminate\Contracts/Broadcasting/Broadcaster	Broadcast::connection()
Illuminate\Contracts/Broadcasting/ShouldBroadcast	
Illuminate\Contracts/Broadcasting/ShouldBroadcastNow	
Illuminate\Contracts/Cache/Factory	Cache
Illuminate\Contracts/Cache/Lock	
Illuminate\Contracts/Cache/LockProvider	
Illuminate\Contracts/Cache/Repository	Cache::driver()
Illuminate\Contracts/Cache/Store	
Illuminate\Contracts/Config/Repository	Config
Illuminate\Contracts/Console/Application	
Illuminate\Contracts/Console/Kernel	Artisan
Illuminate\Contracts/Container/Container	App
Illuminate\Contracts/Cookie/Factory	Cookie
Illuminate\Contracts/Cookie/QueueingFactory	Cookie::queue()
Illuminate\Contracts/Database/ModelIdentifier	
Illuminate\Contracts/Debug/ExceptionHandler	
Illuminate\Contracts/Encryption/Encrypter	Crypt
Illuminate\Contracts/Events/Dispatcher	Event
Illuminate\Contracts/Filesystem/Cloud	Storage::cloud()
Illuminate\Contracts/Filesystem/Factory	Storage
Illuminate\Contracts/Filesystem/Filesystem	Storage::disk()
Illuminate\Contracts/Foundation/Application	App
Illuminate\Contracts/Hashing/Hasher	Hash
Illuminate\Contracts/Http/Kernel	
Illuminate\Contracts/Mail/MailQueue	Mail::queue()
Illuminate\Contracts/Mail/Mailable	
Illuminate\Contracts/Mail/Mailer	Mail
Illuminate\Contracts/Notifications/Dispatcher	Notification
Illuminate\Contracts/Notifications/Factory	Notification
Illuminate\Contracts/Pagination/LengthAwarePaginator	
Illuminate\Contracts/Pagination/Paginator	
Illuminate\Contracts/Pipeline/Hub	
Illuminate\Contracts/Pipeline/Pipeline	
Illuminate\Contracts/Queue/EntityResolver	
Illuminate\Contracts/Queue/Factory	Queue
Illuminate\Contracts/Queue/Job	
Illuminate\Contracts/Queue/Monitor	Queue
Illuminate\Contracts/Queue/Queue	Queue::connection()
Illuminate\Contracts/Queue/QueueableCollection	
Illuminate\Contracts/Queue/QueueableEntity	
Illuminate\Contracts/Queue/ShouldQueue	
Illuminate\Contracts/Redis/Factory	Redis
Illuminate\Contracts/Routing/BindingRegistrar	Route
Illuminate\Contracts/Routing/Registrar	Route

<u>Illuminate\Contracts\Routing\ResponseFactory</u>	Response
<u>Illuminate\Contracts\Routing\UrlGenerator</u>	URL
<u>Illuminate\Contracts\Routing\UrlRoutable</u>	
<u>Illuminate\Contracts\Session\Session</u>	Session::driver()
<u>Illuminate\Contracts\Support\Arrayable</u>	
<u>Illuminate\Contracts\Support\Htmlable</u>	
<u>Illuminate\Contracts\Support\Jsonable</u>	
<u>Illuminate\Contracts\Support\MessageBag</u>	
<u>Illuminate\Contracts\Support\MessageProvider</u>	
<u>Illuminate\Contracts\Support\Renderable</u>	
<u>Illuminate\Contracts\Support\Responsable</u>	
<u>Illuminate\Contracts\Translation\Loader</u>	
<u>Illuminate\Contracts\Translation\Translator</u>	Lang
<u>Illuminate\Contracts\Validation\Factory</u>	Validator
<u>Illuminate\Contracts\Validation\ImplicitRule</u>	
<u>Illuminate\Contracts\Validation\Rule</u>	
<u>Illuminate\Contracts\Validation\ValidatesWhenResolved</u>	
<u>Illuminate\Contracts\Validation\Validator</u>	Validator::make()
<u>Illuminate\Contracts\View\Engine</u>	
<u>Illuminate\Contracts\View\Factory</u>	View
<u>Illuminate\Contracts\View\View</u>	View::make()

The Basics

Routing

Routing

- [Basic Routing](#)
 - [Redirect Routes](#)
 - [View Routes](#)
- [Route Parameters](#)
 - [Required Parameters](#)
 - [Optional Parameters](#)
 - [Regular Expression Constraints](#)
- [Named Routes](#)
- [Route Groups](#)
 - [Middleware](#)
 - [Namespaces](#)
 - [Subdomain Routing](#)
 - [Route Prefixes](#)
 - [Route Name Prefixes](#)
- [Route Model Binding](#)
 - [Implicit Binding](#)
 - [Explicit Binding](#)
- [Fallback Routes](#)
- [Rate Limiting](#)
- [Form Method Spoofing](#)
- [Accessing The Current Route](#)
- [Cross-Origin Resource Sharing \(CORS\)](#)

Basic Routing

The most basic Laravel routes accept a URI and a closure, providing a very simple and expressive method of defining routes:

```
Route::get('foo', function () {  
    return 'Hello World';  
});
```

The Default Route Files

All Laravel routes are defined in your route files, which are located in the `routes` directory. These files are automatically loaded by the framework. The `routes/web.php` file defines routes that are for your web interface. These routes are assigned the `web` middleware group, which provides features like session state and CSRF protection. The routes in `routes/api.php` are stateless and are assigned the `api` middleware group.

For most applications, you will begin by defining routes in your `routes/web.php` file. The routes defined in `routes/web.php` may be accessed by entering the defined route's URL in your browser. For example, you may access the following route by navigating to `http://your-app.test/user` in your browser:

```
Route::get('/user', 'UserController@index');
```

Routes defined in the `routes/api.php` file are nested within a route group by the `RouteServiceProvider`. Within this group, the `/api` URI prefix is automatically applied so you do not need to manually apply it to every route in the file. You may modify the prefix and other route group options by modifying your `RouteServiceProvider` class.

Available Router Methods

The router allows you to register routes that respond to any HTTP verb:


```
Route::get($uri, $callback);
Route::post($uri, $callback);
Route::put($uri, $callback);
Route::patch($uri, $callback);
Route::delete($uri, $callback);
Route::options($uri, $callback);
```

Sometimes you may need to register a route that responds to multiple HTTP verbs. You may do so using the `match` method. Or, you may even register a route that responds to all HTTP verbs using the `any` method:

```
Route::match(['get', 'post'], '/', function () {
    //
});

Route::any('/', function () {
    //
});
```

CSRF Protection

Any HTML forms pointing to `POST`, `PUT`, `PATCH`, or `DELETE` routes that are defined in the web routes file should include a CSRF token field. Otherwise, the request will be rejected. You can read more about CSRF protection in the [CSRF documentation](#):

```
<form method="POST" action="/profile">
    @csrf
    ...
</form>
```

Redirect Routes

If you are defining a route that redirects to another URI, you may use the `Route::redirect` method. This method provides a convenient shortcut so that you do not have to define a full route or controller for performing a simple redirect:

```
Route::redirect('/here', '/there');
```

By default, `Route::redirect` returns a `302` status code. You may customize the status code using the optional third parameter:

```
Route::redirect('/here', '/there', 301);
```

You may use the `Route::permanentRedirect` method to return a `301` status code:

```
Route::permanentRedirect('/here', '/there');
```

View Routes

If your route only needs to return a view, you may use the `Route::view` method. Like the `redirect` method, this method provides a simple shortcut so that you do not have to define a full route or controller. The `view` method accepts a URI as its first argument and a view name as its second argument. In addition, you may provide an array of data to pass to the view as an optional third argument:

```
Route::view('/welcome', 'welcome');

Route::view('/welcome', 'welcome', ['name' => 'Taylor']);
```

Route Parameters

Required Parameters

Sometimes you will need to capture segments of the URI within your route. For example, you may need to capture a user's ID from the URL. You may do so by defining route parameters:

```
Route::get('user/{id}', function ($id) {
    return 'User '.$id;
});
```

```
});
```

You may define as many route parameters as required by your route:

```
Route::get('posts/{post}/comments/{comment}', function ($postId, $commentId) {  
    //  
});
```

Route parameters are always encased within {} braces and should consist of alphabetic characters, and may not contain a - character. Instead of using the - character, use an underscore (_). Route parameters are injected into route callbacks / controllers based on their order - the names of the callback / controller arguments do not matter.

Optional Parameters

Occasionally you may need to specify a route parameter, but make the presence of that route parameter optional. You may do so by placing a ? mark after the parameter name. Make sure to give the route's corresponding variable a default value:

```
Route::get('user/{name?}', function ($name = null) {  
    return $name;  
});  
  
Route::get('user/{name?}', function ($name = 'John') {  
    return $name;  
});
```

Regular Expression Constraints

You may constrain the format of your route parameters using the `where` method on a route instance. The `where` method accepts the name of the parameter and a regular expression defining how the parameter should be constrained:

```
Route::get('user/{name}', function ($name) {  
    //  
})->where('name', '[A-Za-z]+');  
  
Route::get('user/{id}', function ($id) {  
    //  
})->where('id', '[0-9]+');  
  
Route::get('user/{id}/{name}', function ($id, $name) {  
    //  
})->where(['id' => '[0-9]+', 'name' => '[a-z]+']);
```

Global Constraints

If you would like a route parameter to always be constrained by a given regular expression, you may use the `pattern` method. You should define these patterns in the `boot` method of your `RouteServiceProvider`:

```
/**  
 * Define your route model bindings, pattern filters, etc.  
 *  
 * @return void  
 */  
public function boot()  
{  
    Route::pattern('id', '[0-9]+');  
  
    parent::boot();  
}
```

Once the pattern has been defined, it is automatically applied to all routes using that parameter name:

```
Route::get('user/{id}', function ($id) {  
    // Only executed if {id} is numeric...  
});
```

Encoded Forward Slashes

The Laravel routing component allows all characters except /. You must explicitly allow / to be part of your placeholder using a where condition regular expression:

```
Route::get('search/{search}', function ($search) {
    return $search;
})->where('search', '.*');
```

NOTE Encoded forward slashes are only supported within the last route segment.

Named Routes

Named routes allow the convenient generation of URLs or redirects for specific routes. You may specify a name for a route by chaining the `name` method onto the route definition:

```
Route::get('user/profile', function () {
    //
})->name('profile');
```

You may also specify route names for controller actions:

```
Route::get('user/profile', 'UserProfileController@show')->name('profile');
```

NOTE Route names should always be unique.

Generating URLs To Named Routes

Once you have assigned a name to a given route, you may use the route's name when generating URLs or redirects via the global `route` function:

```
// Generating URLs...
$url = route('profile');

// Generating Redirects...
return redirect()->route('profile');
```

If the named route defines parameters, you may pass the parameters as the second argument to the `route` function. The given parameters will automatically be inserted into the URL in their correct positions:

```
Route::get('user/{id}/profile', function ($id) {
    //
})->name('profile');

$url = route('profile', ['id' => 1]);
```

If you pass additional parameters in the array, those key / value pairs will automatically be added to the generated URL's query string:

```
Route::get('user/{id}/profile', function ($id) {
    //
})->name('profile');

$url = route('profile', ['id' => 1, 'photos' => 'yes']);

// /user/1/profile?photos=yes
```

TIP Sometimes, you may wish to specify request-wide default values for URL parameters, such as the current locale. To accomplish this, you may use the [URL::defaults method](#).

Inspecting The Current Route

If you would like to determine if the current request was routed to a given named route, you may use the `named` method on a Route instance. For example, you may check the current route name from a route middleware:

```
/**
 * Handle an incoming request.
 *
 * @param  \Illuminate\Http\Request  $request
 * @param  \Closure  $next
 * @return mixed
 */
```

```
*/
public function handle($request, Closure $next)
{
    if ($request->route()->named('profile')) {
        //
    }

    return $next($request);
}
```

Route Groups

Route groups allow you to share route attributes, such as middleware or namespaces, across a large number of routes without needing to define those attributes on each individual route. Shared attributes are specified in an array format as the first parameter to the `Route::group` method.

Nested groups attempt to intelligently "merge" attributes with their parent group. Middleware and `where` conditions are merged while names, namespaces, and prefixes are appended. Namespace delimiters and slashes in URI prefixes are automatically added where appropriate.

Middleware

To assign middleware to all routes within a group, you may use the `middleware` method before defining the group. Middleware are executed in the order they are listed in the array:

```
Route::middleware(['first', 'second'])->group(function () {
    Route::get('/', function () {
        // Uses first & second Middleware
    });

    Route::get('user/profile', function () {
        // Uses first & second Middleware
    });
});
```

Namespaces

Another common use-case for route groups is assigning the same PHP namespace to a group of controllers using the `namespace` method:

```
Route::namespace('Admin')->group(function () {
    // Controllers Within The "App\Http\Controllers\Admin" Namespace
});
```

Remember, by default, the `RouteServiceProvider` includes your route files within a namespace group, allowing you to register controller routes without specifying the full `App\Http\Controllers` namespace prefix. So, you only need to specify the portion of the namespace that comes after the base `App\Http\Controllers` namespace.

Subdomain Routing

Route groups may also be used to handle subdomain routing. Subdomains may be assigned route parameters just like route URIs, allowing you to capture a portion of the subdomain for usage in your route or controller. The subdomain may be specified by calling the `domain` method before defining the group:

```
Route::domain('{account}.myapp.com')->group(function () {
    Route::get('user/{id}', function ($account, $id) {
        //
    });
});
```

NOTE In order to ensure your subdomain routes are reachable, you should register subdomain routes before registering root domain routes. This will prevent root domain routes from overwriting subdomain routes which have the same URI path.

Route Prefixes

The `prefix` method may be used to prefix each route in the group with a given URI. For example, you may want to prefix all route URIs within the group with `admin`:

```
Route::prefix('admin')->group(function () {
    Route::get('users', function () {
        // Matches The "/admin/users" URL
    });
});
```

Route Name Prefixes

The `name` method may be used to prefix each route name in the group with a given string. For example, you may want to prefix all of the grouped route's names with `admin`. The given string is prefixed to the route name exactly as it is specified, so we will be sure to provide the trailing `.` character in the prefix:

```
Route::name('admin.')->group(function () {
    Route::get('users', function () {
        // Route assigned name "admin.users"...
    })->name('users');
});
```

Route Model Binding

When injecting a model ID to a route or controller action, you will often query to retrieve the model that corresponds to that ID. Laravel route model binding provides a convenient way to automatically inject the model instances directly into your routes. For example, instead of injecting a user's ID, you can inject the entire user model instance that matches the given ID.

Implicit Binding

Laravel automatically resolves Eloquent models defined in routes or controller actions whose type-hinted variable names match a route segment name. For example:

```
Route::get('api/users/{user}', function (App\User $user) {
    return $user->email;
});
```

Since the `$user` variable is type-hinted as the `App\User` Eloquent model and the variable name matches the `{user}` URI segment, Laravel will automatically inject the model instance that has an ID matching the corresponding value from the request URI. If a matching model instance is not found in the database, a 404 HTTP response will automatically be generated.

Customizing The Key

Sometimes you may wish to resolve Eloquent models using a column other than `id`. To do so, you may specify the column in the route parameter definition:

```
Route::get('api/posts/{post:slug}', function (App\Post $post) {
    return $post;
});
```

Custom Keys & Scoping

Sometimes, when implicitly binding multiple Eloquent models in a single route definition, you may wish to scope the second Eloquent model such that it must be a child of the first Eloquent model. For example, consider this situation that retrieves a blog post by slug for a specific user:

```
use App\Post;
use App\User;

Route::get('api/users/{user}/posts/{post:slug}', function (User $user, Post $post) {
    return $post;
});
```

When using a custom keyed implicit binding as a nested route parameter, Laravel will automatically scope the

query to retrieve the nested model by its parent using conventions to guess the relationship name on the parent. In this case, it will be assumed that the `user` model has a relationship named `posts` (the plural of the route parameter name) which can be used to retrieve the `Post` model.

Customizing The Default Key Name

If you would like model binding to use a default database column other than `id` when retrieving a given model class, you may override the `getRouteKeyName` method on the Eloquent model:

```
/**
 * Get the route key for the model.
 *
 * @return string
 */
public function getRouteKeyName()
{
    return 'slug';
}
```

Explicit Binding

To register an explicit binding, use the router's `model` method to specify the class for a given parameter. You should define your explicit model bindings in the `boot` method of the `RouteServiceProvider` class:

```
public function boot()
{
    parent::boot();

    Route::model('user', App\User::class);
}
```

Next, define a route that contains a `{user}` parameter:

```
Route::get('profile/{user}', function (App\User $user) {
    //
});
```

Since we have bound all `{user}` parameters to the `App\User` model, a `User` instance will be injected into the route. So, for example, a request to `profile/1` will inject the `User` instance from the database which has an ID of 1.

If a matching model instance is not found in the database, a 404 HTTP response will be automatically generated.

Customizing The Resolution Logic

If you wish to use your own resolution logic, you may use the `Route::bind` method. The closure you pass to the `bind` method will receive the value of the URI segment and should return the instance of the class that should be injected into the route:

```
/**
 * Bootstrap any application services.
 *
 * @return void
 */
public function boot()
{
    parent::boot();

    Route::bind('user', function ($value) {
        return App\User::where('name', $value)->firstOrFail();
    });
}
```

Alternatively, you may override the `resolveRouteBinding` method on your Eloquent model. This method will receive the value of the URI segment and should return the instance of the class that should be injected into the route:

```
/**
```

```
* Retrieve the model for a bound value.
*
* @param mixed $value
* @param string|null $field
* @return \Illuminate\Database\Eloquent\Model|null
*/
public function resolveRouteBinding($value, $field = null)
{
    return $this->where('name', $value)->firstOrFail();
}
```

Fallback Routes

Using the `Route::fallback` method, you may define a route that will be executed when no other route matches the incoming request. Typically, unhandled requests will automatically render a "404" page via your application's exception handler. However, since you may define the `fallback` route within your `routes/web.php` file, all middleware in the `web` middleware group will apply to the route. You are free to add additional middleware to this route as needed:

```
Route::fallback(function () {
    //
});
```

NOTE The fallback route should always be the last route registered by your application.

Rate Limiting

Laravel includes a [middleware](#) to rate limit access to routes within your application. To get started, assign the `throttle` middleware to a route or a group of routes. The `throttle` middleware accepts two parameters that determine the maximum number of requests that can be made in a given number of minutes. For example, let's specify that an authenticated user may access the following group of routes 60 times per minute:

```
Route::middleware('auth:api', 'throttle:60,1')->group(function () {
    Route::get('/user', function () {
        //
    });
});
```

Dynamic Rate Limiting

You may specify a dynamic request maximum based on an attribute of the authenticated user model. For example, if your user model contains a `rate_limit` attribute, you may pass the name of the attribute to the `throttle` middleware so that it is used to calculate the maximum request count:

```
Route::middleware('auth:api', 'throttle:rate_limit,1')->group(function () {
    Route::get('/user', function () {
        //
    });
});
```

Distinct Guest & Authenticated User Rate Limits

You may specify different rate limits for guest and authenticated users. For example, you may specify a maximum of 10 requests per minute for guests 60 for authenticated users:

```
Route::middleware('throttle:10|60,1')->group(function () {
    //
});
```

You may also combine this functionality with dynamic rate limits. For example, if your user model contains a `rate_limit` attribute, you may pass the name of the attribute to the `throttle` middleware so that it is used to calculate the maximum request count for authenticated users:

```
Route::middleware('auth:api', 'throttle:10|rate_limit,1')->group(function () {
    Route::get('/user', function () {
        //
    });
});
```

```
});
```

Rate Limit Segments

Typically, you will probably specify one rate limit for your entire API. However, your application may require different rate limits for different segments of your API. If this is the case, you will need to pass a segment name as the third argument to the `throttle` middleware:

```
Route::middleware('auth:api')->group(function () {
    Route::middleware('throttle:60,1,default')->group(function () {
        Route::get('/servers', function () {
            //
        });
    });

    Route::middleware('throttle:60,1,deletes')->group(function () {
        Route::delete('/servers/{id}', function () {
            //
        });
    });
});
```

Form Method Spoofing

HTML forms do not support PUT, PATCH OR DELETE actions. So, when defining PUT, PATCH OR DELETE routes that are called from an HTML form, you will need to add a hidden `_method` field to the form. The value sent with the `_method` field will be used as the HTTP request method:

```
<form action="/foo/bar" method="POST">
    <input type="hidden" name="_method" value="PUT">
    <input type="hidden" name="_token" value="{{ csrf_token() }}">
</form>
```

You may use the `@method` Blade directive to generate the `_method` input:

```
<form action="/foo/bar" method="POST">
    @method('PUT')
    @csrf
</form>
```

Accessing The Current Route

You may use the `current`, `currentRouteName`, and `currentRouteAction` methods on the `Route` facade to access information about the route handling the incoming request:

```
$route = Route::current();

$name = Route::currentRouteName();

$action = Route::currentRouteAction();
```

Refer to the API documentation for both the [underlying class of the Route facade](#) and [Route instance](#) to review all accessible methods.

Cross-Origin Resource Sharing (CORS)

Laravel can automatically respond to CORS OPTIONS requests with values that you configure. All CORS settings may be configured in your `cors` configuration file and OPTIONS requests will automatically be handled by the `HandleCors` middleware that is included by default in your global middleware stack.

TIP For more information on CORS and CORS headers, please consult the [MDN web documentation on CORS](#).

The Basics

Middleware

Middleware

- [Introduction](#)
- [Defining Middleware](#)
- [Registering Middleware](#)
 - [Global Middleware](#)
 - [Assigning Middleware To Routes](#)
 - [Middleware Groups](#)
 - [Sorting Middleware](#)
- [Middleware Parameters](#)
- [Terminable Middleware](#)

Introduction

Middleware provide a convenient mechanism for filtering HTTP requests entering your application. For example, Laravel includes a middleware that verifies the user of your application is authenticated. If the user is not authenticated, the middleware will redirect the user to the login screen. However, if the user is authenticated, the middleware will allow the request to proceed further into the application.

Additional middleware can be written to perform a variety of tasks besides authentication. A CORS middleware might be responsible for adding the proper headers to all responses leaving your application. A logging middleware might log all incoming requests to your application.

There are several middleware included in the Laravel framework, including middleware for authentication and CSRF protection. All of these middleware are located in the `app/Http/Middleware` directory.

Defining Middleware

To create a new middleware, use the `make:middleware` Artisan command:

```
php artisan make:middleware CheckAge
```

This command will place a new `CheckAge` class within your `app/Http/Middleware` directory. In this middleware, we will only allow access to the route if the supplied age is greater than 200. Otherwise, we will redirect the users back to the home URI:

```
<?php

namespace App\Http\Middleware;

use Closure;

class CheckAge
{
    /**
     * Handle an incoming request.
     *
     * @param \Illuminate\Http\Request $request
     * @param \Closure $next
     * @return mixed
     */
    public function handle($request, Closure $next)
    {
        if ($request->age <= 200) {
            return redirect('home');
        }

        return $next($request);
    }
}
```

As you can see, if the given `age` is less than or equal to `200`, the middleware will return an HTTP redirect to the client; otherwise, the request will be passed further into the application. To pass the request deeper into the application (allowing the middleware to "pass"), call the `$next` callback with the `$request`.

It's best to envision middleware as a series of "layers" HTTP requests must pass through before they hit your application. Each layer can examine the request and even reject it entirely.

TIP All middleware are resolved via the [service container](#), so you may type-hint any dependencies you need within a middleware's constructor.

Before & After Middleware

Whether a middleware runs before or after a request depends on the middleware itself. For example, the following middleware would perform some task **before** the request is handled by the application:

```
<?php

namespace App\Http\Middleware;

use Closure;

class BeforeMiddleware
{
    public function handle($request, Closure $next)
    {
        // Perform action

        return $next($request);
    }
}
```

However, this middleware would perform its task **after** the request is handled by the application:

```
<?php

namespace App\Http\Middleware;

use Closure;

class AfterMiddleware
{
    public function handle($request, Closure $next)
    {
        $response = $next($request);

        // Perform action

        return $response;
    }
}
```

Registering Middleware

Global Middleware

If you want a middleware to run during every HTTP request to your application, list the middleware class in the `$middleware` property of your `app/Http/Kernel.php` class.

Assigning Middleware To Routes

If you would like to assign middleware to specific routes, you should first assign the middleware a key in your `app/Http/Kernel.php` file. By default, the `$routeMiddleware` property of this class contains entries for the middleware included with Laravel. To add your own, append it to this list and assign it a key of your choosing:

```
// Within App\Http\Kernel Class...

protected $routeMiddleware = [
    'auth' => \App\Http\Middleware\Authenticate::class,
    'auth.basic' => \Illuminate\Auth\Middleware\AuthenticateWithBasicAuth::class,
```

```
'bindings' => \Illuminate\Routing\Middleware\SubstituteBindings::class,
'cache.headers' => \Illuminate\Http\Middleware\SetCacheHeaders::class,
'can' => \Illuminate\Auth\Middleware\Authorize::class,
'guest' => \App\Http\Middleware\RedirectIfAuthenticated::class,
'signed' => \Illuminate\Routing\Middleware\ValidateSignature::class,
'throttle' => \Illuminate\Routing\Middleware\ThrottleRequests::class,
'verified' => \Illuminate\Auth\Middleware\EnsureEmailIsVerified::class,
];
```

Once the middleware has been defined in the HTTP kernel, you may use the `middleware` method to assign middleware to a route:

```
Route::get('admin/profile', function () {
    //
})->middleware('auth');
```

You may also assign multiple middleware to the route:

```
Route::get('/', function () {
    //
})->middleware('first', 'second');
```

When assigning middleware, you may also pass the fully qualified class name:

```
use App\Http\Middleware\CheckAge;

Route::get('admin/profile', function () {
    //
})->middleware(CheckAge::class);
```

When assigning middleware to a group of routes, you may occasionally need to prevent the middleware from being applied to an individual route within the group. You may accomplish this using the `withoutMiddleware` method:

```
use App\Http\Middleware\CheckAge;

Route::middleware([CheckAge::class])>group(function () {
    Route::get('/', function () {
        //
    });

    Route::get('admin/profile', function () {
        //
    })->withoutMiddleware([CheckAge::class]);
});
```

The `withoutMiddleware` method can only remove route middleware and does not apply to [global middleware](#).

Middleware Groups

Sometimes you may want to group several middleware under a single key to make them easier to assign to routes. You may do this using the `$middlewareGroups` property of your HTTP kernel.

Out of the box, Laravel comes with `web` and `api` middleware groups that contain common middleware you may want to apply to your web UI and API routes:

```
/**
 * The application's route middleware groups.
 *
 * @var array
 */
protected $middlewareGroups = [
    'web' => [
        \App\Http\Middleware\EncryptCookies::class,
        \Illuminate\Cookie\Middleware\AddQueuedCookiesToResponse::class,
        \Illuminate\Session\Middleware\StartSession::class,
        \Illuminate\View\Middleware\ShareErrorsFromSession::class,
        \App\Http\Middleware\VerifyCsrfToken::class,
        \Illuminate\Routing\Middleware\SubstituteBindings::class,
    ],

    'api' => [
        'throttle:60,1',
        'auth:api',
    ],
];
```

```
    ],  
];
```

Middleware groups may be assigned to routes and controller actions using the same syntax as individual middleware. Again, middleware groups make it more convenient to assign many middleware to a route at once:

```
Route::get('/', function () {  
    //  
})->middleware('web');  
  
Route::group(['middleware' => ['web']], function () {  
    //  
});  
  
Route::middleware(['web', 'subscribed'])->group(function () {  
    //  
});
```

TIP Out of the box, the `web` middleware group is automatically applied to your `routes/web.php` file by the `RouteServiceProvider`.

Sorting Middleware

Rarely, you may need your middleware to execute in a specific order but not have control over their order when they are assigned to the route. In this case, you may specify your middleware priority using the `$middlewarePriority` property of your `app/Http/Kernel.php` file:

```
/**  
 * The priority-sorted list of middleware.  
 *  
 * This forces non-global middleware to always be in the given order.  
 *  
 * @var array  
 */  
protected $middlewarePriority = [  
    \Illuminate\Session\Middleware\StartSession::class,  
    \Illuminate\View\Middleware\ShareErrorsFromSession::class,  
    \Illuminate\Contracts\Auth\Middleware\AuthenticatesRequests::class,  
    \Illuminate\Routing\Middleware\ThrottleRequests::class,  
    \Illuminate\Session\Middleware\AuthenticateSession::class,  
    \Illuminate\Routing\Middleware\SubstituteBindings::class,  
    \Illuminate\Auth\Middleware\Authorize::class,  
];
```

Middleware Parameters

Middleware can also receive additional parameters. For example, if your application needs to verify that the authenticated user has a given "role" before performing a given action, you could create a `CheckRole` middleware that receives a role name as an additional argument.

Additional middleware parameters will be passed to the middleware after the `$next` argument:

```
<?php  
  
namespace App\Http\Middleware;  
  
use Closure;  
  
class CheckRole  
{  
    /**  
     * Handle the incoming request.  
     *  
     * @param \Illuminate\Http\Request $request  
     * @param \Closure $next  
     * @param string $role  
     * @return mixed  
     */  
    public function handle($request, Closure $next, $role)  
    {  
        if (! $request->user()->hasRole($role)) {  
            // Redirect...  
        }  
    }  
}
```

```
        return $next($request);
    }
}
```

Middleware parameters may be specified when defining the route by separating the middleware name and parameters with a `:`. Multiple parameters should be delimited by commas:

```
Route::put('post/{id}', function ($id) {
    //
})->middleware('role:editor');
```

Terminable Middleware

Sometimes a middleware may need to do some work after the HTTP response has been sent to the browser. If you define a `terminate` method on your middleware and your web server is using FastCGI, the `terminate` method will automatically be called after the response is sent to the browser:

```
<?php

namespace Illuminate\Session\Middleware;

use Closure;

class StartSession
{
    public function handle($request, Closure $next)
    {
        return $next($request);
    }

    public function terminate($request, $response)
    {
        // Store the session data...
    }
}
```

The `terminate` method should receive both the request and the response. Once you have defined a terminable middleware, you should add it to the list of route or global middleware in the `app/Http/Kernel.php` file.

When calling the `terminate` method on your middleware, Laravel will resolve a fresh instance of the middleware from the [service container](#). If you would like to use the same middleware instance when the `handle` and `terminate` methods are called, register the middleware with the container using the container's `singleton` method. Typically this should be done in the `register` method of your `AppServiceProvider.php`:

```
use App\Http\Middleware\TerminableMiddleware;

/**
 * Register any application services.
 *
 * @return void
 */
public function register()
{
    $this->app->singleton(TerminableMiddleware::class);
}
```

The Basics

CSRF Protection

CSRF Protection

- [Introduction](#)
- [Excluding URIs](#)
- [X-CSRF-Token](#)
- [X-XSRF-Token](#)

Introduction

Laravel makes it easy to protect your application from [cross-site request forgery](#) (CSRF) attacks. Cross-site request forgeries are a type of malicious exploit whereby unauthorized commands are performed on behalf of an authenticated user.

Laravel automatically generates a CSRF "token" for each active user session managed by the application. This token is used to verify that the authenticated user is the one actually making the requests to the application.

Anytime you define an HTML form in your application, you should include a hidden CSRF token field in the form so that the CSRF protection middleware can validate the request. You may use the `@csrf` Blade directive to generate the token field:

```
<form method="POST" action="/profile">
    @csrf
    ...
</form>
```

The `VerifyCsrfToken` [middleware](#), which is included in the `web` middleware group, will automatically verify that the token in the request input matches the token stored in the session.

CSRF Tokens & JavaScript

When building JavaScript driven applications, it is convenient to have your JavaScript HTTP library automatically attach the CSRF token to every outgoing request. By default, the Axios HTTP library provided in the `resources/js/bootstrap.js` file automatically sends an `X-XSRF-TOKEN` header using the value of the encrypted `XSRF-TOKEN` cookie. If you are not using this library, you will need to manually configure this behavior for your application.

Excluding URIs From CSRF Protection

Sometimes you may wish to exclude a set of URIs from CSRF protection. For example, if you are using [Stripe](#) to process payments and are utilizing their webhook system, you will need to exclude your Stripe webhook handler route from CSRF protection since Stripe will not know what CSRF token to send to your routes.

Typically, you should place these kinds of routes outside of the `web` middleware group that the `RouteServiceProvider` applies to all routes in the `routes/web.php` file. However, you may also exclude the routes by adding their URIs to the `$except` property of the `VerifyCsrfToken` middleware:

```
<?php

namespace App\Http\Middleware;

use Illuminate\Foundation\Http\Middleware\VerifyCsrfToken as Middleware;

class VerifyCsrfToken extends Middleware
{
    /**
     * The URIs that should be excluded from CSRF verification.
     *
     * @var array
     */
}
```

```
protected $except = [  
    'stripe/*',  
    'http://example.com/foo/bar',  
    'http://example.com/foo/*',  
];  
}
```

TIP The CSRF middleware is automatically disabled when [running tests](#).

X-CSRF-TOKEN

In addition to checking for the CSRF token as a POST parameter, the `verifyCsrfToken` middleware will also check for the `x-csrf-token` request header. You could, for example, store the token in an HTML meta tag:

```
<meta name="csrf-token" content="{ csrf_token() }">
```

Then, once you have created the meta tag, you can instruct a library like jQuery to automatically add the token to all request headers. This provides simple, convenient CSRF protection for your AJAX based applications:

```
$.ajaxSetup({  
    headers: {  
        'X-CSRF-TOKEN': $('meta[name="csrf-token"]').attr('content')  
    }  
});
```

X-XSRF-TOKEN

Laravel stores the current CSRF token in an encrypted `xsrftoken` cookie that is included with each response generated by the framework. You can use the cookie value to set the `x-xsrf-token` request header.

This cookie is primarily sent as a convenience since some JavaScript frameworks and libraries, like Angular and Axios, automatically place its value in the `x-xsrf-token` header on same-origin requests.

TIP By default, the `resources/js/bootstrap.js` file includes the Axios HTTP library which will automatically send this for you.

The Basics

Controllers

Controllers

- [Introduction](#)
- [Basic Controllers](#)
 - [Defining Controllers](#)
 - [Controllers & Namespaces](#)
 - [Single Action Controllers](#)
- [Controller Middleware](#)
- [Resource Controllers](#)
 - [Partial Resource Routes](#)
 - [Nested Resources](#)
 - [Naming Resource Routes](#)
 - [Naming Resource Route Parameters](#)
 - [Scoping Resource Routes](#)
 - [Localizing Resource URIs](#)
 - [Supplementing Resource Controllers](#)
- [Dependency Injection & Controllers](#)
- [Route Caching](#)

Introduction

Instead of defining all of your request handling logic as Closures in route files, you may wish to organize this behavior using Controller classes. Controllers can group related request handling logic into a single class. Controllers are stored in the `app/Http/Controllers` directory.

Basic Controllers

Defining Controllers

Below is an example of a basic controller class. Note that the controller extends the base controller class included with Laravel. The base class provides a few convenience methods such as the `middleware` method, which may be used to attach middleware to controller actions:

```
<?php

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;
use App\User;

class UserController extends Controller
{
    /**
     * Show the profile for the given user.
     *
     * @param int $id
     * @return View
     */
    public function show($id)
    {
        return view('user.profile', ['user' => User::findOrFail($id)]);
    }
}
```

You can define a route to this controller action like so:

```
Route::get('user/{id}', 'UserController@show');
```

Now, when a request matches the specified route URI, the `show` method on the `UserController` class will be executed. The route parameters will also be passed to the method.

TIP Controllers are not **required** to extend a base class. However, you will not have access to convenience features such as the `middleware`, `validate`, and `dispatch` methods.

Controllers & Namespaces

It is very important to note that we did not need to specify the full controller namespace when defining the controller route. Since the `RouteServiceProvider` loads your route files within a route group that contains the namespace, we only specified the portion of the class name that comes after the `App\Http\Controllers` portion of the namespace.

If you choose to nest your controllers deeper into the `App\Http\Controllers` directory, use the specific class name relative to the `App\Http\Controllers` root namespace. So, if your full controller class is `App\Http\Controllers\Photos\AdminController`, you should register routes to the controller like so:

```
Route::get('foo', 'Photos\AdminController@method');
```

Single Action Controllers

If you would like to define a controller that only handles a single action, you may place a single `__invoke` method on the controller:

```
<?php

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;
use App\User;

class ShowProfile extends Controller
{
    /**
     * Show the profile for the given user.
     *
     * @param int $id
     * @return View
     */
    public function __invoke($id)
    {
        return view('user.profile', ['user' => User::findOrFail($id)];
    }
}
```

When registering routes for single action controllers, you do not need to specify a method:

```
Route::get('user/{id}', 'ShowProfile');
```

You may generate an invokable controller by using the `--invokable` option of the `make:controller` Artisan command:

```
php artisan make:controller ShowProfile --invokable
```

TIP Controller stubs may be customized using [stub publishing](#)

Controller Middleware

[Middleware](#) may be assigned to the controller's routes in your route files:

```
Route::get('profile', 'UserController@show')->middleware('auth');
```

However, it is more convenient to specify middleware within your controller's constructor. Using the `middleware` method from your controller's constructor, you may easily assign middleware to the controller's action. You may even restrict the middleware to only certain methods on the controller class:

```
class UserController extends Controller
{
    /**
     * Instantiate a new controller instance.
     *
     */
}
```

```
* @return void
*/
public function __construct()
{
    $this->middleware('auth');

    $this->middleware('log')->only('index');

    $this->middleware('subscribed')->except('store');
}
}
```

Controllers also allow you to register middleware using a Closure. This provides a convenient way to define a middleware for a single controller without defining an entire middleware class:

```
$this->middleware(function ($request, $next) {
    // ...

    return $next($request);
});
```

TIP You may assign middleware to a subset of controller actions; however, it may indicate your controller is growing too large. Instead, consider breaking your controller into multiple, smaller controllers.

Resource Controllers

Laravel resource routing assigns the typical "CRUD" routes to a controller with a single line of code. For example, you may wish to create a controller that handles all HTTP requests for "photos" stored by your application. Using the `make:controller` Artisan command, we can quickly create such a controller:

```
php artisan make:controller PhotoController --resource
```

This command will generate a controller at `app/Http/Controllers/PhotoController.php`. The controller will contain a method for each of the available resource operations.

Next, you may register a resourceful route to the controller:

```
Route::resource('photos', 'PhotoController');
```

This single route declaration creates multiple routes to handle a variety of actions on the resource. The generated controller will already have methods stubbed for each of these actions, including notes informing you of the HTTP verbs and URIs they handle.

You may register many resource controllers at once by passing an array to the `resources` method:

```
Route::resources([
    'photos' => 'PhotoController',
    'posts' => 'PostController',
]);
```

Actions Handled By Resource Controller

Verb	URI	Action	Route Name
GET	/photos	index	photos.index
GET	/photos/create	create	photos.create
POST	/photos	store	photos.store
GET	/photos/{photo}	show	photos.show
GET	/photos/{photo}/edit	edit	photos.edit
PUT/PATCH	/photos/{photo}	update	photos.update
DELETE	/photos/{photo}	destroy	photos.destroy

Specifying The Resource Model

If you are using route model binding and would like the resource controller's methods to type-hint a model instance, you may use the `--model` option when generating the controller:

```
php artisan make:controller PhotoController --resource --model=Photo
```

Spoofing Form Methods

Since HTML forms can't make PUT, PATCH, or DELETE requests, you will need to add a hidden `_method` field to spoof these HTTP verbs. The `@method` Blade directive can create this field for you:

```
<form action="/foo/bar" method="POST">
    @method('PUT')
</form>
```

Partial Resource Routes

When declaring a resource route, you may specify a subset of actions the controller should handle instead of the full set of default actions:

```
Route::resource('photos', 'PhotoController')->only([
    'index', 'show'
]);

Route::resource('photos', 'PhotoController')->except([
    'create', 'store', 'update', 'destroy'
]);
```

API Resource Routes

When declaring resource routes that will be consumed by APIs, you will commonly want to exclude routes that present HTML templates such as `create` and `edit`. For convenience, you may use the `apiResource` method to automatically exclude these two routes:

```
Route::apiResource('photos', 'PhotoController');
```

You may register many API resource controllers at once by passing an array to the `apiResources` method:

```
Route::apiResources([
    'photos' => 'PhotoController',
    'posts' => 'PostController',
]);
```

To quickly generate an API resource controller that does not include the `create` or `edit` methods, use the `--api` switch when executing the `make:controller` command:

```
php artisan make:controller API/PhotoController --api
```

Nested Resources

Sometimes you may need to define routes to a nested resource. For example, a photo resource may have multiple comments that may be attached to the photo. To nest the resource controllers, use `"dot"` notation in your route declaration:

```
Route::resource('photos.comments', 'PhotoCommentController');
```

This route will register a nested resource that may be accessed with URIs like the following:

```
/photos/{photo}/comments/{comment}
```

Shallow Nesting

Often, it is not entirely necessary to have both the parent and the child IDs within a URI since the child ID is already a unique identifier. When using unique identifier such as auto-incrementing primary keys to identify your models in URI segments, you may choose to use `"shallow nesting"`:

```
Route::resource('photos.comments', 'CommentController')->shallow();
```

The route definition above will define the following routes:

Verb	URI	Action	Route Name
GET	/photos/{photo}/comments	index	photos.comments.index
GET	/photos/{photo}/comments/create	create	photos.comments.create
POST	/photos/{photo}/comments	store	photos.comments.store
GET	/comments/{comment}	show	comments.show
GET	/comments/{comment}/edit	edit	comments.edit
PUT/PATCH	/comments/{comment}	update	comments.update
DELETE	/comments/{comment}	destroy	comments.destroy

Naming Resource Routes

By default, all resource controller actions have a route name; however, you can override these names by passing a `names` array with your options:

```
Route::resource('photos', 'PhotoController')->names([
    'create' => 'photos.build'
]);
```

Naming Resource Route Parameters

By default, `Route::resource` will create the route parameters for your resource routes based on the "singularized" version of the resource name. You can easily override this on a per resource basis by using the `parameters` method. The array passed into the `parameters` method should be an associative array of resource names and parameter names:

```
Route::resource('users', 'AdminUserController')->parameters([
    'users' => 'admin_user'
]);
```

The example above generates the following URIs for the resource's `show` route:

```
/users/{admin_user}
```

Scoping Resource Routes

Sometimes, when implicitly binding multiple Eloquent models in resource route definitions, you may wish to scope the second Eloquent model such that it must be a child of the first Eloquent model. For example, consider this situation that retrieves a blog post by slug for a specific user:

```
use App\Http\Controllers\PostsController;

Route::resource('users.posts', PostsController::class)->scoped();
```

You may override the default model route keys by passing an array to the `scoped` method:

```
use App\Http\Controllers\PostsController;

Route::resource('users.posts', PostsController::class)->scoped([
    'post' => 'slug',
]);
```

When using a custom keyed implicit binding as a nested route parameter, Laravel will automatically scope the query to retrieve the nested model by its parent using conventions to guess the relationship name on the parent. In this case, it will be assumed that the `User` model has a relationship named `posts` (the plural of the route parameter name) which can be used to retrieve the `Post` model.

Localizing Resource URIs

By default, `Route::resource` will create resource URIs using English verbs. If you need to localize the `create` and `edit` action verbs, you may use the `Route::resourceVerbs` method. This may be done in the `boot` method of your `AppServiceProvider`:

```
use Illuminate\Support\Facades\Route;
```

```
/**
 * Bootstrap any application services.
 *
 * @return void
 */
public function boot()
{
    Route::resourceVerbs([
        'create' => 'crear',
        'edit' => 'editar',
    ]);
}
```

Once the verbs have been customized, a resource route registration such as `Route::resource('fotos', 'PhotoController')` will produce the following URIs:

```
/fotos/crear
/fotos/{foto}/editar
```

Supplementing Resource Controllers

If you need to add additional routes to a resource controller beyond the default set of resource routes, you should define those routes before your call to `Route::resource`; otherwise, the routes defined by the `resource` method may unintentionally take precedence over your supplemental routes:

```
Route::get('photos/popular', 'PhotoController@method');

Route::resource('photos', 'PhotoController');
```

TIP Remember to keep your controllers focused. If you find yourself routinely needing methods outside of the typical set of resource actions, consider splitting your controller into two, smaller controllers.

Dependency Injection & Controllers

Constructor Injection

The Laravel [service container](#) is used to resolve all Laravel controllers. As a result, you are able to type-hint any dependencies your controller may need in its constructor. The declared dependencies will automatically be resolved and injected into the controller instance:

```
<?php

namespace App\Http\Controllers;

use App\Repositories\UserRepository;

class UserController extends Controller
{
    /**
     * The user repository instance.
     */
    protected $users;

    /**
     * Create a new controller instance.
     *
     * @param UserRepository $users
     * @return void
     */
    public function __construct(UserRepository $users)
    {
        $this->users = $users;
    }
}
```

You may also type-hint any [Laravel contract](#). If the container can resolve it, you can type-hint it. Depending on your application, injecting your dependencies into your controller may provide better testability.

Method Injection

In addition to constructor injection, you may also type-hint dependencies on your controller's methods. A common use-case for method injection is injecting the `Illuminate\Http\Request` instance into your controller methods:

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

class UserController extends Controller
{
    /**
     * Store a new user.
     *
     * @param Request $request
     * @return Response
     */
    public function store(Request $request)
    {
        $name = $request->name;

        //
    }
}
```

If your controller method is also expecting input from a route parameter, list your route arguments after your other dependencies. For example, if your route is defined like so:

```
Route::put('user/{id}', 'UserController@update');
```

You may still type-hint the `Illuminate\Http\Request` and access your `id` parameter by defining your controller method as follows:

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

class UserController extends Controller
{
    /**
     * Update the given user.
     *
     * @param Request $request
     * @param string $id
     * @return Response
     */
    public function update(Request $request, $id)
    {
        //
    }
}
```

Route Caching

NOTE Closure based routes cannot be cached. To use route caching, you must convert any Closure routes to controller classes.

If your application is exclusively using controller based routes, you should take advantage of Laravel's route cache. Using the route cache will drastically decrease the amount of time it takes to register all of your application's routes. In some cases, your route registration may even be up to 100x faster. To generate a route cache, just execute the `route:cache` Artisan command:

```
php artisan route:cache
```

After running this command, your cached routes file will be loaded on every request. Remember, if you add any new routes you will need to generate a fresh route cache. Because of this, you should only run the `route:cache` command during your project's deployment.

You may use the `route:clear` command to clear the route cache:

```
php artisan route:clear
```

The Basics

Requests

HTTP Requests

- [Accessing The Request](#)
 - [Request Path & Method](#)
 - [PSR-7 Requests](#)
- [Input Trimming & Normalization](#)
- [Retrieving Input](#)
 - [Old Input](#)
 - [Cookies](#)
- [Files](#)
 - [Retrieving Uploaded Files](#)
 - [Storing Uploaded Files](#)
- [Configuring Trusted Proxies](#)

Accessing The Request

To obtain an instance of the current HTTP request via dependency injection, you should type-hint the `Illuminate\Http\Request` class on your controller method. The incoming request instance will automatically be injected by the [service container](#):

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

class UserController extends Controller
{
    /**
     * Store a new user.
     *
     * @param Request $request
     * @return Response
     */
    public function store(Request $request)
    {
        $name = $request->input('name');

        //
    }
}
```

Dependency Injection & Route Parameters

If your controller method is also expecting input from a route parameter you should list your route parameters after your other dependencies. For example, if your route is defined like so:

```
Route::put('user/{id}', 'UserController@update');
```

You may still type-hint the `Illuminate\Http\Request` and access your route parameter `id` by defining your controller method as follows:

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

class UserController extends Controller
{
    /**
     * Update the specified user.
     *
     * @param Request $request
```



```
        * @param string $id
        * @return Response
        */
        public function update(Request $request, $id)
        {
            //
        }
    }
}
```

Accessing The Request Via Route Closures

You may also type-hint the `Illuminate\Http\Request` class on a route Closure. The service container will automatically inject the incoming request into the Closure when it is executed:

```
use Illuminate\Http\Request;

Route::get('/', function (Request $request) {
    //
});
```

Request Path & Method

The `Illuminate\Http\Request` instance provides a variety of methods for examining the HTTP request for your application and extends the `Symfony\Component\HttpFoundation\Request` class. We will discuss a few of the most important methods below.

Retrieving The Request Path

The `path` method returns the request's path information. So, if the incoming request is targeted at `http://domain.com/foo/bar`, the `path` method will return `foo/bar`:

```
$uri = $request->path();
```

The `is` method allows you to verify that the incoming request path matches a given pattern. You may use the `*` character as a wildcard when utilizing this method:

```
if ($request->is('admin/*')) {
    //
}
```

Retrieving The Request URL

To retrieve the full URL for the incoming request you may use the `url` or `fullUrl` methods. The `url` method will return the URL without the query string, while the `fullUrl` method includes the query string:

```
// Without Query String...
$url = $request->url();

// With Query String...
$url = $request->fullUrl();
```

Retrieving The Request Method

The `method` method will return the HTTP verb for the request. You may use the `isMethod` method to verify that the HTTP verb matches a given string:

```
$method = $request->method();

if ($request->isMethod('post')) {
    //
}
```

PSR-7 Requests

The [PSR-7 standard](#) specifies interfaces for HTTP messages, including requests and responses. If you would like to obtain an instance of a PSR-7 request instead of a Laravel request, you will first need to install a few

libraries. Laravel uses the *Symfony HTTP Message Bridge* component to convert typical Laravel requests and responses into PSR-7 compatible implementations:

```
composer require symfony/psr-http-message-bridge
composer require nyholm/psr7
```

Once you have installed these libraries, you may obtain a PSR-7 request by type-hinting the request interface on your route Closure or controller method:

```
use Psr\Http\Message\ServerRequestInterface;

Route::get('/', function (ServerRequestInterface $request) {
    //
});
```

TIP If you return a PSR-7 response instance from a route or controller, it will automatically be converted back to a Laravel response instance and be displayed by the framework.

Input Trimming & Normalization

By default, Laravel includes the `TrimStrings` and `ConvertEmptyStringsToNull` middleware in your application's global middleware stack. These middleware are listed in the stack by the `App\Http\Kernel` class. These middleware will automatically trim all incoming string fields on the request, as well as convert any empty string fields to `null`. This allows you to not have to worry about these normalization concerns in your routes and controllers.

If you would like to disable this behavior, you may remove the two middleware from your application's middleware stack by removing them from the `$middleware` property of your `App\Http\Kernel` class.

Retrieving Input

Retrieving All Input Data

You may also retrieve all of the input data as an array using the `all` method:

```
$input = $request->all();
```

Retrieving An Input Value

Using a few simple methods, you may access all of the user input from your `Illuminate\Http\Request` instance without worrying about which HTTP verb was used for the request. Regardless of the HTTP verb, the `input` method may be used to retrieve user input:

```
$name = $request->input('name');
```

You may pass a default value as the second argument to the `input` method. This value will be returned if the requested input value is not present on the request:

```
$name = $request->input('name', 'Sally');
```

When working with forms that contain array inputs, use "dot" notation to access the arrays:

```
$name = $request->input('products.0.name');
```

```
$names = $request->input('products.*.name');
```

You may call the `input` method without any arguments in order to retrieve all of the input values as an associative array:

```
$input = $request->input();
```

Retrieving Input From The Query String

While the `input` method retrieves values from entire request payload (including the query string), the `query`

method will only retrieve values from the query string:

```
$name = $request->query('name');
```

If the requested query string value data is not present, the second argument to this method will be returned:

```
$name = $request->query('name', 'Helen');
```

You may call the `query` method without any arguments in order to retrieve all of the query string values as an associative array:

```
$query = $request->query();
```

Retrieving Input Via Dynamic Properties

You may also access user input using dynamic properties on the `Illuminate\Http\Request` instance. For example, if one of your application's forms contains a `name` field, you may access the value of the field like so:

```
$name = $request->name;
```

When using dynamic properties, Laravel will first look for the parameter's value in the request payload. If it is not present, Laravel will search for the field in the route parameters.

Retrieving JSON Input Values

When sending JSON requests to your application, you may access the JSON data via the `input` method as long as the `Content-Type` header of the request is properly set to `application/json`. You may even use "dot" syntax to dig into JSON arrays:

```
$name = $request->input('user.name');
```

Retrieving Boolean Input Values

When dealing with HTML elements like checkboxes, your application may receive "truthy" values that are actually strings. For example, "true" or "on". For convenience, you may use the `boolean` method to retrieve these values as booleans. The `boolean` method returns `true` for 1, "1", true, "true", "on", and "yes". All other values will return `false`:

```
$archived = $request->boolean('archived');
```

Retrieving A Portion Of The Input Data

If you need to retrieve a subset of the input data, you may use the `only` and `except` methods. Both of these methods accept a single array or a dynamic list of arguments:

```
$input = $request->only(['username', 'password']);
```

```
$input = $request->only('username', 'password');
```

```
$input = $request->except(['credit_card']);
```

```
$input = $request->except('credit_card');
```

TIP The `only` method returns all of the key / value pairs that you request; however, it will not return key / value pairs that are not present on the request.

Determining If An Input Value Is Present

You should use the `has` method to determine if a value is present on the request. The `has` method returns `true` if the value is present on the request:

```
if ($request->has('name')) {  
    //  
}
```

When given an array, the `has` method will determine if all of the specified values are present:

```
if ($request->has(['name', 'email'])) {  
    //  
}
```

The `hasAny` method returns `true` if any of the specified values are present:

```
if ($request->hasAny(['name', 'email'])) {  
    //  
}
```

If you would like to determine if a value is present on the request and is not empty, you may use the `filled` method:

```
if ($request->filled('name')) {  
    //  
}
```

To determine if a given key is absent from the request, you may use the `missing` method:

```
if ($request->missing('name')) {  
    //  
}
```

Old Input

Laravel allows you to keep input from one request during the next request. This feature is particularly useful for re-populating forms after detecting validation errors. However, if you are using Laravel's included [validation features](#), it is unlikely you will need to manually use these methods, as some of Laravel's built-in validation facilities will call them automatically.

Flashing Input To The Session

The `flash` method on the `Illuminate\Http\Request` class will flash the current input to the [session](#) so that it is available during the user's next request to the application:

```
$request->flash();
```

You may also use the `flashOnly` and `flashExcept` methods to flash a subset of the request data to the session. These methods are useful for keeping sensitive information such as passwords out of the session:

```
$request->flashOnly(['username', 'email']);  
$request->flashExcept('password');
```

Flashing Input Then Redirecting

Since you often will want to flash input to the session and then redirect to the previous page, you may easily chain input flashing onto a redirect using the `withInput` method:

```
return redirect('form')->withInput();  
  
return redirect('form')->withInput(  
    $request->except('password')  
);
```

Retrieving Old Input

To retrieve flashed input from the previous request, use the `old` method on the `Request` instance. The `old` method will pull the previously flashed input data from the [session](#):

```
$username = $request->old('username');
```

Laravel also provides a global `old` helper. If you are displaying old input within a [Blade template](#), it is more convenient to use the `old` helper. If no old input exists for the given field, `null` will be returned:

```
<input type="text" name="username" value="{{ old('username') }}">
```

Cookies

Retrieving Cookies From Requests

All cookies created by the Laravel framework are encrypted and signed with an authentication code, meaning they will be considered invalid if they have been changed by the client. To retrieve a cookie value from the request, use the `cookie` method on a `Illuminate\Http\Request` instance:

```
$value = $request->cookie('name');
```

Alternatively, you may use the `Cookie` facade to access cookie values:

```
use Illuminate\Support\Facades\Cookie;

$value = Cookie::get('name');
```

Attaching Cookies To Responses

You may attach a cookie to an outgoing `Illuminate\Http\Response` instance using the `cookie` method. You should pass the name, value, and number of minutes the cookie should be considered valid to this method:

```
return response('Hello World')->cookie(
    'name', 'value', $minutes
);
```

The `cookie` method also accepts a few more arguments which are used less frequently. Generally, these arguments have the same purpose and meaning as the arguments that would be given to PHP's native [setcookie](#) method:

```
return response('Hello World')->cookie(
    'name', 'value', $minutes, $path, $domain, $secure, $httpOnly
);
```

Alternatively, you can use the `Cookie` facade to "queue" cookies for attachment to the outgoing response from your application. The `queue` method accepts a `Cookie` instance or the arguments needed to create a `Cookie` instance. These cookies will be attached to the outgoing response before it is sent to the browser:

```
Cookie::queue(Cookie::make('name', 'value', $minutes));

Cookie::queue('name', 'value', $minutes);
```

Generating Cookie Instances

If you would like to generate a `Symfony\Component\HttpFoundation\Cookie` instance that can be given to a response instance at a later time, you may use the global `cookie` helper. This cookie will not be sent back to the client unless it is attached to a response instance:

```
$cookie = cookie('name', 'value', $minutes);

return response('Hello World')->cookie($cookie);
```

Expiring Cookies Early

You may remove a cookie by expiring it via the `forget` method of the `Cookie` facade:

```
Cookie::queue(Cookie::forget('name'));
```

Alternatively, you may attach the expired cookie to a response instance:

```
$cookie = Cookie::forget('name');

return response('Hello World')->withCookie($cookie);
```

Files

Retrieving Uploaded Files

You may access uploaded files from a `Illuminate\Http\Request` instance using the `file` method or using dynamic properties. The `file` method returns an instance of the `Illuminate\Http\UploadedFile` class, which extends the PHP `SplFileInfo` class and provides a variety of methods for interacting with the file:

```
$file = $request->file('photo');  
  
$file = $request->photo;
```

You may determine if a file is present on the request using the `hasFile` method:

```
if ($request->hasFile('photo')) {  
    //  
}
```

Validating Successful Uploads

In addition to checking if the file is present, you may verify that there were no problems uploading the file via the `isValid` method:

```
if ($request->file('photo')->isValid()) {  
    //  
}
```

File Paths & Extensions

The `UploadedFile` class also contains methods for accessing the file's fully-qualified path and its extension. The `extension` method will attempt to guess the file's extension based on its contents. This extension may be different from the extension that was supplied by the client:

```
$path = $request->photo->path();  
  
$extension = $request->photo->extension();
```

Other File Methods

There are a variety of other methods available on `UploadedFile` instances. Check out the [API documentation for the class](#) for more information regarding these methods.

Storing Uploaded Files

To store an uploaded file, you will typically use one of your configured [filesystems](#). The `UploadedFile` class has a `store` method which will move an uploaded file to one of your disks, which may be a location on your local filesystem or even a cloud storage location like Amazon S3.

The `store` method accepts the path where the file should be stored relative to the filesystem's configured root directory. This path should not contain a file name, since a unique ID will automatically be generated to serve as the file name.

The `store` method also accepts an optional second argument for the name of the disk that should be used to store the file. The method will return the path of the file relative to the disk's root:

```
$path = $request->photo->store('images');  
  
$path = $request->photo->store('images', 's3');
```

If you do not want a file name to be automatically generated, you may use the `storeAs` method, which accepts the path, file name, and disk name as its arguments:

```
$path = $request->photo->storeAs('images', 'filename.jpg');
```

```
$path = $request->photo->storeAs('images', 'filename.jpg', 's3');
```

Configuring Trusted Proxies

When running your applications behind a load balancer that terminates TLS / SSL certificates, you may notice your application sometimes does not generate HTTPS links. Typically this is because your application is being forwarded traffic from your load balancer on port 80 and does not know it should generate secure links.

To solve this, you may use the `App\Http\Middleware\TrustProxies` middleware that is included in your Laravel application, which allows you to quickly customize the load balancers or proxies that should be trusted by your application. Your trusted proxies should be listed as an array on the `$proxies` property of this middleware. In addition to configuring the trusted proxies, you may configure the proxy `$headers` that should be trusted:

```
<?php

namespace App\Http\Middleware;

use Fideloper\Proxy\TrustProxies as Middleware;
use Illuminate\Http\Request;

class TrustProxies extends Middleware
{
    /**
     * The trusted proxies for this application.
     *
     * @var string|array
     */
    protected $proxies = [
        '192.168.1.1',
        '192.168.1.2',
    ];

    /**
     * The headers that should be used to detect proxies.
     *
     * @var int
     */
    protected $headers = Request::HEADER_X_FORWARDED_ALL;
}
```

TIP If you are using AWS Elastic Load Balancing, your `$headers` value should be `Request::HEADER_X_FORWARDED_AWS_ELB`. For more information on the constants that may be used in the `$headers` property, check out Symfony's documentation on [trusting proxies](#).

Trusting All Proxies

If you are using Amazon AWS or another "cloud" load balancer provider, you may not know the IP addresses of your actual balancers. In this case, you may use `*` to trust all proxies:

```
/**
 * The trusted proxies for this application.
 *
 * @var string|array
 */
protected $proxies = '*';
```

The Basics

Responses

HTTP Responses

- [Creating Responses](#)
 - [Attaching Headers To Responses](#)
 - [Attaching Cookies To Responses](#)
 - [Cookies & Encryption](#)
- [Redirects](#)
 - [Redirecting To Named Routes](#)
 - [Redirecting To Controller Actions](#)
 - [Redirecting To External Domains](#)
 - [Redirecting With Flashed Session Data](#)
- [Other Response Types](#)
 - [View Responses](#)
 - [JSON Responses](#)
 - [File Downloads](#)
 - [File Responses](#)
- [Response Macros](#)

Creating Responses

Strings & Arrays

All routes and controllers should return a response to be sent back to the user's browser. Laravel provides several different ways to return responses. The most basic response is returning a string from a route or controller. The framework will automatically convert the string into a full HTTP response:

```
Route::get('/', function () {  
    return 'Hello World';  
});
```

In addition to returning strings from your routes and controllers, you may also return arrays. The framework will automatically convert the array into a JSON response:

```
Route::get('/', function () {  
    return [1, 2, 3];  
});
```

TIP Did you know you can also return [Eloquent collections](#) from your routes or controllers? They will automatically be converted to JSON. Give it a shot!

Response Objects

Typically, you won't just be returning simple strings or arrays from your route actions. Instead, you will be returning full `Illuminate\Http\Response` instances or [views](#).

Returning a full `Response` instance allows you to customize the response's HTTP status code and headers. A `Response` instance inherits from the `Symfony\Component\HttpFoundation\Response` class, which provides a variety of methods for building HTTP responses:

```
Route::get('home', function () {  
    return response('Hello World', 200)  
        ->header('Content-Type', 'text/plain');  
});
```

Attaching Headers To Responses

Keep in mind that most response methods are chainable, allowing for the fluent construction of response instances. For example, you may use the `header` method to add a series of headers to the response before

sending it back to the user:

```
return response($content)
    ->header('Content-Type', $type)
    ->header('X-Header-One', 'Header Value')
    ->header('X-Header-Two', 'Header Value');
```

Or, you may use the `withHeaders` method to specify an array of headers to be added to the response:

```
return response($content)
    ->withHeaders([
        'Content-Type' => $type,
        'X-Header-One' => 'Header Value',
        'X-Header-Two' => 'Header Value',
    ]);
```

Cache Control Middleware

Laravel includes a `cache.headers` middleware, which may be used to quickly set the `cache-control` header for a group of routes. If `etag` is specified in the list of directives, an MD5 hash of the response content will automatically be set as the ETag identifier:

```
Route::middleware('cache.headers:public;max_age=2628000;etag')->group(function () {
    Route::get('privacy', function () {
        // ...
    });

    Route::get('terms', function () {
        // ...
    });
});
```

Attaching Cookies To Responses

The `cookie` method on response instances allows you to easily attach cookies to the response. For example, you may use the `cookie` method to generate a cookie and fluently attach it to the response instance like so:

```
return response($content)
    ->header('Content-Type', $type)
    ->cookie('name', 'value', $minutes);
```

The `cookie` method also accepts a few more arguments which are used less frequently. Generally, these arguments have the same purpose and meaning as the arguments that would be given to PHP's native [setcookie](#) method:

```
->cookie($name, $value, $minutes, $path, $domain, $secure, $httpOnly)
```

Alternatively, you can use the `cookie` facade to "queue" cookies for attachment to the outgoing response from your application. The `queue` method accepts a `Cookie` instance or the arguments needed to create a `Cookie` instance. These cookies will be attached to the outgoing response before it is sent to the browser:

```
Cookie::queue(Cookie::make('name', 'value', $minutes));

Cookie::queue('name', 'value', $minutes);
```

Cookies & Encryption

By default, all cookies generated by Laravel are encrypted and signed so that they can't be modified or read by the client. If you would like to disable encryption for a subset of cookies generated by your application, you may use the `$except` property of the `App\Http\Middleware\EncryptCookies` middleware, which is located in the `app/Http/Middleware` directory:

```
/**
 * The names of the cookies that should not be encrypted.
 *
 * @var array
 */
protected $except = [
    'cookie_name',
```

```
];
```

Redirects

Redirect responses are instances of the `Illuminate\Http\RedirectResponse` class, and contain the proper headers needed to redirect the user to another URL. There are several ways to generate a `RedirectResponse` instance. The simplest method is to use the global `redirect` helper:

```
Route::get('dashboard', function () {
    return redirect('home/dashboard');
});
```

Sometimes you may wish to redirect the user to their previous location, such as when a submitted form is invalid. You may do so by using the global `back` helper function. Since this feature utilizes the [session](#), make sure the route calling the `back` function is using the `web` middleware group or has all of the session middleware applied:

```
Route::post('user/profile', function () {
    // Validate the request...

    return back()->withInput();
});
```

Redirecting To Named Routes

When you call the `redirect` helper with no parameters, an instance of `Illuminate\Routing\Redirector` is returned, allowing you to call any method on the `Redirector` instance. For example, to generate a `RedirectResponse` to a named route, you may use the `route` method:

```
return redirect()->route('login');
```

If your route has parameters, you may pass them as the second argument to the `route` method:

```
// For a route with the following URI: profile/{id}

return redirect()->route('profile', ['id' => 1]);
```

Populating Parameters Via Eloquent Models

If you are redirecting to a route with an "ID" parameter that is being populated from an Eloquent model, you may pass the model itself. The ID will be extracted automatically:

```
// For a route with the following URI: profile/{id}

return redirect()->route('profile', [$user]);
```

If you would like to customize the value that is placed in the route parameter, you can specify the column in the route parameter definition (`profile/{id:slug}`) or you can override the `getRouteKey` method on your Eloquent model:

```
/**
 * Get the value of the model's route key.
 *
 * @return mixed
 */
public function getRouteKey()
{
    return $this->slug;
}
```

Redirecting To Controller Actions

You may also generate redirects to [controller actions](#). To do so, pass the controller and action name to the `action` method. Remember, you do not need to specify the full namespace to the controller since Laravel's `RouteServiceProvider` will automatically set the base controller namespace:

```
return redirect()->action('HomeController@index');
```

If your controller route requires parameters, you may pass them as the second argument to the `action` method:

```
return redirect()->action(
    'UserController@profile', ['id' => 1]
);
```

Redirecting To External Domains

Sometimes you may need to redirect to a domain outside of your application. You may do so by calling the `away` method, which creates a `RedirectResponse` without any additional URL encoding, validation, or verification:

```
return redirect()->away('https://www.google.com');
```

Redirecting With Flashed Session Data

Redirecting to a new URL and [flashing data to the session](#) are usually done at the same time. Typically, this is done after successfully performing an action when you flash a success message to the session. For convenience, you may create a `RedirectResponse` instance and flash data to the session in a single, fluent method chain:

```
Route::post('user/profile', function () {
    // Update the user's profile...

    return redirect('dashboard')->with('status', 'Profile updated!');
});
```

After the user is redirected, you may display the flashed message from the [session](#). For example, using [Blade syntax](#):

```
@if (session('status'))
    <div class="alert alert-success">
        {{ session('status') }}
    </div>
@endif
```

Other Response Types

The response helper may be used to generate other types of response instances. When the response helper is called without arguments, an implementation of the `Illuminate\Contracts\Routing\ResponseFactory` [contract](#) is returned. This contract provides several helpful methods for generating responses.

View Responses

If you need control over the response's status and headers but also need to return a [view](#) as the response's content, you should use the `view` method:

```
return response()
    ->view('hello', $data, 200)
    ->header('Content-Type', $type);
```

Of course, if you do not need to pass a custom HTTP status code or custom headers, you should use the global `view` helper function.

JSON Responses

The `json` method will automatically set the `Content-Type` header to `application/json`, as well as convert the given array to JSON using the `json_encode` PHP function:

```
return response()->json([
    'name' => 'Abigail',
    'state' => 'CA',
]);
```

If you would like to create a JSONP response, you may use the `json` method in combination with the `withCallback` method:

```
return response()
    ->json(['name' => 'Abigail', 'state' => 'CA'])
    ->withCallback($request->input('callback'));
```

File Downloads

The `download` method may be used to generate a response that forces the user's browser to download the file at the given path. The `download` method accepts a file name as the second argument to the method, which will determine the file name that is seen by the user downloading the file. Finally, you may pass an array of HTTP headers as the third argument to the method:

```
return response()->download($pathToFile);

return response()->download($pathToFile, $name, $headers);

return response()->download($pathToFile)->deleteFileAfterSend();
```

NOTE Symfony HttpFoundation, which manages file downloads, requires the file being downloaded to have an ASCII file name.

Streamed Downloads

Sometimes you may wish to turn the string response of a given operation into a downloadable response without having to write the contents of the operation to disk. You may use the `streamDownload` method in this scenario. This method accepts a callback, file name, and an optional array of headers as its arguments:

```
return response()->streamDownload(function () {
    echo GitHub::api('repo')
        ->contents()
        ->readme('laravel', 'laravel')['contents'];
}, 'laravel-readme.md');
```

File Responses

The `file` method may be used to display a file, such as an image or PDF, directly in the user's browser instead of initiating a download. This method accepts the path to the file as its first argument and an array of headers as its second argument:

```
return response()->file($pathToFile);

return response()->file($pathToFile, $headers);
```

Response Macros

If you would like to define a custom response that you can re-use in a variety of your routes and controllers, you may use the `macro` method on the Response facade. For example, from a [service provider's](#) `boot` method:

```
<?php

namespace App\Providers;

use Illuminate\Support\Facades\Response;
use Illuminate\Support\ServiceProvider;

class ResponseMacroServiceProvider extends ServiceProvider
{
    /**
     * Register the application's response macros.
     *
     * @return void
     */
    public function boot()
    {
        Response::macro('caps', function ($value) {
            return Response::make(strtoupper($value));
        });
    }
}
```

The `macro` function accepts a name as its first argument, and a Closure as its second. The macro's Closure will be executed when calling the macro name from a `ResponseFactory` implementation or the `response` helper:

```
return response()->caps('foo');
```

The Basics

Views

Views

- [Creating Views](#)
- [Passing Data To Views](#)
 - [Sharing Data With All Views](#)
- [View Composers](#)
- [Optimizing Views](#)

Creating Views

TIP Looking for more information on how to write Blade templates? Check out the full [Blade documentation](#) to get started.

Views contain the HTML served by your application and separate your controller / application logic from your presentation logic. Views are stored in the `resources/views` directory. A simple view might look something like this:

```
<!-- View stored in resources/views/greeting.blade.php -->

<html>
  <body>
    <h1>Hello, {{ $name }}</h1>
  </body>
</html>
```

Since this view is stored at `resources/views/greeting.blade.php`, we may return it using the global `view` helper like so:

```
Route::get('/', function () {
    return view('greeting', ['name' => 'James']);
});
```

As you can see, the first argument passed to the `view` helper corresponds to the name of the view file in the `resources/views` directory. The second argument is an array of data that should be made available to the view. In this case, we are passing the `name` variable, which is displayed in the view using [Blade syntax](#).

Views may also be nested within subdirectories of the `resources/views` directory. "Dot" notation may be used to reference nested views. For example, if your view is stored at `resources/views/admin/profile.blade.php`, you may reference it like so:

```
return view('admin.profile', $data);
```

NOTE View directory names should not contain the `.` character.

Determining If A View Exists

If you need to determine if a view exists, you may use the `view` facade. The `exists` method will return `true` if the view exists:

```
use Illuminate\Support\Facades\View;

if (View::exists('emails.customer')) {
    //
}
```

Creating The First Available View

Using the `first` method, you may create the first view that exists in a given array of views. This is useful if your application or package allows views to be customized or overwritten:

```
return view()->first(['custom.admin', 'admin'], $data);
```

You may also call this method via the view [facade](#):

```
use Illuminate\Support\Facades\View;

return View::first(['custom.admin', 'admin'], $data);
```

Passing Data To Views

As you saw in the previous examples, you may pass an array of data to views:

```
return view('greetings', ['name' => 'Victoria']);
```

When passing information in this manner, the data should be an array with key / value pairs. Inside your view, you can then access each value using its corresponding key, such as `<?php echo $key; ?>`. As an alternative to passing a complete array of data to the view helper function, you may use the `with` method to add individual pieces of data to the view:

```
return view('greeting')->with('name', 'Victoria');
```

Sharing Data With All Views

Occasionally, you may need to share a piece of data with all views that are rendered by your application. You may do so using the view facade's `share` method. Typically, you should place calls to `share` within a service provider's `boot` method. You are free to add them to the `AppServiceProvider` or generate a separate service provider to house them:

```
<?php

namespace App\Providers;

use Illuminate\Support\Facades\View;

class AppServiceProvider extends ServiceProvider
{
    /**
     * Register any application services.
     *
     * @return void
     */
    public function register()
    {
        //
    }

    /**
     * Bootstrap any application services.
     *
     * @return void
     */
    public function boot()
    {
        View::share('key', 'value');
    }
}
```

View Composers

View composers are callbacks or class methods that are called when a view is rendered. If you have data that you want to be bound to a view each time that view is rendered, a view composer can help you organize that logic into a single location.

For this example, let's register the view composers within a [service provider](#). We'll use the view facade to access the underlying `Illuminate\Contracts\View\Factory` contract implementation. Remember, Laravel does not include a default directory for view composers. You are free to organize them however you wish. For example, you could create an `app/Http/View/Composers` directory:

```
<?php
```

```
namespace App\Providers;

use Illuminate\Support\Facades\View;
use Illuminate\Support\ServiceProvider;

class ViewServiceProvider extends ServiceProvider
{
    /**
     * Register any application services.
     *
     * @return void
     */
    public function register()
    {
        //
    }

    /**
     * Bootstrap any application services.
     *
     * @return void
     */
    public function boot()
    {
        // Using class based composers...
        View::composer(
            'profile', 'App\Http\View\Composers\ProfileComposer'
        );

        // Using Closure based composers...
        View::composer('dashboard', function ($view) {
            //
        });
    }
}
```

NOTE Remember, if you create a new service provider to contain your view composer registrations, you will need to add the service provider to the providers array in the config/app.php configuration file.

Now that we have registered the composer, the ProfileComposer@compose method will be executed each time the profile view is being rendered. So, let's define the composer class:

```
<?php

namespace App\Http\View\Composers;

use App\Repositories\UserRepository;
use Illuminate\View\View;

class ProfileComposer
{
    /**
     * The user repository implementation.
     *
     * @var UserRepository
     */
    protected $users;

    /**
     * Create a new profile composer.
     *
     * @param UserRepository $users
     * @return void
     */
    public function __construct(UserRepository $users)
    {
        // Dependencies automatically resolved by service container...
        $this->users = $users;
    }

    /**
     * Bind data to the view.
     *
     * @param View $view
     * @return void
     */
    public function compose(View $view)
    {
    }
}
```



```
        $view->with('count', $this->users->count());
    }
}
```

Just before the view is rendered, the composer's `compose` method is called with the `Illuminate\View\View` instance. You may use the `with` method to bind data to the view.

TIP All view composers are resolved via the [service container](#), so you may type-hint any dependencies you need within a composer's constructor.

Attaching A Composer To Multiple Views

You may attach a view composer to multiple views at once by passing an array of views as the first argument to the `composer` method:

```
View::composer(
    ['profile', 'dashboard'],
    'App\Http\View\Composers\MyViewComposer'
);
```

The `composer` method also accepts the `*` character as a wildcard, allowing you to attach a composer to all views:

```
View::composer('*', function ($view) {
    //
});
```

View Creators

View **creators** are very similar to view composers; however, they are executed immediately after the view is instantiated instead of waiting until the view is about to render. To register a view creator, use the `creator` method:

```
View::creator('profile', 'App\Http\View\Creators\ProfileCreator');
```

Optimizing Views

By default, views are compiled on demand. When a request is executed that renders a view, Laravel will determine if a compiled version of the view exists. If the file exists, Laravel will then determine if the uncompiled view has been modified more recently than the compiled view. If the compiled view either does not exist, or the uncompiled view has been modified, Laravel will recompile the view.

Compiling views during the request negatively impacts performance, so Laravel provides the `view:cache` Artisan command to precompile all of the views utilized by your application. For increased performance, you may wish to run this command as part of your deployment process:

```
php artisan view:cache
```

You may use the `view:clear` command to clear the view cache:

```
php artisan view:clear
```

The Basics

URL Generation

URL Generation

- [Introduction](#)
- [The Basics](#)
 - [Generating Basic URLs](#)
 - [Accessing The Current URL](#)
- [URLs For Named Routes](#)
 - [Signed URLs](#)
- [URLs For Controller Actions](#)
- [Default Values](#)

Introduction

Laravel provides several helpers to assist you in generating URLs for your application. These are mainly helpful when building links in your templates and API responses, or when generating redirect responses to another part of your application.

The Basics

Generating Basic URLs

The `url` helper may be used to generate arbitrary URLs for your application. The generated URL will automatically use the scheme (HTTP or HTTPS) and host from the current request:

```
$post = App\Post::find(1);  
  
echo url("/posts/{$post->id}");  
  
// http://example.com/posts/1
```

Accessing The Current URL

If no path is provided to the `url` helper, a `Illuminate\Routing\UrlGenerator` instance is returned, allowing you to access information about the current URL:

```
// Get the current URL without the query string...  
echo url()->current();  
  
// Get the current URL including the query string...  
echo url()->full();  
  
// Get the full URL for the previous request...  
echo url()->previous();
```

Each of these methods may also be accessed via the URL [facade](#):

```
use Illuminate\Support\Facades\Url;  
  
echo Url::current();
```

URLs For Named Routes

The `route` helper may be used to generate URLs to named routes. Named routes allow you to generate URLs without being coupled to the actual URL defined on the route. Therefore, if the route's URL changes, no changes need to be made to your route function calls. For example, imagine your application contains a route defined like the following:

```
Route::get('/post/{post}', function () {
```

```
//
})->name('post.show');
```

To generate a URL to this route, you may use the `route` helper like so:

```
echo route('post.show', ['post' => 1]);

// http://example.com/post/1
```

You will often be generating URLs using the primary key of [Eloquent models](#). For this reason, you may pass Eloquent models as parameter values. The `route` helper will automatically extract the model's primary key:

```
echo route('post.show', ['post' => $post]);
```

The `route` helper may also be used to generate URLs for routes with multiple parameters:

```
Route::get('/post/{post}/comment/{comment}', function () {
    //
})->name('comment.show');

echo route('comment.show', ['post' => 1, 'comment' => 3]);

// http://example.com/post/1/comment/3
```

Signed URLs

Laravel allows you to easily create "signed" URLs to named routes. These URLs have a "signature" hash appended to the query string which allows Laravel to verify that the URL has not been modified since it was created. Signed URLs are especially useful for routes that are publicly accessible yet need a layer of protection against URL manipulation.

For example, you might use signed URLs to implement a public "unsubscribe" link that is emailed to your customers. To create a signed URL to a named route, use the `signedRoute` method of the `URL` facade:

```
use Illuminate\Support\Facades\URL;

return URL::signedRoute('unsubscribe', ['user' => 1]);
```

If you would like to generate a temporary signed route URL that expires, you may use the `temporarySignedRoute` method:

```
use Illuminate\Support\Facades\URL;

return URL::temporarySignedRoute(
    'unsubscribe', now()->addMinutes(30), ['user' => 1]
);
```

Validating Signed Route Requests

To verify that an incoming request has a valid signature, you should call the `hasValidSignature` method on the incoming `Request`:

```
use Illuminate\Http\Request;

Route::get('/unsubscribe/{user}', function (Request $request) {
    if (! $request->hasValidSignature()) {
        abort(401);
    }

    // ...
})->name('unsubscribe');
```

Alternatively, you may assign the `Illuminate\Routing\Middleware\ValidateSignature` middleware to the route. If it is not already present, you should assign this middleware a key in your HTTP kernel's `routeMiddleware` array:

```
/**
 * The application's route middleware.
 *
 * These middleware may be assigned to groups or used individually.
 *
 * @var array
```

```
*/
protected $routeMiddleware = [
    'signed' => \Illuminate\Routing\Middleware\ValidateSignature::class,
];
```

Once you have registered the middleware in your kernel, you may attach it to a route. If the incoming request does not have a valid signature, the middleware will automatically return a 403 error response:

```
Route::post('/unsubscribe/{user}', function (Request $request) {
    // ...
})->name('unsubscribe')->middleware('signed');
```

URLs For Controller Actions

The action function generates a URL for the given controller action. You do not need to pass the full namespace of the controller. Instead, pass the controller class name relative to the `App\Http\Controllers` namespace:

```
$url = action('HomeController@index');
```

You may also reference actions with a "callable" array syntax:

```
use App\Http\Controllers\HomeController;

$url = action([HomeController::class, 'index']);
```

If the controller method accepts route parameters, you may pass them as the second argument to the function:

```
$url = action('UserController@profile', ['id' => 1]);
```

Default Values

For some applications, you may wish to specify request-wide default values for certain URL parameters. For example, imagine many of your routes define a `{locale}` parameter:

```
Route::get('/{locale}/posts', function () {
    //
})->name('post.index');
```

It is cumbersome to always pass the `locale` every time you call the route helper. So, you may use the `URL::defaults` method to define a default value for this parameter that will always be applied during the current request. You may wish to call this method from a [route middleware](#) so that you have access to the current request:

```
<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Support\Facades\URL;

class SetDefaultLocaleForUrls
{
    public function handle($request, Closure $next)
    {
        URL::defaults(['locale' => $request->user()->locale]);

        return $next($request);
    }
}
```

Once the default value for the `locale` parameter has been set, you are no longer required to pass its value when generating URLs via the route helper.

The Basics

Session

HTTP Session

- [Introduction](#)
 - [Configuration](#)
 - [Driver Prerequisites](#)
- [Using The Session](#)
 - [Retrieving Data](#)
 - [Storing Data](#)
 - [Flash Data](#)
 - [Deleting Data](#)
 - [Regenerating The Session ID](#)
- [Session Blocking](#)
- [Adding Custom Session Drivers](#)
 - [Implementing The Driver](#)
 - [Registering The Driver](#)

Introduction

Since HTTP driven applications are stateless, sessions provide a way to store information about the user across multiple requests. Laravel ships with a variety of session backends that are accessed through an expressive, unified API. Support for popular backends such as [Memcached](#), [Redis](#), and databases is included out of the box.

Configuration

The session configuration file is stored at `config/session.php`. Be sure to review the options available to you in this file. By default, Laravel is configured to use the `file` session driver, which will work well for many applications.

The `session.driver` configuration option defines where session data will be stored for each request. Laravel ships with several great drivers out of the box:

- `file` - sessions are stored in `storage/framework/sessions`.
- `cookie` - sessions are stored in secure, encrypted cookies.
- `database` - sessions are stored in a relational database.
- `memcached` / `redis` - sessions are stored in one of these fast, cache based stores.
- `array` - sessions are stored in a PHP array and will not be persisted.

TIP The `array` driver is used during [testing](#) and prevents the data stored in the session from being persisted.

Driver Prerequisites

Database

When using the `database` session driver, you will need to create a table to contain the session items. Below is an example schema declaration for the table:

```
Schema::create('sessions', function ($table) {
    $table->string('id')->unique();
    $table->foreignId('user_id')->nullable();
    $table->string('ip_address', 45)->nullable();
    $table->text('user_agent')->nullable();
    $table->text('payload');
    $table->integer('last_activity');
});
```

You may use the `session:table` Artisan command to generate this migration:

```
php artisan session:table
```

```
php artisan migrate
```

Redis

Before using Redis sessions with Laravel, you will need to either install the PhpRedis PHP extension via PECL or install the `redis/redis` package (~1.0) via Composer. For more information on configuring Redis, consult its [Laravel documentation page](#).

TIP In the session configuration file, the `connection` option may be used to specify which Redis connection is used by the session.

Using The Session

Retrieving Data

There are two primary ways of working with session data in Laravel: the global session helper and via a Request instance. First, let's look at accessing the session via a Request instance, which can be type-hinted on a controller method. Remember, controller method dependencies are automatically injected via the Laravel [service container](#):

```
<?php

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;
use Illuminate\Http\Request;

class UserController extends Controller
{
    /**
     * Show the profile for the given user.
     *
     * @param Request $request
     * @param int $id
     * @return Response
     */
    public function show(Request $request, $id)
    {
        $value = $request->session()->get('key');

        //
    }
}
```

When you retrieve an item from the session, you may also pass a default value as the second argument to the `get` method. This default value will be returned if the specified key does not exist in the session. If you pass a closure as the default value to the `get` method and the requested key does not exist, the closure will be executed and its result returned:

```
$value = $request->session()->get('key', 'default');

$value = $request->session()->get('key', function () {
    return 'default';
});
```

The Global Session Helper

You may also use the global session PHP function to retrieve and store data in the session. When the session helper is called with a single, string argument, it will return the value of that session key. When the helper is called with an array of key / value pairs, those values will be stored in the session:

```
Route::get('home', function () {
    // Retrieve a piece of data from the session...
    $value = session('key');

    // Specifying a default value...
    $value = session('key', 'default');
```

```
// Store a piece of data in the session...
session(['key' => 'value']);
});
```

TIP There is little practical difference between using the session via an HTTP request instance versus using the global session helper. Both methods are [testable](#) via the `assertSessionHas` method which is available in all of your test cases.

Retrieving All Session Data

If you would like to retrieve all the data in the session, you may use the `all` method:

```
$data = $request->session()->all();
```

Determining If An Item Exists In The Session

To determine if an item is present in the session, you may use the `has` method. The `has` method returns `true` if the item is present and is not `null`:

```
if ($request->session()->has('users')) {
    //
}
```

To determine if an item is present in the session, even if its value is `null`, you may use the `exists` method. The `exists` method returns `true` if the item is present:

```
if ($request->session()->exists('users')) {
    //
}
```

Storing Data

To store data in the session, you will typically use the `put` method or the session helper:

```
// Via a request instance...
$request->session()->put('key', 'value');

// Via the global helper...
session(['key' => 'value']);
```

Pushing To Array Session Values

The `push` method may be used to push a new value onto a session value that is an array. For example, if the `user.teams` key contains an array of team names, you may push a new value onto the array like so:

```
$request->session()->push('user.teams', 'developers');
```

Retrieving & Deleting An Item

The `pull` method will retrieve and delete an item from the session in a single statement:

```
$value = $request->session()->pull('key', 'default');
```

Flash Data

Sometimes you may wish to store items in the session only for the next request. You may do so using the `flash` method. Data stored in the session using this method will be available immediately and during the subsequent HTTP request. After the subsequent HTTP request, the flashed data will be deleted. Flash data is primarily useful for short-lived status messages:

```
$request->session()->flash('status', 'Task was successful!');
```

If you need to keep your flash data around for several requests, you may use the `reflash` method, which will keep all of the flash data for an additional request. If you only need to keep specific flash data, you may use the

keep method:

```
$request->session()->reflash();  
$request->session()->keep(['username', 'email']);
```

Deleting Data

The `forget` method will remove a piece of data from the session. If you would like to remove all data from the session, you may use the `flush` method:

```
// Forget a single key...  
$request->session()->forget('key');  
  
// Forget multiple keys...  
$request->session()->forget(['key1', 'key2']);  
  
$request->session()->flush();
```

Regenerating The Session ID

Regenerating the session ID is often done in order to prevent malicious users from exploiting a [session fixation](#) attack on your application.

Laravel automatically regenerates the session ID during authentication if you are using the built-in `LoginController`; however, if you need to manually regenerate the session ID, you may use the `regenerate` method.

```
$request->session()->regenerate();
```

Session Blocking

NOTE To utilize session blocking, your application must be using a cache driver that supports [atomic locks](#). Currently, those cache drivers include the `memcached`, `dynamodb`, `redis`, and `database` drivers. In addition, you may not use the `cookie` session driver.

By default, Laravel allows requests using the same session to execute concurrently. So, for example, if you use a JavaScript HTTP library to make two HTTP requests to your application, they will both execute at the same time. For many applications, this is not a problem; however, session data loss can occur in a small subset of applications that make concurrent requests to two different application endpoints which both write data to the session.

To mitigate this, Laravel provides functionality that allows you to limit concurrent requests for a given session. To get started, you may simply chain the `block` method onto your route definition. In this example, an incoming request to the `/profile` endpoint would acquire a session lock. While this lock is being held, any incoming requests to the `/profile` or `/order` endpoints which share the same session ID will wait for the first request to finish executing before continuing their execution:

```
Route::post('/profile', function () {  
    //  
})->block($lockSeconds = 10, $waitSeconds = 10)  
  
Route::post('/order', function () {  
    //  
})->block($lockSeconds = 10, $waitSeconds = 10)
```

The `block` method accepts two optional arguments. The first argument accepted by the `block` method is the maximum number of seconds the session lock should be held for before it is released. Of course, if the request finishes executing before this time the lock will be released earlier.

The second argument accepted by the `block` method is the number of seconds a request should wait while attempting to obtain a session lock. A `Illuminate\Contracts\Cache\LockTimeoutException` will be thrown if the request is unable to obtain a session lock within the given number of seconds.

If neither of these arguments are passed, the lock will be obtained for a maximum of 10 seconds and requests

will wait a maximum of 10 seconds while attempting to obtain a lock:

```
Route::post('/profile', function () {  
    //  
})->block()
```

Adding Custom Session Drivers

Implementing The Driver

Your custom session driver should implement the `SessionHandlerInterface`. This interface contains just a few simple methods we need to implement. A stubbed MongoDB implementation looks something like this:

```
<?php  
  
namespace App\Extensions;  
  
class MongoSessionHandler implements \SessionHandlerInterface  
{  
    public function open($savePath, $sessionId) {}  
    public function close() {}  
    public function read($sessionId) {}  
    public function write($sessionId, $data) {}  
    public function destroy($sessionId) {}  
    public function gc($lifetime) {}  
}
```

TIP Laravel does not ship with a directory to contain your extensions. You are free to place them anywhere you like. In this example, we have created an `Extensions` directory to house the `MongoSessionHandler`.

Since the purpose of these methods is not readily understandable, let's quickly cover what each of the methods do:

- The `open` method would typically be used in file based session store systems. Since Laravel ships with a file session driver, you will almost never need to put anything in this method. You can leave it as an empty stub. It is a fact of poor interface design (which we'll discuss later) that PHP requires us to implement this method.
- The `close` method, like the `open` method, can also usually be disregarded. For most drivers, it is not needed.
- The `read` method should return the string version of the session data associated with the given `$sessionId`. There is no need to do any serialization or other encoding when retrieving or storing session data in your driver, as Laravel will perform the serialization for you.
- The `write` method should write the given `$data` string associated with the `$sessionId` to some persistent storage system, such as MongoDB, Dynamo, etc. Again, you should not perform any serialization - Laravel will have already handled that for you.
- The `destroy` method should remove the data associated with the `$sessionId` from persistent storage.
- The `gc` method should destroy all session data that is older than the given `$lifetime`, which is a UNIX timestamp. For self-expiring systems like Memcached and Redis, this method may be left empty.

Registering The Driver

Once your driver has been implemented, you are ready to register it with the framework. To add additional drivers to Laravel's session backend, you may use the `extend` method on the `Session` [facade](#). You should call the `extend` method from the `boot` method of a [service provider](#). You may do this from the existing `AppServiceProvider` or create an entirely new provider:

```
<?php  
  
namespace App\Providers;  
  
use App\Extensions\MongoSessionHandler;  
use Illuminate\Support\Facades\Session;  
use Illuminate\Support\ServiceProvider;  
  
class SessionServiceProvider extends ServiceProvider  
{  
    /**
```

```
* Register any application services.
*
* @return void
*/
public function register()
{
    //
}

/**
 * Bootstrap any application services.
 *
 * @return void
 */
public function boot()
{
    Session::extend('mongo', function ($app) {
        // Return implementation of SessionHandlerInterface...
        return new MongoSessionHandler;
    });
}
```

Once the session driver has been registered, you may use the `mongo` driver in your `config/session.php` configuration file.

The Basics

Validation

Validation

- [Introduction](#)
- [Validation Quickstart](#)
 - [Defining The Routes](#)
 - [Creating The Controller](#)
 - [Writing The Validation Logic](#)
 - [Displaying The Validation Errors](#)
 - [A Note On Optional Fields](#)
- [Form Request Validation](#)
 - [Creating Form Requests](#)
 - [Authorizing Form Requests](#)
 - [Customizing The Error Messages](#)
 - [Customizing The Validation Attributes](#)
 - [Prepare Input For Validation](#)
- [Manually Creating Validators](#)
 - [Automatic Redirection](#)
 - [Named Error Bags](#)
 - [After Validation Hook](#)
- [Working With Error Messages](#)
 - [Custom Error Messages](#)
- [Available Validation Rules](#)
- [Conditionally Adding Rules](#)
- [Validating Arrays](#)
- [Custom Validation Rules](#)
 - [Using Rule Objects](#)
 - [Using Closures](#)
 - [Using Extensions](#)
 - [Implicit Extensions](#)

Introduction

Laravel provides several different approaches to validate your application's incoming data. By default, Laravel's base controller class uses a `ValidatesRequests` trait which provides a convenient method to validate incoming HTTP requests with a variety of powerful validation rules.

Validation Quickstart

To learn about Laravel's powerful validation features, let's look at a complete example of validating a form and displaying the error messages back to the user.

Defining The Routes

First, let's assume we have the following routes defined in our `routes/web.php` file:

```
Route::get('post/create', 'PostController@create');  
Route::post('post', 'PostController@store');
```

The `GET` route will display a form for the user to create a new blog post, while the `POST` route will store the new blog post in the database.

Creating The Controller

Next, let's take a look at a simple controller that handles these routes. We'll leave the `store` method empty for

now:

```
<?php

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;
use Illuminate\Http\Request;

class PostController extends Controller
{
    /**
     * Show the form to create a new blog post.
     *
     * @return Response
     */
    public function create()
    {
        return view('post.create');
    }

    /**
     * Store a new blog post.
     *
     * @param Request $request
     * @return Response
     */
    public function store(Request $request)
    {
        // Validate and store the blog post...
    }
}
```

Writing The Validation Logic

Now we are ready to fill in our `store` method with the logic to validate the new blog post. To do this, we will use the `validate` method provided by the `Illuminate\Http\Request` object. If the validation rules pass, your code will keep executing normally; however, if validation fails, an exception will be thrown and the proper error response will automatically be sent back to the user. In the case of a traditional HTTP request, a redirect response will be generated, while a JSON response will be sent for AJAX requests.

To get a better understanding of the `validate` method, let's jump back into the `store` method:

```
/**
 * Store a new blog post.
 *
 * @param Request $request
 * @return Response
 */
public function store(Request $request)
{
    $validatedData = $request->validate([
        'title' => 'required|unique:posts|max:255',
        'body' => 'required',
    ]);

    // The blog post is valid...
}
```

As you can see, we pass the desired validation rules into the `validate` method. Again, if the validation fails, the proper response will automatically be generated. If the validation passes, our controller will continue executing normally.

Alternatively, validation rules may be specified as arrays of rules instead of a single `|` delimited string:

```
$validatedData = $request->validate([
    'title' => ['required', 'unique:posts', 'max:255'],
    'body' => ['required'],
]);
```

You may use the `validateWithBag` method to validate a request and store any error messages within a [named error bag](#):

```
$validatedData = $request->validateWithBag('post', [
```

```
'title' => ['required', 'unique:posts', 'max:255'],
'body' => ['required'],
]);
```

Stopping On First Validation Failure

Sometimes you may wish to stop running validation rules on an attribute after the first validation failure. To do so, assign the `bail` rule to the attribute:

```
$request->validate([
    'title' => 'bail|required|unique:posts|max:255',
    'body' => 'required',
]);
```

In this example, if the `unique` rule on the `title` attribute fails, the `max` rule will not be checked. Rules will be validated in the order they are assigned.

A Note On Nested Attributes

If your HTTP request contains "nested" parameters, you may specify them in your validation rules using "dot" syntax:

```
$request->validate([
    'title' => 'required|unique:posts|max:255',
    'author.name' => 'required',
    'author.description' => 'required',
]);
```

Displaying The Validation Errors

So, what if the incoming request parameters do not pass the given validation rules? As mentioned previously, Laravel will automatically redirect the user back to their previous location. In addition, all of the validation errors will automatically be [flushed to the session](#).

Again, notice that we did not have to explicitly bind the error messages to the view in our `GET` route. This is because Laravel will check for errors in the session data, and automatically bind them to the view if they are available. The `$errors` variable will be an instance of `Illuminate\Support\MessageBag`. For more information on working with this object, [check out its documentation](#).

TIP The `$errors` variable is bound to the view by the `Illuminate\View\Middleware\ShareErrorsFromSession` middleware, which is provided by the `web` middleware group. **When this middleware is applied an `$errors` variable will always be available in your views**, allowing you to conveniently assume the `$errors` variable is always defined and can be safely used.

So, in our example, the user will be redirected to our controller's `create` method when validation fails, allowing us to display the error messages in the view:

```
<!-- /resources/views/post/create.blade.php -->

<h1>Create Post</h1>

@if ($errors->any())
    <div class="alert alert-danger">
        <ul>
            @foreach ($errors->all() as $error)
                <li>{{ $error }}</li>
            @endforeach
        </ul>
    </div>
@endif

<!-- Create Post Form -->
```

The @error Directive

You may also use the `@error` [Blade](#) directive to quickly check if validation error messages exist for a given attribute. Within an `@error` directive, you may echo the `$message` variable to display the error message:

```
<!-- /resources/views/post/create.blade.php -->

<label for="title">Post Title</label>

<input id="title" type="text" class="@error('title') is-invalid @enderror">

@error('title')
    <div class="alert alert-danger">{{ $message }}</div>
@enderror
```

A Note On Optional Fields

By default, Laravel includes the `TrimStrings` and `ConvertEmptyStringsToNull` middleware in your application's global middleware stack. These middleware are listed in the stack by the `App\Http\Kernel` class. Because of this, you will often need to mark your "optional" request fields as `nullable` if you do not want the validator to consider `null` values as invalid. For example:

```
$request->validate([
    'title' => 'required|unique:posts|max:255',
    'body' => 'required',
    'publish_at' => 'nullable|date',
]);
```

In this example, we are specifying that the `publish_at` field may be either `null` or a valid date representation. If the `nullable` modifier is not added to the rule definition, the validator would consider `null` an invalid date.

AJAX Requests & Validation

In this example, we used a traditional form to send data to the application. However, many applications use AJAX requests. When using the `validate` method during an AJAX request, Laravel will not generate a redirect response. Instead, Laravel generates a JSON response containing all of the validation errors. This JSON response will be sent with a 422 HTTP status code.

Form Request Validation

Creating Form Requests

For more complex validation scenarios, you may wish to create a "form request". Form requests are custom request classes that contain validation logic. To create a form request class, use the `make:request` Artisan CLI command:

```
php artisan make:request StoreBlogPost
```

The generated class will be placed in the `app/Http/Requests` directory. If this directory does not exist, it will be created when you run the `make:request` command. Let's add a few validation rules to the `rules` method:

```
/**
 * Get the validation rules that apply to the request.
 *
 * @return array
 */
public function rules()
{
    return [
        'title' => 'required|unique:posts|max:255',
        'body' => 'required',
    ];
}
```

TIP You may type-hint any dependencies you need within the `rules` method's signature. They will automatically be resolved via the Laravel [service container](#).

So, how are the validation rules evaluated? All you need to do is type-hint the request on your controller method. The incoming form request is validated before the controller method is called, meaning you do not need to clutter your controller with any validation logic:

```
/**
```

```
* Store the incoming blog post.
*
* @param StoreBlogPost $request
* @return Response
*/
public function store(StoreBlogPost $request)
{
    // The incoming request is valid...

    // Retrieve the validated input data...
    $validated = $request->validated();
}
```

If validation fails, a redirect response will be generated to send the user back to their previous location. The errors will also be flashed to the session so they are available for display. If the request was an AJAX request, a HTTP response with a 422 status code will be returned to the user including a JSON representation of the validation errors.

Adding After Hooks To Form Requests

If you would like to add an "after" hook to a form request, you may use the `withValidator` method. This method receives the fully constructed validator, allowing you to call any of its methods before the validation rules are actually evaluated:

```
/**
 * Configure the validator instance.
 *
 * @param \Illuminate\Validation\Validator $validator
 * @return void
 */
public function withValidator($validator)
{
    $validator->after(function ($validator) {
        if ($this->somethingElseIsValid()) {
            $validator->errors()->add('field', 'Something is wrong with this field!');
        }
    });
}
```

Authorizing Form Requests

The form request class also contains an `authorize` method. Within this method, you may check if the authenticated user actually has the authority to update a given resource. For example, you may determine if a user actually owns a blog comment they are attempting to update:

```
/**
 * Determine if the user is authorized to make this request.
 *
 * @return bool
 */
public function authorize()
{
    $comment = Comment::find($this->route('comment'));

    return $comment && $this->user()->can('update', $comment);
}
```

Since all form requests extend the base Laravel request class, we may use the `user` method to access the currently authenticated user. Also note the call to the `route` method in the example above. This method grants you access to the URI parameters defined on the route being called, such as the `{comment}` parameter in the example below:

```
Route::post('comment/{comment}');
```

If the `authorize` method returns `false`, a HTTP response with a 403 status code will automatically be returned and your controller method will not execute.

If you plan to have authorization logic in another part of your application, return `true` from the `authorize` method:

```
/**
```

```
* Determine if the user is authorized to make this request.
*
* @return bool
*/
public function authorize()
{
    return true;
}
```

TIP You may type-hint any dependencies you need within the authorize method's signature. They will automatically be resolved via the Laravel [service container](#).

Customizing The Error Messages

You may customize the error messages used by the form request by overriding the `messages` method. This method should return an array of attribute / rule pairs and their corresponding error messages:

```
/**
 * Get the error messages for the defined validation rules.
 *
 * @return array
 */
public function messages()
{
    return [
        'title.required' => 'A title is required',
        'body.required' => 'A message is required',
    ];
}
```

Customizing The Validation Attributes

If you would like the `:attribute` portion of your validation message to be replaced with a custom attribute name, you may specify the custom names by overriding the `attributes` method. This method should return an array of attribute / name pairs:

```
/**
 * Get custom attributes for validator errors.
 *
 * @return array
 */
public function attributes()
{
    return [
        'email' => 'email address',
    ];
}
```

Prepare Input For Validation

If you need to sanitize any data from the request before you apply your validation rules, you can use the `prepareForValidation` method:

```
use Illuminate\Support\Str;

/**
 * Prepare the data for validation.
 *
 * @return void
 */
protected function prepareForValidation()
{
    $this->merge([
        'slug' => Str::slug($this->slug),
    ]);
}
```

Manually Creating Validators

If you do not want to use the `validate` method on the request, you may create a validator instance manually

using the validator [facade](#). The `make` method on the facade generates a new validator instance:

```
<?php

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Validator;

class PostController extends Controller
{
    /**
     * Store a new blog post.
     *
     * @param Request $request
     * @return Response
     */
    public function store(Request $request)
    {
        $validator = Validator::make($request->all(), [
            'title' => 'required|unique:posts|max:255',
            'body' => 'required',
        ]);

        if ($validator->fails()) {
            return redirect('post/create')
                ->withErrors($validator)
                ->withInput();
        }

        // Store the blog post...
    }
}
```

The first argument passed to the `make` method is the data under validation. The second argument is the validation rules that should be applied to the data.

After checking if the request validation failed, you may use the `withErrors` method to flash the error messages to the session. When using this method, the `$errors` variable will automatically be shared with your views after redirection, allowing you to easily display them back to the user. The `withErrors` method accepts a validator, a `MessageBag`, or a PHP array.

Automatic Redirection

If you would like to create a validator instance manually but still take advantage of the automatic redirection offered by the request's `validate` method, you may call the `validate` method on an existing validator instance. If validation fails, the user will automatically be redirected or, in the case of an AJAX request, a JSON response will be returned:

```
Validator::make($request->all(), [
    'title' => 'required|unique:posts|max:255',
    'body' => 'required',
])->validate();
```

You may use the `validateWithBag` method to store the error messages in a [named error bag](#) if validation fails:

```
Validator::make($request->all(), [
    'title' => 'required|unique:posts|max:255',
    'body' => 'required',
])->validateWithBag('post');
```

Named Error Bags

If you have multiple forms on a single page, you may wish to name the `MessageBag` of errors, allowing you to retrieve the error messages for a specific form. Pass a name as the second argument to `withErrors`:

```
return redirect('register')
    ->withErrors($validator, 'login');
```

You may then access the named `MessageBag` instance from the `$errors` variable:

```
{{ $errors->login->first('email') }}
```

After Validation Hook

The validator also allows you to attach callbacks to be run after validation is completed. This allows you to easily perform further validation and even add more error messages to the message collection. To get started, use the `after` method on a validator instance:

```
$validator = Validator::make(...);

$validator->after(function ($validator) {
    if ($this->somethingElseIsInvalid()) {
        $validator->errors()->add('field', 'Something is wrong with this field!');
    }
});

if ($validator->fails()) {
    //
}
```

Working With Error Messages

After calling the `errors` method on a validator instance, you will receive an `Illuminate\Support\MessageBag` instance, which has a variety of convenient methods for working with error messages. The `$errors` variable that is automatically made available to all views is also an instance of the `MessageBag` class.

Retrieving The First Error Message For A Field

To retrieve the first error message for a given field, use the `first` method:

```
$errors = $validator->errors();

echo $errors->first('email');
```

Retrieving All Error Messages For A Field

If you need to retrieve an array of all the messages for a given field, use the `get` method:

```
foreach ($errors->get('email') as $message) {
    //
}
```

If you are validating an array form field, you may retrieve all of the messages for each of the array elements using the `*` character:

```
foreach ($errors->get('attachments.*') as $message) {
    //
}
```

Retrieving All Error Messages For All Fields

To retrieve an array of all messages for all fields, use the `all` method:

```
foreach ($errors->all() as $message) {
    //
}
```

Determining If Messages Exist For A Field

The `has` method may be used to determine if any error messages exist for a given field:

```
if ($errors->has('email')) {
    //
}
```

Custom Error Messages

If needed, you may use custom error messages for validation instead of the defaults. There are several ways to specify custom messages. First, you may pass the custom messages as the third argument to the `Validator::make` method:

```
$messages = [
    'required' => 'The :attribute field is required.',
];

$validator = Validator::make($input, $rules, $messages);
```

In this example, the `:attribute` placeholder will be replaced by the actual name of the field under validation. You may also utilize other placeholders in validation messages. For example:

```
$messages = [
    'same' => 'The :attribute and :other must match.',
    'size' => 'The :attribute must be exactly :size.',
    'between' => 'The :attribute value :input is not between :min - :max.',
    'in' => 'The :attribute must be one of the following types: :values',
];
```

Specifying A Custom Message For A Given Attribute

Sometimes you may wish to specify a custom error message only for a specific field. You may do so using "dot" notation. Specify the attribute's name first, followed by the rule:

```
$messages = [
    'email.required' => 'We need to know your e-mail address!',
];
```

Specifying Custom Messages In Language Files

In most cases, you will probably specify your custom messages in a language file instead of passing them directly to the validator. To do so, add your messages to `custom` array in the `resources/lang/xx/validation.php` language file.

```
'custom' => [
    'email' => [
        'required' => 'We need to know your e-mail address!',
    ],
],
```

Specifying Custom Attribute Values

If you would like the `:attribute` portion of your validation message to be replaced with a custom attribute name, you may specify the custom name in the `attributes` array of your `resources/lang/xx/validation.php` language file:

```
'attributes' => [
    'email' => 'email address',
],
```

You may also pass the custom attributes as the fourth argument to the `Validator::make` method:

```
$customAttributes = [
    'email' => 'email address',
];

$validator = Validator::make($input, $rules, $messages, $customAttributes);
```

Specifying Custom Values In Language Files

Sometimes you may need the `:value` portion of your validation message to be replaced with a custom representation of the value. For example, consider the following rule that specifies that a credit card number is required if the `payment_type` has a value of `cc`:

```
$request->validate([
    'credit_card_number' => 'required_if:payment_type,cc'
]);
```

If this validation rule fails, it will produce the following error message:

The credit card number field is required when payment type is cc.

Instead of displaying cc as the payment type value, you may specify a custom value representation in your validation language file by defining a values array:

```
'values' => [
    'payment_type' => [
        'cc' => 'credit card'
    ],
],
```

Now if the validation rule fails it will produce the following message:

The credit card number field is required when payment type is credit card.

Available Validation Rules

Below is a list of all available validation rules and their function:

Accepted Active URL After (Date) After Or Equal (Date) Alpha Alpha Dash Alpha Numeric Array Bail Before (Date) Before Or Equal (Date) Between Boolean Confirmed Date Date Equals Date Format Different Digits Digits Between Dimensions (Image Files)	Distinct Email Ends With Exclude If Exclude Unless Exists (Database) File Filled Greater Than Greater Than Or Equal Image (File) In In Array Integer IP Address JSON Less Than Less Than Or Equal Max MIME Types MIME Type By File Extension Min Not In Not Regex Nullable	Numeric Password Present Regular Expression Required Required If Required Unless Required With Required With All Required Without Required Without All Same Size Sometimes Starts With String Timezone Unique (Database) URL UUID
---	--	---

accepted

The field under validation must be yes, on, 1, or true. This is useful for validating "Terms of Service" acceptance.

active_url

The field under validation must have a valid A or AAAA record according to the `dns_get_record` PHP function. The hostname of the provided URL is extracted using the `parse_url` PHP function before being passed to `dns_get_record`.

after:date

The field under validation must be a value after a given date. The dates will be passed into the `strtotime` PHP function:

```
'start_date' => 'required|date|after:tomorrow'
```

Instead of passing a date string to be evaluated by `strtotime`, you may specify another field to compare against the date:

```
'finish_date' => 'required|date|after:start_date'
```

after_or_equal:date

The field under validation must be a value after or equal to the given date. For more information, see the [after](#) rule.

alpha

The field under validation must be entirely alphabetic characters.

alpha_dash

The field under validation may have alpha-numeric characters, as well as dashes and underscores.

alpha_num

The field under validation must be entirely alpha-numeric characters.

array

The field under validation must be a PHP array.

bail

Stop running validation rules after the first validation failure.

before:date

The field under validation must be a value preceding the given date. The dates will be passed into the PHP `strtotime` function. In addition, like the [after](#) rule, the name of another field under validation may be supplied as the value of `date`.

before_or_equal:date

The field under validation must be a value preceding or equal to the given date. The dates will be passed into the PHP `strtotime` function. In addition, like the [after](#) rule, the name of another field under validation may be supplied as the value of `date`.

between:min,max

The field under validation must have a size between the given *min* and *max*. Strings, numerics, arrays, and files are evaluated in the same fashion as the [size](#) rule.

boolean

The field under validation must be able to be cast as a boolean. Accepted input are `true`, `false`, `1`, `0`, `"1"`, and `"0"`.

confirmed

The field under validation must have a matching field of `foo_confirmation`. For example, if the field under validation is `password`, a matching `password_confirmation` field must be present in the input.

date

The field under validation must be a valid, non-relative date according to the `strtotime` PHP function.

date_equals:date

The field under validation must be equal to the given date. The dates will be passed into the PHP `strtotime` function.

date_format:format

The field under validation must match the given *format*. You should use **either** `date` or `date_format` when

validating a field, not both. This validation rule supports all formats supported by PHP's [DateTime](#) class.

different:field

The field under validation must have a different value than *field*.

digits:value

The field under validation must be *numeric* and must have an exact length of *value*.

digits_between:min,max

The field under validation must be *numeric* and must have a length between the given *min* and *max*.

dimensions

The file under validation must be an image meeting the dimension constraints as specified by the rule's parameters:

```
'avatar' => 'dimensions:min_width=100,min_height=200'
```

Available constraints are: *min_width*, *max_width*, *min_height*, *max_height*, *width*, *height*, *ratio*.

A *ratio* constraint should be represented as width divided by height. This can be specified either by a statement like 3/2 or a float like 1.5:

```
'avatar' => 'dimensions:ratio=3/2'
```

Since this rule requires several arguments, you may use the `Rule::dimensions` method to fluently construct the rule:

```
use Illuminate\Validation\Rule;

Validator::make($data, [
    'avatar' => [
        'required',
        Rule::dimensions()->maxWidth(1000)->maxHeight(500)->ratio(3 / 2),
    ],
]);
```

distinct

When working with arrays, the field under validation must not have any duplicate values.

```
'foo.*.id' => 'distinct'
```

email

The field under validation must be formatted as an e-mail address. Under the hood, this validation rule makes use of the [egulias/email-validator](#) package for validating the email address. By default the `RFCValidation` validator is applied, but you can apply other validation styles as well:

```
'email' => 'email:rfc,dns'
```

The example above will apply the `RFCValidation` and `DNSCheckValidation` validations. Here's a full list of validation styles you can apply:

- `rfc`: `RFCValidation`
- `strict`: `NoRFCWarningsValidation`
- `dns`: `DNSCheckValidation`
- `spooof`: `SpoofCheckValidation`
- `filter`: `FilterEmailValidation`

The `filter` validator, which uses PHP's `filter_var` function under the hood, ships with Laravel and is Laravel's pre-5.8 behavior. The `dns` and `spooof` validators require the PHP `intl` extension.

`ends_with:foo,bar,...`

The field under validation must end with one of the given values.

`exclude_if:anotherfield,value`

The field under validation will be excluded from the request data returned by the `validate` and `validated` methods if the *anotherfield* field is equal to *value*.

`exclude_unless:anotherfield,value`

The field under validation will be excluded from the request data returned by the `validate` and `validated` methods unless *anotherfield*'s field is equal to *value*.

`exists:table,column`

The field under validation must exist on a given database table.

Basic Usage Of Exists Rule

```
'state' => 'exists:states'
```

If the `column` option is not specified, the field name will be used.

Specifying A Custom Column Name

```
'state' => 'exists:states,abbreviation'
```

Occasionally, you may need to specify a specific database connection to be used for the `exists` query. You can accomplish this by prepending the connection name to the table name using "dot" syntax:

```
'email' => 'exists:connection.staff,email'
```

Instead of specifying the table name directly, you may specify the Eloquent model which should be used to determine the table name:

```
'user_id' => 'exists:App\User,id'
```

If you would like to customize the query executed by the validation rule, you may use the `Rule` class to fluently define the rule. In this example, we'll also specify the validation rules as an array instead of using the `|` character to delimit them:

```
use Illuminate\Validation\Rule;

Validator::make($data, [
    'email' => [
        'required',
        Rule::exists('staff')->where(function ($query) {
            $query->where('account_id', 1);
        }),
    ],
]);
```

file

The field under validation must be a successfully uploaded file.

filled

The field under validation must not be empty when it is present.

gt:field

The field under validation must be greater than the given *field*. The two fields must be of the same type. Strings, numerics, arrays, and files are evaluated using the same conventions as the [size](#) rule.

gte:field

The field under validation must be greater than or equal to the given *field*. The two fields must be of the same type. Strings, numerics, arrays, and files are evaluated using the same conventions as the [size](#) rule.

image

The file under validation must be an image (jpeg, png, bmp, gif, svg, or webp)

in:foo,bar,...

The field under validation must be included in the given list of values. Since this rule often requires you to implode an array, the `Rule::in` method may be used to fluently construct the rule:

```
use Illuminate\Validation\Rule;

Validator::make($data, [
    'zones' => [
        'required',
        Rule::in(['first-zone', 'second-zone']),
    ],
]);
```

in_array:anotherfield.*

The field under validation must exist in *anotherfield*'s values.

integer

The field under validation must be an integer.

NOTE This validation rule does not verify that the input is of the "integer" variable type, only that the input is a string or numeric value that contains an integer.

ip

The field under validation must be an IP address.

ipv4

The field under validation must be an IPv4 address.

ipv6

The field under validation must be an IPv6 address.

json

The field under validation must be a valid JSON string.

lt:field

The field under validation must be less than the given *field*. The two fields must be of the same type. Strings, numerics, arrays, and files are evaluated using the same conventions as the [size](#) rule.

lte:field

The field under validation must be less than or equal to the given *field*. The two fields must be of the same type. Strings, numerics, arrays, and files are evaluated using the same conventions as the [size](#) rule.

max:value

The field under validation must be less than or equal to a maximum *value*. Strings, numerics, arrays, and files are evaluated in the same fashion as the [size](#) rule.

mimetypes:text/plain,...

The file under validation must match one of the given MIME types:

```
'video' => 'mimetypes:video/avi,video/mpeg,video/quicktime'
```

To determine the MIME type of the uploaded file, the file's contents will be read and the framework will attempt to guess the MIME type, which may be different from the client provided MIME type.

mimes:foo,bar,...

The file under validation must have a MIME type corresponding to one of the listed extensions.

Basic Usage Of MIME Rule

```
'photo' => 'mimes:jpeg,bmp,png'
```

Even though you only need to specify the extensions, this rule actually validates against the MIME type of the file by reading the file's contents and guessing its MIME type.

A full listing of MIME types and their corresponding extensions may be found at the following location:
<https://svn.apache.org/repos/asf/httpd/httpd/trunk/docs/conf/mime.types>

min:value

The field under validation must have a minimum *value*. Strings, numerics, arrays, and files are evaluated in the same fashion as the [size](#) rule.

not_in:foo,bar,...

The field under validation must not be included in the given list of values. The `Rule::notIn` method may be used to fluently construct the rule:

```
use Illuminate\Validation\Rule;

Validator::make($data, [
    'toppings' => [
        'required',
        Rule::notIn(['sprinkles', 'cherries']),
    ],
]);
```

not_regex:pattern

The field under validation must not match the given regular expression.

Internally, this rule uses the PHP `preg_match` function. The pattern specified should obey the same formatting required by `preg_match` and thus also include valid delimiters. For example: `'email' => 'not_regex:/^.+$/i'`.

Note: When using the `regex` / `not_regex` patterns, it may be necessary to specify rules in an array instead of using pipe delimiters, especially if the regular expression contains a pipe character.

nullable

The field under validation may be `null`. This is particularly useful when validating primitive such as strings and integers that can contain `null` values.

numeric

The field under validation must be numeric.

password

The field under validation must match the authenticated user's password. You may specify an authentication guard using the rule's first parameter:

```
'password' => 'password:api'
```

present

The field under validation must be present in the input data but can be empty.

regex:pattern

The field under validation must match the given regular expression.

Internally, this rule uses the PHP `preg_match` function. The pattern specified should obey the same formatting required by `preg_match` and thus also include valid delimiters. For example: `'email' => 'regex:/^.+@.+$/'`.

Note: When using the `regex / not_regex` patterns, it may be necessary to specify rules in an array instead of using pipe delimiters, especially if the regular expression contains a pipe character.

required

The field under validation must be present in the input data and not empty. A field is considered "empty" if one of the following conditions are true:

- The value is `null`.
- The value is an empty string.
- The value is an empty array or empty `Countable` object.
- The value is an uploaded file with no path.

required_if:anotherfield,value,...

The field under validation must be present and not empty if the *anotherfield* field is equal to any *value*.

If you would like to construct a more complex condition for the `required_if` rule, you may use the `Rule::requiredIf` method. This method accepts a boolean or a Closure. When passed a Closure, the Closure should return `true` or `false` to indicate if the field under validation is required:

```
use Illuminate\Validation\Rule;

Validator::make($request->all(), [
    'role_id' => Rule::requiredIf($request->user()->is_admin),
]);

Validator::make($request->all(), [
    'role_id' => Rule::requiredIf(function () use ($request) {
        return $request->user()->is_admin;
    }),
]);
```

required_unless:anotherfield,value,...

The field under validation must be present and not empty unless the *anotherfield* field is equal to any *value*.

required_with:foo,bar,...

The field under validation must be present and not empty *only if* any of the other specified fields are present.

required_with_all:foo,bar,...

The field under validation must be present and not empty *only if* all of the other specified fields are present.

required_without:foo,bar,...

The field under validation must be present and not empty *only when* any of the other specified fields are not present.

required_without_all:foo,bar,...

The field under validation must be present and not empty *only when* all of the other specified fields are not present.

same:field

The given *field* must match the field under validation.

size:value

The field under validation must have a size matching the given *value*. For string data, *value* corresponds to the number of characters. For numeric data, *value* corresponds to a given integer value (the attribute must also have the `numeric` or `integer` rule). For an array, *size* corresponds to the count of the array. For files, *size* corresponds to the file size in kilobytes. Let's look at some examples:

```
// Validate that a string is exactly 12 characters long...
'title' => 'size:12';

// Validate that a provided integer equals 10...
'seats' => 'integer|size:10';

// Validate that an array has exactly 5 elements...
'tags' => 'array|size:5';

// Validate that an uploaded file is exactly 512 kilobytes...
'image' => 'file|size:512';
```

starts_with:foo,bar,...

The field under validation must start with one of the given values.

string

The field under validation must be a string. If you would like to allow the field to also be `null`, you should assign the `nullable` rule to the field.

timezone

The field under validation must be a valid timezone identifier according to the `timezone_identifiers_list` PHP function.

unique:table,column,except,idColumn

The field under validation must not exist within the given database table.

Specifying A Custom Table / Column Name:

Instead of specifying the table name directly, you may specify the Eloquent model which should be used to determine the table name:

```
'email' => 'unique:App\User,email_address'
```

The `column` option may be used to specify the field's corresponding database column. If the `column` option is not specified, the field name will be used.

```
'email' => 'unique:users,email_address'
```

Custom Database Connection

Occasionally, you may need to set a custom connection for database queries made by the Validator. As seen above, setting `unique:users` as a validation rule will use the default database connection to query the database. To override this, specify the connection and the table name using "dot" syntax:

```
'email' => 'unique:connection.users,email_address'
```

Forcing A Unique Rule To Ignore A Given ID:

Sometimes, you may wish to ignore a given ID during the unique check. For example, consider an "update profile" screen that includes the user's name, e-mail address, and location. You will probably want to verify that the e-mail address is unique. However, if the user only changes the name field and not the e-mail field, you do not want a validation error to be thrown because the user is already the owner of the e-mail address.

To instruct the validator to ignore the user's ID, we'll use the `Rule` class to fluently define the rule. In this example, we'll also specify the validation rules as an array instead of using the `|` character to delimit the rules:

```
use Illuminate\Validation\Rule;

Validator::make($data, [
    'email' => [
        'required',
        Rule::unique('users')->ignore($user->id),
    ],
]);
```

NOTE You should never pass any user controlled request input into the `ignore` method. Instead, you should only pass a system generated unique ID such as an auto-incrementing ID or UUID from an Eloquent model instance. Otherwise, your application will be vulnerable to an SQL injection attack.

Instead of passing the model key's value to the `ignore` method, you may pass the entire model instance. Laravel will automatically extract the key from the model:

```
Rule::unique('users')->ignore($user)
```

If your table uses a primary key column name other than `id`, you may specify the name of the column when calling the `ignore` method:

```
Rule::unique('users')->ignore($user->id, 'user_id')
```

By default, the `unique` rule will check the uniqueness of the column matching the name of the attribute being validated. However, you may pass a different column name as the second argument to the `unique` method:

```
Rule::unique('users', 'email_address')->ignore($user->id),
```

Adding Additional Where Clauses:

You may also specify additional query constraints by customizing the query using the `where` method. For example, let's add a constraint that verifies the `account_id` is 1:

```
'email' => Rule::unique('users')->where(function ($query) {
    return $query->where('account_id', 1);
})
```

url

The field under validation must be a valid URL.

uuid

The field under validation must be a valid RFC 4122 (version 1, 3, 4, or 5) universally unique identifier (UUID).

Conditionally Adding Rules

Skipping Validation When Fields Have Certain Values

You may occasionally wish to not validate a given field if another field has a given value. You may accomplish this using the `exclude_if` validation rule. In this example, the `appointment_date` and `doctor_name` fields will not be validated if the `has_appointment` field has a value of `false`:

```
$v = Validator::make($data, [
    'has_appointment' => 'required|bool',
    'appointment_date' => 'exclude_if:has_appointment,false|required|date',
    'doctor_name' => 'exclude_if:has_appointment,false|required|string',
]);
```

Alternatively, you may use the `exclude_unless` rule to not validate a given field unless another field has a given value:

```
$v = Validator::make($data, [
    'has_appointment' => 'required|bool',
    'appointment_date' => 'exclude_unless:has_appointment,true|required|date',
    'doctor_name' => 'exclude_unless:has_appointment,true|required|string',
]);
```

Validating When Present

In some situations, you may wish to run validation checks against a field **only** if that field is present in the input array. To quickly accomplish this, add the `sometimes` rule to your rule list:

```
$v = Validator::make($data, [
    'email' => 'sometimes|required|email',
]);
```

In the example above, the `email` field will only be validated if it is present in the `$data` array.

TIP If you are attempting to validate a field that should always be present but may be empty, check out [this note on optional fields](#)

Complex Conditional Validation

Sometimes you may wish to add validation rules based on more complex conditional logic. For example, you may wish to require a given field only if another field has a greater value than 100. Or, you may need two fields to have a given value only when another field is present. Adding these validation rules doesn't have to be a pain. First, create a validator instance with your *static rules* that never change:

```
$v = Validator::make($data, [
    'email' => 'required|email',
    'games' => 'required|numeric',
]);
```

Let's assume our web application is for game collectors. If a game collector registers with our application and they own more than 100 games, we want them to explain why they own so many games. For example, perhaps they run a game resale shop, or maybe they just enjoy collecting. To conditionally add this requirement, we can use the `sometimes` method on the validator instance.

```
$v->sometimes('reason', 'required|max:500', function ($input) {
    return $input->games >= 100;
});
```

The first argument passed to the `sometimes` method is the name of the field we are conditionally validating. The second argument is the rules we want to add. If the closure passed as the third argument returns `true`, the rules will be added. This method makes it a breeze to build complex conditional validations. You may even add conditional validations for several fields at once:

```
$v->sometimes(['reason', 'cost'], 'required', function ($input) {  
    return $input->games >= 100;  
});
```

TIP The `$input` parameter passed to your closure will be an instance of `Illuminate\Support\Fluent` and may be used to access your input and files.

Validating Arrays

Validating array based form input fields doesn't have to be a pain. You may use "dot notation" to validate attributes within an array. For example, if the incoming HTTP request contains a `photos[profile]` field, you may validate it like so:

```
$validator = Validator::make($request->all(), [  
    'photos.profile' => 'required|image',  
]);
```

You may also validate each element of an array. For example, to validate that each e-mail in a given array input field is unique, you may do the following:

```
$validator = Validator::make($request->all(), [  
    'person.*.email' => 'email|unique:users',  
    'person.*.first_name' => 'required_with:person.*.last_name',  
]);
```

Likewise, you may use the `*` character when specifying your validation messages in your language files, making it a breeze to use a single validation message for array based fields:

```
'custom' => [  
    'person.*.email' => [  
        'unique' => 'Each person must have a unique e-mail address',  
    ]  
],
```

Custom Validation Rules

Using Rule Objects

Laravel provides a variety of helpful validation rules; however, you may wish to specify some of your own. One method of registering custom validation rules is using rule objects. To generate a new rule object, you may use the `make:rule` Artisan command. Let's use this command to generate a rule that verifies a string is uppercase. Laravel will place the new rule in the `app/Rules` directory:

```
php artisan make:rule Uppercase
```

Once the rule has been created, we are ready to define its behavior. A rule object contains two methods: `passes` and `message`. The `passes` method receives the attribute value and name, and should return `true` or `false` depending on whether the attribute value is valid or not. The `message` method should return the validation error message that should be used when validation fails:

```
<?php  
  
namespace App\Rules;  
  
use Illuminate\Contracts\Validation\Rule;  
  
class Uppercase implements Rule  
{  
    /**  
     * Determine if the validation rule passes.  
     *  
     * @param string $attribute  
     */  
}
```

```
* @param mixed $value
* @return bool
*/
public function passes($attribute, $value)
{
    return strtoupper($value) === $value;
}

/**
 * Get the validation error message.
 *
 * @return string
 */
public function message()
{
    return 'The :attribute must be uppercase.';
}
}
```

You may call the `trans` helper from your `message` method if you would like to return an error message from your translation files:

```
/**
 * Get the validation error message.
 *
 * @return string
 */
public function message()
{
    return trans('validation.uppercase');
}
```

Once the rule has been defined, you may attach it to a validator by passing an instance of the rule object with your other validation rules:

```
use App\Rules\Uppercase;

$request->validate([
    'name' => ['required', 'string', new Uppercase],
]);
```

Using Closures

If you only need the functionality of a custom rule once throughout your application, you may use a Closure instead of a rule object. The Closure receives the attribute's name, the attribute's value, and a `$fail` callback that should be called if validation fails:

```
$validator = Validator::make($request->all(), [
    'title' => [
        'required',
        'max:255',
        function ($attribute, $value, $fail) {
            if ($value === 'foo') {
                $fail($attribute.' is invalid.');
```

Using Extensions

Another method of registering custom validation rules is using the `extend` method on the validator [facade](#). Let's use this method within a [service provider](#) to register a custom validation rule:

```
<?php

namespace App\Providers;

use Illuminate\Support\ServiceProvider;
use Illuminate\Support\Facades\Validator;

class AppServiceProvider extends ServiceProvider
{
    /**
```

```
* Register any application services.
*
* @return void
*/
public function register()
{
    //
}

/**
 * Bootstrap any application services.
 *
 * @return void
 */
public function boot()
{
    Validator::extend('foo', function ($attribute, $value, $parameters, $validator) {
        return $value == 'foo';
    });
}
```

The custom validator Closure receives four arguments: the name of the `$attribute` being validated, the `$value` of the attribute, an array of `$parameters` passed to the rule, and the `validator` instance.

You may also pass a class and method to the `extend` method instead of a Closure:

```
Validator::extend('foo', 'FooValidator@validate');
```

Defining The Error Message

You will also need to define an error message for your custom rule. You can do so either using an inline custom message array or by adding an entry in the validation language file. This message should be placed in the first level of the array, not within the custom array, which is only for attribute-specific error messages:

```
"foo" => "Your input was invalid!",
"accepted" => "The :attribute must be accepted.",
// The rest of the validation error messages...
```

When creating a custom validation rule, you may sometimes need to define custom placeholder replacements for error messages. You may do so by creating a custom Validator as described above then making a call to the `replacer` method on the validator facade. You may do this within the `boot` method of a [service provider](#):

```
/**
 * Bootstrap any application services.
 *
 * @return void
 */
public function boot()
{
    Validator::extend(...);

    Validator::replacer('foo', function ($message, $attribute, $rule, $parameters) {
        return str_replace(...);
    });
}
```

Implicit Extensions

By default, when an attribute being validated is not present or contains an empty string, normal validation rules, including custom extensions, are not run. For example, the [unique](#) rule will not be run against an empty string:

```
$rules = ['name' => 'unique:users,name'];

$input = ['name' => ''];

Validator::make($input, $rules)->passes(); // true
```

For a rule to run even when an attribute is empty, the rule must imply that the attribute is required. To create such an "implicit" extension, use the `validator::extendImplicit()` method:


```
Validator::extendImplicit('foo', function ($attribute, $value, $parameters, $validator) {  
    return $value == 'foo';  
});
```

NOTE An "implicit" extension only *implies* that the attribute is required. Whether it actually invalidates a missing or empty attribute is up to you.

Implicit Rule Objects

If you would like a rule object to run when an attribute is empty, you should implement the `Illuminate\Contracts\Validation\ImplicitRule` interface. This interface serves as a "marker interface" for the validator; therefore, it does not contain any methods you need to implement.

The Basics

Error Handling

Error Handling

- [Introduction](#)
- [Configuration](#)
- [The Exception Handler](#)
 - [Report Method](#)
 - [Render Method](#)
 - [Reportable & Renderable Exceptions](#)
- [HTTP Exceptions](#)
 - [Custom HTTP Error Pages](#)

Introduction

When you start a new Laravel project, error and exception handling is already configured for you. The `App\Exceptions\Handler` class is where all exceptions triggered by your application are logged and then rendered back to the user. We'll dive deeper into this class throughout this documentation.

Configuration

The `debug` option in your `config/app.php` configuration file determines how much information about an error is actually displayed to the user. By default, this option is set to respect the value of the `APP_DEBUG` environment variable, which is stored in your `.env` file.

For local development, you should set the `APP_DEBUG` environment variable to `true`. In your production environment, this value should always be `false`. If the value is set to `true` in production, you risk exposing sensitive configuration values to your application's end users.

The Exception Handler

The Report Method

All exceptions are handled by the `App\Exceptions\Handler` class. This class contains two methods: `report` and `render`. We'll examine each of these methods in detail. The `report` method is used to log exceptions or send them to an external service like [Flare](#), [Bugsnag](#) or [Sentry](#). By default, the `report` method passes the exception to the base class where the exception is logged. However, you are free to log exceptions however you wish.

For example, if you need to report different types of exceptions in different ways, you may use the PHP `instanceof` comparison operator:

```
/**
 * Report or log an exception.
 *
 * This is a great spot to send exceptions to Flare, Sentry, Bugsnag, etc.
 *
 * @param \Throwable $exception
 * @return void
 */
public function report(Throwable $exception)
{
    if ($exception instanceof CustomException) {
        //
    }

    parent::report($exception);
}
```

TIP Instead of making a lot of `instanceof` checks in your `report` method, consider using [reportable exceptions](#)

Global Log Context

If available, Laravel automatically adds the current user's ID to every exception's log message as contextual data. You may define your own global contextual data by overriding the `context` method of your application's `App\Exceptions\Handler` class. This information will be included in every exception's log message written by your application:

```
/**
 * Get the default context variables for logging.
 *
 * @return array
 */
protected function context()
{
    return array_merge(parent::context(), [
        'foo' => 'bar',
    ]);
}
```

The report Helper

Sometimes you may need to report an exception but continue handling the current request. The `report` helper function allows you to quickly report an exception using your exception handler's `report` method without rendering an error page:

```
public function isValid($value)
{
    try {
        // Validate the value...
    } catch (Throwable $e) {
        report($e);

        return false;
    }
}
```

Ignoring Exceptions By Type

The `$dontReport` property of the exception handler contains an array of exception types that will not be logged. For example, exceptions resulting from 404 errors, as well as several other types of errors, are not written to your log files. You may add other exception types to this array as needed:

```
/**
 * A list of the exception types that should not be reported.
 *
 * @var array
 */
protected $dontReport = [
    \Illuminate\Auth\AuthenticationException::class,
    \Illuminate\Auth\Access\AuthorizationException::class,
    \Symfony\Component\HttpKernel\Exception\HttpException::class,
    \Illuminate\Database\Eloquent\ModelNotFoundException::class,
    \Illuminate\Validation\ValidationException::class,
];
```

The Render Method

The `render` method is responsible for converting a given exception into an HTTP response that should be sent back to the browser. By default, the exception is passed to the base class which generates a response for you. However, you are free to check the exception type or return your own custom response:

```
/**
 * Render an exception into an HTTP response.
 *
 * @param \Illuminate\Http\Request $request
 * @param \Throwable $exception
 * @return \Illuminate\Http\Response
 */
public function render($request, Throwable $exception)
{
    if ($exception instanceof CustomException) {
```

```
        return response()->view('errors.custom', [], 500);
    }

    return parent::render($request, $exception);
}
```

Reportable & Renderable Exceptions

Instead of type-checking exceptions in the exception handler's `report` and `render` methods, you may define `report` and `render` methods directly on your custom exception. When these methods exist, they will be called automatically by the framework:

```
<?php

namespace App\Exceptions;

use Exception;

class RenderException extends Exception
{
    /**
     * Report the exception.
     *
     * @return void
     */
    public function report()
    {
        //
    }

    /**
     * Render the exception into an HTTP response.
     *
     * @param \Illuminate\Http\Request $request
     * @return \Illuminate\Http\Response
     */
    public function render($request)
    {
        return response(...);
    }
}
```

TIP You may type-hint any required dependencies of the `report` method and they will automatically be injected into the method by Laravel's [service container](#).

HTTP Exceptions

Some exceptions describe HTTP error codes from the server. For example, this may be a "page not found" error (404), an "unauthorized error" (401) or even a developer generated 500 error. In order to generate such a response from anywhere in your application, you may use the `abort` helper:

```
abort(404);
```

The `abort` helper will immediately raise an exception which will be rendered by the exception handler. Optionally, you may provide the response text:

```
abort(403, 'Unauthorized action.');
```

Custom HTTP Error Pages

Laravel makes it easy to display custom error pages for various HTTP status codes. For example, if you wish to customize the error page for 404 HTTP status codes, create a `resources/views/errors/404.blade.php`. This file will be served on all 404 errors generated by your application. The views within this directory should be named to match the HTTP status code they correspond to. The `HttpException` instance raised by the `abort` function will be passed to the view as an `$exception` variable:

```
<h2>{{ $exception->getMessage() }}</h2>
```

You may publish Laravel's error page templates using the `vendor:publish` Artisan command. Once the templates

have been published, you may customize them to your liking:

```
php artisan vendor:publish --tag=laravel-errors
```

The Basics

Logging

Logging

- [Introduction](#)
- [Configuration](#)
 - [Building Log Stacks](#)
- [Writing Log Messages](#)
 - [Writing To Specific Channels](#)
- [Advanced Monolog Channel Customization](#)
 - [Customizing Monolog For Channels](#)
 - [Creating Monolog Handler Channels](#)
 - [Creating Channels Via Factories](#)

Introduction

To help you learn more about what's happening within your application, Laravel provides robust logging services that allow you to log messages to files, the system error log, and even to Slack to notify your entire team.

Under the hood, Laravel utilizes the [Monolog](#) library, which provides support for a variety of powerful log handlers. Laravel makes it a cinch to configure these handlers, allowing you to mix and match them to customize your application's log handling.

Configuration

All of the configuration for your application's logging system is housed in the `config/logging.php` configuration file. This file allows you to configure your application's log channels, so be sure to review each of the available channels and their options. We'll review a few common options below.

By default, Laravel will use the `stack` channel when logging messages. The `stack` channel is used to aggregate multiple log channels into a single channel. For more information on building stacks, check out the [documentation below](#).

Configuring The Channel Name

By default, Monolog is instantiated with a "channel name" that matches the current environment, such as `production` or `local`. To change this value, add a `name` option to your channel's configuration:

```
'stack' => [  
    'driver' => 'stack',  
    'name' => 'channel-name',  
    'channels' => ['single', 'slack'],  
],
```

Available Channel Drivers

Name	Description
<code>stack</code>	A wrapper to facilitate creating "multi-channel" channels
<code>single</code>	A single file or path based logger channel (<code>StreamHandler</code>)
<code>daily</code>	A <code>RotatingFileHandler</code> based Monolog driver which rotates daily
<code>slack</code>	A <code>SlackWebhookHandler</code> based Monolog driver
<code>papertrail</code>	A <code>SyslogUdpHandler</code> based Monolog driver
<code>syslog</code>	A <code>SyslogHandler</code> based Monolog driver
<code>errorlog</code>	A <code>ErrorLogHandler</code> based Monolog driver
<code>monolog</code>	A Monolog factory driver that may use any supported Monolog handler

`custom` A driver that calls a specified factory to create a channel

TIP Check out the documentation on [advanced channel customization](#) to learn more about the `monolog` and `custom` drivers.

Configuring The Single and Daily Channels

The `single` and `daily` channels have three optional configuration options: `bubble`, `permission`, and `locking`.

Name	Description	Default
<code>bubble</code>	Indicates if messages should bubble up to other channels after being handled	<code>true</code>
<code>permission</code>	The log file's permissions	<code>0644</code>
<code>locking</code>	Attempt to lock the log file before writing to it	<code>false</code>

Configuring The Papertrail Channel

The `papertrail` channel requires the `url` and `port` configuration options. You can obtain these values from [Papertrail](#).

Configuring The Slack Channel

The `slack` channel requires a `url` configuration option. This URL should match a URL for an [incoming webhook](#) that you have configured for your Slack team. By default, Slack will only receive logs at the `critical` level and above; however, you can adjust this in your logging configuration file.

Building Log Stacks

As previously mentioned, the `stack` driver allows you to combine multiple channels into a single log channel. To illustrate how to use log stacks, let's take a look at an example configuration that you might see in a production application:

```
'channels' => [  
  'stack' => [  
    'driver' => 'stack',  
    'channels' => ['syslog', 'slack'],  
  ],  
  
  'syslog' => [  
    'driver' => 'syslog',  
    'level' => 'debug',  
  ],  
  
  'slack' => [  
    'driver' => 'slack',  
    'url' => env('LOG_SLACK_WEBHOOK_URL'),  
    'username' => 'Laravel Log',  
    'emoji' => ':boom:',  
    'level' => 'critical',  
  ],  
],
```

Let's dissect this configuration. First, notice our `stack` channel aggregates two other channels via its `channels` option: `syslog` and `slack`. So, when logging messages, both of these channels will have the opportunity to log the message.

Log Levels

Take note of the `level` configuration option present on the `syslog` and `slack` channel configurations in the example above. This option determines the minimum "level" a message must be in order to be logged by the channel. Monolog, which powers Laravel's logging services, offers all of the log levels defined in the [RFC 5424 specification](#): **emergency**, **alert**, **critical**, **error**, **warning**, **notice**, **info**, and **debug**.

So, imagine we log a message using the `debug` method:

```
Log::debug('An informational message.');
```

Given our configuration, the `syslog` channel will write the message to the system log; however, since the error message is not **critical** or above, it will not be sent to Slack. However, if we log an emergency message, it will be sent to both the system log and Slack since the **emergency** level is above our minimum level threshold for both channels:

```
Log::emergency('The system is down!');
```

Writing Log Messages

You may write information to the logs using the `Log` [facade](#). As previously mentioned, the logger provides the eight logging levels defined in the [RFC 5424 specification](#): **emergency**, **alert**, **critical**, **error**, **warning**, **notice**, **info** and **debug**:

```
Log::emergency($message);
Log::alert($message);
Log::critical($message);
Log::error($message);
Log::warning($message);
Log::notice($message);
Log::info($message);
Log::debug($message);
```

So, you may call any of these methods to log a message for the corresponding level. By default, the message will be written to the default log channel as configured by your `config/logging.php` configuration file:

```
<?php

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;
use App\User;
use Illuminate\Support\Facades\Log;

class UserController extends Controller
{
    /**
     * Show the profile for the given user.
     *
     * @param int $id
     * @return Response
     */
    public function showProfile($id)
    {
        Log::info('Showing user profile for user: '.$id);

        return view('user.profile', ['user' => User::findOrFail($id)]);
    }
}
```

Contextual Information

An array of contextual data may also be passed to the log methods. This contextual data will be formatted and displayed with the log message:

```
Log::info('User failed to login.', ['id' => $user->id]);
```

Writing To Specific Channels

Sometimes you may wish to log a message to a channel other than your application's default channel. You may use the `channel` method on the `Log` facade to retrieve and log to any channel defined in your configuration file:

```
Log::channel('slack')->info('Something happened!');
```

If you would like to create an on-demand logging stack consisting of multiple channels, you may use the `stack` method:

```
Log::stack(['single', 'slack'])->info('Something happened!');
```


Advanced Monolog Channel Customization

Customizing Monolog For Channels

Sometimes you may need complete control over how Monolog is configured for an existing channel. For example, you may want to configure a custom Monolog `FormatterInterface` implementation for a given channel's handlers.

To get started, define a `tap` array on the channel's configuration. The `tap` array should contain a list of classes that should have an opportunity to customize (or "tap" into) the Monolog instance after it is created:

```
'single' => [
    'driver' => 'single',
    'tap' => [App\Logging\CustomizeFormatter::class],
    'path' => storage_path('logs/laravel.log'),
    'level' => 'debug',
],
```

Once you have configured the `tap` option on your channel, you're ready to define the class that will customize your Monolog instance. This class only needs a single method: `__invoke`, which receives an `Illuminate\Log\Logger` instance. The `Illuminate\Log\Logger` instance proxies all method calls to the underlying Monolog instance:

```
<?php

namespace App\Logging;

use Monolog\Formatter\LineFormatter;

class CustomizeFormatter
{
    /**
     * Customize the given logger instance.
     *
     * @param \Illuminate\Log\Logger $logger
     * @return void
     */
    public function __invoke($logger)
    {
        foreach ($logger->getHandlers() as $handler) {
            $handler->setFormatter(new LineFormatter(
                ' [%datetime%] %channel%.%level_name%: %message% %context% %extra%'
            ));
        }
    }
}
```

TIP All of your "tap" classes are resolved by the [service container](#), so any constructor dependencies they require will automatically be injected.

Creating Monolog Handler Channels

Monolog has a variety of [available handlers](#). In some cases, the type of logger you wish to create is merely a Monolog driver with an instance of a specific handler. These channels can be created using the `monolog` driver.

When using the `monolog` driver, the `handler` configuration option is used to specify which handler will be instantiated. Optionally, any constructor parameters the handler needs may be specified using the `with` configuration option:

```
'logentries' => [
    'driver' => 'monolog',
    'handler' => MonologHandler\SyslogUdpHandler::class,
    'with' => [
        'host' => 'my.logentries.internal.datahubhost.company.com',
        'port' => '10000',
    ],
],
```

Monolog Formatters

When using the `monolog` driver, the `Monolog LineFormatter` will be used as the default formatter. However, you may customize the type of formatter passed to the handler using the `formatter` and `formatter_with` configuration options:

```
'browser' => [
    'driver' => 'monolog',
    'handler' => Monolog\Handler\BrowserConsoleHandler::class,
    'formatter' => Monolog\Formatter\HtmlFormatter::class,
    'formatter_with' => [
        'dateFormat' => 'Y-m-d',
    ],
],
```

If you are using a Monolog handler that is capable of providing its own formatter, you may set the value of the `formatter` configuration option to `default`:

```
'newrelic' => [
    'driver' => 'monolog',
    'handler' => Monolog\Handler\NewRelicHandler::class,
    'formatter' => 'default',
],
```

Creating Channels Via Factories

If you would like to define an entirely custom channel in which you have full control over Monolog's instantiation and configuration, you may specify a custom driver type in your `config/logging.php` configuration file. Your configuration should include a `via` option to point to the factory class which will be invoked to create the Monolog instance:

```
'channels' => [
    'custom' => [
        'driver' => 'custom',
        'via' => App\Logging\CreateCustomLogger::class,
    ],
],
```

Once you have configured the `custom` channel, you're ready to define the class that will create your Monolog instance. This class only needs a single method: `__invoke`, which should return the Monolog instance:

```
<?php

namespace App\Logging;

use Monolog\Logger;

class CreateCustomLogger
{
    /**
     * Create a custom Monolog instance.
     *
     * @param array $config
     * @return \Monolog\Logger
     */
    public function __invoke(array $config)
    {
        return new Logger(...);
    }
}
```

Frontend

Blade Templates

Blade Templates

- [Introduction](#)
- [Template Inheritance](#)
 - [Defining A Layout](#)
 - [Extending A Layout](#)
- [Displaying Data](#)
 - [Blade & JavaScript Frameworks](#)
- [Control Structures](#)
 - [If Statements](#)
 - [Switch Statements](#)
 - [Loops](#)
 - [The Loop Variable](#)
 - [Comments](#)
 - [PHP](#)
 - [The @once Directive](#)
- [Forms](#)
 - [CSRF Field](#)
 - [Method Field](#)
 - [Validation Errors](#)
- [Components](#)
 - [Displaying Components](#)
 - [Passing Data To Components](#)
 - [Managing Attributes](#)
 - [Slots](#)
 - [Inline Component Views](#)
 - [Anonymous Components](#)
- [Including Subviews](#)
 - [Rendering Views For Collections](#)
- [Stacks](#)
- [Service Injection](#)
- [Extending Blade](#)
 - [Custom If Statements](#)

Introduction

Blade is the simple, yet powerful templating engine provided with Laravel. Unlike other popular PHP templating engines, Blade does not restrict you from using plain PHP code in your views. In fact, all Blade views are compiled into plain PHP code and cached until they are modified, meaning Blade adds essentially zero overhead to your application. Blade view files use the `.blade.php` file extension and are typically stored in the `resources/views` directory.

Template Inheritance

Defining A Layout

Two of the primary benefits of using Blade are *template inheritance* and *sections*. To get started, let's take a look at a simple example. First, we will examine a "master" page layout. Since most web applications maintain the same general layout across various pages, it's convenient to define this layout as a single Blade view:

```
<!-- Stored in resources/views/layouts/app.blade.php -->

<html>
  <head>
    <title>App Name - @yield('title')</title>
  </head>
```

```
<body>
    @section('sidebar')
        This is the master sidebar.
    @show

    <div class="container">
        @yield('content')
    </div>
</body>
</html>
```

As you can see, this file contains typical HTML mark-up. However, take note of the `@section` and `@yield` directives. The `@section` directive, as the name implies, defines a section of content, while the `@yield` directive is used to display the contents of a given section.

Now that we have defined a layout for our application, let's define a child page that inherits the layout.

Extending A Layout

When defining a child view, use the Blade `@extends` directive to specify which layout the child view should "inherit". Views which extend a Blade layout may inject content into the layout's sections using `@section` directives. Remember, as seen in the example above, the contents of these sections will be displayed in the layout using `@yield`:

```
<!-- Stored in resources/views/child.blade.php -->

@extends('layouts.app')

@section('title', 'Page Title')

@section('sidebar')
    @@parent

    <p>This is appended to the master sidebar.</p>
@endsection

@section('content')
    <p>This is my body content.</p>
@endsection
```

In this example, the sidebar section is utilizing the `@@parent` directive to append (rather than overwriting) content to the layout's sidebar. The `@@parent` directive will be replaced by the content of the layout when the view is rendered.

TIP Contrary to the previous example, this sidebar section ends with `@endsection` instead of `@show`. The `@endsection` directive will only define a section while `@show` will define and **immediately yield** the section.

The `@yield` directive also accepts a default value as its second parameter. This value will be rendered if the section being yielded is undefined:

```
@yield('content', View::make('view.name'))
```

Blade views may be returned from routes using the global `view` helper:

```
Route::get('blade', function () {
    return view('child');
});
```

Displaying Data

You may display data passed to your Blade views by wrapping the variable in curly braces. For example, given the following route:

```
Route::get('greeting', function () {
    return view('welcome', ['name' => 'Samantha']);
});
```

You may display the contents of the `name` variable like so:

```
Hello, {{ $name }}.
```

TIP Blade `{{ }}` statements are automatically sent through PHP's `htmlspecialchars` function to prevent XSS attacks.

You are not limited to displaying the contents of the variables passed to the view. You may also echo the results of any PHP function. In fact, you can put any PHP code you wish inside of a Blade echo statement:

```
The current UNIX timestamp is {{ time() }}.
```

Displaying Unescaped Data

By default, Blade `{{ }}` statements are automatically sent through PHP's `htmlspecialchars` function to prevent XSS attacks. If you do not want your data to be escaped, you may use the following syntax:

```
Hello, {!! $name !!}.
```

NOTE Be very careful when echoing content that is supplied by users of your application. Always use the escaped, double curly brace syntax to prevent XSS attacks when displaying user supplied data.

Rendering JSON

Sometimes you may pass an array to your view with the intention of rendering it as JSON in order to initialize a JavaScript variable. For example:

```
<script>
    var app = <?php echo json_encode($array); ?>;
</script>
```

However, instead of manually calling `json_encode`, you may use the `@json` Blade directive. The `@json` directive accepts the same arguments as PHP's `json_encode` function:

```
<script>
    var app = @json($array);

    var app = @json($array, JSON_PRETTY_PRINT);
</script>
```

NOTE You should only use the `@json` directive to render existing variables as JSON. The Blade templating is based on regular expressions and attempts to pass a complex expression to the directive may cause unexpected failures.

HTML Entity Encoding

By default, Blade (and the Laravel `e` helper) will double encode HTML entities. If you would like to disable double encoding, call the `Blade::withoutDoubleEncoding` method from the `boot` method of your `AppServiceProvider`:

```
<?php

namespace App\Providers;

use Illuminate\Support\Facades\Blade;
use Illuminate\Support\ServiceProvider;

class AppServiceProvider extends ServiceProvider
{
    /**
     * Bootstrap any application services.
     *
     * @return void
     */
    public function boot()
    {
        Blade::withoutDoubleEncoding();
    }
}
```

Blade & JavaScript Frameworks

Since many JavaScript frameworks also use "curly" braces to indicate a given expression should be displayed in the browser, you may use the `@` symbol to inform the Blade rendering engine an expression should remain untouched. For example:

```
<h1>Laravel</h1>

Hello, @{{ name }}.
```

In this example, the `@` symbol will be removed by Blade; however, `{{ name }}` expression will remain untouched by the Blade engine, allowing it to instead be rendered by your JavaScript framework.

The `@` symbol may also be used to escape Blade directives:

```
{{-- Blade --}}
@@json()

<!-- HTML output -->
@json()
```

The `@verbatim` Directive

If you are displaying JavaScript variables in a large portion of your template, you may wrap the HTML in the `@verbatim` directive so that you do not have to prefix each Blade echo statement with an `@` symbol:

```
@verbatim
    <div class="container">
        Hello, {{ name }}.
    </div>
@endverbatim
```

Control Structures

In addition to template inheritance and displaying data, Blade also provides convenient shortcuts for common PHP control structures, such as conditional statements and loops. These shortcuts provide a very clean, terse way of working with PHP control structures, while also remaining familiar to their PHP counterparts.

If Statements

You may construct `if` statements using the `@if`, `@elseif`, `@else`, and `@endif` directives. These directives function identically to their PHP counterparts:

```
@if (count($records) === 1)
    I have one record!
@elseif (count($records) > 1)
    I have multiple records!
@else
    I don't have any records!
@endif
```

For convenience, Blade also provides an `@unless` directive:

```
@unless (Auth::check())
    You are not signed in.
@endunless
```

In addition to the conditional directives already discussed, the `@isset` and `@empty` directives may be used as convenient shortcuts for their respective PHP functions:

```
@isset($records)
    // $records is defined and is not null...
@endisset

@empty($records)
    // $records is "empty"...
@endempty
```

Authentication Directives

The `@auth` and `@guest` directives may be used to quickly determine if the current user is authenticated or is a guest:

```
@auth
    // The user is authenticated...
@endauth

@guest
    // The user is not authenticated...
@endguest
```

If needed, you may specify the [authentication guard](#) that should be checked when using the `@auth` and `@guest` directives:

```
@auth('admin')
    // The user is authenticated...
@endauth

@guest('admin')
    // The user is not authenticated...
@endguest
```

Section Directives

You may check if a section has content using the `@hasSection` directive:

```
@hasSection('navigation')
    <div class="pull-right">
        @yield('navigation')
    </div>

    <div class="clearfix"></div>
@endif
```

You may use the `sectionMissing` directive to determine if a section does not have content:

```
@sectionMissing('navigation')
    <div class="pull-right">
        @include('default-navigation')
    </div>
@endif
```

Environment Directives

You may check if the application is running in the production environment using the `@production` directive:

```
@production
    // Production specific content...
@endproduction
```

Or, you may determine if the application is running in a specific environment using the `@env` directive:

```
@env('staging')
    // The application is running in "staging"...
@endenv

@env(['staging', 'production'])
    // The application is running in "staging" or "production"...
@endenv
```

Switch Statements

Switch statements can be constructed using the `@switch`, `@case`, `@break`, `@default` and `@endswitch` directives:

```
@switch($i)
    @case(1)
        First case...
    @break
```

```
@case(2)
    Second case...
    @break

@default
    Default case...
@endswitch
```

Loops

In addition to conditional statements, Blade provides simple directives for working with PHP's loop structures. Again, each of these directives functions identically to their PHP counterparts:

```
@for ($i = 0; $i < 10; $i++)
    The current value is {{ $i }}
@endfor

@foreach ($users as $user)
    <p>This is user {{ $user->id }}</p>
@endforeach

@forelse ($users as $user)
    <li>{{ $user->name }}</li>
@empty
    <p>No users</p>
@endforelse

@while (true)
    <p>I'm looping forever.</p>
@endwhile
```

TIP When looping, you may use the [loop variable](#) to gain valuable information about the loop, such as whether you are in the first or last iteration through the loop.

When using loops you may also end the loop or skip the current iteration:

```
@foreach ($users as $user)
    @if ($user->type == 1)
        @continue
    @endif

    <li>{{ $user->name }}</li>

    @if ($user->number == 5)
        @break
    @endif
@endforeach
```

You may also include the condition with the directive declaration in one line:

```
@foreach ($users as $user)
    @continue($user->type == 1)

    <li>{{ $user->name }}</li>

    @break($user->number == 5)
@endforeach
```

The Loop Variable

When looping, a `$loop` variable will be available inside of your loop. This variable provides access to some useful bits of information such as the current loop index and whether this is the first or last iteration through the loop:

```
@foreach ($users as $user)
    @if ($loop->first)
        This is the first iteration.
    @endif

    @if ($loop->last)
        This is the last iteration.
    @endif

    <p>This is user {{ $user->id }}</p>
```



```
@endforeach
```

If you are in a nested loop, you may access the parent loop's `$loop` variable via the `parent` property:

```
@foreach ($users as $user)
    @foreach ($user->posts as $post)
        @if ($loop->parent->first)
            This is first iteration of the parent loop.
        @endif
    @endforeach
@endforeach
```

The `$loop` variable also contains a variety of other useful properties:

Property	Description
<code>\$loop->index</code>	The index of the current loop iteration (starts at 0).
<code>\$loop->iteration</code>	The current loop iteration (starts at 1).
<code>\$loop->remaining</code>	The iterations remaining in the loop.
<code>\$loop->count</code>	The total number of items in the array being iterated.
<code>\$loop->first</code>	Whether this is the first iteration through the loop.
<code>\$loop->last</code>	Whether this is the last iteration through the loop.
<code>\$loop->even</code>	Whether this is an even iteration through the loop.
<code>\$loop->odd</code>	Whether this is an odd iteration through the loop.
<code>\$loop->depth</code>	The nesting level of the current loop.
<code>\$loop->parent</code>	When in a nested loop, the parent's loop variable.

Comments

Blade also allows you to define comments in your views. However, unlike HTML comments, Blade comments are not included in the HTML returned by your application:

```
{{-- This comment will not be present in the rendered HTML --}}
```

PHP

In some situations, it's useful to embed PHP code into your views. You can use the Blade `@php` directive to execute a block of plain PHP within your template:

```
@php
    //
@endphp
```

TIP While Blade provides this feature, using it frequently may be a signal that you have too much logic embedded within your template.

The @once Directive

The `@once` directive allows you to define a portion of the template that will only be evaluated once per rendering cycle. This may be useful for pushing a given piece of JavaScript into the page's header using [stacks](#). For example, if you are rendering a given [component](#) within a loop, you may wish to only push the JavaScript to the header the first time the component is rendered:

```
@once
    @push('scripts')
        <script>
            // Your custom JavaScript...
        </script>
    @endpush
@endonce
```

Forms

CSRF Field

Anytime you define an HTML form in your application, you should include a hidden CSRF token field in the form so that [the CSRF protection](#) middleware can validate the request. You may use the `@csrf` Blade directive to generate the token field:

```
<form method="POST" action="/profile">
    @csrf

    ...
</form>
```

Method Field

Since HTML forms can't make PUT, PATCH, or DELETE requests, you will need to add a hidden `_method` field to spoof these HTTP verbs. The `@method` Blade directive can create this field for you:

```
<form action="/foo/bar" method="POST">
    @method('PUT')

    ...
</form>
```

Validation Errors

The `@error` directive may be used to quickly check if [validation error messages](#) exist for a given attribute. Within an `@error` directive, you may echo the `$message` variable to display the error message:

```
<!-- /resources/views/post/create.blade.php -->

<label for="title">Post Title</label>

<input id="title" type="text" class="@error('title') is-invalid @enderror">

@error('title')
    <div class="alert alert-danger">{{ $message }}</div>
@enderror
```

You may pass [the name of a specific error bag](#) as the second parameter to the `@error` directive to retrieve validation error messages on pages containing multiple forms:

```
<!-- /resources/views/auth.blade.php -->

<label for="email">Email address</label>

<input id="email" type="email" class="@error('email', 'login') is-invalid @enderror">

@error('email', 'login')
    <div class="alert alert-danger">{{ $message }}</div>
@enderror
```

Components

Components and slots provide similar benefits to sections and layouts; however, some may find the mental model of components and slots easier to understand. There are two approaches to writing components: class based components and anonymous components.

To create a class based component, you may use the `make:component` Artisan command. To illustrate how to use components, we will create a simple `Alert` component. The `make:component` command will place the component in the `app/View/Components` directory:

```
php artisan make:component Alert
```

The `make:component` command will also create a view template for the component. The view will be placed in the `resources/views/components` directory.

Manually Registering Package Components

When writing components for your own application, components are automatically discovered within the

`app/View/Components` directory and `resources/views/components` directory.

However, if you are building a package that utilizes Blade components, you will need to manually register your component class and its HTML tag alias. You should typically register your components in the `boot` method of your package's service provider:

```
use Illuminate\Support\Facades\Blade;

/**
 * Bootstrap your package's services.
 */
public function boot()
{
    Blade::component('package-alert', AlertComponent::class);
}
```

Once your component has been registered, it may be rendered using its tag alias:

```
<x-package-alert/>
```

Displaying Components

To display a component, you may use a Blade component tag within one of your Blade templates. Blade component tags start with the string `x-` followed by the kebab case name of the component class:

```
<x-alert/>

<x-user-profile/>
```

If the component class is nested deeper within the `app/View/Components` directory, you may use the `.` character to indicate directory nesting. For example, if we assume a component is located at `App/View/Components/Inputs/Button.php`, we may render it like so:

```
<x-inputs.button/>
```

Passing Data To Components

You may pass data to Blade components using HTML attributes. Hard-coded, primitive values may be passed to the component using simple HTML attributes. PHP expressions and variables should be passed to the component via attributes that are prefixed with `::`:

```
<x-alert type="error" :message="$message"/>
```

You should define the component's required data in its class constructor. All public properties on a component will automatically be made available to the component's view. It is not necessary to pass the data to the view from the component's render method:

```
<?php

namespace App\View\Components;

use Illuminate\View\Component;

class Alert extends Component
{
    /**
     * The alert type.
     *
     * @var string
     */
    public $type;

    /**
     * The alert message.
     *
     * @var string
     */
    public $message;

    /**
     * Create the component instance.
     */
}
```

```
*
* @param string $type
* @param string $message
* @return void
*/
public function __construct($type, $message)
{
    $this->type = $type;
    $this->message = $message;
}

/**
 * Get the view / contents that represent the component.
 *
 * @return \Illuminate\View\View|\Closure|string
 */
public function render()
{
    return view('components.alert');
}
}
```

When your component is rendered, you may display the contents of your component's public variables by echoing the variables by name:

```
<div class="alert alert-{{ $type }}">
    {{ $message }}
</div>
```

Casing

Component constructor arguments should be specified using `camelCase`, while `kebab-case` should be used when referencing the argument names in your HTML attributes. For example, given the following component constructor:

```
/**
 * Create the component instance.
 *
 * @param string $alertType
 * @return void
 */
public function __construct($alertType)
{
    $this->alertType = $alertType;
}
}
```

The `$alertType` argument may be provided like so:

```
<x-alert alert-type="danger" />
```

Component Methods

In addition to public variables being available to your component template, any public methods on the component may also be executed. For example, imagine a component that has a `isSelected` method:

```
/**
 * Determine if the given option is the current selected option.
 *
 * @param string $option
 * @return bool
 */
public function isSelected($option)
{
    return $option === $this->selected;
}
}
```

You may execute this method from your component template by invoking the variable matching the name of the method:

```
<option {{ $isSelected($value) ? 'selected="selected"' : '' }} value="{{ $value }}">
    {{ $label }}
</option>
```

Using Attributes & Slots Inside The Class

Blade components also allow you to access the component name, attributes, and slot inside the class's render method. However, in order to access this data, you should return a Closure from your component's render method. The Closure will receive a `$data` array as its only argument:

```
/**
 * Get the view / contents that represent the component.
 *
 * @return \Illuminate\View\View|\Closure|string
 */
public function render()
{
    return function (array $data) {
        // $data['componentName'];
        // $data['attributes'];
        // $data['slot'];

        return '<div>Component content</div>';
    };
}
```

The `componentName` is equal to the name used in the HTML tag after the `x-` prefix. So `<x-alert />`'s `componentName` will be `alert`. The `attributes` element will contain all of the attributes that were present on the HTML tag. The `slot` element is a `Illuminate\Support\HtmlString` instance with the contents of the slot from the component.

Additional Dependencies

If your component requires dependencies from Laravel's [service container](#), you may list them before any of the component's data attributes and they will automatically be injected by the container:

```
use App\AlertCreator

/**
 * Create the component instance.
 *
 * @param \App\AlertCreator $creator
 * @param string $type
 * @param string $message
 * @return void
 */
public function __construct(AlertCreator $creator, $type, $message)
{
    $this->creator = $creator;
    $this->type = $type;
    $this->message = $message;
}
```

Managing Attributes

We've already examined how to pass data attributes to a component; however, sometimes you may need to specify additional HTML attributes, such as `class`, that are not part of the data required for a component to function. Typically, you want to pass these additional attributes down to the root element of the component template. For example, imagine we want to render an alert component like so:

```
<x-alert type="error" :message="$message" class="mt-4"/>
```

All of the attributes that are not part of the component's constructor will automatically be added to the component's "attribute bag". This attribute bag is automatically made available to the component via the `$attributes` variable. All of the attributes may be rendered within the component by echoing this variable:

```
<div {{ $attributes }}>
    <!-- Component Content -->
</div>
```

NOTE Echoing variables (`{{ $attributes }}`) or using directives such as `@env` directly on a component is not supported at this time.

Default / Merged Attributes

Sometimes you may need to specify default values for attributes or merge additional values into some of the component's attributes. To accomplish this, you may use the attribute bag's `merge` method:

```
<div {{ $attributes->merge(['class' => 'alert alert-'. $type]) }}>
    {{ $message }}
</div>
```

If we assume this component is utilized like so:

```
<x-alert type="error" :message="$message" class="mb-4"/>
```

The final, rendered HTML of the component will appear like the following:

```
<div class="alert alert-error mb-4">
    <!-- Contents of the $message variable -->
</div>
```

Filtering Attributes

You may filter attributes using the `filter` method. This method accepts a Closure which should return `true` if you wish to retain the attribute in the attribute bag:

```
{{ $attributes->filter(fn ($value, $key) => $key == 'foo') }}
```

For convenience, you may use the `whereStartsWith` method to retrieve all attributes whose keys begin with a given string:

```
{{ $attributes->whereStartsWith('wire:model') }}
```

Using the `first` method, you may render the first attribute in a given attribute bag:

```
{{ $attributes->whereStartsWith('wire:model')->first() }}
```

Slots

Often, you will need to pass additional content to your component via "slots". Let's imagine that an alert component we created has the following markup:

```
<!-- /resources/views/components/alert.blade.php -->

<div class="alert alert-danger">
    {{ $slot }}
</div>
```

We may pass content to the `slot` by injecting content into the component:

```
<x-alert>
    <strong>Whoops!</strong> Something went wrong!
</x-alert>
```

Sometimes a component may need to render multiple different slots in different locations within the component. Let's modify our alert component to allow for the injection of a "title":

```
<!-- /resources/views/components/alert.blade.php -->

<span class="alert-title">{{ $title }}</span>

<div class="alert alert-danger">
    {{ $slot }}
</div>
```

You may define the content of the named slot using the `x-slot` tag. Any content not within a `x-slot` tag will be passed to the component in the `$slot` variable:

```
<x-alert>
    <x-slot name="title">
        Server Error
    </x-slot>
    <strong>Whoops!</strong> Something went wrong!
</x-alert>
```

```
        </x-slot>

        <strong>Whoops!</strong> Something went wrong!
    </x-alert>
```

Scoped Slots

If you have used a JavaScript framework such as Vue, you may be familiar with "scoped slots", which allow you to access data or methods from the component within your slot. You may achieve similar behavior in Laravel by defining public methods or properties on your component and accessing the component within your slot via the `$component` variable:

```
<x-alert>
    <x-slot name="title">
        {{ $component->formatAlert('Server Error') }}
    </x-slot>

    <strong>Whoops!</strong> Something went wrong!
</x-alert>
```

Inline Component Views

For very small components, it may feel cumbersome to manage both the component class and the component's view template. For this reason, you may return the component's markup directly from the `render` method:

```
/**
 * Get the view / contents that represent the component.
 *
 * @return \Illuminate\View\View|\Closure|string
 */
public function render()
{
    return <<<'blade'
        <div class="alert alert-danger">
            {{ $slot }}
        </div>
    >>>
    blade;
}
```

Generating Inline View Components

To create a component that renders an inline view, you may use the `inline` option when executing the `make:component` command:

```
php artisan make:component Alert --inline
```

Anonymous Components

Similar to inline components, anonymous components provide a mechanism for managing a component via a single file. However, anonymous components utilize a single view file and have no associated class. To define an anonymous component, you only need to place a Blade template within your `resources/views/components` directory. For example, assuming you have defined a component at `resources/views/components/alert.blade.php`:

```
<x-alert/>
```

You may use the `.` character to indicate if a component is nested deeper inside the `components` directory. For example, assuming the component is defined at `resources/views/components/inputs/button.blade.php`, you may render it like so:

```
<x-inputs.button/>
```

Data Properties / Attributes

Since anonymous components do not have any associated class, you may wonder how you may differentiate which data should be passed to the component as variables and which attributes should be placed in the

component's [attribute bag](#).

You may specify which attributes should be considered data variables using the `@props` directive at the top of your component's Blade template. All other attributes on the component will be available via the component's attribute bag. If you wish to give a data variable a default value, you may specify the variable's name as the array key and the default value as the array value:

```
<!-- /resources/views/components/alert.blade.php -->

@props(['type' => 'info', 'message'])

<div {{ $attributes->merge(['class' => 'alert alert-'. $type]) }}>
    {{ $message }}
</div>
```

Including Subviews

Blade's `@include` directive allows you to include a Blade view from within another view. All variables that are available to the parent view will be made available to the included view:

```
<div>
    @include('shared.errors')

    <form>
        <!-- Form Contents -->
    </form>
</div>
```

Even though the included view will inherit all data available in the parent view, you may also pass an array of extra data to the included view:

```
@include('view.name', ['some' => 'data'])
```

If you attempt to `@include` a view which does not exist, Laravel will throw an error. If you would like to include a view that may or may not be present, you should use the `@includeIf` directive:

```
@includeIf('view.name', ['some' => 'data'])
```

If you would like to `@include` a view if a given boolean expression evaluates to `true`, you may use the `@includeWhen` directive:

```
@includeWhen($boolean, 'view.name', ['some' => 'data'])
```

If you would like to `@include` a view if a given boolean expression evaluates to `false`, you may use the `@includeUnless` directive:

```
@includeUnless($boolean, 'view.name', ['some' => 'data'])
```

To include the first view that exists from a given array of views, you may use the `includeFirst` directive:

```
@includeFirst(['custom.admin', 'admin'], ['some' => 'data'])
```

NOTE You should avoid using the `__DIR__` and `__FILE__` constants in your Blade views, since they will refer to the location of the cached, compiled view.

Aliasing Includes

If your Blade includes are stored in a subdirectory, you may wish to alias them for easier access. For example, imagine a Blade include that is stored at `resources/views/includes/input.blade.php` with the following content:

```
<input type="{{ $type ?? 'text' }}">
```

You may use the `include` method to alias the include from `includes.input` to `input`. Typically, this should be done in the `boot` method of your `AppServiceProvider`:

```
use Illuminate\Support\Facades\Blade;

Blade::include('includes.input', 'input');
```


Once the include has been aliased, you may render it using the alias name as the Blade directive:

```
@input(['type' => 'email'])
```

Rendering Views For Collections

You may combine loops and includes into one line with Blade's `@each` directive:

```
@each('view.name', $jobs, 'job')
```

The first argument is the view partial to render for each element in the array or collection. The second argument is the array or collection you wish to iterate over, while the third argument is the variable name that will be assigned to the current iteration within the view. So, for example, if you are iterating over an array of `jobs`, typically you will want to access each job as a `job` variable within your view partial. The key for the current iteration will be available as the `key` variable within your view partial.

You may also pass a fourth argument to the `@each` directive. This argument determines the view that will be rendered if the given array is empty.

```
@each('view.name', $jobs, 'job', 'view.empty')
```

NOTE Views rendered via `@each` do not inherit the variables from the parent view. If the child view requires these variables, you should use `@foreach` and `@include` instead.

Stacks

Blade allows you to push to named stacks which can be rendered somewhere else in another view or layout. This can be particularly useful for specifying any JavaScript libraries required by your child views:

```
@push('scripts')
    <script src="/example.js"></script>
@endpush
```

You may push to a stack as many times as needed. To render the complete stack contents, pass the name of the stack to the `@stack` directive:

```
<head>
    <!-- Head Contents -->

    @stack('scripts')
</head>
```

If you would like to prepend content onto the beginning of a stack, you should use the `@prepend` directive:

```
@push('scripts')
    This will be second...
@endpush

// Later...

@prepend('scripts')
    This will be first...
@endprepend
```

Service Injection

The `@inject` directive may be used to retrieve a service from the Laravel [service container](#). The first argument passed to `@inject` is the name of the variable the service will be placed into, while the second argument is the class or interface name of the service you wish to resolve:

```
@inject('metrics', 'App\Services\MetricsService')

<div>
    Monthly Revenue: {{ $metrics->monthlyRevenue() }}.
</div>
```

Extending Blade

Blade allows you to define your own custom directives using the `directive` method. When the Blade compiler encounters the custom directive, it will call the provided callback with the expression that the directive contains.

The following example creates a `@datetime($var)` directive which formats a given `$var`, which should be an instance of `DateTime`:

```
<?php

namespace App\Providers;

use Illuminate\Support\Facades\Blade;
use Illuminate\Support\ServiceProvider;

class AppServiceProvider extends ServiceProvider
{
    /**
     * Register any application services.
     *
     * @return void
     */
    public function register()
    {
        //
    }

    /**
     * Bootstrap any application services.
     *
     * @return void
     */
    public function boot()
    {
        Blade::directive('datetime', function ($expression) {
            return "<?php echo ($expression)->format('m/d/Y H:i'); ?>";
        });
    }
}
```

As you can see, we will chain the `format` method onto whatever expression is passed into the directive. So, in this example, the final PHP generated by this directive will be:

```
<?php echo ($var)->format('m/d/Y H:i'); ?>
```

NOTE After updating the logic of a Blade directive, you will need to delete all of the cached Blade views. The cached Blade views may be removed using the `view:clear` Artisan command.

Custom If Statements

Programming a custom directive is sometimes more complex than necessary when defining simple, custom conditional statements. For that reason, Blade provides a `Blade::if` method which allows you to quickly define custom conditional directives using Closures. For example, let's define a custom conditional that checks the current application cloud provider. We may do this in the `boot` method of our `AppServiceProvider`:

```
use Illuminate\Support\Facades\Blade;

/**
 * Bootstrap any application services.
 *
 * @return void
 */
public function boot()
{
    Blade::if('cloud', function ($provider) {
        return config('filesystems.default') === $provider;
    });
}
```

Once the custom conditional has been defined, we can easily use it on our templates:

```
@cloud('digitalocean')
    // The application is using the digitalocean cloud provider...
@elseifcloud('aws')
    // The application is using the aws provider...
@else
    // The application is not using the digitalocean or aws environment...
@endcloud

@unlesscloud('aws')
    // The application is not using the aws environment...
@endcloud
```

Frontend

Localization

Localization

- [Introduction](#)
 - [Configuring The Locale](#)
- [Defining Translation Strings](#)
 - [Using Short Keys](#)
 - [Using Translation Strings As Keys](#)
- [Retrieving Translation Strings](#)
 - [Replacing Parameters In Translation Strings](#)
 - [Pluralization](#)
- [Overriding Package Language Files](#)

Introduction

Laravel's localization features provide a convenient way to retrieve strings in various languages, allowing you to easily support multiple languages within your application. Language strings are stored in files within the `resources/lang` directory. Within this directory there should be a subdirectory for each language supported by the application:

```
/resources
  /lang
    /en
      messages.php
    /es
      messages.php
```

All language files return an array of keyed strings. For example:

```
<?php

return [
    'welcome' => 'Welcome to our application',
];
```

NOTE For languages that differ by territory, you should name the language directories according to the ISO 15897. For example, "en_GB" should be used for British English rather than "en-gb".

Configuring The Locale

The default language for your application is stored in the `config/app.php` configuration file. You may modify this value to suit the needs of your application. You may also change the active language at runtime using the `setLocale` method on the App facade:

```
Route::get('welcome/{locale}', function ($locale) {
    if (! in_array($locale, ['en', 'es', 'fr'])) {
        abort(400);
    }

    App::setLocale($locale);

    //
});
```

You may configure a "fallback language", which will be used when the active language does not contain a given translation string. Like the default language, the fallback language is also configured in the `config/app.php` configuration file:

```
'fallback_locale' => 'en',
```

Determining The Current Locale

You may use the `getLocale` and `isLocale` methods on the App facade to determine the current locale or check if the locale is a given value:

```
$locale = App::getLocale();

if (App::isLocale('en')) {
    //
}
```

Defining Translation Strings

Using Short Keys

Typically, translation strings are stored in files within the `resources/lang` directory. Within this directory there should be a subdirectory for each language supported by the application:

```
/resources
  /lang
    /en
      messages.php
    /es
      messages.php
```

All language files return an array of keyed strings. For example:

```
<?php

// resources/lang/en/messages.php

return [
    'welcome' => 'Welcome to our application',
];
```

Using Translation Strings As Keys

For applications with heavy translation requirements, defining every string with a "short key" can become quickly confusing when referencing them in your views. For this reason, Laravel also provides support for defining translation strings using the "default" translation of the string as the key.

Translation files that use translation strings as keys are stored as JSON files in the `resources/lang` directory. For example, if your application has a Spanish translation, you should create a `resources/lang/es.json` file:

```
{
    "I love programming.": "Me encanta programar."
}
```

Retrieving Translation Strings

You may retrieve lines from language files using the `__` helper function. The `__` method accepts the file and key of the translation string as its first argument. For example, let's retrieve the `welcome` translation string from the `resources/lang/messages.php` language file:

```
echo __('messages.welcome');

echo __('I love programming.');
```

If you are using the [Blade templating engine](#), you may use the `{{ }}` syntax to echo the translation string or use the `@lang` directive:

```
{{ __('messages.welcome') }}

@lang('messages.welcome')
```

If the specified translation string does not exist, the `__` function will return the translation string key. So, using the example above, the `__` function would return `messages.welcome` if the translation string does not exist.

NOTE The `@lang` directive does not escape any output. You are **fully responsible** for escaping your own

output when using this directive.

Replacing Parameters In Translation Strings

If you wish, you may define placeholders in your translation strings. All placeholders are prefixed with a `:`. For example, you may define a welcome message with a placeholder name:

```
'welcome' => 'Welcome, :name',
```

To replace the placeholders when retrieving a translation string, pass an array of replacements as the second argument to the `__` function:

```
echo __('messages.welcome', ['name' => 'dayle']);
```

If your placeholder contains all capital letters, or only has its first letter capitalized, the translated value will be capitalized accordingly:

```
'welcome' => 'Welcome, :NAME', // Welcome, DAYLE  
'goodbye' => 'Goodbye, :Name', // Goodbye, Dayle
```

Pluralization

Pluralization is a complex problem, as different languages have a variety of complex rules for pluralization. By using a "pipe" character, you may distinguish singular and plural forms of a string:

```
'apples' => 'There is one apple|There are many apples',
```

You may even create more complex pluralization rules which specify translation strings for multiple number ranges:

```
'apples' => '{0} There are none|[1,19] There are some|[20,*] There are many',
```

After defining a translation string that has pluralization options, you may use the `trans_choice` function to retrieve the line for a given "count". In this example, since the count is greater than one, the plural form of the translation string is returned:

```
echo trans_choice('messages.apples', 10);
```

You may also define placeholder attributes in pluralization strings. These placeholders may be replaced by passing an array as the third argument to the `trans_choice` function:

```
'minutes_ago' => '{1} :value minute ago|[2,*] :value minutes ago',  
echo trans_choice('time.minutes_ago', 5, ['value' => 5]);
```

If you would like to display the integer value that was passed to the `trans_choice` function, you may use the `:count` placeholder:

```
'apples' => '{0} There are none|{1} There is one|[2,*] There are :count',
```

Overriding Package Language Files

Some packages may ship with their own language files. Instead of changing the package's core files to tweak these lines, you may override them by placing files in the `resources/lang/vendor/{package}/{locale}` directory.

So, for example, if you need to override the English translation strings in `messages.php` for a package named `skyrim/hearthfire`, you should place a language file at: `resources/lang/vendor/hearthfire/en/messages.php`. Within this file, you should only define the translation strings you wish to override. Any translation strings you don't override will still be loaded from the package's original language files.

Frontend

Frontend Scaffolding

JavaScript & CSS Scaffolding

- [Introduction](#)
- [Writing CSS](#)
- [Writing JavaScript](#)
 - [Writing Vue Components](#)
 - [Using React](#)
- [Adding Presets](#)

Introduction

While Laravel does not dictate which JavaScript or CSS pre-processors you use, it does provide a basic starting point using [Bootstrap](#), [React](#), and / or [Vue](#) that will be helpful for many applications. By default, Laravel uses [NPM](#) to install both of these frontend packages.

The Bootstrap and Vue scaffolding provided by Laravel is located in the `laravel/ui` Composer package, which may be installed using Composer:

```
composer require laravel/ui:^2.4
```

Once the `laravel/ui` package has been installed, you may install the frontend scaffolding using the `ui` Artisan command:

```
// Generate basic scaffolding...
php artisan ui bootstrap
php artisan ui vue
php artisan ui react

// Generate login / registration scaffolding...
php artisan ui bootstrap --auth
php artisan ui vue --auth
php artisan ui react --auth
```

CSS

[Laravel Mix](#) provides a clean, expressive API over compiling SASS or Less, which are extensions of plain CSS that add variables, mixins, and other powerful features that make working with CSS much more enjoyable. In this document, we will briefly discuss CSS compilation in general; however, you should consult the full [Laravel Mix documentation](#) for more information on compiling SASS or Less.

JavaScript

Laravel does not require you to use a specific JavaScript framework or library to build your applications. In fact, you don't have to use JavaScript at all. However, Laravel does include some basic scaffolding to make it easier to get started writing modern JavaScript using the [Vue](#) library. Vue provides an expressive API for building robust JavaScript applications using components. As with CSS, we may use Laravel Mix to easily compile JavaScript components into a single, browser-ready JavaScript file.

Writing CSS

After installing the `laravel/ui` Composer package and [generating the frontend scaffolding](#), Laravel's `package.json` file will include the `bootstrap` package to help you get started prototyping your application's frontend using Bootstrap. However, feel free to add or remove packages from the `package.json` file as needed for your own application. You are not required to use the Bootstrap framework to build your Laravel application - it is provided as a good starting point for those who choose to use it.

Before compiling your CSS, install your project's frontend dependencies using the [Node package manager](#)

[\(NPM\)](#):

```
npm install
```

Once the dependencies have been installed using `npm install`, you can compile your SASS files to plain CSS using [Laravel Mix](#). The `npm run dev` command will process the instructions in your `webpack.mix.js` file. Typically, your compiled CSS will be placed in the `public/css` directory:

```
npm run dev
```

The `webpack.mix.js` file included with Laravel's frontend scaffolding will compile the `resources/sass/app.scss` SASS file. This `app.scss` file imports a file of SASS variables and loads Bootstrap, which provides a good starting point for most applications. Feel free to customize the `app.scss` file however you wish or even use an entirely different pre-processor by [configuring Laravel Mix](#).

Writing JavaScript

All of the JavaScript dependencies required by your application can be found in the `package.json` file in the project's root directory. This file is similar to a `composer.json` file except it specifies JavaScript dependencies instead of PHP dependencies. You can install these dependencies using the [Node package manager \(NPM\)](#):

```
npm install
```

TIP By default, the Laravel `package.json` file includes a few packages such as `lodash` and `axios` to help you get started building your JavaScript application. Feel free to add or remove from the `package.json` file as needed for your own application.

Once the packages are installed, you can use the `npm run dev` command to [compile your assets](#). Webpack is a module bundler for modern JavaScript applications. When you run the `npm run dev` command, Webpack will execute the instructions in your `webpack.mix.js` file:

```
npm run dev
```

By default, the Laravel `webpack.mix.js` file compiles your SASS and the `resources/js/app.js` file. Within the `app.js` file you may register your Vue components or, if you prefer a different framework, configure your own JavaScript application. Your compiled JavaScript will typically be placed in the `public/js` directory.

TIP The `app.js` file will load the `resources/js/bootstrap.js` file which bootstraps and configures Vue, Axios, jQuery, and all other JavaScript dependencies. If you have additional JavaScript dependencies to configure, you may do so in this file.

Writing Vue Components

When using the `laravel/ui` package to scaffold your frontend, an `ExampleComponent.vue` Vue component will be placed in the `resources/js/components` directory. The `ExampleComponent.vue` file is an example of a [single file Vue component](#) which defines its JavaScript and HTML template in the same file. Single file components provide a very convenient approach to building JavaScript driven applications. The example component is registered in your `app.js` file:

```
Vue.component(
  'example-component',
  require('./components/ExampleComponent.vue').default
);
```

To use the component in your application, you may drop it into one of your HTML templates. For example, after running the `php artisan ui vue --auth` Artisan command to scaffold your application's authentication and registration screens, you could drop the component into the `home.blade.php` Blade template:

```
@extends('layouts.app')

@section('content')
    <example-component></example-component>
@endsection
```

TIP Remember, you should run the `npm run dev` command each time you change a Vue component. Or,

you may run the `npm run watch` command to monitor and automatically recompile your components each time they are modified.

If you are interested in learning more about writing Vue components, you should read the [Vue documentation](#), which provides a thorough, easy-to-read overview of the entire Vue framework.

Using React

If you prefer to use React to build your JavaScript application, Laravel makes it a cinch to swap the Vue scaffolding with React scaffolding:

```
composer require laravel/ui

// Generate basic scaffolding...
php artisan ui react

// Generate login / registration scaffolding...
php artisan ui react --auth
```

Adding Presets

Presets are "macroable", which allows you to add additional methods to the `UiCommand` class at runtime. For example, the following code adds a `nextjs` method to the `UiCommand` class. Typically, you should declare preset macros in a [service provider](#):

```
use Laravel\Ui\UiCommand;

UiCommand::macro('nextjs', function (UiCommand $command) {
    // Scaffold your frontend...
});
```

Then, you may call the new preset via the `ui` command:

```
php artisan ui nextjs
```

Frontend

Compiling Assets

Compiling Assets (Mix)

- [Introduction](#)
- [Installation & Setup](#)
- [Running Mix](#)
- [Working With Stylesheets](#)
 - [Less](#)
 - [Sass](#)
 - [Stylus](#)
 - [PostCSS](#)
 - [Plain CSS](#)
 - [URL Processing](#)
 - [Source Maps](#)
- [Working With JavaScript](#)
 - [Vendor Extraction](#)
 - [React](#)
 - [Vanilla JS](#)
 - [Custom Webpack Configuration](#)
- [Copying Files & Directories](#)
- [Versioning / Cache Busting](#)
- [Browsersync Reloading](#)
- [Environment Variables](#)
- [Notifications](#)

Introduction

[Laravel Mix](#) provides a fluent API for defining Webpack build steps for your Laravel application using several common CSS and JavaScript pre-processors. Through simple method chaining, you can fluently define your asset pipeline. For example:

```
mix.js('resources/js/app.js', 'public/js')
    .sass('resources/sass/app.scss', 'public/css');
```

If you've ever been confused and overwhelmed about getting started with Webpack and asset compilation, you will love Laravel Mix. However, you are not required to use it while developing your application; you are free to use any asset pipeline tool you wish, or even none at all.

Installation & Setup

Installing Node

Before triggering Mix, you must first ensure that Node.js and NPM are installed on your machine.

```
node -v
npm -v
```

By default, Laravel Homestead includes everything you need; however, if you aren't using Vagrant, then you can easily install the latest version of Node and NPM using simple graphical installers from [their download page](#).

Laravel Mix

The only remaining step is to install Laravel Mix. Within a fresh installation of Laravel, you'll find a `package.json` file in the root of your directory structure. The default `package.json` file includes everything you need to get started. Think of this like your `composer.json` file, except it defines Node dependencies instead of

PHP. You may install the dependencies it references by running:

```
npm install
```

Running Mix

Mix is a configuration layer on top of [Webpack](#), so to run your Mix tasks you only need to execute one of the NPM scripts that is included with the default Laravel `package.json` file:

```
// Run all Mix tasks...
npm run dev

// Run all Mix tasks and minify output...
npm run production
```

Watching Assets For Changes

The `npm run watch` command will continue running in your terminal and watch all relevant files for changes. Webpack will then automatically recompile your assets when it detects a change:

```
npm run watch
```

You may find that in certain environments Webpack isn't updating when your files change. If this is the case on your system, consider using the `watch-poll` command:

```
npm run watch-poll
```

Working With Stylesheets

The `webpack.mix.js` file is your entry point for all asset compilation. Think of it as a light configuration wrapper around Webpack. Mix tasks can be chained together to define exactly how your assets should be compiled.

Less

The `less` method may be used to compile [Less](#) into CSS. Let's compile our primary `app.less` file to `public/css/app.css`.

```
mix.less('resources/less/app.less', 'public/css');
```

Multiple calls to the `less` method may be used to compile multiple files:

```
mix.less('resources/less/app.less', 'public/css')
    .less('resources/less/admin.less', 'public/css');
```

If you wish to customize the file name of the compiled CSS, you may pass a full file path as the second argument to the `less` method:

```
mix.less('resources/less/app.less', 'public/stylesheets/styles.css');
```

If you need to override the [underlying Less plug-in options](#), you may pass an object as the third argument to `mix.less()`:

```
mix.less('resources/less/app.less', 'public/css', {
    strictMath: true
});
```

Sass

The `sass` method allows you to compile [Sass](#) into CSS. You may use the method like so:

```
mix.sass('resources/sass/app.scss', 'public/css');
```

Again, like the `less` method, you may compile multiple Sass files into their own respective CSS files and even customize the output directory of the resulting CSS:

```
mix.sass('resources/sass/app.sass', 'public/css')
    .sass('resources/sass/admin.sass', 'public/css/admin');
```

Additional [Node-Sass plug-in options](#) may be provided as the third argument:

```
mix.sass('resources/sass/app.sass', 'public/css', {
    precision: 5
});
```

Stylus

Similar to Less and Sass, the `stylus` method allows you to compile [Stylus](#) into CSS:

```
mix.stylus('resources/stylus/app.styl', 'public/css');
```

You may also install additional Stylus plug-ins, such as [Rupture](#). First, install the plug-in in question through NPM (`npm install rupture`) and then require it in your call to `mix.stylus()`:

```
mix.stylus('resources/stylus/app.styl', 'public/css', {
    use: [
        require('rupture')()
    ]
});
```

PostCSS

[PostCSS](#), a powerful tool for transforming your CSS, is included with Laravel Mix out of the box. By default, Mix leverages the popular [Autoprefixer](#) plug-in to automatically apply all necessary CSS3 vendor prefixes. However, you're free to add any additional plug-ins that are appropriate for your application. First, install the desired plug-in through NPM and then reference it in your `webpack.mix.js` file:

```
mix.sass('resources/sass/app.scss', 'public/css')
    .options({
        postCss: [
            require('postcss-css-variables')()
        ]
    });
```

Plain CSS

If you would just like to concatenate some plain CSS stylesheets into a single file, you may use the `styles` method.

```
mix.styles([
    'public/css/vendor/normalize.css',
    'public/css/vendor/videojs.css'
], 'public/css/all.css');
```

URL Processing

Because Laravel Mix is built on top of Webpack, it's important to understand a few Webpack concepts. For CSS compilation, Webpack will rewrite and optimize any `url()` calls within your stylesheets. While this might initially sound strange, it's an incredibly powerful piece of functionality. Imagine that we want to compile Sass that includes a relative URL to an image:

```
.example {
    background: url('../images/example.png');
}
```

NOTE Absolute paths for any given `url()` will be excluded from URL-rewriting. For example, `url('/images/thing.png')` OR `url('http://example.com/images/thing.png')` won't be modified.

By default, Laravel Mix and Webpack will find `example.png`, copy it to your `public/images` folder, and then rewrite the `url()` within your generated stylesheet. As such, your compiled CSS will be:

```
.example {
    background: url(/images/example.png?d41d8cd98f00b204e9800998ecf8427e);
}
```

```
}
```

As useful as this feature may be, it's possible that your existing folder structure is already configured in a way you like. If this is the case, you may disable `url()` rewriting like so:

```
mix.sass('resources/sass/app.scss', 'public/css')
  .options({
    processCssUrls: false
  });
```

With this addition to your `webpack.mix.js` file, Mix will no longer match any `url()` or copy assets to your public directory. In other words, the compiled CSS will look just like how you originally typed it:

```
.example {
  background: url("../images/thing.png");
}
```

Source Maps

Though disabled by default, source maps may be activated by calling the `mix.sourceMaps()` method in your `webpack.mix.js` file. Though it comes with a compile/performance cost, this will provide extra debugging information to your browser's developer tools when using compiled assets.

```
mix.js('resources/js/app.js', 'public/js')
  .sourceMaps();
```

Style Of Source Mapping

Webpack offers a variety of [source mapping styles](#). By default, Mix's source mapping style is set to `eval-source-map`, which provides a fast rebuild time. If you want to change the mapping style, you may do so using the `sourceMaps` method:

```
let productionSourceMaps = false;

mix.js('resources/js/app.js', 'public/js')
  .sourceMaps(productionSourceMaps, 'source-map');
```

Working With JavaScript

Mix provides several features to help you work with your JavaScript files, such as compiling ECMAScript 2015, module bundling, minification, and concatenating plain JavaScript files. Even better, this all works seamlessly, without requiring an ounce of custom configuration:

```
mix.js('resources/js/app.js', 'public/js');
```

With this single line of code, you may now take advantage of:

- ES2015 syntax.
- Modules
- Compilation of `.vue` files.
- Minification for production environments.

Vendor Extraction

One potential downside to bundling all application-specific JavaScript with your vendor libraries is that it makes long-term caching more difficult. For example, a single update to your application code will force the browser to re-download all of your vendor libraries even if they haven't changed.

If you intend to make frequent updates to your application's JavaScript, you should consider extracting all of your vendor libraries into their own file. This way, a change to your application code will not affect the caching of your large `vendor.js` file. Mix's `extract` method makes this a breeze:

```
mix.js('resources/js/app.js', 'public/js')
  .extract(['vue'])
```

The `extract` method accepts an array of all libraries or modules that you wish to extract into a `vendor.js` file. Using the above snippet as an example, Mix will generate the following files:

- `public/js/manifest.js`: *The Webpack manifest runtime*
- `public/js/vendor.js`: *Your vendor libraries*
- `public/js/app.js`: *Your application code*

To avoid JavaScript errors, be sure to load these files in the proper order:

```
<script src="/js/manifest.js"></script>
<script src="/js/vendor.js"></script>
<script src="/js/app.js"></script>
```

React

Mix can automatically install the Babel plug-ins necessary for React support. To get started, replace your `mix.js()` call with `mix.react()`:

```
mix.react('resources/js/app.jsx', 'public/js');
```

Behind the scenes, Mix will download and include the appropriate `babel-preset-react` Babel plug-in.

Vanilla JS

Similar to combining stylesheets with `mix.styles()`, you may also combine and minify any number of JavaScript files with the `scripts()` method:

```
mix.scripts([
    'public/js/admin.js',
    'public/js/dashboard.js'
], 'public/js/all.js');
```

This option is particularly useful for legacy projects where you don't require Webpack compilation for your JavaScript.

TIP A slight variation of `mix.scripts()` is `mix.babel()`. Its method signature is identical to `scripts`; however, the concatenated file will receive Babel compilation, which translates any ES2015 code to vanilla JavaScript that all browsers will understand.

Custom Webpack Configuration

Behind the scenes, Laravel Mix references a pre-configured `webpack.config.js` file to get you up and running as quickly as possible. Occasionally, you may need to manually modify this file. You might have a special loader or plug-in that needs to be referenced, or maybe you prefer to use Stylus instead of Sass. In such instances, you have two choices:

Merging Custom Configuration

Mix provides a useful `webpackConfig` method that allows you to merge any short Webpack configuration overrides. This is a particularly appealing choice, as it doesn't require you to copy and maintain your own copy of the `webpack.config.js` file. The `webpackConfig` method accepts an object, which should contain any [Webpack-specific configuration](#) that you wish to apply.

```
mix.webpackConfig({
    resolve: {
        modules: [
            path.resolve(__dirname, 'vendor/laravel/spark/resources/assets/js')
        ]
    }
});
```

Custom Configuration Files

If you would like to completely customize your Webpack configuration, copy the `node_modules/laravel-`

`mix/setup/webpack.config.js` file to your project's root directory. Next, point all of the `--config` references in your `package.json` file to the newly copied configuration file. If you choose to take this approach to customization, any future upstream updates to Mix's `webpack.config.js` must be manually merged into your customized file.

Copying Files & Directories

The `copy` method may be used to copy files and directories to new locations. This can be useful when a particular asset within your `node_modules` directory needs to be relocated to your `public` folder.

```
mix.copy('node_modules/foo/bar.css', 'public/css/bar.css');
```

When copying a directory, the `copy` method will flatten the directory's structure. To maintain the directory's original structure, you should use the `copyDirectory` method instead:

```
mix.copyDirectory('resources/img', 'public/img');
```

Versioning / Cache Busting

Many developers suffix their compiled assets with a timestamp or unique token to force browsers to load the fresh assets instead of serving stale copies of the code. Mix can handle this for you using the `version` method.

The `version` method will automatically append a unique hash to the filenames of all compiled files, allowing for more convenient cache busting:

```
mix.js('resources/js/app.js', 'public/js')
    .version();
```

After generating the versioned file, you won't know the exact file name. So, you should use Laravel's global `mix` function within your [views](#) to load the appropriately hashed asset. The `mix` function will automatically determine the current name of the hashed file:

```
<script src="{{ mix('/js/app.js') }}"></script>
```

Because versioned files are usually unnecessary in development, you may instruct the versioning process to only run during `npm run production`:

```
mix.js('resources/js/app.js', 'public/js');

if (mix.inProduction()) {
    mix.version();
}
```

Custom Mix Base URLs

If your Mix compiled assets are deployed to a CDN separate from your application, you will need to change the base URL generated by the `mix` function. You may do so by adding a `mix_url` configuration option to your `config/app.php` configuration file:

```
'mix_url' => env('MIX_ASSET_URL', null)
```

After configuring the Mix URL, The `mix` function will prefix the configured URL when generating URLs to assets:

```
https://cdn.example.com/js/app.js?id=1964becbdd96414518cd
```

Browsersync Reloading

[BrowserSync](#) can automatically monitor your files for changes, and inject your changes into the browser without requiring a manual refresh. You may enable support by calling the `mix.browserSync()` method:

```
mix.browserSync('my-domain.test');

// Or...
```

```
// https://browsersync.io/docs/options
mix.browserSync({
  proxy: 'my-domain.test'
});
```

You may pass either a string (proxy) or object (BrowserSync settings) to this method. Next, start Webpack's dev server using the `npm run watch` command. Now, when you modify a script or PHP file, watch as the browser instantly refreshes the page to reflect your changes.

Environment Variables

You may inject environment variables into Mix by prefixing a key in your `.env` file with `MIX_`:

```
MIX_SENTRY_DSN_PUBLIC=http://example.com
```

After the variable has been defined in your `.env` file, you may access it via the `process.env` object. If the value changes while you are running a `watch` task, you will need to restart the task:

```
process.env.MIX_SENTRY_DSN_PUBLIC
```

Notifications

When available, Mix will automatically display OS notifications for each bundle. This will give you instant feedback, as to whether the compilation was successful or not. However, there may be instances when you'd prefer to disable these notifications. One such example might be triggering Mix on your production server. Notifications may be deactivated, via the `disableNotifications` method.

```
mix.disableNotifications();
```


Security

Authentication

Authentication

- [Introduction](#)
 - [Database Considerations](#)
- [Authentication Quickstart](#)
 - [Routing](#)
 - [Views](#)
 - [Authenticating](#)
 - [Retrieving The Authenticated User](#)
 - [Protecting Routes](#)
 - [Password Confirmation](#)
 - [Login Throttling](#)
- [Manually Authenticating Users](#)
 - [Remembering Users](#)
 - [Other Authentication Methods](#)
- [HTTP Basic Authentication](#)
 - [Stateless HTTP Basic Authentication](#)
- [Logging Out](#)
 - [Invalidating Sessions On Other Devices](#)
- [Social Authentication](#)
- [Adding Custom Guards](#)
 - [Closure Request Guards](#)
- [Adding Custom User Providers](#)
 - [The User Provider Contract](#)
 - [The Authenticatable Contract](#)
- [Events](#)

Introduction

TIP Want to get started fast? Install the `laravel/ui` Composer package and run `php artisan ui vue --auth` in a fresh Laravel application. After migrating your database, navigate your browser to `http://your-app.test/register` or any other URL that is assigned to your application. These commands will take care of scaffolding your entire authentication system!

Laravel makes implementing authentication very simple. In fact, almost everything is configured for you out of the box. The authentication configuration file is located at `config/auth.php`, which contains several well documented options for tweaking the behavior of the authentication services.

At its core, Laravel's authentication facilities are made up of "guards" and "providers". Guards define how users are authenticated for each request. For example, Laravel ships with a `session` guard which maintains state using session storage and cookies.

Providers define how users are retrieved from your persistent storage. Laravel ships with support for retrieving users using Eloquent and the database query builder. However, you are free to define additional providers as needed for your application.

Don't worry if this all sounds confusing now! Many applications will never need to modify the default authentication configuration.

Database Considerations

By default, Laravel includes an `App\User` [Eloquent model](#) in your `app` directory. This model may be used with the default Eloquent authentication driver. If your application is not using Eloquent, you may use the `database` authentication driver which uses the Laravel query builder.

When building the database schema for the `App\User` model, make sure the password column is at least 60 characters in length. Maintaining the default string column length of 255 characters would be a good choice.

Also, you should verify that your users (or equivalent) table contains a nullable, string `remember_token` column of 100 characters. This column will be used to store a token for users that select the "remember me" option when logging into your application.

Authentication Quickstart

Routing

Laravel's `laravel/ui` package provides a quick way to scaffold all of the routes and views you need for authentication using a few simple commands:

```
composer require laravel/ui:^2.4

php artisan ui vue --auth
```

This command should be used on fresh applications and will install a layout view, registration and login views, as well as routes for all authentication end-points. A `HomeController` will also be generated to handle post-login requests to your application's dashboard.

The `laravel/ui` package also generates several pre-built authentication controllers, which are located in the `App\Http\Controllers\Auth` namespace. The `RegisterController` handles new user registration, the `LoginController` handles authentication, the `ForgotPasswordController` handles e-mailing links for resetting passwords, and the `ResetPasswordController` contains the logic to reset passwords. Each of these controllers uses a trait to include their necessary methods. For many applications, you will not need to modify these controllers at all.

TIP If your application doesn't need registration, you may disable it by removing the newly created `RegisterController` and modifying your route declaration: `Auth::routes(['register' => false]);`.

Creating Applications Including Authentication

If you are starting a brand new application and would like to include the authentication scaffolding, you may use the `--auth` directive when creating your application. This command will create a new application with all of the authentication scaffolding compiled and installed:

```
laravel new blog --auth
```

Views

As mentioned in the previous section, the `laravel/ui` package's `php artisan ui vue --auth` command will create all of the views you need for authentication and place them in the `resources/views/auth` directory.

The `ui` command will also create a `resources/views/layouts` directory containing a base layout for your application. All of these views use the Bootstrap CSS framework, but you are free to customize them however you wish.

Authenticating

Now that you have routes and views setup for the included authentication controllers, you are ready to register and authenticate new users for your application! You may access your application in a browser since the authentication controllers already contain the logic (via their traits) to authenticate existing users and store new users in the database.

Path Customization

When a user is successfully authenticated, they will be redirected to the `/home` URI. You can customize the post-authentication redirect path using the `HOME` constant defined in your `RouteServiceProvider`:

```
public const HOME = '/home';
```

If you need more robust customization of the response returned when a user is authenticated, Laravel provides an empty `authenticated(Request $request, $user)` method within the `AuthenticatesUsers` trait. This trait is used by the `LoginController` class that is installed into your application when using the `laravel/ui` package. Therefore, you can define your own `authenticated` method within the `LoginController` class:

```
/**
 * The user has been authenticated.
 *
 * @param \Illuminate\Http\Request $request
 * @param mixed $user
 * @return mixed
 */
protected function authenticated(Request $request, $user)
{
    return response([
        //
    ]);
}
```

Username Customization

By default, Laravel uses the `email` field for authentication. If you would like to customize this, you may define a `username` method on your `LoginController`:

```
public function username()
{
    return 'username';
}
```

Guard Customization

You may also customize the "guard" that is used to authenticate and register users. To get started, define a `guard` method on your `LoginController`, `RegisterController`, and `ResetPasswordController`. The method should return a guard instance:

```
use Illuminate\Support\Facades\Auth;

protected function guard()
{
    return Auth::guard('guard-name');
}
```

Validation / Storage Customization

To modify the form fields that are required when a new user registers with your application, or to customize how new users are stored into your database, you may modify the `RegisterController` class. This class is responsible for validating and creating new users of your application.

The `validator` method of the `RegisterController` contains the validation rules for new users of the application. You are free to modify this method as you wish.

The `create` method of the `RegisterController` is responsible for creating new `App\User` records in your database using the [Eloquent ORM](#). You are free to modify this method according to the needs of your database.

Retrieving The Authenticated User

You may access the authenticated user via the `Auth` facade:

```
use Illuminate\Support\Facades\Auth;

// Get the currently authenticated user...
$user = Auth::user();

// Get the currently authenticated user's ID...
$id = Auth::id();
```

Alternatively, once a user is authenticated, you may access the authenticated user via an `Illuminate\Http\Request` instance. Remember, type-hinted classes will automatically be injected into your controller methods:

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

class ProfileController extends Controller
{
    /**
     * Update the user's profile.
     *
     * @param Request $request
     * @return Response
     */
    public function update(Request $request)
    {
        // $request->user() returns an instance of the authenticated user...
    }
}
```

Determining If The Current User Is Authenticated

To determine if the user is already logged into your application, you may use the `check` method on the `Auth` facade, which will return `true` if the user is authenticated:

```
use Illuminate\Support\Facades\Auth;

if (Auth::check()) {
    // The user is logged in...
}
```

TIP Even though it is possible to determine if a user is authenticated using the `check` method, you will typically use a middleware to verify that the user is authenticated before allowing the user access to certain routes / controllers. To learn more about this, check out the documentation on [protecting routes](#).

Protecting Routes

[Route middleware](#) can be used to only allow authenticated users to access a given route. Laravel ships with an `auth` middleware, which is defined at `Illuminate\Auth\Middleware\Authenticate`. Since this middleware is already registered in your HTTP kernel, all you need to do is attach the middleware to a route definition:

```
Route::get('profile', function () {
    // Only authenticated users may enter...
})->middleware('auth');
```

If you are using [controllers](#), you may call the `middleware` method from the controller's constructor instead of attaching it in the route definition directly:

```
public function __construct()
{
    $this->middleware('auth');
}
```

Redirecting Unauthenticated Users

When the `auth` middleware detects an unauthorized user, it will redirect the user to the login [named route](#). You may modify this behavior by updating the `redirectTo` function in your `app/Http/Middleware/Authenticate.php` file:

```
/**
 * Get the path the user should be redirected to.
 *
 * @param \Illuminate\Http\Request $request
 * @return string
 */
protected function redirectTo($request)
```

```
{
    return route('login');
}
```

Specifying A Guard

When attaching the `auth` middleware to a route, you may also specify which guard should be used to authenticate the user. The guard specified should correspond to one of the keys in the `guards` array of your `auth.php` configuration file:

```
public function __construct()
{
    $this->middleware('auth:api');
}
```

Password Confirmation

Sometimes, you may wish to require the user to confirm their password before accessing a specific area of your application. For example, you may require this before the user modifies any billing settings within the application.

To accomplish this, Laravel provides a `password.confirm` middleware. Attaching the `password.confirm` middleware to a route will redirect users to a screen where they need to confirm their password before they can continue:

```
Route::get('/settings/security', function () {
    // Users must confirm their password before continuing...
})->middleware(['auth', 'password.confirm']);
```

After the user has successfully confirmed their password, the user is redirected to the route they originally tried to access. By default, after confirming their password, the user will not have to confirm their password again for three hours. You are free to customize the length of time before the user must re-confirm their password using the `auth.password_timeout` configuration option.

Login Throttling

If you are using the `LoginController` class that is shipped with the `laravel/ui` package, the `Illuminate\Foundation\Auth\ThrottlesLogins` trait will already be included in your controller. By default, the user will not be able to login for one minute if they fail to provide the correct credentials after several attempts. The throttling is unique to the user's username / e-mail address and their IP address.

TIP If you would like to rate limit other routes in your application, check out the [rate limiting documentation](#).

Manually Authenticating Users

Note that you are not required to use the authentication controllers included with Laravel. If you choose to remove these controllers, you will need to manage user authentication using the Laravel authentication classes directly. Don't worry, it's a cinch!

We will access Laravel's authentication services via the `Auth facade`, so we'll need to make sure to import the `Auth facade` at the top of the class. Next, let's check out the `attempt` method:

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;

class LoginController extends Controller
{
    /**
     * Handle an authentication attempt.
     */
}
```

```

    * @param \Illuminate\Http\Request $request
    *
    * @return Response
    */
    public function authenticate(Request $request)
    {
        $credentials = $request->only('email', 'password');

        if (Auth::attempt($credentials)) {
            // Authentication passed...
            return redirect()->intended('dashboard');
        }
    }
}

```

The `attempt` method accepts an array of key / value pairs as its first argument. The values in the array will be used to find the user in your database table. So, in the example above, the user will be retrieved by the value of the `email` column. If the user is found, the hashed password stored in the database will be compared with the password value passed to the method via the array. You should not hash the password specified as the `password` value, since the framework will automatically hash the value before comparing it to the hashed password in the database. If the two hashed passwords match an authenticated session will be started for the user.

The `attempt` method will return `true` if authentication was successful. Otherwise, `false` will be returned.

The `intended` method on the redirector will redirect the user to the URL they were attempting to access before being intercepted by the authentication middleware. A fallback URI may be given to this method in case the intended destination is not available.

Specifying Additional Conditions

If you wish, you may also add extra conditions to the authentication query in addition to the user's e-mail and password. For example, we may verify that user is marked as "active":

```

if (Auth::attempt(['email' => $email, 'password' => $password, 'active' => 1])) {
    // The user is active, not suspended, and exists.
}

```

NOTE In these examples, `email` is not a required option, it is merely used as an example. You should use whatever column name corresponds to a "username" in your database.

Accessing Specific Guard Instances

You may specify which guard instance you would like to utilize using the `guard` method on the `Auth` facade. This allows you to manage authentication for separate parts of your application using entirely separate authenticatable models or user tables.

The guard name passed to the `guard` method should correspond to one of the guards configured in your `auth.php` configuration file:

```

if (Auth::guard('admin')->attempt($credentials)) {
    //
}

```

Logging Out

To log users out of your application, you may use the `logout` method on the `Auth` facade. This will clear the authentication information in the user's session:

```
Auth::logout();
```

Remembering Users

If you would like to provide "remember me" functionality in your application, you may pass a boolean value as the second argument to the `attempt` method, which will keep the user authenticated indefinitely, or until they manually logout. Your `users` table must include the string `remember_token` column, which will be used to store the "remember me" token.

```
if (Auth::attempt(['email' => $email, 'password' => $password], $remember)) {  
    // The user is being remembered...  
}
```

TIP If you are using the built-in `LoginController` that is shipped with the `laravel/ui` package, the proper logic to "remember" users is already implemented by the traits used by the controller.

If you are "remembering" users, you may use the `viaRemember` method to determine if the user was authenticated using the "remember me" cookie:

```
if (Auth::viaRemember()) {  
    //  
}
```

Other Authentication Methods

Authenticate A User Instance

If you need to log an existing user instance into your application, you may call the `login` method with the user instance. The given object must be an implementation of the `Illuminate\Contracts\Auth\Authenticatable` [contract](#). The `App\User` model included with Laravel already implements this interface:

```
Auth::login($user);  
  
// Login and "remember" the given user...  
Auth::login($user, true);
```

You may specify the guard instance you would like to use:

```
Auth::guard('admin')->login($user);
```

Authenticate A User By ID

To log a user into the application by their ID, you may use the `loginUsingId` method. This method accepts the primary key of the user you wish to authenticate:

```
Auth::loginUsingId(1);  
  
// Login and "remember" the given user...  
Auth::loginUsingId(1, true);
```

Authenticate A User Once

You may use the `once` method to log a user into the application for a single request. No sessions or cookies will be utilized, which means this method may be helpful when building a stateless API:

```
if (Auth::once($credentials)) {  
    //  
}
```

HTTP Basic Authentication

[HTTP Basic Authentication](#) provides a quick way to authenticate users of your application without setting up a dedicated "login" page. To get started, attach the `auth.basic` [middleware](#) to your route. The `auth.basic` middleware is included with the Laravel framework, so you do not need to define it:

```
Route::get('profile', function () {  
    // Only authenticated users may enter...  
})->middleware('auth.basic');
```

Once the middleware has been attached to the route, you will automatically be prompted for credentials when accessing the route in your browser. By default, the `auth.basic` middleware will use the `email` column on the user record as the "username".

A Note On FastCGI

If you are using PHP FastCGI, HTTP Basic authentication may not work correctly out of the box. The following lines should be added to your `.htaccess` file:

```
RewriteCond %{HTTP:Authorization} ^(.+)$
RewriteRule .* - [E=HTTP_AUTHORIZATION:%{HTTP:Authorization}]
```

Stateless HTTP Basic Authentication

You may also use HTTP Basic Authentication without setting a user identifier cookie in the session, which is particularly useful for API authentication. To do so, [define a middleware](#) that calls the `onceBasic` method. If no response is returned by the `onceBasic` method, the request may be passed further into the application:

```
<?php

namespace App\Http\Middleware;

use Illuminate\Support\Facades\Auth;

class AuthenticateOnceWithBasicAuth
{
    /**
     * Handle an incoming request.
     *
     * @param  \Illuminate\Http\Request  $request
     * @param  \Closure  $next
     * @return mixed
     */
    public function handle($request, $next)
    {
        return Auth::onceBasic() ?: $next($request);
    }
}
```

Next, [register the route middleware](#) and attach it to a route:

```
Route::get('api/user', function () {
    // Only authenticated users may enter...
})->middleware('auth.basic.once');
```

Logging Out

To manually log users out of your application, you may use the `logout` method on the `Auth` facade. This will clear the authentication information in the user's session:

```
use Illuminate\Support\Facades\Auth;

Auth::logout();
```

Invalidating Sessions On Other Devices

Laravel also provides a mechanism for invalidating and "logging out" a user's sessions that are active on other devices without invalidating the session on their current device. This feature is typically utilized when a user is changing or updating their password and you would like to invalidate sessions on other devices while keeping the current device authenticated.

Before getting started, you should make sure that the `Illuminate\Session\Middleware\AuthenticateSession` middleware is present and un-commented in your `app/Http/Kernel.php` class' web middleware group:

```
'web' => [
    // ...
    \Illuminate\Session\Middleware\AuthenticateSession::class,
    // ...
],
```

Then, you may use the `logoutOtherDevices` method on the `Auth` facade. This method requires the user to provide their current password, which your application should accept through an input form:

```
use Illuminate\Support\Facades\Auth;
```



```
Auth::logoutOtherDevices($password);
```

When the `logoutOtherDevices` method is invoked, the user's other sessions will be invalidated entirely, meaning they will be "logged out" of all guards they were previously authenticated by.

NOTE When using the `AuthenticateSession` middleware in combination with a custom route name for the login route, you must override the `unauthenticated` method on your application's exception handler to properly redirect users to your login page.

Adding Custom Guards

You may define your own authentication guards using the `extend` method on the `Auth` facade. You should place this call to `extend` within a [service provider](#). Since Laravel already ships with an `AuthServiceProvider`, we can place the code in that provider:

```
<?php

namespace App\Providers;

use App\Services\Auth\JwtGuard;
use Illuminate\Foundation\Support\Providers\AuthServiceProvider as ServiceProvider;
use Illuminate\Support\Facades\Auth;

class AuthServiceProvider extends ServiceProvider
{
    /**
     * Register any application authentication / authorization services.
     *
     * @return void
     */
    public function boot()
    {
        $this->registerPolicies();

        Auth::extend('jwt', function ($app, $name, array $config) {
            // Return an instance of Illuminate\Contracts\Auth\Guard...

            return new JwtGuard(Auth::createUserProvider($config['provider']));
        });
    }
}
```

As you can see in the example above, the callback passed to the `extend` method should return an implementation of `Illuminate\Contracts\Auth\Guard`. This interface contains a few methods you will need to implement to define a custom guard. Once your custom guard has been defined, you may use this guard in the guards configuration of your `auth.php` configuration file:

```
'guards' => [
    'api' => [
        'driver' => 'jwt',
        'provider' => 'users',
    ],
],
```

Closure Request Guards

The simplest way to implement a custom, HTTP request based authentication system is by using the `Auth::viaRequest` method. This method allows you to quickly define your authentication process using a single Closure.

To get started, call the `Auth::viaRequest` method within the `boot` method of your `AuthServiceProvider`. The `viaRequest` method accepts an authentication driver name as its first argument. This name can be any string that describes your custom guard. The second argument passed to the method should be a Closure that receives the incoming HTTP request and returns a user instance or, if authentication fails, `null`:

```
use App\User;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
```

```
/**
 * Register any application authentication / authorization services.
 *
 * @return void
 */
public function boot()
{
    $this->registerPolicies();

    Auth::viaRequest('custom-token', function ($request) {
        return User::where('token', $request->token)->first();
    });
}
```

Once your custom authentication driver has been defined, you use it as a driver within guards configuration of your `auth.php` configuration file:

```
'guards' => [
    'api' => [
        'driver' => 'custom-token',
    ],
],
```

Adding Custom User Providers

If you are not using a traditional relational database to store your users, you will need to extend Laravel with your own authentication user provider. We will use the `provider` method on the `Auth` facade to define a custom user provider:

```
<?php

namespace App\Providers;

use App\Extensions\iakUserProvider;
use Illuminate\Foundation\Support\Providers\AuthServiceProvider as ServiceProvider;
use Illuminate\Support\Facades\Auth;

class AuthServiceProvider extends ServiceProvider
{
    /**
     * Register any application authentication / authorization services.
     *
     * @return void
     */
    public function boot()
    {
        $this->registerPolicies();

        Auth::provider('riak', function ($app, array $config) {
            // Return an instance of Illuminate\Contracts\Auth\UserProvider...

            return new RiakUserProvider($app->make('riak.connection'));
        });
    }
}
```

After you have registered the provider using the `provider` method, you may switch to the new user provider in your `auth.php` configuration file. First, define a provider that uses your new driver:

```
'providers' => [
    'users' => [
        'driver' => 'riak',
    ],
],
```

Finally, you may use this provider in your guards configuration:

```
'guards' => [
    'web' => [
        'driver' => 'session',
        'provider' => 'users',
    ],
],
```

The User Provider Contract

The `Illuminate\Contracts\Auth\UserProvider` implementations are only responsible for fetching a `Illuminate\Contracts\Auth\Authenticatable` implementation out of a persistent storage system, such as MySQL, Riak, etc. These two interfaces allow the Laravel authentication mechanisms to continue functioning regardless of how the user data is stored or what type of class is used to represent it.

Let's take a look at the `Illuminate\Contracts\Auth\UserProvider` contract:

```
<?php

namespace Illuminate\Contracts\Auth;

interface UserProvider
{
    public function retrieveById($identifier);
    public function retrieveByToken($identifier, $token);
    public function updateRememberToken(Authenticatable $user, $token);
    public function retrieveByCredentials(array $credentials);
    public function validateCredentials(Authenticatable $user, array $credentials);
}
```

The `retrieveById` function typically receives a key representing the user, such as an auto-incrementing ID from a MySQL database. The `Authenticatable` implementation matching the ID should be retrieved and returned by the method.

The `retrieveByToken` function retrieves a user by their unique `$identifier` and "remember me" `$token`, stored in a field `remember_token`. As with the previous method, the `Authenticatable` implementation should be returned.

The `updateRememberToken` method updates the `$user` field `remember_token` with the new `$token`. A fresh token is assigned on a successful "remember me" login attempt or when the user is logging out.

The `retrieveByCredentials` method receives the array of credentials passed to the `Auth::attempt` method when attempting to sign into an application. The method should then "query" the underlying persistent storage for the user matching those credentials. Typically, this method will run a query with a "where" condition on `$credentials['username']`. The method should then return an implementation of `Authenticatable`. **This method should not attempt to do any password validation or authentication.**

The `validateCredentials` method should compare the given `$user` with the `$credentials` to authenticate the user. For example, this method should probably use `Hash::check` to compare the value of `$user->getAuthPassword()` to the value of `$credentials['password']`. This method should return `true` or `false` indicating on whether the password is valid.

The Authenticatable Contract

Now that we have explored each of the methods on the `UserProvider`, let's take a look at the `Authenticatable` contract. Remember, the provider should return implementations of this interface from the `retrieveById`, `retrieveByToken`, and `retrieveByCredentials` methods:

```
<?php

namespace Illuminate\Contracts\Auth;

interface Authenticatable
{
    public function getAuthIdentifierName();
    public function getAuthIdentifier();
    public function getAuthPassword();
    public function getRememberToken();
    public function setRememberToken($value);
    public function getRememberTokenName();
}
```

This interface is simple. The `getAuthIdentifierName` method should return the name of the "primary key" field of the user and the `getAuthIdentifier` method should return the "primary key" of the user. In a MySQL back-end, again, this would be the auto-incrementing primary key. The `getAuthPassword` should return the user's hashed password. This interface allows the authentication system to work with any `User` class, regardless of

what ORM or storage abstraction layer you are using. By default, Laravel includes a `User` class in the `app` directory which implements this interface, so you may consult this class for an implementation example.

Events

Laravel raises a variety of [events](#) during the authentication process. You may attach listeners to these events in your `EventServiceProvider`:

```
/**
 * The event listener mappings for the application.
 *
 * @var array
 */
protected $listen = [
    'Illuminate\Auth\Events\Registered' => [
        'App\Listeners\LogRegisteredUser',
    ],

    'Illuminate\Auth\Events\Attempting' => [
        'App\Listeners\LogAuthenticationAttempt',
    ],

    'Illuminate\Auth\Events\Authenticated' => [
        'App\Listeners\LogAuthenticated',
    ],

    'Illuminate\Auth\Events>Login' => [
        'App\Listeners\LogSuccessfulLogin',
    ],

    'Illuminate\Auth\Events\Failed' => [
        'App\Listeners\LogFailedLogin',
    ],

    'Illuminate\Auth\Events\Validated' => [
        'App\Listeners\LogValidated',
    ],

    'Illuminate\Auth\Events\Verified' => [
        'App\Listeners\LogVerified',
    ],

    'Illuminate\Auth\Events\Logout' => [
        'App\Listeners\LogSuccessfulLogout',
    ],

    'Illuminate\Auth\Events\CurrentDeviceLogout' => [
        'App\Listeners\LogCurrentDeviceLogout',
    ],

    'Illuminate\Auth\Events\OtherDeviceLogout' => [
        'App\Listeners\LogOtherDeviceLogout',
    ],

    'Illuminate\Auth\Events\Lockout' => [
        'App\Listeners\LogLockout',
    ],

    'Illuminate\Auth\Events>PasswordReset' => [
        'App\Listeners\LogPasswordReset',
    ],
];
```

Security

Authorization

Authorization

- [Introduction](#)
- [Gates](#)
 - [Writing Gates](#)
 - [Authorizing Actions](#)
 - [Gate Responses](#)
 - [Intercepting Gate Checks](#)
- [Creating Policies](#)
 - [Generating Policies](#)
 - [Registering Policies](#)
- [Writing Policies](#)
 - [Policy Methods](#)
 - [Policy Responses](#)
 - [Methods Without Models](#)
 - [Guest Users](#)
 - [Policy Filters](#)
- [Authorizing Actions Using Policies](#)
 - [Via The User Model](#)
 - [Via Middleware](#)
 - [Via Controller Helpers](#)
 - [Via Blade Templates](#)
 - [Supplying Additional Context](#)

Introduction

In addition to providing [authentication](#) services out of the box, Laravel also provides a simple way to authorize user actions against a given resource. Like authentication, Laravel's approach to authorization is simple, and there are two primary ways of authorizing actions: gates and policies.

Think of gates and policies like routes and controllers. Gates provide a simple, Closure based approach to authorization while policies, like controllers, group their logic around a particular model or resource. We'll explore gates first and then examine policies.

You do not need to choose between exclusively using gates or exclusively using policies when building an application. Most applications will most likely contain a mixture of gates and policies, and that is perfectly fine! Gates are most applicable to actions which are not related to any model or resource, such as viewing an administrator dashboard. In contrast, policies should be used when you wish to authorize an action for a particular model or resource.

Gates

Writing Gates

Gates are Closures that determine if a user is authorized to perform a given action and are typically defined in the `App\Providers\AuthServiceProvider` class using the `Gate` facade. Gates always receive a user instance as their first argument, and may optionally receive additional arguments such as a relevant Eloquent model:

```
/**
 * Register any authentication / authorization services.
 *
 * @return void
 */
public function boot()
{
    $this->registerPolicies();
}
```

```
Gate::define('edit-settings', function ($user) {
    return $user->isAdmin;
});

Gate::define('update-post', function ($user, $post) {
    return $user->id === $post->user_id;
});
}
```

Gates may also be defined using a `class@method` style callback string, like controllers:

```
/**
 * Register any authentication / authorization services.
 *
 * @return void
 */
public function boot()
{
    $this->registerPolicies();

    Gate::define('update-post', 'App\Policies\PostPolicy@update');
}
```

Authorizing Actions

To authorize an action using gates, you should use the `allows` or `denies` methods. Note that you are not required to pass the currently authenticated user to these methods. Laravel will automatically take care of passing the user into the gate Closure:

```
if (Gate::allows('edit-settings')) {
    // The current user can edit settings
}

if (Gate::allows('update-post', $post)) {
    // The current user can update the post...
}

if (Gate::denies('update-post', $post)) {
    // The current user can't update the post...
}
```

If you would like to determine if a particular user is authorized to perform an action, you may use the `forUser` method on the Gate facade:

```
if (Gate::forUser($user)->allows('update-post', $post)) {
    // The user can update the post...
}

if (Gate::forUser($user)->denies('update-post', $post)) {
    // The user can't update the post...
}
```

You may authorize multiple actions at a time with the `any` or `none` methods:

```
if (Gate::any(['update-post', 'delete-post'], $post)) {
    // The user can update or delete the post
}

if (Gate::none(['update-post', 'delete-post'], $post)) {
    // The user cannot update or delete the post
}
```

Authorizing Or Throwing Exceptions

If you would like to attempt to authorize an action and automatically throw an `Illuminate\Auth\Access\AuthorizationException` if the user is not allowed to perform the given action, you may use the `Gate::authorize` method. Instances of `AuthorizationException` are automatically converted to 403 HTTP response:

```
Gate::authorize('update-post', $post);

// The action is authorized...
```

Supplying Additional Context

The gate methods for authorizing abilities (allows, denies, check, any, none, authorize, can, cannot) and the authorization [Blade directives](#) (@can, @cannot, @canany) can receive an array as the second argument. These array elements are passed as parameters to gate, and can be used for additional context when making authorization decisions:

```
Gate::define('create-post', function ($user, $category, $extraFlag) {
    return $category->group > 3 && $extraFlag === true;
});

if (Gate::check('create-post', [$category, $extraFlag])) {
    // The user can create the post...
}
```

Gate Responses

So far, we have only examined gates that return simple boolean values. However, sometimes you may wish to return a more detailed response, including an error message. To do so, you may return a `Illuminate\Auth\Access\Response` from your gate:

```
use Illuminate\Auth\Access\Response;
use Illuminate\Support\Facades\Gate;

Gate::define('edit-settings', function ($user) {
    return $user->isAdmin
        ? Response::allow()
        : Response::deny('You must be a super administrator.');
```

When returning an authorization response from your gate, the `Gate::allows` method will still return a simple boolean value; however, you may use the `Gate::inspect` method to get the full authorization response returned by the gate:

```
$response = Gate::inspect('edit-settings', $post);

if ($response->allowed()) {
    // The action is authorized...
} else {
    echo $response->message();
}
```

Of course, when using the `Gate::authorize` method to throw an `AuthorizationException` if the action is not authorized, the error message provided by the authorization response will be propagated to the HTTP response:

```
Gate::authorize('edit-settings', $post);

// The action is authorized...
```

Intercepting Gate Checks

Sometimes, you may wish to grant all abilities to a specific user. You may use the `before` method to define a callback that is run before all other authorization checks:

```
Gate::before(function ($user, $ability) {
    if ($user->isSuperAdmin()) {
        return true;
    }
});
```

If the `before` callback returns a non-null result that result will be considered the result of the check.

You may use the `after` method to define a callback to be executed after all other authorization checks:

```
Gate::after(function ($user, $ability, $result, $arguments) {
    if ($user->isSuperAdmin()) {
        return true;
    }
});
```

Similar to the `before` check, if the `after` callback returns a non-null result that result will be considered the result of the check.

Creating Policies

Generating Policies

Policies are classes that organize authorization logic around a particular model or resource. For example, if your application is a blog, you may have a `Post` model and a corresponding `PostPolicy` to authorize user actions such as creating or updating posts.

You may generate a policy using the `make:policy` [artisan command](#). The generated policy will be placed in the `app/Policies` directory. If this directory does not exist in your application, Laravel will create it for you:

```
php artisan make:policy PostPolicy
```

The `make:policy` command will generate an empty policy class. If you would like to generate a class with the basic "CRUD" policy methods already included in the class, you may specify a `--model` when executing the command:

```
php artisan make:policy PostPolicy --model=Post
```

TIP All policies are resolved via the Laravel [service container](#), allowing you to type-hint any needed dependencies in the policy's constructor to have them automatically injected.

Registering Policies

Once the policy exists, it needs to be registered. The `AuthServiceProvider` included with fresh Laravel applications contains a `policies` property which maps your Eloquent models to their corresponding policies. Registering a policy will instruct Laravel which policy to utilize when authorizing actions against a given model:

```
<?php

namespace App\Providers;

use App\Policies\PostPolicy;
use App\Post;
use Illuminate\Foundation\Support\Providers\AuthServiceProvider as ServiceProvider;
use Illuminate\Support\Facades\Gate;

class AuthServiceProvider extends ServiceProvider
{
    /**
     * The policy mappings for the application.
     *
     * @var array
     */
    protected $policies = [
        Post::class => PostPolicy::class,
    ];

    /**
     * Register any application authentication / authorization services.
     *
     * @return void
     */
    public function boot()
    {
        $this->registerPolicies();
    }
}
```

Policy Auto-Discovery

Instead of manually registering model policies, Laravel can auto-discover policies as long as the model and

policy follow standard Laravel naming conventions. Specifically, the policies must be in a `Policies` directory below the directory that contains the models. So, for example, the models may be placed in the `app` directory while the policies may be placed in the `app/Policies` directory. In addition, the policy name must match the model name and have a `Policy` suffix. So, a `User` model would correspond to a `UserPolicy` class.

If you would like to provide your own policy discovery logic, you may register a custom callback using the `Gate::guessPolicyNamesUsing` method. Typically, this method should be called from the `boot` method of your application's `AuthServiceProvider`:

```
use Illuminate\Support\Facades\Gate;

Gate::guessPolicyNamesUsing(function ($modelClass) {
    // return policy class name...
});
```

NOTE Any policies that are explicitly mapped in your `AuthServiceProvider` will take precedence over any potential auto-discovered policies.

Writing Policies

Policy Methods

Once the policy has been registered, you may add methods for each action it authorizes. For example, let's define an `update` method on our `PostPolicy` which determines if a given user can update a given `Post` instance.

The `update` method will receive a `User` and a `Post` instance as its arguments, and should return `true` or `false` indicating whether the user is authorized to update the given `Post`. So, for this example, let's verify that the user's `id` matches the `user_id` on the post:

```
<?php

namespace App\Policies;

use App\Post;
use App\User;

class PostPolicy
{
    /**
     * Determine if the given post can be updated by the user.
     *
     * @param  \App\User  $user
     * @param  \App\Post  $post
     * @return bool
     */
    public function update(User $user, Post $post)
    {
        return $user->id === $post->user_id;
    }
}
```

You may continue to define additional methods on the policy as needed for the various actions it authorizes. For example, you might define `view` or `delete` methods to authorize various `Post` actions, but remember you are free to give your policy methods any name you like.

TIP If you used the `--model` option when generating your policy via the Artisan console, it will already contain methods for the `viewAny`, `view`, `create`, `update`, `delete`, `restore`, and `forceDelete` actions.

Policy Responses

So far, we have only examined policy methods that return simple boolean values. However, sometimes you may wish to return a more detailed response, including an error message. To do so, you may return an `Illuminate\Auth\Access\Response` from your policy method:

```
use Illuminate\Auth\Access\Response;

/**
 * Determine if the given post can be updated by the user.
```

```
*
* @param \App\User $user
* @param \App\Post $post
* @return \Illuminate\Auth\Access\Response
*/
public function update(User $user, Post $post)
{
    return $user->id === $post->user_id
        ? Response::allow()
        : Response::deny('You do not own this post.');
```

When returning an authorization response from your policy, the `Gate::allows` method will still return a simple boolean value; however, you may use the `Gate::inspect` method to get the full authorization response returned by the gate:

```
$response = Gate::inspect('update', $post);

if ($response->allowed()) {
    // The action is authorized...
} else {
    echo $response->message();
}
```

Of course, when using the `Gate::authorize` method to throw an `AuthorizationException` if the action is not authorized, the error message provided by the authorization response will be propagated to the HTTP response:

```
Gate::authorize('update', $post);

// The action is authorized...
```

Methods Without Models

Some policy methods only receive the currently authenticated user and not an instance of the model they authorize. This situation is most common when authorizing `create` actions. For example, if you are creating a blog, you may wish to check if a user is authorized to create any posts at all.

When defining policy methods that will not receive a model instance, such as a `create` method, it will not receive a model instance. Instead, you should define the method as only expecting the authenticated user:

```
/**
 * Determine if the given user can create posts.
 *
 * @param \App\User $user
 * @return bool
 */
public function create(User $user)
{
    //
}
```

Guest Users

By default, all gates and policies automatically return `false` if the incoming HTTP request was not initiated by an authenticated user. However, you may allow these authorization checks to pass through to your gates and policies by declaring an "optional" type-hint or supplying a `null` default value for the user argument definition:

```
<?php

namespace App\Policies;

use App\Post;
use App\User;

class PostPolicy
{
    /**
     * Determine if the given post can be updated by the user.
     *
     * @param \App\User $user
     * @param \App\Post $post
     * @return bool
     */
}
```

```
    */
    public function update(?User $user, Post $post)
    {
        return optional($user)->id === $post->user_id;
    }
}
```

Policy Filters

For certain users, you may wish to authorize all actions within a given policy. To accomplish this, define a `before` method on the policy. The `before` method will be executed before any other methods on the policy, giving you an opportunity to authorize the action before the intended policy method is actually called. This feature is most commonly used for authorizing application administrators to perform any action:

```
public function before($user, $ability)
{
    if ($user->isSuperAdmin()) {
        return true;
    }
}
```

If you would like to deny all authorizations for a user you should return `false` from the `before` method. If `null` is returned, the authorization will fall through to the policy method.

NOTE The `before` method of a policy class will not be called if the class doesn't contain a method with a name matching the name of the ability being checked.

Authorizing Actions Using Policies

Via The User Model

The user model that is included with your Laravel application includes two helpful methods for authorizing actions: `can` and `cant`. The `can` method receives the action you wish to authorize and the relevant model. For example, let's determine if a user is authorized to update a given `Post` model:

```
if ($user->can('update', $post)) {
    //
}
```

If a [policy is registered](#) for the given model, the `can` method will automatically call the appropriate policy and return the boolean result. If no policy is registered for the model, the `can` method will attempt to call the Closure based Gate matching the given action name.

Actions That Don't Require Models

Remember, some actions like `create` may not require a model instance. In these situations, you may pass a class name to the `can` method. The class name will be used to determine which policy to use when authorizing the action:

```
use App\Post;

if ($user->can('create', Post::class)) {
    // Executes the "create" method on the relevant policy...
}
```

Via Middleware

Laravel includes a middleware that can authorize actions before the incoming request even reaches your routes or controllers. By default, the `Illuminate\Auth\Middleware\Authorize` middleware is assigned the `can` key in your `App\Http\Kernel` class. Let's explore an example of using the `can` middleware to authorize that a user can update a blog post:

```
use App\Post;

Route::put('/post/{post}', function (Post $post) {
```

```
// The current user may update the post...
})->middleware('can:update,post');
```

In this example, we're passing the `can` middleware two arguments. The first is the name of the action we wish to authorize and the second is the route parameter we wish to pass to the policy method. In this case, since we are using [implicit model binding](#), a `Post` model will be passed to the policy method. If the user is not authorized to perform the given action, a HTTP response with a 403 status code will be generated by the middleware.

Actions That Don't Require Models

Again, some actions like `create` may not require a model instance. In these situations, you may pass a class name to the middleware. The class name will be used to determine which policy to use when authorizing the action:

```
Route::post('/post', function () {
    // The current user may create posts...
})->middleware('can:create,App\Post');
```

Via Controller Helpers

In addition to helpful methods provided to the `User` model, Laravel provides a helpful `authorize` method to any of your controllers which extend the `App\Http\Controllers\Controller` base class. Like the `can` method, this method accepts the name of the action you wish to authorize and the relevant model. If the action is not authorized, the `authorize` method will throw an `Illuminate\Auth\Access\AuthorizationException`, which the default Laravel exception handler will convert to an HTTP response with a 403 status code:

```
<?php

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;
use App\Post;
use Illuminate\Http\Request;

class PostController extends Controller
{
    /**
     * Update the given blog post.
     *
     * @param Request $request
     * @param Post $post
     * @return Response
     * @throws \Illuminate\Auth\Access\AuthorizationException
     */
    public function update(Request $request, Post $post)
    {
        $this->authorize('update', $post);

        // The current user can update the blog post...
    }
}
```

Actions That Don't Require Models

As previously discussed, some actions like `create` may not require a model instance. In these situations, you should pass a class name to the `authorize` method. The class name will be used to determine which policy to use when authorizing the action:

```
/**
 * Create a new blog post.
 *
 * @param Request $request
 * @return Response
 * @throws \Illuminate\Auth\Access\AuthorizationException
 */
public function create(Request $request)
{
    $this->authorize('create', Post::class);

    // The current user can create blog posts...
}
```

Authorizing Resource Controllers

If you are utilizing [resource controllers](#), you may make use of the `authorizeResource` method in the controller's constructor. This method will attach the appropriate can middleware definitions to the resource controller's methods.

The `authorizeResource` method accepts the model's class name as its first argument, and the name of the route / request parameter that will contain the model's ID as its second argument. You should ensure your [resource controller](#) is created with the `--model` flag to have the required method signatures and type hints:

```
<?php

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;
use App\Post;
use Illuminate\Http\Request;

class PostController extends Controller
{
    public function __construct()
    {
        $this->authorizeResource(Post::class, 'post');
    }
}
```

The following controller methods will be mapped to their corresponding policy method:

Controller Method Policy Method

index	viewAny
show	view
create	create
store	create
edit	update
update	update
destroy	delete

TIP You may use the `make:policy` command with the `--model` option to quickly generate a policy class for a given model: `php artisan make:policy PostPolicy --model=Post`.

Via Blade Templates

When writing Blade templates, you may wish to display a portion of the page only if the user is authorized to perform a given action. For example, you may wish to show an update form for a blog post only if the user can actually update the post. In this situation, you may use the `@can` and `@cannot` family of directives:

```
@can('update', $post)
    <!-- The Current User Can Update The Post -->
@elsecan('create', App\Post::class)
    <!-- The Current User Can Create New Post -->
@endcan

@cannot('update', $post)
    <!-- The Current User Cannot Update The Post -->
@elsecannot('create', App\Post::class)
    <!-- The Current User Cannot Create A New Post -->
@endcannot
```

These directives are convenient shortcuts for writing `@if` and `@unless` statements. The `@can` and `@cannot` statements above respectively translate to the following statements:

```
@if (Auth::user()->can('update', $post))
    <!-- The Current User Can Update The Post -->
@endif

@unless (Auth::user()->can('update', $post))
    <!-- The Current User Cannot Update The Post -->
@endunless
```

You may also determine if a user has any authorization ability from a given list of abilities. To accomplish this, use the `@canany` directive:

```
@canany(['update', 'view', 'delete'], $post)
    // The current user can update, view, or delete the post
@elsecanany(['create'], \App\Post::class)
    // The current user can create a post
@endcanany
```

Actions That Don't Require Models

Like most of the other authorization methods, you may pass a class name to the `@can` and `@cannot` directives if the action does not require a model instance:

```
@can('create', App\Post::class)
    <!-- The Current User Can Create Posts -->
@endcan

@cannot('create', App\Post::class)
    <!-- The Current User Can't Create Posts -->
@endcannot
```

Supplying Additional Context

When authorizing actions using policies, you may pass an array as the second argument to the various authorization functions and helpers. The first element in the array will be used to determine which policy should be invoked, while the rest of the array elements are passed as parameters to the policy method and can be used for additional context when making authorization decisions. For example, consider the following `PostPolicy` method definition which contains an additional `$category` parameter:

```
/**
 * Determine if the given post can be updated by the user.
 *
 * @param \App\User $user
 * @param \App\Post $post
 * @param int $category
 * @return bool
 */
public function update(User $user, Post $post, int $category)
{
    return $user->id === $post->user_id &&
        $category > 3;
}
```

When attempting to determine if the authenticated user can update a given post, we can invoke this policy method like so:

```
/**
 * Update the given blog post.
 *
 * @param Request $request
 * @param Post $post
 * @return Response
 * @throws \Illuminate\Auth\Access\AuthorizationException
 */
public function update(Request $request, Post $post)
{
    $this->authorize('update', [$post, $request->input('category')]);

    // The current user can update the blog post...
}
```

Security

Email Verification

Email Verification

- [Introduction](#)
- [Model Preparation](#)
- [Database Considerations](#)
- [Routing](#)
 - [Protecting Routes](#)
- [Views](#)
- [After Verifying Emails](#)
- [Events](#)

Introduction

Many web applications require users to verify their email addresses before using the application. Rather than forcing you to re-implement this on each application, Laravel provides convenient methods for sending and verifying email verification requests.

Model Preparation

To get started, verify that your `App\User` model implements the `Illuminate\Contracts\Auth\MustVerifyEmail` contract:

```
<?php

namespace App;

use Illuminate\Contracts\Auth\MustVerifyEmail;
use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;

class User extends Authenticatable implements MustVerifyEmail
{
    use Notifiable;

    // ...
}
```

Once this interface has been added to your model, newly registered users will automatically be sent an email containing an email verification link. As you can see by examining your `EventServiceProvider`, Laravel already contains a `SendEmailVerificationNotification` listener that is attached to the `Illuminate\Auth\Events\Registered` event.

Database Considerations

The Email Verification Column

Next, your `users` table must contain an `email_verified_at` column to store the date and time that the email address was verified. By default, the `users` table migration included with the Laravel framework already includes this column. So, all you need to do is run your database migrations:

```
php artisan migrate
```

Routing

Laravel includes the `Auth\VerificationController` class that contains the necessary logic to send verification links and verify emails. To register the necessary routes for this controller, pass the `verify` option to the `Auth::routes` method:

```
Auth::routes(['verify' => true]);
```

Protecting Routes

[Route middleware](#) can be used to only allow verified users to access a given route. Laravel ships with a `verified` middleware, which is defined at `Illuminate\Auth\Middleware\EnsureEmailIsVerified`. Since this middleware is already registered in your application's HTTP kernel, all you need to do is attach the middleware to a route definition:

```
Route::get('profile', function () {
    // Only verified users may enter...
})->middleware('verified');
```

Views

To generate all of the necessary view for email verification, you may use the `laravel/ui` Composer package:

```
composer require laravel/ui

php artisan ui vue --auth
```

The email verification view is placed in `resources/views/auth/verify.blade.php`. You are free to customize this view as needed for your application.

After Verifying Emails

After an email address is verified, the user will automatically be redirected to `/home`. You can customize the post verification redirect location by defining a `redirectTo` method or property on the `VerificationController`:

```
protected $redirectTo = '/dashboard';
```

Events

Laravel dispatches [events](#) during the email verification process. You may attach listeners to these events in your `EventServiceProvider`:

```
/**
 * The event listener mappings for the application.
 *
 * @var array
 */
protected $listen = [
    'Illuminate\Auth\Events\Verified' => [
        'App\Listeners\LogVerifiedUser',
    ],
];
```


Security

Encryption

Encryption

- [Introduction](#)
- [Configuration](#)
- [Using The Encrypter](#)

Introduction

Laravel's encrypter uses OpenSSL to provide AES-256 and AES-128 encryption. You are strongly encouraged to use Laravel's built-in encryption facilities and not attempt to roll your own "home grown" encryption algorithms. All of Laravel's encrypted values are signed using a message authentication code (MAC) so that their underlying value can not be modified once encrypted.

Configuration

Before using Laravel's encrypter, you must set a key option in your `config/app.php` configuration file. You should use the `php artisan key:generate` command to generate this key since this Artisan command will use PHP's secure random bytes generator to build your key. If this value is not properly set, all values encrypted by Laravel will be insecure.

Using The Encrypter

Encrypting A Value

You may encrypt a value using the `encryptString` method of the `crypt` facade. All encrypted values are encrypted using OpenSSL and the AES-256-CBC cipher. Furthermore, all encrypted values are signed with a message authentication code (MAC) to detect any modifications to the encrypted string:

```
<?php

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;
use App\User;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Crypt;

class UserController extends Controller
{
    /**
     * Store a secret message for the user.
     *
     * @param Request $request
     * @param int $id
     * @return Response
     */
    public function storeSecret(Request $request, $id)
    {
        $user = User::findOrFail($id);

        $user->fill([
            'secret' => Crypt::encryptString($request->secret),
        ])->save();
    }
}
```

Decrypting A Value

You may decrypt values using the `decryptString` method of the `crypt` facade. If the value can not be properly decrypted, such as when the MAC is invalid, an `Illuminate\Contracts\Encryption\DecryptException` will be

thrown:

```
use Illuminate\Contracts\Encryption\DecryptException;
use Illuminate\Support\Facades\Crypt;

try {
    $decrypted = Crypt::decryptString($encryptedValue);
} catch (DecryptException $e) {
    //
}
```

Security

Hashing

Hashing

- [Introduction](#)
- [Configuration](#)
- [Basic Usage](#)

Introduction

The Laravel Hash [facade](#) provides secure Bcrypt and Argon2 hashing for storing user passwords. If you are using the built-in `LoginController` and `RegisterController` classes that are included with your Laravel application, they will use Bcrypt for registration and authentication by default.

TIP Bcrypt is a great choice for hashing passwords because its "work factor" is adjustable, which means that the time it takes to generate a hash can be increased as hardware power increases.

Configuration

The default hashing driver for your application is configured in the `config/hashing.php` configuration file. There are currently three supported drivers: [Bcrypt](#) and [Argon2](#) (Argon2i and Argon2id variants).

NOTE The Argon2i driver requires PHP 7.2.0 or greater and the Argon2id driver requires PHP 7.3.0 or greater.

Basic Usage

You may hash a password by calling the `make` method on the `Hash` facade:

```
<?php

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Hash;

class UpdatePasswordController extends Controller
{
    /**
     * Update the password for the user.
     *
     * @param Request $request
     * @return Response
     */
    public function update(Request $request)
    {
        // Validate the new password length...

        $request->user()->fill([
            'password' => Hash::make($request->newPassword)
        ])->save();
    }
}
```

Adjusting The Bcrypt Work Factor

If you are using the Bcrypt algorithm, the `make` method allows you to manage the work factor of the algorithm using the `rounds` option; however, the default is acceptable for most applications:

```
$hashed = Hash::make('password', [
    'rounds' => 12,
]);
```

Adjusting The Argon2 Work Factor

If you are using the Argon2 algorithm, the `make` method allows you to manage the work factor of the algorithm using the `memory`, `time`, and `threads` options; however, the defaults are acceptable for most applications:

```
$hashed = Hash::make('password', [
    'memory' => 1024,
    'time' => 2,
    'threads' => 2,
]);
```

TIP For more information on these options, check out the [official PHP documentation](#).

Verifying A Password Against A Hash

The `check` method allows you to verify that a given plain-text string corresponds to a given hash. However, if you are using the `LoginController` [included with Laravel](#), you will probably not need to use this directly, as this controller automatically calls this method:

```
if (Hash::check('plain-text', $hashedPassword)) {
    // The passwords match...
}
```

Checking If A Password Needs To Be Rehashed

The `needsRehash` function allows you to determine if the work factor used by the hasher has changed since the password was hashed:

```
if (Hash::needsRehash($hashed)) {
    $hashed = Hash::make('plain-text');
}
```

Security

Password Reset

Resetting Passwords

- [Introduction](#)
- [Database Considerations](#)
- [Routing](#)
- [Views](#)
- [After Resetting Passwords](#)
- [Customization](#)

Introduction

TIP Want to get started fast? Install the `laravel/ui` Composer package and run `php artisan ui vue --auth` in a fresh Laravel application. After migrating your database, navigate your browser to `http://your-app.test/register` or any other URL that is assigned to your application. This single command will take care of scaffolding your entire authentication system, including resetting passwords!

Most web applications provide a way for users to reset their forgotten passwords. Rather than forcing you to re-implement this on each application, Laravel provides convenient methods for sending password reminders and performing password resets.

NOTE Before using the password reset features of Laravel, your user must use the `Illuminate\Notifications\Notifiable` trait.

Database Considerations

To get started, verify that your `App\User` model implements the `Illuminate\Contracts\Auth\CanResetPassword` contract. The `App\User` model included with the framework already implements this interface, and uses the `Illuminate\Auth\Passwords\CanResetPassword` trait to include the methods needed to implement the interface.

Generating The Reset Token Table Migration

Next, a table must be created to store the password reset tokens. The migration for this table is included in the `laravel/ui` Composer package. After installing the `laravel/ui` package, you may use the `migrate` command to create the password reset token database table:

```
composer require laravel/ui
php artisan migrate
```

Routing

Laravel includes `Auth\ForgotPasswordController` and `Auth\ResetPasswordController` classes that contains the logic necessary to e-mail password reset links and reset user passwords. All of the routes needed to perform password resets may be generated using the `laravel/ui` Composer package:

```
composer require laravel/ui
php artisan ui vue --auth
```

Views

To generate all of the necessary view for resetting passwords, you may use the `laravel/ui` Composer package:

```
composer require laravel/ui
php artisan ui vue --auth
```

These views are placed in `resources/views/auth/passwords`. You are free to customize them as needed for your application.

After Resetting Passwords

Once you have defined the routes and views to reset your user's passwords, you may access the route in your browser at `/password/reset`. The `ForgotPasswordController` included with the framework already includes the logic to send the password reset link e-mails, while the `ResetPasswordController` includes the logic to reset user passwords.

After a password is reset, the user will automatically be logged into the application and redirected to `/home`. You can customize the post password reset redirect location by defining a `redirectTo` property on the `ResetPasswordController`:

```
protected $redirectTo = '/dashboard';
```

NOTE By default, password reset tokens expire after one hour. You may change this via the password reset expire option in your `config/auth.php` file.

Customization

Authentication Guard Customization

In your `auth.php` configuration file, you may configure multiple "guards", which may be used to define authentication behavior for multiple user tables. You can customize the included `ResetPasswordController` to use the guard of your choice by overriding the `guard` method on the controller. This method should return a guard instance:

```
use Illuminate\Support\Facades\Auth;

/**
 * Get the guard to be used during password reset.
 *
 * @return \Illuminate\Contracts\Auth\StatefulGuard
 */
protected function guard()
{
    return Auth::guard('guard-name');
}
```

Password Broker Customization

In your `auth.php` configuration file, you may configure multiple password "brokers", which may be used to reset passwords on multiple user tables. You can customize the included `ForgotPasswordController` and `ResetPasswordController` to use the broker of your choice by overriding the `broker` method:

```
use Illuminate\Support\Facades>Password;

/**
 * Get the broker to be used during password reset.
 *
 * @return PasswordBroker
 */
public function broker()
{
    return Password::broker('name');
}
```

Reset Email Customization

You may easily modify the notification class used to send the password reset link to the user. To get started, override the `sendPasswordResetNotification` method on your `User` model. Within this method, you may send the notification using any notification class you choose. The password reset `$token` is the first argument received by the method:

```
/**
 * Send the password reset notification.
 *
 * @param string $token
 * @return void
 */
public function sendPasswordResetNotification($token)
{
    $this->notify(new ResetPasswordNotification($token));
}
```

Digging Deeper

Artisan Console

Artisan Console

- [Introduction](#)
 - [Tinker \(REPL\)](#)
- [Writing Commands](#)
 - [Generating Commands](#)
 - [Command Structure](#)
 - [Closure Commands](#)
- [Defining Input Expectations](#)
 - [Arguments](#)
 - [Options](#)
 - [Input Arrays](#)
 - [Input Descriptions](#)
- [Command I/O](#)
 - [Retrieving Input](#)
 - [Prompting For Input](#)
 - [Writing Output](#)
- [Registering Commands](#)
- [Programmatically Executing Commands](#)
 - [Calling Commands From Other Commands](#)
- [Stub Customization](#)

Introduction

Artisan is the command-line interface included with Laravel. It provides a number of helpful commands that can assist you while you build your application. To view a list of all available Artisan commands, you may use the `list` command:

```
php artisan list
```

Every command also includes a "help" screen which displays and describes the command's available arguments and options. To view a help screen, precede the name of the command with `help`:

```
php artisan help migrate
```

Tinker (REPL)

Laravel Tinker is a powerful REPL for the Laravel framework, powered by the [PsySH](#) package.

Installation

All Laravel applications include Tinker by default. However, you may install it manually if needed using Composer:

```
composer require laravel/tinker
```

Usage

Tinker allows you to interact with your entire Laravel application on the command line, including the Eloquent ORM, jobs, events, and more. To enter the Tinker environment, run the `tinker` Artisan command:

```
php artisan tinker
```

You can publish Tinker's configuration file using the `vendor:publish` command:

```
php artisan vendor:publish --provider="Laravel\Tinker\TinkerServiceProvider"
```


NOTE The `dispatch` helper function and `dispatch` method on the `Dispatchable` class depends on garbage collection to place the job on the queue. Therefore, when using `tinker`, you should use `Bus::dispatch` or `Queue::push` to dispatch jobs.

Command Whitelist

Tinker utilizes a white-list to determine which Artisan commands are allowed to be run within its shell. By default, you may run the `clear-compiled`, `down`, `env`, `inspire`, `migrate`, `optimize`, and `up` commands. If you would like to white-list more commands you may add them to the `commands` array in your `tinker.php` configuration file:

```
'commands' => [
    // App\Console\Commands\ExampleCommand::class,
],
```

Classes That Should Not Be Aliased

Typically, Tinker automatically aliases classes as you require them in Tinker. However, you may wish to never alias some classes. You may accomplish this by listing the classes in the `dont_alias` array of your `tinker.php` configuration file:

```
'dont_alias' => [
    App\User::class,
],
```

Writing Commands

In addition to the commands provided with Artisan, you may also build your own custom commands. Commands are typically stored in the `app/Console/Commands` directory; however, you are free to choose your own storage location as long as your commands can be loaded by Composer.

Generating Commands

To create a new command, use the `make:command` Artisan command. This command will create a new command class in the `app/Console/Commands` directory. Don't worry if this directory does not exist in your application, since it will be created the first time you run the `make:command` Artisan command. The generated command will include the default set of properties and methods that are present on all commands:

```
php artisan make:command SendEmails
```

Command Structure

After generating your command, you should fill in the `signature` and `description` properties of the class, which will be used when displaying your command on the `list` screen. The `handle` method will be called when your command is executed. You may place your command logic in this method.

TIP For greater code reuse, it is good practice to keep your console commands light and let them defer to application services to accomplish their tasks. In the example below, note that we inject a service class to do the "heavy lifting" of sending the e-mails.

Let's take a look at an example command. Note that we are able to inject any dependencies we need into the command's `handle` method. The Laravel [service container](#) will automatically inject all dependencies that are type-hinted in this method's signature:

```
<?php

namespace App\Console\Commands;

use App\DripEmailer;
use App\User;
use Illuminate\Console\Command;

class SendEmails extends Command
{
    /**
```

```

    * The name and signature of the console command.
    *
    * @var string
    */
protected $signature = 'email:send {user}';

/**
 * The console command description.
 *
 * @var string
 */
protected $description = 'Send drip e-mails to a user';

/**
 * Create a new command instance.
 *
 * @return void
 */
public function __construct()
{
    parent::__construct();
}

/**
 * Execute the console command.
 *
 * @param \App\DripEmailer $drip
 * @return mixed
 */
public function handle(DripEmailer $drip)
{
    $drip->send(User::find($this->argument('user')));
}
}
```

Closure Commands

Closure based commands provide an alternative to defining console commands as classes. In the same way that route Closures are an alternative to controllers, think of command Closures as an alternative to command classes. Within the commands method of your app/Console/Kernel.php file, Laravel loads the routes/console.php file:

```

/**
 * Register the Closure based commands for the application.
 *
 * @return void
 */
protected function commands()
{
    require base_path('routes/console.php');
}
}
```

Even though this file does not define HTTP routes, it defines console based entry points (routes) into your application. Within this file, you may define all of your Closure based routes using the `Artisan::command` method. The `command` method accepts two arguments: the [command signature](#) and a Closure which receives the commands arguments and options:

```
Artisan::command('build {project}', function ($project) {
    $this->info("Building {$project}!");
});
```

The Closure is bound to the underlying command instance, so you have full access to all of the helper methods you would typically be able to access on a full command class.

Type-Hinting Dependencies

In addition to receiving your command's arguments and options, command Closures may also type-hint additional dependencies that you would like resolved out of the [service container](#):

```
use App\DripEmailer;
use App\User;

Artisan::command('email:send {user}', function (DripEmailer $drip, $user) {
```

```
        $drip->send(User::find($user));
    });
```

Closure Command Descriptions

When defining a Closure based command, you may use the `describe` method to add a description to the command. This description will be displayed when you run the `php artisan list` or `php artisan help` commands:

```
Artisan::command('build {project}', function ($project) {
    $this->info("Building {$project}!");
})->describe('Build the project');
```

Defining Input Expectations

When writing console commands, it is common to gather input from the user through arguments or options. Laravel makes it very convenient to define the input you expect from the user using the `signature` property on your commands. The `signature` property allows you to define the name, arguments, and options for the command in a single, expressive, route-like syntax.

Arguments

All user supplied arguments and options are wrapped in curly braces. In the following example, the command defines one **required** argument: `user`:

```
/**
 * The name and signature of the console command.
 *
 * @var string
 */
protected $signature = 'email:send {user}';
```

You may also make arguments optional and define default values for arguments:

```
// Optional argument...
email:send {user?}

// Optional argument with default value...
email:send {user=foo}
```

Options

Options, like arguments, are another form of user input. Options are prefixed by two hyphens (`--`) when they are specified on the command line. There are two types of options: those that receive a value and those that don't. Options that don't receive a value serve as a boolean "switch". Let's take a look at an example of this type of option:

```
/**
 * The name and signature of the console command.
 *
 * @var string
 */
protected $signature = 'email:send {user} {--queue}';
```

In this example, the `--queue` switch may be specified when calling the Artisan command. If the `--queue` switch is passed, the value of the option will be `true`. Otherwise, the value will be `false`:

```
php artisan email:send 1 --queue
```

Options With Values

Next, let's take a look at an option that expects a value. If the user must specify a value for an option, suffix the option name with a `=` sign:

```
/**
 * The name and signature of the console command.
```

```
*
* @var string
*/
protected $signature = 'email:send {user} [--queue=]';
```

In this example, the user may pass a value for the option like so:

```
php artisan email:send 1 --queue=default
```

You may assign default values to options by specifying the default value after the option name. If no option value is passed by the user, the default value will be used:

```
email:send {user} [--queue=default]
```

Option Shortcuts

To assign a shortcut when defining an option, you may specify it before the option name and use a `|` delimiter to separate the shortcut from the full option name:

```
email:send {user} [--Q|queue]
```

Input Arrays

If you would like to define arguments or options to expect array inputs, you may use the `*` character. First, let's take a look at an example that specifies an array argument:

```
email:send {user*}
```

When calling this method, the `user` arguments may be passed in order to the command line. For example, the following command will set the value of `user` to `['foo', 'bar']`:

```
php artisan email:send foo bar
```

When defining an option that expects an array input, each option value passed to the command should be prefixed with the option name:

```
email:send {user} [--id=*]
```

```
php artisan email:send --id=1 --id=2
```

Input Descriptions

You may assign descriptions to input arguments and options by separating the parameter from the description using a colon. If you need a little extra room to define your command, feel free to spread the definition across multiple lines:

```
/**
 * The name and signature of the console command.
 *
 * @var string
 */
protected $signature = 'email:send
                        {user : The ID of the user}
                        [--queue= : Whether the job should be queued]';
```

Command I/O

Retrieving Input

While your command is executing, you will obviously need to access the values for the arguments and options accepted by your command. To do so, you may use the `argument` and `option` methods:

```
/**
 * Execute the console command.
 *
 * @return mixed
```

```
*/
public function handle()
{
    $userId = $this->argument('user');

    //
}
```

If you need to retrieve all of the arguments as an array, call the `arguments` method:

```
$arguments = $this->arguments();
```

Options may be retrieved just as easily as arguments using the `option` method. To retrieve all of the options as an array, call the `options` method:

```
// Retrieve a specific option...
$queueName = $this->option('queue');

// Retrieve all options...
$options = $this->options();
```

If the argument or option does not exist, `null` will be returned.

Prompting For Input

In addition to displaying output, you may also ask the user to provide input during the execution of your command. The `ask` method will prompt the user with the given question, accept their input, and then return the user's input back to your command:

```
/**
 * Execute the console command.
 *
 * @return mixed
 */
public function handle()
{
    $name = $this->ask('What is your name?');
}
```

The `secret` method is similar to `ask`, but the user's input will not be visible to them as they type in the console. This method is useful when asking for sensitive information such as a password:

```
$password = $this->secret('What is the password?');
```

Asking For Confirmation

If you need to ask the user for a simple confirmation, you may use the `confirm` method. By default, this method will return `false`. However, if the user enters `y` or `yes` in response to the prompt, the method will return `true`.

```
if ($this->confirm('Do you wish to continue?')) {
    //
}
```

Auto-Completion

The `anticipate` method can be used to provide auto-completion for possible choices. The user can still choose any answer, regardless of the auto-completion hints:

```
$name = $this->anticipate('What is your name?', ['Taylor', 'Dayle']);
```

Alternatively, you may pass a Closure as the second argument to the `anticipate` method. The Closure will be called each time the user types an input character. The Closure should accept a string parameter containing the user's input so far, and return an array of options for auto-completion:

```
$name = $this->anticipate('What is your name?', function ($input) {
    // Return auto-completion options...
});
```

Multiple Choice Questions

If you need to give the user a predefined set of choices, you may use the `choice` method. You may set the array index of the default value to be returned if no option is chosen:

```
$name = $this->choice('What is your name?', ['Taylor', 'Dayle'], $defaultIndex);
```

In addition, the `choice` method accepts optional fourth and fifth arguments for determining the maximum number of attempts to select a valid response and whether multiple selections are permitted:

```
$name = $this->choice(
    'What is your name?',
    ['Taylor', 'Dayle'],
    $defaultIndex,
    $maxAttempts = null,
    $allowMultipleSelections = false
);
```

Writing Output

To send output to the console, use the `line`, `info`, `comment`, `question` and `error` methods. Each of these methods will use appropriate ANSI colors for their purpose. For example, let's display some general information to the user. Typically, the `info` method will display in the console as green text:

```
/**
 * Execute the console command.
 *
 * @return mixed
 */
public function handle()
{
    $this->info('Display this on the screen');
}
```

To display an error message, use the `error` method. Error message text is typically displayed in red:

```
$this->error('Something went wrong!');
```

If you would like to display plain, uncolored console output, use the `line` method:

```
$this->line('Display this on the screen');
```

Table Layouts

The `table` method makes it easy to correctly format multiple rows / columns of data. Just pass in the headers and rows to the method. The width and height will be dynamically calculated based on the given data:

```
$headers = ['Name', 'Email'];

$users = App\User::all(['name', 'email'])->toArray();

$this->table($headers, $users);
```

Progress Bars

For long running tasks, it could be helpful to show a progress indicator. Using the output object, we can start, advance and stop the Progress Bar. First, define the total number of steps the process will iterate through. Then, advance the Progress Bar after processing each item:

```
$users = App\User::all();

$bar = $this->output->createProgressBar(count($users));

$bar->start();

foreach ($users as $user) {
    $this->performTask($user);

    $bar->advance();
}

$bar->finish();
```

For more advanced options, check out the [Symfony Progress Bar component documentation](#).

Registering Commands

Because of the `load` method call in your console kernel's `commands` method, all commands within the `app/Console/Commands` directory will automatically be registered with Artisan. In fact, you are free to make additional calls to the `load` method to scan other directories for Artisan commands:

```
/**
 * Register the commands for the application.
 *
 * @return void
 */
protected function commands()
{
    $this->load(__DIR__.'/Commands');
    $this->load(__DIR__.'/MoreCommands');

    // ...
}
```

You may also manually register commands by adding its class name to the `$commands` property of your `app/Console/Kernel.php` file. When Artisan boots, all the commands listed in this property will be resolved by the [service container](#) and registered with Artisan:

```
protected $commands = [
    Commands\SendEmails::class
];
```

Programmatically Executing Commands

Sometimes you may wish to execute an Artisan command outside of the CLI. For example, you may wish to fire an Artisan command from a route or controller. You may use the `call` method on the Artisan facade to accomplish this. The `call` method accepts either the command's name or class as the first argument, and an array of command parameters as the second argument. The exit code will be returned:

```
Route::get('/foo', function () {
    $exitCode = Artisan::call('email:send', [
        'user' => 1, '--queue' => 'default'
    ]);

    //
});
```

Alternatively, you may pass the entire Artisan command to the `call` method as a string:

```
Artisan::call('email:send 1 --queue=default');
```

Using the `queue` method on the Artisan facade, you may even queue Artisan commands so they are processed in the background by your [queue workers](#). Before using this method, make sure you have configured your queue and are running a queue listener:

```
Route::get('/foo', function () {
    Artisan::queue('email:send', [
        'user' => 1, '--queue' => 'default'
    ]);

    //
});
```

You may also specify the connection or queue the Artisan command should be dispatched to:

```
Artisan::queue('email:send', [
    'user' => 1, '--queue' => 'default'
])->onConnection('redis')->onQueue('commands');
```

Passing Array Values

If your command defines an option that accepts an array, you may pass an array of values to that option:

```
Route::get('/foo', function () {
    $exitCode = Artisan::call('email:send', [
        'user' => 1, '--id' => [5, 13]
    ]);
});
```

Passing Boolean Values

If you need to specify the value of an option that does not accept string values, such as the `--force` flag on the `migrate:refresh` command, you should pass `true` or `false`:

```
$exitCode = Artisan::call('migrate:refresh', [
    '--force' => true,
]);
```

Calling Commands From Other Commands

Sometimes you may wish to call other commands from an existing Artisan command. You may do so using the `call` method. This `call` method accepts the command name and an array of command parameters:

```
/**
 * Execute the console command.
 *
 * @return mixed
 */
public function handle()
{
    $this->call('email:send', [
        'user' => 1, '--queue' => 'default'
    ]);

    //
}
```

If you would like to call another console command and suppress all of its output, you may use the `callSilent` method. The `callSilent` method has the same signature as the `call` method:

```
$this->callSilent('email:send', [
    'user' => 1, '--queue' => 'default'
]);
```

Stub Customization

The Artisan console's `make` commands are used to create a variety of classes, such as controllers, jobs, migrations, and tests. These classes are generated using "stub" files that are populated with values based on your input. However, you may sometimes wish to make small changes to files generated by Artisan. To accomplish this, you may use the `stub:publish` command to publish the most common stubs for customization:

```
php artisan stub:publish
```

The published stubs will be located within a `stubs` directory in the root of your application. Any changes you make to these stubs will be reflected when you generate their corresponding classes using Artisan `make` commands.

Digging Deeper

Broadcasting

Broadcasting

- [Introduction](#)
 - [Configuration](#)
 - [Driver Prerequisites](#)
- [Concept Overview](#)
 - [Using An Example Application](#)
- [Defining Broadcast Events](#)
 - [Broadcast Name](#)
 - [Broadcast Data](#)
 - [Broadcast Queue](#)
 - [Broadcast Conditions](#)
- [Authorizing Channels](#)
 - [Defining Authorization Routes](#)
 - [Defining Authorization Callbacks](#)
 - [Defining Channel Classes](#)
- [Broadcasting Events](#)
 - [Only To Others](#)
- [Receiving Broadcasts](#)
 - [Installing Laravel Echo](#)
 - [Listening For Events](#)
 - [Leaving A Channel](#)
 - [Namespaces](#)
- [Presence Channels](#)
 - [Authorizing Presence Channels](#)
 - [Joining Presence Channels](#)
 - [Broadcasting To Presence Channels](#)
- [Client Events](#)
- [Notifications](#)

Introduction

In many modern web applications, WebSockets are used to implement realtime, live-updating user interfaces. When some data is updated on the server, a message is typically sent over a WebSocket connection to be handled by the client. This provides a more robust, efficient alternative to continually polling your application for changes.

To assist you in building these types of applications, Laravel makes it easy to "broadcast" your [events](#) over a WebSocket connection. Broadcasting your Laravel events allows you to share the same event names between your server-side code and your client-side JavaScript application.

TIP Before diving into event broadcasting, make sure you have read all of the documentation regarding Laravel [events and listeners](#).

Configuration

All of your application's event broadcasting configuration is stored in the `config/broadcasting.php` configuration file. Laravel supports several broadcast drivers out of the box: [Pusher Channels](#), [Redis](#), and a `log` driver for local development and debugging. Additionally, a `null` driver is included which allows you to totally disable broadcasting. A configuration example is included for each of these drivers in the `config/broadcasting.php` configuration file.

Broadcast Service Provider

Before broadcasting any events, you will first need to register the `App\Providers\BroadcastServiceProvider`. In fresh Laravel applications, you only need to uncomment this provider in the `providers` array of your `config/app.php` configuration file. This provider will allow you to register the broadcast authorization routes and callbacks.

CSRF Token

[Laravel Echo](#) will need access to the current session's CSRF token. You should verify that your application's head HTML element defines a meta tag containing the CSRF token:

```
<meta name="csrf-token" content="{ {{ csrf_token() }}">
```

Driver Prerequisites

Pusher Channels

If you are broadcasting your events over [Pusher Channels](#), you should install the Pusher Channels PHP SDK using the Composer package manager:

```
composer require pusher/pusher-php-server "~4.0"
```

Next, you should configure your Channels credentials in the `config/broadcasting.php` configuration file. An example Channels configuration is already included in this file, allowing you to quickly specify your Channels key, secret, and application ID. The `config/broadcasting.php` file's pusher configuration also allows you to specify additional options that are supported by Channels, such as the cluster:

```
'options' => [
    'cluster' => 'eu',
    'useTLS' => true
],
```

When using Channels and [Laravel Echo](#), you should specify pusher as your desired broadcaster when instantiating the Echo instance in your `resources/js/bootstrap.js` file:

```
import Echo from "laravel-echo";

window.Pusher = require('pusher-js');

window.Echo = new Echo({
    broadcaster: 'pusher',
    key: 'your-pusher-channels-key'
});
```

Finally, you will need to change your broadcast driver to pusher in your `.env` file:

```
BROADCAST_DRIVER=pusher
```

Pusher Compatible Laravel Websockets

The [laravel-websockets](#) is a pure PHP, Pusher compatible websocket package for Laravel. This package allows you to leverage the full power of Laravel broadcasting without an external websocket provider or Node. For more information on installing and using this package, please consult its [official documentation](#).

Redis

If you are using the Redis broadcaster, you should either install the `phpredis` PHP extension via PECL or install the `Predis` library via Composer:

```
composer require predis/predis
```

Next, you should update your broadcast driver to `redis` in your `.env` file:

```
BROADCAST_DRIVER=redis
```

The Redis broadcaster will broadcast messages using Redis' `pub / sub` feature; however, you will need to pair

this with a WebSocket server that can receive the messages from Redis and broadcast them to your WebSocket channels.

When the Redis broadcaster publishes an event, it will be published on the event's specified channel names and the payload will be a JSON encoded string containing the event name, a data payload, and the user that generated the event's socket ID (if applicable).

Socket.IO

If you are going to pair the Redis broadcaster with a Socket.IO server, you will need to include the Socket.IO JavaScript client library in your application. You may install it via the NPM package manager:

```
npm install --save socket.io-client
```

Next, you will need to instantiate Echo with the `socket.io` connector and a host.

```
import Echo from "laravel-echo"

window.io = require('socket.io-client');

window.Echo = new Echo({
  broadcaster: 'socket.io',
  host: window.location.hostname + ':6001'
});
```

Finally, you will need to run a compatible Socket.IO server. Laravel does not include a Socket.IO server implementation; however, a community driven Socket.IO server is currently maintained at the [tlaverdure/laravel-echo-server](#) GitHub repository.

Queue Prerequisites

Before broadcasting events, you will also need to configure and run a [queue listener](#). All event broadcasting is done via queued jobs so that the response time of your application is not seriously affected.

Concept Overview

Laravel's event broadcasting allows you to broadcast your server-side Laravel events to your client-side JavaScript application using a driver-based approach to WebSockets. Currently, Laravel ships with [Pusher Channels](#) and Redis drivers. The events may be easily consumed on the client-side using the [Laravel Echo](#) Javascript package.

Events are broadcast over "channels", which may be specified as public or private. Any visitor to your application may subscribe to a public channel without any authentication or authorization; however, in order to subscribe to a private channel, a user must be authenticated and authorized to listen on that channel.

TIP If you would like to use an open source, PHP driven alternative to Pusher, check out the [laravel-websockets](#) package.

Using An Example Application

Before diving into each component of event broadcasting, let's take a high level overview using an e-commerce store as an example. We won't discuss the details of configuring [Pusher Channels](#) or [Laravel Echo](#) since that will be discussed in detail in other sections of this documentation.

In our application, let's assume we have a page that allows users to view the shipping status for their orders. Let's also assume that a `ShippingStatusUpdated` event is fired when a shipping status update is processed by the application:

```
event(new ShippingStatusUpdated($update));
```

The ShouldBroadcast Interface

When a user is viewing one of their orders, we don't want them to have to refresh the page to view status updates. Instead, we want to broadcast the updates to the application as they are created. So, we need to mark the `ShippingStatusUpdated` event with the `ShouldBroadcast` interface. This will instruct Laravel to broadcast the event when it is fired:

```
<?php

namespace App\Events;

use Illuminate\Broadcasting\Channel;
use Illuminate\Broadcasting\InteractsWithSockets;
use Illuminate\Broadcasting\PresenceChannel;
use Illuminate\Broadcasting\PrivateChannel;
use Illuminate\Contracts\Broadcasting\ShouldBroadcast;
use Illuminate\Queue\SerializesModels;

class ShippingStatusUpdated implements ShouldBroadcast
{
    /**
     * Information about the shipping status update.
     *
     * @var string
     */
    public $update;
}
```

The `ShouldBroadcast` interface requires our event to define a `broadcastOn` method. This method is responsible for returning the channels that the event should broadcast on. An empty stub of this method is already defined on generated event classes, so we only need to fill in its details. We only want the creator of the order to be able to view status updates, so we will broadcast the event on a private channel that is tied to the order:

```
/**
 * Get the channels the event should broadcast on.
 *
 * @return \Illuminate\Broadcasting\PrivateChannel
 */
public function broadcastOn()
{
    return new PrivateChannel('order.'.$this->update->order_id);
}
```

Authorizing Channels

Remember, users must be authorized to listen on private channels. We may define our channel authorization rules in the `routes/channels.php` file. In this example, we need to verify that any user attempting to listen on the private `order.1` channel is actually the creator of the order:

```
Broadcast::channel('order.{orderId}', function ($user, $orderId) {
    return $user->id === Order::findOrNew($orderId)->user_id;
});
```

The `channel` method accepts two arguments: the name of the channel and a callback which returns `true` or `false` indicating whether the user is authorized to listen on the channel.

All authorization callbacks receive the currently authenticated user as their first argument and any additional wildcard parameters as their subsequent arguments. In this example, we are using the `{orderId}` placeholder to indicate that the "ID" portion of the channel name is a wildcard.

Listening For Event Broadcasts

Next, all that remains is to listen for the event in our JavaScript application. We can do this using Laravel Echo. First, we'll use the `private` method to subscribe to the private channel. Then, we may use the `listen` method to listen for the `ShippingStatusUpdated` event. By default, all of the event's public properties will be included on the broadcast event:

```
Echo.private(`order.${orderId}`)
    .listen('ShippingStatusUpdated', (e) => {
        console.log(e.update);
    });
```

Defining Broadcast Events

To inform Laravel that a given event should be broadcast, implement the `Illuminate\Contracts\Broadcasting\ShouldBroadcast` interface on the event class. This interface is already imported into all event classes generated by the framework so you may easily add it to any of your events.

The `ShouldBroadcast` interface requires you to implement a single method: `broadcastOn`. The `broadcastOn` method should return a channel or array of channels that the event should broadcast on. The channels should be instances of `Channel`, `PrivateChannel`, or `PresenceChannel`. Instances of `Channel` represent public channels that any user may subscribe to, while `PrivateChannels` and `PresenceChannels` represent private channels that require [channel authorization](#):

```
<?php

namespace App\Events;

use App\User;
use Illuminate\Broadcasting\Channel;
use Illuminate\Broadcasting\InteractsWithSockets;
use Illuminate\Broadcasting\PresenceChannel;
use Illuminate\Broadcasting\PrivateChannel;
use Illuminate\Contracts\Broadcasting\ShouldBroadcast;
use Illuminate\Queue\SerializesModels;

class ServerCreated implements ShouldBroadcast
{
    use SerializesModels;

    public $user;

    /**
     * Create a new event instance.
     *
     * @return void
     */
    public function __construct(User $user)
    {
        $this->user = $user;
    }

    /**
     * Get the channels the event should broadcast on.
     *
     * @return Channel|array
     */
    public function broadcastOn()
    {
        return new PrivateChannel('user.'.$this->user->id);
    }
}
```

Then, you only need to [fire the event](#) as you normally would. Once the event has been fired, a [queued job](#) will automatically broadcast the event over your specified broadcast driver.

Broadcast Name

By default, Laravel will broadcast the event using the event's class name. However, you may customize the broadcast name by defining a `broadcastAs` method on the event:

```
/**
 * The event's broadcast name.
 *
 * @return string
 */
public function broadcastAs()
{
    return 'server.created';
}
```

If you customize the broadcast name using the `broadcastAs` method, you should make sure to register your listener with a leading `.` character. This will instruct Echo to not prepend the application's namespace to the event:

```
.listen('server.created', function (e) {
    ....
});
```

Broadcast Data

When an event is broadcast, all of its public properties are automatically serialized and broadcast as the event's payload, allowing you to access any of its public data from your JavaScript application. So, for example, if your event has a single public `$user` property that contains an Eloquent model, the event's broadcast payload would be:

```
{
  "user": {
    "id": 1,
    "name": "Patrick Stewart"
    ...
  }
}
```

However, if you wish to have more fine-grained control over your broadcast payload, you may add a `broadcastWith` method to your event. This method should return the array of data that you wish to broadcast as the event payload:

```
/**
 * Get the data to broadcast.
 *
 * @return array
 */
public function broadcastWith()
{
    return ['id' => $this->user->id];
}
```

Broadcast Queue

By default, each broadcast event is placed on the default queue for the default queue connection specified in your `queue.php` configuration file. You may customize the queue used by the broadcaster by defining a `broadcastQueue` property on your event class. This property should specify the name of the queue you wish to use when broadcasting:

```
/**
 * The name of the queue on which to place the event.
 *
 * @var string
 */
public $broadcastQueue = 'your-queue-name';
```

If you want to broadcast your event using the `sync` queue instead of the default queue driver, you can implement the `ShouldBroadcastNow` interface instead of `ShouldBroadcast`:

```
<?php

use Illuminate\Contracts\Broadcasting\ShouldBroadcastNow;

class ShippingStatusUpdated implements ShouldBroadcastNow
{
    //
}
```

Broadcast Conditions

Sometimes you want to broadcast your event only if a given condition is true. You may define these conditions by adding a `broadcastWhen` method to your event class:

```
/**
 * Determine if this event should broadcast.
 *
 * @return bool
 */
public function broadcastWhen()
```

```
{
    return $this->value > 100;
}
```

Authorizing Channels

Private channels require you to authorize that the currently authenticated user can actually listen on the channel. This is accomplished by making an HTTP request to your Laravel application with the channel name and allowing your application to determine if the user can listen on that channel. When using [Laravel Echo](#), the HTTP request to authorize subscriptions to private channels will be made automatically; however, you do need to define the proper routes to respond to these requests.

Defining Authorization Routes

Thankfully, Laravel makes it easy to define the routes to respond to channel authorization requests. In the `BroadcastServiceProvider` included with your Laravel application, you will see a call to the `Broadcast::routes` method. This method will register the `/broadcasting/auth` route to handle authorization requests:

```
Broadcast::routes();
```

The `Broadcast::routes` method will automatically place its routes within the `web` middleware group; however, you may pass an array of route attributes to the method if you would like to customize the assigned attributes:

```
Broadcast::routes($attributes);
```

Customizing The Authorization Endpoint

By default, Echo will use the `/broadcasting/auth` endpoint to authorize channel access. However, you may specify your own authorization endpoint by passing the `authEndpoint` configuration option to your Echo instance:

```
window.Echo = new Echo({
    broadcaster: 'pusher',
    key: 'your-pusher-channels-key',
    authEndpoint: '/custom/endpoint/auth'
});
```

Defining Authorization Callbacks

Next, we need to define the logic that will actually perform the channel authorization. This is done in the `routes/channels.php` file that is included with your application. In this file, you may use the `Broadcast::channel` method to register channel authorization callbacks:

```
Broadcast::channel('order.{orderId}', function ($user, $orderId) {
    return $user->id === Order::findOrNew($orderId)->user_id;
});
```

The `channel` method accepts two arguments: the name of the channel and a callback which returns `true` or `false` indicating whether the user is authorized to listen on the channel.

All authorization callbacks receive the currently authenticated user as their first argument and any additional wildcard parameters as their subsequent arguments. In this example, we are using the `{orderId}` placeholder to indicate that the "ID" portion of the channel name is a wildcard.

Authorization Callback Model Binding

Just like HTTP routes, channel routes may also take advantage of implicit and explicit [route model binding](#). For example, instead of receiving the string or numeric order ID, you may request an actual `Order` model instance:

```
use App\Order;

Broadcast::channel('order.{order}', function ($user, Order $order) {
    return $user->id === $order->user_id;
});
```

Authorization Callback Authentication

Private and presence broadcast channels authenticate the current user via your application's default authentication guard. If the user is not authenticated, channel authorization is automatically denied and the authorization callback is never executed. However, you may assign multiple, custom guards that should authenticate the incoming request if necessary:

```
Broadcast::channel('channel', function () {
    // ...
}, ['guards' => ['web', 'admin']]);
```

Defining Channel Classes

If your application is consuming many different channels, your `routes/channels.php` file could become bulky. So, instead of using Closures to authorize channels, you may use channel classes. To generate a channel class, use the `make:channel` Artisan command. This command will place a new channel class in the `App/Broadcasting` directory.

```
php artisan make:channel OrderChannel
```

Next, register your channel in your `routes/channels.php` file:

```
use App\Broadcasting\OrderChannel;

Broadcast::channel('order.{order}', OrderChannel::class);
```

Finally, you may place the authorization logic for your channel in the channel class' `join` method. This `join` method will house the same logic you would have typically placed in your channel authorization Closure. You may also take advantage of channel model binding:

```
<?php

namespace App\Broadcasting;

use App\Order;
use App\User;

class OrderChannel
{
    /**
     * Create a new channel instance.
     *
     * @return void
     */
    public function __construct()
    {
        //
    }

    /**
     * Authenticate the user's access to the channel.
     *
     * @param \App\User $user
     * @param \App\Order $order
     * @return array|bool
     */
    public function join(User $user, Order $order)
    {
        return $user->id === $order->user_id;
    }
}
```

TIP Like many other classes in Laravel, channel classes will automatically be resolved by the [service container](#). So, you may type-hint any dependencies required by your channel in its constructor.

Broadcasting Events

Once you have defined an event and marked it with the `ShouldBroadcast` interface, you only need to fire the event using the `event` function. The event dispatcher will notice that the event is marked with the `ShouldBroadcast` interface and will queue the event for broadcasting:


```
event(new ShippingStatusUpdated($update));
```

Only To Others

When building an application that utilizes event broadcasting, you may substitute the event function with the broadcast function. Like the event function, the broadcast function dispatches the event to your server-side listeners:

```
broadcast(new ShippingStatusUpdated($update));
```

However, the broadcast function also exposes the `toOthers` method which allows you to exclude the current user from the broadcast's recipients:

```
broadcast(new ShippingStatusUpdated($update))->toOthers();
```

To better understand when you may want to use the `toOthers` method, let's imagine a task list application where a user may create a new task by entering a task name. To create a task, your application might make a request to a `/task` end-point which broadcasts the task's creation and returns a JSON representation of the new task. When your JavaScript application receives the response from the end-point, it might directly insert the new task into its task list like so:

```
axios.post('/task', task)
  .then((response) => {
    this.tasks.push(response.data);
  });
```

However, remember that we also broadcast the task's creation. If your JavaScript application is listening for this event in order to add tasks to the task list, you will have duplicate tasks in your list: one from the end-point and one from the broadcast. You may solve this by using the `toOthers` method to instruct the broadcaster to not broadcast the event to the current user.

NOTE Your event must use the `Illuminate\Broadcasting\InteractsWithSockets` trait in order to call the `toOthers` method.

Configuration

When you initialize a Laravel Echo instance, a socket ID is assigned to the connection. If you are using [Vue](#) and [Axios](#), the socket ID will automatically be attached to every outgoing request as a `x-socket-id` header. Then, when you call the `toOthers` method, Laravel will extract the socket ID from the header and instruct the broadcaster to not broadcast to any connections with that socket ID.

If you are not using Vue and Axios, you will need to manually configure your JavaScript application to send the `x-socket-id` header. You may retrieve the socket ID using the `Echo.socketId` method:

```
var socketId = Echo.socketId();
```

Receiving Broadcasts

Installing Laravel Echo

Laravel Echo is a JavaScript library that makes it painless to subscribe to channels and listen for events broadcast by Laravel. You may install Echo via the NPM package manager. In this example, we will also install the `pusher-js` package since we will be using the Pusher Channels broadcaster:

```
npm install --save laravel-echo pusher-js
```

Once Echo is installed, you are ready to create a fresh Echo instance in your application's JavaScript. A great place to do this is at the bottom of the `resources/js/bootstrap.js` file that is included with the Laravel framework:

```
import Echo from "laravel-echo"

window.Echo = new Echo({
  broadcaster: 'pusher',
```

```
    key: 'your-pusher-channels-key'
  });
```

When creating an Echo instance that uses the `pusher` connector, you may also specify a `cluster` as well as whether the connection must be made over TLS (by default, when `forceTLS` is `false`, a non-TLS connection will be made if the page was loaded over HTTP, or as a fallback if a TLS connection fails):

```
window.Echo = new Echo({
  broadcaster: 'pusher',
  key: 'your-pusher-channels-key',
  cluster: 'eu',
  forceTLS: true
});
```

Using An Existing Client Instance

If you already have a Pusher Channels or Socket.io client instance that you would like Echo to utilize, you may pass it to Echo via the `client` configuration option:

```
const client = require('pusher-js');

window.Echo = new Echo({
  broadcaster: 'pusher',
  key: 'your-pusher-channels-key',
  client: client
});
```

Listening For Events

Once you have installed and instantiated Echo, you are ready to start listening for event broadcasts. First, use the `channel` method to retrieve an instance of a channel, then call the `listen` method to listen for a specified event:

```
Echo.channel('orders')
  .listen('OrderShipped', (e) => {
    console.log(e.order.name);
  });
```

If you would like to listen for events on a private channel, use the `private` method instead. You may continue to chain calls to the `listen` method to listen for multiple events on a single channel:

```
Echo.private('orders')
  .listen(...)
  .listen(...)
  .listen(...);
```

Leaving A Channel

To leave a channel, you may call the `leaveChannel` method on your Echo instance:

```
Echo.leaveChannel('orders');
```

If you would like to leave a channel and also its associated private and presence channels, you may call the `leave` method:

```
Echo.leave('orders');
```

Namespaces

You may have noticed in the examples above that we did not specify the full namespace for the event classes. This is because Echo will automatically assume the events are located in the `App\Events` namespace. However, you may configure the root namespace when you instantiate Echo by passing a `namespace` configuration option:

```
window.Echo = new Echo({
  broadcaster: 'pusher',
  key: 'your-pusher-channels-key',
  namespace: 'App.Other.Namespace'
});
```

Alternatively, you may prefix event classes with a `.` when subscribing to them using Echo. This will allow you to always specify the fully-qualified class name:

```
Echo.channel('orders')
    .listen('.Namespace\\Event\\Class', (e) => {
        //
    });
```

Presence Channels

Presence channels build on the security of private channels while exposing the additional feature of awareness of who is subscribed to the channel. This makes it easy to build powerful, collaborative application features such as notifying users when another user is viewing the same page.

Authorizing Presence Channels

All presence channels are also private channels; therefore, users must be [authorized to access them](#). However, when defining authorization callbacks for presence channels, you will not return `true` if the user is authorized to join the channel. Instead, you should return an array of data about the user.

The data returned by the authorization callback will be made available to the presence channel event listeners in your JavaScript application. If the user is not authorized to join the presence channel, you should return `false` or `null`:

```
Broadcast::channel('chat.{roomId}', function ($user, $roomId) {
    if ($user->canJoinRoom($roomId)) {
        return ['id' => $user->id, 'name' => $user->name];
    }
});
```

Joining Presence Channels

To join a presence channel, you may use Echo's `join` method. The `join` method will return a `PresenceChannel` implementation which, along with exposing the `listen` method, allows you to subscribe to the `here`, `joining`, and `leaving` events.

```
Echo.join(`chat.${roomId}`)
    .here((users) => {
        //
    })
    .joining((user) => {
        console.log(user.name);
    })
    .leaving((user) => {
        console.log(user.name);
    });
```

The `here` callback will be executed immediately once the channel is joined successfully, and will receive an array containing the user information for all of the other users currently subscribed to the channel. The `joining` method will be executed when a new user joins a channel, while the `leaving` method will be executed when a user leaves the channel.

Broadcasting To Presence Channels

Presence channels may receive events just like public or private channels. Using the example of a chatroom, we may want to broadcast `NewMessage` events to the room's presence channel. To do so, we'll return an instance of `PresenceChannel` from the event's `broadcastOn` method:

```
/**
 * Get the channels the event should broadcast on.
 *
 * @return Channel|array
 */
public function broadcastOn()
{
    return new PresenceChannel('room.' . $this->message->room_id);
}
```

```
}
```

Like public or private events, presence channel events may be broadcast using the `broadcast` function. As with other events, you may use the `toOthers` method to exclude the current user from receiving the broadcast:

```
broadcast(new NewMessage($message));

broadcast(new NewMessage($message))->toOthers();
```

You may listen for the join event via Echo's `listen` method:

```
Echo.join(`chat.${roomId}`)
    .here(...)
    .joining(...)
    .leaving(...)
    .listen('NewMessage', (e) => {
        //
    });
```

Client Events

TIP When using [Pusher Channels](#), you must enable the "Client Events" option in the "App Settings" section of your [application dashboard](#) in order to send client events.

Sometimes you may wish to broadcast an event to other connected clients without hitting your Laravel application at all. This can be particularly useful for things like "typing" notifications, where you want to alert users of your application that another user is typing a message on a given screen.

To broadcast client events, you may use Echo's `whisper` method:

```
Echo.private('chat')
    .whisper('typing', {
        name: this.user.name
    });
```

To listen for client events, you may use the `listenForWhisper` method:

```
Echo.private('chat')
    .listenForWhisper('typing', (e) => {
        console.log(e.name);
    });
```

Notifications

By pairing event broadcasting with [notifications](#), your JavaScript application may receive new notifications as they occur without needing to refresh the page. First, be sure to read over the documentation on using [the broadcast notification channel](#).

Once you have configured a notification to use the broadcast channel, you may listen for the broadcast events using Echo's `notification` method. Remember, the channel name should match the class name of the entity receiving the notifications:

```
Echo.private(`App.User.${userId}`)
    .notification((notification) => {
        console.log(notification.type);
    });
```

In this example, all notifications sent to `App\User` instances via the `broadcast` channel would be received by the callback. A channel authorization callback for the `App\User.{id}` channel is included in the default `BroadcastServiceProvider` that ships with the Laravel framework.

Digging Deeper

Cache

Cache

- [Configuration](#)
 - [Driver Prerequisites](#)
- [Cache Usage](#)
 - [Obtaining A Cache Instance](#)
 - [Retrieving Items From The Cache](#)
 - [Storing Items In The Cache](#)
 - [Removing Items From The Cache](#)
 - [The Cache Helper](#)
- [Cache Tags](#)
 - [Storing Tagged Cache Items](#)
 - [Accessing Tagged Cache Items](#)
 - [Removing Tagged Cache Items](#)
- [Atomic Locks](#)
 - [Driver Prerequisites](#)
 - [Managing Locks](#)
 - [Managing Locks Across Processes](#)
- [Adding Custom Cache Drivers](#)
 - [Writing The Driver](#)
 - [Registering The Driver](#)
- [Events](#)

Configuration

Laravel provides an expressive, unified API for various caching backends. The cache configuration is located at `config/cache.php`. In this file you may specify which cache driver you would like to be used by default throughout your application. Laravel supports popular caching backends like [Memcached](#) and [Redis](#) out of the box.

The cache configuration file also contains various other options, which are documented within the file, so make sure to read over these options. By default, Laravel is configured to use the `file` cache driver, which stores the serialized, cached objects in the filesystem. For larger applications, it is recommended that you use a more robust driver such as Memcached or Redis. You may even configure multiple cache configurations for the same driver.

Driver Prerequisites

Database

When using the database cache driver, you will need to setup a table to contain the cache items. You'll find an example schema declaration for the table below:

```
Schema::create('cache', function ($table) {
    $table->string('key')->unique();
    $table->text('value');
    $table->integer('expiration');
});
```

TIP You may also use the `php artisan cache:table` Artisan command to generate a migration with the proper schema.

Memcached

Using the Memcached driver requires the [Memcached PECL package](#) to be installed. You may list all of your Memcached servers in the `config/cache.php` configuration file:

```
'memcached' => [
    [
        'host' => '127.0.0.1',
        'port' => 11211,
        'weight' => 100
    ],
],
```

You may also set the `host` option to a UNIX socket path. If you do this, the `port` option should be set to `0`:

```
'memcached' => [
    [
        'host' => '/var/run/memcached/memcached.sock',
        'port' => 0,
        'weight' => 100
    ],
],
```

Redis

Before using a Redis cache with Laravel, you will need to either install the PhpRedis PHP extension via PECL or install the `predis/predis` package (~1.0) via Composer.

For more information on configuring Redis, consult its [Laravel documentation page](#).

Cache Usage

Obtaining A Cache Instance

The `Illuminate\Contracts\Cache\Factory` and `Illuminate\Contracts\Cache\Repository` [contracts](#) provide access to Laravel's cache services. The `Factory` contract provides access to all cache drivers defined for your application. The `Repository` contract is typically an implementation of the default cache driver for your application as specified by your cache configuration file.

However, you may also use the cache facade, which is what we will use throughout this documentation. The cache facade provides convenient, terse access to the underlying implementations of the Laravel cache contracts:

```
<?php

namespace App\Http\Controllers;

use Illuminate\Support\Facades\Cache;

class UserController extends Controller
{
    /**
     * Show a list of all users of the application.
     *
     * @return Response
     */
    public function index()
    {
        $value = Cache::get('key');

        //
    }
}
```

Accessing Multiple Cache Stores

Using the cache facade, you may access various cache stores via the `store` method. The key passed to the `store` method should correspond to one of the stores listed in the `stores` configuration array in your cache configuration file:

```
$value = Cache::store('file')->get('foo');

Cache::store('redis')->put('bar', 'baz', 600); // 10 Minutes
```

Retrieving Items From The Cache

The `get` method on the `cache` facade is used to retrieve items from the cache. If the item does not exist in the cache, `null` will be returned. If you wish, you may pass a second argument to the `get` method specifying the default value you wish to be returned if the item doesn't exist:

```
$value = Cache::get('key');  
  
$value = Cache::get('key', 'default');
```

You may even pass a `closure` as the default value. The result of the `closure` will be returned if the specified item does not exist in the cache. Passing a `Closure` allows you to defer the retrieval of default values from a database or other external service:

```
$value = Cache::get('key', function () {  
    return DB::table(...)->get();  
});
```

Checking For Item Existence

The `has` method may be used to determine if an item exists in the cache. This method will return `false` if the value is `null`:

```
if (Cache::has('key')) {  
    //  
}
```

Incrementing / Decrementing Values

The `increment` and `decrement` methods may be used to adjust the value of integer items in the cache. Both of these methods accept an optional second argument indicating the amount by which to increment or decrement the item's value:

```
Cache::increment('key');  
Cache::increment('key', $amount);  
Cache::decrement('key');  
Cache::decrement('key', $amount);
```

Retrieve & Store

Sometimes you may wish to retrieve an item from the cache, but also store a default value if the requested item doesn't exist. For example, you may wish to retrieve all users from the cache or, if they don't exist, retrieve them from the database and add them to the cache. You may do this using the `cache::remember` method:

```
$value = Cache::remember('users', $seconds, function () {  
    return DB::table('users')->get();  
});
```

If the item does not exist in the cache, the `closure` passed to the `remember` method will be executed and its result will be placed in the cache.

You may use the `rememberForever` method to retrieve an item from the cache or store it forever:

```
$value = Cache::rememberForever('users', function () {  
    return DB::table('users')->get();  
});
```

Retrieve & Delete

If you need to retrieve an item from the cache and then delete the item, you may use the `pull` method. Like the `get` method, `null` will be returned if the item does not exist in the cache:

```
$value = Cache::pull('key');
```

Storing Items In The Cache

You may use the `put` method on the cache facade to store items in the cache:

```
Cache::put('key', 'value', $seconds);
```

If the storage time is not passed to the `put` method, the item will be stored indefinitely:

```
Cache::put('key', 'value');
```

Instead of passing the number of seconds as an integer, you may also pass a `DateTime` instance representing the expiration time of the cached item:

```
Cache::put('key', 'value', now()->addMinutes(10));
```

Store If Not Present

The `add` method will only add the item to the cache if it does not already exist in the cache store. The method will return `true` if the item is actually added to the cache. Otherwise, the method will return `false`:

```
Cache::add('key', 'value', $seconds);
```

Storing Items Forever

The `forever` method may be used to store an item in the cache permanently. Since these items will not expire, they must be manually removed from the cache using the `forget` method:

```
Cache::forever('key', 'value');
```

TIP If you are using the Memcached driver, items that are stored "forever" may be removed when the cache reaches its size limit.

Removing Items From The Cache

You may remove items from the cache using the `forget` method:

```
Cache::forget('key');
```

You may also remove items by providing a zero or negative TTL:

```
Cache::put('key', 'value', 0);
```

```
Cache::put('key', 'value', -5);
```

You may clear the entire cache using the `flush` method:

```
Cache::flush();
```

NOTE Flushing the cache does not respect the cache prefix and will remove all entries from the cache. Consider this carefully when clearing a cache which is shared by other applications.

The Cache Helper

In addition to using the cache facade or [cache contract](#), you may also use the global `cache` function to retrieve and store data via the cache. When the `cache` function is called with a single, string argument, it will return the value of the given key:

```
$value = cache('key');
```

If you provide an array of key / value pairs and an expiration time to the function, it will store values in the cache for the specified duration:

```
cache(['key' => 'value'], $seconds);
```

```
cache(['key' => 'value'], now()->addMinutes(10));
```

When the `cache` function is called without any arguments, it returns an instance of the

Illuminate\Contracts\Cache\Factory implementation, allowing you to call other caching methods:

```
cache()->remember('users', $seconds, function () {  
    return DB::table('users')->get();  
});
```

TIP When testing call to the global cache function, you may use the `Cache::shouldReceive` method just as if you were [testing a facade](#).

Cache Tags

NOTE Cache tags are not supported when using the `file`, `dynamodb`, or `database` cache drivers. Furthermore, when using multiple tags with caches that are stored "forever", performance will be best with a driver such as `memcached`, which automatically purges stale records.

Storing Tagged Cache Items

Cache tags allow you to tag related items in the cache and then flush all cached values that have been assigned a given tag. You may access a tagged cache by passing in an ordered array of tag names. For example, let's access a tagged cache and put value in the cache:

```
Cache::tags(['people', 'artists']->put('John', $john, $seconds);  
Cache::tags(['people', 'authors']->put('Anne', $anne, $seconds);
```

Accessing Tagged Cache Items

To retrieve a tagged cache item, pass the same ordered list of tags to the `tags` method and then call the `get` method with the key you wish to retrieve:

```
$john = Cache::tags(['people', 'artists']->get('John');  
$anne = Cache::tags(['people', 'authors']->get('Anne');
```

Removing Tagged Cache Items

You may flush all items that are assigned a tag or list of tags. For example, this statement would remove all caches tagged with either `people`, `authors`, or both. So, both Anne and John would be removed from the cache:

```
Cache::tags(['people', 'authors']->flush();
```

In contrast, this statement would remove only caches tagged with `authors`, so Anne would be removed, but not John:

```
Cache::tags('authors')->flush();
```

Atomic Locks

NOTE To utilize this feature, your application must be using the `memcached`, `dynamodb`, `redis`, `database`, or `array` cache driver as your application's default cache driver. In addition, all servers must be communicating with the same central cache server.

Driver Prerequisites

Database

When using the `database` cache driver, you will need to setup a table to contain the cache locks. You'll find an example schema declaration for the table below:

```
Schema::create('cache_locks', function ($table) {  
    $table->string('key')->primary();  
    $table->string('owner');
```

```
$table->integer('expiration');
});
```

Managing Locks

Atomic locks allow for the manipulation of distributed locks without worrying about race conditions. For example, [Laravel Forge](#) uses atomic locks to ensure that only one remote task is being executed on a server at a time. You may create and manage locks using the `cache::lock` method:

```
use Illuminate\Support\Facades\Cache;

$lock = Cache::lock('foo', 10);

if ($lock->get()) {
    // Lock acquired for 10 seconds...

    $lock->release();
}
```

The `get` method also accepts a Closure. After the Closure is executed, Laravel will automatically release the lock:

```
Cache::lock('foo')->get(function () {
    // Lock acquired indefinitely and automatically released...
});
```

If the lock is not available at the moment you request it, you may instruct Laravel to wait for a specified number of seconds. If the lock can not be acquired within the specified time limit, an `Illuminate\Contracts\Cache\LockTimeoutException` will be thrown:

```
use Illuminate\Contracts\Cache\LockTimeoutException;

$lock = Cache::lock('foo', 10);

try {
    $lock->block(5);

    // Lock acquired after waiting maximum of 5 seconds...
} catch (LockTimeoutException $e) {
    // Unable to acquire lock...
} finally {
    optional($lock)->release();
}

Cache::lock('foo', 10)->block(5, function () {
    // Lock acquired after waiting maximum of 5 seconds...
});
```

Managing Locks Across Processes

Sometimes, you may wish to acquire a lock in one process and release it in another process. For example, you may acquire a lock during a web request and wish to release the lock at the end of a queued job that is triggered by that request. In this scenario, you should pass the lock's scoped "owner token" to the queued job so that the job can re-instantiate the lock using the given token:

```
// Within Controller...
$podcast = Podcast::find($id);

$lock = Cache::lock('foo', 120);

if ($result = $lock->get()) {
    ProcessPodcast::dispatch($podcast, $lock->owner());
}

// Within ProcessPodcast Job...
Cache::restoreLock('foo', $this->owner)->release();
```

If you would like to release a lock without respecting its current owner, you may use the `forceRelease` method:

```
Cache::lock('foo')->forceRelease();
```

Adding Custom Cache Drivers

Writing The Driver

To create our custom cache driver, we first need to implement the `Illuminate\Contracts\Cache\Store` [contract](#). So, a MongoDB cache implementation would look something like this:

```
<?php

namespace App\Extensions;

use Illuminate\Contracts\Cache\Store;

class MongoStore implements Store
{
    public function get($key) {}
    public function many(array $keys) {}
    public function put($key, $value, $seconds) {}
    public function putMany(array $values, $seconds) {}
    public function increment($key, $value = 1) {}
    public function decrement($key, $value = 1) {}
    public function forever($key, $value) {}
    public function forget($key) {}
    public function flush() {}
    public function getPrefix() {}
}
```

We just need to implement each of these methods using a MongoDB connection. For an example of how to implement each of these methods, take a look at the `Illuminate\Cache\MemcachedStore` in the framework source code. Once our implementation is complete, we can finish our custom driver registration.

```
Cache::extend('mongo', function ($app) {
    return Cache::repository(new MongoStore);
});
```

TIP If you're wondering where to put your custom cache driver code, you could create an `Extensions` namespace within your app directory. However, keep in mind that Laravel does not have a rigid application structure and you are free to organize your application according to your preferences.

Registering The Driver

To register the custom cache driver with Laravel, we will use the `extend` method on the cache facade. The call to `Cache::extend` could be done in the `boot` method of the default `App\Providers\AppServiceProvider` that ships with fresh Laravel applications, or you may create your own service provider to house the extension - just don't forget to register the provider in the `config/app.php` provider array:

```
<?php

namespace App\Providers;

use App\Extensions\MongoStore;
use Illuminate\Support\Facades\Cache;
use Illuminate\Support\ServiceProvider;

class CacheServiceProvider extends ServiceProvider
{
    /**
     * Register any application services.
     *
     * @return void
     */
    public function register()
    {
        //
    }

    /**
     * Bootstrap any application services.
     *
     * @return void
     */
    public function boot()
```

```
{
    Cache::extend('mongo', function ($app) {
        return Cache::repository(new MongoStore);
    });
}
```

The first argument passed to the `extend` method is the name of the driver. This will correspond to your `driver` option in the `config/cache.php` configuration file. The second argument is a Closure that should return an `Illuminate\Cache\Repository` instance. The Closure will be passed an `$app` instance, which is an instance of the [service container](#).

Once your extension is registered, update your `config/cache.php` configuration file's `driver` option to the name of your extension.

Events

To execute code on every cache operation, you may listen for the [events](#) fired by the cache. Typically, you should place these event listeners within your `EventServiceProvider`:

```
/**
 * The event listener mappings for the application.
 *
 * @var array
 */
protected $listen = [
    'Illuminate\Cache\Events\CacheHit' => [
        'App\Listeners\LogCacheHit',
    ],
    'Illuminate\Cache\Events\CacheMissed' => [
        'App\Listeners\LogCacheMissed',
    ],
    'Illuminate\Cache\Events\KeyForgotten' => [
        'App\Listeners\LogKeyForgotten',
    ],
    'Illuminate\Cache\Events\KeyWritten' => [
        'App\Listeners\LogKeyWritten',
    ],
];
```

Digging Deeper

Collections

Collections

- [Introduction](#)
 - [Creating Collections](#)
 - [Extending Collections](#)
- [Available Methods](#)
- [Higher Order Messages](#)
- [Lazy Collections](#)
 - [Introduction](#)
 - [Creating Lazy Collections](#)
 - [The Enumerable Contract](#)
 - [Lazy Collection Methods](#)

Introduction

The `Illuminate\Support\Collection` class provides a fluent, convenient wrapper for working with arrays of data. For example, check out the following code. We'll use the `collect` helper to create a new collection instance from the array, run the `strtoupper` function on each element, and then remove all empty elements:

```
$collection = collect(['taylor', 'abigail', null])->map(function ($name) {
    return strtoupper($name);
})
->reject(function ($name) {
    return empty($name);
});
```

As you can see, the `collection` class allows you to chain its methods to perform fluent mapping and reducing of the underlying array. In general, collections are immutable, meaning every `collection` method returns an entirely new `collection` instance.

Creating Collections

As mentioned above, the `collect` helper returns a new `Illuminate\Support\Collection` instance for the given array. So, creating a collection is as simple as:

```
$collection = collect([1, 2, 3]);
```

TIP The results of [Eloquent](#) queries are always returned as `collection` instances.

Extending Collections

Collections are "macroable", which allows you to add additional methods to the `collection` class at run time. For example, the following code adds a `toUpper` method to the `collection` class:

```
use Illuminate\Support\Collection;
use Illuminate\Support\Str;

Collection::macro('toUpper', function () {
    return $this->map(function ($value) {
        return Str::upper($value);
    });
});

$collection = collect(['first', 'second']);

$upper = $collection->toUpper();

// ['FIRST', 'SECOND']
```

Typically, you should declare collection macros in a [service provider](#).

Available Methods

For the remainder of this documentation, we'll discuss each method available on the `Collection` class. Remember, all of these methods may be chained to fluently manipulate the underlying array. Furthermore, almost every method returns a new `Collection` instance, allowing you to preserve the original copy of the collection when necessary:

[all](#) [average](#) [avg](#) [chunk](#) [collapse](#)
[collect](#) [combine](#) [concat](#) [contains](#)
[containsStrict](#) [count](#) [countBy](#)
[crossJoin](#) [dd](#) [diff](#) [diffAssoc](#)
[diffKeys](#) [dump](#) [duplicates](#)
[duplicatesStrict](#) [each](#) [eachSpread](#)
[every](#) [except](#) [filter](#) [first](#) [firstWhere](#)
[flatMap](#) [flatten](#) [flip](#) [forget](#) [forPage](#)
[get](#) [groupBy](#) [has](#) [implode](#)
[intersect](#) [intersectByKey](#)
[isEmpty](#) [isNotEmpty](#) [join](#) [keyBy](#)
[keys](#) [last](#) [macro](#) [make](#) [map](#)

[mapInto](#) [mapSpread](#)
[mapToGroups](#) [mapWithKeys](#)
[max](#) [median](#) [merge](#)
[mergeRecursive](#) [min](#) [mode](#) [nth](#)
[only](#) [pad](#) [partition](#) [pipe](#) [pluck](#) [pop](#)
[prepend](#) [pull](#) [push](#) [put](#) [random](#)
[reduce](#) [reject](#) [replace](#)
[replaceRecursive](#) [reverse](#) [search](#)
[shift](#) [shuffle](#) [skip](#) [skipUntil](#)
[skipWhile](#) [slice](#) [some](#) [sort](#) [sortBy](#)
[sortByDesc](#) [sortDesc](#) [sortKeys](#)
[sortKeysDesc](#) [splice](#) [split](#) [sum](#)

[take](#) [takeUntil](#) [takeWhile](#) [tap](#)
[times](#) [toArray](#) [toJson](#) [transform](#)
[union](#) [unique](#) [uniqueStrict](#) [unless](#)
[unlessEmpty](#) [unlessNotEmpty](#)
[unwrap](#) [values](#) [when](#) [whenEmpty](#)
[whenNotEmpty](#) [where](#)
[whereStrict](#) [whereBetween](#)
[whereIn](#) [whereInStrict](#)
[whereInstanceOf](#)
[whereNotBetween](#) [whereNotIn](#)
[whereNotInStrict](#) [whereNotNull](#)
[whereNull](#) [wrap](#) [zip](#)

Method Listing

`all() {collection-method .first-collection-method}`

The `all` method returns the underlying array represented by the collection:

```
collect([1, 2, 3])->all();  
  
// [1, 2, 3]
```

`average() {collection-method}`

Alias for the [avg](#) method.

`avg() {collection-method}`

The `avg` method returns the [average value](#) of a given key:

```
$average = collect(['foo' => 10], ['foo' => 10], ['foo' => 20], ['foo' => 40])->avg('foo');  
  
// 20  
  
$average = collect([1, 1, 2, 4])->avg();  
  
// 2
```

`chunk() {collection-method}`

The `chunk` method breaks the collection into multiple, smaller collections of a given size:

```
$collection = collect([1, 2, 3, 4, 5, 6, 7]);  
  
$chunks = $collection->chunk(4);  
  
$chunks->toArray();  
  
// [[1, 2, 3, 4], [5, 6, 7]]
```

This method is especially useful in [views](#) when working with a grid system such as [Bootstrap](#). Imagine you have a collection of [Eloquent](#) models you want to display in a grid:

```
@foreach ($products->chunk(3) as $chunk)  
    <div class="row">
```

```
        @foreach ($chunk as $product)
            <div class="col-xs-4">{{ $product->name }}</div>
        @endforeach
    </div>
@endforeach
```

collapse() {collection-method}

The collapse method collapses a collection of arrays into a single, flat collection:

```
$collection = collect([[1, 2, 3], [4, 5, 6], [7, 8, 9]]);

$collapsed = $collection->collapse();

$collapsed->all();

// [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

combine() {collection-method}

The combine method combines the values of the collection, as keys, with the values of another array or collection:

```
$collection = collect(['name', 'age']);

$combined = $collection->combine(['George', 29]);

$combined->all();

// ['name' => 'George', 'age' => 29]
```

collect() {collection-method}

The collect method returns a new Collection instance with the items currently in the collection:

```
$collectionA = collect([1, 2, 3]);

$collectionB = $collectionA->collect();

$collectionB->all();

// [1, 2, 3]
```

The collect method is primarily useful for converting [lazy collections](#) into standard collection instances:

```
$lazyCollection = LazyCollection::make(function () {
    yield 1;
    yield 2;
    yield 3;
});

$collection = $lazyCollection->collect();

get_class($collection);

// 'Illuminate\Support\Collection'

$collection->all();

// [1, 2, 3]
```

TIP The collect method is especially useful when you have an instance of `Enumerable` and need a non-lazy collection instance. Since `collect()` is part of the `Enumerable` contract, you can safely use it to get a `Collection` instance.

concat() {collection-method}

The concat method appends the given array or collection values onto the end of the collection:

```
$collection = collect(['John Doe']);
```

```
$concatenated = $collection->concat(['Jane Doe'])->concat(['name' => 'Johnny Doe']);  
$concatenated->all();  
// ['John Doe', 'Jane Doe', 'Johnny Doe']
```

contains() {collection-method}

The contains method determines whether the collection contains a given item:

```
$collection = collect(['name' => 'Desk', 'price' => 100]);  
$collection->contains('Desk');  
// true  
$collection->contains('New York');  
// false
```

You may also pass a key / value pair to the contains method, which will determine if the given pair exists in the collection:

```
$collection = collect([  
    ['product' => 'Desk', 'price' => 200],  
    ['product' => 'Chair', 'price' => 100],  
]);  
$collection->contains('product', 'Bookcase');  
// false
```

Finally, you may also pass a callback to the contains method to perform your own truth test:

```
$collection = collect([1, 2, 3, 4, 5]);  
$collection->contains(function ($value, $key) {  
    return $value > 5;  
});  
// false
```

The contains method uses "loose" comparisons when checking item values, meaning a string with an integer value will be considered equal to an integer of the same value. Use the [containsStrict](#) method to filter using "strict" comparisons.

containsStrict() {collection-method}

This method has the same signature as the [contains](#) method; however, all values are compared using "strict" comparisons.

TIP This method's behavior is modified when using [Eloquent Collections](#).

count() {collection-method}

The count method returns the total number of items in the collection:

```
$collection = collect([1, 2, 3, 4]);  
$collection->count();  
// 4
```

countBy() {collection-method}

The countBy method counts the occurrences of values in the collection. By default, the method counts the occurrences of every element:

```
$collection = collect([1, 2, 2, 2, 3]);
```



```
$counted = $collection->countBy();  
  
$counted->all();  
  
// [1 => 1, 2 => 3, 3 => 1]
```

However, you pass a callback to the `countBy` method to count all items by a custom value:

```
$collection = collect(['alice@gmail.com', 'bob@yahoo.com', 'carlos@gmail.com']);  
  
$counted = $collection->countBy(function ($email) {  
    return substr(strrchr($email, "@"), 1);  
});  
  
$counted->all();  
  
// ['gmail.com' => 2, 'yahoo.com' => 1]
```

crossJoin() {collection-method}

The `crossJoin` method cross joins the collection's values among the given arrays or collections, returning a Cartesian product with all possible permutations:

```
$collection = collect([1, 2]);  
  
$matrix = $collection->crossJoin(['a', 'b']);  
  
$matrix->all();  
  
/*  
    [  
        [1, 'a'],  
        [1, 'b'],  
        [2, 'a'],  
        [2, 'b'],  
    ]  
*/  
  
$collection = collect([1, 2]);  
  
$matrix = $collection->crossJoin(['a', 'b'], ['I', 'II']);  
  
$matrix->all();  
  
/*  
    [  
        [1, 'a', 'I'],  
        [1, 'a', 'II'],  
        [1, 'b', 'I'],  
        [1, 'b', 'II'],  
        [2, 'a', 'I'],  
        [2, 'a', 'II'],  
        [2, 'b', 'I'],  
        [2, 'b', 'II'],  
    ]  
*/
```

dd() {collection-method}

The `dd` method dumps the collection's items and ends execution of the script:

```
$collection = collect(['John Doe', 'Jane Doe']);  
  
$collection->dd();  
  
/*  
    Collection {  
        #items: array:2 [  
            0 => "John Doe"  
            1 => "Jane Doe"  
        ]  
    }  
*/
```

If you do not want to stop executing the script, use the [dump](#) method instead.

diff() {collection-method}

The `diff` method compares the collection against another collection or a plain PHP array based on its values. This method will return the values in the original collection that are not present in the given collection:

```
$collection = collect([1, 2, 3, 4, 5]);  
$diff = $collection->diff([2, 4, 6, 8]);  
$diff->all();  
// [1, 3, 5]
```

TIP This method's behavior is modified when using [Eloquent Collections](#).

diffAssoc() {collection-method}

The `diffAssoc` method compares the collection against another collection or a plain PHP array based on its keys and values. This method will return the key / value pairs in the original collection that are not present in the given collection:

```
$collection = collect([  
    'color' => 'orange',  
    'type' => 'fruit',  
    'remain' => 6,  
]);  
  
$diff = $collection->diffAssoc([  
    'color' => 'yellow',  
    'type' => 'fruit',  
    'remain' => 3,  
    'used' => 6,  
]);  
  
$diff->all();  
// ['color' => 'orange', 'remain' => 6]
```

diffKeys() {collection-method}

The `diffKeys` method compares the collection against another collection or a plain PHP array based on its keys. This method will return the key / value pairs in the original collection that are not present in the given collection:

```
$collection = collect([  
    'one' => 10,  
    'two' => 20,  
    'three' => 30,  
    'four' => 40,  
    'five' => 50,  
]);  
  
$diff = $collection->diffKeys([  
    'two' => 2,  
    'four' => 4,  
    'six' => 6,  
    'eight' => 8,  
]);  
  
$diff->all();  
// ['one' => 10, 'three' => 30, 'five' => 50]
```

dump() {collection-method}

The `dump` method dumps the collection's items:

```
$collection = collect(['John Doe', 'Jane Doe']);  
  
$collection->dump();  
/*
```

```
Collection {
  #items: array:2 [
    0 => "John Doe"
    1 => "Jane Doe"
  ]
}
```

If you want to stop executing the script after dumping the collection, use the [dd](#) method instead.

duplicates() {collection-method}

The `duplicates` method retrieves and returns duplicate values from the collection:

```
$collection = collect(['a', 'b', 'a', 'c', 'b']);
$collection->duplicates();
// [2 => 'a', 4 => 'b']
```

If the collection contains arrays or objects, you can pass the key of the attributes that you wish to check for duplicate values:

```
$employees = collect([
  ['email' => 'abigail@example.com', 'position' => 'Developer'],
  ['email' => 'james@example.com', 'position' => 'Designer'],
  ['email' => 'victoria@example.com', 'position' => 'Developer'],
]);
$employees->duplicates('position');
// [2 => 'Developer']
```

duplicatesStrict() {collection-method}

This method has the same signature as the [duplicates](#) method; however, all values are compared using "strict" comparisons.

each() {collection-method}

The `each` method iterates over the items in the collection and passes each item to a callback:

```
$collection->each(function ($item, $key) {
  //
});
```

If you would like to stop iterating through the items, you may return `false` from your callback:

```
$collection->each(function ($item, $key) {
  if (/* some condition */) {
    return false;
  }
});
```

eachSpread() {collection-method}

The `eachSpread` method iterates over the collection's items, passing each nested item value into the given callback:

```
$collection = collect([['John Doe', 35], ['Jane Doe', 33]]);
$collection->eachSpread(function ($name, $age) {
  //
});
```

You may stop iterating through the items by returning `false` from the callback:

```
$collection->eachSpread(function ($name, $age) {
  return false;
});
```

every() {collection-method}

The `every` method may be used to verify that all elements of a collection pass a given truth test:

```
collect([1, 2, 3, 4])->every(function ($value, $key) {  
    return $value > 2;  
});  
  
// false
```

If the collection is empty, `every` will return `true`:

```
$collection = collect([]);  
  
$collection->every(function ($value, $key) {  
    return $value > 2;  
});  
  
// true
```

except() {collection-method}

The `except` method returns all items in the collection except for those with the specified keys:

```
$collection = collect(['product_id' => 1, 'price' => 100, 'discount' => false]);  
  
$filtered = $collection->except(['price', 'discount']);  
  
$filtered->all();  
  
// ['product_id' => 1]
```

For the inverse of `except`, see the [only](#) method.

TIP This method's behavior is modified when using [Eloquent Collections](#).

filter() {collection-method}

The `filter` method filters the collection using the given callback, keeping only those items that pass a given truth test:

```
$collection = collect([1, 2, 3, 4]);  
  
$filtered = $collection->filter(function ($value, $key) {  
    return $value > 2;  
});  
  
$filtered->all();  
  
// [3, 4]
```

If no callback is supplied, all entries of the collection that are equivalent to `false` will be removed:

```
$collection = collect([1, 2, 3, null, false, '', 0, []]);  
  
$collection->filter()->all();  
  
// [1, 2, 3]
```

For the inverse of `filter`, see the [reject](#) method.

first() {collection-method}

The `first` method returns the first element in the collection that passes a given truth test:

```
collect([1, 2, 3, 4])->first(function ($value, $key) {  
    return $value > 2;  
});  
  
// 3
```

```
$collection = collect([
    'Apple' => [
```

```
        ['name' => 'iPhone 6S', 'brand' => 'Apple'],
    ],
    'Samsung' => [
        ['name' => 'Galaxy S7', 'brand' => 'Samsung'],
    ],
]);

$products = $collection->flatten(1);

$products->values()->all();

/*
    [
        ['name' => 'iPhone 6S', 'brand' => 'Apple'],
        ['name' => 'Galaxy S7', 'brand' => 'Samsung'],
    ]
*/
```

In this example, calling `flatten` without providing the depth would have also flattened the nested arrays, resulting in `['iPhone 6S', 'Apple', 'Galaxy S7', 'Samsung']`. Providing a depth allows you to restrict the levels of nested arrays that will be flattened.

`flip()` {collection-method}

The `flip` method swaps the collection's keys with their corresponding values:

```
$collection = collect(['name' => 'taylor', 'framework' => 'laravel']);

$flipped = $collection->flip();

$flipped->all();

// ['taylor' => 'name', 'laravel' => 'framework']
```

`forget()` {collection-method}

The `forget` method removes an item from the collection by its key:

```
$collection = collect(['name' => 'taylor', 'framework' => 'laravel']);

$collection->forget('name');

$collection->all();

// ['framework' => 'laravel']
```

NOTE Unlike most other collection methods, `forget` does not return a new modified collection; it modifies the collection it is called on.

`forPage()` {collection-method}

The `forPage` method returns a new collection containing the items that would be present on a given page number. The method accepts the page number as its first argument and the number of items to show per page as its second argument:

```
$collection = collect([1, 2, 3, 4, 5, 6, 7, 8, 9]);

$chunk = $collection->forPage(2, 3);

$chunk->all();

// [4, 5, 6]
```

`get()` {collection-method}

The `get` method returns the item at a given key. If the key does not exist, `null` is returned:

```
$collection = collect(['name' => 'taylor', 'framework' => 'laravel']);

$value = $collection->get('name');
```

```
// taylor
```

You may optionally pass a default value as the second argument:

```
$collection = collect(['name' => 'taylor', 'framework' => 'laravel']);

$value = $collection->get('foo', 'default-value');

// default-value
```

You may even pass a callback as the default value. The result of the callback will be returned if the specified key does not exist:

```
$collection->get('email', function () {
    return 'default-value';
});

// default-value
```

groupBy() {collection-method}

The groupBy method groups the collection's items by a given key:

```
$collection = collect([
    ['account_id' => 'account-x10', 'product' => 'Chair'],
    ['account_id' => 'account-x10', 'product' => 'Bookcase'],
    ['account_id' => 'account-x11', 'product' => 'Desk'],
]);

$grouped = $collection->groupBy('account_id');

$grouped->toArray();

/*
    [
        'account-x10' => [
            ['account_id' => 'account-x10', 'product' => 'Chair'],
            ['account_id' => 'account-x10', 'product' => 'Bookcase'],
        ],
        'account-x11' => [
            ['account_id' => 'account-x11', 'product' => 'Desk'],
        ],
    ]
*/
```

Instead of passing a string key, you may pass a callback. The callback should return the value you wish to key the group by:

```
$grouped = $collection->groupBy(function ($item, $key) {
    return substr($item['account_id'], -3);
});

$grouped->toArray();

/*
    [
        'x10' => [
            ['account_id' => 'account-x10', 'product' => 'Chair'],
            ['account_id' => 'account-x10', 'product' => 'Bookcase'],
        ],
        'x11' => [
            ['account_id' => 'account-x11', 'product' => 'Desk'],
        ],
    ]
*/
```

Multiple grouping criteria may be passed as an array. Each array element will be applied to the corresponding level within a multi-dimensional array:

```
$data = new Collection([
    10 => ['user' => 1, 'skill' => 1, 'roles' => ['Role_1', 'Role_3']],
    20 => ['user' => 2, 'skill' => 1, 'roles' => ['Role_1', 'Role_2']],
    30 => ['user' => 3, 'skill' => 2, 'roles' => ['Role_1']],
    40 => ['user' => 4, 'skill' => 2, 'roles' => ['Role_2']],
]);
```

```

$result = $data->groupBy([
    'skill',
    function ($item) {
        return $item['roles'];
    },
], $preserveKeys = true);

/*
[
    1 => [
        'Role_1' => [
            10 => ['user' => 1, 'skill' => 1, 'roles' => ['Role_1', 'Role_3']],
            20 => ['user' => 2, 'skill' => 1, 'roles' => ['Role_1', 'Role_2']],
        ],
        'Role_2' => [
            20 => ['user' => 2, 'skill' => 1, 'roles' => ['Role_1', 'Role_2']],
        ],
        'Role_3' => [
            10 => ['user' => 1, 'skill' => 1, 'roles' => ['Role_1', 'Role_3']],
        ],
    ],
    2 => [
        'Role_1' => [
            30 => ['user' => 3, 'skill' => 2, 'roles' => ['Role_1']],
        ],
        'Role_2' => [
            40 => ['user' => 4, 'skill' => 2, 'roles' => ['Role_2']],
        ],
    ],
];
*/

```

has() {collection-method}

The has method determines if a given key exists in the collection:

```

$collection = collect(['account_id' => 1, 'product' => 'Desk', 'amount' => 5]);

$collection->has('product');

// true

$collection->has(['product', 'amount']);

// true

$collection->has(['amount', 'price']);

// false

```

implode() {collection-method}

The implode method joins the items in a collection. Its arguments depend on the type of items in the collection. If the collection contains arrays or objects, you should pass the key of the attributes you wish to join, and the "glue" string you wish to place between the values:

```

$collection = collect([
    ['account_id' => 1, 'product' => 'Desk'],
    ['account_id' => 2, 'product' => 'Chair'],
]);

$collection->implode('product', ' ', '');

// Desk, Chair

```

If the collection contains simple strings or numeric values, pass the "glue" as the only argument to the method:

```

collect([1, 2, 3, 4, 5])>implode('-');

// '1-2-3-4-5'

```

intersect() {collection-method}

The `intersect` method removes any values from the original collection that are not present in the given array or collection. The resulting collection will preserve the original collection's keys:

```
$collection = collect(['Desk', 'Sofa', 'Chair']);
$intersect = $collection->intersect(['Desk', 'Chair', 'Bookcase']);
$intersect->all();
// [0 => 'Desk', 2 => 'Chair']
```

TIP This method's behavior is modified when using [Eloquent Collections](#).

`intersectByKeys()` {.collection-method}

The `intersectByKeys` method removes any keys from the original collection that are not present in the given array or collection:

```
$collection = collect([
    'serial' => 'UX301', 'type' => 'screen', 'year' => 2009,
]);
$intersect = $collection->intersectByKeys([
    'reference' => 'UX404', 'type' => 'tab', 'year' => 2011,
]);
$intersect->all();
// ['type' => 'screen', 'year' => 2009]
```

`isEmpty()` {.collection-method}

The `isEmpty` method returns `true` if the collection is empty; otherwise, `false` is returned:

```
collect([])->isEmpty();
// true
```

`isNotEmpty()` {.collection-method}

The `isNotEmpty` method returns `true` if the collection is not empty; otherwise, `false` is returned:

```
collect([])->isNotEmpty();
// false
```

`join()` {.collection-method}

The `join` method joins the collection's values with a string:

```
collect(['a', 'b', 'c'])->join(', '); // 'a, b, c'
collect(['a', 'b', 'c'])->join(' ', ' and '); // 'a, b, and c'
collect(['a', 'b'])->join(' ', ' and '); // 'a and b'
collect(['a'])->join(' ', ' and '); // 'a'
collect([])->join(' ', ' and '); // ''
```

`keyBy()` {.collection-method}

The `keyBy` method keys the collection by the given key. If multiple items have the same key, only the last one will appear in the new collection:

```
$collection = collect([
    ['product_id' => 'prod-100', 'name' => 'Desk'],
    ['product_id' => 'prod-200', 'name' => 'Chair'],
]);
$keyed = $collection->keyBy('product_id');
$keyed->all();
```

```
/*
    [
        'prod-100' => ['product_id' => 'prod-100', 'name' => 'Desk'],
        'prod-200' => ['product_id' => 'prod-200', 'name' => 'Chair'],
    ]
*/
```

You may also pass a callback to the method. The callback should return the value to key the collection by:

```
$keyed = $collection->keyBy(function ($item) {
    return strtoupper($item['product_id']);
});

$keyed->all();

/*
    [
        'PROD-100' => ['product_id' => 'prod-100', 'name' => 'Desk'],
        'PROD-200' => ['product_id' => 'prod-200', 'name' => 'Chair'],
    ]
*/
```

keys() {collection-method}

The `keys` method returns all of the collection's keys:

```
$collection = collect([
    'prod-100' => ['product_id' => 'prod-100', 'name' => 'Desk'],
    'prod-200' => ['product_id' => 'prod-200', 'name' => 'Chair'],
]);

$keys = $collection->keys();

$keys->all();

// ['prod-100', 'prod-200']
```

last() {collection-method}

The `last` method returns the last element in the collection that passes a given truth test:

```
collect([1, 2, 3, 4])->last(function ($value, $key) {
    return $value < 3;
});

// 2
```

You may also call the `last` method with no arguments to get the last element in the collection. If the collection is empty, `null` is returned:

```
collect([1, 2, 3, 4])->last();

// 4
```

macro() {collection-method}

The static `macro` method allows you to add methods to the `Collection` class at run time. Refer to the documentation on [extending collections](#) for more information.

make() {collection-method}

The static `make` method creates a new collection instance. See the [Creating Collections](#) section.

map() {collection-method}

The `map` method iterates through the collection and passes each value to the given callback. The callback is free to modify the item and return it, thus forming a new collection of modified items:

```
$collection = collect([1, 2, 3, 4, 5]);
```

```
$multiplied = $collection->map(function ($item, $key) {  
    return $item * 2;  
});  
  
$multiplied->all();  
  
// [2, 4, 6, 8, 10]
```

NOTE Like most other collection methods, `map` returns a new collection instance; it does not modify the collection it is called on. If you want to transform the original collection, use the [transform](#) method.

`mapInto()` {.collection-method}

The `mapInto()` method iterates over the collection, creating a new instance of the given class by passing the value into the constructor:

```
class Currency  
{  
    /**  
     * Create a new currency instance.  
     *  
     * @param string $code  
     * @return void  
     */  
    function __construct(string $code)  
    {  
        $this->code = $code;  
    }  
}  
  
$collection = collect(['USD', 'EUR', 'GBP']);  
  
$currencies = $collection->mapInto(Currency::class);  
  
$currencies->all();  
  
// [Currency('USD'), Currency('EUR'), Currency('GBP')]
```

`mapSpread()` {.collection-method}

The `mapSpread` method iterates over the collection's items, passing each nested item value into the given callback. The callback is free to modify the item and return it, thus forming a new collection of modified items:

```
$collection = collect([0, 1, 2, 3, 4, 5, 6, 7, 8, 9]);  
  
$chunks = $collection->chunk(2);  
  
$sequence = $chunks->mapSpread(function ($even, $odd) {  
    return $even + $odd;  
});  
  
$sequence->all();  
  
// [1, 5, 9, 13, 17]
```

`mapToGroups()` {.collection-method}

The `mapToGroups` method groups the collection's items by the given callback. The callback should return an associative array containing a single key / value pair, thus forming a new collection of grouped values:

```
$collection = collect([  
    [  
        'name' => 'John Doe',  
        'department' => 'Sales',  
    ],  
    [  
        'name' => 'Jane Doe',  
        'department' => 'Sales',  
    ],  
    [  
        'name' => 'Johnny Doe',  
        'department' => 'Marketing',  
    ],  
]);
```

```
    ]
  });

$grouped = $collection->mapToGroups(function ($item, $key) {
    return [$item['department'] => $item['name']];
});

$grouped->toArray();

/*
    [
        'Sales' => ['John Doe', 'Jane Doe'],
        'Marketing' => ['Johnny Doe'],
    ]
*/

$grouped->get('Sales')->all();

// ['John Doe', 'Jane Doe']
```

mapWithKeys() {collection-method}

The `mapWithKeys` method iterates through the collection and passes each value to the given callback. The callback should return an associative array containing a single key / value pair:

```
$collection = collect([
    [
        'name' => 'John',
        'department' => 'Sales',
        'email' => 'john@example.com',
    ],
    [
        'name' => 'Jane',
        'department' => 'Marketing',
        'email' => 'jane@example.com',
    ]
]);

$keyed = $collection->mapWithKeys(function ($item) {
    return [$item['email'] => $item['name']];
});

$keyed->all();

/*
    [
        'john@example.com' => 'John',
        'jane@example.com' => 'Jane',
    ]
*/
```

max() {collection-method}

The `max` method returns the maximum value of a given key:

```
$max = collect(['foo' => 10], ['foo' => 20])->max('foo');

// 20

$max = collect([1, 2, 3, 4, 5])->max();

// 5
```

median() {collection-method}

The `median` method returns the [median value](#) of a given key:

```
$median = collect(['foo' => 10], ['foo' => 10], ['foo' => 20], ['foo' => 40])->median('foo');

// 15

$median = collect([1, 1, 2, 4])->median();

// 1.5
```

merge() {.collection-method}

The `merge` method merges the given array or collection with the original collection. If a string key in the given items matches a string key in the original collection, the given items's value will overwrite the value in the original collection:

```
$collection = collect(['product_id' => 1, 'price' => 100]);
$merged = $collection->merge(['price' => 200, 'discount' => false]);
$merged->all();
// ['product_id' => 1, 'price' => 200, 'discount' => false]
```

If the given items's keys are numeric, the values will be appended to the end of the collection:

```
$collection = collect(['Desk', 'Chair']);
$merged = $collection->merge(['Bookcase', 'Door']);
$merged->all();
// ['Desk', 'Chair', 'Bookcase', 'Door']
```

mergeRecursive() {.collection-method}

The `mergeRecursive` method merges the given array or collection recursively with the original collection. If a string key in the given items matches a string key in the original collection, then the values for these keys are merged together into an array, and this is done recursively:

```
$collection = collect(['product_id' => 1, 'price' => 100]);
$merged = $collection->mergeRecursive(['product_id' => 2, 'price' => 200, 'discount' => false]);
$merged->all();
// ['product_id' => [1, 2], 'price' => [100, 200], 'discount' => false]
```

min() {.collection-method}

The `min` method returns the minimum value of a given key:

```
$min = collect(['foo' => 10], ['foo' => 20])->min('foo');
// 10
$min = collect([1, 2, 3, 4, 5])->min();
// 1
```

mode() {.collection-method}

The `mode` method returns the [mode value](#) of a given key:

```
$mode = collect(['foo' => 10], ['foo' => 10], ['foo' => 20], ['foo' => 40])->mode('foo');
// [10]
$mode = collect([1, 1, 2, 4])->mode();
// [1]
```

nth() {.collection-method}

The `nth` method creates a new collection consisting of every n-th element:

```
$collection = collect(['a', 'b', 'c', 'd', 'e', 'f']);
$collection->nth(4);
```

```
// ['a', 'e']
```

You may optionally pass an offset as the second argument:

```
$collection->nth(4, 1);  
// ['b', 'f']
```

only() {collection-method}

The `only` method returns the items in the collection with the specified keys:

```
$collection = collect(['product_id' => 1, 'name' => 'Desk', 'price' => 100, 'discount' => false]);  
$filtered = $collection->only(['product_id', 'name']);  
$filtered->all();  
// ['product_id' => 1, 'name' => 'Desk']
```

For the inverse of `only`, see the [except](#) method.

TIP This method's behavior is modified when using [Eloquent Collections](#).

pad() {collection-method}

The `pad` method will fill the array with the given value until the array reaches the specified size. This method behaves like the [array_pad](#) PHP function.

To pad to the left, you should specify a negative size. No padding will take place if the absolute value of the given size is less than or equal to the length of the array:

```
$collection = collect(['A', 'B', 'C']);  
$filtered = $collection->pad(5, 0);  
$filtered->all();  
// ['A', 'B', 'C', 0, 0]  
$filtered = $collection->pad(-5, 0);  
$filtered->all();  
// [0, 0, 'A', 'B', 'C']
```

partition() {collection-method}

The `partition` method may be combined with the `list` PHP function to separate elements that pass a given truth test from those that do not:

```
$collection = collect([1, 2, 3, 4, 5, 6]);  
list($underThree, $equalOrAboveThree) = $collection->partition(function ($i) {  
    return $i < 3;  
});  
$underThree->all();  
// [1, 2]  
$equalOrAboveThree->all();  
// [3, 4, 5, 6]
```

pipe() {collection-method}

The `pipe` method passes the collection to the given callback and returns the result:

```
$collection = collect([1, 2, 3]);
```

```
$piped = $collection->pipe(function ($collection) {  
    return $collection->sum();  
});  
  
// 6
```

pluck() {collection-method}

The `pluck` method retrieves all of the values for a given key:

```
$collection = collect([  
    ['product_id' => 'prod-100', 'name' => 'Desk'],  
    ['product_id' => 'prod-200', 'name' => 'Chair'],  
]);  
  
$plucked = $collection->pluck('name');  
  
$plucked->all();  
  
// ['Desk', 'Chair']
```

You may also specify how you wish the resulting collection to be keyed:

```
$plucked = $collection->pluck('name', 'product_id');  
  
$plucked->all();  
  
// ['prod-100' => 'Desk', 'prod-200' => 'Chair']
```

The `pluck` method also supports retrieving nested values using "dot" notation:

```
$collection = collect([  
    [  
        'speakers' => [  
            'first_day' => ['Rosa', 'Judith'],  
            'second_day' => ['Angela', 'Kathleen'],  
        ],  
    ],  
]);  
  
$plucked = $collection->pluck('speakers.first_day');  
  
$plucked->all();  
  
// ['Rosa', 'Judith']
```

If duplicate keys exist, the last matching element will be inserted into the plucked collection:

```
$collection = collect([  
    ['brand' => 'Tesla', 'color' => 'red'],  
    ['brand' => 'Pagani', 'color' => 'white'],  
    ['brand' => 'Tesla', 'color' => 'black'],  
    ['brand' => 'Pagani', 'color' => 'orange'],  
]);  
  
$plucked = $collection->pluck('color', 'brand');  
  
$plucked->all();  
  
// ['Tesla' => 'black', 'Pagani' => 'orange']
```

pop() {collection-method}

The `pop` method removes and returns the last item from the collection:

```
$collection = collect([1, 2, 3, 4, 5]);  
  
$collection->pop();  
  
// 5  
  
$collection->all();  
  
// [1, 2, 3, 4]
```

prepend() {collection-method}

The prepend method adds an item to the beginning of the collection:

```
$collection = collect([1, 2, 3, 4, 5]);  
$collection->prepend(0);  
$collection->all();  
// [0, 1, 2, 3, 4, 5]
```

You may also pass a second argument to set the key of the prepended item:

```
$collection = collect(['one' => 1, 'two' => 2]);  
$collection->prepend(0, 'zero');  
$collection->all();  
// ['zero' => 0, 'one' => 1, 'two' => 2]
```

pull() {collection-method}

The pull method removes and returns an item from the collection by its key:

```
$collection = collect(['product_id' => 'prod-100', 'name' => 'Desk']);  
$collection->pull('name');  
// 'Desk'  
$collection->all();  
// ['product_id' => 'prod-100']
```

push() {collection-method}

The push method appends an item to the end of the collection:

```
$collection = collect([1, 2, 3, 4]);  
$collection->push(5);  
$collection->all();  
// [1, 2, 3, 4, 5]
```

put() {collection-method}

The put method sets the given key and value in the collection:

```
$collection = collect(['product_id' => 1, 'name' => 'Desk']);  
$collection->put('price', 100);  
$collection->all();  
// ['product_id' => 1, 'name' => 'Desk', 'price' => 100]
```

random() {collection-method}

The random method returns a random item from the collection:

```
$collection = collect([1, 2, 3, 4, 5]);  
$collection->random();  
// 4 - (retrieved randomly)
```

You may optionally pass an integer to random to specify how many items you would like to randomly retrieve. A

collection of items is always returned when explicitly passing the number of items you wish to receive:

```
$random = $collection->random(3);  
$random->all();  
// [2, 4, 5] - (retrieved randomly)
```

If the Collection has fewer items than requested, the method will throw an `InvalidArgumentException`.

reduce() {collection-method}

The `reduce` method reduces the collection to a single value, passing the result of each iteration into the subsequent iteration:

```
$collection = collect([1, 2, 3]);  
$total = $collection->reduce(function ($carry, $item) {  
    return $carry + $item;  
});  
// 6
```

The value for `$carry` on the first iteration is `null`; however, you may specify its initial value by passing a second argument to `reduce`:

```
$collection->reduce(function ($carry, $item) {  
    return $carry + $item;  
}, 4);  
// 10
```

reject() {collection-method}

The `reject` method filters the collection using the given callback. The callback should return `true` if the item should be removed from the resulting collection:

```
$collection = collect([1, 2, 3, 4]);  
$filtered = $collection->reject(function ($value, $key) {  
    return $value > 2;  
});  
$filtered->all();  
// [1, 2]
```

For the inverse of the `reject` method, see the [filter](#) method.

replace() {collection-method}

The `replace` method behaves similarly to `merge`; however, in addition to overwriting matching items with string keys, the `replace` method will also overwrite items in the collection that have matching numeric keys:

```
$collection = collect(['Taylor', 'Abigail', 'James']);  
$replaced = $collection->replace([1 => 'Victoria', 3 => 'Finn']);  
$replaced->all();  
// ['Taylor', 'Victoria', 'James', 'Finn']
```

replaceRecursive() {collection-method}

This method works like `replace`, but it will recur into arrays and apply the same replacement process to the inner values:

```
$collection = collect(['Taylor', 'Abigail', ['James', 'Victoria', 'Finn']]);  
$replaced = $collection->replaceRecursive(['Charlie', 2 => [1 => 'King']]);
```

```
$replaced->all();  
// ['Charlie', 'Abigail', ['James', 'King', 'Finn']]
```

reverse() {collection-method}

The reverse method reverses the order of the collection's items, preserving the original keys:

```
$collection = collect(['a', 'b', 'c', 'd', 'e']);  
$reversed = $collection->reverse();  
$reversed->all();  
/*  
    [  
        4 => 'e',  
        3 => 'd',  
        2 => 'c',  
        1 => 'b',  
        0 => 'a',  
    ]  
*/
```

search() {collection-method}

The search method searches the collection for the given value and returns its key if found. If the item is not found, false is returned.

```
$collection = collect([2, 4, 6, 8]);  
$collection->search(4);  
// 1
```

The search is done using a "loose" comparison, meaning a string with an integer value will be considered equal to an integer of the same value. To use "strict" comparison, pass true as the second argument to the method:

```
$collection->search('4', true);  
// false
```

Alternatively, you may pass in your own callback to search for the first item that passes your truth test:

```
$collection->search(function ($item, $key) {  
    return $item > 5;  
});  
// 2
```

shift() {collection-method}

The shift method removes and returns the first item from the collection:

```
$collection = collect([1, 2, 3, 4, 5]);  
$collection->shift();  
// 1  
$collection->all();  
// [2, 3, 4, 5]
```

shuffle() {collection-method}

The shuffle method randomly shuffles the items in the collection:

```
$collection = collect([1, 2, 3, 4, 5]);
```

```
$shuffled = $collection->shuffle();  
$shuffled->all();  
// [3, 2, 5, 1, 4] - (generated randomly)
```

skip() {.collection-method}

The `skip` method returns a new collection, without the first given amount of items:

```
$collection = collect([1, 2, 3, 4, 5, 6, 7, 8, 9, 10]);  
$collection = $collection->skip(4);  
$collection->all();  
// [5, 6, 7, 8, 9, 10]
```

skipUntil() {.collection-method}

The `skipUntil` method skips items until the given callback returns `true` and then returns the remaining items in the collection:

```
$collection = collect([1, 2, 3, 4]);  
$subset = $collection->skipUntil(function ($item) {  
    return $item >= 3;  
});  
$subset->all();  
// [3, 4]
```

You may also pass a simple value to the `skipUntil` method to skip all items until the given value is found:

```
$collection = collect([1, 2, 3, 4]);  
$subset = $collection->skipUntil(3);  
$subset->all();  
// [3, 4]
```

NOTE If the given value is not found or the callback never returns `true`, the `skipUntil` method will return an empty collection.

skipWhile() {.collection-method}

The `skipWhile` method skips items while the given callback returns `true` and then returns the remaining items in the collection:

```
$collection = collect([1, 2, 3, 4]);  
$subset = $collection->skipWhile(function ($item) {  
    return $item <= 3;  
});  
$subset->all();  
// [4]
```

NOTE If the callback never returns `true`, the `skipWhile` method will return an empty collection.

slice() {.collection-method}

The `slice` method returns a slice of the collection starting at the given index:

```
$collection = collect([1, 2, 3, 4, 5, 6, 7, 8, 9, 10]);  
$slice = $collection->slice(4);
```

```
$slice->all();  
// [5, 6, 7, 8, 9, 10]
```

If you would like to limit the size of the returned slice, pass the desired size as the second argument to the method:

```
$slice = $collection->slice(4, 2);  
$slice->all();  
// [5, 6]
```

The returned slice will preserve keys by default. If you do not wish to preserve the original keys, you can use the [values](#) method to reindex them.

some() {.collection-method}

Alias for the [contains](#) method.

sort() {.collection-method}

The `sort` method sorts the collection. The sorted collection keeps the original array keys, so in this example we'll use the [values](#) method to reset the keys to consecutively numbered indexes:

```
$collection = collect([5, 3, 1, 2, 4]);  
$sorted = $collection->sort();  
$sorted->values()->all();  
// [1, 2, 3, 4, 5]
```

If your sorting needs are more advanced, you may pass a callback to `sort` with your own algorithm. Refer to the PHP documentation on [uasort](#), which is what the collection's `sort` method calls under the hood.

TIP If you need to sort a collection of nested arrays or objects, see the [sortBy](#) and [sortByDesc](#) methods.

sortBy() {.collection-method}

The `sortBy` method sorts the collection by the given key. The sorted collection keeps the original array keys, so in this example we'll use the [values](#) method to reset the keys to consecutively numbered indexes:

```
$collection = collect([  
    ['name' => 'Desk', 'price' => 200],  
    ['name' => 'Chair', 'price' => 100],  
    ['name' => 'Bookcase', 'price' => 150],  
]);  
$sorted = $collection->sortBy('price');  
$sorted->values()->all();  
/*  
    [  
        ['name' => 'Chair', 'price' => 100],  
        ['name' => 'Bookcase', 'price' => 150],  
        ['name' => 'Desk', 'price' => 200],  
    ]  
*/
```

You can also pass your own callback to determine how to sort the collection values:

```
$collection = collect([  
    ['name' => 'Desk', 'colors' => ['Black', 'Mahogany']],  
    ['name' => 'Chair', 'colors' => ['Black']],  
    ['name' => 'Bookcase', 'colors' => ['Red', 'Beige', 'Brown']],  
]);  
$sorted = $collection->sortBy(function ($product, $key) {  
    return count($product['colors']);  
});
```

```
});

$sorted->values()->all();

/*
    [
        ['name' => 'Chair', 'colors' => ['Black']],
        ['name' => 'Desk', 'colors' => ['Black', 'Mahogany']],
        ['name' => 'Bookcase', 'colors' => ['Red', 'Beige', 'Brown']],
    ]
*/
```

sortByDesc() {collection-method}

This method has the same signature as the [sortBy](#) method, but will sort the collection in the opposite order.

sortDesc() {collection-method}

This method will sort the collection in the opposite order as the [sort](#) method:

```
$collection = collect([5, 3, 1, 2, 4]);

$sorted = $collection->sortDesc();

$sorted->values()->all();

// [5, 4, 3, 2, 1]
```

Unlike sort, you may not pass a callback to sortDesc. If you wish to use a callback, you should use [sort](#) and invert your comparison.

sortKeys() {collection-method}

The sortKeys method sorts the collection by the keys of the underlying associative array:

```
$collection = collect([
    'id' => 22345,
    'first' => 'John',
    'last' => 'Doe',
]);

$sorted = $collection->sortKeys();

$sorted->all();

/*
    [
        'first' => 'John',
        'id' => 22345,
        'last' => 'Doe',
    ]
*/
```

sortKeysDesc() {collection-method}

This method has the same signature as the [sortKeys](#) method, but will sort the collection in the opposite order.

splice() {collection-method}

The splice method removes and returns a slice of items starting at the specified index:

```
$collection = collect([1, 2, 3, 4, 5]);

$chunk = $collection->splice(2);

$chunk->all();

// [3, 4, 5]

$collection->all();
```

```
// [1, 2]
```

You may pass a second argument to limit the size of the resulting chunk:

```
$collection = collect([1, 2, 3, 4, 5]);
```

```
$chunk = $collection->splice(2, 1);
```

```
$chunk->all();
```

```
// [3]
```

```
$collection->all();
```

```
// [1, 2, 4, 5]
```

In addition, you can pass a third argument containing the new items to replace the items removed from the collection:

```
$collection = collect([1, 2, 3, 4, 5]);
```

```
$chunk = $collection->splice(2, 1, [10, 11]);
```

```
$chunk->all();
```

```
// [3]
```

```
$collection->all();
```

```
// [1, 2, 10, 11, 4, 5]
```

split() {collection-method}

The `split` method breaks a collection into the given number of groups:

```
$collection = collect([1, 2, 3, 4, 5]);
```

```
$groups = $collection->split(3);
```

```
$groups->toArray();
```

```
// [[1, 2], [3, 4], [5]]
```

sum() {collection-method}

The `sum` method returns the sum of all items in the collection:

```
collect([1, 2, 3, 4, 5])->sum();
```

```
// 15
```

If the collection contains nested arrays or objects, you should pass a key to use for determining which values to sum:

```
$collection = collect([
    ['name' => 'JavaScript: The Good Parts', 'pages' => 176],
    ['name' => 'JavaScript: The Definitive Guide', 'pages' => 1096],
]);
```

```
$collection->sum('pages');
```

```
// 1272
```

In addition, you may pass your own callback to determine which values of the collection to sum:

```
$collection = collect([
    ['name' => 'Chair', 'colors' => ['Black']],
    ['name' => 'Desk', 'colors' => ['Black', 'Mahogany']],
    ['name' => 'Bookcase', 'colors' => ['Red', 'Beige', 'Brown']],
]);
```

```
$collection->sum(function ($product) {
    return count($product['colors']);
});
```

```
// 6
```

take() {collection-method}

The take method returns a new collection with the specified number of items:

```
$collection = collect([0, 1, 2, 3, 4, 5]);  
$chunk = $collection->take(3);  
$chunk->all();  
// [0, 1, 2]
```

You may also pass a negative integer to take the specified amount of items from the end of the collection:

```
$collection = collect([0, 1, 2, 3, 4, 5]);  
$chunk = $collection->take(-2);  
$chunk->all();  
// [4, 5]
```

takeUntil() {collection-method}

The takeUntil method returns items in the collection until the given callback returns true:

```
$collection = collect([1, 2, 3, 4]);  
$subset = $collection->takeUntil(function ($item) {  
    return $item >= 3;  
});  
$subset->all();  
// [1, 2]
```

You may also pass a simple value to the takeUntil method to get the items until the given value is found:

```
$collection = collect([1, 2, 3, 4]);  
$subset = $collection->takeUntil(3);  
$subset->all();  
// [1, 2]
```

NOTE If the given value is not found or the callback never returns true, the takeUntil method will return all items in the collection.

takeWhile() {collection-method}

The takeWhile method returns items in the collection until the given callback returns false:

```
$collection = collect([1, 2, 3, 4]);  
$subset = $collection->takeWhile(function ($item) {  
    return $item < 3;  
});  
$subset->all();  
// [1, 2]
```

NOTE If the callback never returns false, the takeWhile method will return all items in the collection.

tap() {collection-method}

The tap method passes the collection to the given callback, allowing you to "tap" into the collection at a

specific point and do something with the items while not affecting the collection itself:

```
collect([2, 4, 3, 1, 5])
    ->sort()
    ->tap(function ($collection) {
        Log::debug('Values after sorting', $collection->values()->toArray());
    })
    ->shift();

// 1
```

times() {collection-method}

The static `times` method creates a new collection by invoking the callback a given amount of times:

```
$collection = Collection::times(10, function ($number) {
    return $number * 9;
});

$collection->all();

// [9, 18, 27, 36, 45, 54, 63, 72, 81, 90]
```

This method can be useful when combined with factories to create [Eloquent](#) models:

```
$categories = Collection::times(3, function ($number) {
    return factory(Category::class)->create(['name' => "Category No. $number"]);
});

$categories->all();

/*
    [
        ['id' => 1, 'name' => 'Category No. 1'],
        ['id' => 2, 'name' => 'Category No. 2'],
        ['id' => 3, 'name' => 'Category No. 3'],
    ]
*/
```

toArray() {collection-method}

The `toArray` method converts the collection into a plain PHP array. If the collection's values are [Eloquent](#) models, the models will also be converted to arrays:

```
$collection = collect(['name' => 'Desk', 'price' => 200]);

$collection->toArray();

/*
    [
        ['name' => 'Desk', 'price' => 200],
    ]
*/
```

NOTE `toArray` also converts all of the collection's nested objects that are an instance of `Arrayable` to an array. If you want to get the raw underlying array, use the [all](#) method instead.

toJson() {collection-method}

The `toJson` method converts the collection into a JSON serialized string:

```
$collection = collect(['name' => 'Desk', 'price' => 200]);

$collection->toJson();

// '{"name":"Desk", "price":200}'
```

transform() {collection-method}

The `transform` method iterates over the collection and calls the given callback with each item in the collection. The items in the collection will be replaced by the values returned by the callback:


```
$collection = collect([1, 2, 3, 4, 5]);

$collection->transform(function ($item, $key) {
    return $item * 2;
});

$collection->all();

// [2, 4, 6, 8, 10]
```

NOTE Unlike most other collection methods, `transform` modifies the collection itself. If you wish to create a new collection instead, use the [map](#) method.

`union()` {collection-method}

The `union` method adds the given array to the collection. If the given array contains keys that are already in the original collection, the original collection's values will be preferred:

```
$collection = collect([1 => ['a'], 2 => ['b']]);

$union = $collection->union([3 => ['c'], 1 => ['b']]);

$union->all();

// [1 => ['a'], 2 => ['b'], 3 => ['c']]
```

`unique()` {collection-method}

The `unique` method returns all of the unique items in the collection. The returned collection keeps the original array keys, so in this example we'll use the [values](#) method to reset the keys to consecutively numbered indexes:

```
$collection = collect([1, 1, 2, 2, 3, 4, 2]);

$unique = $collection->unique();

$unique->values()->all();

// [1, 2, 3, 4]
```

When dealing with nested arrays or objects, you may specify the key used to determine uniqueness:

```
$collection = collect([
    ['name' => 'iPhone 6', 'brand' => 'Apple', 'type' => 'phone'],
    ['name' => 'iPhone 5', 'brand' => 'Apple', 'type' => 'phone'],
    ['name' => 'Apple Watch', 'brand' => 'Apple', 'type' => 'watch'],
    ['name' => 'Galaxy S6', 'brand' => 'Samsung', 'type' => 'phone'],
    ['name' => 'Galaxy Gear', 'brand' => 'Samsung', 'type' => 'watch'],
]);

$unique = $collection->unique('brand');

$unique->values()->all();

/*
    [
        ['name' => 'iPhone 6', 'brand' => 'Apple', 'type' => 'phone'],
        ['name' => 'Galaxy S6', 'brand' => 'Samsung', 'type' => 'phone'],
    ]
*/
```

You may also pass your own callback to determine item uniqueness:

```
$unique = $collection->unique(function ($item) {
    return $item['brand'].$item['type'];
});

$unique->values()->all();

/*
    [
        ['name' => 'iPhone 6', 'brand' => 'Apple', 'type' => 'phone'],
        ['name' => 'Apple Watch', 'brand' => 'Apple', 'type' => 'watch'],
        ['name' => 'Galaxy S6', 'brand' => 'Samsung', 'type' => 'phone'],
        ['name' => 'Galaxy Gear', 'brand' => 'Samsung', 'type' => 'watch'],
    ]
*/
```

```
    ]  
*/
```

The `unique` method uses "loose" comparisons when checking item values, meaning a string with an integer value will be considered equal to an integer of the same value. Use the [uniqueStrict](#) method to filter using "strict" comparisons.

TIP This method's behavior is modified when using [Eloquent Collections](#).

`uniqueStrict() {collection-method}`

This method has the same signature as the [unique](#) method; however, all values are compared using "strict" comparisons.

`unless() {collection-method}`

The `unless` method will execute the given callback unless the first argument given to the method evaluates to `true`:

```
$collection = collect([1, 2, 3]);  
  
$collection->unless(true, function ($collection) {  
    return $collection->push(4);  
});  
  
$collection->unless(false, function ($collection) {  
    return $collection->push(5);  
});  
  
$collection->all();  
  
// [1, 2, 3, 5]
```

For the inverse of `unless`, see the [when](#) method.

`unlessEmpty() {collection-method}`

Alias for the [whenNotEmpty](#) method.

`unlessNotEmpty() {collection-method}`

Alias for the [whenEmpty](#) method.

`unwrap() {collection-method}`

The static `unwrap` method returns the collection's underlying items from the given value when applicable:

```
Collection::unwrap(collect('John Doe'));  
  
// ['John Doe']  
  
Collection::unwrap(['John Doe']);  
  
// ['John Doe']  
  
Collection::unwrap('John Doe');  
  
// 'John Doe'
```

`values() {collection-method}`

The `values` method returns a new collection with the keys reset to consecutive integers:

```
$collection = collect([  
    10 => ['product' => 'Desk', 'price' => 200],  
    11 => ['product' => 'Desk', 'price' => 200],  
]);
```

```
$values = $collection->values();  
$values->all();  
  
/*  
    [  
        0 => ['product' => 'Desk', 'price' => 200],  
        1 => ['product' => 'Desk', 'price' => 200],  
    ]  
*/
```

when() {collection-method}

The when method will execute the given callback when the first argument given to the method evaluates to true:

```
$collection = collect([1, 2, 3]);  
  
$collection->when(true, function ($collection) {  
    return $collection->push(4);  
});  
  
$collection->when(false, function ($collection) {  
    return $collection->push(5);  
});  
  
$collection->all();  
  
// [1, 2, 3, 4]
```

For the inverse of when, see the [unless](#) method.

whenEmpty() {collection-method}

The whenEmpty method will execute the given callback when the collection is empty:

```
$collection = collect(['michael', 'tom']);  
  
$collection->whenEmpty(function ($collection) {  
    return $collection->push('adam');  
});  
  
$collection->all();  
  
// ['michael', 'tom']  
  
$collection = collect();  
  
$collection->whenEmpty(function ($collection) {  
    return $collection->push('adam');  
});  
  
$collection->all();  
  
// ['adam']  
  
$collection = collect(['michael', 'tom']);  
  
$collection->whenEmpty(function ($collection) {  
    return $collection->push('adam');  
}, function ($collection) {  
    return $collection->push('taylor');  
});  
  
$collection->all();  
  
// ['michael', 'tom', 'taylor']
```

For the inverse of whenEmpty, see the [whenNotEmpty](#) method.

whenNotEmpty() {collection-method}

The `whenNotEmpty` method will execute the given callback when the collection is not empty:

```
$collection = collect(['michael', 'tom']);

$collection->whenNotEmpty(function ($collection) {
    return $collection->push('adam');
});

$collection->all();

// ['michael', 'tom', 'adam']

$collection = collect();

$collection->whenNotEmpty(function ($collection) {
    return $collection->push('adam');
});

$collection->all();

// []

$collection = collect();

$collection->whenNotEmpty(function ($collection) {
    return $collection->push('adam');
}, function ($collection) {
    return $collection->push('taylor');
});

$collection->all();

// ['taylor']
```

For the inverse of `whenNotEmpty`, see the [whenEmpty](#) method.

`where() {collection-method}`

The `where` method filters the collection by a given key / value pair:

```
$collection = collect([
    ['product' => 'Desk', 'price' => 200],
    ['product' => 'Chair', 'price' => 100],
    ['product' => 'Bookcase', 'price' => 150],
    ['product' => 'Door', 'price' => 100],
]);

$filtered = $collection->where('price', 100);

$filtered->all();

/*
    [
        ['product' => 'Chair', 'price' => 100],
        ['product' => 'Door', 'price' => 100],
    ]
*/
```

The `where` method uses "loose" comparisons when checking item values, meaning a string with an integer value will be considered equal to an integer of the same value. Use the [whereStrict](#) method to filter using "strict" comparisons.

Optionally, you may pass a comparison operator as the second parameter.

```
$collection = collect([
    ['name' => 'Jim', 'deleted_at' => '2019-01-01 00:00:00'],
    ['name' => 'Sally', 'deleted_at' => '2019-01-02 00:00:00'],
    ['name' => 'Sue', 'deleted_at' => null],
]);

$filtered = $collection->where('deleted_at', '!=', null);

$filtered->all();
```

```
/*
    [
        ['name' => 'Jim', 'deleted_at' => '2019-01-01 00:00:00'],
        ['name' => 'Sally', 'deleted_at' => '2019-01-02 00:00:00'],
    ]
*/
```

whereStrict() {collection-method}

This method has the same signature as the [where](#) method; however, all values are compared using "strict" comparisons.

whereBetween() {collection-method}

The whereBetween method filters the collection within a given range:

```
$collection = collect([
    ['product' => 'Desk', 'price' => 200],
    ['product' => 'Chair', 'price' => 80],
    ['product' => 'Bookcase', 'price' => 150],
    ['product' => 'Pencil', 'price' => 30],
    ['product' => 'Door', 'price' => 100],
]);

$filtered = $collection->whereBetween('price', [100, 200]);

$filtered->all();

/*
    [
        ['product' => 'Desk', 'price' => 200],
        ['product' => 'Bookcase', 'price' => 150],
        ['product' => 'Door', 'price' => 100],
    ]
*/
```

whereIn() {collection-method}

The whereIn method filters the collection by a given key / value contained within the given array:

```
$collection = collect([
    ['product' => 'Desk', 'price' => 200],
    ['product' => 'Chair', 'price' => 100],
    ['product' => 'Bookcase', 'price' => 150],
    ['product' => 'Door', 'price' => 100],
]);

$filtered = $collection->whereIn('price', [150, 200]);

$filtered->all();

/*
    [
        ['product' => 'Desk', 'price' => 200],
        ['product' => 'Bookcase', 'price' => 150],
    ]
*/
```

The whereIn method uses "loose" comparisons when checking item values, meaning a string with an integer value will be considered equal to an integer of the same value. Use the [whereInStrict](#) method to filter using "strict" comparisons.

whereInStrict() {collection-method}

This method has the same signature as the [whereIn](#) method; however, all values are compared using "strict" comparisons.

whereInInstanceOf() {collection-method}

The whereInstanceOf method filters the collection by a given class type:

```
use App\User;
use App\Post;

$collection = collect([
    new User,
    new User,
    new Post,
]);

$filtered = $collection->whereIn(User::class);

$filtered->all();

// [App\User, App\User]
```

whereNotBetween() {collection-method}

The whereNotBetween method filters the collection within a given range:

```
$collection = collect([
    ['product' => 'Desk', 'price' => 200],
    ['product' => 'Chair', 'price' => 80],
    ['product' => 'Bookcase', 'price' => 150],
    ['product' => 'Pencil', 'price' => 30],
    ['product' => 'Door', 'price' => 100],
]);

$filtered = $collection->whereNotBetween('price', [100, 200]);

$filtered->all();

/*
    [
        ['product' => 'Chair', 'price' => 80],
        ['product' => 'Pencil', 'price' => 30],
    ]
*/
```

whereNotIn() {collection-method}

The whereNotIn method filters the collection by a given key / value not contained within the given array:

```
$collection = collect([
    ['product' => 'Desk', 'price' => 200],
    ['product' => 'Chair', 'price' => 100],
    ['product' => 'Bookcase', 'price' => 150],
    ['product' => 'Door', 'price' => 100],
]);

$filtered = $collection->whereNotIn('price', [150, 200]);

$filtered->all();

/*
    [
        ['product' => 'Chair', 'price' => 100],
        ['product' => 'Door', 'price' => 100],
    ]
*/
```

The whereNotIn method uses "loose" comparisons when checking item values, meaning a string with an integer value will be considered equal to an integer of the same value. Use the [whereNotInStrict](#) method to filter using "strict" comparisons.

whereNotInStrict() {collection-method}

This method has the same signature as the [whereNotIn](#) method; however, all values are compared using "strict" comparisons.

whereNotNull() {collection-method}

The whereNotNull method filters items where the given key is not null:

```
$collection = collect([
    ['name' => 'Desk'],
    ['name' => null],
    ['name' => 'Bookcase'],
]);

$filtered = $collection->whereNotNull('name');

$filtered->all();

/*
    [
        ['name' => 'Desk'],
        ['name' => 'Bookcase'],
    ]
*/
```

whereNull() {collection-method}

The `whereNull` method filters items where the given key is null:

```
$collection = collect([
    ['name' => 'Desk'],
    ['name' => null],
    ['name' => 'Bookcase'],
]);

$filtered = $collection->whereNull('name');

$filtered->all();

/*
    [
        ['name' => null],
    ]
*/
```

wrap() {collection-method}

The static `wrap` method wraps the given value in a collection when applicable:

```
$collection = Collection::wrap('John Doe');

$collection->all();

// ['John Doe']

$collection = Collection::wrap(['John Doe']);

$collection->all();

// ['John Doe']

$collection = Collection::wrap(collect('John Doe'));

$collection->all();

// ['John Doe']
```

zip() {collection-method}

The `zip` method merges together the values of the given array with the values of the original collection at the corresponding index:

```
$collection = collect(['Chair', 'Desk']);

$zipped = $collection->zip([100, 200]);

$zipped->all();

// [['Chair', 100], ['Desk', 200]]
```

Higher Order Messages

Collections also provide support for "higher order messages", which are short-cuts for performing common actions on collections. The collection methods that provide higher order messages are: [average](#), [avg](#), [contains](#), [each](#), [every](#), [filter](#), [first](#), [flatMap](#), [groupBy](#), [keyBy](#), [map](#), [max](#), [min](#), [partition](#), [reject](#), [skipUntil](#), [skipWhile](#), [some](#), [sortBy](#), [sortByDesc](#), [sum](#), [takeUntil](#), [takeWhile](#) and [unique](#).

Each higher order message can be accessed as a dynamic property on a collection instance. For instance, let's use the `each` higher order message to call a method on each object within a collection:

```
$users = User::where('votes', '>', 500)->get();

$users->each->markAsVip();
```

Likewise, we can use the `sum` higher order message to gather the total number of "votes" for a collection of users:

```
$users = User::where('group', 'Development')->get();

return $users->sum->votes;
```

Lazy Collections

Introduction

NOTE Before learning more about Laravel's lazy collections, take some time to familiarize yourself with [PHP generators](#).

To supplement the already powerful `Collection` class, the `LazyCollection` class leverages PHP's [generators](#) to allow you to work with very large datasets while keeping memory usage low.

For example, imagine your application needs to process a multi-gigabyte log file while taking advantage of Laravel's collection methods to parse the logs. Instead of reading the entire file into memory at once, lazy collections may be used to keep only a small part of the file in memory at a given time:

```
use App\LogEntry;
use Illuminate\Support\LazyCollection;

LazyCollection::make(function () {
    $handle = fopen('log.txt', 'r');

    while (($line = fgets($handle)) !== false) {
        yield $line;
    }
})->chunk(4)->map(function ($lines) {
    return LogEntry::fromLines($lines);
})->each(function (LogEntry $logEntry) {
    // Process the log entry...
});
```

Or, imagine you need to iterate through 10,000 Eloquent models. When using traditional Laravel collections, all 10,000 Eloquent models must be loaded into memory at the same time:

```
$users = App\User::all()->filter(function ($user) {
    return $user->id > 500;
});
```

However, the query builder's `cursor` method returns a `LazyCollection` instance. This allows you to still only run a single query against the database but also only keep one Eloquent model loaded in memory at a time. In this example, the `filter` callback is not executed until we actually iterate over each user individually, allowing for a drastic reduction in memory usage:

```
$users = App\User::cursor()->filter(function ($user) {
    return $user->id > 500;
});

foreach ($users as $user) {
    echo $user->id;
}
```


Creating Lazy Collections

To create a lazy collection instance, you should pass a PHP generator function to the collection's `make` method:

```
use Illuminate\Support\LazyCollection;

LazyCollection::make(function () {
    $handle = fopen('log.txt', 'r');

    while (($line = fgets($handle)) !== false) {
        yield $line;
    }
});
```

The Enumerable Contract

Almost all methods available on the `Collection` class are also available on the `LazyCollection` class. Both of these classes implement the `Illuminate\Support\Enumerable` contract, which defines the following methods:

all average avg chunk collapse	macro make map mapInto	unique uniqueStrict unless
collect combine concat contains	mapSpread mapToGroups	unlessEmpty unlessNotEmpty
containsStrict count countBy	mapWithKeys max median merge	unwrap values when whenEmpty
crossJoin dd diff diffAssoc	mergeRecursive min mode nth	whenNotEmpty where
diffKeys dump duplicates	only pad partition pipe pluck	whereStrict whereBetween
duplicatesStrict each eachSpread	random reduce reject replace	whereIn whereInStrict
every except filter first firstWhere	replaceRecursive reverse search	whereInStrict
flatMap flatten flip forPage get	shuffle skip slice some sort	whereInStrict
groupBy has implode intersect	sortBy sortByDesc sortKeys	whereInStrict
intersectByKeys isEmpty	sortByDesc split sum take tap	whereInStrict
isNotEmpty join keyBy keys last	times toArray toJson union	whereInStrict

NOTE Methods that mutate the collection (such as `shift`, `pop`, `prepend` etc.) are *not* available on the `LazyCollection` class.

Lazy Collection Methods

In addition to the methods defined in the `Enumerable` contract, the `LazyCollection` class contains the following methods:

`tapEach()` {.collection-method}

While the `each` method calls the given callback for each item in the collection right away, the `tapEach` method only calls the given callback as the items are being pulled out of the list one by one:

```
$lazyCollection = LazyCollection::times(INF)->tapEach(function ($value) {
    dump($value);
});

// Nothing has been dumped so far...

$array = $lazyCollection->take(3)->all();

// 1
// 2
// 3
```

`remember()` {.collection-method}

The `remember` method returns a new lazy collection that will remember any values that have already been enumerated and will not retrieve them again when the collection is enumerated again:

```
$users = User::cursor()->remember();

// No query has been executed yet...

$users->take(5)->all();
```

```
// The query has been executed and the first 5 users have been hydrated from the database...  
$users->take(20)->all();  
// First 5 users come from the collection's cache... The rest are hydrated from the database...
```

Digging Deeper

Events

Events

- [Introduction](#)
- [Registering Events & Listeners](#)
 - [Generating Events & Listeners](#)
 - [Manually Registering Events](#)
 - [Event Discovery](#)
- [Defining Events](#)
- [Defining Listeners](#)
- [Queued Event Listeners](#)
 - [Manually Accessing The Queue](#)
 - [Handling Failed Jobs](#)
- [Dispatching Events](#)
- [Event Subscribers](#)
 - [Writing Event Subscribers](#)
 - [Registering Event Subscribers](#)

Introduction

Laravel's events provide a simple observer implementation, allowing you to subscribe and listen for various events that occur in your application. Event classes are typically stored in the `app/Events` directory, while their listeners are stored in `app/Listeners`. Don't worry if you don't see these directories in your application, since they will be created for you as you generate events and listeners using Artisan console commands.

Events serve as a great way to decouple various aspects of your application, since a single event can have multiple listeners that do not depend on each other. For example, you may wish to send a Slack notification to your user each time an order has shipped. Instead of coupling your order processing code to your Slack notification code, you can raise an `OrderShipped` event, which a listener can receive and transform into a Slack notification.

Registering Events & Listeners

The `EventServiceProvider` included with your Laravel application provides a convenient place to register all of your application's event listeners. The `listen` property contains an array of all events (keys) and their listeners (values). You may add as many events to this array as your application requires. For example, let's add a `OrderShipped` event:

```
/**
 * The event listener mappings for the application.
 *
 * @var array
 */
protected $listen = [
    'App\Events\OrderShipped' => [
        'App\Listeners\SendShipmentNotification',
    ],
];
```

Generating Events & Listeners

Of course, manually creating the files for each event and listener is cumbersome. Instead, add listeners and events to your `EventServiceProvider` and use the `event:generate` command. This command will generate any events or listeners that are listed in your `EventServiceProvider`. Events and listeners that already exist will be left untouched:

```
php artisan event:generate
```

Manually Registering Events

Typically, events should be registered via the `EventServiceProvider $listen` array; however, you may also register Closure based events manually in the `boot` method of your `EventServiceProvider`:

```
/**
 * Register any other events for your application.
 *
 * @return void
 */
public function boot()
{
    parent::boot();

    Event::listen('event.name', function ($foo, $bar) {
        //
    });
}
```

Wildcard Event Listeners

You may even register listeners using the `*` as a wildcard parameter, allowing you to catch multiple events on the same listener. Wildcard listeners receive the event name as their first argument, and the entire event data array as their second argument:

```
Event::listen('event.*', function ($eventName, array $data) {
    //
});
```

Event Discovery

Instead of registering events and listeners manually in the `$listen` array of the `EventServiceProvider`, you can enable automatic event discovery. When event discovery is enabled, Laravel will automatically find and register your events and listeners by scanning your application's `Listeners` directory. In addition, any explicitly defined events listed in the `EventServiceProvider` will still be registered.

Laravel finds event listeners by scanning the listener classes using reflection. When Laravel finds any listener class method that begins with `handle`, Laravel will register those methods as event listeners for the event that is type-hinted in the method's signature:

```
use App\Events\PodcastProcessed;

class SendPodcastProcessedNotification
{
    /**
     * Handle the given event.
     *
     * @param  \App\Events\PodcastProcessed
     * @return void
     */
    public function handle(PodcastProcessed $event)
    {
        //
    }
}
```

Event discovery is disabled by default, but you can enable it by overriding the `shouldDiscoverEvents` method of your application's `EventServiceProvider`:

```
/**
 * Determine if events and listeners should be automatically discovered.
 *
 * @return bool
 */
public function shouldDiscoverEvents()
{
    return true;
}
```

By default, all listeners within your application's `Listeners` directory will be scanned. If you would like to

define additional directories to scan, you may override the `discoverEventsWithin` method in your `EventServiceProvider`:

```
/**
 * Get the listener directories that should be used to discover events.
 *
 * @return array
 */
protected function discoverEventsWithin()
{
    return [
        $this->app->path('Listeners'),
    ];
}
```

In production, you likely do not want the framework to scan all of your listeners on every request. Therefore, during your deployment process, you should run the `event:cache` Artisan command to cache a manifest of all of your application's events and listeners. This manifest will be used by the framework to speed up the event registration process. The `event:clear` command may be used to destroy the cache.

TIP The `event:list` command may be used to display a list of all events and listeners registered by your application.

Defining Events

An event class is a data container which holds the information related to the event. For example, let's assume our generated `orderShipped` event receives an [Eloquent ORM](#) object:

```
<?php

namespace App\Events;

use App\Order;
use Illuminate\Broadcasting\InteractsWithSockets;
use Illuminate\Foundation\Events\Dispatchable;
use Illuminate\Queue\SerializesModels;

class OrderShipped
{
    use Dispatchable, InteractsWithSockets, SerializesModels;

    public $order;

    /**
     * Create a new event instance.
     *
     * @param \App\Order $order
     * @return void
     */
    public function __construct(Order $order)
    {
        $this->order = $order;
    }
}
```

As you can see, this event class contains no logic. It is a container for the `order` instance that was purchased. The `SerializesModels` trait used by the event will gracefully serialize any Eloquent models if the event object is serialized using PHP's `serialize` function.

Defining Listeners

Next, let's take a look at the listener for our example event. Event listeners receive the event instance in their `handle` method. The `event:generate` command will automatically import the proper event class and type-hint the event on the `handle` method. Within the `handle` method, you may perform any actions necessary to respond to the event:

```
<?php

namespace App\Listeners;
```

```
use App\Events\OrderShipped;

class SendShipmentNotification
{
    /**
     * Create the event listener.
     *
     * @return void
     */
    public function __construct()
    {
        //
    }

    /**
     * Handle the event.
     *
     * @param  \App\Events\OrderShipped  $event
     * @return void
     */
    public function handle(OrderShipped $event)
    {
        // Access the order using $event->order...
    }
}
```

TIP Your event listeners may also type-hint any dependencies they need on their constructors. All event listeners are resolved via the Laravel [service container](#), so dependencies will be injected automatically.

Stopping The Propagation Of An Event

Sometimes, you may wish to stop the propagation of an event to other listeners. You may do so by returning `false` from your listener's `handle` method.

Queued Event Listeners

Queueing listeners can be beneficial if your listener is going to perform a slow task such as sending an e-mail or making an HTTP request. Before getting started with queued listeners, make sure to [configure your queue](#) and start a queue listener on your server or local development environment.

To specify that a listener should be queued, add the `ShouldQueue` interface to the listener class. Listeners generated by the `event:generate` Artisan command already have this interface imported into the current namespace, so you can use it immediately:

```
<?php

namespace App\Listeners;

use App\Events\OrderShipped;
use Illuminate\Contracts\Queue\ShouldQueue;

class SendShipmentNotification implements ShouldQueue
{
    //
}
```

That's it! Now, when this listener is called for an event, it will be automatically queued by the event dispatcher using Laravel's [queue system](#). If no exceptions are thrown when the listener is executed by the queue, the queued job will automatically be deleted after it has finished processing.

Customizing The Queue Connection & Queue Name

If you would like to customize the queue connection, queue name, or queue delay time of an event listener, you may define the `$connection`, `$queue`, or `$delay` properties on your listener class:

```
<?php

namespace App\Listeners;

use App\Events\OrderShipped;
```

```
use Illuminate\Contracts\Queue\ShouldQueue;

class SendShipmentNotification implements ShouldQueue
{
    /**
     * The name of the connection the job should be sent to.
     *
     * @var string|null
     */
    public $connection = 'sqs';

    /**
     * The name of the queue the job should be sent to.
     *
     * @var string|null
     */
    public $queue = 'listeners';

    /**
     * The time (seconds) before the job should be processed.
     *
     * @var int
     */
    public $delay = 60;
}
```

If you would like to define the listener's queue at runtime, you may define a `viaQueue` method on the listener:

```
/**
 * Get the name of the listener's queue.
 *
 * @return string
 */
public function viaQueue()
{
    return 'listeners';
}
```

Conditionally Queueing Listeners

Sometimes, you may need to determine whether a listener should be queued based on some data that's only available at runtime. To accomplish this, a `shouldQueue` method may be added to a listener to determine whether the listener should be queued. If the `shouldQueue` method returns `false`, the listener will not be executed:

```
<?php

namespace App\Listeners;

use App\Events\OrderPlaced;
use Illuminate\Contracts\Queue\ShouldQueue;

class RewardGiftCard implements ShouldQueue
{
    /**
     * Reward a gift card to the customer.
     *
     * @param \App\Events\OrderPlaced $event
     * @return void
     */
    public function handle(OrderPlaced $event)
    {
        //
    }

    /**
     * Determine whether the listener should be queued.
     *
     * @param \App\Events\OrderPlaced $event
     * @return bool
     */
    public function shouldQueue(OrderPlaced $event)
    {
        return $event->order->subtotal >= 5000;
    }
}
```

Manually Accessing The Queue

If you need to manually access the listener's underlying queue job's `delete` and `release` methods, you may do so using the `Illuminate\Queue\InteractsWithQueue` trait. This trait is imported by default on generated listeners and provides access to these methods:

```
<?php

namespace App\Listeners;

use App\Events\OrderShipped;
use Illuminate\Contracts\Queue\ShouldQueue;
use Illuminate\Queue\InteractsWithQueue;

class SendShipmentNotification implements ShouldQueue
{
    use InteractsWithQueue;

    /**
     * Handle the event.
     *
     * @param  \App\Events\OrderShipped  $event
     * @return void
     */
    public function handle(OrderShipped $event)
    {
        if (true) {
            $this->release(30);
        }
    }
}
```

Handling Failed Jobs

Sometimes your queued event listeners may fail. If queued listener exceeds the maximum number of attempts as defined by your queue worker, the `failed` method will be called on your listener. The `failed` method receives the event instance and the exception that caused the failure:

```
<?php

namespace App\Listeners;

use App\Events\OrderShipped;
use Illuminate\Contracts\Queue\ShouldQueue;
use Illuminate\Queue\InteractsWithQueue;

class SendShipmentNotification implements ShouldQueue
{
    use InteractsWithQueue;

    /**
     * Handle the event.
     *
     * @param  \App\Events\OrderShipped  $event
     * @return void
     */
    public function handle(OrderShipped $event)
    {
        //
    }

    /**
     * Handle a job failure.
     *
     * @param  \App\Events\OrderShipped  $event
     * @param  \Throwable  $exception
     * @return void
     */
    public function failed(OrderShipped $event, $exception)
    {
        //
    }
}
```


Dispatching Events

To dispatch an event, you may pass an instance of the event to the event helper. The helper will dispatch the event to all of its registered listeners. Since the event helper is globally available, you may call it from anywhere in your application:

```
<?php

namespace App\Http\Controllers;

use App\Events\OrderShipped;
use App\Http\Controllers\Controller;
use App\Order;

class OrderController extends Controller
{
    /**
     * Ship the given order.
     *
     * @param int $orderId
     * @return Response
     */
    public function ship($orderId)
    {
        $order = Order::findOrFail($orderId);

        // Order shipment logic...

        event(new OrderShipped($order));
    }
}
```

Alternatively, if your event uses the `Illuminate\Foundation\Events\Dispatchable` trait, you may call the static `dispatch` method on the event. Any arguments passed to the `dispatch` method will be passed to the event's constructor:

```
OrderShipped::dispatch($order);
```

TIP When testing, it can be helpful to assert that certain events were dispatched without actually triggering their listeners. Laravel's [built-in testing helpers](#) makes it a cinch.

Event Subscribers

Writing Event Subscribers

Event subscribers are classes that may subscribe to multiple events from within the class itself, allowing you to define several event handlers within a single class. Subscribers should define a `subscribe` method, which will be passed an event dispatcher instance. You may call the `listen` method on the given dispatcher to register event listeners:

```
<?php

namespace App\Listeners;

class UserEventSubscriber
{
    /**
     * Handle user login events.
     */
    public function handleUserLogin($event) {}

    /**
     * Handle user logout events.
     */
    public function handleUserLogout($event) {}

    /**
     * Register the listeners for the subscriber.
     *
     * @param \Illuminate\Events\Dispatcher $events
     */
    public function subscribe($events)
```

```
{
    $events->listen(
        'Illuminate\Auth\Events\Login',
        'App\Listeners\UserEventSubscriber@handleUserLogin'
    );

    $events->listen(
        'Illuminate\Auth\Events\Logout',
        'App\Listeners\UserEventSubscriber@handleUserLogout'
    );
}
```

Registering Event Subscribers

After writing the subscriber, you are ready to register it with the event dispatcher. You may register subscribers using the `$subscribe` property on the `EventServiceProvider`. For example, let's add the `UserEventSubscriber` to the list:

```
<?php

namespace App\Providers;

use Illuminate\Foundation\Support\Providers\EventServiceProvider as ServiceProvider;

class EventServiceProvider extends ServiceProvider
{
    /**
     * The event listener mappings for the application.
     *
     * @var array
     */
    protected $listen = [
        //
    ];

    /**
     * The subscriber classes to register.
     *
     * @var array
     */
    protected $subscribe = [
        'App\Listeners\UserEventSubscriber',
    ];
}
```

Digging Deeper

File Storage

File Storage

- [Introduction](#)
- [Configuration](#)
 - [The Public Disk](#)
 - [The Local Driver](#)
 - [Driver Prerequisites](#)
 - [Caching](#)
- [Obtaining Disk Instances](#)
- [Retrieving Files](#)
 - [Downloading Files](#)
 - [File URLs](#)
 - [File Metadata](#)
- [Storing Files](#)
 - [File Uploads](#)
 - [File Visibility](#)
- [Deleting Files](#)
- [Directories](#)
- [Custom Filesystems](#)

Introduction

Laravel provides a powerful filesystem abstraction thanks to the wonderful [Flysystem](#) PHP package by Frank de Jonge. The Laravel Flysystem integration provides simple to use drivers for working with local filesystems and Amazon S3. Even better, it's amazingly simple to switch between these storage options as the API remains the same for each system.

Configuration

The filesystem configuration file is located at `config/filesystems.php`. Within this file you may configure all of your "disks". Each disk represents a particular storage driver and storage location. Example configurations for each supported driver are included in the configuration file. So, modify the configuration to reflect your storage preferences and credentials.

You may configure as many disks as you like, and may even have multiple disks that use the same driver.

The Public Disk

The `public` disk is intended for files that are going to be publicly accessible. By default, the `public` disk uses the `local` driver and stores these files in `storage/app/public`. To make them accessible from the web, you should create a symbolic link from `public/storage` to `storage/app/public`. This convention will keep your publicly accessible files in one directory that can be easily shared across deployments when using zero down-time deployment systems like [Envoyer](#).

To create the symbolic link, you may use the `storage:link` Artisan command:

```
php artisan storage:link
```

Once a file has been stored and the symbolic link has been created, you can create a URL to the files using the `asset` helper:

```
echo asset('storage/file.txt');
```

You may configure additional symbolic links in your `filesystems` configuration file. Each of the configured links will be created when you run the `storage:link` command:

```
'links' => [
    public_path('storage') => storage_path('app/public'),
    public_path('images') => storage_path('app/images'),
],
```

The Local Driver

When using the `local` driver, all file operations are relative to the `root` directory defined in your `filesystems` configuration file. By default, this value is set to the `storage/app` directory. Therefore, the following method would store a file in `storage/app/file.txt`:

```
Storage::disk('local')->put('file.txt', 'Contents');
```

Permissions

The public [visibility](#) translates to `0755` for directories and `0644` for files. You can modify the permissions mappings in your `filesystems` configuration file:

```
'local' => [
    'driver' => 'local',
    'root' => storage_path('app'),
    'permissions' => [
        'file' => [
            'public' => 0664,
            'private' => 0600,
        ],
        'dir' => [
            'public' => 0775,
            'private' => 0700,
        ],
    ],
],
```

Driver Prerequisites

Composer Packages

Before using the SFTP or S3 drivers, you will need to install the appropriate package via Composer:

- SFTP: `league/flysystem-sftp ~1.0`
- Amazon S3: `league/flysystem-aws-s3-v3 ~1.0`

An absolute must for performance is to use a cached adapter. You will need an additional package for this:

- CachedAdapter: `league/flysystem-cached-adapter ~1.0`

S3 Driver Configuration

The S3 driver configuration information is located in your `config/filesystems.php` configuration file. This file contains an example configuration array for an S3 driver. You are free to modify this array with your own S3 configuration and credentials. For convenience, these environment variables match the naming convention used by the AWS CLI.

FTP Driver Configuration

Laravel's Flysystem integrations works great with FTP; however, a sample configuration is not included with the framework's default `filesystems.php` configuration file. If you need to configure a FTP filesystem, you may use the example configuration below:

```
'ftp' => [
    'driver' => 'ftp',
    'host' => 'ftp.example.com',
    'username' => 'your-username',
    'password' => 'your-password',

    // Optional FTP Settings...
```

```
        // 'port' => 21,  
        // 'root' => '',  
        // 'passive' => true,  
        // 'ssl' => true,  
        // 'timeout' => 30,  
    ],
```

SFTP Driver Configuration

Laravel's Flysystem integrations works great with SFTP; however, a sample configuration is not included with the framework's default `filesystems.php` configuration file. If you need to configure a SFTP filesystem, you may use the example configuration below:

```
'sftp' => [  
    'driver' => 'sftp',  
    'host' => 'example.com',  
    'username' => 'your-username',  
    'password' => 'your-password',  
  
    // Settings for SSH key based authentication...  
    // 'privateKey' => '/path/to/privateKey',  
    // 'password' => 'encryption-password',  
  
    // Optional SFTP Settings...  
    // 'port' => 22,  
    // 'root' => '',  
    // 'timeout' => 30,  
],
```

Caching

To enable caching for a given disk, you may add a `cache` directive to the disk's configuration options. The `cache` option should be an array of caching options containing the `disk` name, the `expire` time in seconds, and the `cache prefix`:

```
's3' => [  
    'driver' => 's3',  
  
    // Other Disk Options...  
  
    'cache' => [  
        'store' => 'memcached',  
        'expire' => 600,  
        'prefix' => 'cache-prefix',  
    ],  
],
```

Obtaining Disk Instances

The storage facade may be used to interact with any of your configured disks. For example, you may use the `put` method on the facade to store an avatar on the default disk. If you call methods on the storage facade without first calling the `disk` method, the method call will automatically be passed to the default disk:

```
use Illuminate\Support\Facades\Storage;  
  
Storage::put('avatars/1', $fileContents);
```

If your application interacts with multiple disks, you may use the `disk` method on the storage facade to work with files on a particular disk:

```
Storage::disk('s3')->put('avatars/1', $fileContents);
```

Retrieving Files

The `get` method may be used to retrieve the contents of a file. The raw string contents of the file will be returned by the method. Remember, all file paths should be specified relative to the `"root"` location configured for the disk:

```
$contents = Storage::get('file.jpg');
```

The `exists` method may be used to determine if a file exists on the disk:

```
$exists = Storage::disk('s3')->exists('file.jpg');
```

The `missing` method may be used to determine if a file is missing from the disk:

```
$missing = Storage::disk('s3')->missing('file.jpg');
```

Downloading Files

The `download` method may be used to generate a response that forces the user's browser to download the file at the given path. The `download` method accepts a file name as the second argument to the method, which will determine the file name that is seen by the user downloading the file. Finally, you may pass an array of HTTP headers as the third argument to the method:

```
return Storage::download('file.jpg');

return Storage::download('file.jpg', $name, $headers);
```

File URLs

You may use the `url` method to get the URL for the given file. If you are using the `local` driver, this will typically just prepend `/storage` to the given path and return a relative URL to the file. If you are using the `s3` driver, the fully qualified remote URL will be returned:

```
use Illuminate\Support\Facades\Storage;

$url = Storage::url('file.jpg');
```

NOTE Remember, if you are using the `local` driver, all files that should be publicly accessible should be placed in the `storage/app/public` directory. Furthermore, you should [create a symbolic link](#) at `public/storage` which points to the `storage/app/public` directory.

Temporary URLs

For files stored using the `s3` you may create a temporary URL to a given file using the `temporaryUrl` method. This method accepts a path and a `DateTime` instance specifying when the URL should expire:

```
$url = Storage::temporaryUrl(
    'file.jpg', now()->addMinutes(5)
);
```

If you need to specify additional [S3 request parameters](#), you may pass the array of request parameters as the third argument to the `temporaryUrl` method:

```
$url = Storage::temporaryUrl(
    'file.jpg',
    now()->addMinutes(5),
    ['ResponseContentType' => 'application/octet-stream']
);
```

URL Host Customization

If you would like to pre-define the host for file URLs generated using the storage facade, you may add a `url` option to the disk's configuration array:

```
'public' => [
    'driver' => 'local',
    'root' => storage_path('app/public'),
    'url' => env('APP_URL').'/storage',
    'visibility' => 'public',
],
```

File Metadata

In addition to reading and writing files, Laravel can also provide information about the files themselves. For

example, the `size` method may be used to get the size of the file in bytes:

```
use Illuminate\Support\Facades\Storage;

$size = Storage::size('file.jpg');
```

The `lastModified` method returns the UNIX timestamp of the last time the file was modified:

```
$time = Storage::lastModified('file.jpg');
```

Storing Files

The `put` method may be used to store raw file contents on a disk. You may also pass a PHP resource to the `put` method, which will use Flysystem's underlying stream support. Remember, all file paths should be specified relative to the "root" location configured for the disk:

```
use Illuminate\Support\Facades\Storage;

Storage::put('file.jpg', $contents);

Storage::put('file.jpg', $resource);
```

Automatic Streaming

If you would like Laravel to automatically manage streaming a given file to your storage location, you may use the `putFile` or `putFileAs` method. This method accepts either a `Illuminate\Http\File` or `Illuminate\Http\UploadedFile` instance and will automatically stream the file to your desired location:

```
use Illuminate\Http\File;
use Illuminate\Support\Facades\Storage;

// Automatically generate a unique ID for file name...
Storage::putFile('photos', new File('/path/to/photo'));

// Manually specify a file name...
Storage::putFileAs('photos', new File('/path/to/photo'), 'photo.jpg');
```

There are a few important things to note about the `putFile` method. Note that we only specified a directory name, not a file name. By default, the `putFile` method will generate a unique ID to serve as the file name. The file's extension will be determined by examining the file's MIME type. The path to the file will be returned by the `putFile` method so you can store the path, including the generated file name, in your database.

The `putFile` and `putFileAs` methods also accept an argument to specify the "visibility" of the stored file. This is particularly useful if you are storing the file on a cloud disk such as S3 and would like the file to be publicly accessible:

```
Storage::putFile('photos', new File('/path/to/photo'), 'public');
```

Prepending & Appending To Files

The `prepend` and `append` methods allow you to write to the beginning or end of a file:

```
Storage::prepend('file.log', 'Prepended Text');

Storage::append('file.log', 'Appended Text');
```

Copying & Moving Files

The `copy` method may be used to copy an existing file to a new location on the disk, while the `move` method may be used to rename or move an existing file to a new location:

```
Storage::copy('old/file.jpg', 'new/file.jpg');

Storage::move('old/file.jpg', 'new/file.jpg');
```

File Uploads

In web applications, one of the most common use-cases for storing files is storing user uploaded files such as profile pictures, photos, and documents. Laravel makes it very easy to store uploaded files using the `store` method on an uploaded file instance. Call the `store` method with the path at which you wish to store the uploaded file:

```
<?php

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;
use Illuminate\Http\Request;

class UserAvatarController extends Controller
{
    /**
     * Update the avatar for the user.
     *
     * @param Request $request
     * @return Response
     */
    public function update(Request $request)
    {
        $path = $request->file('avatar')->store('avatars');

        return $path;
    }
}
```

There are a few important things to note about this example. Note that we only specified a directory name, not a file name. By default, the `store` method will generate a unique ID to serve as the file name. The file's extension will be determined by examining the file's MIME type. The path to the file will be returned by the `store` method so you can store the path, including the generated file name, in your database.

You may also call the `putFile` method on the storage facade to perform the same file manipulation as the example above:

```
$path = Storage::putFile('avatars', $request->file('avatar'));
```

Specifying A File Name

If you would not like a file name to be automatically assigned to your stored file, you may use the `storeAs` method, which receives the path, the file name, and the (optional) disk as its arguments:

```
$path = $request->file('avatar')->storeAs(
    'avatars', $request->user()->id
);
```

You may also use the `putFileAs` method on the storage facade, which will perform the same file manipulation as the example above:

```
$path = Storage::putFileAs(
    'avatars', $request->file('avatar'), $request->user()->id
);
```

NOTE Unprintable and invalid unicode characters will automatically be removed from file paths. Therefore, you may wish to sanitize your file paths before passing them to Laravel's file storage methods. File paths are normalized using the `League\Flysystem\Util::normalizePath` method.

Specifying A Disk

By default, this method will use your default disk. If you would like to specify another disk, pass the disk name as the second argument to the `store` method:

```
$path = $request->file('avatar')->store(
    'avatars/'.$request->user()->id, 's3'
);
```

If you are using the `storeAs` method, you may pass the disk name as the third argument to the method:

```
$path = $request->file('avatar')->storeAs(
```



```
        'avatars',  
        $request->user()->id,  
        's3'  
    );
```

Other File Information

If you would like to get original name of the uploaded file, you may do so using the `getClientOriginalName` method:

```
$name = $request->file('avatar')->getClientOriginalName();
```

The `extension` method may be used to get the file extension of the uploaded file:

```
$extension = $request->file('avatar')->extension();
```

File Visibility

In Laravel's Flysystem integration, "visibility" is an abstraction of file permissions across multiple platforms. Files may either be declared `public` or `private`. When a file is declared `public`, you are indicating that the file should generally be accessible to others. For example, when using the S3 driver, you may retrieve URLs for public files.

You can set the visibility when setting the file via the `put` method:

```
use Illuminate\Support\Facades\Storage;  
  
Storage::put('file.jpg', $contents, 'public');
```

If the file has already been stored, its visibility can be retrieved and set via the `getVisibility` and `setVisibility` methods:

```
$visibility = Storage::getVisibility('file.jpg');  
  
Storage::setVisibility('file.jpg', 'public');
```

When interacting with uploaded files, you may use the `storePublicly` and `storePubliclyAs` methods to store the uploaded file with public visibility:

```
$path = $request->file('avatar')->storePublicly('avatars', 's3');  
  
$path = $request->file('avatar')->storePubliclyAs(  
    'avatars',  
    $request->user()->id,  
    's3'  
);
```

Deleting Files

The `delete` method accepts a single filename or an array of files to remove from the disk:

```
use Illuminate\Support\Facades\Storage;  
  
Storage::delete('file.jpg');  
  
Storage::delete(['file.jpg', 'file2.jpg']);
```

If necessary, you may specify the disk that the file should be deleted from:

```
use Illuminate\Support\Facades\Storage;  
  
Storage::disk('s3')->delete('folder_path/file_name.jpg');
```

Directories

Get All Files Within A Directory

The `files` method returns an array of all of the files in a given directory. If you would like to retrieve a list of all files within a given directory including all subdirectories, you may use the `allFiles` method:

```
use Illuminate\Support\Facades\Storage;

$files = Storage::files($directory);

$files = Storage::allFiles($directory);
```

Get All Directories Within A Directory

The `directories` method returns an array of all the directories within a given directory. Additionally, you may use the `allDirectories` method to get a list of all directories within a given directory and all of its subdirectories:

```
$directories = Storage::directories($directory);

// Recursive...
$directories = Storage::allDirectories($directory);
```

Create A Directory

The `makeDirectory` method will create the given directory, including any needed subdirectories:

```
Storage::makeDirectory($directory);
```

Delete A Directory

Finally, the `deleteDirectory` method may be used to remove a directory and all of its files:

```
Storage::deleteDirectory($directory);
```

Custom Filesystems

Laravel's Flysystem integration provides drivers for several "drivers" out of the box; however, Flysystem is not limited to these and has adapters for many other storage systems. You can create a custom driver if you want to use one of these additional adapters in your Laravel application.

In order to set up the custom filesystem you will need a Flysystem adapter. Let's add a community maintained Dropbox adapter to our project:

```
composer require spatie/flysystem-dropbox
```

Next, you should create a [service provider](#) such as `DropboxServiceProvider`. In the provider's `boot` method, you may use the storage facade's `extend` method to define the custom driver:

```
<?php

namespace App\Providers;

use Illuminate\Support\Facades\Storage;
use Illuminate\Support\ServiceProvider;
use League\Flysystem\Filesystem;
use Spatie\Dropbox\Client as DropboxClient;
use Spatie\FlysystemDropbox\DropboxAdapter;

class DropboxServiceProvider extends ServiceProvider
{
    /**
     * Register any application services.
     *
     * @return void
     */
    public function register()
    {
        //
    }

    /**
```

```
* Bootstrap any application services.
*
* @return void
*/
public function boot()
{
    Storage::extend('dropbox', function ($app, $config) {
        $client = new DropboxClient(
            $config['authorization_token']
        );

        return new Filesystem(new DropboxAdapter($client));
    });
}
```

The first argument of the `extend` method is the name of the driver and the second is a Closure that receives the `$app` and `$config` variables. The resolver Closure must return an instance of `League\Flysystem\Filesystem`. The `$config` variable contains the values defined in `config/filesystems.php` for the specified disk.

Next, register the service provider in your `config/app.php` configuration file:

```
'providers' => [
    // ...
    App\Providers\DropboxServiceProvider::class,
];
```

Once you have created and registered the extension's service provider, you may use the `dropbox` driver in your `config/filesystems.php` configuration file.

Digging Deeper

Helpers

Helpers

- [Introduction](#)
- [Available Methods](#)

Introduction

Laravel includes a variety of global "helper" PHP functions. Many of these functions are used by the framework itself; however, you are free to use them in your own applications if you find them convenient.

Available Methods

Arrays & Objects

Arr::accessible Arr::add	Arr::forget Arr::get Arr::has	Arr::random Arr::set Arr::shuffle
Arr::collapse Arr::crossJoin	Arr::hasAny Arr::isAssoc	Arr::sort Arr::sortRecursive
Arr::divide Arr::dot Arr::except	Arr::last Arr::only Arr::pluck	Arr::where Arr::wrap data fill
Arr::exists Arr::first Arr::flatten	Arr::prepend Arr::pull Arr::query	data get data set head last

Paths

app_path base_path config_path	database_path mix public_path	resource_path storage_path
--	---	--

Strings

__ class basename e	Str::isUuid Str::kebab Str::length	Str::slug Str::snake Str::start
preg_replace array Str::after	Str::limit Str::lower	Str::startsWith Str::studly
Str::afterLast Str::ascii Str::before	Str::orderedUuid Str::padBoth	Str::substr Str::title Str::ucfirst
Str::beforeLast Str::between	Str::padLeft Str::padRight	Str::upper Str::uuid Str::words
Str::camel Str::contains	Str::plural Str::random	trans trans choice
Str::containsAll Str::endsWith	Str::replaceArray Str::replaceFirst	
Str::finish Str::is Str::isAscii	Str::replaceLast Str::singular	

Fluent Strings

after afterLast append ascii	isEmpty kebab length limit	replaceMatches rtrim singular
basename before beforeLast	lower ltrim match matchAll	slug snake split start startsWith
camel contains containsAll	padBoth padLeft padRight plural	studly substr title trim ucfirst
dirname endsWith exactly	prepend replace replaceArray	upper when whenEmpty words
explode finish is isAscii isEmpty	replaceFirst replaceLast	

URLs

action asset route secure_asset	secure_url url
---	--

Miscellaneous

abort abort if abort unless app	dispatch now dump env event	session tap throw if throw unless
auth back bcrypt blank broadcast	factory filled info logger	today trait uses recursive
cache class uses recursive	method field now old optional	transform validator value view
collect config cookie csrf field	policy redirect report request	with

[csrf](#) [token](#) [dd](#) [dispatch](#)

[rescue](#) [resolve](#) [response](#) [retry](#)

Method Listing

Arrays & Objects

Arr::accessible() {collection-method .first-collection-method}

The Arr::accessible method checks that the given value is array accessible:

```
use Illuminate\Support\Arr;
use Illuminate\Support\Collection;

$isAccessible = Arr::accessible(['a' => 1, 'b' => 2]);

// true

$isAccessible = Arr::accessible(new Collection);

// true

$isAccessible = Arr::accessible('abc');

// false

$isAccessible = Arr::accessible(new stdClass);

// false
```

Arr::add() {collection-method}

The Arr::add method adds a given key / value pair to an array if the given key doesn't already exist in the array or is set to null:

```
use Illuminate\Support\Arr;

$array = Arr::add(['name' => 'Desk'], 'price', 100);

// ['name' => 'Desk', 'price' => 100]

$array = Arr::add(['name' => 'Desk', 'price' => null], 'price', 100);

// ['name' => 'Desk', 'price' => 100]
```

Arr::collapse() {collection-method}

The Arr::collapse method collapses an array of arrays into a single array:

```
use Illuminate\Support\Arr;

$array = Arr::collapse([[1, 2, 3], [4, 5, 6], [7, 8, 9]]);

// [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Arr::crossJoin() {collection-method}

The Arr::crossJoin method cross joins the given arrays, returning a Cartesian product with all possible permutations:

```
use Illuminate\Support\Arr;

$matrix = Arr::crossJoin([1, 2], ['a', 'b']);

/*
    [
        [1, 'a'],
        [1, 'b'],
        [2, 'a'],
        [2, 'b'],
    ]
*/
```

```
    ]
*/

$matrix = Arr::crossJoin([1, 2], ['a', 'b'], ['I', 'II']);

/*
    [
        [1, 'a', 'I'],
        [1, 'a', 'II'],
        [1, 'b', 'I'],
        [1, 'b', 'II'],
        [2, 'a', 'I'],
        [2, 'a', 'II'],
        [2, 'b', 'I'],
        [2, 'b', 'II'],
    ]
*/
```

Arr::divide() {collection-method}

The `Arr::divide` method returns two arrays, one containing the keys, and the other containing the values of the given array:

```
use Illuminate\Support\Arr;

[$keys, $values] = Arr::divide(['name' => 'Desk']);

// $keys: ['name']
// $values: ['Desk']
```

Arr::dot() {collection-method}

The `Arr::dot` method flattens a multi-dimensional array into a single level array that uses "dot" notation to indicate depth:

```
use Illuminate\Support\Arr;

$array = ['products' => ['desk' => ['price' => 100]]];

$flattened = Arr::dot($array);

// ['products.desk.price' => 100]
```

Arr::except() {collection-method}

The `Arr::except` method removes the given key / value pairs from an array:

```
use Illuminate\Support\Arr;

$array = ['name' => 'Desk', 'price' => 100];

$filtered = Arr::except($array, ['price']);

// ['name' => 'Desk']
```

Arr::exists() {collection-method}

The `Arr::exists` method checks that the given key exists in the provided array:

```
use Illuminate\Support\Arr;

$array = ['name' => 'John Doe', 'age' => 17];

$exists = Arr::exists($array, 'name');

// true

$exists = Arr::exists($array, 'salary');

// false
```

Arr::first() {collection-method}

The Arr::first method returns the first element of an array passing a given truth test:

```
use Illuminate\Support\Arr;

$array = [100, 200, 300];

$first = Arr::first($array, function ($value, $key) {
    return $value >= 150;
});

// 200
```

A default value may also be passed as the third parameter to the method. This value will be returned if no value passes the truth test:

```
use Illuminate\Support\Arr;

$first = Arr::first($array, $callback, $default);
```

Arr::flatten() {collection-method}

The Arr::flatten method flattens a multi-dimensional array into a single level array:

```
use Illuminate\Support\Arr;

$array = ['name' => 'Joe', 'languages' => ['PHP', 'Ruby']];

$flattened = Arr::flatten($array);

// ['Joe', 'PHP', 'Ruby']
```

Arr::forget() {collection-method}

The Arr::forget method removes a given key / value pair from a deeply nested array using "dot" notation:

```
use Illuminate\Support\Arr;

$array = ['products' => ['desk' => ['price' => 100]]];

Arr::forget($array, 'products.desk');

// ['products' => []]
```

Arr::get() {collection-method}

The Arr::get method retrieves a value from a deeply nested array using "dot" notation:

```
use Illuminate\Support\Arr;

$array = ['products' => ['desk' => ['price' => 100]]];

$price = Arr::get($array, 'products.desk.price');

// 100
```

The Arr::get method also accepts a default value, which will be returned if the specific key is not found:

```
use Illuminate\Support\Arr;

$discount = Arr::get($array, 'products.desk.discount', 0);

// 0
```

Arr::has() {collection-method}

The Arr::has method checks whether a given item or items exists in an array using "dot" notation:

```
use Illuminate\Support\Arr;
```

```
$array = ['product' => ['name' => 'Desk', 'price' => 100]];
$contains = Arr::has($array, 'product.name');

// true

$contains = Arr::has($array, ['product.price', 'product.discount']);

// false
```

Arr::hasAny() {collection-method}

The Arr::hasAny method checks whether any item in a given set exists in an array using "dot" notation:

```
use Illuminate\Support\Arr;

$array = ['product' => ['name' => 'Desk', 'price' => 100]];
$contains = Arr::hasAny($array, 'product.name');

// true

$contains = Arr::hasAny($array, ['product.name', 'product.discount']);

// true

$contains = Arr::hasAny($array, ['category', 'product.discount']);

// false
```

Arr::isAssoc() {collection-method}

The Arr::isAssoc returns true if the given array is an associative array. An array is considered "associative" if it doesn't have sequential numerical keys beginning with zero:

```
use Illuminate\Support\Arr;

$isAssoc = Arr::isAssoc(['product' => ['name' => 'Desk', 'price' => 100]]);

// true

$isAssoc = Arr::isAssoc([1, 2, 3]);

// false
```

Arr::last() {collection-method}

The Arr::last method returns the last element of an array passing a given truth test:

```
use Illuminate\Support\Arr;

$array = [100, 200, 300, 110];

$last = Arr::last($array, function ($value, $key) {
    return $value >= 150;
});

// 300
```

A default value may be passed as the third argument to the method. This value will be returned if no value passes the truth test:

```
use Illuminate\Support\Arr;

$last = Arr::last($array, $callback, $default);
```

Arr::only() {collection-method}

The Arr::only method returns only the specified key / value pairs from the given array:

```
use Illuminate\Support\Arr;
```



```
$array = ['name' => 'Desk', 'price' => 100, 'orders' => 10];  
$slice = Arr::only($array, ['name', 'price']);  
  
// ['name' => 'Desk', 'price' => 100]
```

Arr::pluck() {collection-method}

The Arr::pluck method retrieves all of the values for a given key from an array:

```
use Illuminate\Support\Arr;  
  
$array = [  
    ['developer' => ['id' => 1, 'name' => 'Taylor']],  
    ['developer' => ['id' => 2, 'name' => 'Abigail']],  
];  
  
$names = Arr::pluck($array, 'developer.name');  
  
// ['Taylor', 'Abigail']
```

You may also specify how you wish the resulting list to be keyed:

```
use Illuminate\Support\Arr;  
  
$names = Arr::pluck($array, 'developer.name', 'developer.id');  
  
// [1 => 'Taylor', 2 => 'Abigail']
```

Arr::prepend() {collection-method}

The Arr::prepend method will push an item onto the beginning of an array:

```
use Illuminate\Support\Arr;  
  
$array = ['one', 'two', 'three', 'four'];  
$array = Arr::prepend($array, 'zero');  
  
// ['zero', 'one', 'two', 'three', 'four']
```

If needed, you may specify the key that should be used for the value:

```
use Illuminate\Support\Arr;  
  
$array = ['price' => 100];  
$array = Arr::prepend($array, 'Desk', 'name');  
  
// ['name' => 'Desk', 'price' => 100]
```

Arr::pull() {collection-method}

The Arr::pull method returns and removes a key / value pair from an array:

```
use Illuminate\Support\Arr;  
  
$array = ['name' => 'Desk', 'price' => 100];  
$name = Arr::pull($array, 'name');  
  
// $name: Desk  
  
// $array: ['price' => 100]
```

A default value may be passed as the third argument to the method. This value will be returned if the key doesn't exist:

```
use Illuminate\Support\Arr;  
  
$value = Arr::pull($array, $key, $default);
```

Arr::query() {collection-method}

The Arr::query method converts the array into a query string:

```
use Illuminate\Support\Arr;

$array = ['name' => 'Taylor', 'order' => ['column' => 'created_at', 'direction' => 'desc']];

Arr::query($array);

// name=Taylor&order[column]=created_at&order[direction]=desc
```

Arr::random() {collection-method}

The Arr::random method returns a random value from an array:

```
use Illuminate\Support\Arr;

$array = [1, 2, 3, 4, 5];

$random = Arr::random($array);

// 4 - (retrieved randomly)
```

You may also specify the number of items to return as an optional second argument. Note that providing this argument will return an array, even if only one item is desired:

```
use Illuminate\Support\Arr;

$items = Arr::random($array, 2);

// [2, 5] - (retrieved randomly)
```

Arr::set() {collection-method}

The Arr::set method sets a value within a deeply nested array using "dot" notation:

```
use Illuminate\Support\Arr;

$array = ['products' => ['desk' => ['price' => 100]]];

Arr::set($array, 'products.desk.price', 200);

// ['products' => ['desk' => ['price' => 200]]
```

Arr::shuffle() {collection-method}

The Arr::shuffle method randomly shuffles the items in the array:

```
use Illuminate\Support\Arr;

$array = Arr::shuffle([1, 2, 3, 4, 5]);

// [3, 2, 5, 1, 4] - (generated randomly)
```

Arr::sort() {collection-method}

The Arr::sort method sorts an array by its values:

```
use Illuminate\Support\Arr;

$array = ['Desk', 'Table', 'Chair'];

$sorted = Arr::sort($array);

// ['Chair', 'Desk', 'Table']
```

You may also sort the array by the results of the given Closure:

```
use Illuminate\Support\Arr;
```

```
$array = [
    ['name' => 'Desk'],
    ['name' => 'Table'],
    ['name' => 'Chair'],
];

$sorted = array_values(Arr::sort($array, function ($value) {
    return $value['name'];
}));

/*
    [
        ['name' => 'Chair'],
        ['name' => 'Desk'],
        ['name' => 'Table'],
    ]
*/
```

Arr::sortRecursive() {collection-method}

The Arr::sortRecursive method recursively sorts an array using the sort function for numeric sub=arrays and ksort for associative subarrays:

```
use Illuminate\Support\Arr;

$array = [
    ['Roman', 'Taylor', 'Li'],
    ['PHP', 'Ruby', 'JavaScript'],
    ['one' => 1, 'two' => 2, 'three' => 3],
];

$sorted = Arr::sortRecursive($array);

/*
    [
        ['JavaScript', 'PHP', 'Ruby'],
        ['one' => 1, 'three' => 3, 'two' => 2],
        ['Li', 'Roman', 'Taylor'],
    ]
*/
```

Arr::where() {collection-method}

The Arr::where method filters an array using the given Closure:

```
use Illuminate\Support\Arr;

$array = [100, '200', 300, '400', 500];

$filtered = Arr::where($array, function ($value, $key) {
    return is_string($value);
});

// [1 => '200', 3 => '400']
```

Arr::wrap() {collection-method}

The Arr::wrap method wraps the given value in an array. If the given value is already an array it will not be changed:

```
use Illuminate\Support\Arr;

$string = 'Laravel';

$array = Arr::wrap($string);

// ['Laravel']
```

If the given value is null, an empty array will be returned:

```
use Illuminate\Support\Arr;

$nothing = null;
```

```
$array = Arr::wrap($nothing);  
// []
```

data_fill() {collection-method}

The `data_fill` function sets a missing value within a nested array or object using "dot" notation:

```
$data = ['products' => ['desk' => ['price' => 100]]];  
data_fill($data, 'products.desk.price', 200);  
// ['products' => ['desk' => ['price' => 100]]  
data_fill($data, 'products.desk.discount', 10);  
// ['products' => ['desk' => ['price' => 100, 'discount' => 10]]
```

This function also accepts asterisks as wildcards and will fill the target accordingly:

```
$data = [  
    'products' => [  
        ['name' => 'Desk 1', 'price' => 100],  
        ['name' => 'Desk 2'],  
    ],  
];  
data_fill($data, 'products.*.price', 200);  
/*  
    [  
        'products' => [  
            ['name' => 'Desk 1', 'price' => 100],  
            ['name' => 'Desk 2', 'price' => 200],  
        ],  
    ]  
*/
```

data_get() {collection-method}

The `data_get` function retrieves a value from a nested array or object using "dot" notation:

```
$data = ['products' => ['desk' => ['price' => 100]]];  
$price = data_get($data, 'products.desk.price');  
// 100
```

The `data_get` function also accepts a default value, which will be returned if the specified key is not found:

```
$discount = data_get($data, 'products.desk.discount', 0);  
// 0
```

The function also accepts wildcards using asterisks, which may target any key of the array or object:

```
$data = [  
    'product-one' => ['name' => 'Desk 1', 'price' => 100],  
    'product-two' => ['name' => 'Desk 2', 'price' => 150],  
];  
data_get($data, '*.name');  
// ['Desk 1', 'Desk 2'];
```

data_set() {collection-method}

The `data_set` function sets a value within a nested array or object using "dot" notation:

```
$data = ['products' => ['desk' => ['price' => 100]]];  
data_set($data, 'products.desk.price', 200);
```

```
// ['products' => ['desk' => ['price' => 200]]]
```

This function also accepts wildcards and will set values on the target accordingly:

```
$data = [
    'products' => [
        ['name' => 'Desk 1', 'price' => 100],
        ['name' => 'Desk 2', 'price' => 150],
    ],
];

data_set($data, 'products.*.price', 200);

/*
    [
        'products' => [
            ['name' => 'Desk 1', 'price' => 200],
            ['name' => 'Desk 2', 'price' => 200],
        ],
    ]
*/
```

By default, any existing values are overwritten. If you wish to only set a value if it doesn't exist, you may pass false as the fourth argument:

```
$data = ['products' => ['desk' => ['price' => 100]]];

data_set($data, 'products.desk.price', 200, false);

// ['products' => ['desk' => ['price' => 100]]]
```

head() {collection-method}

The head function returns the first element in the given array:

```
$array = [100, 200, 300];

$first = head($array);

// 100
```

last() {collection-method}

The last function returns the last element in the given array:

```
$array = [100, 200, 300];

$last = last($array);

// 300
```

Paths

app_path() {collection-method}

The app_path function returns the fully qualified path to the app directory. You may also use the app_path function to generate a fully qualified path to a file relative to the application directory:

```
$path = app_path();

$path = app_path('Http/Controllers/Controller.php');
```

base_path() {collection-method}

The base_path function returns the fully qualified path to the project root. You may also use the base_path function to generate a fully qualified path to a given file relative to the project root directory:

```
$path = base_path();
```

```
$path = base_path('vendor/bin');
```

config_path() {collection-method}

The `config_path` function returns the fully qualified path to the `config` directory. You may also use the `config_path` function to generate a fully qualified path to a given file within the application's configuration directory:

```
$path = config_path();
```

```
$path = config_path('app.php');
```

database_path() {collection-method}

The `database_path` function returns the fully qualified path to the `database` directory. You may also use the `database_path` function to generate a fully qualified path to a given file within the database directory:

```
$path = database_path();
```

```
$path = database_path('factories/UserFactory.php');
```

mix() {collection-method}

The `mix` function returns the path to a [versioned Mix file](#):

```
$path = mix('css/app.css');
```

public_path() {collection-method}

The `public_path` function returns the fully qualified path to the `public` directory. You may also use the `public_path` function to generate a fully qualified path to a given file within the public directory:

```
$path = public_path();
```

```
$path = public_path('css/app.css');
```

resource_path() {collection-method}

The `resource_path` function returns the fully qualified path to the `resources` directory. You may also use the `resource_path` function to generate a fully qualified path to a given file within the resources directory:

```
$path = resource_path();
```

```
$path = resource_path('sass/app.scss');
```

storage_path() {collection-method}

The `storage_path` function returns the fully qualified path to the `storage` directory. You may also use the `storage_path` function to generate a fully qualified path to a given file within the storage directory:

```
$path = storage_path();
```

```
$path = storage_path('app/file.txt');
```

Strings

__() {collection-method}

The `__` function translates the given translation string or translation key using your [localization files](#):

```
echo __('Welcome to our application');
```

```
echo __('messages.welcome');
```

If the specified translation string or key does not exist, the `__` function will return the given value. So, using the example above, the `__` function would return `messages.welcome` if that translation key does not exist.

`class_basename()` {.collection-method}

The `class_basename` function returns the class name of the given class with the class's namespace removed:

```
$class = class_basename('Foo\Bar\Baz');  
  
// Baz
```

`e()` {.collection-method}

The `e` function runs PHP's `htmlspecialchars` function with the `double_encode` option set to `true` by default:

```
echo e('<html>foo</html>');  
  
// &lt;html&gt;foo&lt;/html&gt;
```

`preg_replace_array()` {.collection-method}

The `preg_replace_array` function replaces a given pattern in the string sequentially using an array:

```
$string = 'The event will take place between :start and :end';  
$replaced = preg_replace_array('/: [a-z_]+/', ['8:30', '9:00'], $string);  
  
// The event will take place between 8:30 and 9:00
```

`Str::after()` {.collection-method}

The `Str::after` method returns everything after the given value in a string. The entire string will be returned if the value does not exist within the string:

```
use Illuminate\Support\Str;  
  
$slice = Str::after('This is my name', 'This is');  
  
// ' my name'
```

`Str::afterLast()` {.collection-method}

The `Str::afterLast` method returns everything after the last occurrence of the given value in a string. The entire string will be returned if the value does not exist within the string:

```
use Illuminate\Support\Str;  
  
$slice = Str::afterLast('App\Http\Controllers\Controller', '\\');  
  
// 'Controller'
```

`Str::ascii()` {.collection-method}

The `Str::ascii` method will attempt to transliterate the string into an ASCII value:

```
use Illuminate\Support\Str;  
  
$slice = Str::ascii('ü');  
  
// 'u'
```

`Str::before()` {.collection-method}

The `Str::before` method returns everything before the given value in a string:

```
use Illuminate\Support\Str;
```

```
$slice = Str::before('This is my name', 'my name');  
// 'This is '
```

Str::beforeLast() {collection-method}

The `Str::beforeLast` method returns everything before the last occurrence of the given value in a string:

```
use Illuminate\Support\Str;  
$slice = Str::beforeLast('This is my name', 'is');  
// 'This '
```

Str::between() {collection-method}

The `Str::between` method returns the portion of a string between two values:

```
use Illuminate\Support\Str;  
$slice = Str::between('This is my name', 'This', 'name');  
// ' is my '
```

Str::camel() {collection-method}

The `Str::camel` method converts the given string to camelCase:

```
use Illuminate\Support\Str;  
$converted = Str::camel('foo_bar');  
// fooBar
```

Str::contains() {collection-method}

The `Str::contains` method determines if the given string contains the given value (case sensitive):

```
use Illuminate\Support\Str;  
$contains = Str::contains('This is my name', 'my');  
// true
```

You may also pass an array of values to determine if the given string contains any of the values:

```
use Illuminate\Support\Str;  
$contains = Str::contains('This is my name', ['my', 'foo']);  
// true
```

Str::containsAll() {collection-method}

The `Str::containsAll` method determines if the given string contains all array values:

```
use Illuminate\Support\Str;  
$containsAll = Str::containsAll('This is my name', ['my', 'name']);  
// true
```

Str::endsWith() {collection-method}

The `Str::endsWith` method determines if the given string ends with the given value:

```
use Illuminate\Support\Str;  
$result = Str::endsWith('This is my name', 'name');
```



```
// true
```

You may also pass an array of values to determine if the given string ends with any of the given values:

```
use Illuminate\Support\Str;

$result = Str::endsWith('This is my name', ['name', 'foo']);

// true

$result = Str::endsWith('This is my name', ['this', 'foo']);

// false
```

Str::finish() {collection-method}

The `Str::finish` method adds a single instance of the given value to a string if it does not already end with the value:

```
use Illuminate\Support\Str;

$adjusted = Str::finish('this/string', '/');

// this/string/

$adjusted = Str::finish('this/string/', '/');

// this/string/
```

Str::is() {collection-method}

The `Str::is` method determines if a given string matches a given pattern. Asterisks may be used to indicate wildcards:

```
use Illuminate\Support\Str;

$matches = Str::is('foo*', 'foobar');

// true

$matches = Str::is('baz*', 'foobar');

// false
```

Str::isAscii() {collection-method}

The `Str::isAscii` method determines if a given string is 7 bit ASCII:

```
use Illuminate\Support\Str;

$isAscii = Str::isAscii('Taylor');

// true

$isAscii = Str::isAscii('ü');

// false
```

Str::isUuid() {collection-method}

The `Str::isUuid` method determines if the given string is a valid UUID:

```
use Illuminate\Support\Str;

$isUuid = Str::isUuid('a0a2a2d2-0b87-4a18-83f2-2529882be2de');

// true

$isUuid = Str::isUuid('laravel');

// false
```

Str::kebab() {collection-method}

The `Str::kebab` method converts the given string to kebab-case:

```
use Illuminate\Support\Str;

$converted = Str::kebab('fooBar');

// foo-bar
```

Str::length() {collection-method}

The `Str::length` method returns the length of the given string:

```
use Illuminate\Support\Str;

$length = Str::length('Laravel');

// 7
```

Str::limit() {collection-method}

The `Str::limit` method truncates the given string at the specified length:

```
use Illuminate\Support\Str;

$truncated = Str::limit('The quick brown fox jumps over the lazy dog', 20);

// The quick brown fox...
```

You may also pass a third argument to change the string that will be appended to the end:

```
use Illuminate\Support\Str;

$truncated = Str::limit('The quick brown fox jumps over the lazy dog', 20, ' (...)');

// The quick brown fox (...)
```

Str::lower() {collection-method}

The `Str::lower` method converts the given string to lowercase:

```
use Illuminate\Support\Str;

$converted = Str::lower('LARAVEL');

// laravel
```

Str::orderedUuid() {collection-method}

The `Str::orderedUuid` method generates a "timestamp first" UUID that may be efficiently stored in an indexed database column:

```
use Illuminate\Support\Str;

return (string) Str::orderedUuid();
```

Str::padBoth() {collection-method}

The `Str::padBoth` method wraps PHP's `str_pad` function, padding both sides of a string with another:

```
use Illuminate\Support\Str;

$padded = Str::padBoth('James', 10, '_');

// '__James__'

$padded = Str::padBoth('James', 10);
```

```
// ' James '
```

Str::padLeft() {collection-method}

The `Str::padLeft` method wraps PHP's `str_pad` function, padding the left side of a string with another:

```
use Illuminate\Support\Str;

$padding = Str::padLeft('James', 10, '-');

// '----James'

$padding = Str::padLeft('James', 10);

// ' James '
```

Str::padRight() {collection-method}

The `Str::padRight` method wraps PHP's `str_pad` function, padding the right side of a string with another:

```
use Illuminate\Support\Str;

$padding = Str::padRight('James', 10, '-');

// 'James-----'

$padding = Str::padRight('James', 10);

// 'James      '
```

Str::plural() {collection-method}

The `Str::plural` method converts a single word string to its plural form. This function currently only supports the English language:

```
use Illuminate\Support\Str;

$plural = Str::plural('car');

// cars

$plural = Str::plural('child');

// children
```

You may provide an integer as a second argument to the function to retrieve the singular or plural form of the string:

```
use Illuminate\Support\Str;

$plural = Str::plural('child', 2);

// children

$plural = Str::plural('child', 1);

// child
```

Str::random() {collection-method}

The `Str::random` method generates a random string of the specified length. This function uses PHP's `random_bytes` function:

```
use Illuminate\Support\Str;

$random = Str::random(40);
```

Str::replaceArray() {collection-method}

The `Str::replaceArray` method replaces a given value in the string sequentially using an array:

```
use Illuminate\Support\Str;

$string = 'The event will take place between ? and ?';

$replaced = Str::replaceArray('?', ['8:30', '9:00'], $string);

// The event will take place between 8:30 and 9:00
```

Str::replaceFirst() {collection-method}

The `Str::replaceFirst` method replaces the first occurrence of a given value in a string:

```
use Illuminate\Support\Str;

$replaced = Str::replaceFirst('the', 'a', 'the quick brown fox jumps over the lazy dog');

// a quick brown fox jumps over the lazy dog
```

Str::replaceLast() {collection-method}

The `Str::replaceLast` method replaces the last occurrence of a given value in a string:

```
use Illuminate\Support\Str;

$replaced = Str::replaceLast('the', 'a', 'the quick brown fox jumps over the lazy dog');

// the quick brown fox jumps over a lazy dog
```

Str::singular() {collection-method}

The `Str::singular` method converts a string to its singular form. This function currently only supports the English language:

```
use Illuminate\Support\Str;

$singular = Str::singular('cars');

// car

$singular = Str::singular('children');

// child
```

Str::slug() {collection-method}

The `Str::slug` method generates a URL friendly "slug" from the given string:

```
use Illuminate\Support\Str;

$slug = Str::slug('Laravel 5 Framework', '-');

// laravel-5-framework
```

Str::snake() {collection-method}

The `Str::snake` method converts the given string to `snake_case`:

```
use Illuminate\Support\Str;

$converted = Str::snake('fooBar');

// foo_bar
```

Str::start() {collection-method}

The `Str::start` method adds a single instance of the given value to a string if it does not already start with the value:

```
use Illuminate\Support\Str;
```

```
$adjusted = Str::start('this/string', '/');  
// /this/string  
  
$adjusted = Str::start('/this/string', '/');  
// /this/string
```

Str::startsWith() {collection-method}

The `Str::startsWith` method determines if the given string begins with the given value:

```
use Illuminate\Support\Str;  
  
$result = Str::startsWith('This is my name', 'This');  
  
// true
```

Str::studly() {collection-method}

The `Str::studly` method converts the given string to StudlyCase:

```
use Illuminate\Support\Str;  
  
$converted = Str::studly('foo_bar');  
  
// FooBar
```

Str::substr() {collection-method}

The `Str::substr` method returns the portion of string specified by the start and length parameters:

```
use Illuminate\Support\Str;  
  
$converted = Str::substr('The Laravel Framework', 4, 7);  
  
// Laravel
```

Str::title() {collection-method}

The `Str::title` method converts the given string to Title Case:

```
use Illuminate\Support\Str;  
  
$converted = Str::title('a nice title uses the correct case');  
  
// A Nice Title Uses The Correct Case
```

Str::ucfirst() {collection-method}

The `Str::ucfirst` method returns the given string with the first character capitalized:

```
use Illuminate\Support\Str;  
  
$string = Str::ucfirst('foo bar');  
  
// Foo bar
```

Str::upper() {collection-method}

The `Str::upper` method converts the given string to uppercase:

```
use Illuminate\Support\Str;  
  
$string = Str::upper('laravel');  
  
// LARAVEL
```

Str::uuid() {.collection-method}

The Str::uuid method generates a UUID (version 4):

```
use Illuminate\Support\Str;

return (string) Str::uuid();
```

Str::words() {.collection-method}

The Str::words method limits the number of words in a string:

```
use Illuminate\Support\Str;

return Str::words('Perfectly balanced, as all things should be.', 3, ' >>>');

// Perfectly balanced, as >>>
```

trans() {.collection-method}

The trans function translates the given translation key using your [localization files](#):

```
echo trans('messages.welcome');
```

If the specified translation key does not exist, the trans function will return the given key. So, using the example above, the trans function would return `messages.welcome` if the translation key does not exist.

trans_choice() {.collection-method}

The trans_choice function translates the given translation key with inflection:

```
echo trans_choice('messages.notifications', $unreadCount);
```

If the specified translation key does not exist, the trans_choice function will return the given key. So, using the example above, the trans_choice function would return `messages.notifications` if the translation key does not exist.

Fluent Strings

Fluent strings provide a more fluent, object-oriented interface for working with string values, allowing you to chain multiple string operations together using a more readable syntax compared to traditional string operations.

after {.collection-method}

The after method returns everything after the given value in a string. The entire string will be returned if the value does not exist within the string:

```
use Illuminate\Support\Str;

$slice = Str::of('This is my name')->after('This is');

// ' my name'
```

afterLast {.collection-method}

The afterLast method returns everything after the last occurrence of the given value in a string. The entire string will be returned if the value does not exist within the string:

```
use Illuminate\Support\Str;

$slice = Str::of('App\Http\Controllers\Controller')->afterLast('\');

// 'Controller'
```

append {collection-method}

The append method appends the given values to the string:

```
use Illuminate\Support\Str;

$string = Str::of('Taylor')->append(' Otwell');

// 'Taylor Otwell'
```

ascii {collection-method}

The ascii method will attempt to transliterate the string into an ASCII value:

```
use Illuminate\Support\Str;

$string = Str::of('ü')->ascii();

// 'u'
```

basename {collection-method}

The basename method will return the trailing name component of the given string:

```
use Illuminate\Support\Str;

$string = Str::of('/foo/bar/baz')->basename();

// 'baz'
```

If needed, you may provide an "extension" that will be removed from the trailing component:

```
use Illuminate\Support\Str;

$string = Str::of('/foo/bar/baz.jpg')->basename('.jpg');

// 'baz'
```

before {collection-method}

The before method returns everything before the given value in a string:

```
use Illuminate\Support\Str;

$slice = Str::of('This is my name')->before('my name');

// 'This is '
```

beforeLast {collection-method}

The beforeLast method returns everything before the last occurrence of the given value in a string:

```
use Illuminate\Support\Str;

$slice = Str::of('This is my name')->beforeLast('is');

// 'This '
```

camel {collection-method}

The camel method converts the given string to camelCase:

```
use Illuminate\Support\Str;

$converted = Str::of('foo_bar')->camel();

// fooBar
```

contains {collection-method}

The `contains` method determines if the given string contains the given value (case sensitive):

```
use Illuminate\Support\Str;

$contains = Str::of('This is my name')->contains('my');

// true
```

You may also pass an array of values to determine if the given string contains any of the values:

```
use Illuminate\Support\Str;

$contains = Str::of('This is my name')->contains(['my', 'foo']);

// true
```

`containsAll {collection-method}`

The `containsAll` method determines if the given string contains all array values:

```
use Illuminate\Support\Str;

$containsAll = Str::of('This is my name')->containsAll(['my', 'name']);

// true
```

`dirname {collection-method}`

The `dirname` method returns the parent directory portion of the given string:

```
use Illuminate\Support\Str;

$string = Str::of('/foo/bar/baz')->dirname();

// '/foo/bar'
```

Optionally, You may specify how many directory levels you wish to trim from the string:

```
use Illuminate\Support\Str;

$string = Str::of('/foo/bar/baz')->dirname(2);

// '/foo'
```

`endsWith {collection-method}`

The `endsWith` method determines if the given string ends with the given value:

```
use Illuminate\Support\Str;

$result = Str::of('This is my name')->endsWith('name');

// true
```

You may also pass an array of values to determine if the given string ends with any of the given values:

```
use Illuminate\Support\Str;

$result = Str::of('This is my name')->endsWith(['name', 'foo']);

// true

$result = Str::of('This is my name')->endsWith(['this', 'foo']);

// false
```

`exactly {collection-method}`

The `exactly` method determines if the given string is an exact match with another string:

```
use Illuminate\Support\Str;
```



```
$result = Str::of('Laravel')->exactly('Laravel');  
// true
```

explode {collection-method}

The `explode` method splits the string by the given delimiter and returns a collection containing each section of the split string:

```
use Illuminate\Support\Str;  
$collection = Str::of('foo bar baz')->explode(' ');  
// collect(['foo', 'bar', 'baz'])
```

finish {collection-method}

The `finish` method adds a single instance of the given value to a string if it does not already end with the value:

```
use Illuminate\Support\Str;  
$adjusted = Str::of('this/string')->finish('/');  
// this/string/  
$adjusted = Str::of('this/string/')->finish('/');  
// this/string/
```

is {collection-method}

The `is` method determines if a given string matches a given pattern. Asterisks may be used to indicate wildcards:

```
use Illuminate\Support\Str;  
$matches = Str::of('foobar')->is('foo*');  
// true  
$matches = Str::of('foobar')->is('baz*');  
// false
```

isAscii {collection-method}

The `isAscii` method determines if a given string is an ASCII string:

```
use Illuminate\Support\Str;  
$result = Str::of('Taylor')->isAscii();  
// true  
$result = Str::of('ü')->isAscii();  
// false
```

isEmpty {collection-method}

The `isEmpty` method determines if the given string is empty:

```
use Illuminate\Support\Str;  
$result = Str::of(' ')->trim()->isEmpty();  
// true  
$result = Str::of('Laravel')->trim()->isEmpty();
```

```
// false
```

isEmpty {collection-method}

The `isEmpty` method determines if the given string is not empty:

```
use Illuminate\Support\Str;

$result = Str::of(' ')->trim()->isEmpty();

// false

$result = Str::of('Laravel')->trim()->isEmpty();

// true
```

kebab {collection-method}

The `kebab` method converts the given string to kebab-case:

```
use Illuminate\Support\Str;

$converted = Str::of('fooBar')->kebab();

// foo-bar
```

length {collection-method}

The `length` method returns the length of the given string:

```
use Illuminate\Support\Str;

$length = Str::of('Laravel')->length();

// 7
```

limit {collection-method}

The `limit` method truncates the given string at the specified length:

```
use Illuminate\Support\Str;

$truncated = Str::of('The quick brown fox jumps over the lazy dog')->limit(20);

// The quick brown fox...
```

You may also pass a second argument to change the string that will be appended to the end:

```
use Illuminate\Support\Str;

$truncated = Str::of('The quick brown fox jumps over the lazy dog')->limit(20, ' (...)');

// The quick brown fox (...)
```

lower {collection-method}

The `lower` method converts the given string to lowercase:

```
use Illuminate\Support\Str;

$result = Str::of('LARAVEL')->lower();

// 'laravel'
```

ltrim {collection-method}

The `ltrim` method left trims the given string:

```
use Illuminate\Support\Str;
```

```
$string = Str::of(' Laravel ')->ltrim();  
// 'Laravel '  
  
$string = Str::of('/Laravel/')->ltrim('/');  
// 'Laravel/'
```

match {collection-method}

The `match` method will return the portion of a string that matches a given regular expression pattern:

```
use Illuminate\Support\Str;  
  
$result = Str::of('foo bar')->match('/bar/');  
// 'bar'  
  
$result = Str::of('foo bar')->match('/foo (.*)/');  
// 'bar'
```

matchAll {collection-method}

The `matchAll` method will return a collection containing the portions of a string that match a given regular expression pattern:

```
use Illuminate\Support\Str;  
  
$result = Str::of('bar foo bar')->matchAll('/bar/');  
// collect(['bar', 'bar'])
```

If you specify a matching group within the expression, Laravel will return a collection of that group's matches:

```
use Illuminate\Support\Str;  
  
$result = Str::of('bar fun bar fly')->matchAll('/f(\w*)/');  
// collect(['un', 'ly'])
```

If no matches are found, an empty collection will be returned.

padBoth {collection-method}

The `padBoth` method wraps PHP's `str_pad` function, padding both sides of a string with another:

```
use Illuminate\Support\Str;  
  
$padded = Str::of('James')->padBoth(10, '_');  
// '__James__'  
  
$padded = Str::of('James')->padBoth(10);  
// ' James '
```

padLeft {collection-method}

The `padLeft` method wraps PHP's `str_pad` function, padding the left side of a string with another:

```
use Illuminate\Support\Str;  
  
$padded = Str::of('James')->padLeft(10, '-=');  
// '-==James'  
  
$padded = Str::of('James')->padLeft(10);  
// ' James'
```

padRight {collection-method}

The `padRight` method wraps PHP's `str_pad` function, padding the right side of a string with another:

```
use Illuminate\Support\Str;

$padding = Str::of('James')->padRight(10, '-');

// 'James-----'

$padding = Str::of('James')->padRight(10);

// 'James      '
```

plural {collection-method}

The `plural` method converts a single word string to its plural form. This function currently only supports the English language:

```
use Illuminate\Support\Str;

$plural = Str::of('car')->plural();

// cars

$plural = Str::of('child')->plural();

// children
```

You may provide an integer as a second argument to the function to retrieve the singular or plural form of the string:

```
use Illuminate\Support\Str;

$plural = Str::of('child')->plural(2);

// children

$plural = Str::of('child')->plural(1);

// child
```

prepend {collection-method}

The `prepend` method prepends the given values onto the string:

```
use Illuminate\Support\Str;

$string = Str::of('Framework')->prepend('Laravel ');

// Laravel Framework
```

replace {collection-method}

The `replace` method replaces a given string within the string:

```
use Illuminate\Support\Str;

$replaced = Str::of('Laravel 6.x')->replace('6.x', '7.x');

// Laravel 7.x
```

replaceArray {collection-method}

The `replaceArray` method replaces a given value in the string sequentially using an array:

```
use Illuminate\Support\Str;

$string = 'The event will take place between ? and ?';

$replaced = Str::of($string)->replaceArray('?', ['8:30', '9:00']);
```

```
// The event will take place between 8:30 and 9:00
```

replaceFirst {collection-method}

The `replaceFirst` method replaces the first occurrence of a given value in a string:

```
use Illuminate\Support\Str;

$replaced = Str::of('the quick brown fox jumps over the lazy dog')->replaceFirst('the', 'a');

// a quick brown fox jumps over the lazy dog
```

replaceLast {collection-method}

The `replaceLast` method replaces the last occurrence of a given value in a string:

```
use Illuminate\Support\Str;

$replaced = Str::of('the quick brown fox jumps over the lazy dog')->replaceLast('the', 'a');

// the quick brown fox jumps over a lazy dog
```

replaceMatches {collection-method}

The `replaceMatches` method replaces all portions of a string matching a given pattern with the given replacement string:

```
use Illuminate\Support\Str;

$replaced = Str::of('(+) 501-555-1000')->replaceMatches('/^[^A-Za-z0-9]++/', '')

// '15015551000'
```

The `replaceMatches` method also accepts a Closure that will be invoked with each portion of the string matching the given party, allowing you to perform the replacement logic within the Closure and return the replaced value:

```
use Illuminate\Support\Str;

$replaced = Str::of('123')->replaceMatches('/\d/', function ($match) {
    return '['.$match[0].']';
});

// '[1][2][3]'
```

rtrim {collection-method}

The `rtrim` method right trims the given string:

```
use Illuminate\Support\Str;

$string = Str::of(' Laravel ')->rtrim();

// ' Laravel'

$string = Str::of('/Laravel/')->rtrim('/');

// '/Laravel'
```

singular {collection-method}

The `singular` method converts a string to its singular form. This function currently only supports the English language:

```
use Illuminate\Support\Str;

$singular = Str::of('cars')->singular();

// car
```

```
$singular = Str::of('children')->singular();  
  
// child
```

slug {collection-method}

The `slug` method generates a URL friendly "slug" from the given string:

```
use Illuminate\Support\Str;  
  
$slug = Str::of('Laravel Framework')->slug('-');  
  
// laravel-framework
```

snake {collection-method}

The `snake` method converts the given string to `snake_case`:

```
use Illuminate\Support\Str;  
  
$converted = Str::of('fooBar')->snake();  
  
// foo_bar
```

split {collection-method}

The `split` method splits a string into a collection using a regular expression:

```
use Illuminate\Support\Str;  
  
$segments = Str::of('one, two, three')->split('/[\s,]+/');  
  
// collect(["one", "two", "three"])
```

start {collection-method}

The `start` method adds a single instance of the given value to a string if it does not already start with the value:

```
use Illuminate\Support\Str;  
  
$adjusted = Str::of('this/string')->start('/');  
  
// /this/string  
  
$adjusted = Str::of('/this/string')->start('/');  
  
// /this/string
```

startswith {collection-method}

The `startswith` method determines if the given string begins with the given value:

```
use Illuminate\Support\Str;  
  
$result = Str::of('This is my name')->startsWith('This');  
  
// true
```

studly {collection-method}

The `studly` method converts the given string to `StudlyCase`:

```
use Illuminate\Support\Str;  
  
$converted = Str::of('foo_bar')->studly();  
  
// FooBar
```

substr {collection-method}

The `substr` method returns the portion of the string specified by the given start and length parameters:

```
use Illuminate\Support\Str;

$string = Str::of('Laravel Framework')->substr(8);

// Framework

$string = Str::of('Laravel Framework')->substr(8, 5);

// Frame
```

title {collection-method}

The `title` method converts the given string to Title Case:

```
use Illuminate\Support\Str;

$converted = Str::of('a nice title uses the correct case')->title();

// A Nice Title Uses The Correct Case
```

trim {collection-method}

The `trim` method trims the given string:

```
use Illuminate\Support\Str;

$string = Str::of('  Laravel  ')->trim();

// 'Laravel'

$string = Str::of('/Laravel/')->trim('/');

// 'Laravel'
```

ucfirst {collection-method}

The `ucfirst` method returns the given string with the first character capitalized:

```
use Illuminate\Support\Str;

$string = Str::of('foo bar')->ucfirst();

// Foo bar
```

upper {collection-method}

The `upper` method converts the given string to uppercase:

```
use Illuminate\Support\Str;

$adjusted = Str::of('laravel')->upper();

// LARAVEL
```

when {collection-method}

The `when` method invokes the given Closure if a given condition is true. The Closure will receive the fluent string instance:

```
use Illuminate\Support\Str;

$string = Str::of('Taylor')
    ->when(true, function ($string) {
        return $string->append(' Otwell');
    });

// 'Taylor Otwell'
```

If necessary, you may pass another Closure as the third parameter to the `when` method. This Closure will execute if the condition parameter evaluates to `false`.

`whenEmpty {collection-method}`

The `whenEmpty` method invokes the given Closure if the string is empty. If the Closure returns a value, that value will also be returned by the `whenEmpty` method. If the Closure does not return a value, the fluent string instance will be returned:

```
use Illuminate\Support\Str;

$string = Str::of(' ')->whenEmpty(function ($string) {
    return $string->trim()->prepend('Laravel');
});

// 'Laravel'
```

`words {collection-method}`

The `words` method limits the number of words in a string:

```
use Illuminate\Support\Str;

$string = Str::of('Perfectly balanced, as all things should be.')->words(3, ' >>>');

// Perfectly balanced, as >>>
```

URLs

`action() {collection-method}`

The `action` function generates a URL for the given controller action. You do not need to pass the full namespace of the controller. Instead, pass the controller class name relative to the `App\Http\Controllers` namespace:

```
$url = action('HomeController@index');

$url = action([HomeController::class, 'index']);
```

If the method accepts route parameters, you may pass them as the second argument to the method:

```
$url = action('UserController@profile', ['id' => 1]);
```

`asset() {collection-method}`

The `asset` function generates a URL for an asset using the current scheme of the request (HTTP or HTTPS):

```
$url = asset('img/photo.jpg');
```

You can configure the asset URL host by setting the `ASSET_URL` variable in your `.env` file. This can be useful if you host your assets on an external service like Amazon S3:

```
// ASSET_URL=http://example.com/assets

$url = asset('img/photo.jpg'); // http://example.com/assets/img/photo.jpg
```

`route() {collection-method}`

The `route` function generates a URL for the given named route:

```
$url = route('routeName');
```

If the route accepts parameters, you may pass them as the second argument to the method:

```
$url = route('routeName', ['id' => 1]);
```


By default, the `route` function generates an absolute URL. If you wish to generate a relative URL, you may pass `false` as the third argument:

```
$url = route('routeName', ['id' => 1], false);
```

`secure_asset()` {collection-method}

The `secure_asset` function generates a URL for an asset using HTTPS:

```
$url = secure_asset('img/photo.jpg');
```

`secure_url()` {collection-method}

The `secure_url` function generates a fully qualified HTTPS URL to the given path:

```
$url = secure_url('user/profile');
```

```
$url = secure_url('user/profile', [1]);
```

`url()` {collection-method}

The `url` function generates a fully qualified URL to the given path:

```
$url = url('user/profile');
```

```
$url = url('user/profile', [1]);
```

If no path is provided, a `Illuminate\Routing\UrlGenerator` instance is returned:

```
$current = url()->current();
```

```
$full = url()->full();
```

```
$previous = url()->previous();
```

Miscellaneous

`abort()` {collection-method}

The `abort` function throws [an HTTP exception](#) which will be rendered by the [exception handler](#):

```
abort(403);
```

You may also provide the exception's response text and custom response headers:

```
abort(403, 'Unauthorized.', $headers);
```

`abort_if()` {collection-method}

The `abort_if` function throws an HTTP exception if a given boolean expression evaluates to `true`:

```
abort_if(! Auth::user()->isAdmin(), 403);
```

Like the `abort` method, you may also provide the exception's response text as the third argument and an array of custom response headers as the fourth argument.

`abort_unless()` {collection-method}

The `abort_unless` function throws an HTTP exception if a given boolean expression evaluates to `false`:

```
abort_unless(Auth::user()->isAdmin(), 403);
```

Like the `abort` method, you may also provide the exception's response text as the third argument and an array of custom response headers as the fourth argument.

app() {.collection-method}

The app function returns the [service container](#) instance:

```
$container = app();
```

You may pass a class or interface name to resolve it from the container:

```
$api = app('HelpSpot\API');
```

auth() {.collection-method}

The auth function returns an [authenticator](#) instance. You may use it instead of the Auth facade for convenience:

```
$user = auth()->user();
```

If needed, you may specify which guard instance you would like to access:

```
$user = auth('admin')->user();
```

back() {.collection-method}

The back function generates a [redirect HTTP response](#) to the user's previous location:

```
return back($status = 302, $headers = [], $fallback = false);
```

```
return back();
```

bcrypt() {.collection-method}

The bcrypt function [hashes](#) the given value using Bcrypt. You may use it as an alternative to the Hash facade:

```
$password = bcrypt('my-secret-password');
```

blank() {.collection-method}

The blank function returns whether the given value is "blank":

```
blank('');  
blank(' ');  
blank(null);  
blank(collect());
```

```
// true
```

```
blank(0);  
blank(true);  
blank(false);
```

```
// false
```

For the inverse of blank, see the [filled](#) method.

broadcast() {.collection-method}

The broadcast function [broadcasts](#) the given [event](#) to its listeners:

```
broadcast(new UserRegistered($user));
```

cache() {.collection-method}

The cache function may be used to get values from the [cache](#). If the given key does not exist in the cache, an optional default value will be returned:

```
$value = cache('key');
```

```
$value = cache('key', 'default');
```

You may add items to the cache by passing an array of key / value pairs to the function. You should also pass the number of seconds or duration the cached value should be considered valid:

```
cache(['key' => 'value'], 300);  
cache(['key' => 'value'], now()->addSeconds(10));
```

class_uses_recursive() {collection-method}

The `class_uses_recursive` function returns all traits used by a class, including traits used by all of its parent classes:

```
$traits = class_uses_recursive(App\User::class);
```

collect() {collection-method}

The `collect` function creates a [collection](#) instance from the given value:

```
$collection = collect(['taylor', 'abigail']);
```

config() {collection-method}

The `config` function gets the value of a [configuration](#) variable. The configuration values may be accessed using "dot" syntax, which includes the name of the file and the option you wish to access. A default value may be specified and is returned if the configuration option does not exist:

```
$value = config('app.timezone');  
$value = config('app.timezone', $default);
```

You may set configuration variables at runtime by passing an array of key / value pairs:

```
config(['app.debug' => true]);
```

cookie() {collection-method}

The `cookie` function creates a new [cookie](#) instance:

```
$cookie = cookie('name', 'value', $minutes);
```

csrf_field() {collection-method}

The `csrf_field` function generates an HTML hidden input field containing the value of the CSRF token. For example, using [Blade syntax](#):

```
{{ csrf_field() }}
```

csrf_token() {collection-method}

The `csrf_token` function retrieves the value of the current CSRF token:

```
$token = csrf_token();
```

dd() {collection-method}

The `dd` function dumps the given variables and ends execution of the script:

```
dd($value);  
dd($value1, $value2, $value3, ...);
```

If you do not want to halt the execution of your script, use the [dump](#) function instead.

dispatch() {collection-method}

The `dispatch` function pushes the given [job](#) onto the Laravel [job queue](#):

```
dispatch(new App\Jobs\SendEmails);
```

dispatch_now() {collection-method}

The `dispatch_now` function runs the given [job](#) immediately and returns the value from its `handle` method:

```
$result = dispatch_now(new App\Jobs\SendEmails);
```

dump() {collection-method}

The `dump` function dumps the given variables:

```
dump($value);  
dump($value1, $value2, $value3, ...);
```

If you want to stop executing the script after dumping the variables, use the [dd](#) function instead.

env() {collection-method}

The `env` function retrieves the value of an [environment variable](#) or returns a default value:

```
$env = env('APP_ENV');  
  
// Returns 'production' if APP_ENV is not set...  
$env = env('APP_ENV', 'production');
```

NOTE If you execute the `config:cache` command during your deployment process, you should be sure that you are only calling the `env` function from within your configuration files. Once the configuration has been cached, the `.env` file will not be loaded and all calls to the `env` function will return `null`.

event() {collection-method}

The `event` function dispatches the given [event](#) to its listeners:

```
event(new UserRegistered($user));
```

factory() {collection-method}

The `factory` function creates a model factory builder for a given class, name, and amount. It can be used while [testing](#) or [seeding](#):

```
$user = factory(App\User::class)->make();
```

filled() {collection-method}

The `filled` function returns whether the given value is not "blank":

```
filled(0);  
filled(true);  
filled(false);  
  
// true  
  
filled('');  
filled(' ');  
filled(null);  
filled(collect());  
  
// false
```

For the inverse of `filled`, see the [blank](#) method.

info() {collection-method}

The `info` function will write information to the [log](#):

```
info('Some helpful information!');
```

An array of contextual data may also be passed to the function:

```
info('User login attempt failed.', ['id' => $user->id]);
```

logger() {collection-method}

The `logger` function can be used to write a debug level message to the [log](#):

```
logger('Debug message');
```

An array of contextual data may also be passed to the function:

```
logger('User has logged in.', ['id' => $user->id]);
```

A [logger](#) instance will be returned if no value is passed to the function:

```
logger()->error('You are not allowed here.');
```

method_field() {collection-method}

The `method_field` function generates an HTML hidden input field containing the spoofed value of the form's HTTP verb. For example, using [Blade syntax](#):

```
<form method="POST">
    {{ method_field('DELETE') }}
</form>
```

now() {collection-method}

The `now` function creates a new `Illuminate\Support\Carbon` instance for the current time:

```
$now = now();
```

old() {collection-method}

The `old` function [retrieves](#) an [old input](#) value flashed into the session:

```
$value = old('value');
```

```
$value = old('value', 'default');
```

optional() {collection-method}

The `optional` function accepts any argument and allows you to access properties or call methods on that object. If the given object is `null`, properties and methods will return `null` instead of causing an error:

```
return optional($user->address)->street;
```

```
{!! old('name', optional($user)->name) !!}
```

The `optional` function also accepts a Closure as its second argument. The Closure will be invoked if the value provided as the first argument is not `null`:

```
return optional(User::find($id), function ($user) {
    return new DummyUser;
});
```

policy() {collection-method}

The policy method retrieves a [policy](#) instance for a given class:

```
$policy = policy(App\User::class);
```

redirect() {collection-method}

The redirect function returns a [redirect HTTP response](#), or returns the redirector instance if called with no arguments:

```
return redirect($to = null, $status = 302, $headers = [], $secure = null);  
return redirect('/home');  
return redirect()->route('route.name');
```

report() {collection-method}

The report function will report an exception using your [exception handler](#)'s report method:

```
report($e);
```

request() {collection-method}

The request function returns the current [request](#) instance or obtains an input item:

```
$request = request();  
$value = request('key', $default);
```

rescue() {collection-method}

The rescue function executes the given Closure and catches any exceptions that occur during its execution. All exceptions that are caught will be sent to your [exception handler](#)'s report method; however, the request will continue processing:

```
return rescue(function () {  
    return $this->method();  
});
```

You may also pass a second argument to the rescue function. This argument will be the "default" value that should be returned if an exception occurs while executing the Closure:

```
return rescue(function () {  
    return $this->method();  
}, false);  
  
return rescue(function () {  
    return $this->method();  
}, function () {  
    return $this->failure();  
});
```

resolve() {collection-method}

The resolve function resolves a given class or interface name to its instance using the [service container](#):

```
$api = resolve('HelpSpot\API');
```

response() {collection-method}

The response function creates a [response](#) instance or obtains an instance of the response factory:

```
return response('Hello World', 200, $headers);  
return response()->json(['foo' => 'bar'], 200, $headers);
```

retry() {collection-method}

The `retry` function attempts to execute the given callback until the given maximum attempt threshold is met. If the callback does not throw an exception, its return value will be returned. If the callback throws an exception, it will automatically be retried. If the maximum attempt count is exceeded, the exception will be thrown:

```
return retry(5, function () {  
    // Attempt 5 times while resting 100ms in between attempts...  
}, 100);
```

session() {collection-method}

The `session` function may be used to get or set [session](#) values:

```
$value = session('key');
```

You may set values by passing an array of key / value pairs to the function:

```
session(['chairs' => 7, 'instruments' => 3]);
```

The session store will be returned if no value is passed to the function:

```
$value = session()->get('key');  
session()->put('key', $value);
```

tap() {collection-method}

The `tap` function accepts two arguments: an arbitrary `$value` and a Closure. The `$value` will be passed to the Closure and then be returned by the `tap` function. The return value of the Closure is irrelevant:

```
$user = tap(User::first(), function ($user) {  
    $user->name = 'taylor';  
  
    $user->save();  
});
```

If no Closure is passed to the `tap` function, you may call any method on the given `$value`. The return value of the method you call will always be `$value`, regardless of what the method actually returns in its definition. For example, the Eloquent `update` method typically returns an integer. However, we can force the method to return the model itself by chaining the `update` method call through the `tap` function:

```
$user = tap($user)->update([  
    'name' => $name,  
    'email' => $email,  
]);
```

To add a `tap` method to a class, you may add the `Illuminate\Support\Traits\Tappable` trait to the class. The `tap` method of this trait accepts a Closure as its only argument. The object instance itself will be passed to the Closure and then be returned by the `tap` method:

```
return $user->tap(function ($user) {  
    //  
});
```

throw_if() {collection-method}

The `throw_if` function throws the given exception if a given boolean expression evaluates to true:

```
throw_if(! Auth::user()->isAdmin(), AuthorizationException::class);  
  
throw_if(  
    ! Auth::user()->isAdmin(),  
    AuthorizationException::class,  
    'You are not allowed to access this page'  
);
```

throw_unless() {collection-method}

The `throw_unless` function throws the given exception if a given boolean expression evaluates to `false`:

```
throw_unless(Auth::user()->isAdmin(), AuthorizationException::class);

throw_unless(
    Auth::user()->isAdmin(),
    AuthorizationException::class,
    'You are not allowed to access this page'
);
```

today() {.collection-method}

The `today` function creates a new `Illuminate\Support\Carbon` instance for the current date:

```
$today = today();
```

trait_uses_recursive() {.collection-method}

The `trait_uses_recursive` function returns all traits used by a trait:

```
$traits = trait_uses_recursive(\Illuminate\Notifications\Notifiable::class);
```

transform() {.collection-method}

The `transform` function executes a closure on a given value if the value is not [blank](#) and returns the result of the closure:

```
$callback = function ($value) {
    return $value * 2;
};

$result = transform(5, $callback);

// 10
```

A default value or closure may also be passed as the third parameter to the method. This value will be returned if the given value is blank:

```
$result = transform(null, $callback, 'The value is blank');

// The value is blank
```

validator() {.collection-method}

The `validator` function creates a new [validator](#) instance with the given arguments. You may use it instead of the `validator` facade for convenience:

```
$validator = validator($data, $rules, $messages);
```

value() {.collection-method}

The `value` function returns the value it is given. However, if you pass a closure to the function, the closure will be executed then its result will be returned:

```
$result = value(true);

// true

$result = value(function () {
    return false;
});

// false
```

view() {.collection-method}

The `view` function retrieves a [view](#) instance:


```
return view('auth.login');
```

with() {.collection-method}

The `with` function returns the value it is given. If a closure is passed as the second argument to the function, the closure will be executed and its result will be returned:

```
$callback = function ($value) {  
    return (is_numeric($value)) ? $value * 2 : 0;  
};  
  
$result = with(5, $callback);  
  
// 10  
  
$result = with(null, $callback);  
  
// 0  
  
$result = with(5, null);  
  
// 5
```

Digging Deeper

HTTP Client

HTTP Client

- [Introduction](#)
- [Making Requests](#)
 - [Request Data](#)
 - [Headers](#)
 - [Authentication](#)
 - [Timeout](#)
 - [Retries](#)
 - [Error Handling](#)
 - [Guzzle Options](#)
- [Testing](#)
 - [Faking Responses](#)
 - [Inspecting Requests](#)

Introduction

Laravel provides an expressive, minimal API around the [Guzzle HTTP client](#), allowing you to quickly make outgoing HTTP requests to communicate with other web applications. Laravel's wrapper around Guzzle is focused on its most common use cases and a wonderful developer experience.

Before getting started, you should ensure that you have installed the Guzzle package as a dependency of your application. By default, Laravel automatically includes this dependency:

```
composer require guzzlehttp/guzzle
```

Making Requests

To make requests, you may use the `get`, `post`, `put`, `patch`, and `delete` methods. First, let's examine how to make a basic `GET` request:

```
use Illuminate\Support\Facades\Http;

$response = Http::get('http://test.com');
```

The `get` method returns an instance of `Illuminate\Http\Client\Response`, which provides a variety of methods that may be used to inspect the response:

```
$response->body() : string;
$response->json() : array|mixed;
$response->status() : int;
$response->ok() : bool;
$response->successful() : bool;
$response->failed() : bool;
$response->serverError() : bool;
$response->clientError() : bool;
$response->header($header) : string;
$response->headers() : array;
```

The `Illuminate\Http\Client\Response` object also implements the `PHP ArrayAccess` interface, allowing you to access JSON response data directly on the response:

```
return Http::get('http://test.com/users/1')['name'];
```

Request Data

Of course, it is common when using `POST`, `PUT`, and `PATCH` to send additional data with your request. So, these methods accept an array of data as their second argument. By default, data will be sent using the `application/json` content type:

```
$response = Http::post('http://test.com/users', [
    'name' => 'Steve',
    'role' => 'Network Administrator',
]);
```

GET Request Query Parameters

When making GET requests, you may either append a query string to the URL directly or pass an array of key / value pairs as the second argument to the get method:

```
$response = Http::get('http://test.com/users', [
    'name' => 'Taylor',
    'page' => 1,
]);
```

Sending Form URL Encoded Requests

If you would like to send data using the application/x-www-form-urlencoded content type, you should call the asForm method before making your request:

```
$response = Http::asForm()->post('http://test.com/users', [
    'name' => 'Sara',
    'role' => 'Privacy Consultant',
]);
```

Sending A Raw Request Body

You may use the withBody method if you would like to provide a raw request body when making a request:

```
$response = Http::withBody(
    base64_encode($photo), 'image/jpeg'
)->post('http://test.com/photo');
```

Multi-Part Requests

If you would like to send files as multi-part requests, you should call the attach method before making your request. This method accepts the name of the file and its contents. Optionally, you may provide a third argument which will be considered the file's filename:

```
$response = Http::attach(
    'attachment', file_get_contents('photo.jpg'), 'photo.jpg'
)->post('http://test.com/attachments');
```

Instead of passing the raw contents of a file, you may also pass a stream resource:

```
$photo = fopen('photo.jpg', 'r');

$response = Http::attach(
    'attachment', $photo, 'photo.jpg'
)->post('http://test.com/attachments');
```

Headers

Headers may be added to requests using the withHeaders method. This withHeaders method accepts an array of key / value pairs:

```
$response = Http::withHeaders([
    'X-First' => 'foo',
    'X-Second' => 'bar'
])->post('http://test.com/users', [
    'name' => 'Taylor',
]);
```

Authentication

You may specify basic and digest authentication credentials using the withBasicAuth and withDigestAuth methods, respectively:

```
// Basic authentication...
$response = Http::withBasicAuth('taylor@laravel.com', 'secret')->post(...);

// Digest authentication...
$response = Http::withDigestAuth('taylor@laravel.com', 'secret')->post(...);
```

Bearer Tokens

If you would like to quickly add an Authorization bearer token header to the request, you may use the `withToken` method:

```
$response = Http::withToken('token')->post(...);
```

Timeout

The `timeout` method may be used to specify the maximum number of seconds to wait for a response:

```
$response = Http::timeout(3)->get(...);
```

If the given timeout is exceeded, an instance of `Illuminate\Http\Client\ConnectionException` will be thrown.

Retries

If you would like HTTP client to automatically retry the request if a client or server error occurs, you may use the `retry` method. The `retry` method accepts two arguments: the number of times the request should be attempted and the number of milliseconds that Laravel should wait in between attempts:

```
$response = Http::retry(3, 100)->post(...);
```

If all of the requests fail, an instance of `Illuminate\Http\Client\RequestException` will be thrown.

Error Handling

Unlike Guzzle's default behavior, Laravel's HTTP client wrapper does not throw exceptions on client or server errors (400 and 500 level responses from servers). You may determine if one of these errors was returned using the `successful`, `clientError`, or `serverError` methods:

```
// Determine if the status code was >= 200 and < 300...
$response->successful();

// Determine if the status code was >= 400...
$response->failed();

// Determine if the response has a 400 level status code...
$response->clientError();

// Determine if the response has a 500 level status code...
$response->serverError();
```

Throwing Exceptions

If you have a response instance and would like to throw an instance of `Illuminate\Http\Client\RequestException` if the response is a client or server error, you may use the `throw` method:

```
$response = Http::post(...);

// Throw an exception if a client or server error occurred...
$response->throw();

return $response['user']['id'];
```

The `Illuminate\Http\Client\RequestException` instance has a public `$response` property which will allow you to inspect the returned response.

The `throw` method returns the response instance if no error occurred, allowing you to chain other operations

onto the `throw` method:

```
return Http::post(...)->throw()->json();
```

Guzzle Options

You may specify additional [Guzzle request options](#) using the `withOptions` method. The `withOptions` method accepts an array of key / value pairs:

```
$response = Http::withOptions([
    'debug' => true,
])->get('http://test.com/users');
```

Testing

Many Laravel services provide functionality to help you easily and expressively write tests, and Laravel's HTTP wrapper is no exception. The `Http` facade's `fake` method allows you to instruct the HTTP client to return stubbed / dummy responses when requests are made.

Faking Responses

For example, to instruct the HTTP client to return empty, 200 status code responses for every request, you may call the `fake` method with no arguments:

```
use Illuminate\Support\Facades\Http;

Http::fake();

$response = Http::post(...);
```

Faking Specific URLs

Alternatively, you may pass an array to the `fake` method. The array's keys should represent URL patterns that you wish to fake and their associated responses. The `*` character may be used as a wildcard character. Any requests made to URLs that have not been faked will actually be executed. You may use the `response` method to construct stub / fake responses for these endpoints:

```
Http::fake([
    // Stub a JSON response for GitHub endpoints...
    'github.com/*' => Http::response(['foo' => 'bar'], 200, ['Headers']),

    // Stub a string response for Google endpoints...
    'google.com/*' => Http::response('Hello World', 200, ['Headers']),
]);
```

If you would like to specify a fallback URL pattern that will stub all unmatched URLs, you may use a single `*` character:

```
Http::fake([
    // Stub a JSON response for GitHub endpoints...
    'github.com/*' => Http::response(['foo' => 'bar'], 200, ['Headers']),

    // Stub a string response for all other endpoints...
    '*' => Http::response('Hello World', 200, ['Headers']),
]);
```

Faking Response Sequences

Sometimes you may need to specify that a single URL should return a series of fake responses in a specific order. You may accomplish this using the `Http::sequence` method to build the responses:

```
Http::fake([
    // Stub a series of responses for GitHub endpoints...
    'github.com/*' => Http::sequence()
        ->push('Hello World', 200)
        ->push(['foo' => 'bar'], 200)
        ->pushStatus(404),
]);
```

```
]);
```

When all of the responses in a response sequence have been consumed, any further requests will cause the response sequence to throw an exception. If you would like to specify a default response that should be returned when a sequence is empty, you may use the `whenEmpty` method:

```
Http::fake([
    // Stub a series of responses for GitHub endpoints...
    'github.com/*' => Http::sequence()
        ->push('Hello World', 200)
        ->push(['foo' => 'bar'], 200)
        ->whenEmpty(Http::response()),
]);
```

If you would like to fake a sequence of responses but do not need to specify a specific URL pattern that should be faked, you may use the `Http::fakeSequence` method:

```
Http::fakeSequence()
    ->push('Hello World', 200)
    ->whenEmpty(Http::response());
```

Fake Callback

If you require more complicated logic to determine what responses to return for certain endpoints, you may pass a callback to the `fake` method. This callback will receive an instance of `Illuminate\Http\Client\Request` and should return a response instance:

```
Http::fake(function ($request) {
    return Http::response('Hello World', 200);
});
```

Inspecting Requests

When faking responses, you may occasionally wish to inspect the requests the client receives in order to make sure your application is sending the correct data or headers. You may accomplish this by calling the `Http::assertSent` method after calling `Http::fake`.

The `assertSent` method accepts a callback which will be given an `Illuminate\Http\Client\Request` instance and should return a boolean value indicating if the request matches your expectations. In order for the test to pass, at least one request must have been issued matching the given expectations:

```
Http::fake();

Http::withHeaders([
    'X-First' => 'foo',
])->post('http://test.com/users', [
    'name' => 'Taylor',
    'role' => 'Developer',
]);

Http::assertSent(function ($request) {
    return $request->hasHeader('X-First', 'foo') &&
        $request->url() == 'http://test.com/users' &&
        $request['name'] == 'Taylor' &&
        $request['role'] == 'Developer';
});
```

If needed, you may assert that a specific request was not sent using the `assertNotSent` method:

```
Http::fake();

Http::post('http://test.com/users', [
    'name' => 'Taylor',
    'role' => 'Developer',
]);

Http::assertNotSent(function (Request $request) {
    return $request->url() === 'http://test.com/posts';
});
```

Or, if you would like to assert that no requests were sent, you may use the `assertNothingSent` method:

```
Http::fake();
```

```
Http::assertNothingSent();
```

Digging Deeper

Mail

Mail

- [Introduction](#)
 - [Configuration](#)
 - [Driver Prerequisites](#)
- [Generating Mailables](#)
- [Writing Mailables](#)
 - [Configuring The Sender](#)
 - [Configuring The View](#)
 - [View Data](#)
 - [Attachments](#)
 - [Inline Attachments](#)
 - [Customizing The SwiftMailer Message](#)
- [Markdown Mailables](#)
 - [Generating Markdown Mailables](#)
 - [Writing Markdown Messages](#)
 - [Customizing The Components](#)
- [Sending Mail](#)
 - [Queueing Mail](#)
- [Rendering Mailables](#)
 - [Previewing Mailables In The Browser](#)
- [Localizing Mailables](#)
- [Mail & Local Development](#)
- [Events](#)

Introduction

Laravel provides a clean, simple API over the popular [SwiftMailer](#) library with drivers for SMTP, Mailgun, Postmark, Amazon SES, and `sendmail`, allowing you to quickly get started sending mail through a local or cloud based service of your choice.

Configuration

Laravel's email services may be configured via the `mail` configuration file. Each mailer configured within this file may have its own options and even its own unique "transport", allowing your application to use different email services to send certain email messages. For example, your application might use Postmark to send transactional mail while using Amazon SES to send bulk mail.

Driver Prerequisites

The API based drivers such as Mailgun and Postmark are often simpler and faster than SMTP servers. If possible, you should use one of these drivers. All of the API drivers require the Guzzle HTTP library, which may be installed via the Composer package manager:

```
composer require guzzlehttp/guzzle
```

Mailgun Driver

To use the Mailgun driver, first install Guzzle, then set the `default` option in your `config/mail.php` configuration file to `mailgun`. Next, verify that your `config/services.php` configuration file contains the following options:

```
'mailgun' => [  
    'domain' => 'your-mailgun-domain',  
    'secret' => 'your-mailgun-key',  
],
```


If you are not using the "US" [Mailgun region](#), you may define your region's endpoint in the services configuration file:

```
'mailgun' => [
    'domain' => 'your-mailgun-domain',
    'secret' => 'your-mailgun-key',
    'endpoint' => 'api.eu.mailgun.net',
],
```

Postmark Driver

To use the Postmark driver, install Postmark's SwiftMailer transport via Composer:

```
composer require wilddbit/swiftmailer-postmark
```

Next, install Guzzle and set the default option in your `config/mail.php` configuration file to `postmark`. Finally, verify that your `config/services.php` configuration file contains the following options:

```
'postmark' => [
    'token' => 'your-postmark-token',
],
```

SES Driver

To use the Amazon SES driver you must first install the Amazon AWS SDK for PHP. You may install this library by adding the following line to your `composer.json` file's `require` section and running the `composer update` command:

```
"aws/aws-sdk-php": "~3.0"
```

Next, set the default option in your `config/mail.php` configuration file to `ses` and verify that your `config/services.php` configuration file contains the following options:

```
'ses' => [
    'key' => 'your-ses-key',
    'secret' => 'your-ses-secret',
    'region' => 'ses-region', // e.g. us-east-1
],
```

If you need to include [additional options](#) when executing the SES `SendRawEmail` request, you may define an options array within your `ses` configuration:

```
'ses' => [
    'key' => 'your-ses-key',
    'secret' => 'your-ses-secret',
    'region' => 'ses-region', // e.g. us-east-1
    'options' => [
        'ConfigurationSetName' => 'MyConfigurationSet',
        'Tags' => [
            [
                'Name' => 'foo',
                'Value' => 'bar',
            ],
        ],
    ],
],
```

Generating Mailables

In Laravel, each type of email sent by your application is represented as a "mailable" class. These classes are stored in the `app/Mail` directory. Don't worry if you don't see this directory in your application, since it will be generated for you when you create your first mailable class using the `make:mail` command:

```
php artisan make:mail OrderShipped
```

Writing Mailables

All of a mailable class' configuration is done in the `build` method. Within this method, you may call various

methods such as `from`, `subject`, `view`, and `attach` to configure the email's presentation and delivery.

Configuring The Sender

Using The `from` Method

First, let's explore configuring the sender of the email. Or, in other words, who the email is going to be "from". There are two ways to configure the sender. First, you may use the `from` method within your mailable class' `build` method:

```
/**
 * Build the message.
 *
 * @return $this
 */
public function build()
{
    return $this->from('example@example.com')
        ->view('emails.orders.shipped');
}
```

Using A Global `from` Address

However, if your application uses the same "from" address for all of its emails, it can become cumbersome to call the `from` method in each mailable class you generate. Instead, you may specify a global "from" address in your `config/mail.php` configuration file. This address will be used if no other "from" address is specified within the mailable class:

```
'from' => ['address' => 'example@example.com', 'name' => 'App Name'],
```

In addition, you may define a global "reply_to" address within your `config/mail.php` configuration file:

```
'reply_to' => ['address' => 'example@example.com', 'name' => 'App Name'],
```

Configuring The View

Within a mailable class' `build` method, you may use the `view` method to specify which template should be used when rendering the email's contents. Since each email typically uses a [Blade template](#) to render its contents, you have the full power and convenience of the Blade templating engine when building your email's HTML:

```
/**
 * Build the message.
 *
 * @return $this
 */
public function build()
{
    return $this->view('emails.orders.shipped');
}
```

TIP You may wish to create a `resources/views/emails` directory to house all of your email templates; however, you are free to place them wherever you wish within your `resources/views` directory.

Plain Text Emails

If you would like to define a plain-text version of your email, you may use the `text` method. Like the `view` method, the `text` method accepts a template name which will be used to render the contents of the email. You are free to define both an HTML and plain-text version of your message:

```
/**
 * Build the message.
 *
 * @return $this
 */
public function build()
{
    return $this->view('emails.orders.shipped')
```

```
        ->text('emails.orders.shipped_plain');
    }
}
```

View Data

Via Public Properties

Typically, you will want to pass some data to your view that you can utilize when rendering the email's HTML. There are two ways you may make data available to your view. First, any public property defined on your mailable class will automatically be made available to the view. So, for example, you may pass data into your mailable class' constructor and set that data to public properties defined on the class:

```
<?php

namespace App\Mail;

use App\Order;
use Illuminate\Bus\Queueable;
use Illuminate\Mail\Mailable;
use Illuminate\Queue\SerializesModels;

class OrderShipped extends Mailable
{
    use Queueable, SerializesModels;

    /**
     * The order instance.
     *
     * @var Order
     */
    public $order;

    /**
     * Create a new message instance.
     *
     * @param \App\Order $order
     * @return void
     */
    public function __construct(Order $order)
    {
        $this->order = $order;
    }

    /**
     * Build the message.
     *
     * @return $this
     */
    public function build()
    {
        return $this->view('emails.orders.shipped');
    }
}
```

Once the data has been set to a public property, it will automatically be available in your view, so you may access it like you would access any other data in your Blade templates:

```
<div>
    Price: {{ $order->price }}
</div>
```

Via The with Method:

If you would like to customize the format of your email's data before it is sent to the template, you may manually pass your data to the view via the `with` method. Typically, you will still pass data via the mailable class' constructor; however, you should set this data to `protected` or `private` properties so the data is not automatically made available to the template. Then, when calling the `with` method, pass an array of data that you wish to make available to the template:

```
<?php

namespace App\Mail;
```

```
use App\Order;
use Illuminate\Bus\Queueable;
use Illuminate\Mail\Mailable;
use Illuminate\Queue\SerializesModels;

class OrderShipped extends Mailable
{
    use Queueable, SerializesModels;

    /**
     * The order instance.
     *
     * @var Order
     */
    protected $order;

    /**
     * Create a new message instance.
     *
     * @param  \App\Order $order
     * @return void
     */
    public function __construct(Order $order)
    {
        $this->order = $order;
    }

    /**
     * Build the message.
     *
     * @return $this
     */
    public function build()
    {
        return $this->view('emails.orders.shipped')
            ->with([
                'orderName' => $this->order->name,
                'orderPrice' => $this->order->price,
            ]);
    }
}
```

Once the data has been passed to the with method, it will automatically be available in your view, so you may access it like you would access any other data in your Blade templates:

```
<div>
    Price: {{ $orderPrice }}
</div>
```

Attachments

To add attachments to an email, use the attach method within the mailable class' build method. The attach method accepts the full path to the file as its first argument:

```
/**
 * Build the message.
 *
 * @return $this
 */
public function build()
{
    return $this->view('emails.orders.shipped')
        ->attach('/path/to/file');
}
```

When attaching files to a message, you may also specify the display name and / or MIME type by passing an array as the second argument to the attach method:

```
/**
 * Build the message.
 *
 * @return $this
 */
public function build()
{
    return $this->view('emails.orders.shipped')
```

```
        ->attach('/path/to/file', [
            'as' => 'name.pdf',
            'mime' => 'application/pdf',
        ]);
    }
}
```

Attaching Files from Disk

If you have stored a file on one of your [filesystem disks](#), you may attach it to the email using the `attachFromStorage` method:

```
/**
 * Build the message.
 *
 * @return $this
 */
public function build()
{
    return $this->view('emails.orders.shipped')
        ->attachFromStorage('/path/to/file');
}
```

If necessary, you may specify the file's attachment name and additional options using the second and third arguments to the `attachFromStorage` method:

```
/**
 * Build the message.
 *
 * @return $this
 */
public function build()
{
    return $this->view('emails.orders.shipped')
        ->attachFromStorage('/path/to/file', 'name.pdf', [
            'mime' => 'application/pdf'
        ]);
}
```

The `attachFromStorageDisk` method may be used if you need to specify a storage disk other than your default disk:

```
/**
 * Build the message.
 *
 * @return $this
 */
public function build()
{
    return $this->view('emails.orders.shipped')
        ->attachFromStorageDisk('s3', '/path/to/file');
}
```

Raw Data Attachments

The `attachData` method may be used to attach a raw string of bytes as an attachment. For example, you might use this method if you have generated a PDF in memory and want to attach it to the email without writing it to disk. The `attachData` method accepts the raw data bytes as its first argument, the name of the file as its second argument, and an array of options as its third argument:

```
/**
 * Build the message.
 *
 * @return $this
 */
public function build()
{
    return $this->view('emails.orders.shipped')
        ->attachData($this->pdf, 'name.pdf', [
            'mime' => 'application/pdf',
        ]);
}
```

Inline Attachments

Embedding inline images into your emails is typically cumbersome; however, Laravel provides a convenient way to attach images to your emails and retrieving the appropriate CID. To embed an inline image, use the `embed` method on the `$message` variable within your email template. Laravel automatically makes the `$message` variable available to all of your email templates, so you don't need to worry about passing it in manually:

```
<body>
    Here is an image:

    
</body>
```

NOTE `$message` variable is not available in plain-text messages since plain-text messages do not utilize inline attachments.

Embedding Raw Data Attachments

If you already have a raw data string you wish to embed into an email template, you may use the `embedData` method on the `$message` variable:

```
<body>
    Here is an image from raw data:

    
</body>
```

Customizing The SwiftMailer Message

The `withSwiftMessage` method of the `Mailable` base class allows you to register a callback which will be invoked with the raw SwiftMailer message instance before sending the message. This gives you an opportunity to customize the message before it is delivered:

```
/**
 * Build the message.
 *
 * @return $this
 */
public function build()
{
    $this->view('emails.orders.shipped');

    $this->withSwiftMessage(function ($message) {
        $message->getHeaders()
            ->addTextHeader('Custom-Header', 'HeaderValue');
    });
}
```

Markdown Mailables

Markdown mailable messages allow you to take advantage of the pre-built templates and components of mail notifications in your mailables. Since the messages are written in Markdown, Laravel is able to render beautiful, responsive HTML templates for the messages while also automatically generating a plain-text counterpart.

Generating Markdown Mailables

To generate a mailable with a corresponding Markdown template, you may use the `--markdown` option of the `make:mail` Artisan command:

```
php artisan make:mail OrderShipped --markdown=emails.orders.shipped
```

Then, when configuring the mailable within its `build` method, call the `markdown` method instead of the `view` method. The `markdown` method accepts the name of the Markdown template and an optional array of data to make available to the template:

```
/**
 * Build the message.
 *
```

```
* @return $this
*/
public function build()
{
    return $this->from('example@example.com')
        ->markdown('emails.orders.shipped');
}
```

Writing Markdown Messages

Markdown mailables use a combination of Blade components and Markdown syntax which allow you to easily construct mail messages while leveraging Laravel's pre-crafted components:

```
@component('mail::message')
# Order Shipped

Your order has been shipped!

@component('mail::button', ['url' => $url])
View Order
@endcomponent

Thanks,<br>
{{ config('app.name') }}
```

TIP Do not use excess indentation when writing Markdown emails. Markdown parsers will render indented content as code blocks.

Button Component

The button component renders a centered button link. The component accepts two arguments, a `url` and an optional `color`. Supported colors are `primary`, `success`, and `error`. You may add as many button components to a message as you wish:

```
@component('mail::button', ['url' => $url, 'color' => 'success'])
View Order
@endcomponent
```

Panel Component

The panel component renders the given block of text in a panel that has a slightly different background color than the rest of the message. This allows you to draw attention to a given block of text:

```
@component('mail::panel')
This is the panel content.
@endcomponent
```

Table Component

The table component allows you to transform a Markdown table into an HTML table. The component accepts the Markdown table as its content. Table column alignment is supported using the default Markdown table alignment syntax:

```
@component('mail::table')
| Laravel | Table | Example |
| :-----: | :-----: | :-----: |
| Col 2 is | Centered | $10 |
| Col 3 is | Right-Aligned | $20 |
@endcomponent
```

Customizing The Components

You may export all of the Markdown mail components to your own application for customization. To export the components, use the `vendor:publish` Artisan command to publish the `laravel-mail` asset tag:

```
php artisan vendor:publish --tag=laravel-mail
```

This command will publish the Markdown mail components to the `resources/views/vendor/mail` directory. The `mail` directory will contain an `html` and a `text` directory, each containing their respective representations of every available component. You are free to customize these components however you like.

Customizing The CSS

After exporting the components, the `resources/views/vendor/mail/html/themes` directory will contain a `default.css` file. You may customize the CSS in this file and your styles will automatically be in-lined within the HTML representations of your Markdown mail messages.

If you would like to build an entirely new theme for Laravel's Markdown components, you may place a CSS file within the `html/themes` directory. After naming and saving your CSS file, update the `theme` option of the `mail` configuration file to match the name of your new theme.

To customize the theme for an individual mailable, you may set the `$theme` property of the mailable class to the name of the theme that should be used when sending that mailable.

Sending Mail

To send a message, use the `to` method on the `Mail` [facade](#). The `to` method accepts an email address, a user instance, or a collection of users. If you pass an object or collection of objects, the mailer will automatically use their `email` and `name` properties when setting the email recipients, so make sure these attributes are available on your objects. Once you have specified your recipients, you may pass an instance of your mailable class to the `send` method:

```
<?php

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;
use App\Mail\OrderShipped;
use App\Order;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Mail;

class OrderController extends Controller
{
    /**
     * Ship the given order.
     *
     * @param Request $request
     * @param int $orderId
     * @return Response
     */
    public function ship(Request $request, $orderId)
    {
        $order = Order::findOrFail($orderId);

        // Ship order...

        Mail::to($request->user())->send(new OrderShipped($order));
    }
}
```

You are not limited to just specifying the "to" recipients when sending a message. You are free to set "to", "cc", and "bcc" recipients all within a single, chained method call:

```
use Illuminate\Support\Facades\Mail;

Mail::to($request->user())
    ->cc($moreUsers)
    ->bcc($sevenMoreUsers)
    ->send(new OrderShipped($order));
```

Looping Over Recipients

Occasionally, you may need to send a mailable to a list of recipients by iterating over an array of recipients / email addresses. Since the `to` method appends email addresses to the mailable's list of recipients, you should

always re-create the mailable instance for each recipient:

```
foreach (['taylor@example.com', 'dries@example.com'] as $recipient) {  
    Mail::to($recipient)->send(new OrderShipped($order));  
}
```

Sending Mail Via A Specific Mailer

By default, Laravel will use the mailer configured as the default mailer in your mail configuration file. However, you may use the `mailer` method to send a message using a specific mailer configuration:

```
Mail::mailer('postmark')  
    ->to($request->user())  
    ->send(new OrderShipped($order));
```

Queueing Mail

Queueing A Mail Message

Since sending email messages can drastically lengthen the response time of your application, many developers choose to queue email messages for background sending. Laravel makes this easy using its built-in [unified queue API](#). To queue a mail message, use the `queue` method on the `Mail` facade after specifying the message's recipients:

```
Mail::to($request->user())  
    ->cc($moreUsers)  
    ->bcc($sevenMoreUsers)  
    ->queue(new OrderShipped($order));
```

This method will automatically take care of pushing a job onto the queue so the message is sent in the background. You will need to [configure your queues](#) before using this feature.

Delayed Message Queueing

If you wish to delay the delivery of a queued email message, you may use the `later` method. As its first argument, the `later` method accepts a `DateTime` instance indicating when the message should be sent:

```
$when = now()->addMinutes(10);  
  
Mail::to($request->user())  
    ->cc($moreUsers)  
    ->bcc($sevenMoreUsers)  
    ->later($when, new OrderShipped($order));
```

Pushing To Specific Queues

Since all mailable classes generated using the `make:mail` command make use of the `Illuminate\Bus\Queueable` trait, you may call the `onQueue` and `onConnection` methods on any mailable class instance, allowing you to specify the connection and queue name for the message:

```
$message = (new OrderShipped($order))  
    ->onConnection('sqs')  
    ->onQueue('emails');  
  
Mail::to($request->user())  
    ->cc($moreUsers)  
    ->bcc($sevenMoreUsers)  
    ->queue($message);
```

Queueing By Default

If you have mailable classes that you want to always be queued, you may implement the `ShouldQueue` contract on the class. Now, even if you call the `send` method when mailing, the mailable will still be queued since it implements the contract:

```
use Illuminate\Contracts\Queue\ShouldQueue;
```

```
class OrderShipped extends Mailable implements ShouldQueue
{
    //
}
```

Rendering Mailables

Sometimes you may wish to capture the HTML content of a mailable without sending it. To accomplish this, you may call the `render` method of the mailable. This method will return the evaluated contents of the mailable as a string:

```
$invoice = App\Invoice::find(1);

return (new App\Mail\InvoicePaid($invoice))->render();
```

Previewing Mailables In The Browser

When designing a mailable's template, it is convenient to quickly preview the rendered mailable in your browser like a typical Blade template. For this reason, Laravel allows you to return any mailable directly from a route Closure or controller. When a mailable is returned, it will be rendered and displayed in the browser, allowing you to quickly preview its design without needing to send it to an actual email address:

```
Route::get('mailable', function () {
    $invoice = App\Invoice::find(1);

    return new App\Mail\InvoicePaid($invoice);
});
```

Localizing Mailables

Laravel allows you to send mailables in a locale other than the current language, and will even remember this locale if the mail is queued.

To accomplish this, the `Mail` facade offers a `locale` method to set the desired language. The application will change into this locale when the mailable is being formatted and then revert back to the previous locale when formatting is complete:

```
Mail::to($request->user())->locale('es')->send(
    new OrderShipped($order)
);
```

User Preferred Locales

Sometimes, applications store each user's preferred locale. By implementing the `HasLocalePreference` contract on one or more of your models, you may instruct Laravel to use this stored locale when sending mail:

```
use Illuminate\Contracts\Translation\HasLocalePreference;

class User extends Model implements HasLocalePreference
{
    /**
     * Get the user's preferred locale.
     *
     * @return string
     */
    public function preferredLocale()
    {
        return $this->locale;
    }
}
```

Once you have implemented the interface, Laravel will automatically use the preferred locale when sending mailables and notifications to the model. Therefore, there is no need to call the `locale` method when using this interface:

```
Mail::to($request->user())->send(new OrderShipped($order));
```

Mail & Local Development

When developing an application that sends email, you probably don't want to actually send emails to live email addresses. Laravel provides several ways to "disable" the actual sending of emails during local development.

Log Driver

Instead of sending your emails, the log mail driver will write all email messages to your log files for inspection. For more information on configuring your application per environment, check out the [configuration documentation](#).

Universal To

Another solution provided by Laravel is to set a universal recipient of all emails sent by the framework. This way, all the emails generated by your application will be sent to a specific address, instead of the address actually specified when sending the message. This can be done via the `to` option in your `config/mail.php` configuration file:

```
'to' => [
    'address' => 'example@example.com',
    'name' => 'Example'
],
```

Mailtrap

Finally, you may use a service like [Mailtrap](#) and the `smtp` driver to send your email messages to a "dummy" mailbox where you may view them in a true email client. This approach has the benefit of allowing you to actually inspect the final emails in Mailtrap's message viewer.

Events

Laravel fires two events during the process of sending mail messages. The `MessageSending` event is fired prior to a message being sent, while the `MessageSent` event is fired after a message has been sent. Remember, these events are fired when the mail is being *sent*, not when it is queued. You may register an event listener for this event in your `EventServiceProvider`:

```
/**
 * The event listener mappings for the application.
 *
 * @var array
 */
protected $listen = [
    'Illuminate\Mail\Events\MessageSending' => [
        'App\Listeners\LogSendingMessage',
    ],
    'Illuminate\Mail\Events\MessageSent' => [
        'App\Listeners\LogSentMessage',
    ],
];
```

Digging Deeper

Notifications

Notifications

- [Introduction](#)
- [Creating Notifications](#)
- [Sending Notifications](#)
 - [Using The Notifiable Trait](#)
 - [Using The Notification Facade](#)
 - [Specifying Delivery Channels](#)
 - [Queueing Notifications](#)
 - [On-Demand Notifications](#)
- [Mail Notifications](#)
 - [Formatting Mail Messages](#)
 - [Customizing The Sender](#)
 - [Customizing The Recipient](#)
 - [Customizing The Subject](#)
 - [Customizing The Mailer](#)
 - [Customizing The Templates](#)
 - [Previewing Mail Notifications](#)
- [Markdown Mail Notifications](#)
 - [Generating The Message](#)
 - [Writing The Message](#)
 - [Customizing The Components](#)
- [Database Notifications](#)
 - [Prerequisites](#)
 - [Formatting Database Notifications](#)
 - [Accessing The Notifications](#)
 - [Marking Notifications As Read](#)
- [Broadcast Notifications](#)
 - [Prerequisites](#)
 - [Formatting Broadcast Notifications](#)
 - [Listening For Notifications](#)
- [SMS Notifications](#)
 - [Prerequisites](#)
 - [Formatting SMS Notifications](#)
 - [Formatting Shortcode Notifications](#)
 - [Customizing The "From" Number](#)
 - [Routing SMS Notifications](#)
- [Slack Notifications](#)
 - [Prerequisites](#)
 - [Formatting Slack Notifications](#)
 - [Slack Attachments](#)
 - [Routing Slack Notifications](#)
- [Localizing Notifications](#)
- [Notification Events](#)
- [Custom Channels](#)

Introduction

In addition to support for [sending email](#), Laravel provides support for sending notifications across a variety of delivery channels, including mail, SMS (via [Vonage](#), formerly known as Nexmo), and [Slack](#). Notifications may also be stored in a database so they may be displayed in your web interface.

Typically, notifications should be short, informational messages that notify users of something that occurred in your application. For example, if you are writing a billing application, you might send an "Invoice Paid" notification to your users via the email and SMS channels.

Creating Notifications

In Laravel, each notification is represented by a single class (typically stored in the `app/Notifications` directory). Don't worry if you don't see this directory in your application, it will be created for you when you run the `make:notification` Artisan command:

```
php artisan make:notification InvoicePaid
```

This command will place a fresh notification class in your `app/Notifications` directory. Each notification class contains a `via` method and a variable number of message building methods (such as `toMail` or `toDatabase`) that convert the notification to a message optimized for that particular channel.

Sending Notifications

Using The Notifiable Trait

Notifications may be sent in two ways: using the `notify` method of the `Notifiable` trait or using the `Notification` [facade](#). First, let's explore using the trait:

```
<?php

namespace App;

use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;

class User extends Authenticatable
{
    use Notifiable;
}
```

This trait is utilized by the default `App\User` model and contains one method that may be used to send notifications: `notify`. The `notify` method expects to receive a notification instance:

```
use App\Notifications\InvoicePaid;

$user->notify(new InvoicePaid($invoice));
```

TIP Remember, you may use the `Illuminate\Notifications\Notifiable` trait on any of your models. You are not limited to only including it on your `User` model.

Using The Notification Facade

Alternatively, you may send notifications via the `Notification` [facade](#). This is useful primarily when you need to send a notification to multiple notifiable entities such as a collection of users. To send notifications using the facade, pass all of the notifiable entities and the notification instance to the `send` method:

```
Notification::send($users, new InvoicePaid($invoice));
```

Specifying Delivery Channels

Every notification class has a `via` method that determines on which channels the notification will be delivered. Notifications may be sent on the `mail`, `database`, `broadcast`, `nexmo`, and `slack` channels.

TIP If you would like to use other delivery channels such as Telegram or Pusher, check out the community driven [Laravel Notification Channels website](#).

The `via` method receives a `$notifiable` instance, which will be an instance of the class to which the notification is being sent. You may use `$notifiable` to determine which channels the notification should be delivered on:

```
/**
 * Get the notification's delivery channels.
 *
 * @param mixed $notifiable
 * @return array
```

```
*/
public function via($notifiable)
{
    return $notifiable->prefers_sms ? ['nexmo'] : ['mail', 'database'];
}
```

Queueing Notifications

NOTE Before queueing notifications you should configure your queue and [start a worker](#).

Sending notifications can take time, especially if the channel needs an external API call to deliver the notification. To speed up your application's response time, let your notification be queued by adding the `ShouldQueue` interface and `Queueable` trait to your class. The interface and trait are already imported for all notifications generated using `make:notification`, so you may immediately add them to your notification class:

```
<?php

namespace App\Notifications;

use Illuminate\Bus\Queueable;
use Illuminate\Contracts\Queue\ShouldQueue;
use Illuminate\Notifications\Notification;

class InvoicePaid extends Notification implements ShouldQueue
{
    use Queueable;

    // ...
}
```

Once the `ShouldQueue` interface has been added to your notification, you may send the notification like normal. Laravel will detect the `ShouldQueue` interface on the class and automatically queue the delivery of the notification:

```
$user->notify(new InvoicePaid($invoice));
```

If you would like to delay the delivery of the notification, you may chain the `delay` method onto your notification instantiation:

```
$when = now()->addMinutes(10);

$user->notify((new InvoicePaid($invoice))->delay($when));
```

Customizing Notification Channel Queues

If you would like to specify a specific queue that should be used for each notification channel supported by the notification, you may define a `viaQueues` method on your notification. This method should return an array of channel name / queue name pairs:

```
/**
 * Determine which queues should be used for each notification channel.
 *
 * @return array
 */
public function viaQueues()
{
    return [
        'mail' => 'mail-queue',
        'slack' => 'slack-queue',
    ];
}
```

On-Demand Notifications

Sometimes you may need to send a notification to someone who is not stored as a "user" of your application. Using the `Notification::route` facade method, you may specify ad-hoc notification routing information before sending the notification:

```
Notification::route('mail', 'taylor@example.com')
    ->route('nexmo', '5555555555')
```

```
->route('slack', 'https://hooks.slack.com/services/...')
->notify(new InvoicePaid($invoice));
```

Mail Notifications

Formatting Mail Messages

If a notification supports being sent as an email, you should define a `toMail` method on the notification class. This method will receive a `$notifiable` entity and should return a `\Illuminate\Notifications\Messages\MailMessage` instance. Mail messages may contain lines of text as well as a "call to action". Let's take a look at an example `toMail` method:

```
/**
 * Get the mail representation of the notification.
 *
 * @param mixed $notifiable
 * @return \Illuminate\Notifications\Messages\MailMessage
 */
public function toMail($notifiable)
{
    $url = url('/invoice/'. $this->invoice->id);

    return (new MailMessage)
        ->greeting('Hello!')
        ->line('One of your invoices has been paid!')
        ->action('View Invoice', $url)
        ->line('Thank you for using our application!');
}
```

TIP Note we are using `$this->invoice->id` in our `toMail` method. You may pass any data your notification needs to generate its message into the notification's constructor.

In this example, we register a greeting, a line of text, a call to action, and then another line of text. These methods provided by the `MailMessage` object make it simple and fast to format small transactional emails. The mail channel will then translate the message components into a nice, responsive HTML email template with a plain-text counterpart. Here is an example of an email generated by the `mail` channel:

Laravel

Hello!

One of your invoices has been paid!

[View Invoice](#)

Thank you for using our application!

Regards,
Laravel

If you're having trouble clicking the "View Invoice" button, copy and paste the URL below into your web browser:

<https://example.com/invoice/1>

© 2016 [Laravel](#). All rights reserved.

TIP When sending mail notifications, be sure to set the `name` value in your `config/app.php` configuration file. This value will be used in the header and footer of your mail notification messages.

Other Notification Formatting Options

Instead of defining the "lines" of text in the notification class, you may use the `view` method to specify a custom template that should be used to render the notification email:

```
/**
 * Get the mail representation of the notification.
 *
 * @param mixed $notifiable
 * @return \Illuminate\Notifications\Messages\MailMessage
 */
public function toMail($notifiable)
{
    return (new MailMessage)->view(
        'emails.name', ['invoice' => $this->invoice]
    );
}
```

You may specify a plain-text view for the mail message by passing the view name as the second element of an array that is given to the `view` method of the `MailMessage`:

```
/**
 * Get the mail representation of the notification.
```



```
*
* @param mixed $notifiable
* @return \Illuminate\Notifications\Messages\MailMessage
*/
public function toMail($notifiable)
{
    return (new MailMessage)->view(
        ['emails.name.html', 'emails.name.plain'],
        ['invoice' => $this->invoice]
    );
}
```

In addition, you may return a full [mailable object](#) from the `toMail` method:

```
use App\Mail\InvoicePaid as Mailable;
```

```
/**
 * Get the mail representation of the notification.
 *
 * @param mixed $notifiable
 * @return Mailable
 */
public function toMail($notifiable)
{
    return (new Mailable($this->invoice))->to($notifiable->email);
}
```

Error Messages

Some notifications inform users of errors, such as a failed invoice payment. You may indicate that a mail message is regarding an error by calling the `error` method when building your message. When using the `error` method on a mail message, the call to action button will be red instead of blue:

```
/**
 * Get the mail representation of the notification.
 *
 * @param mixed $notifiable
 * @return \Illuminate\Notifications\Message
 */
public function toMail($notifiable)
{
    return (new MailMessage)
        ->error()
        ->subject('Notification Subject')
        ->line('...');
}
```

Customizing The Sender

By default, the email's sender / from address is defined in the `config/mail.php` configuration file. However, you may specify the from address for a specific notification using the `from` method:

```
/**
 * Get the mail representation of the notification.
 *
 * @param mixed $notifiable
 * @return \Illuminate\Notifications\Messages\MailMessage
 */
public function toMail($notifiable)
{
    return (new MailMessage)
        ->from('test@example.com', 'Example')
        ->line('...');
}
```

Customizing The Recipient

When sending notifications via the `mail` channel, the notification system will automatically look for an `email` property on your notifiable entity. You may customize which email address is used to deliver the notification by defining a `routeNotificationForMail` method on the entity:

```
<?php
```

```
namespace App;

use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;

class User extends Authenticatable
{
    use Notifiable;

    /**
     * Route notifications for the mail channel.
     *
     * @param \Illuminate\Notifications\Notification $notification
     * @return array|string
     */
    public function routeNotificationForMail($notification)
    {
        // Return email address only...
        return $this->email_address;

        // Return name and email address...
        return [$this->email_address => $this->name];
    }
}
```

Customizing The Subject

By default, the email's subject is the class name of the notification formatted to "title case". So, if your notification class is named `InvoicePaid`, the email's subject will be `Invoice Paid`. If you would like to specify an explicit subject for the message, you may call the `subject` method when building your message:

```
/**
 * Get the mail representation of the notification.
 *
 * @param mixed $notifiable
 * @return \Illuminate\Notifications\Messages\MailMessage
 */
public function toMail($notifiable)
{
    return (new MailMessage)
        ->subject('Notification Subject')
        ->line('...');
}
```

Customizing The Mailer

By default, the email notification will be sent using the default driver defined in the `config/mail.php` configuration file. However, you may specify a different mailer at runtime by calling the `mailer` method when building your message:

```
/**
 * Get the mail representation of the notification.
 *
 * @param mixed $notifiable
 * @return \Illuminate\Notifications\Messages\MailMessage
 */
public function toMail($notifiable)
{
    return (new MailMessage)
        ->mailer('postmark')
        ->line('...');
}
```

Customizing The Templates

You can modify the HTML and plain-text template used by mail notifications by publishing the notification package's resources. After running this command, the mail notification templates will be located in the `resources/views/vendor/notifications` directory:

```
php artisan vendor:publish --tag=laravel-notifications
```

Previewing Mail Notifications

When designing a mail notification template, it is convenient to quickly preview the rendered mail message in your browser like a typical Blade template. For this reason, Laravel allows you to return any mail message generated by a mail notification directly from a route Closure or controller. When a `MailMessage` is returned, it will be rendered and displayed in the browser, allowing you to quickly preview its design without needing to send it to an actual email address:

```
Route::get('mail', function () {
    $invoice = App\Invoice::find(1);

    return (new App\Notifications\InvoicePaid($invoice))
        ->toMail($invoice->user);
});
```

Markdown Mail Notifications

Markdown mail notifications allow you to take advantage of the pre-built templates of mail notifications, while giving you more freedom to write longer, customized messages. Since the messages are written in Markdown, Laravel is able to render beautiful, responsive HTML templates for the messages while also automatically generating a plain-text counterpart.

Generating The Message

To generate a notification with a corresponding Markdown template, you may use the `--markdown` option of the `make:notification` Artisan command:

```
php artisan make:notification InvoicePaid --markdown=mail.invoice.paid
```

Like all other mail notifications, notifications that use Markdown templates should define a `toMail` method on their notification class. However, instead of using the `line` and `action` methods to construct the notification, use the `markdown` method to specify the name of the Markdown template that should be used:

```
/**
 * Get the mail representation of the notification.
 *
 * @param mixed $notifiable
 * @return \Illuminate\Notifications\Messages\MailMessage
 */
public function toMail($notifiable)
{
    $url = url('/invoice/'. $this->invoice->id);

    return (new MailMessage)
        ->subject('Invoice Paid')
        ->markdown('mail.invoice.paid', ['url' => $url]);
}
```

Writing The Message

Markdown mail notifications use a combination of Blade components and Markdown syntax which allow you to easily construct notifications while leveraging Laravel's pre-crafted notification components:

```
@component('mail::message')
# Invoice Paid

Your invoice has been paid!

@component('mail::button', ['url' => $url])
View Invoice
@endcomponent

Thanks,<br>
{{ config('app.name') }}
@endcomponent
```

Button Component

The button component renders a centered button link. The component accepts two arguments, a `url` and an optional `color`. Supported colors are `blue`, `green`, and `red`. You may add as many button components to a notification as you wish:

```
@component('mail::button', ['url' => $url, 'color' => 'green'])
View Invoice
@endcomponent
```

Panel Component

The panel component renders the given block of text in a panel that has a slightly different background color than the rest of the notification. This allows you to draw attention to a given block of text:

```
@component('mail::panel')
This is the panel content.
@endcomponent
```

Table Component

The table component allows you to transform a Markdown table into an HTML table. The component accepts the Markdown table as its content. Table column alignment is supported using the default Markdown table alignment syntax:

```
@component('mail::table')
| Laravel | Table | Example |
| :-----: | :-----: | :-----: |
| Col 2 is | Centered | $10 |
| Col 3 is | Right-Aligned | $20 |
@endcomponent
```

Customizing The Components

You may export all of the Markdown notification components to your own application for customization. To export the components, use the `vendor:publish` Artisan command to publish the `laravel-mail` asset tag:

```
php artisan vendor:publish --tag=laravel-mail
```

This command will publish the Markdown mail components to the `resources/views/vendor/mail` directory. The `mail` directory will contain an `html` and a `text` directory, each containing their respective representations of every available component. You are free to customize these components however you like.

Customizing The CSS

After exporting the components, the `resources/views/vendor/mail/html/themes` directory will contain a `default.css` file. You may customize the CSS in this file and your styles will automatically be in-lined within the HTML representations of your Markdown notifications.

If you would like to build an entirely new theme for Laravel's Markdown components, you may place a CSS file within the `html/themes` directory. After naming and saving your CSS file, update the `theme` option of the `mail` configuration file to match the name of your new theme.

To customize the theme for an individual notification, you may call the `theme` method while building the notification's mail message. The `theme` method accepts the name of the theme that should be used when sending the notification:

```
/**
 * Get the mail representation of the notification.
 *
 * @param mixed $notifiable
 * @return \Illuminate\Notifications\Messages\MailMessage
 */
public function toMail($notifiable)
{
    return (new MailMessage)
        ->theme('invoice')
        ->subject('Invoice Paid')
        ->markdown('mail.invoice.paid', ['url' => $url]);
}
```

```
}
```

Database Notifications

Prerequisites

The database notification channel stores the notification information in a database table. This table will contain information such as the notification type as well as custom JSON data that describes the notification.

You can query the table to display the notifications in your application's user interface. But, before you can do that, you will need to create a database table to hold your notifications. You may use the `notifications:table` command to generate a migration with the proper table schema:

```
php artisan notifications:table
php artisan migrate
```

Formatting Database Notifications

If a notification supports being stored in a database table, you should define a `toDatabase` or `toArray` method on the notification class. This method will receive a `$notifiable` entity and should return a plain PHP array. The returned array will be encoded as JSON and stored in the `data` column of your `notifications` table. Let's take a look at an example `toArray` method:

```
/**
 * Get the array representation of the notification.
 *
 * @param mixed $notifiable
 * @return array
 */
public function toArray($notifiable)
{
    return [
        'invoice_id' => $this->invoice->id,
        'amount' => $this->invoice->amount,
    ];
}
```

toDatabase Vs. toArray

The `toArray` method is also used by the `broadcast` channel to determine which data to broadcast to your JavaScript client. If you would like to have two different array representations for the database and broadcast channels, you should define a `toDatabase` method instead of a `toArray` method.

Accessing The Notifications

Once notifications are stored in the database, you need a convenient way to access them from your notifiable entities. The `Illuminate\Notifications\Notifiable` trait, which is included on Laravel's default `App\User` model, includes a `notifications` Eloquent relationship that returns the notifications for the entity. To fetch notifications, you may access this method like any other Eloquent relationship. By default, notifications will be sorted by the `created_at` timestamp:

```
$user = App\User::find(1);

foreach ($user->notifications as $notification) {
    echo $notification->type;
}
```

If you want to retrieve only the "unread" notifications, you may use the `unreadNotifications` relationship. Again, these notifications will be sorted by the `created_at` timestamp:

```
$user = App\User::find(1);

foreach ($user->unreadNotifications as $notification) {
    echo $notification->type;
}
```

TIP To access your notifications from your JavaScript client, you should define a notification controller for your application which returns the notifications for a notifiable entity, such as the current user. You may then make an HTTP request to that controller's URI from your JavaScript client.

Marking Notifications As Read

Typically, you will want to mark a notification as "read" when a user views it. The `Illuminate\Notifications\Notifiable` trait provides a `markAsRead` method, which updates the `read_at` column on the notification's database record:

```
$user = App\User::find(1);

foreach ($user->unreadNotifications as $notification) {
    $notification->markAsRead();
}
```

However, instead of looping through each notification, you may use the `markAsRead` method directly on a collection of notifications:

```
$user->unreadNotifications->markAsRead();
```

You may also use a mass-update query to mark all of the notifications as read without retrieving them from the database:

```
$user = App\User::find(1);

$user->unreadNotifications()->update(['read_at' => now()]);
```

You may delete the notifications to remove them from the table entirely:

```
$user->notifications()->delete();
```

Broadcast Notifications

Prerequisites

Before broadcasting notifications, you should configure and be familiar with Laravel's [event broadcasting](#) services. Event broadcasting provides a way to react to server-side fired Laravel events from your JavaScript client.

Formatting Broadcast Notifications

The broadcast channel broadcasts notifications using Laravel's [event broadcasting](#) services, allowing your JavaScript client to catch notifications in realtime. If a notification supports broadcasting, you can define a `toBroadcast` method on the notification class. This method will receive a `$notifiable` entity and should return a `BroadcastMessage` instance. If the `toBroadcast` method does not exist, the `toArray` method will be used to gather the data that should be broadcast. The returned data will be encoded as JSON and broadcast to your JavaScript client. Let's take a look at an example `toBroadcast` method:

```
use Illuminate\Notifications\Messages\BroadcastMessage;

/**
 * Get the broadcastable representation of the notification.
 *
 * @param mixed $notifiable
 * @return BroadcastMessage
 */
public function toBroadcast($notifiable)
{
    return new BroadcastMessage([
        'invoice_id' => $this->invoice->id,
        'amount' => $this->invoice->amount,
    ]);
}
```

Broadcast Queue Configuration

All broadcast notifications are queued for broadcasting. If you would like to configure the queue connection or queue name that is used to queue the broadcast operation, you may use the `onConnection` and `onQueue` methods of the `BroadcastMessage`:

```
return (new BroadcastMessage($data))
    ->onConnection('sqs')
    ->onQueue('broadcasts');
```

Customizing The Notification Type

In addition to the data you specify, all broadcast notifications also have a `type` field containing the full class name of the notification. If you would like to customize the notification type that is provided to your JavaScript client, you may define a `broadcastType` method on the notification class:

```
use Illuminate\Notifications\Messages\BroadcastMessage;

/**
 * Get the type of the notification being broadcast.
 *
 * @return string
 */
public function broadcastType()
{
    return 'broadcast.message';
}
```

Listening For Notifications

Notifications will broadcast on a private channel formatted using a `{notifiable}.{id}` convention. So, if you are sending a notification to a `App\User` instance with an ID of 1, the notification will be broadcast on the `App.User.1` private channel. When using [Laravel Echo](#), you may easily listen for notifications on a channel using the notification helper method:

```
Echo.private('App.User.' + userId)
    .notification((notification) => {
        console.log(notification.type);
    });
```

Customizing The Notification Channel

If you would like to customize which channels a notifiable entity receives its broadcast notifications on, you may define a `receivesBroadcastNotificationsOn` method on the notifiable entity:

```
<?php

namespace App;

use Illuminate\Broadcasting\PrivateChannel;
use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;

class User extends Authenticatable
{
    use Notifiable;

    /**
     * The channels the user receives notification broadcasts on.
     *
     * @return string
     */
    public function receivesBroadcastNotificationsOn()
    {
        return 'users.'.$this->id;
    }
}
```

SMS Notifications

Prerequisites

Sending SMS notifications in Laravel is powered by [Nexmo](#). Before you can send notifications via Nexmo, you need to install the `laravel/nexmo-notification-channel` Composer package:

```
composer require laravel/nexmo-notification-channel
```

This will also install the [nexmo/laravel](#) package. This package includes [its own configuration file](#). You can use the `NEXMO_KEY` and `NEXMO_SECRET` environment variables to set your Nexmo public and secret key.

Next, you will need to add a configuration option to your `config/services.php` configuration file. You may copy the example configuration below to get started:

```
'nexmo' => [
    'sms_from' => '15556666666',
],
```

The `sms_from` option is the phone number that your SMS messages will be sent from. You should generate a phone number for your application in the Nexmo control panel.

Formatting SMS Notifications

If a notification supports being sent as an SMS, you should define a `toNexmo` method on the notification class. This method will receive a `$notifiable` entity and should return a `Illuminate\Notifications\Messages\NexmoMessage` instance:

```
/**
 * Get the Nexmo / SMS representation of the notification.
 *
 * @param mixed $notifiable
 * @return NexmoMessage
 */
public function toNexmo($notifiable)
{
    return (new NexmoMessage)
        ->content('Your SMS message content');
}
```

Formatting Shortcode Notifications

Laravel also supports sending shortcode notifications, which are pre-defined message templates in your Nexmo account. You may specify the type of notification (`alert`, `2fa`, or `marketing`), as well as the custom values that will populate the template:

```
/**
 * Get the Nexmo / Shortcode representation of the notification.
 *
 * @param mixed $notifiable
 * @return array
 */
public function toShortcode($notifiable)
{
    return [
        'type' => 'alert',
        'custom' => [
            'code' => 'ABC123',
        ],
    ];
}
```

TIP Like [routing SMS Notifications](#), you should implement the `routeNotificationForShortcode` method on your notifiable model.

Unicode Content

If your SMS message will contain unicode characters, you should call the `unicode` method when constructing the `NexmoMessage` instance:

```
/**
 * Get the Nexmo / SMS representation of the notification.
 *
```



```
* @param mixed $notifiable
* @return NexmoMessage
*/
public function toNexmo($notifiable)
{
    return (new NexmoMessage)
        ->content('Your unicode message')
        ->unicode();
}
```

Customizing The "From" Number

If you would like to send some notifications from a phone number that is different from the phone number specified in your config/services.php file, you may use the from method on a NexmoMessage instance:

```
/**
 * Get the Nexmo / SMS representation of the notification.
 *
 * @param mixed $notifiable
 * @return NexmoMessage
 */
public function toNexmo($notifiable)
{
    return (new NexmoMessage)
        ->content('Your SMS message content')
        ->from('15554443333');
}
```

Routing SMS Notifications

To route Nexmo notifications to the proper phone number, define a routeNotificationForNexmo method on your notifiable entity:

```
<?php

namespace App;

use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;

class User extends Authenticatable
{
    use Notifiable;

    /**
     * Route notifications for the Nexmo channel.
     *
     * @param \Illuminate\Notifications\Notification $notification
     * @return string
     */
    public function routeNotificationForNexmo($notification)
    {
        return $this->phone_number;
    }
}
```

Slack Notifications

Prerequisites

Before you can send notifications via Slack, you must install the notification channel via Composer:

```
composer require laravel/slack-notification-channel
```

You will also need to configure an ["Incoming Webhook"](#) integration for your Slack team. This integration will provide you with a URL you may use when [routing Slack notifications](#).

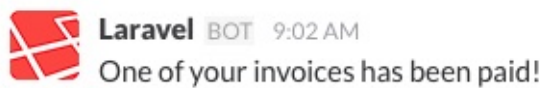
Formatting Slack Notifications

If a notification supports being sent as a Slack message, you should define a toSlack method on the notification

class. This method will receive a `$notifiable` entity and should return a `Illuminate\Notifications\Messages\SlackMessage` instance. Slack messages may contain text content as well as an "attachment" that formats additional text or an array of fields. Let's take a look at a basic `toSlack` example:

```
/**
 * Get the Slack representation of the notification.
 *
 * @param mixed $notifiable
 * @return SlackMessage
 */
public function toSlack($notifiable)
{
    return (new SlackMessage)
        ->content('One of your invoices has been paid!');
}
```

In this example we are just sending a single line of text to Slack, which will create a message that looks like the following:



Customizing The Sender & Recipient

You may use the `from` and `to` methods to customize the sender and recipient. The `from` method accepts a username and emoji identifier, while the `to` method accepts a channel or username:

```
/**
 * Get the Slack representation of the notification.
 *
 * @param mixed $notifiable
 * @return SlackMessage
 */
public function toSlack($notifiable)
{
    return (new SlackMessage)
        ->from('Ghost', ':ghost:')
        ->to('#other')
        ->content('This will be sent to #other');
}
```

You may also use an image as your logo instead of an emoji:

```
/**
 * Get the Slack representation of the notification.
 *
 * @param mixed $notifiable
 * @return SlackMessage
 */
public function toSlack($notifiable)
{
    return (new SlackMessage)
        ->from('Laravel')
        ->image('https://laravel.com/img/favicon/favicon.ico')
        ->content('This will display the Laravel logo next to the message');
}
```

Slack Attachments

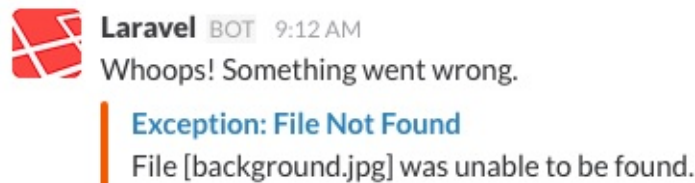
You may also add "attachments" to Slack messages. Attachments provide richer formatting options than simple text messages. In this example, we will send an error notification about an exception that occurred in an application, including a link to view more details about the exception:

```
/**
 * Get the Slack representation of the notification.
 *
 * @param mixed $notifiable
 * @return SlackMessage
 */
public function toSlack($notifiable)
```

```
{
    $url = url('/exceptions/'. $this->exception->id);

    return (new SlackMessage)
        ->error()
        ->content('Whoops! Something went wrong.')
        ->attachment(function ($attachment) use ($url) {
            $attachment->title('Exception: File Not Found', $url)
                ->content('File [background.jpg] was not found.');
        });
}
```

The example above will generate a Slack message that looks like the following:

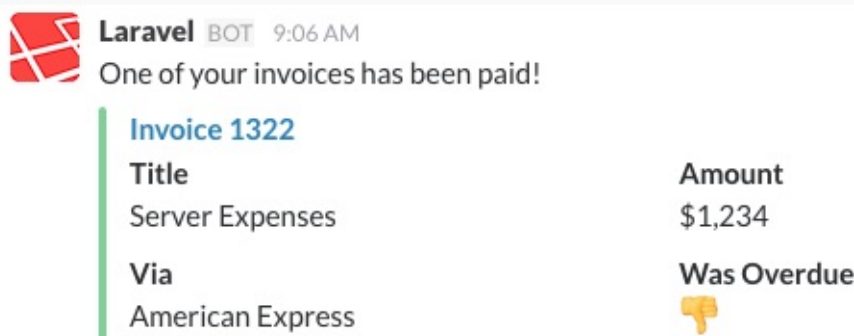


Attachments also allow you to specify an array of data that should be presented to the user. The given data will be presented in a table-style format for easy reading:

```
/**
 * Get the Slack representation of the notification.
 *
 * @param mixed $notifiable
 * @return SlackMessage
 */
public function toSlack($notifiable)
{
    $url = url('/invoices/'. $this->invoice->id);

    return (new SlackMessage)
        ->success()
        ->content('One of your invoices has been paid!')
        ->attachment(function ($attachment) use ($url) {
            $attachment->title('Invoice 1322', $url)
                ->fields([
                    'Title' => 'Server Expenses',
                    'Amount' => '$1,234',
                    'Via' => 'American Express',
                    'Was Overdue' => ':-1:',
                ]);
        });
}
```

The example above will create a Slack message that looks like the following:



Markdown Attachment Content

If some of your attachment fields contain Markdown, you may use the `markdown` method to instruct Slack to parse and display the given attachment fields as Markdown formatted text. The values accepted by this method are: `pretext`, `text`, and `/ or fields`. For more information about Slack attachment formatting, check out the

[Slack API documentation:](#)

```
/**
 * Get the Slack representation of the notification.
 *
 * @param mixed $notifiable
 * @return SlackMessage
 */
public function toSlack($notifiable)
{
    $url = url('/exceptions/'. $this->exception->id);

    return (new SlackMessage)
        ->error()
        ->content('Whoops! Something went wrong.')
        ->attachment(function ($attachment) use ($url) {
            $attachment->title('Exception: File Not Found', $url)
                ->content('File [background.jpg] was *not found*.')
                ->markdown(['text']);
        });
}
```

Routing Slack Notifications

To route Slack notifications to the proper location, define a `routeNotificationForSlack` method on your notifiable entity. This should return the webhook URL to which the notification should be delivered. Webhook URLs may be generated by adding an "Incoming Webhook" service to your Slack team:

```
<?php

namespace App;

use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;

class User extends Authenticatable
{
    use Notifiable;

    /**
     * Route notifications for the Slack channel.
     *
     * @param \Illuminate\Notifications\Notification $notification
     * @return string
     */
    public function routeNotificationForSlack($notification)
    {
        return 'https://hooks.slack.com/services/...';
    }
}
```

Localizing Notifications

Laravel allows you to send notifications in a locale other than the current language, and will even remember this locale if the notification is queued.

To accomplish this, the `Illuminate\Notifications\Notification` class offers a `locale` method to set the desired language. The application will change into this locale when the notification is being formatted and then revert back to the previous locale when formatting is complete:

```
$user->notify((new InvoicePaid($invoice))->locale('es'));
```

Localization of multiple notifiable entries may also be achieved via the `Notification` facade:

```
Notification::locale('es')->send($users, new InvoicePaid($invoice));
```

User Preferred Locales

Sometimes, applications store each user's preferred locale. By implementing the `HasLocalePreference` contract on your notifiable model, you may instruct Laravel to use this stored locale when sending a notification:

```
use Illuminate\Contracts\Translation\HasLocalePreference;

class User extends Model implements HasLocalePreference
{
    /**
     * Get the user's preferred locale.
     *
     * @return string
     */
    public function preferredLocale()
    {
        return $this->locale;
    }
}
```

Once you have implemented the interface, Laravel will automatically use the preferred locale when sending notifications and mailables to the model. Therefore, there is no need to call the `locale` method when using this interface:

```
$user->notify(new InvoicePaid($invoice));
```

Notification Events

When a notification is sent, the `Illuminate\Notifications\Events\NotificationSent` event is fired by the notification system. This contains the "notifiable" entity and the notification instance itself. You may register listeners for this event in your `EventServiceProvider`:

```
/**
 * The event listener mappings for the application.
 *
 * @var array
 */
protected $listen = [
    'Illuminate\Notifications\Events\NotificationSent' => [
        'App\Listeners\LogNotification',
    ],
];
```

TIP After registering listeners in your `EventServiceProvider`, use the `event:generate` Artisan command to quickly generate listener classes.

Within an event listener, you may access the `notifiable`, `notification`, and `channel` properties on the event to learn more about the notification recipient or the notification itself:

```
/**
 * Handle the event.
 *
 * @param NotificationSent $event
 * @return void
 */
public function handle(NotificationSent $event)
{
    // $event->channel
    // $event->notifiable
    // $event->notification
    // $event->response
}
```

Custom Channels

Laravel ships with a handful of notification channels, but you may want to write your own drivers to deliver notifications via other channels. Laravel makes it simple. To get started, define a class that contains a `send` method. The method should receive two arguments: a `$notifiable` and a `$notification`:

```
<?php

namespace App\Channels;

use Illuminate\Notifications\Notification;

class VoiceChannel
{
```

```
/**
 * Send the given notification.
 *
 * @param mixed $notifiable
 * @param \Illuminate\Notifications\Notification $notification
 * @return void
 */
public function send($notifiable, Notification $notification)
{
    $message = $notification->toVoice($notifiable);

    // Send notification to the $notifiable instance...
}
}
```

Once your notification channel class has been defined, you may return the class name from the via method of any of your notifications:

```
<?php

namespace App\Notifications;

use App\Channels\Messages\VoiceMessage;
use App\Channels\VoiceChannel;
use Illuminate\Bus\Queueable;
use Illuminate\Contracts\Queue\ShouldQueue;
use Illuminate\Notifications\Notification;

class InvoicePaid extends Notification
{
    use Queueable;

    /**
     * Get the notification channels.
     *
     * @param mixed $notifiable
     * @return array|string
     */
    public function via($notifiable)
    {
        return [VoiceChannel::class];
    }

    /**
     * Get the voice representation of the notification.
     *
     * @param mixed $notifiable
     * @return VoiceMessage
     */
    public function toVoice($notifiable)
    {
        // ...
    }
}
```

Digging Deeper

Package Development

Package Development

- [Introduction](#)
 - [A Note On Facades](#)
- [Package Discovery](#)
- [Service Providers](#)
- [Resources](#)
 - [Configuration](#)
 - [Migrations](#)
 - [Factories](#)
 - [Routes](#)
 - [Translations](#)
 - [Views](#)
 - [View Components](#)
- [Commands](#)
- [Public Assets](#)
- [Publishing File Groups](#)

Introduction

Packages are the primary way of adding functionality to Laravel. Packages might be anything from a great way to work with dates like [Carbon](#), or an entire BDD testing framework like [Behat](#).

There are different types of packages. Some packages are stand-alone, meaning they work with any PHP framework. Carbon and Behat are examples of stand-alone packages. Any of these packages may be used with Laravel by requesting them in your `composer.json` file.

On the other hand, other packages are specifically intended for use with Laravel. These packages may have routes, controllers, views, and configuration specifically intended to enhance a Laravel application. This guide primarily covers the development of those packages that are Laravel specific.

A Note On Facades

When writing a Laravel application, it generally does not matter if you use contracts or facades since both provide essentially equal levels of testability. However, when writing packages, your package will not typically have access to all of Laravel's testing helpers. If you would like to be able to write your package tests as if they existed inside a typical Laravel application, you may use the [Orchestral Testbench](#) package.

Package Discovery

In a Laravel application's `config/app.php` configuration file, the `providers` option defines a list of service providers that should be loaded by Laravel. When someone installs your package, you will typically want your service provider to be included in this list. Instead of requiring users to manually add your service provider to the list, you may define the provider in the `extra` section of your package's `composer.json` file. In addition to service providers, you may also list any [facades](#) you would like to be registered:

```
"extra": {
    "laravel": {
        "providers": [
            "Barryvdh\\Debugbar\\ServiceProvider"
        ],
        "aliases": {
            "Debugbar": "Barryvdh\\Debugbar\\Facade"
        }
    }
},
```

Once your package has been configured for discovery, Laravel will automatically register its service providers and facades when it is installed, creating a convenient installation experience for your package's users.

Opting Out Of Package Discovery

If you are the consumer of a package and would like to disable package discovery for a package, you may list the package name in the `extra` section of your application's `composer.json` file:

```
"extra": {
    "laravel": {
        "dont-discover": [
            "barryvdh/laravel-debugbar"
        ]
    }
},
```

You may disable package discovery for all packages using the `*` character inside of your application's `dont-discover` directive:

```
"extra": {
    "laravel": {
        "dont-discover": [
            "*"
        ]
    }
},
```

Service Providers

[Service providers](#) are the connection points between your package and Laravel. A service provider is responsible for binding things into Laravel's [service container](#) and informing Laravel where to load package resources such as views, configuration, and localization files.

A service provider extends the `Illuminate\Support\ServiceProvider` class and contains two methods: `register` and `boot`. The base `ServiceProvider` class is located in the `illuminate/support` Composer package, which you should add to your own package's dependencies. To learn more about the structure and purpose of service providers, check out [their documentation](#).

Resources

Configuration

Typically, you will need to publish your package's configuration file to the application's own `config` directory. This will allow users of your package to easily override your default configuration options. To allow your configuration files to be published, call the `publishes` method from the `boot` method of your service provider:

```
/**
 * Bootstrap any application services.
 *
 * @return void
 */
public function boot()
{
    $this->publishes([
        __DIR__.'/path/to/config/courier.php' => config_path('courier.php'),
    ]);
}
```

Now, when users of your package execute Laravel's `vendor:publish` command, your file will be copied to the specified publish location. Once your configuration has been published, its values may be accessed like any other configuration file:

```
$value = config('courier.option');
```

NOTE You should not define Closures in your configuration files. They can not be serialized correctly when users execute the `config:cache` Artisan command.

Default Package Configuration

You may also merge your own package configuration file with the application's published copy. This will allow your users to define only the options they actually want to override in the published copy of the configuration. To merge the configurations, use the `mergeConfigFrom` method within your service provider's `register` method:

```
/**
 * Register any application services.
 *
 * @return void
 */
public function register()
{
    $this->mergeConfigFrom(
        __DIR__.'/path/to/config/courier.php', 'courier'
    );
}
```

NOTE This method only merges the first level of the configuration array. If your users partially define a multi-dimensional configuration array, the missing options will not be merged.

Routes

If your package contains routes, you may load them using the `loadRoutesFrom` method. This method will automatically determine if the application's routes are cached and will not load your routes file if the routes have already been cached:

```
/**
 * Bootstrap any application services.
 *
 * @return void
 */
public function boot()
{
    $this->loadRoutesFrom(__DIR__.'/routes.php');
}
```

Migrations

If your package contains [database migrations](#), you may use the `loadMigrationsFrom` method to inform Laravel how to load them. The `loadMigrationsFrom` method accepts the path to your package's migrations as its only argument:

```
/**
 * Bootstrap any application services.
 *
 * @return void
 */
public function boot()
{
    $this->loadMigrationsFrom(__DIR__.'/path/to/migrations');
}
```

Once your package's migrations have been registered, they will automatically be run when the `php artisan migrate` command is executed. You do not need to export them to the application's main `database/migrations` directory.

Factories

If your package contains [database factories](#), you may use the `loadFactoriesFrom` method to inform Laravel how to load them. The `loadFactoriesFrom` method accepts the path to your package's factories as its only argument:

```
/**
 * Bootstrap any application services.
 *
 * @return void
 */
public function boot()
{
```

```
$this->loadFactoriesFrom(__DIR__.'/path/to/factories');  
}
```

Once your package's factories have been registered, you can use them in your application:

```
factory(Package\Namespace\Model::class)->create();
```

Translations

If your package contains [translation files](#), you may use the `loadTranslationsFrom` method to inform Laravel how to load them. For example, if your package is named `courier`, you should add the following to your service provider's `boot` method:

```
/**  
 * Bootstrap any application services.  
 *  
 * @return void  
 */  
public function boot()  
{  
    $this->loadTranslationsFrom(__DIR__.'/path/to/translations', 'courier');  
}
```

Package translations are referenced using the `package::file.line` syntax convention. So, you may load the `courier` package's `welcome` line from the `messages` file like so:

```
echo trans('courier::messages.welcome');
```

Publishing Translations

If you would like to publish your package's translations to the application's `resources/lang/vendor` directory, you may use the service provider's `publishes` method. The `publishes` method accepts an array of package paths and their desired publish locations. For example, to publish the translation files for the `courier` package, you may do the following:

```
/**  
 * Bootstrap any application services.  
 *  
 * @return void  
 */  
public function boot()  
{  
    $this->loadTranslationsFrom(__DIR__.'/path/to/translations', 'courier');  
  
    $this->publishes([  
        __DIR__.'/path/to/translations' => resource_path('lang/vendor/courier'),  
    ]);  
}
```

Now, when users of your package execute Laravel's `vendor:publish` Artisan command, your package's translations will be published to the specified publish location.

Views

To register your package's [views](#) with Laravel, you need to tell Laravel where the views are located. You may do this using the service provider's `loadViewsFrom` method. The `loadViewsFrom` method accepts two arguments: the path to your view templates and your package's name. For example, if your package's name is `courier`, you would add the following to your service provider's `boot` method:

```
/**  
 * Bootstrap any application services.  
 *  
 * @return void  
 */  
public function boot()  
{  
    $this->loadViewsFrom(__DIR__.'/path/to/views', 'courier');  
}
```

Package views are referenced using the `package::view` syntax convention. So, once your view path is registered in a service provider, you may load the `admin` view from the `courier` package like so:

```
Route::get('admin', function () {
    return view('courier::admin');
});
```

Overriding Package Views

When you use the `loadViewsFrom` method, Laravel actually registers two locations for your views: the application's `resources/views/vendor` directory and the directory you specify. So, using the `courier` example, Laravel will first check if a custom version of the view has been provided by the developer in `resources/views/vendor/courier`. Then, if the view has not been customized, Laravel will search the package view directory you specified in your call to `loadViewsFrom`. This makes it easy for package users to customize / override your package's views.

Publishing Views

If you would like to make your views available for publishing to the application's `resources/views/vendor` directory, you may use the service provider's `publishes` method. The `publishes` method accepts an array of package view paths and their desired publish locations:

```
/**
 * Bootstrap any application services.
 *
 * @return void
 */
public function boot()
{
    $this->loadViewsFrom(__DIR__.'/path/to/views', 'courier');

    $this->publishes([
        __DIR__.'/path/to/views' => resource_path('views/vendor/courier'),
    ]);
}
```

Now, when users of your package execute Laravel's `vendor:publish` Artisan command, your package's views will be copied to the specified publish location.

View Components

If your package contains [view components](#), you may use the `loadViewComponentsAs` method to inform Laravel how to load them. The `loadViewComponentsAs` method accepts two arguments: the tag prefix for your view components and an array of your view components class. For example, if your package's prefix is `courier` and you have `Alert` and `Button` view components, you would add the following to your service provider's `boot` method:

```
/**
 * Bootstrap any application services.
 *
 * @return void
 */
public function boot()
{
    $this->loadViewComponentsAs('courier', [
        Alert::class,
        Button::class,
    ]);
}
```

Once your view components are registered in a service provider, you may reference them in your view like so:

```
<x-courier-alert />

<x-courier-button />
```

Anonymous Components

If your package contains anonymous components, they must be placed within a `components` directory of your package's "views" directory (as specified by `loadViewsFrom`). Then, you may render them by prefixing the component name with the package's view namespace:

```
<x-courier::alert />
```

Commands

To register your package's Artisan commands with Laravel, you may use the `commands` method. This method expects an array of command class names. Once the commands have been registered, you may execute them using the [Artisan CLI](#):

```
/**
 * Bootstrap the application services.
 *
 * @return void
 */
public function boot()
{
    if ($this->app->runningInConsole()) {
        $this->commands([
            FooCommand::class,
            BarCommand::class,
        ]);
    }
}
```

Public Assets

Your package may have assets such as JavaScript, CSS, and images. To publish these assets to the application's public directory, use the service provider's `publishes` method. In this example, we will also add a `public` asset group tag, which may be used to publish groups of related assets:

```
/**
 * Bootstrap any application services.
 *
 * @return void
 */
public function boot()
{
    $this->publishes([
        __DIR__.'/path/to/assets' => public_path('vendor/courier'),
    ], 'public');
}
```

Now, when your package's users execute the `vendor:publish` command, your assets will be copied to the specified publish location. Since you will typically need to overwrite the assets every time the package is updated, you may use the `--force` flag:

```
php artisan vendor:publish --tag=public --force
```

Publishing File Groups

You may want to publish groups of package assets and resources separately. For instance, you might want to allow your users to publish your package's configuration files without being forced to publish your package's assets. You may do this by "tagging" them when calling the `publishes` method from a package's service provider. For example, let's use tags to define two publish groups in the `boot` method of a package service provider:

```
/**
 * Bootstrap any application services.
 *
 * @return void
 */
public function boot()
{
    $this->publishes([
        __DIR__.'/../config/package.php' => config_path('package.php')
    ], 'config');
```

```
$this->publishes([
    __DIR__.'/../database/migrations/' => database_path('migrations')
], 'migrations');
```

Now your users may publish these groups separately by referencing their tag when executing the `vendor:publish` command:

```
php artisan vendor:publish --tag=config
```

Digging Deeper

Queues

Queues

- [Introduction](#)
 - [Connections Vs. Queues](#)
 - [Driver Notes & Prerequisites](#)
- [Creating Jobs](#)
 - [Generating Job Classes](#)
 - [Class Structure](#)
 - [Job Middleware](#)
- [Dispatching Jobs](#)
 - [Delayed Dispatching](#)
 - [Synchronous Dispatching](#)
 - [Job Chaining](#)
 - [Customizing The Queue & Connection](#)
 - [Specifying Max Job Attempts / Timeout Values](#)
 - [Rate Limiting](#)
 - [Error Handling](#)
- [Queueing Closures](#)
- [Running The Queue Worker](#)
 - [Queue Priorities](#)
 - [Queue Workers & Deployment](#)
 - [Job Expirations & Timeouts](#)
- [Supervisor Configuration](#)
- [Dealing With Failed Jobs](#)
 - [Cleaning Up After Failed Jobs](#)
 - [Failed Job Events](#)
 - [Retrying Failed Jobs](#)
 - [Ignoring Missing Models](#)
- [Job Events](#)

Introduction

TIP Laravel now offers Horizon, a beautiful dashboard and configuration system for your Redis powered queues. Check out the full [Horizon documentation](#) for more information.

Laravel queues provide a unified API across a variety of different queue backends, such as Beanstalk, Amazon SQS, Redis, or even a relational database. Queues allow you to defer the processing of a time consuming task, such as sending an email, until a later time. Deferring these time consuming tasks drastically speeds up web requests to your application.

The queue configuration file is stored in `config/queue.php`. In this file you will find connection configurations for each of the queue drivers that are included with the framework, which includes a database, [Beanstalkd](#), [Amazon SQS](#), [Redis](#), and a synchronous driver that will execute jobs immediately (for local use). A `null` queue driver is also included which discards queued jobs.

Connections Vs. Queues

Before getting started with Laravel queues, it is important to understand the distinction between "connections" and "queues". In your `config/queue.php` configuration file, there is a `connections` configuration option. This option defines a particular connection to a backend service such as Amazon SQS, Beanstalk, or Redis. However, any given queue connection may have multiple "queues" which may be thought of as different stacks or piles of queued jobs.

Note that each connection configuration example in the `queue` configuration file contains a `queue` attribute. This is the default queue that jobs will be dispatched to when they are sent to a given connection. In other words, if

you dispatch a job without explicitly defining which queue it should be dispatched to, the job will be placed on the queue that is defined in the `queue` attribute of the connection configuration:

```
// This job is sent to the default queue...
Job::dispatch();

// This job is sent to the "emails" queue...
Job::dispatch()->onQueue('emails');
```

Some applications may not need to ever push jobs onto multiple queues, instead preferring to have one simple queue. However, pushing jobs to multiple queues can be especially useful for applications that wish to prioritize or segment how jobs are processed, since the Laravel queue worker allows you to specify which queues it should process by priority. For example, if you push jobs to a `high` queue, you may run a worker that gives them higher processing priority:

```
php artisan queue:work --queue=high,default
```

Driver Notes & Prerequisites

Database

In order to use the database queue driver, you will need a database table to hold the jobs. To generate a migration that creates this table, run the `queue:table` Artisan command. Once the migration has been created, you may migrate your database using the `migrate` command:

```
php artisan queue:table

php artisan migrate
```

Redis

In order to use the `redis` queue driver, you should configure a Redis database connection in your `config/database.php` configuration file.

Redis Cluster

If your Redis queue connection uses a Redis Cluster, your queue names must contain a [key hash tag](#). This is required in order to ensure all of the Redis keys for a given queue are placed into the same hash slot:

```
'redis' => [
    'driver' => 'redis',
    'connection' => 'default',
    'queue' => '{default}',
    'retry_after' => 90,
],
```

Blocking

When using the Redis queue, you may use the `block_for` configuration option to specify how long the driver should wait for a job to become available before iterating through the worker loop and re-polling the Redis database.

Adjusting this value based on your queue load can be more efficient than continually polling the Redis database for new jobs. For instance, you may set the value to `5` to indicate that the driver should block for five seconds while waiting for a job to become available:

```
'redis' => [
    'driver' => 'redis',
    'connection' => 'default',
    'queue' => 'default',
    'retry_after' => 90,
    'block_for' => 5,
],
```

NOTE Setting `block_for` to `0` will cause queue workers to block indefinitely until a job is available. This will also prevent signals such as `SIGTERM` from being handled until the next job has been processed.

Other Driver Prerequisites

The following dependencies are needed for the listed queue drivers:

- Amazon SQS: `aws/aws-sdk-php ~3.0`
- Beanstalkd: `pda/pheanstalk ~4.0`
- Redis: `redis/predis ~1.0` or `phpredis` PHP extension

Creating Jobs

Generating Job Classes

By default, all of the queueable jobs for your application are stored in the `app/Jobs` directory. If the `app/Jobs` directory doesn't exist, it will be created when you run the `make:job` Artisan command. You may generate a new queued job using the Artisan CLI:

```
php artisan make:job ProcessPodcast
```

The generated class will implement the `Illuminate\Contracts\Queue\ShouldQueue` interface, indicating to Laravel that the job should be pushed onto the queue to run asynchronously.

TIP Job stubs may be customized using [stub publishing](#)

Class Structure

Job classes are very simple, normally containing only a `handle` method which is called when the job is processed by the queue. To get started, let's take a look at an example job class. In this example, we'll pretend we manage a podcast publishing service and need to process the uploaded podcast files before they are published:

```
<?php

namespace App\Jobs;

use App\AudioProcessor;
use App\Podcast;
use Illuminate\Bus\Queueable;
use Illuminate\Contracts\Queue\ShouldQueue;
use Illuminate\Foundation\Bus\Dispatchable;
use Illuminate\Queue\InteractsWithQueue;
use Illuminate\Queue\SerializesModels;

class ProcessPodcast implements ShouldQueue
{
    use Dispatchable, InteractsWithQueue, Queueable, SerializesModels;

    protected $podcast;

    /**
     * Create a new job instance.
     *
     * @param Podcast $podcast
     * @return void
     */
    public function __construct(Podcast $podcast)
    {
        $this->podcast = $podcast;
    }

    /**
     * Execute the job.
     *
     * @param AudioProcessor $processor
     * @return void
     */
    public function handle(AudioProcessor $processor)
    {
        // Process uploaded podcast...
    }
}
```


In this example, note that we were able to pass an [Eloquent model](#) directly into the queued job's constructor. Because of the `SerializesModels` trait that the job is using, Eloquent models and their loaded relationships will be gracefully serialized and unserialized when the job is processing. If your queued job accepts an Eloquent model in its constructor, only the identifier for the model will be serialized onto the queue. When the job is actually handled, the queue system will automatically re-retrieve the full model instance and its loaded relationships from the database. It's all totally transparent to your application and prevents issues that can arise from serializing full Eloquent model instances.

The `handle` method is called when the job is processed by the queue. Note that we are able to type-hint dependencies on the `handle` method of the job. The Laravel [service container](#) automatically injects these dependencies.

If you would like to take total control over how the container injects dependencies into the `handle` method, you may use the container's `bindMethod` method. The `bindMethod` method accepts a callback which receives the job and the container. Within the callback, you are free to invoke the `handle` method however you wish. Typically, you should call this method from a [service provider](#):

```
use App\Jobs\ProcessPodcast;

$this->app->bindMethod(ProcessPodcast::class, '@handle', function ($job, $app) {
    return $job->handle($app->make(AudioProcessor::class));
});
```

NOTE Binary data, such as raw image contents, should be passed through the `base64_encode` function before being passed to a queued job. Otherwise, the job may not properly serialize to JSON when being placed on the queue.

Handling Relationships

Because loaded relationships also get serialized, the serialized job string can become quite large. To prevent relations from being serialized, you can call the `withoutRelations` method on the model when setting a property value. This method will return an instance of the model with no loaded relationships:

```
/**
 * Create a new job instance.
 *
 * @param \App\Podcast $podcast
 * @return void
 */
public function __construct(Podcast $podcast)
{
    $this->podcast = $podcast->withoutRelations();
}
```

Job Middleware

Job middleware allow you to wrap custom logic around the execution of queued jobs, reducing boilerplate in the jobs themselves. For example, consider the following `handle` method which leverages Laravel's Redis rate limiting features to allow only one job to process every five seconds:

```
/**
 * Execute the job.
 *
 * @return void
 */
public function handle()
{
    Redis::throttle('key')->block(0)->allow(1)->every(5)->then(function () {
        info('Lock obtained...');

        // Handle job...
    }, function () {
        // Could not obtain lock...

        return $this->release(5);
    });
}
```

While this code is valid, the structure of the `handle` method becomes noisy since it is cluttered with Redis rate

limiting logic. In addition, this rate limiting logic must be duplicated for any other jobs that we want to rate limit.

Instead of rate limiting in the handle method, we could define a job middleware that handles rate limiting. Laravel does not have a default location for job middleware, so you are welcome to place job middleware anywhere in your application. In this example, we will place the middleware in a `app/Jobs/Middleware` directory:

```
<?php

namespace App\Jobs\Middleware;

use Illuminate\Support\Facades\Redis;

class RateLimited
{
    /**
     * Process the queued job.
     *
     * @param mixed $job
     * @param callable $next
     * @return mixed
     */
    public function handle($job, $next)
    {
        Redis::throttle('key')
            ->block(0)->allow(1)->every(5)
            ->then(function () use ($job, $next) {
                // Lock obtained...

                $next($job);
            }, function () use ($job) {
                // Could not obtain lock...

                $job->release(5);
            });
    }
}
```

As you can see, like [route middleware](#), job middleware receive the job being processed and a callback that should be invoked to continue processing the job.

After creating job middleware, they may be attached to a job by returning them from the job's `middleware` method. This method does not exist on jobs scaffolded by the `make:job` Artisan command, so you will need to add it to your own job class definition:

```
use App\Jobs\Middleware\RateLimited;

/**
 * Get the middleware the job should pass through.
 *
 * @return array
 */
public function middleware()
{
    return [new RateLimited];
}
```

Dispatching Jobs

Once you have written your job class, you may dispatch it using the `dispatch` method on the job itself. The arguments passed to the `dispatch` method will be given to the job's constructor:

```
<?php

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;
use App\Jobs\ProcessPodcast;
use Illuminate\Http\Request;

class PodcastController extends Controller
{
```

```
/**
 * Store a new podcast.
 *
 * @param Request $request
 * @return Response
 */
public function store(Request $request)
{
    // Create podcast...

    ProcessPodcast::dispatch($podcast);
}
}
```

If you would like to conditionally dispatch a job, you may use the `dispatchIf` and `dispatchUnless` methods:

```
ProcessPodcast::dispatchIf($accountActive === true, $podcast);

ProcessPodcast::dispatchUnless($accountSuspended === false, $podcast);
```

Delayed Dispatching

If you would like to delay the execution of a queued job, you may use the `delay` method when dispatching a job. For example, let's specify that a job should not be available for processing until 10 minutes after it has been dispatched:

```
<?php

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;
use App\Jobs\ProcessPodcast;
use Illuminate\Http\Request;

class PodcastController extends Controller
{
    /**
     * Store a new podcast.
     *
     * @param Request $request
     * @return Response
     */
    public function store(Request $request)
    {
        // Create podcast...

        ProcessPodcast::dispatch($podcast)
            ->delay(now()->addMinutes(10));
    }
}
```

NOTE The Amazon SQS queue service has a maximum delay time of 15 minutes.

Dispatching After The Response Is Sent To Browser

Alternatively, the `dispatchAfterResponse` method delays dispatching a job until after the response is sent to the user's browser. This will still allow the user to begin using the application even though a queued job is still executing. This should typically only be used for jobs that take about a second, such as sending an email:

```
use App\Jobs\SendNotification;

SendNotification::dispatchAfterResponse();
```

You may dispatch a Closure and chain the `afterResponse` method onto the helper to execute a Closure after the response has been sent to the browser:

```
use App\Mail\WelcomeMessage;
use Illuminate\Support\Facades\Mail;

dispatch(function () {
    Mail::to('taylor@laravel.com')->send(new WelcomeMessage);
})->afterResponse();
```

Synchronous Dispatching

If you would like to dispatch a job immediately (synchronously), you may use the `dispatchNow` method. When using this method, the job will not be queued and will be run immediately within the current process:

```
<?php

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;
use App\Jobs\ProcessPodcast;
use Illuminate\Http\Request;

class PodcastController extends Controller
{
    /**
     * Store a new podcast.
     *
     * @param Request $request
     * @return Response
     */
    public function store(Request $request)
    {
        // Create podcast...

        ProcessPodcast::dispatchNow($podcast);
    }
}
```

Job Chaining

Job chaining allows you to specify a list of queued jobs that should be run in sequence after the primary job has executed successfully. If one job in the sequence fails, the rest of the jobs will not be run. To execute a queued job chain, you may use the `withChain` method on any of your dispatchable jobs:

```
ProcessPodcast::withChain([
    new OptimizePodcast,
    new ReleasePodcast
])->dispatch();
```

In addition to chaining job class instances, you may also chain Closures:

```
ProcessPodcast::withChain([
    new OptimizePodcast,
    new ReleasePodcast,
    function () {
        Podcast::update(...);
    },
])->dispatch();
```

NOTE Deleting jobs using the `$this->delete()` method will not prevent chained jobs from being processed. The chain will only stop executing if a job in the chain fails.

Chain Connection & Queue

If you would like to specify the default connection and queue that should be used for the chained jobs, you may use the `allOnConnection` and `allOnQueue` methods. These methods specify the queue connection and queue name that should be used unless the queued job is explicitly assigned a different connection / queue:

```
ProcessPodcast::withChain([
    new OptimizePodcast,
    new ReleasePodcast
])->dispatch()->allOnConnection('redis')->allOnQueue('podcasts');
```

Customizing The Queue & Connection

Dispatching To A Particular Queue

By pushing jobs to different queues, you may "categorize" your queued jobs and even prioritize how many

workers you assign to various queues. Keep in mind, this does not push jobs to different queue "connections" as defined by your queue configuration file, but only to specific queues within a single connection. To specify the queue, use the `onQueue` method when dispatching the job:

```
<?php

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;
use App\Jobs\ProcessPodcast;
use Illuminate\Http\Request;

class PodcastController extends Controller
{
    /**
     * Store a new podcast.
     *
     * @param Request $request
     * @return Response
     */
    public function store(Request $request)
    {
        // Create podcast...

        ProcessPodcast::dispatch($podcast)->onQueue('processing');
    }
}
```

Dispatching To A Particular Connection

If you are working with multiple queue connections, you may specify which connection to push a job to. To specify the connection, use the `onConnection` method when dispatching the job:

```
<?php

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;
use App\Jobs\ProcessPodcast;
use Illuminate\Http\Request;

class PodcastController extends Controller
{
    /**
     * Store a new podcast.
     *
     * @param Request $request
     * @return Response
     */
    public function store(Request $request)
    {
        // Create podcast...

        ProcessPodcast::dispatch($podcast)->onConnection('sqs');
    }
}
```

You may chain the `onConnection` and `onQueue` methods to specify the connection and the queue for a job:

```
ProcessPodcast::dispatch($podcast)
    ->onConnection('sqs')
    ->onQueue('processing');
```

Specifying Max Job Attempts / Timeout Values

Max Attempts

One approach to specifying the maximum number of times a job may be attempted is via the `--tries` switch on the Artisan command line:

```
php artisan queue:work --tries=3
```

However, you may take a more granular approach by defining the maximum number of attempts on the job

class itself. If the maximum number of attempts is specified on the job, it will take precedence over the value provided on the command line:

```
<?php

namespace App\Jobs;

class ProcessPodcast implements ShouldQueue
{
    /**
     * The number of times the job may be attempted.
     *
     * @var int
     */
    public $tries = 5;
}
```

Time Based Attempts

As an alternative to defining how many times a job may be attempted before it fails, you may define a time at which the job should timeout. This allows a job to be attempted any number of times within a given time frame. To define the time at which a job should timeout, add a `retryUntil` method to your job class:

```
/**
 * Determine the time at which the job should timeout.
 *
 * @return \DateTime
 */
public function retryUntil()
{
    return now()->addSeconds(5);
}
```

TIP You may also define a `retryUntil` method on your queued event listeners.

Max Exceptions

Sometimes you may wish to specify that a job may be attempted many times, but should fail if the retries are triggered by a given number of exceptions. To accomplish this, you may define a `maxExceptions` property on your job class:

```
<?php

namespace App\Jobs;

class ProcessPodcast implements ShouldQueue
{
    /**
     * The number of times the job may be attempted.
     *
     * @var int
     */
    public $tries = 25;

    /**
     * The maximum number of exceptions to allow before failing.
     *
     * @var int
     */
    public $maxExceptions = 3;

    /**
     * Execute the job.
     *
     * @return void
     */
    public function handle()
    {
        Redis::throttle('key')->allow(10)->every(60)->then(function () {
            // Lock obtained, process the podcast...
        }, function () {
            // Unable to obtain lock...
            return $this->release(10);
        });
    }
}
```

```
}  
}
```

In this example, the job is released for ten seconds if the application is unable to obtain a Redis lock and will continue to be retried up to 25 times. However, the job will fail if three unhandled exceptions are thrown by the job.

Timeout

NOTE The `pcntl` PHP extension must be installed in order to specify job timeouts.

Likewise, the maximum number of seconds that jobs can run may be specified using the `--timeout` switch on the Artisan command line:

```
php artisan queue:work --timeout=30
```

However, you may also define the maximum number of seconds a job should be allowed to run on the job class itself. If the timeout is specified on the job, it will take precedence over any timeout specified on the command line:

```
<?php  
  
namespace App\Jobs;  
  
class ProcessPodcast implements ShouldQueue  
{  
    /**  
     * The number of seconds the job can run before timing out.  
     */  
    * @var int  
    */  
    public $timeout = 120;  
}
```

Sometimes, IO blocking processes such as sockets or outgoing HTTP connections may not respect your specified timeout. Therefore, when using these features, you should always attempt to specify a timeout using their APIs as well. For example, when using Guzzle, you should always specify a connection and request timeout value.

Rate Limiting

NOTE This feature requires that your application can interact with a [Redis server](#).

If your application interacts with Redis, you may throttle your queued jobs by time or concurrency. This feature can be of assistance when your queued jobs are interacting with APIs that are also rate limited.

For example, using the `throttle` method, you may throttle a given type of job to only run 10 times every 60 seconds. If a lock can not be obtained, you should typically release the job back onto the queue so it can be retried later:

```
Redis::throttle('key')->allow(10)->every(60)->then(function () {  
    // Job logic...  
}, function () {  
    // Could not obtain lock...  
  
    return $this->release(10);  
});
```

TIP In the example above, the key may be any string that uniquely identifies the type of job you would like to rate limit. For example, you may wish to construct the key based on the class name of the job and the IDs of the Eloquent models it operates on.

NOTE Releasing a throttled job back onto the queue will still increment the job's total number of attempts.

Alternatively, you may specify the maximum number of workers that may simultaneously process a given job. This can be helpful when a queued job is modifying a resource that should only be modified by one job at a time. For example, using the `funnel` method, you may limit jobs of a given type to only be processed by one

worker at a time:

```
Redis::funnel('key')->limit(1)->then(function () {
    // Job logic...
}, function () {
    // Could not obtain lock...

    return $this->release(10);
});
```

TIP When using rate limiting, the number of attempts your job will need to run successfully can be hard to determine. Therefore, it is useful to combine rate limiting with [time based attempts](#).

Error Handling

If an exception is thrown while the job is being processed, the job will automatically be released back onto the queue so it may be attempted again. The job will continue to be released until it has been attempted the maximum number of times allowed by your application. The maximum number of attempts is defined by the `--tries` switch used on the `queue:work` Artisan command. Alternatively, the maximum number of attempts may be defined on the job class itself. More information on running the queue worker [can be found below](#).

Queueing Closures

Instead of dispatching a job class to the queue, you may also dispatch a Closure. This is great for quick, simple tasks that need to be executed outside of the current request cycle:

```
$podcast = App\Podcast::find(1);

dispatch(function () use ($podcast) {
    $podcast->publish();
});
```

When dispatching Closures to the queue, the Closure's code contents is cryptographically signed so it can not be modified in transit.

Running The Queue Worker

Laravel includes a queue worker that will process new jobs as they are pushed onto the queue. You may run the worker using the `queue:work` Artisan command. Note that once the `queue:work` command has started, it will continue to run until it is manually stopped or you close your terminal:

```
php artisan queue:work
```

TIP To keep the `queue:work` process running permanently in the background, you should use a process monitor such as [Supervisor](#) to ensure that the queue worker does not stop running.

Remember, queue workers are long-lived processes and store the booted application state in memory. As a result, they will not notice changes in your code base after they have been started. So, during your deployment process, be sure to [restart your queue workers](#). In addition, remember that any static state created or modified by your application will not be automatically reset between jobs.

Alternatively, you may run the `queue:listen` command. When using the `queue:listen` command, you don't have to manually restart the worker when you want to reload your updated code or reset the application state; however, this command is not as efficient as `queue:work`:

```
php artisan queue:listen
```

Specifying The Connection & Queue

You may also specify which queue connection the worker should utilize. The connection name passed to the `work` command should correspond to one of the connections defined in your `config/queue.php` configuration file:

```
php artisan queue:work redis
```


You may customize your queue worker even further by only processing particular queues for a given connection. For example, if all of your emails are processed in an `emails` queue on your `redis` queue connection, you may issue the following command to start a worker that only processes that queue:

```
php artisan queue:work redis --queue=emails
```

Processing A Single Job

The `--once` option may be used to instruct the worker to only process a single job from the queue:

```
php artisan queue:work --once
```

Processing All Queued Jobs & Then Exiting

The `--stop-when-empty` option may be used to instruct the worker to process all jobs and then exit gracefully. This option can be useful when working Laravel queues within a Docker container if you wish to shutdown the container after the queue is empty:

```
php artisan queue:work --stop-when-empty
```

Resource Considerations

Daemon queue workers do not "reboot" the framework before processing each job. Therefore, you should free any heavy resources after each job completes. For example, if you are doing image manipulation with the GD library, you should free the memory with `imagedestroy` when you are done.

Queue Priorities

Sometimes you may wish to prioritize how your queues are processed. For example, in your `config/queue.php` you may set the default queue for your `redis` connection to `low`. However, occasionally you may wish to push a job to a high priority queue like so:

```
dispatch((new Job)->onQueue('high'));
```

To start a worker that verifies that all of the high queue jobs are processed before continuing to any jobs on the low queue, pass a comma-delimited list of queue names to the `work` command:

```
php artisan queue:work --queue=high,low
```

Queue Workers & Deployment

Since queue workers are long-lived processes, they will not pick up changes to your code without being restarted. So, the simplest way to deploy an application using queue workers is to restart the workers during your deployment process. You may gracefully restart all of the workers by issuing the `queue:restart` command:

```
php artisan queue:restart
```

This command will instruct all queue workers to gracefully "die" after they finish processing their current job so that no existing jobs are lost. Since the queue workers will die when the `queue:restart` command is executed, you should be running a process manager such as [Supervisor](#) to automatically restart the queue workers.

TIP The queue uses the [cache](#) to store restart signals, so you should verify a cache driver is properly configured for your application before using this feature.

Job Expirations & Timeouts

Job Expiration

In your `config/queue.php` configuration file, each queue connection defines a `retry_after` option. This option specifies how many seconds the queue connection should wait before retrying a job that is being processed. For

example, if the value of `retry_after` is set to 90, the job will be released back onto the queue if it has been processing for 90 seconds without being deleted. Typically, you should set the `retry_after` value to the maximum number of seconds your jobs should reasonably take to complete processing.

NOTE The only queue connection which does not contain a `retry_after` value is Amazon SQS. SQS will retry the job based on the [Default Visibility Timeout](#) which is managed within the AWS console.

Worker Timeouts

The `queue:work` Artisan command exposes a `--timeout` option. The `--timeout` option specifies how long the Laravel queue master process will wait before killing off a child queue worker that is processing a job. Sometimes a child queue process can become "frozen" for various reasons. The `--timeout` option removes frozen processes that have exceeded that specified time limit:

```
php artisan queue:work --timeout=60
```

The `retry_after` configuration option and the `--timeout` CLI option are different, but work together to ensure that jobs are not lost and that jobs are only successfully processed once.

NOTE The `--timeout` value should always be at least several seconds shorter than your `retry_after` configuration value. This will ensure that a worker processing a given job is always killed before the job is retried. If your `--timeout` option is longer than your `retry_after` configuration value, your jobs may be processed twice.

Worker Sleep Duration

When jobs are available on the queue, the worker will keep processing jobs with no delay in between them. However, the `sleep` option determines how long (in seconds) the worker will "sleep" if there are no new jobs available. While sleeping, the worker will not process any new jobs - the jobs will be processed after the worker wakes up again.

```
php artisan queue:work --sleep=3
```

Supervisor Configuration

Installing Supervisor

Supervisor is a process monitor for the Linux operating system, and will automatically restart your `queue:work` process if it fails. To install Supervisor on Ubuntu, you may use the following command:

```
sudo apt-get install supervisor
```

TIP If configuring Supervisor yourself sounds overwhelming, consider using [Laravel Forge](#), which will automatically install and configure Supervisor for your Laravel projects.

Configuring Supervisor

Supervisor configuration files are typically stored in the `/etc/supervisor/conf.d` directory. Within this directory, you may create any number of configuration files that instruct supervisor how your processes should be monitored. For example, let's create a `laravel-worker.conf` file that starts and monitors a `queue:work` process:

```
[program:laravel-worker]
process_name=%(program_name)s_%(process_num)02d
command=php /home/forge/app.com/artisan queue:work sqs --sleep=3 --tries=3
autostart=true
autorestart=true
user=forge
numprocs=8
redirect_stderr=true
stdout_logfile=/home/forge/app.com/worker.log
stopwaitsecs=3600
```

In this example, the `numprocs` directive will instruct Supervisor to run 8 `queue:work` processes and monitor all of

them, automatically restarting them if they fail. You should change the `queue:work sqs` portion of the command directive to reflect your desired queue connection.

NOTE You should ensure that the value of `stopwaitsecs` is greater than the number of seconds consumed by your longest running job. Otherwise, Supervisor may kill the job before it is finished processing.

Starting Supervisor

Once the configuration file has been created, you may update the Supervisor configuration and start the processes using the following commands:

```
sudo supervisorctl reread
sudo supervisorctl update
sudo supervisorctl start laravel-worker:*
```

For more information on Supervisor, consult the [Supervisor documentation](#).

Dealing With Failed Jobs

Sometimes your queued jobs will fail. Don't worry, things don't always go as planned! Laravel includes a convenient way to specify the maximum number of times a job should be attempted. After a job has exceeded this amount of attempts, it will be inserted into the `failed_jobs` database table. To create a migration for the `failed_jobs` table, you may use the `queue:failed-table` command:

```
php artisan queue:failed-table
php artisan migrate
```

Then, when running your [queue worker](#), you can specify the maximum number of times a job should be attempted using the `--tries` switch on the `queue:work` command. If you do not specify a value for the `--tries` option, jobs will only be attempted once:

```
php artisan queue:work redis --tries=3
```

In addition, you may specify how many seconds Laravel should wait before retrying a job that has failed using the `--delay` option. By default, a job is retried immediately:

```
php artisan queue:work redis --tries=3 --delay=3
```

If you would like to configure the failed job retry delay on a per-job basis, you may do so by defining a `retryAfter` property on your queued job class:

```
/**
 * The number of seconds to wait before retrying the job.
 *
 * @var int
 */
public $retryAfter = 3;
```

If you require more complex logic for determining the retry delay, you may define a `retryAfter` method on your queued job class:

```
/**
 * Calculate the number of seconds to wait before retrying the job.
 *
 * @return int
 */
public function retryAfter()
{
    return 3;
}
```

Cleaning Up After Failed Jobs

You may define a `failed` method directly on your job class, allowing you to perform job specific clean-up when

a failure occurs. This is the perfect location to send an alert to your users or revert any actions performed by the job. The `Throwable` exception that caused the job to fail will be passed to the `failed` method:

```
<?php

namespace App\Jobs;

use App\AudioProcessor;
use App\Podcast;
use Throwable;
use Illuminate\Bus\Queueable;
use Illuminate\Contracts\Queue\ShouldQueue;
use Illuminate\Queue\InteractsWithQueue;
use Illuminate\Queue\SerializesModels;

class ProcessPodcast implements ShouldQueue
{
    use InteractsWithQueue, Queueable, SerializesModels;

    protected $podcast;

    /**
     * Create a new job instance.
     *
     * @param \App\Podcast $podcast
     * @return void
     */
    public function __construct(Podcast $podcast)
    {
        $this->podcast = $podcast;
    }

    /**
     * Execute the job.
     *
     * @param \App\AudioProcessor $processor
     * @return void
     */
    public function handle(AudioProcessor $processor)
    {
        // Process uploaded podcast...
    }

    /**
     * Handle a job failure.
     *
     * @param \Throwable $exception
     * @return void
     */
    public function failed(Throwable $exception)
    {
        // Send user notification of failure, etc...
    }
}
```

NOTE The `failed` method will not be called if the job was dispatched using the `dispatchNow` method.

Failed Job Events

If you would like to register an event that will be called when a job fails, you may use the `Queue::failing` method. This event is a great opportunity to notify your team via email or [Slack](#). For example, we may attach a callback to this event from the `AppServiceProvider` that is included with Laravel:

```
<?php

namespace App\Providers;

use Illuminate\Support\Facades\Queue;
use Illuminate\Support\ServiceProvider;
use Illuminate\Queue\Events\JobFailed;

class AppServiceProvider extends ServiceProvider
{
    /**
     * Register any application services.
     *
     */
}
```

```
        * @return void
        */
    public function register()
    {
        //
    }

    /**
     * Bootstrap any application services.
     *
     * @return void
     */
    public function boot()
    {
        Queue::failing(function (JobFailed $event) {
            // $event->connectionName
            // $event->job
            // $event->exception
        });
    }
}
```

Retrying Failed Jobs

To view all of your failed jobs that have been inserted into your `failed_jobs` database table, you may use the `queue:failed` Artisan command:

```
php artisan queue:failed
```

The `queue:failed` command will list the job ID, connection, queue, failure time, and other information about the job. The job ID may be used to retry the failed job. For instance, to retry a failed job that has an ID of 5, issue the following command:

```
php artisan queue:retry 5
```

If necessary, you may pass multiple IDs or an ID range (when using numeric IDs) to the command:

```
php artisan queue:retry 5 6 7 8 9 10
```

```
php artisan queue:retry --range=5-10
```

To retry all of your failed jobs, execute the `queue:retry` command and pass `all` as the ID:

```
php artisan queue:retry all
```

If you would like to delete a failed job, you may use the `queue:forget` command:

```
php artisan queue:forget 5
```

To delete all of your failed jobs, you may use the `queue:flush` command:

```
php artisan queue:flush
```

Ignoring Missing Models

When injecting an Eloquent model into a job, it is automatically serialized before being placed on the queue and restored when the job is processed. However, if the model has been deleted while the job was waiting to be processed by a worker, your job may fail with a `ModelNotFoundException`.

For convenience, you may choose to automatically delete jobs with missing models by setting your job's `deleteWhenMissingModels` property to `true`:

```
/**
 * Delete the job if its models no longer exist.
 *
 * @var bool
 */
public $deleteWhenMissingModels = true;
```

Job Events

Using the `before` and `after` methods on the queue [facade](#), you may specify callbacks to be executed before or after a queued job is processed. These callbacks are a great opportunity to perform additional logging or increment statistics for a dashboard. Typically, you should call these methods from a [service provider](#). For example, we may use the `AppServiceProvider` that is included with Laravel:

```
<?php

namespace App\Providers;

use Illuminate\Support\Facades\Queue;
use Illuminate\Support\ServiceProvider;
use Illuminate\Queue\Events\JobProcessed;
use Illuminate\Queue\Events\JobProcessing;

class AppServiceProvider extends ServiceProvider
{
    /**
     * Register any application services.
     *
     * @return void
     */
    public function register()
    {
        //
    }

    /**
     * Bootstrap any application services.
     *
     * @return void
     */
    public function boot()
    {
        Queue::before(function (JobProcessing $event) {
            // $event->connectionName
            // $event->job
            // $event->job->payload()
        });

        Queue::after(function (JobProcessed $event) {
            // $event->connectionName
            // $event->job
            // $event->job->payload()
        });
    }
}
```

Using the `looping` method on the queue [facade](#), you may specify callbacks that execute before the worker attempts to fetch a job from a queue. For example, you might register a Closure to rollback any transactions that were left open by a previously failed job:

```
Queue::looping(function () {
    while (DB::transactionLevel() > 0) {
        DB::rollBack();
    }
});
```

Digging Deeper

Task Scheduling

Task Scheduling

- [Introduction](#)
- [Defining Schedules](#)
 - [Scheduling Artisan Commands](#)
 - [Scheduling Queued Jobs](#)
 - [Scheduling Shell Commands](#)
 - [Schedule Frequency Options](#)
 - [Timezones](#)
 - [Preventing Task Overlaps](#)
 - [Running Tasks On One Server](#)
 - [Background Tasks](#)
 - [Maintenance Mode](#)
- [Task Output](#)
- [Task Hooks](#)

Introduction

In the past, you may have generated a Cron entry for each task you needed to schedule on your server. However, this can quickly become a pain, because your task schedule is no longer in source control and you must SSH into your server to add additional Cron entries.

Laravel's command scheduler allows you to fluently and expressively define your command schedule within Laravel itself. When using the scheduler, only a single Cron entry is needed on your server. Your task schedule is defined in the `app/Console/Kernel.php` file's `schedule` method. To help you get started, a simple example is defined within the method.

Starting The Scheduler

When using the scheduler, you only need to add the following Cron entry to your server. If you do not know how to add Cron entries to your server, consider using a service such as [Laravel Forge](#) which can manage the Cron entries for you:

```
* * * * * cd /path-to-your-project && php artisan schedule:run >> /dev/null 2>&1
```

This Cron will call the Laravel command scheduler every minute. When the `schedule:run` command is executed, Laravel will evaluate your scheduled tasks and runs the tasks that are due.

Defining Schedules

You may define all of your scheduled tasks in the `schedule` method of the `App\Console\Kernel` class. To get started, let's look at an example of scheduling a task. In this example, we will schedule a `closure` to be called every day at midnight. Within the `closure` we will execute a database query to clear a table:

```
<?php

namespace App\Console;

use Illuminate\Console\Scheduling\Schedule;
use Illuminate\Foundation\Console\Kernel as ConsoleKernel;
use Illuminate\Support\Facades\DB;

class Kernel extends ConsoleKernel
{
    /**
     * The Artisan commands provided by your application.
     *
     * @var array
     */
}
```

```

        */
        protected $commands = [
            //
        ];

        /**
         * Define the application's command schedule.
         *
         * @param \Illuminate\Console\Scheduling\Schedule $schedule
         * @return void
         */
        protected function schedule(Schedule $schedule)
        {
            $schedule->call(function () {
                DB::table('recent_users')->delete();
            })->daily();
        }
    }
}

```

In addition to scheduling using Closures, you may also use [invokable objects](#). Invokable objects are simple PHP classes that contain an `__invoke` method:

```
$schedule->call(new DeleteRecentUsers)->daily();
```

Scheduling Artisan Commands

In addition to scheduling Closure calls, you may also schedule [Artisan commands](#) and operating system commands. For example, you may use the `command` method to schedule an Artisan command using either the command's name or class:

```

$schedule->command('emails:send Taylor --force')->daily();

$schedule->command(EmailsCommand::class, ['Taylor', '--force'])->daily();

```

Scheduling Queued Jobs

The `job` method may be used to schedule a [queued job](#). This method provides a convenient way to schedule jobs without using the `call` method to manually create Closures to queue the job:

```

$schedule->job(new Heartbeat)->everyFiveMinutes();

// Dispatch the job to the "heartbeats" queue...
$schedule->job(new Heartbeat, 'heartbeats')->everyFiveMinutes();

```

Scheduling Shell Commands

The `exec` method may be used to issue a command to the operating system:

```
$schedule->exec('node /home/forged/script.js')->daily();
```

Schedule Frequency Options

There are a variety of schedules you may assign to your task:

Method	Description
<code>->cron('* * * * *');</code>	Run the task on a custom Cron schedule
<code>->everyMinute();</code>	Run the task every minute
<code>->everyTwoMinutes();</code>	Run the task every two minutes
<code>->everyThreeMinutes();</code>	Run the task every three minutes
<code>->everyFourMinutes();</code>	Run the task every four minutes
<code>->everyFiveMinutes();</code>	Run the task every five minutes
<code>->everyTenMinutes();</code>	Run the task every ten minutes
<code>->everyFifteenMinutes();</code>	Run the task every fifteen minutes
<code>->everyThirtyMinutes();</code>	Run the task every thirty minutes
<code>->hourly();</code>	Run the task every hour

->hourlyAt(17);	Run the task every hour at 17 minutes past the hour
->everyTwoHours();	Run the task every two hours
->everyThreeHours();	Run the task every three hours
->everyFourHours();	Run the task every four hours
->everySixHours();	Run the task every six hours
->daily();	Run the task every day at midnight
->dailyAt('13:00');	Run the task every day at 13:00
->twiceDaily(1, 13);	Run the task daily at 1:00 & 13:00
->weekly();	Run the task every sunday at 00:00
->weeklyOn(1, '8:00');	Run the task every week on Monday at 8:00
->monthly();	Run the task on the first day of every month at 00:00
->monthlyOn(4, '15:00');	Run the task every month on the 4th at 15:00
->lastDayOfMonth('15:00');	Run the task on the last day of the month at 15:00
->quarterly();	Run the task on the first day of every quarter at 00:00
->yearly();	Run the task on the first day of every year at 00:00
->timezone('America/New_York');	Set the timezone

These methods may be combined with additional constraints to create even more finely tuned schedules that only run on certain days of the week. For example, to schedule a command to run weekly on Monday:

```
// Run once per week on Monday at 1 PM...
$schedule->call(function () {
    //
})->weekly()->mondays()->at('13:00');

// Run hourly from 8 AM to 5 PM on weekdays...
$schedule->command('foo')
    ->weekdays()
    ->hourly()
    ->timezone('America/Chicago')
    ->between('8:00', '17:00');
```

Below is a list of the additional schedule constraints:

Method	Description
->weekdays();	Limit the task to weekdays
->weekends();	Limit the task to weekends
->sundays();	Limit the task to Sunday
->mondays();	Limit the task to Monday
->tuesdays();	Limit the task to Tuesday
->>wednesdays();	Limit the task to Wednesday
->thursdays();	Limit the task to Thursday
->fridays();	Limit the task to Friday
->saturdays();	Limit the task to Saturday
->days(array mixed);	Limit the task to specific days
->between(\$start, \$end);	Limit the task to run between start and end times
->when(Closure);	Limit the task based on a truth test
->environments(\$env);	Limit the task to specific environments

Day Constraints

The `days` method may be used to limit the execution of a task to specific days of the week. For example, you may schedule a command to run hourly on Sundays and Wednesdays:

```
$schedule->command('reminders:send')
    ->hourly()
    ->days([0, 3]);
```

Between Time Constraints

The `between` method may be used to limit the execution of a task based on the time of day:

```
$schedule->command('reminders:send')
    ->hourly()
    ->between('7:00', '22:00');
```

Similarly, the `unlessBetween` method can be used to exclude the execution of a task for a period of time:

```
$schedule->command('reminders:send')
    ->hourly()
    ->unlessBetween('23:00', '4:00');
```

Truth Test Constraints

The `when` method may be used to limit the execution of a task based on the result of a given truth test. In other words, if the given closure returns `true`, the task will execute as long as no other constraining conditions prevent the task from running:

```
$schedule->command('emails:send')->daily()->when(function () {
    return true;
});
```

The `skip` method may be seen as the inverse of `when`. If the `skip` method returns `true`, the scheduled task will not be executed:

```
$schedule->command('emails:send')->daily()->skip(function () {
    return true;
});
```

When using chained `when` methods, the scheduled command will only execute if all `when` conditions return `true`.

Environment Constraints

The `environments` method may be used to execute tasks only on the given environments:

```
$schedule->command('emails:send')
    ->daily()
    ->environments(['staging', 'production']);
```

Timezones

Using the `timezone` method, you may specify that a scheduled task's time should be interpreted within a given timezone:

```
$schedule->command('report:generate')
    ->timezone('America/New_York')
    ->at('02:00')
```

If you are assigning the same timezone to all of your scheduled tasks, you may wish to define a `scheduleTimezone` method in your `app/Console/Kernel.php` file. This method should return the default timezone that should be assigned to all scheduled tasks:

```
/**
 * Get the timezone that should be used by default for scheduled events.
 *
 * @return \DateTimeZone|string|null
 */
protected function scheduleTimezone()
{
    return 'America/Chicago';
}
```

NOTE Remember that some timezones utilize daylight savings time. When daylight saving time changes occur, your scheduled task may run twice or even not run at all. For this reason, we recommend avoiding timezone scheduling when possible.

Preventing Task Overlaps

By default, scheduled tasks will be run even if the previous instance of the task is still running. To prevent this, you may use the `withoutOverlapping` method:

```
$schedule->command('emails:send')->withoutOverlapping();
```

In this example, the `emails:send` [Artisan command](#) will be run every minute if it is not already running. The `withoutOverlapping` method is especially useful if you have tasks that vary drastically in their execution time, preventing you from predicting exactly how long a given task will take.

If needed, you may specify how many minutes must pass before the "without overlapping" lock expires. By default, the lock will expire after 24 hours:

```
$schedule->command('emails:send')->withoutOverlapping(10);
```

Running Tasks On One Server

NOTE To utilize this feature, your application must be using the database, memcached, or redis cache driver as your application's default cache driver. In addition, all servers must be communicating with the same central cache server.

If your application is running on multiple servers, you may limit a scheduled job to only execute on a single server. For instance, assume you have a scheduled task that generates a new report every Friday night. If the task scheduler is running on three worker servers, the scheduled task will run on all three servers and generate the report three times. Not good!

To indicate that the task should run on only one server, use the `onOneServer` method when defining the scheduled task. The first server to obtain the task will secure an atomic lock on the job to prevent other servers from running the same task at the same time:

```
$schedule->command('report:generate')
    ->fridays()
    ->at('17:00')
    ->onOneServer();
```

Background Tasks

By default, multiple commands scheduled at the same time will execute sequentially. If you have long-running commands, this may cause subsequent commands to start much later than anticipated. If you would like to run commands in the background so that they may all run simultaneously, you may use the `runInBackground` method:

```
$schedule->command('analytics:report')
    ->daily()
    ->runInBackground();
```

NOTE The `runInBackground` method may only be used when scheduling tasks via the `command` and `exec` methods.

Maintenance Mode

Laravel's scheduled tasks will not run when Laravel is in [maintenance mode](#), since we don't want your tasks to interfere with any unfinished maintenance you may be performing on your server. However, if you would like to force a task to run even in maintenance mode, you may use the `evenInMaintenanceMode` method:

```
$schedule->command('emails:send')->evenInMaintenanceMode();
```

Task Output

The Laravel scheduler provides several convenient methods for working with the output generated by scheduled tasks. First, using the `sendOutputTo` method, you may send the output to a file for later inspection:

```
$schedule->command('emails:send')
    ->daily()
```

```
->sendOutputTo($filePath);
```

If you would like to append the output to a given file, you may use the `appendOutputTo` method:

```
$schedule->command('emails:send')
    ->daily()
    ->appendOutputTo($filePath);
```

Using the `emailOutputTo` method, you may e-mail the output to an e-mail address of your choice. Before e-mailing the output of a task, you should configure Laravel's [e-mail services](#):

```
$schedule->command('foo')
    ->daily()
    ->sendOutputTo($filePath)
    ->emailOutputTo('foo@example.com');
```

If you only want to e-mail the output if the command fails, use the `emailOutputOnFailure` method:

```
$schedule->command('foo')
    ->daily()
    ->emailOutputOnFailure('foo@example.com');
```

NOTE The `emailOutputTo`, `emailOutputOnFailure`, `sendOutputTo`, and `appendOutputTo` methods are exclusive to the `command` and `exec` methods.

Task Hooks

Using the `before` and `after` methods, you may specify code to be executed before and after the scheduled task is complete:

```
$schedule->command('emails:send')
    ->daily()
    ->before(function () {
        // Task is about to start...
    })
    ->after(function () {
        // Task is complete...
    });
```

The `onSuccess` and `onFailure` methods allow you to specify code to be executed if the scheduled task succeeds or fails:

```
$schedule->command('emails:send')
    ->daily()
    ->onSuccess(function () {
        // The task succeeded...
    })
    ->onFailure(function () {
        // The task failed...
    });
```

Pinging URLs

Using the `pingBefore` and `thenPing` methods, the scheduler can automatically ping a given URL before or after a task is complete. This method is useful for notifying an external service, such as [Laravel Envoyer](#), that your scheduled task is commencing or has finished execution:

```
$schedule->command('emails:send')
    ->daily()
    ->pingBefore($url)
    ->thenPing($url);
```

The `pingBeforeIf` and `thenPingIf` methods may be used to ping a given URL only if the given condition is true:

```
$schedule->command('emails:send')
    ->daily()
    ->pingBeforeIf($condition, $url)
    ->thenPingIf($condition, $url);
```

The `pingOnSuccess` and `pingOnFailure` methods may be used to ping a given URL only if the task succeeds or

fails:

```
$schedule->command('emails:send')  
    ->daily()  
    ->pingOnSuccess($successUrl)  
    ->pingOnFailure($failureUrl);
```

All of the ping methods require the Guzzle HTTP library. You can add Guzzle to your project using the Composer package manager:

```
composer require guzzlehttp/guzzle
```

Database

Getting Started

Database: Getting Started

- [Introduction](#)
 - [Configuration](#)
 - [Read & Write Connections](#)
 - [Using Multiple Database Connections](#)
- [Running Raw SQL Queries](#)
- [Listening For Query Events](#)
- [Database Transactions](#)

Introduction

Laravel makes interacting with databases extremely simple across a variety of database backends using either raw SQL, the [fluent query builder](#), and the [Eloquent ORM](#). Currently, Laravel supports four databases:

- MySQL 5.6+ ([Version Policy](#))
- PostgreSQL 9.4+ ([Version Policy](#))
- SQLite 3.8.8+
- SQL Server 2017+ ([Version Policy](#))

Configuration

The database configuration for your application is located at `config/database.php`. In this file you may define all of your database connections, as well as specify which connection should be used by default. Examples for most of the supported database systems are provided in this file.

By default, Laravel's sample [environment configuration](#) is ready to use with [Laravel Homestead](#), which is a convenient virtual machine for doing Laravel development on your local machine. You are free to modify this configuration as needed for your local database.

SQLite Configuration

After creating a new SQLite database using a command such as `touch database/database.sqlite`, you can easily configure your environment variables to point to this newly created database by using the database's absolute path:

```
DB_CONNECTION=sqlite
DB_DATABASE=/absolute/path/to/database.sqlite
```

To enable foreign key constraints for SQLite connections, you should set the `DB_FOREIGN_KEYS` environment variable to `true`:

```
DB_FOREIGN_KEYS=true
```

Configuration Using URLs

Typically, database connections are configured using multiple configuration values such as `host`, `database`, `username`, `password`, etc. Each of these configuration values has its own corresponding environment variable. This means that when configuring your database connection information on a production server, you need to manage several environment variables.

Some managed database providers such as Heroku provide a single database "URL" that contains all of the connection information for the database in a single string. An example database URL may look something like the following:

```
mysql://root:password@127.0.0.1/forge?charset=UTF-8
```

These URLs typically follow a standard schema convention:

```
driver://username:password@host:port/database?options
```

For convenience, Laravel supports these URLs as an alternative to configuring your database with multiple configuration options. If the `url` (or corresponding `DATABASE_URL` environment variable) configuration option is present, it will be used to extract the database connection and credential information.

Read & Write Connections

Sometimes you may wish to use one database connection for `SELECT` statements, and another for `INSERT`, `UPDATE`, and `DELETE` statements. Laravel makes this a breeze, and the proper connections will always be used whether you are using raw queries, the query builder, or the Eloquent ORM.

To see how read / write connections should be configured, let's look at this example:

```
'mysql' => [
    'read' => [
        'host' => [
            '192.168.1.1',
            '196.168.1.2',
        ],
    ],
    'write' => [
        'host' => [
            '196.168.1.3',
        ],
    ],
    'sticky' => true,
    'driver' => 'mysql',
    'database' => 'database',
    'username' => 'root',
    'password' => '',
    'charset' => 'utf8mb4',
    'collation' => 'utf8mb4_unicode_ci',
    'prefix' => '',
],
```

Note that three keys have been added to the configuration array: `read`, `write` and `sticky`. The `read` and `write` keys have array values containing a single key: `host`. The rest of the database options for the `read` and `write` connections will be merged from the main `mysql` array.

You only need to place items in the `read` and `write` arrays if you wish to override the values from the main array. So, in this case, `192.168.1.1` will be used as the host for the "read" connection, while `192.168.1.3` will be used for the "write" connection. The database credentials, prefix, character set, and all other options in the main `mysql` array will be shared across both connections.

The sticky Option

The `sticky` option is an *optional* value that can be used to allow the immediate reading of records that have been written to the database during the current request cycle. If the `sticky` option is enabled and a "write" operation has been performed against the database during the current request cycle, any further "read" operations will use the "write" connection. This ensures that any data written during the request cycle can be immediately read back from the database during that same request. It is up to you to decide if this is the desired behavior for your application.

Using Multiple Database Connections

When using multiple connections, you may access each connection via the `connection` method on the `DB` facade. The name passed to the `connection` method should correspond to one of the connections listed in your `config/database.php` configuration file:

```
$users = DB::connection('foo')->select(...);
```

You may also access the raw, underlying PDO instance using the `getPdo` method on a connection instance:

```
$pdo = DB::connection()->getPdo();
```

Running Raw SQL Queries

Once you have configured your database connection, you may run queries using the `DB` facade. The `DB` facade provides methods for each type of query: `select`, `update`, `insert`, `delete`, and `statement`.

Running A Select Query

To run a basic query, you may use the `select` method on the `DB` facade:

```
<?php

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;
use Illuminate\Support\Facades\DB;

class UserController extends Controller
{
    /**
     * Show a list of all of the application's users.
     *
     * @return Response
     */
    public function index()
    {
        $users = DB::select('select * from users where active = ?', [1]);

        return view('user.index', ['users' => $users]);
    }
}
```

The first argument passed to the `select` method is the raw SQL query, while the second argument is any parameter bindings that need to be bound to the query. Typically, these are the values of the `where` clause constraints. Parameter binding provides protection against SQL injection.

The `select` method will always return an array of results. Each result within the array will be a PHP `stdClass` object, allowing you to access the values of the results:

```
foreach ($users as $user) {
    echo $user->name;
}
```

Using Named Bindings

Instead of using `?` to represent your parameter bindings, you may execute a query using named bindings:

```
$results = DB::select('select * from users where id = :id', ['id' => 1]);
```

Running An Insert Statement

To execute an `insert` statement, you may use the `insert` method on the `DB` facade. Like `select`, this method takes the raw SQL query as its first argument and bindings as its second argument:

```
DB::insert('insert into users (id, name) values (?, ?)', [1, 'Dayle']);
```

Running An Update Statement

The `update` method should be used to update existing records in the database. The number of rows affected by the statement will be returned:

```
$affected = DB::update('update users set votes = 100 where name = ?', ['John']);
```

Running A Delete Statement

The `delete` method should be used to delete records from the database. Like `update`, the number of rows affected will be returned:


```
$deleted = DB::delete('delete from users');
```

Running A General Statement

Some database statements do not return any value. For these types of operations, you may use the `statement` method on the `DB` facade:

```
DB::statement('drop table users');
```

Listening For Query Events

If you would like to receive each SQL query executed by your application, you may use the `listen` method. This method is useful for logging queries or debugging. You may register your query listener in a [service provider](#):

```
<?php

namespace App\Providers;

use Illuminate\Support\Facades\DB;
use Illuminate\Support\ServiceProvider;

class AppServiceProvider extends ServiceProvider
{
    /**
     * Register any application services.
     *
     * @return void
     */
    public function register()
    {
        //
    }

    /**
     * Bootstrap any application services.
     *
     * @return void
     */
    public function boot()
    {
        DB::listen(function ($query) {
            // $query->sql
            // $query->bindings
            // $query->time
        });
    }
}
```

Database Transactions

You may use the `transaction` method on the `DB` facade to run a set of operations within a database transaction. If an exception is thrown within the transaction closure, the transaction will automatically be rolled back. If the closure executes successfully, the transaction will automatically be committed. You don't need to worry about manually rolling back or committing while using the `transaction` method:

```
DB::transaction(function () {
    DB::table('users')->update(['votes' => 1]);

    DB::table('posts')->delete();
});
```

Handling Deadlocks

The `transaction` method accepts an optional second argument which defines the number of times a transaction should be reattempted when a deadlock occurs. Once these attempts have been exhausted, an exception will be thrown:

```
DB::transaction(function () {
    DB::table('users')->update(['votes' => 1]);
}, 5);
```

```
DB::table('posts')->delete();  
, 5);
```

Manually Using Transactions

If you would like to begin a transaction manually and have complete control over rollbacks and commits, you may use the `beginTransaction` method on the `DB` facade:

```
DB::beginTransaction();
```

You can rollback the transaction via the `rollBack` method:

```
DB::rollBack();
```

Lastly, you can commit a transaction via the `commit` method:

```
DB::commit();
```

TIP The `DB` facade's transaction methods control the transactions for both the [query builder](#) and [Eloquent ORM](#).

Database

Query Builder

Database: Query Builder

- [Introduction](#)
- [Retrieving Results](#)
 - [Chunking Results](#)
 - [Aggregates](#)
- [Selects](#)
- [Raw Expressions](#)
- [Joins](#)
- [Unions](#)
- [Where Clauses](#)
 - [Parameter Grouping](#)
 - [Where Exists Clauses](#)
 - [Subquery Where Clauses](#)
 - [JSON Where Clauses](#)
- [Ordering, Grouping, Limit & Offset](#)
- [Conditional Clauses](#)
- [Inserts](#)
- [Updates](#)
 - [Updating JSON Columns](#)
 - [Increment & Decrement](#)
- [Deletes](#)
- [Pessimistic Locking](#)
- [Debugging](#)

Introduction

Laravel's database query builder provides a convenient, fluent interface to creating and running database queries. It can be used to perform most database operations in your application and works on all supported database systems.

The Laravel query builder uses PDO parameter binding to protect your application against SQL injection attacks. There is no need to clean strings being passed as bindings.

NOTE PDO does not support binding column names. Therefore, you should never allow user input to dictate the column names referenced by your queries, including "order by" columns, etc. If you must allow the user to select certain columns to query against, always validate the column names against a white-list of allowed columns.

Retrieving Results

Retrieving All Rows From A Table

You may use the `table` method on the `DB` facade to begin a query. The `table` method returns a fluent query builder instance for the given table, allowing you to chain more constraints onto the query and then finally get the results using the `get` method:

```
<?php

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;
use Illuminate\Support\Facades\DB;

class UserController extends Controller
{
    /**
```

```
* Show a list of all of the application's users.
*
* @return Response
*/
public function index()
{
    $users = DB::table('users')->get();

    return view('user.index', ['users' => $users]);
}
}
```

The `get` method returns an `Illuminate\Support\Collection` containing the results where each result is an instance of the PHP `stdClass` object. You may access each column's value by accessing the column as a property of the object:

```
foreach ($users as $user) {
    echo $user->name;
}
```

Retrieving A Single Row / Column From A Table

If you just need to retrieve a single row from the database table, you may use the `first` method. This method will return a single `stdClass` object:

```
$user = DB::table('users')->where('name', 'John')->first();

echo $user->name;
```

If you don't even need an entire row, you may extract a single value from a record using the `value` method. This method will return the value of the column directly:

```
$email = DB::table('users')->where('name', 'John')->value('email');
```

To retrieve a single row by its `id` column value, use the `find` method:

```
$user = DB::table('users')->find(3);
```

Retrieving A List Of Column Values

If you would like to retrieve a `Collection` containing the values of a single column, you may use the `pluck` method. In this example, we'll retrieve a `Collection` of role titles:

```
$titles = DB::table('roles')->pluck('title');

foreach ($titles as $title) {
    echo $title;
}
```

You may also specify a custom key column for the returned `Collection`:

```
$roles = DB::table('roles')->pluck('title', 'name');

foreach ($roles as $name => $title) {
    echo $title;
}
```

Chunking Results

If you need to work with thousands of database records, consider using the `chunk` method. This method retrieves a small chunk of the results at a time and feeds each chunk into a `closure` for processing. This method is very useful for writing [Artisan commands](#) that process thousands of records. For example, let's work with the entire `users` table in chunks of 100 records at a time:

```
DB::table('users')->orderBy('id')->chunk(100, function ($users) {
    foreach ($users as $user) {
        //
    }
});
```

You may stop further chunks from being processed by returning `false` from the closure:

```
DB::table('users')->orderBy('id')->chunk(100, function ($users) {  
    // Process the records...  
  
    return false;  
});
```

If you are updating database records while chunking results, your chunk results could change in unexpected ways. So, when updating records while chunking, it is always best to use the `chunkById` method instead. This method will automatically paginate the results based on the record's primary key:

```
DB::table('users')->where('active', false)  
->chunkById(100, function ($users) {  
    foreach ($users as $user) {  
        DB::table('users')  
            ->where('id', $user->id)  
            ->update(['active' => true]);  
    }  
});
```

NOTE When updating or deleting records inside the chunk callback, any changes to the primary key or foreign keys could affect the chunk query. This could potentially result in records not being included in the chunked results.

Aggregates

The query builder also provides a variety of aggregate methods such as `count`, `max`, `min`, `avg`, and `sum`. You may call any of these methods after constructing your query:

```
$users = DB::table('users')->count();  
$price = DB::table('orders')->max('price');
```

You may combine these methods with other clauses:

```
$price = DB::table('orders')  
    ->where('finalized', 1)  
    ->avg('price');
```

Determining If Records Exist

Instead of using the `count` method to determine if any records exist that match your query's constraints, you may use the `exists` and `doesntExist` methods:

```
return DB::table('orders')->where('finalized', 1)->exists();  
return DB::table('orders')->where('finalized', 1)->doesntExist();
```

Selects

Specifying A Select Clause

You may not always want to select all columns from a database table. Using the `select` method, you can specify a custom select clause for the query:

```
$users = DB::table('users')->select('name', 'email as user_email')->get();
```

The `distinct` method allows you to force the query to return distinct results:

```
$users = DB::table('users')->distinct()->get();
```

If you already have a query builder instance and you wish to add a column to its existing select clause, you may use the `addSelect` method:

```
$query = DB::table('users')->select('name');  
$users = $query->addSelect('age')->get();
```

Raw Expressions

Sometimes you may need to use a raw expression in a query. To create a raw expression, you may use the `DB::raw` method:

```
$users = DB::table('users')
    ->select(DB::raw('count(*) as user_count, status'))
    ->where('status', '<>', 1)
    ->groupBy('status')
    ->get();
```

NOTE Raw statements will be injected into the query as strings, so you should be extremely careful to not create SQL injection vulnerabilities.

Raw Methods

Instead of using `DB::raw`, you may also use the following methods to insert a raw expression into various parts of your query.

`selectRaw`

The `selectRaw` method can be used in place of `addSelect(DB::raw(...))`. This method accepts an optional array of bindings as its second argument:

```
$orders = DB::table('orders')
    ->selectRaw('price * ? as price_with_tax', [1.0825])
    ->get();
```

`whereRaw` / `orWhereRaw`

The `whereRaw` and `orWhereRaw` methods can be used to inject a raw where clause into your query. These methods accept an optional array of bindings as their second argument:

```
$orders = DB::table('orders')
    ->whereRaw('price > IF(state = "TX", ?, 100)', [200])
    ->get();
```

`havingRaw` / `orHavingRaw`

The `havingRaw` and `orHavingRaw` methods may be used to set a raw string as the value of the having clause. These methods accept an optional array of bindings as their second argument:

```
$orders = DB::table('orders')
    ->select('department', DB::raw('SUM(price) as total_sales'))
    ->groupBy('department')
    ->havingRaw('SUM(price) > ?', [2500])
    ->get();
```

`orderByRaw`

The `orderByRaw` method may be used to set a raw string as the value of the order by clause:

```
$orders = DB::table('orders')
    ->orderByRaw('updated_at - created_at DESC')
    ->get();
```

`groupByRaw`

The `groupByRaw` method may be used to set a raw string as the value of the group by clause:

```
$orders = DB::table('orders')
    ->select('city', 'state')
    ->groupByRaw('city, state')
    ->get();
```

Joins

Inner Join Clause

The query builder may also be used to write join statements. To perform a basic "inner join", you may use the `join` method on a query builder instance. The first argument passed to the `join` method is the name of the table you need to join to, while the remaining arguments specify the column constraints for the join. You can even join to multiple tables in a single query:

```
$users = DB::table('users')
    ->join('contacts', 'users.id', '=', 'contacts.user_id')
    ->join('orders', 'users.id', '=', 'orders.user_id')
    ->select('users.*', 'contacts.phone', 'orders.price')
    ->get();
```

Left Join / Right Join Clause

If you would like to perform a "left join" or "right join" instead of an "inner join", use the `leftJoin` or `rightJoin` methods. These methods have the same signature as the `join` method:

```
$users = DB::table('users')
    ->leftJoin('posts', 'users.id', '=', 'posts.user_id')
    ->get();

$users = DB::table('users')
    ->rightJoin('posts', 'users.id', '=', 'posts.user_id')
    ->get();
```

Cross Join Clause

To perform a "cross join" use the `crossJoin` method with the name of the table you wish to cross join to. Cross joins generate a cartesian product between the first table and the joined table:

```
$sizes = DB::table('sizes')
    ->crossJoin('colors')
    ->get();
```

Advanced Join Clauses

You may also specify more advanced join clauses. To get started, pass a closure as the second argument into the `join` method. The closure will receive a `JoinClause` object which allows you to specify constraints on the join clause:

```
DB::table('users')
    ->join('contacts', function ($join) {
        $join->on('users.id', '=', 'contacts.user_id')->orOn(...);
    })
    ->get();
```

If you would like to use a "where" style clause on your joins, you may use the `where` and `orWhere` methods on a join. Instead of comparing two columns, these methods will compare the column against a value:

```
DB::table('users')
    ->join('contacts', function ($join) {
        $join->on('users.id', '=', 'contacts.user_id')
            ->where('contacts.user_id', '>', 5);
    })
    ->get();
```

Subquery Joins

You may use the `joinSub`, `leftJoinSub`, and `rightJoinSub` methods to join a query to a subquery. Each of these methods receive three arguments: the subquery, its table alias, and a Closure that defines the related columns:

```
$latestPosts = DB::table('posts')
    ->select('user_id', DB::raw('MAX(created_at) as last_post_created_at'))
    ->where('is_published', true)
    ->groupBy('user_id');

$users = DB::table('users')
```

```
->joinSub($latestPosts, 'latest_posts', function ($join) {  
    $join->on('users.id', '=', 'latest_posts.user_id');  
})->get();
```

Unions

The query builder also provides a quick way to "union" two queries together. For example, you may create an initial query and use the `union` method to union it with a second query:

```
$first = DB::table('users')  
    ->whereNull('first_name');  
  
$users = DB::table('users')  
    ->whereNull('last_name')  
    ->union($first)  
    ->get();
```

TIP The `unionAll` method is also available and has the same method signature as `union`.

Where Clauses

Simple Where Clauses

You may use the `where` method on a query builder instance to add `where` clauses to the query. The most basic call to `where` requires three arguments. The first argument is the name of the column. The second argument is an operator, which can be any of the database's supported operators. Finally, the third argument is the value to evaluate against the column.

For example, here is a query that verifies the value of the "votes" column is equal to 100:

```
$users = DB::table('users')->where('votes', '=', 100)->get();
```

For convenience, if you want to verify that a column is equal to a given value, you may pass the value directly as the second argument to the `where` method:

```
$users = DB::table('users')->where('votes', 100)->get();
```

You may use a variety of other operators when writing a `where` clause:

```
$users = DB::table('users')  
    ->where('votes', '>=', 100)  
    ->get();  
  
$users = DB::table('users')  
    ->where('votes', '<>', 100)  
    ->get();  
  
$users = DB::table('users')  
    ->where('name', 'like', 'T%')  
    ->get();
```

You may also pass an array of conditions to the `where` function:

```
$users = DB::table('users')->where([  
    ['status', '=', '1'],  
    ['subscribed', '<>', '1'],  
])->get();
```

Or Statements

You may chain `where` constraints together as well as add `or` clauses to the query. The `orWhere` method accepts the same arguments as the `where` method:

```
$users = DB::table('users')  
    ->where('votes', '>', 100)  
    ->orWhere('name', 'John')  
    ->get();
```


If you need to group an "or" condition within parentheses, you may pass a Closure as the first argument to the `orWhere` method:

```
$users = DB::table('users')
    ->where('votes', '>', 100)
    ->orWhere(function($query) {
        $query->where('name', 'Abigail')
        ->where('votes', '>', 50);
    })
    ->get();

// SQL: select * from users where votes > 100 or (name = 'Abigail' and votes > 50)
```

Additional Where Clauses

whereBetween / orWhereBetween

The `whereBetween` method verifies that a column's value is between two values:

```
$users = DB::table('users')
    ->whereBetween('votes', [1, 100])
    ->get();
```

whereNotBetween / orWhereNotBetween

The `whereNotBetween` method verifies that a column's value lies outside of two values:

```
$users = DB::table('users')
    ->whereNotBetween('votes', [1, 100])
    ->get();
```

whereIn / whereNotIn / orWhereIn / orWhereNotIn

The `whereIn` method verifies that a given column's value is contained within the given array:

```
$users = DB::table('users')
    ->whereIn('id', [1, 2, 3])
    ->get();
```

The `whereNotIn` method verifies that the given column's value is **not** contained in the given array:

```
$users = DB::table('users')
    ->whereNotIn('id', [1, 2, 3])
    ->get();
```

NOTE If you are adding a huge array of integer bindings to your query, the `whereIntegerInRaw` OR `whereIntegerNotInRaw` methods may be used to greatly reduce your memory usage.

whereNull / whereNotNull / orWhereNull / orWhereNotNull

The `whereNull` method verifies that the value of the given column is `NULL`:

```
$users = DB::table('users')
    ->whereNull('updated_at')
    ->get();
```

The `whereNotNull` method verifies that the column's value is not `NULL`:

```
$users = DB::table('users')
    ->whereNotNull('updated_at')
    ->get();
```

whereDate / whereMonth / whereDay / whereYear / whereTime

The `whereDate` method may be used to compare a column's value against a date:

```
$users = DB::table('users')
    ->whereDate('created_at', '2016-12-31')
    ->get();
```

The `whereMonth` method may be used to compare a column's value against a specific month of a year:

```
$users = DB::table('users')
    ->whereMonth('created_at', '12')
    ->get();
```

The `whereDay` method may be used to compare a column's value against a specific day of a month:

```
$users = DB::table('users')
    ->whereDay('created_at', '31')
    ->get();
```

The `whereYear` method may be used to compare a column's value against a specific year:

```
$users = DB::table('users')
    ->whereYear('created_at', '2016')
    ->get();
```

The `whereTime` method may be used to compare a column's value against a specific time:

```
$users = DB::table('users')
    ->whereTime('created_at', '=', '11:20:45')
    ->get();
```

whereColumn / orWhereColumn

The `whereColumn` method may be used to verify that two columns are equal:

```
$users = DB::table('users')
    ->whereColumn('first_name', 'last_name')
    ->get();
```

You may also pass a comparison operator to the method:

```
$users = DB::table('users')
    ->whereColumn('updated_at', '>', 'created_at')
    ->get();
```

The `whereColumn` method can also be passed an array of multiple conditions. These conditions will be joined using the `and` operator:

```
$users = DB::table('users')
    ->whereColumn([
        ['first_name', '=', 'last_name'],
        ['updated_at', '>', 'created_at'],
    ])->get();
```

Parameter Grouping

Sometimes you may need to create more advanced where clauses such as "where exists" clauses or nested parameter groupings. The Laravel query builder can handle these as well. To get started, let's look at an example of grouping constraints within parenthesis:

```
$users = DB::table('users')
    ->where('name', '=', 'John')
    ->where(function ($query) {
        $query->where('votes', '>', 100)
            ->orWhere('title', '=', 'Admin');
    })
    ->get();
```

As you can see, passing a `closure` into the `where` method instructs the query builder to begin a constraint group. The `closure` will receive a query builder instance which you can use to set the constraints that should be contained within the parenthesis group. The example above will produce the following SQL:

```
select * from users where name = 'John' and (votes > 100 or title = 'Admin')
```

TIP You should always group `orWhere` calls in order to avoid unexpected behavior when global scopes are applied.

Where Exists Clauses

The `whereExists` method allows you to write `where exists` SQL clauses. The `whereExists` method accepts a closure argument, which will receive a query builder instance allowing you to define the query that should be placed inside of the "exists" clause:

```
$users = DB::table('users')
    ->whereExists(function ($query) {
        $query->select(DB::raw(1))
        ->from('orders')
        ->whereRaw('orders.user_id = users.id');
    })
    ->get();
```

The query above will produce the following SQL:

```
select * from users
where exists (
    select 1 from orders where orders.user_id = users.id
)
```

Subquery Where Clauses

Sometimes you may need to construct a where clause that compares the results of a subquery to a given value. You may accomplish this by passing a Closure and a value to the `where` method. For example, the following query will retrieve all users who have a recent "membership" of a given type;

```
use App\User;

$users = User::where(function ($query) {
    $query->select('type')
    ->from('membership')
    ->whereColumn('user_id', 'users.id')
    ->orderByDesc('start_date')
    ->limit(1);
}, 'Pro')->get();
```

JSON Where Clauses

Laravel also supports querying JSON column types on databases that provide support for JSON column types. Currently, this includes MySQL 5.7, PostgreSQL, SQL Server 2016, and SQLite 3.9.0 (with the [JSON1 extension](#)). To query a JSON column, use the `->operator`:

```
$users = DB::table('users')
    ->where('options->language', 'en')
    ->get();

$users = DB::table('users')
    ->where('preferences->dining->meal', 'salad')
    ->get();
```

You may use `whereJsonContains` to query JSON arrays (not supported on SQLite):

```
$users = DB::table('users')
    ->whereJsonContains('options->languages', 'en')
    ->get();
```

MySQL and PostgreSQL support `whereJsonContains` with multiple values:

```
$users = DB::table('users')
    ->whereJsonContains('options->languages', ['en', 'de'])
    ->get();
```

You may use `whereJsonLength` to query JSON arrays by their length:

```
$users = DB::table('users')
    ->whereJsonLength('options->languages', 0)
    ->get();

$users = DB::table('users')
    ->whereJsonLength('options->languages', '>', 1)
    ->get();
```

Ordering, Grouping, Limit & Offset

orderBy

The `orderBy` method allows you to sort the result of the query by a given column. The first argument to the `orderBy` method should be the column you wish to sort by, while the second argument controls the direction of the sort and may be either `asc` or `desc`:

```
$users = DB::table('users')
    ->orderBy('name', 'desc')
    ->get();
```

If you need to sort by multiple columns, you may invoke `orderBy` as many times as needed:

```
$users = DB::table('users')
    ->orderBy('name', 'desc')
    ->orderBy('email', 'asc')
    ->get();
```

latest / oldest

The `latest` and `oldest` methods allow you to easily order results by date. By default, result will be ordered by the `created_at` column. Or, you may pass the column name that you wish to sort by:

```
$user = DB::table('users')
    ->latest()
    ->first();
```

inRandomOrder

The `inRandomOrder` method may be used to sort the query results randomly. For example, you may use this method to fetch a random user:

```
$randomUser = DB::table('users')
    ->inRandomOrder()
    ->first();
```

reorder

The `reorder` method allows you to remove all the existing orders and optionally apply a new order. For example, you can remove all the existing orders:

```
$query = DB::table('users')->orderBy('name');
$unorderedUsers = $query->reorder()->get();
```

To remove all existing orders and apply a new order, provide the column and direction as arguments to the method:

```
$query = DB::table('users')->orderBy('name');
$usersOrderedByEmail = $query->reorder('email', 'desc')->get();
```

groupBy / having

The `groupBy` and `having` methods may be used to group the query results. The `having` method's signature is similar to that of the `where` method:

```
$users = DB::table('users')
    ->groupBy('account_id')
    ->having('account_id', '>', 100)
    ->get();
```

You may pass multiple arguments to the `groupBy` method to group by multiple columns:

```
$users = DB::table('users')
```

```
->groupBy('first_name', 'status')
->having('account_id', '>', 100)
->get();
```

For more advanced having statements, see the [havingRaw](#) method.

skip / take

To limit the number of results returned from the query, or to skip a given number of results in the query, you may use the `skip` and `take` methods:

```
$users = DB::table('users')->skip(10)->take(5)->get();
```

Alternatively, you may use the `limit` and `offset` methods:

```
$users = DB::table('users')
->offset(10)
->limit(5)
->get();
```

Conditional Clauses

Sometimes you may want clauses to apply to a query only when something else is true. For instance you may only want to apply a `where` statement if a given input value is present on the incoming request. You may accomplish this using the `when` method:

```
$role = $request->input('role');

$users = DB::table('users')
->when($role, function ($query, $role) {
    return $query->where('role_id', $role);
})
->get();
```

The `when` method only executes the given Closure when the first parameter is `true`. If the first parameter is `false`, the Closure will not be executed.

You may pass another Closure as the third parameter to the `when` method. This Closure will execute if the first parameter evaluates as `false`. To illustrate how this feature may be used, we will use it to configure the default sorting of a query:

```
$sortBy = null;

$users = DB::table('users')
->when($sortBy, function ($query, $sortBy) {
    return $query->orderBy($sortBy);
}, function ($query) {
    return $query->orderBy('name');
})
->get();
```

Inserts

The query builder also provides an `insert` method for inserting records into the database table. The `insert` method accepts an array of column names and values:

```
DB::table('users')->insert(
    ['email' => 'john@example.com', 'votes' => 0]
);
```

You may even insert several records into the table with a single call to `insert` by passing an array of arrays. Each array represents a row to be inserted into the table:

```
DB::table('users')->insert([
    ['email' => 'taylor@example.com', 'votes' => 0],
    ['email' => 'dayle@example.com', 'votes' => 0],
]);
```

The `insertOrIgnore` method will ignore duplicate record errors while inserting records into the database:

```
DB::table('users')->insertOrIgnore([
    ['id' => 1, 'email' => 'taylor@example.com'],
    ['id' => 2, 'email' => 'dayle@example.com'],
]);
```

Auto-Incrementing IDs

If the table has an auto-incrementing id, use the `insertGetId` method to insert a record and then retrieve the ID:

```
$id = DB::table('users')->insertGetId(
    ['email' => 'john@example.com', 'votes' => 0]
);
```

NOTE When using PostgreSQL the `insertGetId` method expects the auto-incrementing column to be named `id`. If you would like to retrieve the ID from a different "sequence", you may pass the column name as the second parameter to the `insertGetId` method.

Updates

In addition to inserting records into the database, the query builder can also update existing records using the `update` method. The `update` method, like the `insert` method, accepts an array of column and value pairs containing the columns to be updated. You may constrain the `update` query using `where` clauses:

```
$affected = DB::table('users')
    ->where('id', 1)
    ->update(['votes' => 1]);
```

Update Or Insert

Sometimes you may want to update an existing record in the database or create it if no matching record exists. In this scenario, the `updateOrInsert` method may be used. The `updateOrInsert` method accepts two arguments: an array of conditions by which to find the record, and an array of column and value pairs containing the columns to be updated.

The `updateOrInsert` method will first attempt to locate a matching database record using the first argument's column and value pairs. If the record exists, it will be updated with the values in the second argument. If the record can not be found, a new record will be inserted with the merged attributes of both arguments:

```
DB::table('users')
    ->updateOrInsert(
        ['email' => 'john@example.com', 'name' => 'John'],
        ['votes' => '2']
    );
```

Updating JSON Columns

When updating a JSON column, you should use `->` syntax to access the appropriate key in the JSON object. This operation is supported on MySQL 5.7+ and PostgreSQL 9.5+:

```
$affected = DB::table('users')
    ->where('id', 1)
    ->update(['options->enabled' => true]);
```

Increment & Decrement

The query builder also provides convenient methods for incrementing or decrementing the value of a given column. This is a shortcut, providing a more expressive and terse interface compared to manually writing the `update` statement.

Both of these methods accept at least one argument: the column to modify. A second argument may optionally be passed to control the amount by which the column should be incremented or decremented:

```
DB::table('users')->increment('votes');  
DB::table('users')->increment('votes', 5);  
DB::table('users')->decrement('votes');  
DB::table('users')->decrement('votes', 5);
```

You may also specify additional columns to update during the operation:

```
DB::table('users')->increment('votes', 1, ['name' => 'John']);
```

NOTE Model events are not fired when using the `increment` and `decrement` methods.

Deletes

The query builder may also be used to delete records from the table via the `delete` method. You may constrain `delete` statements by adding `where` clauses before calling the `delete` method:

```
DB::table('users')->delete();  
DB::table('users')->where('votes', '>', 100)->delete();
```

If you wish to truncate the entire table, which will remove all rows and reset the auto-incrementing ID to zero, you may use the `truncate` method:

```
DB::table('users')->truncate();
```

Pessimistic Locking

The query builder also includes a few functions to help you do "pessimistic locking" on your `select` statements. To run the statement with a "shared lock", you may use the `sharedLock` method on a query. A shared lock prevents the selected rows from being modified until your transaction commits:

```
DB::table('users')->where('votes', '>', 100)->sharedLock()->get();
```

Alternatively, you may use the `lockForUpdate` method. A "for update" lock prevents the rows from being modified or from being selected with another shared lock:

```
DB::table('users')->where('votes', '>', 100)->lockForUpdate()->get();
```

Debugging

You may use the `dd` or `dump` methods while building a query to dump the query bindings and SQL. The `dd` method will display the debug information and then stop executing the request. The `dump` method will display the debug information but allow the request to keep executing:

```
DB::table('users')->where('votes', '>', 100)->dd();  
DB::table('users')->where('votes', '>', 100)->dump();
```

Database

Pagination

Database: Pagination

- [Introduction](#)
- [Basic Usage](#)
 - [Paginating Query Builder Results](#)
 - [Paginating Eloquent Results](#)
 - [Manually Creating A Paginator](#)
- [Displaying Pagination Results](#)
 - [Converting Results To JSON](#)
- [Customizing The Pagination View](#)
 - [Using Tailwind](#)
- [Paginator Instance Methods](#)

Introduction

In other frameworks, pagination can be very painful. Laravel's paginator is integrated with the [query builder](#) and [Eloquent ORM](#) and provides convenient, easy-to-use pagination of database results out of the box. The HTML generated by the paginator is compatible with the [Bootstrap CSS framework](#).

Basic Usage

Paginating Query Builder Results

There are several ways to paginate items. The simplest is by using the `paginate` method on the [query builder](#) or an [Eloquent query](#). The `paginate` method automatically takes care of setting the proper limit and offset based on the current page being viewed by the user. By default, the current page is detected by the value of the `page` query string argument on the HTTP request. This value is automatically detected by Laravel, and is also automatically inserted into links generated by the paginator.

In this example, the only argument passed to the `paginate` method is the number of items you would like displayed "per page". In this case, let's specify that we would like to display 15 items per page:

```
<?php

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;
use Illuminate\Support\Facades\DB;

class UserController extends Controller
{
    /**
     * Show all of the users for the application.
     *
     * @return Response
     */
    public function index()
    {
        $users = DB::table('users')->paginate(15);

        return view('user.index', ['users' => $users]);
    }
}
```

NOTE Currently, pagination operations that use a `groupBy` statement cannot be executed efficiently by Laravel. If you need to use a `groupBy` with a paginated result set, it is recommended that you query the database and create a paginator manually.

"Simple Pagination"

If you only need to display simple "Next" and "Previous" links in your pagination view, you may use the `simplePaginate` method to perform a more efficient query. This is very useful for large datasets when you do not need to display a link for each page number when rendering your view:

```
$users = DB::table('users')->simplePaginate(15);
```

Paginating Eloquent Results

You may also paginate [Eloquent](#) queries. In this example, we will paginate the `User` model with 15 items per page. As you can see, the syntax is nearly identical to paginating query builder results:

```
$users = App\User::paginate(15);
```

You may call `paginate` after setting other constraints on the query, such as `where` clauses:

```
$users = User::where('votes', '>', 100)->paginate(15);
```

You may also use the `simplePaginate` method when paginating Eloquent models:

```
$users = User::where('votes', '>', 100)->simplePaginate(15);
```

Manually Creating A Paginator

Sometimes you may wish to create a pagination instance manually, passing it an array of items. You may do so by creating either an `Illuminate\Pagination\Paginator` or `Illuminate\Pagination\LengthAwarePaginator` instance, depending on your needs.

The `Paginator` class does not need to know the total number of items in the result set; however, because of this, the class does not have methods for retrieving the index of the last page. The `LengthAwarePaginator` accepts almost the same arguments as the `Paginator`; however, it does require a count of the total number of items in the result set.

In other words, the `Paginator` corresponds to the `simplePaginate` method on the query builder and Eloquent, while the `LengthAwarePaginator` corresponds to the `paginate` method.

NOTE When manually creating a paginator instance, you should manually "slice" the array of results you pass to the paginator. If you're unsure how to do this, check out the [array slice](#) PHP function.

Displaying Pagination Results

When calling the `paginate` method, you will receive an instance of `Illuminate\Pagination\LengthAwarePaginator`. When calling the `simplePaginate` method, you will receive an instance of `Illuminate\Pagination\Paginator`. These objects provide several methods that describe the result set. In addition to these helpers methods, the paginator instances are iterators and may be looped as an array. So, once you have retrieved the results, you may display the results and render the page links using [Blade](#):

```
<div class="container">
    @foreach ($users as $user)
        {{ $user->name }}
    @endforeach
</div>

{{ $users->links() }}
```

The `links` method will render the links to the rest of the pages in the result set. Each of these links will already contain the proper page query string variable. Remember, the HTML generated by the `links` method is compatible with the [Bootstrap CSS framework](#).

Customizing The Paginator URI

The `withPath` method allows you to customize the URI used by the paginator when generating links. For example, if you want the paginator to generate links like `http://example.com/custom/url?page=N`, you should pass `custom/url` to the `withPath` method:

```
Route::get('users', function () {
    $users = App\User::paginate(15);

    $users->withPath('custom/url');

    //
});
```

Appending To Pagination Links

You may append to the query string of pagination links using the `appends` method. For example, to append `sort=votes` to each pagination link, you should make the following call to `appends`:

```
{{ $users->appends(['sort' => 'votes'])->links() }}
```

If you wish to append all current query string values to the pagination links you may use the `withQueryString` method:

```
{{ $users->withQueryString()->links() }}
```

If you wish to append a "hash fragment" to the paginator's URLs, you may use the `fragment` method. For example, to append `#foo` to the end of each pagination link, make the following call to the `fragment` method:

```
{{ $users->fragment('foo')->links() }}
```

Adjusting The Pagination Link Window

You may control how many additional links are displayed on each side of the paginator's URL "window". By default, three links are displayed on each side of the primary paginator links. However, you may control this number using the `onEachSide` method:

```
{{ $users->onEachSide(5)->links() }}
```

Converting Results To JSON

The Laravel paginator result classes implement the `Illuminate\Contracts\Support\Jsonable` Interface contract and expose the `toJson` method, so it's very easy to convert your pagination results to JSON. You may also convert a paginator instance to JSON by returning it from a route or controller action:

```
Route::get('users', function () {
    return App\User::paginate();
});
```

The JSON from the paginator will include meta information such as `total`, `current_page`, `last_page`, and more. The actual result objects will be available via the `data` key in the JSON array. Here is an example of the JSON created by returning a paginator instance from a route:

```
{
  "total": 50,
  "per_page": 15,
  "current_page": 1,
  "last_page": 4,
  "first_page_url": "http://laravel.app?page=1",
  "last_page_url": "http://laravel.app?page=4",
  "next_page_url": "http://laravel.app?page=2",
  "prev_page_url": null,
  "path": "http://laravel.app",
  "from": 1,
  "to": 15,
  "data": [
    {
      // Result Object
    },
    {
      // Result Object
    }
  ]
}
```

Customizing The Pagination View

By default, the views rendered to display the pagination links are compatible with the Bootstrap CSS framework. However, if you are not using Bootstrap, you are free to define your own views to render these links. When calling the `links` method on a paginator instance, pass the view name as the first argument to the method:

```
{{ $paginator->links('view.name') }}
```

```
// Passing data to the view...
{{ $paginator->links('view.name', ['foo' => 'bar']) }}
```

However, the easiest way to customize the pagination views is by exporting them to your `resources/views/vendor` directory using the `vendor:publish` command:

```
php artisan vendor:publish --tag=laravel-pagination
```

This command will place the views in the `resources/views/vendor/pagination` directory. The `bootstrap-4.blade.php` file within this directory corresponds to the default pagination view. You may edit this file to modify the pagination HTML.

If you would like to designate a different file as the default pagination view, you may use the paginator's `defaultView` and `defaultSimpleView` methods within your `AppServiceProvider`:

```
use Illuminate\Pagination\Paginator;

public function boot()
{
    Paginator::defaultView('view-name');

    Paginator::defaultSimpleView('view-name');
}
```

Using Tailwind

Laravel includes pagination views built using [Tailwind CSS](#). To use these views instead of the default Bootstrap views, you may call the paginator's `useTailwind` method within your `AppServiceProvider`:

```
use Illuminate\Pagination\Paginator;

public function boot()
{
    Paginator::useTailwind();
}
```

Paginator Instance Methods

Each paginator instance provides additional pagination information via the following methods:

Method	Description
<code>\$paginator->count()</code>	Get the number of items for the current page.
<code>\$paginator->currentPage()</code>	Get the current page number.
<code>\$paginator->firstItem()</code>	Get the result number of the first item in the results.
<code>\$paginator->getOptions()</code>	Get the paginator options.
<code>\$paginator->getUrlRange(\$start, \$end)</code>	Create a range of pagination URLs.
<code>\$paginator->hasPages()</code>	Determine if there are enough items to split into multiple pages.
<code>\$paginator->hasMorePages()</code>	Determine if there is more items in the data store.
<code>\$paginator->items()</code>	Get the items for the current page.
<code>\$paginator->lastItem()</code>	Get the result number of the last item in the results.
<code>\$paginator->lastPage()</code>	Get the page number of the last available page. (Not available when using <code>simplePaginate</code>).
<code>\$paginator->nextPageUrl()</code>	Get the URL for the next page.
<code>\$paginator->onFirstPage()</code>	Determine if the paginator is on the first page.

<code>\$paginator->perPage()</code>	The number of items to be shown per page.
<code>\$paginator->previousPageUrl()</code>	Get the URL for the previous page.
<code>\$paginator->total()</code>	Determine the total number of matching items in the data store. (Not available when using <code>simplePaginate</code>).
<code>\$paginator->url(\$page)</code>	Get the URL for a given page number.
<code>\$paginator->getPageName()</code>	Get the query string variable used to store the page.
<code>\$paginator->setPageName(\$name)</code>	Set the query string variable used to store the page.

Database

Migrations

Database: Migrations

- [Introduction](#)
- [Generating Migrations](#)
- [Migration Structure](#)
- [Running Migrations](#)
 - [Rolling Back Migrations](#)
- [Tables](#)
 - [Creating Tables](#)
 - [Renaming / Dropping Tables](#)
- [Columns](#)
 - [Creating Columns](#)
 - [Column Modifiers](#)
 - [Modifying Columns](#)
 - [Dropping Columns](#)
- [Indexes](#)
 - [Creating Indexes](#)
 - [Renaming Indexes](#)
 - [Dropping Indexes](#)
 - [Foreign Key Constraints](#)

Introduction

Migrations are like version control for your database, allowing your team to modify and share the application's database schema. Migrations are typically paired with Laravel's schema builder to build your application's database schema. If you have ever had to tell a teammate to manually add a column to their local database schema, you've faced the problem that database migrations solve.

The Laravel schema [facade](#) provides database agnostic support for creating and manipulating tables across all of Laravel's supported database systems.

Generating Migrations

To create a migration, use the `make:migration` [Artisan command](#):

```
php artisan make:migration create_users_table
```

The new migration will be placed in your `database/migrations` directory. Each migration file name contains a timestamp, which allows Laravel to determine the order of the migrations.

TIP Migration stubs may be customized using [stub publishing](#)

The `--table` and `--create` options may also be used to indicate the name of the table and whether or not the migration will be creating a new table. These options pre-fill the generated migration stub file with the specified table:

```
php artisan make:migration create_users_table --create=users
```

```
php artisan make:migration add_votes_to_users_table --table=users
```

If you would like to specify a custom output path for the generated migration, you may use the `--path` option when executing the `make:migration` command. The given path should be relative to your application's base path.

Migration Structure

A migration class contains two methods: `up` and `down`. The `up` method is used to add new tables, columns, or

indexes to your database, while the `down` method should reverse the operations performed by the `up` method.

Within both of these methods you may use the Laravel schema builder to expressively create and modify tables. To learn about all of the methods available on the Schema builder, [check out its documentation](#). For example, the following migration creates a `flights` table:

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

class CreateFlightsTable extends Migration
{
    /**
     * Run the migrations.
     *
     * @return void
     */
    public function up()
    {
        Schema::create('flights', function (Blueprint $table) {
            $table->id();
            $table->string('name');
            $table->string('airline');
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     *
     * @return void
     */
    public function down()
    {
        Schema::drop('flights');
    }
}
```

Running Migrations

To run all of your outstanding migrations, execute the `migrate` Artisan command:

```
php artisan migrate
```

NOTE If you are using the [Homestead virtual machine](#), you should run this command from within your virtual machine.

Forcing Migrations To Run In Production

Some migration operations are destructive, which means they may cause you to lose data. In order to protect you from running these commands against your production database, you will be prompted for confirmation before the commands are executed. To force the commands to run without a prompt, use the `--force` flag:

```
php artisan migrate --force
```

Rolling Back Migrations

To roll back the latest migration operation, you may use the `rollback` command. This command rolls back the last "batch" of migrations, which may include multiple migration files:

```
php artisan migrate:rollback
```

You may roll back a limited number of migrations by providing the `step` option to the `rollback` command. For example, the following command will roll back the last five migrations:

```
php artisan migrate:rollback --step=5
```

The `migrate:reset` command will roll back all of your application's migrations:

```
php artisan migrate:reset
```

Roll Back & Migrate Using A Single Command

The `migrate:refresh` command will roll back all of your migrations and then execute the `migrate` command. This command effectively re-creates your entire database:

```
php artisan migrate:refresh

// Refresh the database and run all database seeds...
php artisan migrate:refresh --seed
```

You may roll back & re-migrate a limited number of migrations by providing the `step` option to the `refresh` command. For example, the following command will roll back & re-migrate the last five migrations:

```
php artisan migrate:refresh --step=5
```

Drop All Tables & Migrate

The `migrate:fresh` command will drop all tables from the database and then execute the `migrate` command:

```
php artisan migrate:fresh

php artisan migrate:fresh --seed
```

Tables

Creating Tables

To create a new database table, use the `create` method on the `Schema` facade. The `create` method accepts two arguments: the first is the name of the table, while the second is a `Closure` which receives a `Blueprint` object that may be used to define the new table:

```
Schema::create('users', function (Blueprint $table) {
    $table->id();
});
```

When creating the table, you may use any of the schema builder's [column methods](#) to define the table's columns.

Checking For Table / Column Existence

You may check for the existence of a table or column using the `hasTable` and `hasColumn` methods:

```
if (Schema::hasTable('users')) {
    //
}

if (Schema::hasColumn('users', 'email')) {
    //
}
```

Database Connection & Table Options

If you want to perform a schema operation on a database connection that is not your default connection, use the `connection` method:

```
Schema::connection('foo')->create('users', function (Blueprint $table) {
    $table->id();
});
```

You may use the following commands on the schema builder to define the table's options:

Command	Description
<code>\$table->engine = 'InnoDB';</code>	Specify the table storage engine (MySQL).

<code>\$table->charset = 'utf8mb4';</code>	Specify a default character set for the table (MySQL).
<code>\$table->collation = 'utf8mb4_unicode_ci';</code>	Specify a default collation for the table (MySQL).
<code>\$table->temporary();</code>	Create a temporary table (except SQL Server).

Renaming / Dropping Tables

To rename an existing database table, use the `rename` method:

```
Schema::rename($from, $to);
```

To drop an existing table, you may use the `drop` or `dropIfExists` methods:

```
Schema::drop('users');
```

```
Schema::dropIfExists('users');
```

Renaming Tables With Foreign Keys

Before renaming a table, you should verify that any foreign key constraints on the table have an explicit name in your migration files instead of letting Laravel assign a convention based name. Otherwise, the foreign key constraint name will refer to the old table name.

Columns

Creating Columns

The `table` method on the schema facade may be used to update existing tables. Like the `create` method, the `table` method accepts two arguments: the name of the table and a `closure` that receives a `Blueprint` instance you may use to add columns to the table:

```
Schema::table('users', function (Blueprint $table) {  
    $table->string('email');  
});
```

Available Column Types

The schema builder contains a variety of column types that you may specify when building your tables:

Command	Description
<code>\$table->id();</code>	Alias of <code>\$table->bigIncrements('id')</code> .
<code>\$table->foreignId('user_id');</code>	Alias of <code>\$table->unsignedBigInteger('user_id')</code> .
<code>\$table->bigIncrements('id');</code>	Auto-incrementing UNSIGNED BIGINT (primary key) equivalent column.
<code>\$table->bigInteger('votes');</code>	BIGINT equivalent column.
<code>\$table->binary('data');</code>	BLOB equivalent column.
<code>\$table->boolean('confirmed');</code>	BOOLEAN equivalent column.
<code>\$table->char('name', 100);</code>	CHAR equivalent column with a length.
<code>\$table->date('created_at');</code>	DATE equivalent column.
<code>\$table->dateTime('created_at', 0);</code>	DATETIME equivalent column with precision (total digits).
<code>\$table->dateTimeTz('created_at', 0);</code>	DATETIME (with timezone) equivalent column with precision (total digits).
<code>\$table->decimal('amount', 8, 2);</code>	DECIMAL equivalent column with precision (total digits) and scale (decimal digits).
<code>\$table->double('amount', 8, 2);</code>	DOUBLE equivalent column with precision (total digits) and scale (decimal digits).
<code>\$table->enum('level', ['easy', 'hard']);</code>	ENUM equivalent column.
<code>\$table->float('amount', 8, 2);</code>	FLOAT equivalent column with a precision (total digits) and scale (decimal digits).
<code>\$table->geometry('positions');</code>	GEOMETRY equivalent column.
<code>\$table->geometryCollection('positions');</code>	GEOMETRYCOLLECTION equivalent column.
<code>\$table->increments('id');</code>	Auto-incrementing UNSIGNED INTEGER (primary key) equivalent column.
<code>\$table->integer('votes');</code>	INTEGER equivalent column.

<code>\$table->ipAddress('visitor');</code>	IP address equivalent column.
<code>\$table->json('options');</code>	JSON equivalent column.
<code>\$table->jsonb('options');</code>	JSONB equivalent column.
<code>\$table->lineString('positions');</code>	LINESTRING equivalent column.
<code>\$table->longText('description');</code>	LONGTEXT equivalent column.
<code>\$table->macAddress('device');</code>	MAC address equivalent column.
<code>\$table->mediumIncrements('id');</code>	Auto-incrementing UNSIGNED MEDIUMINT (primary key) equivalent column.
<code>\$table->mediumInteger('votes');</code>	MEDIUMINT equivalent column.
<code>\$table->mediumText('description');</code>	MEDIUMTEXT equivalent column.
<code>\$table->morphs('taggable');</code>	Adds taggable_id UNSIGNED BIGINT and taggable_type VARCHAR equivalent columns.
<code>\$table->uuidMorphs('taggable');</code>	Adds taggable_id CHAR(36) and taggable_type VARCHAR(255) UUID equivalent columns.
<code>\$table->multiLineString('positions');</code>	MULTILINESTRING equivalent column.
<code>\$table->multiPoint('positions');</code>	MULTIPOINT equivalent column.
<code>\$table->multiPolygon('positions');</code>	MULTIPOLYGON equivalent column.
<code>\$table->nullableMorphs('taggable');</code>	Adds nullable versions of morphs() columns.
<code>\$table->nullableUuidMorphs('taggable');</code>	Adds nullable versions of uuidMorphs() columns.
<code>\$table->nullableTimestamps(0);</code>	Alias of timestamps() method.
<code>\$table->point('position');</code>	POINT equivalent column.
<code>\$table->polygon('positions');</code>	POLYGON equivalent column.
<code>\$table->rememberToken();</code>	Adds a nullable remember_token VARCHAR(100) equivalent column.
<code>\$table->set('flavors', ['strawberry', 'vanilla']);</code>	SET equivalent column.
<code>\$table->smallIncrements('id');</code>	Auto-incrementing UNSIGNED SMALLINT (primary key) equivalent column.
<code>\$table->smallInteger('votes');</code>	SMALLINT equivalent column.
<code>\$table->softDeletes('deleted_at', 0);</code>	Adds a nullable deleted_at TIMESTAMP equivalent column for soft deletes with precision (total digits).
<code>\$table->softDeletesTz('deleted_at', 0);</code>	Adds a nullable deleted_at TIMESTAMP (with timezone) equivalent column for soft deletes with precision (total digits).
<code>\$table->string('name', 100);</code>	VARCHAR equivalent column with a length.
<code>\$table->text('description');</code>	TEXT equivalent column.
<code>\$table->time('sunrise', 0);</code>	TIME equivalent column with precision (total digits).
<code>\$table->timeTz('sunrise', 0);</code>	TIME (with timezone) equivalent column with precision (total digits).
<code>\$table->timestamp('added_on', 0);</code>	TIMESTAMP equivalent column with precision (total digits).
<code>\$table->timestampTz('added_on', 0);</code>	TIMESTAMP (with timezone) equivalent column with precision (total digits).
<code>\$table->timestamps(0);</code>	Adds nullable created_at and updated_at TIMESTAMP equivalent columns with precision (total digits).
<code>\$table->timestampsTz(0);</code>	Adds nullable created_at and updated_at TIMESTAMP (with timezone) equivalent columns with precision (total digits).
<code>\$table->tinyIncrements('id');</code>	Auto-incrementing UNSIGNED TINYINT (primary key) equivalent column.
<code>\$table->tinyInteger('votes');</code>	TINYINT equivalent column.
<code>\$table->unsignedBigInteger('votes');</code>	UNSIGNED BIGINT equivalent column.
<code>\$table->unsignedDecimal('amount', 8, 2);</code>	UNSIGNED DECIMAL equivalent column with a precision (total digits) and scale (decimal digits).
<code>\$table->unsignedInteger('votes');</code>	UNSIGNED INTEGER equivalent column.
<code>\$table->unsignedMediumInteger('votes');</code>	UNSIGNED MEDIUMINT equivalent column.
<code>\$table->unsignedSmallInteger('votes');</code>	UNSIGNED SMALLINT equivalent column.
<code>\$table->unsignedTinyInteger('votes');</code>	UNSIGNED TINYINT equivalent column.
<code>\$table->uuid('id');</code>	UUID equivalent column.
<code>\$table->year('birth_year');</code>	YEAR equivalent column.

Column Modifiers

In addition to the column types listed above, there are several column "modifiers" you may use while adding a column to a database table. For example, to make the column "nullable", you may use the `nullable()` method:

```
Schema::table('users', function (Blueprint $table) {
    $table->string('email')->nullable();
});
```

The following list contains all available column modifiers. This list does not include the [index modifiers](#):

Modifier	Description
<code>->after('column')</code>	Place the column "after" another column (MySQL)
<code>->autoIncrement()</code>	Set INTEGER columns as auto-increment (primary key)
<code>->charset('utf8mb4')</code>	Specify a character set for the column (MySQL)
<code>->collation('utf8mb4_unicode_ci')</code>	Specify a collation for the column (MySQL/PostgreSQL/SQL Server)
<code>->comment('my comment')</code>	Add a comment to a column (MySQL/PostgreSQL)
<code>->default(\$value)</code>	Specify a "default" value for the column
<code>->first()</code>	Place the column "first" in the table (MySQL)
<code>->nullable(\$value = true)</code>	Allows (by default) NULL values to be inserted into the column
<code>->storedAs(\$expression)</code>	Create a stored generated column (MySQL)
<code>->unsigned()</code>	Set INTEGER columns as UNSIGNED (MySQL)
<code>->useCurrent()</code>	Set TIMESTAMP columns to use CURRENT_TIMESTAMP as default value
<code>->virtualAs(\$expression)</code>	Create a virtual generated column (MySQL)
<code>->generatedAs(\$expression)</code>	Create an identity column with specified sequence options (PostgreSQL)
<code>->always()</code>	Defines the precedence of sequence values over input for an identity column (PostgreSQL)

Default Expressions

The default modifier accepts a value or an `\Illuminate\Database\Query\Expression` instance. Using an `Expression` instance will prevent wrapping the value in quotes and allow you to use database specific functions. One situation where this is particularly useful is when you need to assign default values to JSON columns:

```
<?php

use Illuminate\Support\Facades\Schema;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Database\Query\Expression;
use Illuminate\Database\Migrations\Migration;

class CreateFlightsTable extends Migration
{
    /**
     * Run the migrations.
     *
     * @return void
     */
    public function up()
    {
        Schema::create('flights', function (Blueprint $table) {
            $table->id();
            $table->json('movies')->default(new Expression('(JSON_ARRAY())'));
            $table->timestamps();
        });
    }
}
```

NOTE Support for default expressions depends on your database driver, database version, and the field type. Please refer to the appropriate documentation for compatibility. Also note that using database specific functions may tightly couple you to a specific driver.

Modifying Columns

Prerequisites

Before modifying a column, be sure to add the `doctrine/dbal` dependency to your `composer.json` file. The

Doctrine DBAL library is used to determine the current state of the column and create the SQL queries needed to make the required adjustments:

```
composer require doctrine/dbal
```

Updating Column Attributes

The `change` method allows you to modify type and attributes of existing columns. For example, you may wish to increase the size of a `string` column. To see the `change` method in action, let's increase the size of the `name` column from 25 to 50:

```
Schema::table('users', function (Blueprint $table) {
    $table->string('name', 50)->change();
});
```

We could also modify a column to be nullable:

```
Schema::table('users', function (Blueprint $table) {
    $table->string('name', 50)->nullable()->change();
});
```

NOTE Only the following column types can be "changed": `bigInteger`, `binary`, `boolean`, `date`, `dateTime`, `dateTimeTz`, `decimal`, `integer`, `json`, `longText`, `mediumText`, `smallInteger`, `string`, `text`, `time`, `unsignedBigInteger`, `unsignedInteger`, `unsignedSmallInteger` and `uuid`.

Renaming Columns

To rename a column, you may use the `renameColumn` method on the schema builder. Before renaming a column, be sure to add the `doctrine/dbal` dependency to your `composer.json` file:

```
Schema::table('users', function (Blueprint $table) {
    $table->renameColumn('from', 'to');
});
```

NOTE Renaming any column in a table that also has a column of type `enum` is not currently supported.

Dropping Columns

To drop a column, use the `dropColumn` method on the schema builder. Before dropping columns from a SQLite database, you will need to add the `doctrine/dbal` dependency to your `composer.json` file and run the `composer update` command in your terminal to install the library:

```
Schema::table('users', function (Blueprint $table) {
    $table->dropColumn('votes');
});
```

You may drop multiple columns from a table by passing an array of column names to the `dropColumn` method:

```
Schema::table('users', function (Blueprint $table) {
    $table->dropColumn(['votes', 'avatar', 'location']);
});
```

NOTE Dropping or modifying multiple columns within a single migration while using a SQLite database is not supported.

Available Command Aliases

Command	Description
<code>\$table->dropMorphs('morphable');</code>	Drop the <code>morphable_id</code> and <code>morphable_type</code> columns.
<code>\$table->dropRememberToken();</code>	Drop the <code>remember_token</code> column.
<code>\$table->dropSoftDeletes();</code>	Drop the <code>deleted_at</code> column.
<code>\$table->dropSoftDeletesTz();</code>	Alias of <code>dropSoftDeletes()</code> method.
<code>\$table->dropTimestamps();</code>	Drop the <code>created_at</code> and <code>updated_at</code> columns.
<code>\$table->dropTimestampsTz();</code>	Alias of <code>dropTimestamps()</code> method.

Indexes

Creating Indexes

The Laravel schema builder supports several types of indexes. The following example creates a new `email` column and specifies that its values should be unique. To create the index, we can chain the `unique` method onto the column definition:

```
$table->string('email')->unique();
```

Alternatively, you may create the index after defining the column. For example:

```
$table->unique('email');
```

You may even pass an array of columns to an index method to create a compound (or composite) index:

```
$table->index(['account_id', 'created_at']);
```

Laravel will automatically generate an index name based on the table, column names, and the index type, but you may pass a second argument to the method to specify the index name yourself:

```
$table->unique('email', 'unique_email');
```

Available Index Types

Each index method accepts an optional second argument to specify the name of the index. If omitted, the name will be derived from the names of the table and column(s) used for the index, as well as the index type.

Command	Description
<code>\$table->primary('id');</code>	Adds a primary key.
<code>\$table->primary(['id', 'parent_id']);</code>	Adds composite keys.
<code>\$table->unique('email');</code>	Adds a unique index.
<code>\$table->index('state');</code>	Adds a plain index.
<code>\$table->spatialIndex('location');</code>	Adds a spatial index. (except SQLite)

Index Lengths & MySQL / MariaDB

Laravel uses the `utf8mb4` character set by default, which includes support for storing "emojis" in the database. If you are running a version of MySQL older than the 5.7.7 release or MariaDB older than the 10.2.2 release, you may need to manually configure the default string length generated by migrations in order for MySQL to create indexes for them. You may configure this by calling the `Schema::defaultStringLength` method within your `AppServiceProvider`:

```
use Illuminate\Support\Facades\Schema;

/**
 * Bootstrap any application services.
 *
 * @return void
 */
public function boot()
{
    Schema::defaultStringLength(191);
}
```

Alternatively, you may enable the `innodb_large_prefix` option for your database. Refer to your database's documentation for instructions on how to properly enable this option.

Renaming Indexes

To rename an index, you may use the `renameIndex` method. This method accepts the current index name as its first argument and the desired new name as its second argument:

```
$table->renameIndex('from', 'to')
```

Dropping Indexes

To drop an index, you must specify the index's name. By default, Laravel automatically assigns an index name based on the table name, the name of the indexed column, and the index type. Here are some examples:

Command	Description
<code>\$table->dropPrimary('users_id_primary');</code>	Drop a primary key from the "users" table.
<code>\$table->dropUnique('users_email_unique');</code>	Drop a unique index from the "users" table.
<code>\$table->dropIndex('geo_state_index');</code>	Drop a basic index from the "geo" table.
<code>\$table->dropSpatialIndex('geo_location_spatialindex');</code>	Drop a spatial index from the "geo" table (except SQLite).

If you pass an array of columns into a method that drops indexes, the conventional index name will be generated based on the table name, columns and key type:

```
Schema::table('geo', function (Blueprint $table) {
    $table->dropIndex(['state']); // Drops index 'geo_state_index'
});
```

Foreign Key Constraints

Laravel also provides support for creating foreign key constraints, which are used to force referential integrity at the database level. For example, let's define a `user_id` column on the `posts` table that references the `id` column on a `users` table:

```
Schema::table('posts', function (Blueprint $table) {
    $table->unsignedBigInteger('user_id');

    $table->foreign('user_id')->references('id')->on('users');
});
```

Since this syntax is rather verbose, Laravel provides additional, terser methods that use convention to provide a better developer experience. The example above could be written like so:

```
Schema::table('posts', function (Blueprint $table) {
    $table->foreignId('user_id')->constrained();
});
```

The `foreignId` method is an alias for `unsignedBigInteger` while the `constrained` method will use convention to determine the table and column name being referenced. If your table name does not match the convention, you may specify the table name by passing it as an argument to the `constrained` method:

```
Schema::table('posts', function (Blueprint $table) {
    $table->foreignId('user_id')->constrained('users');
});
```

You may also specify the desired action for the "on delete" and "on update" properties of the constraint:

```
$table->foreignId('user_id')
    ->constrained()
    ->onDelete('cascade');
```

Any additional [column modifiers](#) must be called before `constrained`:

```
$table->foreignId('user_id')
    ->nullable()
    ->constrained();
```

To drop a foreign key, you may use the `dropForeign` method, passing the foreign key constraint to be deleted as an argument. Foreign key constraints use the same naming convention as indexes, based on the table name and the columns in the constraint, followed by a `"_foreign"` suffix:

```
$table->dropForeign('posts_user_id_foreign');
```

Alternatively, you may pass an array containing the column name that holds the foreign key to the `dropForeign`

method. The array will be automatically converted using the constraint name convention used by Laravel's schema builder:

```
$table->dropForeign(['user_id']);
```

You may enable or disable foreign key constraints within your migrations by using the following methods:

```
Schema::enableForeignKeyConstraints();
```

```
Schema::disableForeignKeyConstraints();
```

NOTE SQLite disables foreign key constraints by default. When using SQLite, make sure to [enable foreign key support](#) in your database configuration before attempting to create them in your migrations. In addition, SQLite only supports foreign keys upon creation of the table and [not when tables are altered](#).

Database

Seeding

Database: Seeding

- [Introduction](#)
- [Writing Seeders](#)
 - [Using Model Factories](#)
 - [Calling Additional Seeders](#)
- [Running Seeders](#)

Introduction

Laravel includes a simple method of seeding your database with test data using seed classes. All seed classes are stored in the `database/seeds` directory. Seed classes may have any name you wish, but probably should follow some sensible convention, such as `UserSeeder`, etc. By default, a `DatabaseSeeder` class is defined for you. From this class, you may use the `call` method to run other seed classes, allowing you to control the seeding order.

Writing Seeders

To generate a seeder, execute the `make:seeder` [Artisan command](#). All seeders generated by the framework will be placed in the `database/seeds` directory:

```
php artisan make:seeder UserSeeder
```

A seeder class only contains one method by default: `run`. This method is called when the `db:seed` [Artisan command](#) is executed. Within the `run` method, you may insert data into your database however you wish. You may use the [query builder](#) to manually insert data or you may use [Eloquent model factories](#).

TIP [Mass assignment protection](#) is automatically disabled during database seeding.

As an example, let's modify the default `DatabaseSeeder` class and add a database insert statement to the `run` method:

```
<?php

use Illuminate\Database\Seeder;
use Illuminate\Support\Facades\DB;
use Illuminate\Support\Facades\Hash;
use Illuminate\Support\Str;

class DatabaseSeeder extends Seeder
{
    /**
     * Run the database seeds.
     *
     * @return void
     */
    public function run()
    {
        DB::table('users')->insert([
            'name' => Str::random(10),
            'email' => Str::random(10).'@gmail.com',
            'password' => Hash::make('password'),
        ]);
    }
}
```

TIP You may type-hint any dependencies you need within the `run` method's signature. They will automatically be resolved via the Laravel [service container](#).

Using Model Factories

Of course, manually specifying the attributes for each model seed is cumbersome. Instead, you can use [model factories](#) to conveniently generate large amounts of database records. First, review the [model factory documentation](#) to learn how to define your factories. Once you have defined your factories, you may use the factory helper function to insert records into your database.

For example, let's create 50 users and attach a relationship to each user:

```
/**
 * Run the database seeds.
 *
 * @return void
 */
public function run()
{
    factory(App\User::class, 50)->create()->each(function ($user) {
        $user->posts()->save(factory(App\Post::class)->make());
    });
}
```

Calling Additional Seeders

Within the DatabaseSeeder class, you may use the call method to execute additional seed classes. Using the call method allows you to break up your database seeding into multiple files so that no single seeder class becomes overwhelmingly large. Pass the name of the seeder class you wish to run:

```
/**
 * Run the database seeds.
 *
 * @return void
 */
public function run()
{
    $this->call([
        UserSeeder::class,
        PostSeeder::class,
        CommentSeeder::class,
    ]);
}
```

Running Seeders

Once you have written your seeder, you may need to regenerate Composer's autoloader using the dump-autoload command:

```
composer dump-autoload
```

Now you may use the db:seed Artisan command to seed your database. By default, the db:seed command runs the DatabaseSeeder class, which may be used to call other seed classes. However, you may use the --class option to specify a specific seeder class to run individually:

```
php artisan db:seed
```

```
php artisan db:seed --class=UserSeeder
```

You may also seed your database using the migrate:fresh command, which will drop all tables and re-run all of your migrations. This command is useful for completely re-building your database:

```
php artisan migrate:fresh --seed
```

Forcing Seeders To Run In Production

Some seeding operations may cause you to alter or lose data. In order to protect you from running seeding commands against your production database, you will be prompted for confirmation before the seeders are executed. To force the seeders to run without a prompt, use the --force flag:

```
php artisan db:seed --force
```


Database

Redis

Redis

- [Introduction](#)
 - [Configuration](#)
 - [Predis](#)
 - [PhpRedis](#)
- [Interacting With Redis](#)
 - [Pipelining Commands](#)
- [Pub / Sub](#)

Introduction

[Redis](#) is an open source, advanced key-value store. It is often referred to as a data structure server since keys can contain [strings](#), [hashes](#), [lists](#), [sets](#), and [sorted sets](#).

Before using Redis with Laravel, we encourage you to install and use the [PhpRedis](#) PHP extension via PECL. The extension is more complex to install but may yield better performance for applications that make heavy use of Redis.

Alternatively, you can install the `redis/redis` package via Composer:

```
composer require redis/redis
```

Configuration

The Redis configuration for your application is located in the `config/database.php` configuration file. Within this file, you will see a `redis` array containing the Redis servers utilized by your application:

```
'redis' => [  
    'client' => env('REDIS_CLIENT', 'phpredis'),  
    'default' => [  
        'host' => env('REDIS_HOST', '127.0.0.1'),  
        'password' => env('REDIS_PASSWORD', null),  
        'port' => env('REDIS_PORT', 6379),  
        'database' => env('REDIS_DB', 0),  
    ],  
    'cache' => [  
        'host' => env('REDIS_HOST', '127.0.0.1'),  
        'password' => env('REDIS_PASSWORD', null),  
        'port' => env('REDIS_PORT', 6379),  
        'database' => env('REDIS_CACHE_DB', 1),  
    ],  
],
```

The default server configuration should suffice for development. However, you are free to modify this array based on your environment. Each Redis server defined in your configuration file is required to have a name, host, and port unless you define a single URL to represent the Redis connection:

```
'redis' => [  
    'client' => env('REDIS_CLIENT', 'phpredis'),  
    'default' => [  
        'url' => 'tcp://127.0.0.1:6379?database=0',  
    ],  
    'cache' => [  
        'url' => 'tls://user:password@127.0.0.1:6380?database=1',  
    ],  
],
```

```
],
```

Configuring The Connection Scheme

By default, Redis clients will use the `tcp` scheme when connecting to your Redis servers; however, you may use TLS / SSL encryption by specifying a `scheme` configuration option in your Redis server configuration:

```
'redis' => [
    'client' => env('REDIS_CLIENT', 'phpredis'),

    'default' => [
        'scheme' => 'tls',
        'host' => env('REDIS_HOST', '127.0.0.1'),
        'password' => env('REDIS_PASSWORD', null),
        'port' => env('REDIS_PORT', 6379),
        'database' => env('REDIS_DB', 0),
    ],
],
```

Configuring Clusters

If your application is utilizing a cluster of Redis servers, you should define these clusters within a `clusters` key of your Redis configuration:

```
'redis' => [
    'client' => env('REDIS_CLIENT', 'phpredis'),

    'clusters' => [
        'default' => [
            [
                'host' => env('REDIS_HOST', 'localhost'),
                'password' => env('REDIS_PASSWORD', null),
                'port' => env('REDIS_PORT', 6379),
                'database' => 0,
            ],
        ],
    ],
],
```

By default, clusters will perform client-side sharding across your nodes, allowing you to pool nodes and create a large amount of available RAM. However, note that client-side sharding does not handle failover; therefore, is primarily suited for cached data that is available from another primary data store. If you would like to use native Redis clustering, you should specify this in the `options` key of your Redis configuration:

```
'redis' => [
    'client' => env('REDIS_CLIENT', 'phpredis'),

    'options' => [
        'cluster' => env('REDIS_CLUSTER', 'redis'),
    ],

    'clusters' => [
        // ...
    ],
],
```

Predis

To utilize the Predis extension, you should change the `REDIS_CLIENT` environment variable from `phpredis` to `predis`:

```
'redis' => [
    'client' => env('REDIS_CLIENT', 'predis'),
```

```
    // Rest of Redis configuration...
],
```

In addition to the default host, port, database, and password server configuration options, Predis supports additional [connection parameters](#) that may be defined for each of your Redis servers. To utilize these additional configuration options, add them to your Redis server configuration in the `config/database.php` configuration file:

```
'default' => [
    'host' => env('REDIS_HOST', 'localhost'),
    'password' => env('REDIS_PASSWORD', null),
    'port' => env('REDIS_PORT', 6379),
    'database' => 0,
    'read_write_timeout' => 60,
],
```

PhpRedis

The PhpRedis extension is configured as default at `REDIS_CLIENT` env and in your `config/database.php`:

```
'redis' => [

    'client' => env('REDIS_CLIENT', 'predis'),

    // Rest of Redis configuration...
],
```

If you plan to use PhpRedis extension along with the Redis Facade alias, you should rename it to something else, like `RedisManager`, to avoid a collision with the `Redis` class. You can do that in the `aliases` section of your `app.php` config file.

```
'RedisManager' => Illuminate\Support\Facades\Redis::class,
```

In addition to the default host, port, database, and password server configuration options, PhpRedis supports the following additional connection parameters: `persistent`, `prefix`, `read_timeout`, `timeout`, and `context`. You may add any of these options to your Redis server configuration in the `config/database.php` configuration file:

```
'default' => [
    'host' => env('REDIS_HOST', 'localhost'),
    'password' => env('REDIS_PASSWORD', null),
    'port' => env('REDIS_PORT', 6379),
    'database' => 0,
    'read_timeout' => 60,
    'context' => [
        // 'auth' => ['username', 'secret'],
        // 'stream' => ['verify_peer' => false],
    ],
],
```

The Redis Facade

To avoid class naming collisions with the Redis PHP extension itself, you will need to delete or rename the `Illuminate\Support\Facades\Redis` facade alias from your app configuration file's `aliases` array. Generally, you should remove this alias entirely and only reference the facade by its fully qualified class name while using the Redis PHP extension.

Interacting With Redis

You may interact with Redis by calling various methods on the `Redis` [facade](#). The `Redis` facade supports dynamic methods, meaning you may call any [Redis command](#) on the facade and the command will be passed directly to Redis. In this example, we will call the Redis `GET` command by calling the `get` method on the `Redis` facade:

```
<?php

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;
```

```
use Illuminate\Support\Facades\Redis;

class UserController extends Controller
{
    /**
     * Show the profile for the given user.
     *
     * @param int $id
     * @return Response
     */
    public function showProfile($id)
    {
        $user = Redis::get('user:profile:'.$id);

        return view('user.profile', ['user' => $user]);
    }
}
```

As mentioned above, you may call any of the Redis commands on the `Redis` facade. Laravel uses magic methods to pass the commands to the Redis server, so pass the arguments the Redis command expects:

```
Redis::set('name', 'Taylor');

$values = Redis::lrange('names', 5, 10);
```

Alternatively, you may also pass commands to the server using the `command` method, which accepts the name of the command as its first argument, and an array of values as its second argument:

```
$values = Redis::command('lrange', ['name', 5, 10]);
```

Using Multiple Redis Connections

You may get a Redis instance by calling the `Redis::connection` method:

```
$redis = Redis::connection();
```

This will give you an instance of the default Redis server. You may also pass the connection or cluster name to the `connection` method to get a specific server or cluster as defined in your Redis configuration:

```
$redis = Redis::connection('my-connection');
```

Pipelining Commands

Pipelining should be used when you need to send many commands to the server. The `pipeline` method accepts one argument: a `closure` that receives a Redis instance. You may issue all of your commands to this Redis instance and they will all be streamed to the server thus providing better performance:

```
Redis::pipeline(function ($pipe) {
    for ($i = 0; $i < 1000; $i++) {
        $pipe->set("key:$i", $i);
    }
});
```

Pub / Sub

Laravel provides a convenient interface to the Redis `publish` and `subscribe` commands. These Redis commands allow you to listen for messages on a given "channel". You may publish messages to the channel from another application, or even using another programming language, allowing easy communication between applications and processes.

First, let's setup a channel listener using the `subscribe` method. We'll place this method call within an [Artisan command](#) since calling the `subscribe` method begins a long-running process:

```
<?php

namespace App\Console\Commands;

use Illuminate\Console\Command;
use Illuminate\Support\Facades\Redis;
```

```
class RedisSubscribe extends Command
{
    /**
     * The name and signature of the console command.
     *
     * @var string
     */
    protected $signature = 'redis:subscribe';

    /**
     * The console command description.
     *
     * @var string
     */
    protected $description = 'Subscribe to a Redis channel';

    /**
     * Execute the console command.
     *
     * @return mixed
     */
    public function handle()
    {
        Redis::subscribe(['test-channel'], function ($message) {
            echo $message;
        });
    }
}
```

Now we may publish messages to the channel using the `publish` method:

```
Route::get('publish', function () {
    // Route logic...

    Redis::publish('test-channel', json_encode(['foo' => 'bar']));
});
```

Wildcard Subscriptions

Using the `psubscribe` method, you may subscribe to a wildcard channel, which may be useful for catching all messages on all channels. The `$channel` name will be passed as the second argument to the provided callback closure:

```
Redis::psubscribe(['*'], function ($message, $channel) {
    echo $message;
});

Redis::psubscribe(['users.*'], function ($message, $channel) {
    echo $message;
});
```

Eloquent ORM

Getting Started

Eloquent: Getting Started

- [Introduction](#)
- [Defining Models](#)
 - [Eloquent Model Conventions](#)
 - [Default Attribute Values](#)
- [Retrieving Models](#)
 - [Collections](#)
 - [Chunking Results](#)
 - [Advanced Subqueries](#)
- [Retrieving Single Models / Aggregates](#)
 - [Retrieving Aggregates](#)
- [Inserting & Updating Models](#)
 - [Inserts](#)
 - [Updates](#)
 - [Mass Assignment](#)
 - [Other Creation Methods](#)
- [Deleting Models](#)
 - [Soft Deleting](#)
 - [Querying Soft Deleted Models](#)
- [Replicating Models](#)
- [Query Scopes](#)
 - [Global Scopes](#)
 - [Local Scopes](#)
- [Comparing Models](#)
- [Events](#)
 - [Using Closures](#)
 - [Observers](#)
 - [Muting Events](#)

Introduction

The Eloquent ORM included with Laravel provides a beautiful, simple ActiveRecord implementation for working with your database. Each database table has a corresponding "Model" which is used to interact with that table. Models allow you to query for data in your tables, as well as insert new records into the table.

Before getting started, be sure to configure a database connection in `config/database.php`. For more information on configuring your database, check out [the documentation](#).

Defining Models

To get started, let's create an Eloquent model. Models typically live in the `app` directory, but you are free to place them anywhere that can be auto-loaded according to your `composer.json` file. All Eloquent models extend `Illuminate\Database\Eloquent\Model` class.

The easiest way to create a model instance is using the `make:model` [Artisan command](#):

```
php artisan make:model Flight
```

If you would like to generate a [database migration](#) when you generate the model, you may use the `--migration` or `-m` option:

```
php artisan make:model Flight --migration
```

```
php artisan make:model Flight -m
```

Eloquent Model Conventions

Now, let's look at an example `Flight` model, which we will use to retrieve and store information from our `flights` database table:

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;

class Flight extends Model
{
    //
}
```

Table Names

Note that we did not tell Eloquent which table to use for our `Flight` model. By convention, the "snake case", plural name of the class will be used as the table name unless another name is explicitly specified. So, in this case, Eloquent will assume the `Flight` model stores records in the `flights` table. You may specify a custom table by defining a `table` property on your model:

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;

class Flight extends Model
{
    /**
     * The table associated with the model.
     *
     * @var string
     */
    protected $table = 'my_flights';
}
```

Primary Keys

Eloquent will also assume that each table has a primary key column named `id`. You may define a protected `$primaryKey` property to override this convention:

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;

class Flight extends Model
{
    /**
     * The primary key associated with the table.
     *
     * @var string
     */
    protected $primaryKey = 'flight_id';
}
```

In addition, Eloquent assumes that the primary key is an incrementing integer value, which means that by default the primary key will automatically be cast to an `int`. If you wish to use a non-incrementing or a non-numeric primary key you must set the public `$incrementing` property on your model to `false`:

```
<?php

class Flight extends Model
{
    /**
     * Indicates if the IDs are auto-incrementing.
     *
     * @var bool
     */
}
```

```
    */
    public $incrementing = false;
}
```

If your primary key is not an integer, you should set the protected `$keyType` property on your model to `string`:

```
<?php

class Flight extends Model
{
    /**
     * The "type" of the auto-incrementing ID.
     *
     * @var string
     */
    protected $keyType = 'string';
}
```

Timestamps

By default, Eloquent expects `created_at` and `updated_at` columns to exist on your tables. If you do not wish to have these columns automatically managed by Eloquent, set the `$timestamps` property on your model to `false`:

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;

class Flight extends Model
{
    /**
     * Indicates if the model should be timestamped.
     *
     * @var bool
     */
    public $timestamps = false;
}
```

If you need to customize the format of your timestamps, set the `$dateFormat` property on your model. This property determines how date attributes are stored in the database, as well as their format when the model is serialized to an array or JSON:

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;

class Flight extends Model
{
    /**
     * The storage format of the model's date columns.
     *
     * @var string
     */
    protected $dateFormat = 'U';
}
```

If you need to customize the names of the columns used to store the timestamps, you may set the `CREATED_AT` and `UPDATED_AT` constants in your model:

```
<?php

class Flight extends Model
{
    const CREATED_AT = 'creation_date';
    const UPDATED_AT = 'last_update';
}
```

Database Connection

By default, all Eloquent models will use the default database connection configured for your application. If you would like to specify a different connection for the model, use the `$connection` property:


```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;

class Flight extends Model
{
    /**
     * The connection name for the model.
     *
     * @var string
     */
    protected $connection = 'connection-name';
}
```

Default Attribute Values

If you would like to define the default values for some of your model's attributes, you may define an `$attributes` property on your model:

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;

class Flight extends Model
{
    /**
     * The model's default values for attributes.
     *
     * @var array
     */
    protected $attributes = [
        'delayed' => false,
    ];
}
```

Retrieving Models

Once you have created a model and [its associated database table](#), you are ready to start retrieving data from your database. Think of each Eloquent model as a powerful [query builder](#) allowing you to fluently query the database table associated with the model. For example:

```
<?php

$flights = App\Flight::all();

foreach ($flights as $flight) {
    echo $flight->name;
}
```

Adding Additional Constraints

The Eloquent `all` method will return all of the results in the model's table. Since each Eloquent model serves as a [query builder](#), you may also add constraints to queries, and then use the `get` method to retrieve the results:

```
$flights = App\Flight::where('active', 1)
    ->orderBy('name', 'desc')
    ->take(10)
    ->get();
```

TIP Since Eloquent models are query builders, you should review all of the methods available on the [query builder](#). You may use any of these methods in your Eloquent queries.

Refreshing Models

You can refresh models using the `fresh` and `refresh` methods. The `fresh` method will re-retrieve the model from the database. The existing model instance will not be affected:

```
$flight = App\Flight::where('number', 'FR 900')->first();  
$freshFlight = $flight->fresh();
```

The refresh method will re-hydrate the existing model using fresh data from the database. In addition, all of its loaded relationships will be refreshed as well:

```
$flight = App\Flight::where('number', 'FR 900')->first();  
$flight->number = 'FR 456';  
$flight->refresh();  
$flight->number; // "FR 900"
```

Collections

For Eloquent methods like `all` and `get` which retrieve multiple results, an instance of `Illuminate\Database\Eloquent\Collection` will be returned. The `Collection` class provides [a variety of helpful methods](#) for working with your Eloquent results:

```
$flights = $flights->reject(function ($flight) {  
    return $flight->cancelled;  
});
```

You may also loop over the collection like an array:

```
foreach ($flights as $flight) {  
    echo $flight->name;  
}
```

Chunking Results

If you need to process thousands of Eloquent records, use the `chunk` command. The `chunk` method will retrieve a "chunk" of Eloquent models, feeding them to a given closure for processing. Using the `chunk` method will conserve memory when working with large result sets:

```
Flight::chunk(200, function ($flights) {  
    foreach ($flights as $flight) {  
        //  
    }  
});
```

The first argument passed to the method is the number of records you wish to receive per "chunk". The Closure passed as the second argument will be called for each chunk that is retrieved from the database. A database query will be executed to retrieve each chunk of records passed to the Closure.

Using Cursors

The `cursor` method allows you to iterate through your database records using a cursor, which will only execute a single query. When processing large amounts of data, the `cursor` method may be used to greatly reduce your memory usage:

```
foreach (Flight::where('foo', 'bar')->cursor() as $flight) {  
    //  
}
```

The cursor returns an `Illuminate\Support\LazyCollection` instance. [Lazy collections](#) allow you to use many of collection methods available on typical Laravel collections while only loading a single model into memory at a time:

```
$users = App\User::cursor()->filter(function ($user) {  
    return $user->id > 500;  
});  
  
foreach ($users as $user) {  
    echo $user->id;  
}
```

Advanced Subqueries

Subquery Selects

Eloquent also offers advanced subquery support, which allows you to pull information from related tables in a single query. For example, let's imagine that we have a table of `flight_destinations` and a table of `flights` to destinations. The `flights` table contains an `arrived_at` column which indicates when the flight arrived at the destination.

Using the subquery functionality available to the `select` and `addSelect` methods, we can select all of the destinations and the name of the flight that most recently arrived at that destination using a single query:

```
use App\Destination;
use App\Flight;

return Destination::addSelect(['last_flight' => Flight::select('name')
    ->whereColumn('destination_id', 'destinations.id')
    ->orderBy('arrived_at', 'desc')
    ->limit(1)
])->get();
```

Subquery Ordering

In addition, the query builder's `orderBy` function supports subqueries. We may use this functionality to sort all destinations based on when the last flight arrived at that destination. Again, this may be done while executing a single query against the database:

```
return Destination::orderByDesc(
    Flight::select('arrived_at')
        ->whereColumn('destination_id', 'destinations.id')
        ->orderBy('arrived_at', 'desc')
        ->limit(1)
)->get();
```

Retrieving Single Models / Aggregates

In addition to retrieving all of the records for a given table, you may also retrieve single records using `find`, `first`, or `firstWhere`. Instead of returning a collection of models, these methods return a single model instance:

```
// Retrieve a model by its primary key...
$flight = App\Flight::find(1);

// Retrieve the first model matching the query constraints...
$flight = App\Flight::where('active', 1)->first();

// Shorthand for retrieving the first model matching the query constraints...
$flight = App\Flight::firstWhere('active', 1);
```

You may also call the `find` method with an array of primary keys, which will return a collection of the matching records:

```
$flights = App\Flight::find([1, 2, 3]);
```

Sometimes you may wish to retrieve the first result of a query or perform some other action if no results are found. The `firstor` method will return the first result that is found or, if no results are found, execute the given callback. The result of the callback will be considered the result of the `firstor` method:

```
$model = App\Flight::where('legs', '>', 100)->firstOr(function () {
    // ...
});
```

The `firstor` method also accepts an array of columns to retrieve:

```
$model = App\Flight::where('legs', '>', 100)
    ->firstOr(['id', 'legs'], function () {
        // ...
    });
```

Not Found Exceptions

Sometimes you may wish to throw an exception if a model is not found. This is particularly useful in routes or controllers. The `findOrFail` and `firstOrFail` methods will retrieve the first result of the query; however, if no result is found, a `Illuminate\Database\Eloquent\ModelNotFoundException` will be thrown:

```
$model = App\Flight::findOrFail(1);

$model = App\Flight::where('legs', '>', 100)->firstOrFail();
```

If the exception is not caught, a 404 HTTP response is automatically sent back to the user. It is not necessary to write explicit checks to return 404 responses when using these methods:

```
Route::get('/api/flights/{id}', function ($id) {
    return App\Flight::findOrFail($id);
});
```

Retrieving Aggregates

You may also use the `count`, `sum`, `max`, and other [aggregate methods](#) provided by the [query builder](#). These methods return the appropriate scalar value instead of a full model instance:

```
$count = App\Flight::where('active', 1)->count();

$max = App\Flight::where('active', 1)->max('price');
```

Inserting & Updating Models

Inserts

To create a new record in the database, create a new model instance, set attributes on the model, then call the `save` method:

```
<?php

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;
use App\Flight;
use Illuminate\Http\Request;

class FlightController extends Controller
{
    /**
     * Create a new flight instance.
     *
     * @param Request $request
     * @return Response
     */
    public function store(Request $request)
    {
        // Validate the request...

        $flight = new Flight;

        $flight->name = $request->name;

        $flight->save();
    }
}
```

In this example, we assign the `name` parameter from the incoming HTTP request to the `name` attribute of the `App\Flight` model instance. When we call the `save` method, a record will be inserted into the database. The `created_at` and `updated_at` timestamps will automatically be set when the `save` method is called, so there is no need to set them manually.

Updates

The `save` method may also be used to update models that already exist in the database. To update a model, you should retrieve it, set any attributes you wish to update, and then call the `save` method. Again, the `updated_at` timestamp will automatically be updated, so there is no need to manually set its value:

```
$flight = App\Flight::find(1);  
$flight->name = 'New Flight Name';  
$flight->save();
```

Mass Updates

Updates can also be performed against any number of models that match a given query. In this example, all flights that are active and have a destination of San Diego will be marked as delayed:

```
App\Flight::where('active', 1)  
    ->where('destination', 'San Diego')  
    ->update(['delayed' => 1]);
```

The `update` method expects an array of column and value pairs representing the columns that should be updated.

NOTE When issuing a mass update via Eloquent, the `saving`, `saved`, `updating`, and `updated` model events will not be fired for the updated models. This is because the models are never actually retrieved when issuing a mass update.

Examining Attribute Changes

Eloquent provides the `isDirty`, `isClean`, and `wasChanged` methods to examine the internal state of your model and determine how its attributes have changed from when they were originally loaded.

The `isDirty` method determines if any attributes have been changed since the model was loaded. You may pass a specific attribute name to determine if a particular attribute is dirty. The `isClean` method is the opposite of `isDirty` and also accepts an optional attribute argument:

```
$user = User::create([  
    'first_name' => 'Taylor',  
    'last_name' => 'Otwell',  
    'title' => 'Developer',  
]);  
  
$user->title = 'Painter';  
  
$user->isDirty(); // true  
$user->isDirty('title'); // true  
$user->isDirty('first_name'); // false  
  
$user->isClean(); // false  
$user->isClean('title'); // false  
$user->isClean('first_name'); // true  
  
$user->save();  
  
$user->isDirty(); // false  
$user->isClean(); // true
```

The `wasChanged` method determines if any attributes were changed when the model was last saved within the current request cycle. You may also pass an attribute name to see if a particular attribute was changed:

```
$user = User::create([  
    'first_name' => 'Taylor',  
    'last_name' => 'Otwell',  
    'title' => 'Developer',  
]);  
  
$user->title = 'Painter';  
$user->save();  
  
$user->wasChanged(); // true  
$user->wasChanged('title'); // true  
$user->wasChanged('first_name'); // false
```

The `getOriginal` method returns an array containing the original attributes of the model regardless of any changes since the model was loaded. You may pass a specific attribute name to get the original value of a particular attribute:

```
$user = User::find(1);

$user->name; // John
$user->email; // john@example.com

$user->name = "Jack";
$user->name; // Jack

$user->getOriginal('name'); // John
$user->getOriginal(); // Array of original attributes...
```

Mass Assignment

You may also use the `create` method to save a new model in a single line. The inserted model instance will be returned to you from the method. However, before doing so, you will need to specify either a `fillable` or `guarded` attribute on the model, as all Eloquent models protect against mass-assignment by default.

A mass-assignment vulnerability occurs when a user passes an unexpected HTTP parameter through a request, and that parameter changes a column in your database you did not expect. For example, a malicious user might send an `is_admin` parameter through an HTTP request, which is then passed into your model's `create` method, allowing the user to escalate themselves to an administrator.

So, to get started, you should define which model attributes you want to make mass assignable. You may do this using the `fillable` property on the model. For example, let's make the `name` attribute of our `Flight` model mass assignable:

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;

class Flight extends Model
{
    /**
     * The attributes that are mass assignable.
     *
     * @var array
     */
    protected $fillable = ['name'];
}
```

Once we have made the attributes mass assignable, we can use the `create` method to insert a new record in the database. The `create` method returns the saved model instance:

```
$flight = App\Flight::create(['name' => 'Flight 10']);
```

If you already have a model instance, you may use the `fill` method to populate it with an array of attributes:

```
$flight->fill(['name' => 'Flight 22']);
```

Allowing Mass Assignment

If you would like to make all attributes mass assignable, you may define the `$guarded` property as an empty array:

```
/**
 * The attributes that aren't mass assignable.
 *
 * @var array
 */
protected $guarded = [];
```

Other Creation Methods

firstOrCreate/ firstOrCreateNew

There are two other methods you may use to create models by mass assigning attributes: `firstOrCreate` and `firstOrCreateNew`. The `firstOrCreate` method will attempt to locate a database record using the given column / value pairs. If the model can not be found in the database, a record will be inserted with the attributes from the first parameter, along with those in the optional second parameter.

The `firstOrCreateNew` method, like `firstOrCreate` will attempt to locate a record in the database matching the given attributes. However, if a model is not found, a new model instance will be returned. Note that the model returned by `firstOrCreateNew` has not yet been persisted to the database. You will need to call `save` manually to persist it:

```
// Retrieve flight by name, or create it if it doesn't exist...
$flight = App\Flight::firstOrCreate(['name' => 'Flight 10']);

// Retrieve flight by name, or create it with the name, delayed, and arrival_time attributes...
$flight = App\Flight::firstOrCreate(
    ['name' => 'Flight 10'],
    ['delayed' => 1, 'arrival_time' => '11:30']
);

// Retrieve by name, or instantiate...
$flight = App\Flight::firstOrCreate(['name' => 'Flight 10']);

// Retrieve by name, or instantiate with the name, delayed, and arrival_time attributes...
$flight = App\Flight::firstOrCreate(
    ['name' => 'Flight 10'],
    ['delayed' => 1, 'arrival_time' => '11:30']
);
```

updateOrCreate

You may also come across situations where you want to update an existing model or create a new model if none exists. Laravel provides an `updateOrCreate` method to do this in one step. Like the `firstOrCreate` method, `updateOrCreate` persists the model, so there's no need to call `save()`:

```
// If there's a flight from Oakland to San Diego, set the price to $99.
// If no matching model exists, create one.
$flight = App\Flight::updateOrCreate(
    ['departure' => 'Oakland', 'destination' => 'San Diego'],
    ['price' => 99, 'discounted' => 1]
);
```

Deleting Models

To delete a model, call the `delete` method on a model instance:

```
$flight = App\Flight::find(1);

$flight->delete();
```

Deleting An Existing Model By Key

In the example above, we are retrieving the model from the database before calling the `delete` method. However, if you know the primary key of the model, you may delete the model without explicitly retrieving it by calling the `destroy` method. In addition to a single primary key as its argument, the `destroy` method will accept multiple primary keys, an array of primary keys, or a [collection](#) of primary keys:

```
App\Flight::destroy(1);

App\Flight::destroy(1, 2, 3);

App\Flight::destroy([1, 2, 3]);

App\Flight::destroy(collect([1, 2, 3]));
```

NOTE The `destroy` method loads each model individually and calls the `delete` method on them so that the `deleting` and `deleted` events are fired.

Deleting Models By Query

You can also run a delete statement on a set of models. In this example, we will delete all flights that are marked as inactive. Like mass updates, mass deletes will not fire any model events for the models that are deleted:

```
$deletedRows = App\Flight::where('active', 0)->delete();
```

NOTE When executing a mass delete statement via Eloquent, the `deleting` and `deleted` model events will not be fired for the deleted models. This is because the models are never actually retrieved when executing the delete statement.

Soft Deleting

In addition to actually removing records from your database, Eloquent can also "soft delete" models. When models are soft deleted, they are not actually removed from your database. Instead, a `deleted_at` attribute is set on the model and inserted into the database. If a model has a non-null `deleted_at` value, the model has been soft deleted. To enable soft deletes for a model, use the `Illuminate\Database\Eloquent\SoftDeletes` trait on the model:

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\SoftDeletes;

class Flight extends Model
{
    use SoftDeletes;
}
```

TIP The `SoftDeletes` trait will automatically cast the `deleted_at` attribute to a `DateTime` / `Carbon` instance for you.

You should also add the `deleted_at` column to your database table. The Laravel [schema builder](#) contains a helper method to create this column:

```
public function up()
{
    Schema::table('flights', function (Blueprint $table) {
        $table->softDeletes();
    });
}

public function down()
{
    Schema::table('flights', function (Blueprint $table) {
        $table->dropSoftDeletes();
    });
}
```

Now, when you call the `delete` method on the model, the `deleted_at` column will be set to the current date and time. And, when querying a model that uses soft deletes, the soft deleted models will automatically be excluded from all query results.

To determine if a given model instance has been soft deleted, use the `trashed` method:

```
if ($flight->trashed()) {
    //
}
```

Querying Soft Deleted Models

Including Soft Deleted Models

As noted above, soft deleted models will automatically be excluded from query results. However, you may

force soft deleted models to appear in a result set using the `withTrashed` method on the query:

```
$flights = App\Flight::withTrashed()
    ->where('account_id', 1)
    ->get();
```

The `withTrashed` method may also be used on a [relationship](#) query:

```
$flight->history()->withTrashed()->get();
```

Retrieving Only Soft Deleted Models

The `onlyTrashed` method will retrieve **only** soft deleted models:

```
$flights = App\Flight::onlyTrashed()
    ->where('airline_id', 1)
    ->get();
```

Restoring Soft Deleted Models

Sometimes you may wish to "un-delete" a soft deleted model. To restore a soft deleted model into an active state, use the `restore` method on a model instance:

```
$flight->restore();
```

You may also use the `restore` method in a query to quickly restore multiple models. Again, like other "mass" operations, this will not fire any model events for the models that are restored:

```
App\Flight::withTrashed()
    ->where('airline_id', 1)
    ->restore();
```

Like the `withTrashed` method, the `restore` method may also be used on [relationships](#):

```
$flight->history()->restore();
```

Permanently Deleting Models

Sometimes you may need to truly remove a model from your database. To permanently remove a soft deleted model from the database, use the `forceDelete` method:

```
// Force deleting a single model instance...
$flight->forceDelete();

// Force deleting all related models...
$flight->history()->forceDelete();
```

Replicating Models

You may create an unsaved copy of a model instance using the `replicate` method. This is particularly useful when you have model instances that share many of the same attributes:

```
$shipping = App\Address::create([
    'type' => 'shipping',
    'line_1' => '123 Example Street',
    'city' => 'Victorville',
    'state' => 'CA',
    'postcode' => '90001',
]);

$billing = $shipping->replicate()->fill([
    'type' => 'billing'
]);

$billing->save();
```

Query Scopes

Global Scopes

Global scopes allow you to add constraints to all queries for a given model. Laravel's own [soft delete](#) functionality utilizes global scopes to only pull "non-deleted" models from the database. Writing your own global scopes can provide a convenient, easy way to make sure every query for a given model receives certain constraints.

Writing Global Scopes

Writing a global scope is simple. Define a class that implements the `Illuminate\Database\Eloquent\Scope` interface. This interface requires you to implement one method: `apply`. The `apply` method may add `where` constraints to the query as needed:

```
<?php

namespace App\Scopes;

use Illuminate\Database\Eloquent\Builder;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Scope;

class AgeScope implements Scope
{
    /**
     * Apply the scope to a given Eloquent query builder.
     *
     * @param \Illuminate\Database\Eloquent\Builder $builder
     * @param \Illuminate\Database\Eloquent\Model $model
     * @return void
     */
    public function apply(Builder $builder, Model $model)
    {
        $builder->where('age', '>', 200);
    }
}
```

TIP If your global scope is adding columns to the select clause of the query, you should use the `addSelect` method instead of `select`. This will prevent the unintentional replacement of the query's existing select clause.

Applying Global Scopes

To assign a global scope to a model, you should override a given model's `booted` method and use the `addGlobalScope` method:

```
<?php

namespace App;

use App\Scopes\AgeScope;
use Illuminate\Database\Eloquent\Model;

class User extends Model
{
    /**
     * The "booted" method of the model.
     *
     * @return void
     */
    protected static function booted()
    {
        static::addGlobalScope(new AgeScope);
    }
}
```

After adding the scope, a query to `User::all()` will produce the following SQL:

```
select * from `users` where `age` > 200
```

Anonymous Global Scopes

Eloquent also allows you to define global scopes using Closures, which is particularly useful for simple scopes that do not warrant a separate class:

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Builder;
use Illuminate\Database\Eloquent\Model;

class User extends Model
{
    /**
     * The "booted" method of the model.
     *
     * @return void
     */
    protected static function booted()
    {
        static::addGlobalScope('age', function (Builder $builder) {
            $builder->where('age', '>', 200);
        });
    }
}
```

Removing Global Scopes

If you would like to remove a global scope for a given query, you may use the `withoutGlobalScope` method. The method accepts the class name of the global scope as its only argument:

```
User::withoutGlobalScope(AgeScope::class)->get();
```

Or, if you defined the global scope using a Closure:

```
User::withoutGlobalScope('age')->get();
```

If you would like to remove several or even all of the global scopes, you may use the `withoutGlobalScopes` method:

```
// Remove all of the global scopes...
User::withoutGlobalScopes()->get();

// Remove some of the global scopes...
User::withoutGlobalScopes([
    FirstScope::class, SecondScope::class
])->get();
```

Local Scopes

Local scopes allow you to define common sets of constraints that you may easily re-use throughout your application. For example, you may need to frequently retrieve all users that are considered "popular". To define a scope, prefix an Eloquent model method with `scope`.

Scopes should always return a query builder instance:

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;

class User extends Model
{
    /**
     * Scope a query to only include popular users.
     *
     * @param \Illuminate\Database\Eloquent\Builder $query
     * @return \Illuminate\Database\Eloquent\Builder
     */
    public function scopePopular($query)
    {
        return $query->where('votes', '>', 100);
    }
}
```

```
/**
 * Scope a query to only include active users.
 *
 * @param \Illuminate\Database\Eloquent\Builder $query
 * @return \Illuminate\Database\Eloquent\Builder
 */
public function scopeActive($query)
{
    return $query->where('active', 1);
}
}
```

Utilizing A Local Scope

Once the scope has been defined, you may call the scope methods when querying the model. However, you should not include the `scope` prefix when calling the method. You can even chain calls to various scopes, for example:

```
$users = App\User::popular()->active()->orderBy('created_at')->get();
```

Combining multiple Eloquent model scopes via an `or` query operator may require the use of Closure callbacks:

```
$users = App\User::popular()->orWhere(function (Builder $query) {
    $query->active();
})->get();
```

However, since this can be cumbersome, Laravel provides a "higher order" `orWhere` method that allows you to fluently chain these scopes together without the use of Closures:

```
$users = App\User::popular()->orWhere->active()->get();
```

Dynamic Scopes

Sometimes you may wish to define a scope that accepts parameters. To get started, just add your additional parameters to your scope. Scope parameters should be defined after the `$query` parameter:

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;

class User extends Model
{
    /**
     * Scope a query to only include users of a given type.
     *
     * @param \Illuminate\Database\Eloquent\Builder $query
     * @param mixed $type
     * @return \Illuminate\Database\Eloquent\Builder
     */
    public function scopeOfType($query, $type)
    {
        return $query->where('type', $type);
    }
}
```

Now, you may pass the parameters when calling the scope:

```
$users = App\User::ofTypes('admin')->get();
```

Comparing Models

Sometimes you may need to determine if two models are the "same". The `is` method may be used to quickly verify two models have same primary key, table, and database connection:

```
if ($post->is($anotherPost)) {
    //
}
```

Events

Eloquent models fire several events, allowing you to hook into the following points in a model's lifecycle: retrieved, creating, created, updating, updated, saving, saved, deleting, deleted, restoring, restored. Events allow you to easily execute code each time a specific model class is saved or updated in the database. Each event receives the instance of the model through its constructor.

The `retrieved` event will fire when an existing model is retrieved from the database. When a new model is saved for the first time, the `creating` and `created` events will fire. The `updating` / `updated` events will fire when an existing model is modified and the `save` method is called. The `saving` / `saved` events will fire when a model is created or updated.

NOTE When issuing a mass update or delete via Eloquent, the `saved`, `updated`, `deleting`, and `deleted` model events will not be fired for the affected models. This is because the models are never actually retrieved when issuing a mass update or delete.

To get started, define a `$dispatchesEvents` property on your Eloquent model that maps various points of the Eloquent model's lifecycle to your own [event classes](#):

```
<?php

namespace App;

use App\Events\UserDeleted;
use App\Events\UserSaved;
use Illuminate\Foundation\Auth\User as Authenticatable;

class User extends Authenticatable
{
    use Notifiable;

    /**
     * The event map for the model.
     *
     * @var array
     */
    protected $dispatchesEvents = [
        'saved' => UserSaved::class,
        'deleted' => UserDeleted::class,
    ];
}
```

After defining and mapping your Eloquent events, you may use [event listeners](#) to handle the events.

Using Closures

Instead of using custom event classes, you may register Closures that execute when various model events are fired. Typically, you should register these Closures in the `booted` method of your model:

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;

class User extends Model
{
    /**
     * The "booted" method of the model.
     *
     * @return void
     */
    protected static function booted()
    {
        static::created(function ($user) {
            //
        });
    }
}
```

Observers

Defining Observers

If you are listening for many events on a given model, you may use observers to group all of your listeners into a single class. Observers classes have method names which reflect the Eloquent events you wish to listen for. Each of these methods receives the model as their only argument. The `make:observer` Artisan command is the easiest way to create a new observer class:

```
php artisan make:observer UserObserver --model=User
```

This command will place the new observer in your `App/Observers` directory. If this directory does not exist, Artisan will create it for you. Your fresh observer will look like the following:

```
<?php

namespace App\Observers;

use App\User;

class UserObserver
{
    /**
     * Handle the User "created" event.
     *
     * @param \App\User $user
     * @return void
     */
    public function created(User $user)
    {
        //
    }

    /**
     * Handle the User "updated" event.
     *
     * @param \App\User $user
     * @return void
     */
    public function updated(User $user)
    {
        //
    }

    /**
     * Handle the User "deleted" event.
     *
     * @param \App\User $user
     * @return void
     */
    public function deleted(User $user)
    {
        //
    }

    /**
     * Handle the User "forceDeleted" event.
     *
     * @param \App\User $user
     * @return void
     */
    public function forceDeleted(User $user)
    {
        //
    }
}
```

To register an observer, use the `observe` method on the model you wish to observe. You may register observers in the `boot` method of one of your service providers. In this example, we'll register the observer in the `AppServiceProvider`:

```
<?php

namespace App\Providers;
```

```
use App\Observers\UserObserver;
use App\User;
use Illuminate\Support\ServiceProvider;

class AppServiceProvider extends ServiceProvider
{
    /**
     * Register any application services.
     *
     * @return void
     */
    public function register()
    {
        //
    }

    /**
     * Bootstrap any application services.
     *
     * @return void
     */
    public function boot()
    {
        User::observe(UserObserver::class);
    }
}
```

Muting Events

You may occasionally wish to temporarily "mute" all events fired by a model. You may achieve this using the `withoutEvents` method. The `withoutEvents` method accepts a Closure as its only argument. Any code executed within this Closure will not fire model events. For example, the following will fetch and delete an `App\User` instance without firing any model events. Any value returned by the given Closure will be returned by the `withoutEvents` method:

```
use App\User;

$user = User::withoutEvents(function () use () {
    User::findOrFail(1)->delete();

    return User::find(2);
});
```

Eloquent ORM

Relationships

Eloquent: Relationships

- [Introduction](#)
- [Defining Relationships](#)
 - [One To One](#)
 - [One To Many](#)
 - [One To Many \(Inverse\)](#)
 - [Many To Many](#)
 - [Defining Custom Intermediate Table Models](#)
 - [Has One Through](#)
 - [Has Many Through](#)
- [Polymorphic Relationships](#)
 - [One To One](#)
 - [One To Many](#)
 - [Many To Many](#)
 - [Custom Polymorphic Types](#)
- [Dynamic Relationships](#)
- [Querying Relations](#)
 - [Relationship Methods Vs. Dynamic Properties](#)
 - [Querying Relationship Existence](#)
 - [Querying Relationship Absence](#)
 - [Querying Polymorphic Relationships](#)
 - [Counting Related Models](#)
 - [Counting Related Models On Polymorphic Relationships](#)
- [Eager Loading](#)
 - [Constraining Eager Loads](#)
 - [Lazy Eager Loading](#)
- [Inserting & Updating Related Models](#)
 - [The save Method](#)
 - [The create Method](#)
 - [Belongs To Relationships](#)
 - [Many To Many Relationships](#)
- [Touching Parent Timestamps](#)

Introduction

Database tables are often related to one another. For example, a blog post may have many comments, or an order could be related to the user who placed it. Eloquent makes managing and working with these relationships easy, and supports several different types of relationships:

- [One To One](#)
- [One To Many](#)
- [Many To Many](#)
- [Has One Through](#)
- [Has Many Through](#)
- [One To One \(Polymorphic\)](#)
- [One To Many \(Polymorphic\)](#)
- [Many To Many \(Polymorphic\)](#)

Defining Relationships

Eloquent relationships are defined as methods on your Eloquent model classes. Since, like Eloquent models themselves, relationships also serve as powerful [query builders](#), defining relationships as methods provides powerful method chaining and querying capabilities. For example, we may chain additional constraints on this

posts relationship:

```
$user->posts()->where('active', 1)->get();
```

But, before diving too deep into using relationships, let's learn how to define each type.

NOTE Relationship names cannot collide with attribute names as that could lead to your model not being able to know which one to resolve.

One To One

A one-to-one relationship is a very basic relation. For example, a user model might be associated with one Phone. To define this relationship, we place a phone method on the user model. The phone method should call the `hasOne` method and return its result:

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;

class User extends Model
{
    /**
     * Get the phone record associated with the user.
     */
    public function phone()
    {
        return $this->hasOne('App\Phone');
    }
}
```

The first argument passed to the `hasOne` method is the name of the related model. Once the relationship is defined, we may retrieve the related record using Eloquent's dynamic properties. Dynamic properties allow you to access relationship methods as if they were properties defined on the model:

```
$phone = User::find(1)->phone;
```

Eloquent determines the foreign key of the relationship based on the model name. In this case, the `Phone` model is automatically assumed to have a `user_id` foreign key. If you wish to override this convention, you may pass a second argument to the `hasOne` method:

```
return $this->hasOne('App\Phone', 'foreign_key');
```

Additionally, Eloquent assumes that the foreign key should have a value matching the `id` (or the custom `$primaryKey`) column of the parent. In other words, Eloquent will look for the value of the user's `id` column in the `user_id` column of the `Phone` record. If you would like the relationship to use a value other than `id`, you may pass a third argument to the `hasOne` method specifying your custom key:

```
return $this->hasOne('App\Phone', 'foreign_key', 'local_key');
```

Defining The Inverse Of The Relationship

So, we can access the `Phone` model from our `User`. Now, let's define a relationship on the `Phone` model that will let us access the `User` that owns the phone. We can define the inverse of a `hasOne` relationship using the `belongsTo` method:

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;

class Phone extends Model
{
    /**
     * Get the user that owns the phone.
     */
    public function user()
```

```
    {
        return $this->belongsTo('App\User');
    }
}
```

In the example above, Eloquent will try to match the `user_id` from the `Phone` model to an `id` on the `User` model. Eloquent determines the default foreign key name by examining the name of the relationship method and suffixing the method name with `_id`. However, if the foreign key on the `Phone` model is not `user_id`, you may pass a custom key name as the second argument to the `belongsTo` method:

```
/**
 * Get the user that owns the phone.
 */
public function user()
{
    return $this->belongsTo('App\User', 'foreign_key');
}
```

If your parent model does not use `id` as its primary key, or you wish to join the child model to a different column, you may pass a third argument to the `belongsTo` method specifying your parent table's custom key:

```
/**
 * Get the user that owns the phone.
 */
public function user()
{
    return $this->belongsTo('App\User', 'foreign_key', 'other_key');
}
```

One To Many

A one-to-many relationship is used to define relationships where a single model owns any amount of other models. For example, a blog post may have an infinite number of comments. Like all other Eloquent relationships, one-to-many relationships are defined by placing a function on your Eloquent model:

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;

class Post extends Model
{
    /**
     * Get the comments for the blog post.
     */
    public function comments()
    {
        return $this->hasMany('App\Comment');
    }
}
```

Remember, Eloquent will automatically determine the proper foreign key column on the `Comment` model. By convention, Eloquent will take the "snake case" name of the owning model and suffix it with `_id`. So, for this example, Eloquent will assume the foreign key on the `Comment` model is `post_id`.

Once the relationship has been defined, we can access the collection of comments by accessing the `comments` property. Remember, since Eloquent provides "dynamic properties", we can access relationship methods as if they were defined as properties on the model:

```
$comments = App\Post::find(1)->comments;

foreach ($comments as $comment) {
    //
}
```

Since all relationships also serve as query builders, you can add further constraints to which comments are retrieved by calling the `comments` method and continuing to chain conditions onto the query:

```
$comment = App\Post::find(1)->comments()->where('title', 'foo')->first();
```

Like the `hasOne` method, you may also override the foreign and local keys by passing additional arguments to

the `hasMany` method:

```
return $this->hasMany('App\Comment', 'foreign_key');  
return $this->hasMany('App\Comment', 'foreign_key', 'local_key');
```

One To Many (Inverse)

Now that we can access all of a post's comments, let's define a relationship to allow a comment to access its parent post. To define the inverse of a `hasMany` relationship, define a relationship function on the child model which calls the `belongsTo` method:

```
<?php  
  
namespace App;  
  
use Illuminate\Database\Eloquent\Model;  
  
class Comment extends Model  
{  
    /**  
     * Get the post that owns the comment.  
     */  
    public function post()  
    {  
        return $this->belongsTo('App\Post');  
    }  
}
```

Once the relationship has been defined, we can retrieve the `Post` model for a `Comment` by accessing the `post` "dynamic property":

```
$comment = App\Comment::find(1);  
  
echo $comment->post->title;
```

In the example above, Eloquent will try to match the `post_id` from the `Comment` model to an `id` on the `Post` model. Eloquent determines the default foreign key name by examining the name of the relationship method and suffixing the method name with a `_` followed by the name of the primary key column. However, if the foreign key on the `Comment` model is not `post_id`, you may pass a custom key name as the second argument to the `belongsTo` method:

```
/**  
 * Get the post that owns the comment.  
 */  
public function post()  
{  
    return $this->belongsTo('App\Post', 'foreign_key');  
}
```

If your parent model does not use `id` as its primary key, or you wish to join the child model to a different column, you may pass a third argument to the `belongsTo` method specifying your parent table's custom key:

```
/**  
 * Get the post that owns the comment.  
 */  
public function post()  
{  
    return $this->belongsTo('App\Post', 'foreign_key', 'other_key');  
}
```

Many To Many

Many-to-many relations are slightly more complicated than `hasOne` and `hasMany` relationships. An example of such a relationship is a user with many roles, where the roles are also shared by other users. For example, many users may have the role of "Admin".

Table Structure

To define this relationship, three database tables are needed: `users`, `roles`, and `role_user`. The `role_user` table is derived from the alphabetical order of the related model names, and contains the `user_id` and `role_id` columns:

```
users
    id - integer
    name - string

roles
    id - integer
    name - string

role_user
    user_id - integer
    role_id - integer
```

Model Structure

Many-to-many relationships are defined by writing a method that returns the result of the `belongsToMany` method. For example, let's define the `roles` method on our `User` model:

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;

class User extends Model
{
    /**
     * The roles that belong to the user.
     */
    public function roles()
    {
        return $this->belongsToMany('App\Role');
    }
}
```

Once the relationship is defined, you may access the user's roles using the `roles` dynamic property:

```
$user = App\User::find(1);

foreach ($user->roles as $role) {
    //
}
```

Like all other relationship types, you may call the `roles` method to continue chaining query constraints onto the relationship:

```
$roles = App\User::find(1)->roles()->orderBy('name')->get();
```

As mentioned previously, to determine the table name of the relationship's joining table, Eloquent will join the two related model names in alphabetical order. However, you are free to override this convention. You may do so by passing a second argument to the `belongsToMany` method:

```
return $this->belongsToMany('App\Role', 'role_user');
```

In addition to customizing the name of the joining table, you may also customize the column names of the keys on the table by passing additional arguments to the `belongsToMany` method. The third argument is the foreign key name of the model on which you are defining the relationship, while the fourth argument is the foreign key name of the model that you are joining to:

```
return $this->belongsToMany('App\Role', 'role_user', 'user_id', 'role_id');
```

Defining The Inverse Of The Relationship

To define the inverse of a many-to-many relationship, you place another call to `belongsToMany` on your related model. To continue our user roles example, let's define the `users` method on the `Role` model:

```
<?php

namespace App;
```

```
use Illuminate\Database\Eloquent\Model;

class Role extends Model
{
    /**
     * The users that belong to the role.
     */
    public function users()
    {
        return $this->belongsToMany('App\User');
    }
}
```

As you can see, the relationship is defined exactly the same as its `User` counterpart, with the exception of referencing the `App\User` model. Since we're reusing the `belongsToMany` method, all of the usual table and key customization options are available when defining the inverse of many-to-many relationships.

Retrieving Intermediate Table Columns

As you have already learned, working with many-to-many relations requires the presence of an intermediate table. Eloquent provides some very helpful ways of interacting with this table. For example, let's assume our `User` object has many `Role` objects that it is related to. After accessing this relationship, we may access the intermediate table using the `pivot` attribute on the models:

```
$user = App\User::find(1);

foreach ($user->roles as $role) {
    echo $role->pivot->created_at;
}
```

Notice that each `Role` model we retrieve is automatically assigned a `pivot` attribute. This attribute contains a model representing the intermediate table, and may be used like any other Eloquent model.

By default, only the model keys will be present on the `pivot` object. If your pivot table contains extra attributes, you must specify them when defining the relationship:

```
return $this->belongsToMany('App\Role')->withPivot('column1', 'column2');
```

If you want your pivot table to have automatically maintained `created_at` and `updated_at` timestamps, use the `withTimestamps` method on the relationship definition:

```
return $this->belongsToMany('App\Role')->withTimestamps();
```

Customizing The `pivot` Attribute Name

As noted earlier, attributes from the intermediate table may be accessed on models using the `pivot` attribute. However, you are free to customize the name of this attribute to better reflect its purpose within your application.

For example, if your application contains users that may subscribe to podcasts, you probably have a many-to-many relationship between users and podcasts. If this is the case, you may wish to rename your intermediate table accessor to `subscription` instead of `pivot`. This can be done using the `as` method when defining the relationship:

```
return $this->belongsToMany('App\Podcast')
    ->as('subscription')
    ->withTimestamps();
```

Once this is done, you may access the intermediate table data using the customized name:

```
$users = User::with('podcasts')->get();

foreach ($users->flatMap->podcasts as $podcast) {
    echo $podcast->subscription->created_at;
}
```

Filtering Relationships Via Intermediate Table Columns

You can also filter the results returned by `belongsToMany` using the `wherePivot`, `wherePivotIn`, and `wherePivotNotIn` methods when defining the relationship:

```
return $this->belongsToMany('App\Role')->wherePivot('approved', 1);

return $this->belongsToMany('App\Role')->wherePivotIn('priority', [1, 2]);

return $this->belongsToMany('App\Role')->wherePivotNotIn('priority', [1, 2]);
```

Defining Custom Intermediate Table Models

If you would like to define a custom model to represent the intermediate table of your relationship, you may call the `using` method when defining the relationship. Custom many-to-many pivot models should extend the `Illuminate\Database\Eloquent\Relations\Pivot` class while custom polymorphic many-to-many pivot models should extend the `Illuminate\Database\Eloquent\Relations\MorphPivot` class. For example, we may define a `Role` which uses a custom `RoleUser` pivot model:

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;

class Role extends Model
{
    /**
     * The users that belong to the role.
     */
    public function users()
    {
        return $this->belongsToMany('App\User')->using('App\RoleUser');
    }
}
```

When defining the `RoleUser` model, we will extend the `Pivot` class:

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Relations\Pivot;

class RoleUser extends Pivot
{
    //
}
```

You can combine `using` and `withPivot` in order to retrieve columns from the intermediate table. For example, you may retrieve the `created_by` and `updated_by` columns from the `RoleUser` pivot table by passing the column names to the `withPivot` method:

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;

class Role extends Model
{
    /**
     * The users that belong to the role.
     */
    public function users()
    {
        return $this->belongsToMany('App\User')
            ->using('App\RoleUser')
            ->withPivot([
                'created_by',
                'updated_by',
            ]);
    }
}
```

Note: Pivot models may not use the `SoftDeletes` trait. If you need to soft delete pivot records consider

converting your pivot model to an actual Eloquent model.

Custom Pivot Models And Incrementing IDs

If you have defined a many-to-many relationship that uses a custom pivot model, and that pivot model has an auto-incrementing primary key, you should ensure your custom pivot model class defines an `incrementing` property that is set to `true`.

```
/**
 * Indicates if the IDs are auto-incrementing.
 *
 * @var bool
 */
public $incrementing = true;
```

Has One Through

The "has-one-through" relationship links models through a single intermediate relation.

For example, in a vehicle repair shop application, each `Mechanic` may have one car, and each car may have one owner. While the `Mechanic` and the `Owner` have no direct connection, the `Mechanic` can access the `Owner` *through* the car itself. Let's look at the tables necessary to define this relationship:

```
mechanics
  id - integer
  name - string

cars
  id - integer
  model - string
  mechanic_id - integer

owners
  id - integer
  name - string
  car_id - integer
```

Now that we have examined the table structure for the relationship, let's define the relationship on the `Mechanic` model:

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;

class Mechanic extends Model
{
    /**
     * Get the car's owner.
     */
    public function carOwner()
    {
        return $this->hasOneThrough('App\Owner', 'App\Car');
    }
}
```

The first argument passed to the `hasOneThrough` method is the name of the final model we wish to access, while the second argument is the name of the intermediate model.

Typical Eloquent foreign key conventions will be used when performing the relationship's queries. If you would like to customize the keys of the relationship, you may pass them as the third and fourth arguments to the `hasOneThrough` method. The third argument is the name of the foreign key on the intermediate model. The fourth argument is the name of the foreign key on the final model. The fifth argument is the local key, while the sixth argument is the local key of the intermediate model:

```
class Mechanic extends Model
{
    /**
     * Get the car's owner.
     */
```

```
public function carOwner()
{
    return $this->hasOneThrough(
        'App\Owner',
        'App\Car',
        'mechanic_id', // Foreign key on cars table...
        'car_id', // Foreign key on owners table...
        'id', // Local key on mechanics table...
        'id' // Local key on cars table...
    );
}
```

Has Many Through

The "has-many-through" relationship provides a convenient shortcut for accessing distant relations via an intermediate relation. For example, a country model might have many post models through an intermediate user model. In this example, you could easily gather all blog posts for a given country. Let's look at the tables required to define this relationship:

```
countries
  id - integer
  name - string

users
  id - integer
  country_id - integer
  name - string

posts
  id - integer
  user_id - integer
  title - string
```

Though posts does not contain a country_id column, the hasManyThrough relation provides access to a country's posts via \$country->posts. To perform this query, Eloquent inspects the country_id on the intermediate users table. After finding the matching user IDs, they are used to query the posts table.

Now that we have examined the table structure for the relationship, let's define it on the country model:

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;

class Country extends Model
{
    /**
     * Get all of the posts for the country.
     */
    public function posts()
    {
        return $this->hasManyThrough('App\Post', 'App\User');
    }
}
```

The first argument passed to the hasManyThrough method is the name of the final model we wish to access, while the second argument is the name of the intermediate model.

Typical Eloquent foreign key conventions will be used when performing the relationship's queries. If you would like to customize the keys of the relationship, you may pass them as the third and fourth arguments to the hasManyThrough method. The third argument is the name of the foreign key on the intermediate model. The fourth argument is the name of the foreign key on the final model. The fifth argument is the local key, while the sixth argument is the local key of the intermediate model:

```
class Country extends Model
{
    public function posts()
    {
        return $this->hasManyThrough(
            'App\Post',
            'App\User',
            'mechanic_id',
            'car_id',
            'id',
            'id'
        );
    }
}
```



```
        'country_id', // Foreign key on users table...
        'user_id', // Foreign key on posts table...
        'id', // Local key on countries table...
        'id' // Local key on users table...
    );
}
```

Polymorphic Relationships

A polymorphic relationship allows the target model to belong to more than one type of model using a single association.

One To One (Polymorphic)

Table Structure

A one-to-one polymorphic relation is similar to a simple one-to-one relation; however, the target model can belong to more than one type of model on a single association. For example, a blog `Post` and a `User` may share a polymorphic relation to an `Image` model. Using a one-to-one polymorphic relation allows you to have a single list of unique images that are used for both blog posts and user accounts. First, let's examine the table structure:

```
posts
  id - integer
  name - string

users
  id - integer
  name - string

images
  id - integer
  url - string
  imageable_id - integer
  imageable_type - string
```

Take note of the `imageable_id` and `imageable_type` columns on the `images` table. The `imageable_id` column will contain the ID value of the post or user, while the `imageable_type` column will contain the class name of the parent model. The `imageable_type` column is used by Eloquent to determine which "type" of parent model to return when accessing the `imageable` relation.

Model Structure

Next, let's examine the model definitions needed to build this relationship:

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;

class Image extends Model
{
    /**
     * Get the owning imageable model.
     */
    public function imageable()
    {
        return $this->morphTo();
    }
}

class Post extends Model
{
    /**
     * Get the post's image.
     */
    public function image()
    {
        return $this->morphOne('App\Image', 'imageable');
    }
}
```

```
}

class User extends Model
{
    /**
     * Get the user's image.
     */
    public function image()
    {
        return $this->morphOne('App\Image', 'imageable');
    }
}
```

Retrieving The Relationship

Once your database table and models are defined, you may access the relationships via your models. For example, to retrieve the image for a post, we can use the `image` dynamic property:

```
$post = App\Post::find(1);

$image = $post->image;
```

You may also retrieve the parent from the polymorphic model by accessing the name of the method that performs the call to `morphTo`. In our case, that is the `imageable` method on the `Image` model. So, we will access that method as a dynamic property:

```
$image = App\Image::find(1);

$imageable = $image->imageable;
```

The `imageable` relation on the `Image` model will return either a `Post` or `User` instance, depending on which type of model owns the image. If you need to specify custom `type` and `id` columns for the `morphTo` relation, always ensure you pass the relationship name (which should exactly match the method name) as the first parameter:

```
/**
 * Get the model that the image belongs to.
 */
public function imageable()
{
    return $this->morphTo(__FUNCTION__, 'imageable_type', 'imageable_id');
}
```

One To Many (Polymorphic)

Table Structure

A one-to-many polymorphic relation is similar to a simple one-to-many relation; however, the target model can belong to more than one type of model on a single association. For example, imagine users of your application can "comment" on both posts and videos. Using polymorphic relationships, you may use a single `comments` table for both of these scenarios. First, let's examine the table structure required to build this relationship:

```
posts
  id - integer
  title - string
  body - text

videos
  id - integer
  title - string
  url - string

comments
  id - integer
  body - text
  commentable_id - integer
  commentable_type - string
```

Model Structure

Next, let's examine the model definitions needed to build this relationship:

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;

class Comment extends Model
{
    /**
     * Get the owning commentable model.
     */
    public function commentable()
    {
        return $this->morphTo();
    }
}

class Post extends Model
{
    /**
     * Get all of the post's comments.
     */
    public function comments()
    {
        return $this->morphMany('App\Comment', 'commentable');
    }
}

class Video extends Model
{
    /**
     * Get all of the video's comments.
     */
    public function comments()
    {
        return $this->morphMany('App\Comment', 'commentable');
    }
}
```

Retrieving The Relationship

Once your database table and models are defined, you may access the relationships via your models. For example, to access all of the comments for a post, we can use the `comments` dynamic property:

```
$post = App\Post::find(1);

foreach ($post->comments as $comment) {
    //
}
```

You may also retrieve the owner of a polymorphic relation from the polymorphic model by accessing the name of the method that performs the call to `morphTo`. In our case, that is the `commentable` method on the `Comment` model. So, we will access that method as a dynamic property:

```
$comment = App\Comment::find(1);

$commentable = $comment->commentable;
```

The `commentable` relation on the `Comment` model will return either a `Post` or `Video` instance, depending on which type of model owns the comment.

Many To Many (Polymorphic)

Table Structure

Many-to-many polymorphic relations are slightly more complicated than `morphOne` and `morphMany` relationships. For example, a blog `Post` and `Video` model could share a polymorphic relation to a `Tag` model. Using a many-to-many polymorphic relation allows you to have a single list of unique tags that are shared across blog posts and videos. First, let's examine the table structure:

```
posts
  id - integer
```

```
        name - string

videos
    id - integer
    name - string

tags
    id - integer
    name - string

taggables
    tag_id - integer
    taggable_id - integer
    taggable_type - string
```

Model Structure

Next, we're ready to define the relationships on the model. The `Post` and `Video` models will both have a `tags` method that calls the `morphToMany` method on the base Eloquent class:

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;

class Post extends Model
{
    /**
     * Get all of the tags for the post.
     */
    public function tags()
    {
        return $this->morphToMany('App\Tag', 'taggable');
    }
}
```

Defining The Inverse Of The Relationship

Next, on the `Tag` model, you should define a method for each of its related models. So, for this example, we will define a `posts` method and a `videos` method:

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;

class Tag extends Model
{
    /**
     * Get all of the posts that are assigned this tag.
     */
    public function posts()
    {
        return $this->morphedByMany('App\Post', 'taggable');
    }

    /**
     * Get all of the videos that are assigned this tag.
     */
    public function videos()
    {
        return $this->morphedByMany('App\Video', 'taggable');
    }
}
```

Retrieving The Relationship

Once your database table and models are defined, you may access the relationships via your models. For example, to access all of the tags for a post, you can use the `tags` dynamic property:

```
$post = App\Post::find(1);
```

```
foreach ($post->tags as $tag) {  
    //  
}
```

You may also retrieve the owner of a polymorphic relation from the polymorphic model by accessing the name of the method that performs the call to `morphedByMany`. In our case, that is the `posts` or `videos` methods on the `Tag` model. So, you will access those methods as dynamic properties:

```
$tag = App\Tag::find(1);  
  
foreach ($tag->videos as $video) {  
    //  
}
```

Custom Polymorphic Types

By default, Laravel will use the fully qualified class name to store the type of the related model. For instance, given the one-to-many example above where a `Comment` may belong to a `Post` or a `Video`, the default `commentable_type` would be either `App\Post` or `App\Video`, respectively. However, you may wish to decouple your database from your application's internal structure. In that case, you may define a "morph map" to instruct Eloquent to use a custom name for each model instead of the class name:

```
use Illuminate\Database\Eloquent\Relations\Relation;  
  
Relation::morphMap([  
    'posts' => 'App\Post',  
    'videos' => 'App\Video',  
]);
```

You may register the `morphMap` in the `boot` function of your `AppServiceProvider` or create a separate service provider if you wish.

NOTE When adding a "morph map" to your existing application, every morphable `*_type` column value in your database that still contains a fully-qualified class will need to be converted to its "map" name.

You may determine the morph alias of a given model at runtime using the `getMorphClass` method. Conversely, you may determine the fully-qualified class name associated with a morph alias using the `Relation::getMorphedModel` method:

```
use Illuminate\Database\Eloquent\Relations\Relation;  
  
$alias = $post->getMorphClass();  
  
$class = Relation::getMorphedModel($alias);
```

Dynamic Relationships

You may use the `resolveRelationUsing` method to define relations between Eloquent models at runtime. While not typically recommended for normal application development, this may occasionally be useful when developing Laravel packages:

```
use App\Order;  
use App\Customer;  
  
Order::resolveRelationUsing('customer', function ($orderModel) {  
    return $orderModel->belongsTo(Customer::class, 'customer_id');  
});
```

NOTE When defining dynamic relationships, always provide explicit key name arguments to the Eloquent relationship methods.

Querying Relations

Since all types of Eloquent relationships are defined via methods, you may call those methods to obtain an instance of the relationship without actually executing the relationship queries. In addition, all types of Eloquent relationships also serve as [query builders](#), allowing you to continue to chain constraints onto the relationship query before finally executing the SQL against your database.

For example, imagine a blog system in which a user model has many associated Post models:

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;

class User extends Model
{
    /**
     * Get all of the posts for the user.
     */
    public function posts()
    {
        return $this->hasMany('App\Post');
    }
}
```

You may query the posts relationship and add additional constraints to the relationship like so:

```
$user = App\User::find(1);

$user->posts()->where('active', 1)->get();
```

You are able to use any of the [query builder](#) methods on the relationship, so be sure to explore the query builder documentation to learn about all of the methods that are available to you.

Chaining `orWhere` Clauses After Relationships

As demonstrated in the example above, you are free to add additional constraints to relationships when querying them. However, use caution when chaining `orWhere` clauses onto a relationship, as the `orWhere` clauses will be logically grouped at the same level as the relationship constraint:

```
$user->posts()
    ->where('active', 1)
    ->orWhere('votes', '>=', 100)
    ->get();

// select * from posts
// where user_id = ? and active = 1 or votes >= 100
```

In most situations, you likely intend to use [constraint groups](#) to logically group the conditional checks between parentheses:

```
use Illuminate\Database\Eloquent\Builder;

$user->posts()
    ->where(function (Builder $query) {
        return $query->where('active', 1)
            ->orWhere('votes', '>=', 100);
    })
    ->get();

// select * from posts
// where user_id = ? and (active = 1 or votes >= 100)
```

Relationship Methods Vs. Dynamic Properties

If you do not need to add additional constraints to an Eloquent relationship query, you may access the relationship as if it were a property. For example, continuing to use our user and Post example models, we may access all of a user's posts like so:

```
$user = App\User::find(1);

foreach ($user->posts as $post) {
    //
}
```

Dynamic properties are "lazy loading", meaning they will only load their relationship data when you actually access them. Because of this, developers often use [eager loading](#) to pre-load relationships they know will be

accessed after loading the model. Eager loading provides a significant reduction in SQL queries that must be executed to load a model's relations.

Querying Relationship Existence

When accessing the records for a model, you may wish to limit your results based on the existence of a relationship. For example, imagine you want to retrieve all blog posts that have at least one comment. To do so, you may pass the name of the relationship to the `has` and `orHas` methods:

```
// Retrieve all posts that have at least one comment...
$posts = App\Post::has('comments')->get();
```

You may also specify an operator and count to further customize the query:

```
// Retrieve all posts that have three or more comments...
$posts = App\Post::has('comments', '>=', 3)->get();
```

Nested `has` statements may also be constructed using "dot" notation. For example, you may retrieve all posts that have at least one comment and vote:

```
// Retrieve posts that have at least one comment with votes...
$posts = App\Post::has('comments.votes')->get();
```

If you need even more power, you may use the `whereHas` and `orWhereHas` methods to put "where" conditions on your `has` queries. These methods allow you to add customized constraints to a relationship constraint, such as checking the content of a comment:

```
use Illuminate\Database\Eloquent\Builder;

// Retrieve posts with at least one comment containing words like foo...
$posts = App\Post::whereHas('comments', function (Builder $query) {
    $query->where('content', 'like', 'foo%');
})->get();

// Retrieve posts with at least ten comments containing words like foo...
$posts = App\Post::whereHas('comments', function (Builder $query) {
    $query->where('content', 'like', 'foo%');
}, '>=', 10)->get();
```

Querying Relationship Absence

When accessing the records for a model, you may wish to limit your results based on the absence of a relationship. For example, imagine you want to retrieve all blog posts that **don't** have any comments. To do so, you may pass the name of the relationship to the `doesn'tHave` and `orDoesntHave` methods:

```
$posts = App\Post::doesn'tHave('comments')->get();
```

If you need even more power, you may use the `whereDoesntHave` and `orWhereDoesntHave` methods to put "where" conditions on your `doesn'tHave` queries. These methods allows you to add customized constraints to a relationship constraint, such as checking the content of a comment:

```
use Illuminate\Database\Eloquent\Builder;

$posts = App\Post::whereDoesntHave('comments', function (Builder $query) {
    $query->where('content', 'like', 'foo%');
})->get();
```

You may use "dot" notation to execute a query against a nested relationship. For example, the following query will retrieve all posts that do not have comments and posts that have comments from authors that are not banned:

```
use Illuminate\Database\Eloquent\Builder;

$posts = App\Post::whereDoesntHave('comments.author', function (Builder $query) {
    $query->where('banned', 0);
})->get();
```

Querying Polymorphic Relationships

To query the existence of `MorphTo` relationships, you may use the `whereHasMorph` method and its corresponding methods:

```
use Illuminate\Database\Eloquent\Builder;

// Retrieve comments associated to posts or videos with a title like foo%...
$comments = App\Comment::whereHasMorph(
    'commentable',
    ['App\Post', 'App\Video'],
    function (Builder $query) {
        $query->where('title', 'like', 'foo%');
    }
)->get();

// Retrieve comments associated to posts with a title not like foo%...
$comments = App\Comment::whereDoesntHaveMorph(
    'commentable',
    'App\Post',
    function (Builder $query) {
        $query->where('title', 'like', 'foo%');
    }
)->get();
```

You may use the `$type` parameter to add different constraints depending on the related model:

```
use Illuminate\Database\Eloquent\Builder;

$comments = App\Comment::whereHasMorph(
    'commentable',
    ['App\Post', 'App\Video'],
    function (Builder $query, $type) {
        $query->where('title', 'like', 'foo%');

        if ($type === 'App\Post') {
            $query->orWhere('content', 'like', 'foo%');
        }
    }
)->get();
```

Instead of passing an array of possible polymorphic models, you may provide `*` as a wildcard and let Laravel retrieve all the possible polymorphic types from the database. Laravel will execute an additional query in order to perform this operation:

```
use Illuminate\Database\Eloquent\Builder;

$comments = App\Comment::whereHasMorph('commentable', '*', function (Builder $query) {
    $query->where('title', 'like', 'foo%');
})->get();
```

Counting Related Models

If you want to count the number of results from a relationship without actually loading them you may use the `withCount` method, which will place a `{relation}_count` column on your resulting models. For example:

```
$posts = App\Post::withCount('comments')->get();

foreach ($posts as $post) {
    echo $post->comments_count;
}
```

You may add the "counts" for multiple relations as well as add constraints to the queries:

```
use Illuminate\Database\Eloquent\Builder;

$posts = App\Post::withCount(['votes', 'comments' => function (Builder $query) {
    $query->where('content', 'like', 'foo%');
}])->get();

echo $posts[0]->votes_count;
echo $posts[0]->comments_count;
```

You may also alias the relationship count result, allowing multiple counts on the same relationship:

```
use Illuminate\Database\Eloquent\Builder;
```



```
$posts = App\Post::withCount([
    'comments',
    'comments as pending_comments_count' => function (Builder $query) {
        $query->where('approved', false);
    },
])->get();

echo $posts[0]->comments_count;

echo $posts[0]->pending_comments_count;
```

If you're combining `withCount` with a `select` statement, ensure that you call `withCount` after the `select` method:

```
$posts = App\Post::select(['title', 'body'])->withCount('comments')->get();

echo $posts[0]->title;
echo $posts[0]->body;
echo $posts[0]->comments_count;
```

In addition, using the `loadCount` method, you may load a relationship count after the parent model has already been retrieved:

```
$book = App\Book::first();

$book->loadCount('genres');
```

If you need to set additional query constraints on the eager loading query, you may pass an array keyed by the relationships you wish to load. The array values should be `closure` instances which receive the query builder instance:

```
$book->loadCount(['reviews' => function ($query) {
    $query->where('rating', 5);
}])
```

Counting Related Models On Polymorphic Relationships

If you would like to eager load a `morphTo` relationship, as well as nested relationship counts on the various entities that may be returned by that relationship, you may use the `with` method in combination with the `morphTo` relationship's `morphWithCount` method.

In this example, let's assume `Photo` and `Post` models may create `ActivityFeed` models. Additionally, let's assume that `Photo` models are associated with `tag` models, and `Post` models are associated with `comment` models.

Using these model definitions and relationships, we may retrieve `ActivityFeed` model instances and eager load all `parentable` models and their respective nested relationship counts:

```
use Illuminate\Database\Eloquent\Relations\MorphTo;

$activities = ActivityFeed::query()
    ->with(['parentable' => function (MorphTo $morphTo) {
        $morphTo->morphWithCount([
            Photo::class => ['tags'],
            Post::class => ['comments'],
        ]);
    }])->get();
```

In addition, you may use the `loadMorphCount` method to eager load all nested relationship counts on the various entities of the polymorphic relation if the `ActivityFeed` models have already been retrieved:

```
$activities = ActivityFeed::with('parentable')
    ->get()
    ->loadMorphCount('parentable', [
        Photo::class => ['tags'],
        Post::class => ['comments'],
    ]);
```

Eager Loading

When accessing Eloquent relationships as properties, the relationship data is "lazy loaded". This means the

relationship data is not actually loaded until you first access the property. However, Eloquent can "eager load" relationships at the time you query the parent model. Eager loading alleviates the N + 1 query problem. To illustrate the N + 1 query problem, consider a `Book` model that is related to `Author`:

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;

class Book extends Model
{
    /**
     * Get the author that wrote the book.
     */
    public function author()
    {
        return $this->belongsTo('App\Author');
    }
}
```

Now, let's retrieve all books and their authors:

```
$books = App\Book::all();

foreach ($books as $book) {
    echo $book->author->name;
}
```

This loop will execute 1 query to retrieve all of the books on the table, then another query for each book to retrieve the author. So, if we have 25 books, this loop would run 26 queries: 1 for the original book, and 25 additional queries to retrieve the author of each book.

Thankfully, we can use eager loading to reduce this operation to just 2 queries. When querying, you may specify which relationships should be eager loaded using the `with` method:

```
$books = App\Book::with('author')->get();

foreach ($books as $book) {
    echo $book->author->name;
}
```

For this operation, only two queries will be executed:

```
select * from books

select * from authors where id in (1, 2, 3, 4, 5, ...)
```

Eager Loading Multiple Relationships

Sometimes you may need to eager load several different relationships in a single operation. To do so, just pass additional arguments to the `with` method:

```
$books = App\Book::with(['author', 'publisher'])->get();
```

Nested Eager Loading

To eager load nested relationships, you may use "dot" syntax. For example, let's eager load all of the book's authors and all of the author's personal contacts in one Eloquent statement:

```
$books = App\Book::with('author.contacts')->get();
```

Nested Eager Loading `morphTo` Relationships

If you would like to eager load a `morphTo` relationship, as well as nested relationships on the various entities that may be returned by that relationship, you may use the `with` method in combination with the `morphTo` relationship's `morphWith` method. To help illustrate this method, let's consider the following model:

```
<?php
```

```
use Illuminate\Database\Eloquent\Model;

class ActivityFeed extends Model
{
    /**
     * Get the parent of the activity feed record.
     */
    public function parentable()
    {
        return $this->morphTo();
    }
}
```

In this example, let's assume `Event`, `Photo`, and `Post` models may create `ActivityFeed` models. Additionally, let's assume that `Event` models belong to a `calendar` model, `Photo` models are associated with `Tag` models, and `Post` models belong to an `Author` model.

Using these model definitions and relationships, we may retrieve `ActivityFeed` model instances and eager load all `parentable` models and their respective nested relationships:

```
use Illuminate\Database\Eloquent\Relations\MorphTo;

$activities = ActivityFeed::query()
    ->with(['parentable' => function (MorphTo $morphTo) {
        $morphTo->morphWith([
            Event::class => ['calendar'],
            Photo::class => ['tags'],
            Post::class => ['author'],
        ]);
    }])->get();
```

Eager Loading Specific Columns

You may not always need every column from the relationships you are retrieving. For this reason, Eloquent allows you to specify which columns of the relationship you would like to retrieve:

```
$books = App\Book::with('author:id,name')->get();
```

NOTE When using this feature, you should always include the `id` column and any relevant foreign key columns in the list of columns you wish to retrieve.

Eager Loading By Default

Sometimes you might want to always load some relationships when retrieving a model. To accomplish this, you may define a `$with` property on the model:

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;

class Book extends Model
{
    /**
     * The relationships that should always be loaded.
     *
     * @var array
     */
    protected $with = ['author'];

    /**
     * Get the author that wrote the book.
     */
    public function author()
    {
        return $this->belongsTo('App\Author');
    }
}
```

If you would like to remove an item from the `$with` property for a single query, you may use the `without` method:

```
$books = App\Book::without('author')->get();
```

Constraining Eager Loads

Sometimes you may wish to eager load a relationship, but also specify additional query conditions for the eager loading query. Here's an example:

```
$users = App\User::with(['posts' => function ($query) {  
    $query->where('title', 'like', '%first%');  
}])->get();
```

In this example, Eloquent will only eager load posts where the post's `title` column contains the word `first`. You may call other [query builder](#) methods to further customize the eager loading operation:

```
$users = App\User::with(['posts' => function ($query) {  
    $query->orderBy('created_at', 'desc');  
}])->get();
```

NOTE The `limit` and `take` query builder methods may not be used when constraining eager loads.

Lazy Eager Loading

Sometimes you may need to eager load a relationship after the parent model has already been retrieved. For example, this may be useful if you need to dynamically decide whether to load related models:

```
$books = App\Book::all();  
  
if ($someCondition) {  
    $books->load('author', 'publisher');  
}
```

If you need to set additional query constraints on the eager loading query, you may pass an array keyed by the relationships you wish to load. The array values should be `closure` instances which receive the query instance:

```
$author->load(['books' => function ($query) {  
    $query->orderBy('published_date', 'asc');  
}]);
```

To load a relationship only when it has not already been loaded, use the `loadMissing` method:

```
public function format(Book $book)  
{  
    $book->loadMissing('author');  
  
    return [  
        'name' => $book->name,  
        'author' => $book->author->name,  
    ];  
}
```

Nested Lazy Eager Loading & `morphTo`

If you would like to eager load a `morphTo` relationship, as well as nested relationships on the various entities that may be returned by that relationship, you may use the `loadMorph` method.

This method accepts the name of the `morphTo` relationship as its first argument, and an array of model / relationship pairs as its second argument. To help illustrate this method, let's consider the following model:

```
<?php  
  
use Illuminate\Database\Eloquent\Model;  
  
class ActivityFeed extends Model  
{  
    /**  
     * Get the parent of the activity feed record.  
     */  
    public function parentable()  
    {  
        return $this->morphTo();  
    }  
}
```

```
}  
}
```

In this example, let's assume `Event`, `Photo`, and `Post` models may create `ActivityFeed` models. Additionally, let's assume that `Event` models belong to a `Calendar` model, `Photo` models are associated with `Tag` models, and `Post` models belong to an `Author` model.

Using these model definitions and relationships, we may retrieve `ActivityFeed` model instances and eager load all `parentable` models and their respective nested relationships:

```
$activities = ActivityFeed::with('parentable')  
    ->get()  
    ->loadMorph('parentable', [  
        Event::class => ['calendar'],  
        Photo::class => ['tags'],  
        Post::class => ['author'],  
    ]);
```

Inserting & Updating Related Models

The Save Method

Eloquent provides convenient methods for adding new models to relationships. For example, perhaps you need to insert a new `Comment` for a `Post` model. Instead of manually setting the `post_id` attribute on the `Comment`, you may insert the `Comment` directly from the relationship's `save` method:

```
$comment = new App\Comment(['message' => 'A new comment.']);  
  
$post = App\Post::find(1);  
  
$post->comments()->save($comment);
```

Notice that we did not access the `comments` relationship as a dynamic property. Instead, we called the `comments` method to obtain an instance of the relationship. The `save` method will automatically add the appropriate `post_id` value to the new `Comment` model.

If you need to save multiple related models, you may use the `saveMany` method:

```
$post = App\Post::find(1);  
  
$post->comments()->saveMany([  
    new App\Comment(['message' => 'A new comment.']),  
    new App\Comment(['message' => 'Another comment.']),  
]);
```

The `save` and `saveMany` methods will not add the new models to any in-memory relationships that are already loaded onto the parent model. If you plan on accessing the relationship after using the `save` or `saveMany` methods, you may wish to use the `refresh` method to reload the model and its relationships:

```
$post->comments()->save($comment);  
  
$post->refresh();  
  
// All comments, including the newly saved comment...  
$post->comments;
```

Recursively Saving Models & Relationships

If you would like to save your model and all of its associated relationships, you may use the `push` method:

```
$post = App\Post::find(1);  
  
$post->comments[0]->message = 'Message';  
$post->comments[0]->author->name = 'Author Name';  
  
$post->push();
```

The Create Method

In addition to the `save` and `saveMany` methods, you may also use the `create` method, which accepts an array of attributes, creates a model, and inserts it into the database. Again, the difference between `save` and `create` is that `save` accepts a full Eloquent model instance while `create` accepts a plain PHP array:

```
$post = App\Post::find(1);

$comment = $post->comments()->create([
    'message' => 'A new comment.',
]);
```

TIP Before using the `create` method, be sure to review the documentation on attribute [mass assignment](#).

You may use the `createMany` method to create multiple related models:

```
$post = App\Post::find(1);

$post->comments()->createMany([
    [
        'message' => 'A new comment.',
    ],
    [
        'message' => 'Another new comment.',
    ],
]);
```

You may also use the `findOrCreate`, `firstOrCreate`, `firstOrCreate` and `updateOrCreate` methods to [create and update models on relationships](#).

Belongs To Relationships

When updating a `belongsTo` relationship, you may use the `associate` method. This method will set the foreign key on the child model:

```
$account = App\Account::find(10);

$user->account()->associate($account);

$user->save();
```

When removing a `belongsTo` relationship, you may use the `dissociate` method. This method will set the relationship's foreign key to `null`:

```
$user->account()->dissociate();

$user->save();
```

Default Models

The `belongsTo`, `hasOne`, `hasOneThrough`, and `morphOne` relationships allow you to define a default model that will be returned if the given relationship is `null`. This pattern is often referred to as the [Null Object pattern](#) and can help remove conditional checks in your code. In the following example, the `user` relation will return an empty `App\User` model if no user is attached to the post:

```
/**
 * Get the author of the post.
 */
public function user()
{
    return $this->belongsTo('App\User')->withDefault();
}
```

To populate the default model with attributes, you may pass an array or Closure to the `withDefault` method:

```
/**
 * Get the author of the post.
 */
public function user()
{
    return $this->belongsTo('App\User')->withDefault([
        'name' => 'Guest Author',
    ]);
}
```

```
}

/**
 * Get the author of the post.
 */
public function user()
{
    return $this->belongsTo('App\User')->withDefault(function ($user, $post) {
        $user->name = 'Guest Author';
    });
}
```

Many To Many Relationships

Attaching / Detaching

Eloquent also provides a few additional helper methods to make working with related models more convenient. For example, let's imagine a user can have many roles and a role can have many users. To attach a role to a user by inserting a record in the intermediate table that joins the models, use the `attach` method:

```
$user = App\User::find(1);
$user->roles()->attach($roleId);
```

When attaching a relationship to a model, you may also pass an array of additional data to be inserted into the intermediate table:

```
$user->roles()->attach($roleId, ['expires' => $expires]);
```

Sometimes it may be necessary to remove a role from a user. To remove a many-to-many relationship record, use the `detach` method. The `detach` method will delete the appropriate record out of the intermediate table; however, both models will remain in the database:

```
// Detach a single role from the user...
$user->roles()->detach($roleId);

// Detach all roles from the user...
$user->roles()->detach();
```

For convenience, `attach` and `detach` also accept arrays of IDs as input:

```
$user = App\User::find(1);

$user->roles()->detach([1, 2, 3]);

$user->roles()->attach([
    1 => ['expires' => $expires],
    2 => ['expires' => $expires],
]);
```

Syncing Associations

You may also use the `sync` method to construct many-to-many associations. The `sync` method accepts an array of IDs to place on the intermediate table. Any IDs that are not in the given array will be removed from the intermediate table. So, after this operation is complete, only the IDs in the given array will exist in the intermediate table:

```
$user->roles()->sync([1, 2, 3]);
```

You may also pass additional intermediate table values with the IDs:

```
$user->roles()->sync([1 => ['expires' => true], 2, 3]);
```

If you do not want to detach existing IDs, you may use the `syncWithoutDetaching` method:

```
$user->roles()->syncWithoutDetaching([1, 2, 3]);
```

Toggling Associations

The many-to-many relationship also provides a `toggle` method which "toggles" the attachment status of the given IDs. If the given ID is currently attached, it will be detached. Likewise, if it is currently detached, it will be attached:

```
$user->roles()->toggle([1, 2, 3]);
```

Saving Additional Data On A Pivot Table

When working with a many-to-many relationship, the `save` method accepts an array of additional intermediate table attributes as its second argument:

```
App\User::find(1)->roles()->save($role, ['expires' => $expires]);
```

Updating A Record On A Pivot Table

If you need to update an existing row in your pivot table, you may use `updateExistingPivot` method. This method accepts the pivot record foreign key and an array of attributes to update:

```
$user = App\User::find(1);  
$user->roles()->updateExistingPivot($roleId, $attributes);
```

Touching Parent Timestamps

When a model `belongsTo` or `belongsToMany` another model, such as a `Comment` which belongs to a `Post`, it is sometimes helpful to update the parent's timestamp when the child model is updated. For example, when a `Comment` model is updated, you may want to automatically "touch" the `updated_at` timestamp of the owning `Post`. Eloquent makes it easy. Just add a `touches` property containing the names of the relationships to the child model:

```
<?php  
  
namespace App;  
  
use Illuminate\Database\Eloquent\Model;  
  
class Comment extends Model  
{  
    /**  
     * All of the relationships to be touched.  
     *  
     * @var array  
     */  
    protected $touches = ['post'];  
  
    /**  
     * Get the post that the comment belongs to.  
     */  
    public function post()  
    {  
        return $this->belongsTo('App\Post');  
    }  
}
```

Now, when you update a `Comment`, the owning `Post` will have its `updated_at` column updated as well, making it more convenient to know when to invalidate a cache of the `Post` model:

```
$comment = App\Comment::find(1);  
  
$comment->text = 'Edit to this comment!';  
  
$comment->save();
```


Eloquent ORM

Collections

Eloquent: Collections

- [Introduction](#)
- [Available Methods](#)
- [Custom Collections](#)

Introduction

All multi-result sets returned by Eloquent are instances of the `Illuminate\Database\Eloquent\Collection` object, including results retrieved via the `get` method or accessed via a relationship. The Eloquent collection object extends the Laravel [base collection](#), so it naturally inherits dozens of methods used to fluently work with the underlying array of Eloquent models.

All collections also serve as iterators, allowing you to loop over them as if they were simple PHP arrays:

```
$users = App\User::where('active', 1)->get();

foreach ($users as $user) {
    echo $user->name;
}
```

However, collections are much more powerful than arrays and expose a variety of map / reduce operations that may be chained using an intuitive interface. For example, let's remove all inactive models and gather the first name for each remaining user:

```
$users = App\User::all();

$names = $users->reject(function ($user) {
    return $user->active === false;
})
->map(function ($user) {
    return $user->name;
});
```

NOTE While most Eloquent collection methods return a new instance of an Eloquent collection, the `pluck`, `keys`, `zip`, `collapse`, `flatten` and `flip` methods return a [base collection](#) instance. Likewise, if a map operation returns a collection that does not contain any Eloquent models, it will be automatically cast to a base collection.

Available Methods

All Eloquent collections extend the base [Laravel collection](#) object; therefore, they inherit all of the powerful methods provided by the base collection class.

In addition, the `Illuminate\Database\Eloquent\Collection` class provides a superset of methods to aid with managing your model collections. Most methods return `Illuminate\Database\Eloquent\Collection` instances; however, some methods return a base `Illuminate\Support\Collection` instance.

[contains](#) [diff](#) [except](#) [find](#) [fresh](#) [intersect](#) [load](#) [loadMissing](#) [modelKeys](#) [makeVisible](#) [makeHidden](#) [only](#) [toQuery](#) [unique](#)

```
contains($key, $operator = null, $value = null) {collection-method.first-collection-method}
```

The `contains` method may be used to determine if a given model instance is contained by the collection. This method accepts a primary key or a model instance:

```
$users->contains(1);

$users->contains(User::find(1));
```

diff(\$items) {.collection-method}

The `diff` method returns all of the models that are not present in the given collection:

```
use App\User;

$users = $users->diff(User::whereIn('id', [1, 2, 3])->get());
```

except(\$keys) {.collection-method}

The `except` method returns all of the models that do not have the given primary keys:

```
$users = $users->except([1, 2, 3]);
```

find(\$key) {.collection-method}

The `find` method finds a model that has a given primary key. If `$key` is a model instance, `find` will attempt to return a model matching the primary key. If `$key` is an array of keys, `find` will return all models which match the `$keys` using `whereIn()`:

```
$users = User::all();

$user = $users->find(1);
```

fresh(\$with = []) {.collection-method}

The `fresh` method retrieves a fresh instance of each model in the collection from the database. In addition, any specified relationships will be eager loaded:

```
$users = $users->fresh();

$users = $users->fresh('comments');
```

intersect(\$items) {.collection-method}

The `intersect` method returns all of the models that are also present in the given collection:

```
use App\User;

$users = $users->intersect(User::whereIn('id', [1, 2, 3])->get());
```

load(\$relations) {.collection-method}

The `load` method eager loads the given relationships for all models in the collection:

```
$users->load('comments', 'posts');

$users->load('comments.author');
```

loadMissing(\$relations) {.collection-method}

The `loadMissing` method eager loads the given relationships for all models in the collection if the relationships are not already loaded:

```
$users->loadMissing('comments', 'posts');

$users->loadMissing('comments.author');
```

modelKeys() {.collection-method}

The `modelKeys` method returns the primary keys for all models in the collection:

```
$users->modelKeys();

// [1, 2, 3, 4, 5]
```

makeVisible(\$attributes) {.collection-method}

The `makeVisible` method makes attributes visible that are typically "hidden" on each model in the collection:

```
$users = $users->makeVisible(['address', 'phone_number']);
```

makeHidden(\$attributes) {.collection-method}

The `makeHidden` method hides attributes that are typically "visible" on each model in the collection:

```
$users = $users->makeHidden(['address', 'phone_number']);
```

only(\$keys) {.collection-method}

The `only` method returns all of the models that have the given primary keys:

```
$users = $users->only([1, 2, 3]);
```

toQuery() {.collection-method}

The `toQuery` method returns an Eloquent query builder instance containing a `whereIn` constraint on the collection model's primary keys:

```
$users = App\User::where('status', 'VIP')->get();

$users->toQuery()->update([
    'status' => 'Administrator',
]);
```

unique(\$key = null, \$strict = false) {.collection-method}

The `unique` method returns all of the unique models in the collection. Any models of the same type with the same primary key as another model in the collection are removed.

```
$users = $users->unique();
```

Custom Collections

If you need to use a custom `Collection` object with your own extension methods, you may override the `newCollection` method on your model:

```
<?php

namespace App;

use App\CustomCollection;
use Illuminate\Database\Eloquent\Model;

class User extends Model
{
    /**
     * Create a new Eloquent Collection instance.
     *
     * @param array $models
     * @return \Illuminate\Database\Eloquent\Collection
     */
    public function newCollection(array $models = [])
    {
        return new CustomCollection($models);
    }
}
```

Once you have defined a `newCollection` method, you will receive an instance of your custom collection anytime Eloquent returns a `Collection` instance of that model. If you would like to use a custom collection for every model in your application, you should override the `newCollection` method on a base model class that is extended by all of your models.

Eloquent ORM

Mutators

Eloquent: Mutators

- [Introduction](#)
- [Accessors & Mutators](#)
 - [Defining An Accessor](#)
 - [Defining A Mutator](#)
- [Date Mutators](#)
- [Attribute Casting](#)
 - [Custom Casts](#)
 - [Array & JSON Casting](#)
 - [Date Casting](#)
 - [Query Time Casting](#)

Introduction

Accessors and mutators allow you to format Eloquent attribute values when you retrieve or set them on model instances. For example, you may want to use the [Laravel encrypter](#) to encrypt a value while it is stored in the database, and then automatically decrypt the attribute when you access it on an Eloquent model.

In addition to custom accessors and mutators, Eloquent can also automatically cast date fields to [Carbon](#) instances or even [cast text fields to JSON](#).

Accessors & Mutators

Defining An Accessor

To define an accessor, create a `getFooAttribute` method on your model where `Foo` is the "studly" cased name of the column you wish to access. In this example, we'll define an accessor for the `first_name` attribute. The accessor will automatically be called by Eloquent when attempting to retrieve the value of the `first_name` attribute:

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;

class User extends Model
{
    /**
     * Get the user's first name.
     *
     * @param string $value
     * @return string
     */
    public function getFirstNameAttribute($value)
    {
        return ucfirst($value);
    }
}
```

As you can see, the original value of the column is passed to the accessor, allowing you to manipulate and return the value. To access the value of the accessor, you may access the `first_name` attribute on a model instance:

```
$user = App\User::find(1);

$firstName = $user->first_name;
```

You may also use accessors to return new, computed values from existing attributes:

```
/**
 * Get the user's full name.
 *
 * @return string
 */
public function getFullNameAttribute()
{
    return "{$this->first_name} {$this->last_name}";
}
```

TIP If you would like these computed values to be added to the array / JSON representations of your model, [you will need to append them](#).

Defining A Mutator

To define a mutator, define a `setFooAttribute` method on your model where `Foo` is the "studly" cased name of the column you wish to access. So, again, let's define a mutator for the `first_name` attribute. This mutator will be automatically called when we attempt to set the value of the `first_name` attribute on the model:

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;

class User extends Model
{
    /**
     * Set the user's first name.
     *
     * @param string $value
     * @return void
     */
    public function setFirstNameAttribute($value)
    {
        $this->attributes['first_name'] = strtolower($value);
    }
}
```

The mutator will receive the value that is being set on the attribute, allowing you to manipulate the value and set the manipulated value on the Eloquent model's internal `$attributes` property. So, for example, if we attempt to set the `first_name` attribute to Sally:

```
$user = App\User::find(1);

$user->first_name = 'Sally';
```

In this example, the `setFirstNameAttribute` function will be called with the value `sally`. The mutator will then apply the `strtolower` function to the name and set its resulting value in the internal `$attributes` array.

Date Mutators

By default, Eloquent will convert the `created_at` and `updated_at` columns to instances of [Carbon](#), which extends the PHP `DateTime` class and provides an assortment of helpful methods. You may add additional date attributes by setting the `$dates` property of your model:

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;

class User extends Model
{
    /**
     * The attributes that should be mutated to dates.
     *
     * @var array
     */
    protected $dates = [
        'seen_at',
    ];
}
```

```
}
```

TIP You may disable the default `created_at` and `updated_at` timestamps by setting the public `$timestamps` property of your model to `false`.

When a column is considered a date, you may set its value to a UNIX timestamp, date string (`Y-m-d`), date-time string, or a `DateTime` / `Carbon` instance. The date's value will be correctly converted and stored in your database:

```
$user = App\User::find(1);

$user->deleted_at = now();

$user->save();
```

As noted above, when retrieving attributes that are listed in your `$dates` property, they will automatically be cast to [Carbon](#) instances, allowing you to use any of Carbon's methods on your attributes:

```
$user = App\User::find(1);

return $user->deleted_at->getTimestamp();
```

Date Formats

By default, timestamps are formatted as `'Y-m-d H:i:s'`. If you need to customize the timestamp format, set the `$dateFormat` property on your model. This property determines how date attributes are stored in the database:

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;

class Flight extends Model
{
    /**
     * The storage format of the model's date columns.
     *
     * @var string
     */
    protected $dateFormat = 'U';
}
```

Attribute Casting

The `$casts` property on your model provides a convenient method of converting attributes to common data types. The `$casts` property should be an array where the key is the name of the attribute being cast and the value is the type you wish to cast the column to. The supported cast types are: `integer`, `real`, `float`, `double`, `decimal:<digits>`, `string`, `boolean`, `object`, `array`, `collection`, `date`, `datetime`, and `timestamp`. When casting to `decimal`, you must define the number of digits (`decimal:2`).

To demonstrate attribute casting, let's cast the `is_admin` attribute, which is stored in our database as an integer (0 or 1) to a boolean value:

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;

class User extends Model
{
    /**
     * The attributes that should be cast.
     *
     * @var array
     */
    protected $casts = [
        'is_admin' => 'boolean',
    ];
}
```

Now the `is_admin` attribute will always be cast to a boolean when you access it, even if the underlying value is stored in the database as an integer:

```
$user = App\User::find(1);

if ($user->is_admin) {
    //
}
```

NOTE Attributes that are `null` will not be cast. In addition, you should never define a cast (or an attribute) that has the same name as a relationship.

Custom Casts

Laravel has a variety of built-in, helpful cast types; however, you may occasionally need to define your own cast types. You may accomplish this by defining a class that implements the `Castable` interface.

Classes that implement this interface must define a `get` and `set` method. The `get` method is responsible for transforming a raw value from the database into a cast value, while the `set` method should transform a cast value into a raw value that can be stored in the database. As an example, we will re-implement the built-in `json` cast type as a custom cast type:

```
<?php

namespace App\Casts;

use Illuminate\Contracts\Database\Eloquent\Castable;

class Json implements Castable
{
    /**
     * Cast the given value.
     *
     * @param  \Illuminate\Database\Eloquent\Model  $model
     * @param  string  $key
     * @param  mixed  $value
     * @param  array  $attributes
     * @return array
     */
    public function get($model, $key, $value, $attributes)
    {
        return json_decode($value, true);
    }

    /**
     * Prepare the given value for storage.
     *
     * @param  \Illuminate\Database\Eloquent\Model  $model
     * @param  string  $key
     * @param  array  $value
     * @param  array  $attributes
     * @return string
     */
    public function set($model, $key, $value, $attributes)
    {
        return json_encode($value);
    }
}
```

Once you have defined a custom cast type, you may attach it to a model attribute using its class name:

```
<?php

namespace App;

use App\Casts\Json;
use Illuminate\Database\Eloquent\Model;

class User extends Model
{
    /**
     * The attributes that should be cast.
     *
     * @var array
     */
}
```

```

        */
        protected $casts = [
            'options' => Json::class,
        ];
    }

```

Value Object Casting

You are not limited to casting values to primitive types. You may also cast values to objects. Defining custom casts that cast values to objects is very similar to casting to primitive types; however, the `set` method should return an array of key / value pairs that will be used to set raw, storable values on the model.

As an example, we will define a custom cast class that casts multiple model values into a single `Address` value object. We will assume the `Address` value has two public properties: `lineOne` and `lineTwo`:

```

<?php

namespace App\Casts;

use App\Address;
use Illuminate\Contracts\Database\Eloquent\CastsAttributes;
use InvalidArgumentException;

class Address implements CastsAttributes
{
    /**
     * Cast the given value.
     *
     * @param \Illuminate\Database\Eloquent\Model $model
     * @param string $key
     * @param mixed $value
     * @param array $attributes
     * @return \App\Address
     */
    public function get($model, $key, $value, $attributes)
    {
        return new Address(
            $attributes['address_line_one'],
            $attributes['address_line_two']
        );
    }

    /**
     * Prepare the given value for storage.
     *
     * @param \Illuminate\Database\Eloquent\Model $model
     * @param string $key
     * @param \App\Address $value
     * @param array $attributes
     * @return array
     */
    public function set($model, $key, $value, $attributes)
    {
        if (! $value instanceof Address) {
            throw new InvalidArgumentException('The given value is not an Address instance.');
```

When casting to value objects, any changes made to the value object will automatically be synced back to the model before the model is saved:

```

$user = App\User::find(1);

$user->address->lineOne = 'Updated Address Value';

$user->save();

```

TIP If you plan to serialize your Eloquent models containing value objects to JSON or arrays, you should implement the `Illuminate\Contracts\Support\Arrayable` and `JsonSerializable` interfaces on the value

object.

Inbound Casting

Occasionally, you may need to write a custom cast that only transforms values that are being set on the model and does not perform any operations when attributes are being retrieved from the model. A classic example of an inbound only cast is a "hashing" cast. Inbound only custom casts should implement the `CastsinboundAttributes` interface, which only requires a `set` method to be defined.

```
<?php

namespace App\Casts;

use Illuminate\Contracts\Database\Eloquent\CastsInboundAttributes;

class Hash implements CastsinboundAttributes
{
    /**
     * The hashing algorithm.
     *
     * @var string
     */
    protected $algorithm;

    /**
     * Create a new cast class instance.
     *
     * @param string|null $algorithm
     * @return void
     */
    public function __construct($algorithm = null)
    {
        $this->algorithm = $algorithm;
    }

    /**
     * Prepare the given value for storage.
     *
     * @param \Illuminate\Database\Eloquent\Model $model
     * @param string $key
     * @param array $value
     * @param array $attributes
     * @return string
     */
    public function set($model, $key, $value, $attributes)
    {
        return is_null($this->algorithm)
            ? bcrypt($value)
            : hash($this->algorithm, $value);
    }
}
```

Cast Parameters

When attaching a custom cast to a model, cast parameters may be specified by separating them from the class name using a `:` character and comma-delimiting multiple parameters. The parameters will be passed to the constructor of the cast class:

```
/**
 * The attributes that should be cast.
 *
 * @var array
 */
protected $casts = [
    'secret' => Hash::class.':sha256',
];
```

Castables

Instead of attaching the custom cast to your model, you may alternatively attach a class that implements the `Illuminate\Contracts\Database\Eloquent\Castable` interface:

```
protected $casts = [
```

```
        'address' => \App\Address::class,  
    ];
```

Objects that implement the `Castable` interface must define a `castUsing` method that returns the class name of the custom caster class that is responsible for casting to and from the `Castable` class:

```
<?php  
  
namespace App;  
  
use Illuminate\Contracts\Database\Eloquent\Castable;  
use App\Casts\Address as AddressCast;  
  
class Address implements Castable  
{  
    /**  
     * Get the name of the caster class to use when casting from / to this cast target.  
     *  
     * @return string  
     */  
    public static function castUsing()  
    {  
        return AddressCast::class;  
    }  
}
```

When using `Castable` classes, you may still provide arguments in the `$casts` definition. The arguments will be passed directly to the caster class:

```
protected $casts = [  
    'address' => \App\Address::class.':argument',  
];
```

Array & JSON Casting

The `array` cast type is particularly useful when working with columns that are stored as serialized JSON. For example, if your database has a `JSON` or `TEXT` field type that contains serialized JSON, adding the `array` cast to that attribute will automatically deserialize the attribute to a PHP array when you access it on your Eloquent model:

```
<?php  
  
namespace App;  
  
use Illuminate\Database\Eloquent\Model;  
  
class User extends Model  
{  
    /**  
     * The attributes that should be cast.  
     *  
     * @var array  
     */  
    protected $casts = [  
        'options' => 'array',  
    ];  
}
```

Once the cast is defined, you may access the `options` attribute and it will automatically be deserialized from JSON into a PHP array. When you set the value of the `options` attribute, the given array will automatically be serialized back into JSON for storage:

```
$user = App\User::find(1);  
  
$options = $user->options;  
  
$options['key'] = 'value';  
  
$user->options = $options;  
  
$user->save();
```

Date Casting

When using the `date` or `datetime` cast type, you may specify the date's format. This format will be used when the [model is serialized to an array or JSON](#):

```
/**
 * The attributes that should be cast.
 *
 * @var array
 */
protected $casts = [
    'created_at' => 'datetime:Y-m-d',
];
```

Query Time Casting

Sometimes you may need to apply casts while executing a query, such as when selecting a raw value from a table. For example, consider the following query:

```
use App\Post;
use App\User;

$users = User::select([
    'users.*',
    'last_posted_at' => Post::selectRaw('MAX(created_at)')
    ->whereColumn('user_id', 'users.id')
])->get();
```

The `last_posted_at` attribute on the results of this query will be a raw string. It would be convenient if we could apply a date cast to this attribute when executing the query. To accomplish this, we may use the `withCasts` method:

```
$users = User::select([
    'users.*',
    'last_posted_at' => Post::selectRaw('MAX(created_at)')
    ->whereColumn('user_id', 'users.id')
])->withCasts([
    'last_posted_at' => 'datetime'
])->get();
```

Eloquent ORM

API Resources

Eloquent: API Resources

- [Introduction](#)
- [Generating Resources](#)
- [Concept Overview](#)
 - [Resource Collections](#)
- [Writing Resources](#)
 - [Data Wrapping](#)
 - [Pagination](#)
 - [Conditional Attributes](#)
 - [Conditional Relationships](#)
 - [Adding Meta Data](#)
- [Resource Responses](#)

Introduction

When building an API, you may need a transformation layer that sits between your Eloquent models and the JSON responses that are actually returned to your application's users. Laravel's resource classes allow you to expressively and easily transform your models and model collections into JSON.

Generating Resources

To generate a resource class, you may use the `make:resource` Artisan command. By default, resources will be placed in the `app/Http/Resources` directory of your application. Resources extend the `Illuminate\Http\Resources\Json\JsonResource` class:

```
php artisan make:resource User
```

Resource Collections

In addition to generating resources that transform individual models, you may generate resources that are responsible for transforming collections of models. This allows your response to include links and other meta information that is relevant to an entire collection of a given resource.

To create a resource collection, you should use the `--collection` flag when creating the resource. Or, including the word `collection` in the resource name will indicate to Laravel that it should create a collection resource. Collection resources extend the `Illuminate\Http\Resources\Json\ResourceCollection` class:

```
php artisan make:resource Users --collection
```

```
php artisan make:resource UserCollection
```

Concept Overview

TIP This is a high-level overview of resources and resource collections. You are highly encouraged to read the other sections of this documentation to gain a deeper understanding of the customization and power offered to you by resources.

Before diving into all of the options available to you when writing resources, let's first take a high-level look at how resources are used within Laravel. A resource class represents a single model that needs to be transformed into a JSON structure. For example, here is a simple `User` resource class:

```
<?php
```

```
namespace App\Http\Resources;
```

```
use Illuminate\Http\Resources\Json\JsonResource;

class User extends JsonResource
{
    /**
     * Transform the resource into an array.
     *
     * @param \Illuminate\Http\Request $request
     * @return array
     */
    public function toArray($request)
    {
        return [
            'id' => $this->id,
            'name' => $this->name,
            'email' => $this->email,
            'created_at' => $this->created_at,
            'updated_at' => $this->updated_at,
        ];
    }
}
```

Every resource class defines a `toArray` method which returns the array of attributes that should be converted to JSON when sending the response. Notice that we can access model properties directly from the `$this` variable. This is because a resource class will automatically proxy property and method access down to the underlying model for convenient access. Once the resource is defined, it may be returned from a route or controller:

```
use App\Http\Resources\User as UserResource;
use App\User;

Route::get('/user', function () {
    return new UserResource(User::find(1));
});
```

Resource Collections

If you are returning a collection of resources or a paginated response, you may use the `collection` method when creating the resource instance in your route or controller:

```
use App\Http\Resources\User as UserResource;
use App\User;

Route::get('/user', function () {
    return UserResource::collection(User::all());
});
```

Note that this does not allow any addition of meta data that may need to be returned with the collection. If you would like to customize the resource collection response, you may create a dedicated resource to represent the collection:

```
php artisan make:resource UserCollection
```

Once the resource collection class has been generated, you may easily define any meta data that should be included with the response:

```
<?php

namespace App\Http\Resources;

use Illuminate\Http\Resources\Json\ResourceCollection;

class UserCollection extends ResourceCollection
{
    /**
     * Transform the resource collection into an array.
     *
     * @param \Illuminate\Http\Request $request
     * @return array
     */
    public function toArray($request)
    {
        return [
            'data' => $this->collection,
            'links' => [

```

```
        'self' => 'link-value',
    ],
    ];
}
```

After defining your resource collection, it may be returned from a route or controller:

```
use App\Http\Resources\UserCollection;
use App\User;

Route::get('/users', function () {
    return new UserCollection(User::all());
});
```

Preserving Collection Keys

When returning a resource collection from a route, Laravel resets the collection's keys so that they are in simple numerical order. However, you may add a `preserveKeys` property to your resource class indicating if collection keys should be preserved:

```
<?php

namespace App\Http\Resources;

use Illuminate\Http\Resources\Json\JsonResource;

class User extends JsonResource
{
    /**
     * Indicates if the resource's collection keys should be preserved.
     *
     * @var bool
     */
    public $preserveKeys = true;
}
```

When the `preserveKeys` property is set to `true`, collection keys will be preserved:

```
use App\Http\Resources\User as UserResource;
use App\User;

Route::get('/user', function () {
    return UserResource::collection(User::all()->keyBy->id);
});
```

Customizing The Underlying Resource Class

Typically, the `$this->collection` property of a resource collection is automatically populated with the result of mapping each item of the collection to its singular resource class. The singular resource class is assumed to be the collection's class name without the trailing `collection` string.

For example, `UserCollection` will attempt to map the given user instances into the `user` resource. To customize this behavior, you may override the `$collects` property of your resource collection:

```
<?php

namespace App\Http\Resources;

use Illuminate\Http\Resources\Json\ResourceCollection;

class UserCollection extends ResourceCollection
{
    /**
     * The resource that this resource collects.
     *
     * @var string
     */
    public $collects = 'App\Http\Resources\Member';
}
```

Writing Resources

TIP If you have not read the [concept overview](#), you are highly encouraged to do so before proceeding with this documentation.

In essence, resources are simple. They only need to transform a given model into an array. So, each resource contains a `toArray` method which translates your model's attributes into an API friendly array that can be returned to your users:

```
<?php

namespace App\Http\Resources;

use Illuminate\Http\Resources\Json\JsonResource;

class User extends JsonResource
{
    /**
     * Transform the resource into an array.
     *
     * @param \Illuminate\Http\Request $request
     * @return array
     */
    public function toArray($request)
    {
        return [
            'id' => $this->id,
            'name' => $this->name,
            'email' => $this->email,
            'created_at' => $this->created_at,
            'updated_at' => $this->updated_at,
        ];
    }
}
```

Once a resource has been defined, it may be returned directly from a route or controller:

```
use App\Http\Resources\User as UserResource;
use App\User;

Route::get('/user', function () {
    return new UserResource(User::find(1));
});
```

Relationships

If you would like to include related resources in your response, you may add them to the array returned by your `toArray` method. In this example, we will use the `Post` resource's `collection` method to add the user's blog posts to the resource response:

```
/**
 * Transform the resource into an array.
 *
 * @param \Illuminate\Http\Request $request
 * @return array
 */
public function toArray($request)
{
    return [
        'id' => $this->id,
        'name' => $this->name,
        'email' => $this->email,
        'posts' => PostResource::collection($this->posts),
        'created_at' => $this->created_at,
        'updated_at' => $this->updated_at,
    ];
}
```

TIP If you would like to include relationships only when they have already been loaded, check out the documentation on [conditional relationships](#).

Resource Collections

While resources translate a single model into an array, resource collections translate a collection of models into an array. It is not absolutely necessary to define a resource collection class for each one of your model types

since all resources provide a `collection` method to generate an "ad-hoc" resource collection on the fly:

```
use App\Http\Resources\User as UserResource;
use App\User;

Route::get('/user', function () {
    return UserResource::collection(User::all());
});
```

However, if you need to customize the meta data returned with the collection, it will be necessary to define a resource collection:

```
<?php

namespace App\Http\Resources;

use Illuminate\Http\Resources\Json\ResourceCollection;

class UserCollection extends ResourceCollection
{
    /**
     * Transform the resource collection into an array.
     *
     * @param  \Illuminate\Http\Request  $request
     * @return array
     */
    public function toArray($request)
    {
        return [
            'data' => $this->collection,
            'links' => [
                'self' => 'link-value',
            ],
        ];
    }
}
```

Like singular resources, resource collections may be returned directly from routes or controllers:

```
use App\Http\Resources\UserCollection;
use App\User;

Route::get('/users', function () {
    return new UserCollection(User::all());
});
```

Data Wrapping

By default, your outermost resource is wrapped in a `data` key when the resource response is converted to JSON. So, for example, a typical resource collection response looks like the following:

```
{
  "data": [
    {
      "id": 1,
      "name": "Eladio Schroeder Sr.",
      "email": "therese28@example.com",
    },
    {
      "id": 2,
      "name": "Liliana Mayert",
      "email": "evandervort@example.com",
    }
  ]
}
```

If you would like to use a custom key instead of `data`, you may define a `$wrap` attribute on the resource class:

```
<?php

namespace App\Http\Resources;

use Illuminate\Http\Resources\Json\JsonResource;

class User extends JsonResource
{
    //
```



```
/**
 * The "data" wrapper that should be applied.
 *
 * @var string
 */
public static $wrap = 'user';
}
```

If you would like to disable the wrapping of the outermost resource, you may use the `withoutWrapping` method on the base resource class. Typically, you should call this method from your `AppServiceProvider` or another [service provider](#) that is loaded on every request to your application:

```
<?php

namespace App\Providers;

use Illuminate\Http\Resources\Json\JsonResource;
use Illuminate\Support\ServiceProvider;

class AppServiceProvider extends ServiceProvider
{
    /**
     * Register any application services.
     *
     * @return void
     */
    public function register()
    {
        //
    }

    /**
     * Bootstrap any application services.
     *
     * @return void
     */
    public function boot()
    {
        JsonResource::withoutWrapping();
    }
}
```

NOTE The `withoutWrapping` method only affects the outermost response and will not remove data keys that you manually add to your own resource collections.

Wrapping Nested Resources

You have total freedom to determine how your resource's relationships are wrapped. If you would like all resource collections to be wrapped in a data key, regardless of their nesting, you should define a resource collection class for each resource and return the collection within a data key.

You may be wondering if this will cause your outermost resource to be wrapped in two data keys. Don't worry, Laravel will never let your resources be accidentally double-wrapped, so you don't have to be concerned about the nesting level of the resource collection you are transforming:

```
<?php

namespace App\Http\Resources;

use Illuminate\Http\Resources\Json\ResourceCollection;

class CommentsCollection extends ResourceCollection
{
    /**
     * Transform the resource collection into an array.
     *
     * @param  \Illuminate\Http\Request  $request
     * @return array
     */
    public function toArray($request)
    {
        return ['data' => $this->collection];
    }
}
```

Data Wrapping And Pagination

When returning paginated collections in a resource response, Laravel will wrap your resource data in a `data` key even if the `withoutWrapping` method has been called. This is because paginated responses always contain `meta` and `links` keys with information about the paginator's state:

```
{
  "data": [
    {
      "id": 1,
      "name": "Eladio Schroeder Sr.",
      "email": "therese28@example.com",
    },
    {
      "id": 2,
      "name": "Liliana Mayert",
      "email": "evandervort@example.com",
    }
  ],
  "links": {
    "first": "http://example.com/pagination?page=1",
    "last": "http://example.com/pagination?page=1",
    "prev": null,
    "next": null
  },
  "meta": {
    "current_page": 1,
    "from": 1,
    "last_page": 1,
    "path": "http://example.com/pagination",
    "per_page": 15,
    "to": 10,
    "total": 10
  }
}
```

Pagination

You may always pass a paginator instance to the `collection` method of a resource or to a custom resource collection:

```
use App\Http\Resources\UserCollection;
use App\User;

Route::get('/users', function () {
    return new UserCollection(User::paginate());
});
```

Paginated responses always contain `meta` and `links` keys with information about the paginator's state:

```
{
  "data": [
    {
      "id": 1,
      "name": "Eladio Schroeder Sr.",
      "email": "therese28@example.com",
    },
    {
      "id": 2,
      "name": "Liliana Mayert",
      "email": "evandervort@example.com",
    }
  ],
  "links": {
    "first": "http://example.com/pagination?page=1",
    "last": "http://example.com/pagination?page=1",
    "prev": null,
    "next": null
  },
  "meta": {
    "current_page": 1,
    "from": 1,
    "last_page": 1,
    "path": "http://example.com/pagination",
    "per_page": 15,
  }
```

```
        "to": 10,
        "total": 10
    }
}
```

Conditional Attributes

Sometimes you may wish to only include an attribute in a resource response if a given condition is met. For example, you may wish to only include a value if the current user is an "administrator". Laravel provides a variety of helper methods to assist you in this situation. The `when` method may be used to conditionally add an attribute to a resource response:

```
/**
 * Transform the resource into an array.
 *
 * @param \Illuminate\Http\Request $request
 * @return array
 */
public function toArray($request)
{
    return [
        'id' => $this->id,
        'name' => $this->name,
        'email' => $this->email,
        'secret' => $this->when(Auth::user()->isAdmin(), 'secret-value'),
        'created_at' => $this->created_at,
        'updated_at' => $this->updated_at,
    ];
}
```

In this example, the `secret` key will only be returned in the final resource response if the authenticated user's `isAdmin` method returns `true`. If the method returns `false`, the `secret` key will be removed from the resource response entirely before it is sent back to the client. The `when` method allows you to expressively define your resources without resorting to conditional statements when building the array.

The `when` method also accepts a Closure as its second argument, allowing you to calculate the resulting value only if the given condition is `true`:

```
'secret' => $this->when(Auth::user()->isAdmin(), function () {
    return 'secret-value';
}),
```

Merging Conditional Attributes

Sometimes you may have several attributes that should only be included in the resource response based on the same condition. In this case, you may use the `mergeWhen` method to include the attributes in the response only when the given condition is `true`:

```
/**
 * Transform the resource into an array.
 *
 * @param \Illuminate\Http\Request $request
 * @return array
 */
public function toArray($request)
{
    return [
        'id' => $this->id,
        'name' => $this->name,
        'email' => $this->email,
        $this->mergeWhen(Auth::user()->isAdmin(), [
            'first-secret' => 'value',
            'second-secret' => 'value',
        ]),
        'created_at' => $this->created_at,
        'updated_at' => $this->updated_at,
    ];
}
```

Again, if the given condition is `false`, these attributes will be removed from the resource response entirely before it is sent to the client.

NOTE The `mergeWhen` method should not be used within arrays that mix string and numeric keys. Furthermore, it should not be used within arrays with numeric keys that are not ordered sequentially.

Conditional Relationships

In addition to conditionally loading attributes, you may conditionally include relationships on your resource responses based on if the relationship has already been loaded on the model. This allows your controller to decide which relationships should be loaded on the model and your resource can easily include them only when they have actually been loaded.

Ultimately, this makes it easier to avoid "N+1" query problems within your resources. The `whenLoaded` method may be used to conditionally load a relationship. In order to avoid unnecessarily loading relationships, this method accepts the name of the relationship instead of the relationship itself:

```
/**
 * Transform the resource into an array.
 *
 * @param \Illuminate\Http\Request $request
 * @return array
 */
public function toArray($request)
{
    return [
        'id' => $this->id,
        'name' => $this->name,
        'email' => $this->email,
        'posts' => PostResource::collection($this->whenLoaded('posts')),
        'created_at' => $this->created_at,
        'updated_at' => $this->updated_at,
    ];
}
```

In this example, if the relationship has not been loaded, the `posts` key will be removed from the resource response entirely before it is sent to the client.

Conditional Pivot Information

In addition to conditionally including relationship information in your resource responses, you may conditionally include data from the intermediate tables of many-to-many relationships using the `whenPivotLoaded` method. The `whenPivotLoaded` method accepts the name of the pivot table as its first argument. The second argument should be a Closure that defines the value to be returned if the pivot information is available on the model:

```
/**
 * Transform the resource into an array.
 *
 * @param \Illuminate\Http\Request $request
 * @return array
 */
public function toArray($request)
{
    return [
        'id' => $this->id,
        'name' => $this->name,
        'expires_at' => $this->whenPivotLoaded('role_user', function () {
            return $this->pivot->expires_at;
        }),
    ];
}
```

If your intermediate table is using an accessor other than `pivot`, you may use the `whenPivotLoadedAs` method:

```
/**
 * Transform the resource into an array.
 *
 * @param \Illuminate\Http\Request $request
 * @return array
 */
public function toArray($request)
{
    return [
```

```
        'id' => $this->id,
        'name' => $this->name,
        'expires_at' => $this->whenPivotLoadedAs('subscription', 'role_user', function () {
            return $this->subscription->expires_at;
        }),
    ];
}
```

Adding Meta Data

Some JSON API standards require the addition of meta data to your resource and resource collections responses. This often includes things like `links` to the resource or related resources, or meta data about the resource itself. If you need to return additional meta data about a resource, include it in your `toArray` method. For example, you might include `link` information when transforming a resource collection:

```
/**
 * Transform the resource into an array.
 *
 * @param \Illuminate\Http\Request $request
 * @return array
 */
public function toArray($request)
{
    return [
        'data' => $this->collection,
        'links' => [
            'self' => 'link-value',
        ],
    ];
}
```

When returning additional meta data from your resources, you never have to worry about accidentally overriding the `links` or `meta` keys that are automatically added by Laravel when returning paginated responses. Any additional links you define will be merged with the links provided by the paginator.

Top Level Meta Data

Sometimes you may wish to only include certain meta data with a resource response if the resource is the outermost resource being returned. Typically, this includes meta information about the response as a whole. To define this meta data, add a `with` method to your resource class. This method should return an array of meta data to be included with the resource response only when the resource is the outermost resource being rendered:

```
<?php

namespace App\Http\Resources;

use Illuminate\Http\Resources\Json\ResourceCollection;

class UserCollection extends ResourceCollection
{
    /**
     * Transform the resource collection into an array.
     *
     * @param \Illuminate\Http\Request $request
     * @return array
     */
    public function toArray($request)
    {
        return parent::toArray($request);
    }

    /**
     * Get additional data that should be returned with the resource array.
     *
     * @param \Illuminate\Http\Request $request
     * @return array
     */
    public function with($request)
    {
        return [
            'meta' => [
                'key' => 'value',
            ],
        ];
    }
}
```

```
    }  
}
```

Adding Meta Data When Constructing Resources

You may also add top-level data when constructing resource instances in your route or controller. The `additional` method, which is available on all resources, accepts an array of data that should be added to the resource response:

```
return (new UserCollection(User::all()->load('roles')))  
    ->additional(['meta' => [  
        'key' => 'value',  
    ]]);
```

Resource Responses

As you have already read, resources may be returned directly from routes and controllers:

```
use App\Http\Resources\User as UserResource;  
use App\User;  
  
Route::get('/user', function () {  
    return new UserResource(User::find(1));  
});
```

However, sometimes you may need to customize the outgoing HTTP response before it is sent to the client. There are two ways to accomplish this. First, you may chain the `response` method onto the resource. This method will return an `Illuminate\Http\JsonResponse` instance, allowing you full control of the response's headers:

```
use App\Http\Resources\User as UserResource;  
use App\User;  
  
Route::get('/user', function () {  
    return (new UserResource(User::find(1)))  
        ->response()  
        ->header('X-Value', 'True');  
});
```

Alternatively, you may define a `withResponse` method within the resource itself. This method will be called when the resource is returned as the outermost resource in a response:

```
<?php  
  
namespace App\Http\Resources;  
  
use Illuminate\Http\Resources\Json\JsonResource;  
  
class User extends JsonResource  
{  
    /**  
     * Transform the resource into an array.  
     *  
     * @param \Illuminate\Http\Request $request  
     * @return array  
     */  
    public function toArray($request)  
    {  
        return [  
            'id' => $this->id,  
        ];  
    }  
  
    /**  
     * Customize the outgoing response for the resource.  
     *  
     * @param \Illuminate\Http\Request $request  
     * @param \Illuminate\Http\Response $response  
     * @return void  
     */  
    public function withResponse($request, $response)  
    {  
        $response->header('X-Value', 'True');  
    }  
}
```

```
} }
```

Eloquent ORM

Serialization

Eloquent: Serialization

- [Introduction](#)
- [Serializing Models & Collections](#)
 - [Serializing To Arrays](#)
 - [Serializing To JSON](#)
- [Hiding Attributes From JSON](#)
- [Appending Values To JSON](#)
- [Date Serialization](#)

Introduction

When building JSON APIs, you will often need to convert your models and relationships to arrays or JSON. Eloquent includes convenient methods for making these conversions, as well as controlling which attributes are included in your serializations.

Serializing Models & Collections

Serializing To Arrays

To convert a model and its loaded [relationships](#) to an array, you should use the `toArray` method. This method is recursive, so all attributes and all relations (including the relations of relations) will be converted to arrays:

```
$user = App\User::with('roles')->first();  
  
return $user->toArray();
```

To convert only a model's attributes to an array, use the `attributesToArray` method:

```
$user = App\User::first();  
  
return $user->attributesToArray();
```

You may also convert entire [collections](#) of models to arrays:

```
$users = App\User::all();  
  
return $users->toArray();
```

Serializing To JSON

To convert a model to JSON, you should use the `toJson` method. Like `toArray`, the `toJson` method is recursive, so all attributes and relations will be converted to JSON. You may also specify JSON encoding options [supported by PHP](#):

```
$user = App\User::find(1);  
  
return $user->toJson();  
  
return $user->toJson(JSON_PRETTY_PRINT);
```

Alternatively, you may cast a model or collection to a string, which will automatically call the `toJson` method on the model or collection:

```
$user = App\User::find(1);  
  
return (string) $user;
```

Since models and collections are converted to JSON when cast to a string, you can return Eloquent objects

directly from your application's routes or controllers:

```
Route::get('users', function () {
    return App\User::all();
});
```

Relationships

When an Eloquent model is converted to JSON, its loaded relationships will automatically be included as attributes on the JSON object. Also, though Eloquent relationship methods are defined using "camel case", a relationship's JSON attribute will be "snake case".

Hiding Attributes From JSON

Sometimes you may wish to limit the attributes, such as passwords, that are included in your model's array or JSON representation. To do so, add a `$hidden` property to your model:

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;

class User extends Model
{
    /**
     * The attributes that should be hidden for arrays.
     *
     * @var array
     */
    protected $hidden = ['password'];
}
```

NOTE When hiding relationships, use the relationship's method name.

Alternatively, you may use the `visible` property to define a white-list of attributes that should be included in your model's array and JSON representation. All other attributes will be hidden when the model is converted to an array or JSON:

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;

class User extends Model
{
    /**
     * The attributes that should be visible in arrays.
     *
     * @var array
     */
    protected $visible = ['first_name', 'last_name'];
}
```

Temporarily Modifying Attribute Visibility

If you would like to make some typically hidden attributes visible on a given model instance, you may use the `makeVisible` method. The `makeVisible` method returns the model instance for convenient method chaining:

```
return $user->makeVisible('attribute')->toArray();
```

Likewise, if you would like to make some typically visible attributes hidden on a given model instance, you may use the `makeHidden` method.

```
return $user->makeHidden('attribute')->toArray();
```

Appending Values To JSON

Occasionally, when casting models to an array or JSON, you may wish to add attributes that do not have a corresponding column in your database. To do so, first define an [accessor](#) for the value:

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;

class User extends Model
{
    /**
     * Get the administrator flag for the user.
     *
     * @return bool
     */
    public function getIsAdminAttribute()
    {
        return $this->attributes['admin'] === 'yes';
    }
}
```

After creating the accessor, add the attribute name to the `appends` property on the model. Note that attribute names are typically referenced in "snake case", even though the accessor is defined using "camel case":

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;

class User extends Model
{
    /**
     * The accessors to append to the model's array form.
     *
     * @var array
     */
    protected $appends = ['is_admin'];
}
```

Once the attribute has been added to the `appends` list, it will be included in both the model's array and JSON representations. Attributes in the `appends` array will also respect the `visible` and `hidden` settings configured on the model.

Appending At Run Time

You may instruct a single model instance to append attributes using the `append` method. Or, you may use the `setAppends` method to override the entire array of appended properties for a given model instance:

```
return $user->append('is_admin')->toArray();

return $user->setAppends(['is_admin'])->toArray();
```

Date Serialization

Customizing The Default Date Format

You may customize the default serialization format by overriding the `serializeDate` method:

```
/**
 * Prepare a date for array / JSON serialization.
 *
 * @param \DateTimeInterface $date
 * @return string
 */
protected function serializeDate(DateTimeInterface $date)
{
    return $date->format('Y-m-d');
}
```

Customizing The Date Format Per Attribute

You may customize the serialization format of individual Eloquent date attributes by specifying the date format in the [cast declaration](#):

```
protected $casts = [  
    'birthday' => 'date:Y-m-d',  
    'joined_at' => 'datetime:Y-m-d H:00',  
];
```

Testing

Getting Started

Testing: Getting Started

- [Introduction](#)
- [Environment](#)
- [Creating & Running Tests](#)
 - [Artisan Test Runner](#)

Introduction

Laravel is built with testing in mind. In fact, support for testing with PHPUnit is included out of the box and a `phpunit.xml` file is already set up for your application. The framework also ships with convenient helper methods that allow you to expressively test your applications.

By default, your application's `tests` directory contains two directories: `Feature` and `Unit`. Unit tests are tests that focus on a very small, isolated portion of your code. In fact, most unit tests probably focus on a single method. Feature tests may test a larger portion of your code, including how several objects interact with each other or even a full HTTP request to a JSON endpoint.

An `ExampleTest.php` file is provided in both the `Feature` and `Unit` test directories. After installing a new Laravel application, run `vendor/bin/phpunit` on the command line to run your tests.

Environment

When running tests via `vendor/bin/phpunit`, Laravel will automatically set the configuration environment to `testing` because of the environment variables defined in the `phpunit.xml` file. Laravel also automatically configures the session and cache to the `array` driver while testing, meaning no session or cache data will be persisted while testing.

You are free to define other testing environment configuration values as necessary. The testing environment variables may be configured in the `phpunit.xml` file, but make sure to clear your configuration cache using the `config:clear` Artisan command before running your tests!

In addition, you may create a `.env.testing` file in the root of your project. This file will override the `.env` file when running PHPUnit tests or executing Artisan commands with the `--env=testing` option.

Creating & Running Tests

To create a new test case, use the `make:test` Artisan command:

```
// Create a test in the Feature directory...
php artisan make:test UserTest

// Create a test in the Unit directory...
php artisan make:test UserTest --unit
```

TIP Test stubs may be customized using [stub publishing](#)

Once the test has been generated, you may define test methods as you normally would using PHPUnit. To run your tests, execute the `phpunit` or `artisan test` command from your terminal:

```
<?php

namespace Tests\Unit;

use PHPUnit\Framework\TestCase;

class ExampleTest extends TestCase
{
```

```
/**
 * A basic test example.
 *
 * @return void
 */
public function testBasicTest()
{
    $this->assertTrue(true);
}
}
```

NOTE If you define your own `setUp` / `tearDown` methods within a test class, be sure to call the respective `parent::setUp()` / `parent::tearDown()` methods on the parent class.

Artisan Test Runner

In addition to the `phpunit` command, you may use the `test` Artisan command to run your tests. The Artisan test runner provides more information regarding the test that is currently running and will automatically stop on the first test failure:

```
php artisan test
```

Any arguments that can be passed to the `phpunit` command may also be passed to the Artisan test command:

```
php artisan test --group=feature
```

Testing

HTTP Tests

HTTP Tests

- [Introduction](#)
 - [Customizing Request Headers](#)
 - [Cookies](#)
 - [Debugging Responses](#)
- [Session / Authentication](#)
- [Testing JSON APIs](#)
- [Testing File Uploads](#)
- [Available Assertions](#)
 - [Response Assertions](#)
 - [Authentication Assertions](#)

Introduction

Laravel provides a very fluent API for making HTTP requests to your application and examining the output. For example, take a look at the feature test defined below:

```
<?php

namespace Tests\Feature;

use Illuminate\Foundation\Testing\RefreshDatabase;
use Illuminate\Foundation\Testing\WithoutMiddleware;
use Tests\TestCase;

class ExampleTest extends TestCase
{
    /**
     * A basic test example.
     *
     * @return void
     */
    public function testBasicTest()
    {
        $response = $this->get('/');

        $response->assertStatus(200);
    }
}
```

The `get` method makes a GET request into the application, while the `assertStatus` method asserts that the returned response should have the given HTTP status code. In addition to this simple assertion, Laravel also contains a variety of assertions for inspecting the response headers, content, JSON structure, and more.

Customizing Request Headers

You may use the `withHeaders` method to customize the request's headers before it is sent to the application. This allows you to add any custom headers you would like to the request:

```
<?php

class ExampleTest extends TestCase
{
    /**
     * A basic functional test example.
     *
     * @return void
     */
    public function testBasicExample()
    {
        $response = $this->withHeaders([
            'X-Header' => 'Value',
        ]->json('POST', '/user', ['name' => 'Sally']));
    }
}
```

```
        $response
            ->assertStatus(201)
            ->assertJson([
                'created' => true,
            ]);
    }
}
```

TIP The CSRF middleware is automatically disabled when running tests.

Cookies

You may use the `withCookie` or `withCookies` methods to set cookie values before making a request. The `withCookie` method accepts a cookie name and value as its two arguments, while the `withCookies` method accepts an array of name / value pairs:

```
<?php

class ExampleTest extends TestCase
{
    public function testCookies()
    {
        $response = $this->withCookie('color', 'blue')->get('/');

        $response = $this->withCookies([
            'color' => 'blue',
            'name' => 'Taylor',
        ])->get('/');
    }
}
```

Debugging Responses

After making a test request to your application, the `dump`, `dumpHeaders`, and `dumpSession` methods may be used to examine and debug the response contents:

```
<?php

namespace Tests\Feature;

use Illuminate\Foundation\Testing\RefreshDatabase;
use Illuminate\Foundation\Testing\WithoutMiddleware;
use Tests\TestCase;

class ExampleTest extends TestCase
{
    /**
     * A basic test example.
     *
     * @return void
     */
    public function testBasicTest()
    {
        $response = $this->get('/');

        $response->dumpHeaders();

        $response->dumpSession();

        $response->dump();
    }
}
```

Session / Authentication

Laravel provides several helpers for working with the session during HTTP testing. First, you may set the session data to a given array using the `withSession` method. This is useful for loading the session with data before issuing a request to your application:

```
<?php
```

```
class ExampleTest extends TestCase
{
    public function testApplication()
    {
        $response = $this->withSession(['foo' => 'bar'])
            ->get('/');
    }
}
```

One common use of the session is for maintaining state for the authenticated user. The `actingAs` helper method provides a simple way to authenticate a given user as the current user. For example, we may use a [model factory](#) to generate and authenticate a user:

```
<?php

use App\User;

class ExampleTest extends TestCase
{
    public function testApplication()
    {
        $user = factory(User::class)->create();

        $response = $this->actingAs($user)
            ->withSession(['foo' => 'bar'])
            ->get('/');
    }
}
```

You may also specify which guard should be used to authenticate the given user by passing the guard name as the second argument to the `actingAs` method:

```
$this->actingAs($user, 'api')
```

Testing JSON APIs

Laravel also provides several helpers for testing JSON APIs and their responses. For example, the `json`, `getJson`, `postJson`, `putJson`, `patchJson`, `deleteJson`, and `optionsJson` methods may be used to issue JSON requests with various HTTP verbs. You may also easily pass data and headers to these methods. To get started, let's write a test to make a `POST` request to `/user` and assert that the expected data was returned:

```
<?php

class ExampleTest extends TestCase
{
    /**
     * A basic functional test example.
     *
     * @return void
     */
    public function testBasicExample()
    {
        $response = $this->postJson('/user', ['name' => 'Sally']);

        $response
            ->assertStatus(201)
            ->assertJson([
                'created' => true,
            ]);
    }
}
```

TIP The `assertJson` method converts the response to an array and utilizes `PHPUnit::assertArraySubset` to verify that the given array exists within the JSON response returned by the application. So, if there are other properties in the JSON response, this test will still pass as long as the given fragment is present.

In addition, JSON response data may be accessed as array variables on the response:

```
$this->assertTrue($response['created']);
```

Verifying An Exact JSON Match

If you would like to verify that the given array is an **exact** match for the JSON returned by the application, you should use the `assertExactJson` method:

```
<?php

class ExampleTest extends TestCase
{
    /**
     * A basic functional test example.
     *
     * @return void
     */
    public function testBasicExample()
    {
        $response = $this->json('POST', '/user', ['name' => 'Sally']);

        $response
            ->assertStatus(201)
            ->assertExactJson([
                'created' => true,
            ]);
    }
}
```

Verifying JSON Paths

If you would like to verify that the JSON response contains some given data at a specified path, you should use the `assertJsonPath` method:

```
<?php

class ExampleTest extends TestCase
{
    /**
     * A basic functional test example.
     *
     * @return void
     */
    public function testBasicExample()
    {
        $response = $this->json('POST', '/user', ['name' => 'Sally']);

        $response
            ->assertStatus(201)
            ->assertJsonPath('team.owner.name', 'foo')
    }
}
```

Testing File Uploads

The `Illuminate\Http\UploadedFile` class provides a `fake` method which may be used to generate dummy files or images for testing. This, combined with the `Storage` facade's `fake` method greatly simplifies the testing of file uploads. For example, you may combine these two features to easily test an avatar upload form:

```
<?php

namespace Tests\Feature;

use Illuminate\Foundation\Testing\RefreshDatabase;
use Illuminate\Foundation\Testing\WithoutMiddleware;
use Illuminate\Http\UploadedFile;
use Illuminate\Support\Facades\Storage;
use Tests\TestCase;

class ExampleTest extends TestCase
{
    public function testAvatarUpload()
    {
        Storage::fake('avatars');

        $file = UploadedFile::fake()->image('avatar.jpg');

        $response = $this->json('POST', '/avatar', [
            'avatar' => $file,
        ]);
    }
}
```

```
    });

    // Assert the file was stored...
    Storage::disk('avatars')->assertExists($file->hashName());

    // Assert a file does not exist...
    Storage::disk('avatars')->assertMissing('missing.jpg');
}
}
```

Fake File Customization

When creating files using the `fake` method, you may specify the width, height, and size of the image in order to better test your validation rules:

```
UploadedFile::fake()->image('avatar.jpg', $width, $height)->size(100);
```

In addition to creating images, you may create files of any other type using the `create` method:

```
UploadedFile::fake()->create('document.pdf', $sizeInKilobytes);
```

If needed, you may pass a `$mimeType` argument to the method to explicitly define the MIME type that should be returned by the file:

```
UploadedFile::fake()->create('document.pdf', $sizeInKilobytes, 'application/pdf');
```

Available Assertions

Response Assertions

Laravel provides a variety of custom assertion methods for your [PHPUnit](#) feature tests. These assertions may be accessed on the response that is returned from the `json`, `get`, `post`, `put`, and `delete` test methods:

assertCookie	assertOk
assertCookieExpired	assertPlainCookie
assertCookieNotExpired	assertRedirect
assertCookieMissing	assertSee
assertCreated	assertSeeInOrder
assertDontSee	assertSeeText
assertDontSeeText	assertSeeTextInOrder
assertExactJson	assertSessionHas
assertForbidden	assertSessionHasInput
assertHeader	assertSessionHasAll
assertHeaderMissing	assertSessionHasErrors
assertJson	assertSessionHasErrorsIn
assertJsonCount	assertSessionHasNoErrors
assertJsonFragment	assertSessionDoesntHaveErrors
assertJsonMissing	assertSessionMissing
assertJsonMissingExact	assertStatus
assertJsonMissingValidationErrors	assertSuccessful
assertJsonPath	assertUnauthorized
assertJsonStructure	assertViewHas
assertJsonValidationErrors	assertViewHasAll
assertLocation	assertViewIs
assertNoContent	assertViewMissing
assertNotFound	

assertCookie

Assert that the response contains the given cookie:

```
$response->assertCookie($cookieName, $value = null);
```

assertCookieExpired

Assert that the response contains the given cookie and it is expired:

```
$response->assertCookieExpired($cookieName);
```

assertCookieNotExpired

Assert that the response contains the given cookie and it is not expired:

```
$response->assertCookieNotExpired($cookieName);
```

assertCookieMissing

Assert that the response does not contains the given cookie:

```
$response->assertCookieMissing($cookieName);
```

assertCreated

Assert that the response has a 201 status code:

```
$response->assertCreated();
```

assertDontSee

Assert that the given string is not contained within the response. This assertion will automatically escape the given string unless you pass a second argument of `false`:

```
$response->assertDontSee($value, $escaped = true);
```

assertDontSeeText

Assert that the given string is not contained within the response text. This assertion will automatically escape the given string unless you pass a second argument of `false`:

```
$response->assertDontSeeText($value, $escaped = true);
```

assertExactJson

Assert that the response contains an exact match of the given JSON data:

```
$response->assertExactJson(array $data);
```

assertForbidden

Assert that the response has a forbidden (403) status code:

```
$response->assertForbidden();
```

assertHeader

Assert that the given header is present on the response:

```
$response->assertHeader($headerName, $value = null);
```

assertHeaderMissing

Assert that the given header is not present on the response:

```
$response->assertHeaderMissing($headerName);
```

assertJson

Assert that the response contains the given JSON data:

```
$response->assertJson(array $data, $strict = false);
```

assertJsonCount

Assert that the response JSON has an array with the expected number of items at the given key:

```
$response->assertJsonCount($count, $key = null);
```

assertJsonFragment

Assert that the response contains the given JSON fragment:

```
$response->assertJsonFragment(array $data);
```

assertJsonMissing

Assert that the response does not contain the given JSON fragment:

```
$response->assertJsonMissing(array $data);
```

assertJsonMissingExact

Assert that the response does not contain the exact JSON fragment:

```
$response->assertJsonMissingExact(array $data);
```

assertJsonMissingValidationErrors

Assert that the response has no JSON validation errors for the given keys:

```
$response->assertJsonMissingValidationErrors($keys);
```

assertJsonPath

Assert that the response contains the given data at the specified path:

```
$response->assertJsonPath($path, array $data, $strict = false);
```

assertJsonStructure

Assert that the response has a given JSON structure:

```
$response->assertJsonStructure(array $structure);
```

assertJsonValidationErrors

Assert that the response has the given JSON validation errors:

```
$response->assertJsonValidationErrors(array $data);
```

assertLocation

Assert that the response has the given URI value in the `Location` header:

```
$response->assertLocation($uri);
```

assertNoContent

Assert that the response has the given status code and no content.

```
$response->assertNoContent($status = 204);
```

assertNotFound

Assert that the response has a not found status code:

```
$response->assertNotFound();
```

assertOk

Assert that the response has a 200 status code:

```
$response->assertOk();
```

assertPlainCookie

Assert that the response contains the given cookie (unencrypted):

```
$response->assertPlainCookie($cookieName, $value = null);
```

assertRedirect

Assert that the response is a redirect to a given URI:

```
$response->assertRedirect($uri);
```

assertSee

Assert that the given string is contained within the response. This assertion will automatically escape the given string unless you pass a second argument of `false`:

```
$response->assertSee($value, $escaped = true);
```

assertSeeInOrder

Assert that the given strings are contained in order within the response. This assertion will automatically escape the given strings unless you pass a second argument of `false`:

```
$response->assertSeeInOrder(array $values, $escaped = true);
```

assertSeeText

Assert that the given string is contained within the response text. This assertion will automatically escape the given string unless you pass a second argument of `false`:

```
$response->assertSeeText($value, $escaped = true);
```

assertSeeTextInOrder

Assert that the given strings are contained in order within the response text. This assertion will automatically escape the given strings unless you pass a second argument of `false`:

```
$response->assertSeeTextInOrder(array $values, $escaped = true);
```

assertSessionHas

Assert that the session contains the given piece of data:

```
$response->assertSessionHas($key, $value = null);
```

assertSessionHasInput

Assert that the session has a given value in the flashed input array:

```
$response->assertSessionHasInput($key, $value = null);
```

assertSessionHasAll

Assert that the session has a given list of values:

```
$response->assertSessionHasAll(array $data);
```

assertSessionHasErrors

Assert that the session contains an error for the given \$keys. If \$keys is an associative array, assert that the session contains a specific error message (value) for each field (key):

```
$response->assertSessionHasErrors(array $keys, $format = null, $errorBag = 'default');
```

assertSessionHasErrorsIn

Assert that the session contains an error for the given \$keys, within a specific error bag. If \$keys is an associative array, assert that the session contains a specific error message (value) for each field (key), within the error bag:

```
$response->assertSessionHasErrorsIn($errorBag, $keys = [], $format = null);
```

assertSessionHasNoErrors

Assert that the session has no errors:

```
$response->assertSessionHasNoErrors();
```

assertSessionDoesntHaveErrors

Assert that the session has no errors for the given keys:

```
$response->assertSessionDoesntHaveErrors($keys = [], $format = null, $errorBag = 'default');
```

assertSessionMissing

Assert that the session does not contain the given key:

```
$response->assertSessionMissing($key);
```

assertStatus

Assert that the response has a given code:

```
$response->assertStatus($code);
```

assertSuccessful

Assert that the response has a successful (≥ 200 and < 300) status code:

```
$response->assertSuccessful();
```

assertUnauthorized

Assert that the response has an unauthorized (401) status code:

```
$response->assertUnauthorized();
```

assertViewHas

Assert that the response view was given a piece of data:

```
$response->assertViewHas($key, $value = null);
```

In addition, view data may be accessed as array variables on the response:

```
$this->assertEquals('Taylor', $response['name']);
```

assertViewHasAll

Assert that the response view has a given list of data:

```
$response->assertViewHasAll(array $data);
```

assertViewIs

Assert that the given view was returned by the route:

```
$response->assertViewIs($value);
```

assertViewMissing

Assert that the response view is missing a piece of bound data:

```
$response->assertViewMissing($key);
```

Authentication Assertions

Laravel also provides a variety of authentication related assertions for your [PHPUnit](#) feature tests:

Method	Description
<code>\$this->assertAuthenticated(\$guard = null);</code>	Assert that the user is authenticated.
<code>\$this->assertGuest(\$guard = null);</code>	Assert that the user is not authenticated.
<code>\$this->assertAuthenticatedAs(\$user, \$guard = null);</code>	Assert that the given user is authenticated.
<code>\$this->assertCredentials(array \$credentials, \$guard = null);</code>	Assert that the given credentials are valid.
<code>\$this->assertInvalidCredentials(array \$credentials, \$guard = null);</code>	Assert that the given credentials are invalid.

Testing

Console Tests

Console Tests

- [Introduction](#)
- [Expecting Input / Output](#)

Introduction

In addition to simplifying HTTP testing, Laravel provides a simple API for testing console applications that ask for user input.

Expecting Input / Output

Laravel allows you to easily "mock" user input for your console commands using the `expectsQuestion` method. In addition, you may specify the exit code and text that you expect to be output by the console command using the `assertExitCode` and `expectsOutput` methods. For example, consider the following console command:

```
Artisan::command('question', function () {
    $name = $this->ask('What is your name?');

    $language = $this->choice('Which language do you program in?', [
        'PHP',
        'Ruby',
        'Python',
    ]);

    $this->line('Your name is '.$name.' and you program in '.$language.'.');
});
```

You may test this command with the following test which utilizes the `expectsQuestion`, `expectsOutput`, and `assertExitCode` methods:

```
/**
 * Test a console command.
 *
 * @return void
 */
public function testConsoleCommand()
{
    $this->artisan('question')
        ->expectsQuestion('What is your name?', 'Taylor Otwell')
        ->expectsQuestion('Which language do you program in?', 'PHP')
        ->expectsOutput('Your name is Taylor Otwell and you program in PHP.')
        ->assertExitCode(0);
}
```

When writing a command which expects a confirmation in the form of a "yes" or "no" answer, you may utilize the `expectsConfirmation` method:

```
$this->artisan('module:import')
    ->expectsConfirmation('Do you really wish to run this command?', 'no')
    ->assertExitCode(1);
```


Testing

Browser Tests

Laravel Dusk

- [Introduction](#)
- [Installation](#)
 - [Managing ChromeDriver Installations](#)
 - [Using Other Browsers](#)
- [Getting Started](#)
 - [Generating Tests](#)
 - [Running Tests](#)
 - [Environment Handling](#)
 - [Creating Browsers](#)
 - [Browser Macros](#)
 - [Authentication](#)
 - [Database Migrations](#)
 - [Cookies](#)
 - [Taking A Screenshot](#)
 - [Storing Console Output To Disk](#)
 - [Storing Page Source To Disk](#)
- [Interacting With Elements](#)
 - [Dusk Selectors](#)
 - [Clicking Links](#)
 - [Text, Values, & Attributes](#)
 - [Using Forms](#)
 - [Attaching Files](#)
 - [Using The Keyboard](#)
 - [Using The Mouse](#)
 - [JavaScript Dialogs](#)
 - [Scoping Selectors](#)
 - [Waiting For Elements](#)
 - [Scrolling An Element Into View](#)
 - [Making Vue Assertions](#)
- [Available Assertions](#)
- [Pages](#)
 - [Generating Pages](#)
 - [Configuring Pages](#)
 - [Navigating To Pages](#)
 - [Shorthand Selectors](#)
 - [Page Methods](#)
- [Components](#)
 - [Generating Components](#)
 - [Using Components](#)
- [Continuous Integration](#)
 - [CircleCI](#)
 - [Codeship](#)
 - [Heroku CI](#)
 - [Travis CI](#)
 - [GitHub Actions](#)

Introduction

Laravel Dusk provides an expressive, easy-to-use browser automation and testing API. By default, Dusk does not require you to install JDK or Selenium on your machine. Instead, Dusk uses a standalone [ChromeDriver](#) installation. However, you are free to utilize any other Selenium compatible driver you wish.

Installation

To get started, you should add the `laravel/dusk` Composer dependency to your project:

```
composer require --dev laravel/dusk
```

NOTE If you are manually registering Dusk's service provider, you should **never** register it in your production environment, as doing so could lead to arbitrary users being able to authenticate with your application.

After installing the Dusk package, run the `dusk:install` Artisan command:

```
php artisan dusk:install
```

A `Browser` directory will be created within your `tests` directory and will contain an example test. Next, set the `APP_URL` environment variable in your `.env` file. This value should match the URL you use to access your application in a browser.

To run your tests, use the `dusk` Artisan command. The `dusk` command accepts any argument that is also accepted by the `phpunit` command:

```
php artisan dusk
```

If you had test failures the last time you ran the `dusk` command, you may save time by re-running the failing tests first using the `dusk:fails` command:

```
php artisan dusk:fails
```

Managing ChromeDriver Installations

If you would like to install a different version of ChromeDriver than what is included with Laravel Dusk, you may use the `dusk:chrome-driver` command:

```
# Install the latest version of ChromeDriver for your OS...
php artisan dusk:chrome-driver
```

```
# Install a given version of ChromeDriver for your OS...
php artisan dusk:chrome-driver 74
```

```
# Install a given version of ChromeDriver for all supported OSs...
php artisan dusk:chrome-driver --all
```

NOTE Dusk requires the `chromedriver` binaries to be executable. If you're having problems running Dusk, you should ensure the binaries are executable using the following command: `chmod -R 0755 vendor/laravel/dusk/bin/`.

Using Other Browsers

By default, Dusk uses Google Chrome and a standalone [ChromeDriver](#) installation to run your browser tests. However, you may start your own Selenium server and run your tests against any browser you wish.

To get started, open your `tests/DuskTestCase.php` file, which is the base Dusk test case for your application. Within this file, you can remove the call to the `startChromeDriver` method. This will stop Dusk from automatically starting the ChromeDriver:

```
/**
 * Prepare for Dusk test execution.
 *
 * @beforeClass
 * @return void
 */
public static function prepare()
{
    // static::startChromeDriver();
}
```

Next, you may modify the `driver` method to connect to the URL and port of your choice. In addition, you may

modify the "desired capabilities" that should be passed to the WebDriver:

```
/**
 * Create the RemoteWebDriver instance.
 *
 * @return \Facebook\WebDriver\Remote\RemoteWebDriver
 */
protected function driver()
{
    return RemoteWebDriver::create(
        'http://localhost:4444/wd/hub', DesiredCapabilities::phantomjs()
    );
}
```

Getting Started

Generating Tests

To generate a Dusk test, use the `dusk:make` Artisan command. The generated test will be placed in the `tests/Browser` directory:

```
php artisan dusk:make LoginTest
```

Running Tests

To run your browser tests, use the `dusk` Artisan command:

```
php artisan dusk
```

If you had test failures the last time you ran the `dusk` command, you may save time by re-running the failing tests first using the `dusk:fails` command:

```
php artisan dusk:fails
```

The `dusk` command accepts any argument that is normally accepted by the PHPUnit test runner, allowing you to only run the tests for a given [group](#), etc:

```
php artisan dusk --group=foo
```

Manually Starting ChromeDriver

By default, Dusk will automatically attempt to start ChromeDriver. If this does not work for your particular system, you may manually start ChromeDriver before running the `dusk` command. If you choose to start ChromeDriver manually, you should comment out the following line of your `tests/DuskTestCase.php` file:

```
/**
 * Prepare for Dusk test execution.
 *
 * @beforeClass
 * @return void
 */
public static function prepare()
{
    // static::startChromeDriver();
}
```

In addition, if you start ChromeDriver on a port other than 9515, you should modify the `driver` method of the same class:

```
/**
 * Create the RemoteWebDriver instance.
 *
 * @return \Facebook\WebDriver\Remote\RemoteWebDriver
 */
protected function driver()
{
    return RemoteWebDriver::create(
        'http://localhost:9515', DesiredCapabilities::chrome()
    );
}
```

```
}
```

Environment Handling

To force Dusk to use its own environment file when running tests, create a `.env.dusk.{environment}` file in the root of your project. For example, if you will be initiating the `dusk` command from your `local` environment, you should create a `.env.dusk.local` file.

When running tests, Dusk will back-up your `.env` file and rename your Dusk environment to `.env`. Once the tests have completed, your `.env` file will be restored.

Creating Browsers

To get started, let's write a test that verifies we can log into our application. After generating a test, we can modify it to navigate to the login page, enter some credentials, and click the "Login" button. To create a browser instance, call the `browse` method:

```
<?php

namespace Tests\Browser;

use App\User;
use Illuminate\Foundation\Testing\DatabaseMigrations;
use Laravel\Dusk\Chrome;
use Tests\DuskTestCase;

class ExampleTest extends DuskTestCase
{
    use DatabaseMigrations;

    /**
     * A basic browser test example.
     *
     * @return void
     */
    public function testBasicExample()
    {
        $user = factory(User::class)->create([
            'email' => 'taylor@laravel.com',
        ]);

        $this->browse(function ($browser) use ($user) {
            $browser->visit('/login')
                ->type('email', $user->email)
                ->type('password', 'password')
                ->press('Login')
                ->assertPathIs('/home');
        });
    }
}
```

As you can see in the example above, the `browse` method accepts a callback. A browser instance will automatically be passed to this callback by Dusk and is the main object used to interact with and make assertions against your application.

Creating Multiple Browsers

Sometimes you may need multiple browsers in order to properly carry out a test. For example, multiple browsers may be needed to test a chat screen that interacts with websockets. To create multiple browsers, "ask" for more than one browser in the signature of the callback given to the `browse` method:

```
$this->browse(function ($first, $second) {
    $first->loginAs(User::find(1))
        ->visit('/home')
        ->waitForText('Message');

    $second->loginAs(User::find(2))
        ->visit('/home')
        ->waitForText('Message')
        ->type('message', 'Hey Taylor')
        ->press('Send');
```

```
$first->waitForText('Hey Taylor')
    ->assertSee('Jeffrey Way');
});
```

Resizing Browser Windows

You may use the `resize` method to adjust the size of the browser window:

```
$browser->resize(1920, 1080);
```

The `maximize` method may be used to maximize the browser window:

```
$browser->maximize();
```

The `fitContent` method will resize the browser window to match the size of the content:

```
$browser->fitContent();
```

When a test fails, Dusk will automatically resize the browser to fit the content prior to taking a screenshot. You may disable this feature by calling the `disableFitOnFailure` method within your test:

```
$browser->disableFitOnFailure();
```

You may use the `move` method to move the browser window to a different position on your screen:

```
$browser->move(100, 100);
```

Browser Macros

If you would like to define a custom browser method that you can re-use in a variety of your tests, you may use the `macro` method on the `Browser` class. Typically, you should call this method from a [service provider's](#) `boot` method:

```
<?php

namespace App\Providers;

use Illuminate\Support\ServiceProvider;
use Laravel\Dusk\Browser;

class DuskServiceProvider extends ServiceProvider
{
    /**
     * Register the Dusk's browser macros.
     *
     * @return void
     */
    public function boot()
    {
        Browser::macro('scrollToElement', function ($element = null) {
            $this->script("$('#html, body').animate({ scrollTop: $('#$element').offset().top }, 0);");

            return $this;
        });
    }
}
```

The `macro` function accepts a name as its first argument, and a Closure as its second. The macro's Closure will be executed when calling the macro as a method on a `Browser` implementation:

```
$this->browse(function ($browser) use ($user) {
    $browser->visit('/pay')
        ->scrollToElement('#credit-card-details')
        ->assertSee('Enter Credit Card Details');
});
```

Authentication

Often, you will be testing pages that require authentication. You can use Dusk's `loginAs` method in order to

avoid interacting with the login screen during every test. The `loginAs` method accepts a user ID or user model instance:

```
$this->browse(function ($first, $second) {  
    $first->loginAs(User::find(1))  
        ->visit('/home');  
});
```

NOTE After using the `loginAs` method, the user session will be maintained for all tests within the file.

Database Migrations

When your test requires migrations, like the authentication example above, you should never use the `RefreshDatabase` trait. The `RefreshDatabase` trait leverages database transactions which will not be applicable across HTTP requests. Instead, use the `DatabaseMigrations` trait:

```
<?php  
  
namespace Tests\Browser;  
  
use App\User;  
use Illuminate\Foundation\Testing\DatabaseMigrations;  
use Laravel\Dusk\Chrome;  
use Tests\DuskTestCase;  
  
class ExampleTest extends DuskTestCase  
{  
    use DatabaseMigrations;  
}
```

Cookies

You may use the `cookie` method to get or set an encrypted cookie's value:

```
$browser->cookie('name');  
  
$browser->cookie('name', 'Taylor');
```

You may use the `plainCookie` method to get or set an unencrypted cookie's value:

```
$browser->plainCookie('name');  
  
$browser->plainCookie('name', 'Taylor');
```

You may use the `deleteCookie` method to delete the given cookie:

```
$browser->deleteCookie('name');
```

Taking A Screenshot

You may use the `screenshot` method to take a screenshot and store it with the given filename. All screenshots will be stored within the `tests/Browser/screenshots` directory:

```
$browser->screenshot('filename');
```

Storing Console Output To Disk

You may use the `storeConsoleLog` method to write the console output to disk with the given filename. Console output will be stored within the `tests/Browser/console` directory:

```
$browser->storeConsoleLog('filename');
```

Storing Page Source To Disk

You may use the `storeSource` method to write the page's current source to disk with the given filename. The page source will be stored within the `tests/Browser/source` directory:

```
$browser->storeSource('filename');
```

Interacting With Elements

Dusk Selectors

Choosing good CSS selectors for interacting with elements is one of the hardest parts of writing Dusk tests. Over time, frontend changes can cause CSS selectors like the following to break your tests:

```
// HTML...

<button>Login</button>

// Test...

$browser->click('.login-page .container div > button');
```

Dusk selectors allow you to focus on writing effective tests rather than remembering CSS selectors. To define a selector, add a dusk attribute to your HTML element. Then, prefix the selector with @ to manipulate the attached element within a Dusk test:

```
// HTML...

<button dusk="login-button">Login</button>

// Test...

$browser->click('@login-button');
```

Clicking Links

To click a link, you may use the `clickLink` method on the browser instance. The `clickLink` method will click the link that has the given display text:

```
$browser->clickLink($linkText);
```

You may use the `seeLink` method to determine if a link that has the given display text is visible on the page:

```
if ($browser->seeLink($linkText)) {
    // ...
}
```

NOTE These methods interact with jQuery. If jQuery is not available on the page, Dusk will automatically inject it into the page so it is available for the test's duration.

Text, Values, & Attributes

Retrieving & Setting Values

Dusk provides several methods for interacting with the current display text, value, and attributes of elements on the page. For example, to get the "value" of an element that matches a given selector, use the `value` method:

```
// Retrieve the value...
$value = $browser->value('selector');

// Set the value...
$browser->value('selector', 'value');
```

You may use the `inputValue` method to get the "value" of an input element that has a given field name:

```
// Retrieve the value of an input element...
$inputValue = $browser->inputValue('field');
```

Retrieving Text

The `text` method may be used to retrieve the display text of an element that matches the given selector:

```
$text = $browser->text('selector');
```

Retrieving Attributes

Finally, the `attribute` method may be used to retrieve an attribute of an element matching the given selector:

```
$attribute = $browser->attribute('selector', 'value');
```

Using Forms

Typing Values

Dusk provides a variety of methods for interacting with forms and input elements. First, let's take a look at an example of typing text into an input field:

```
$browser->type('email', 'taylor@laravel.com');
```

Note that, although the method accepts one if necessary, we are not required to pass a CSS selector into the `type` method. If a CSS selector is not provided, Dusk will search for an input field with the given `name` attribute. Finally, Dusk will attempt to find a `textarea` with the given `name` attribute.

To append text to a field without clearing its content, you may use the `append` method:

```
$browser->type('tags', 'foo')
    ->append('tags', ', bar, baz');
```

You may clear the value of an input using the `clear` method:

```
$browser->clear('email');
```

You can instruct Dusk to type slowly using the `typeSlowly` method. By default, Dusk will pause for 100 milliseconds between key presses. To customize the amount of time between key presses, you may pass the appropriate number of milliseconds as the third argument to the method:

```
$browser->typeSlowly('mobile', '+1 (202) 555-5555');
$browser->typeSlowly('mobile', '+1 (202) 555-5555', 300);
```

You may use the `appendSlowly` method to append text slowly:

```
$browser->type('tags', 'foo')
    ->appendSlowly('tags', ', bar, baz');
```

Dropdowns

To select a value in a dropdown selection box, you may use the `select` method. Like the `type` method, the `select` method does not require a full CSS selector. When passing a value to the `select` method, you should pass the underlying option value instead of the display text:

```
$browser->select('size', 'Large');
```

You may select a random option by omitting the second parameter:

```
$browser->select('size');
```

Checkboxes

To "check" a checkbox field, you may use the `check` method. Like many other input related methods, a full CSS selector is not required. If an exact selector match can't be found, Dusk will search for a checkbox with a matching `name` attribute:

```
$browser->check('terms');
$browser->unchecked('terms');
```


Radio Buttons

To "select" a radio button option, you may use the `radio` method. Like many other input related methods, a full CSS selector is not required. If an exact selector match can't be found, Dusk will search for a radio with matching `name` and `value` attributes:

```
$browser->radio('version', 'php7');
```

Attaching Files

The `attach` method may be used to attach a file to a `file` input element. Like many other input related methods, a full CSS selector is not required. If an exact selector match can't be found, Dusk will search for a file input with matching `name` attribute:

```
$browser->attach('photo', __DIR__.'/photos/me.png');
```

NOTE The `attach` function requires the `zip` PHP extension to be installed and enabled on your server.

Using The Keyboard

The `keys` method allows you to provide more complex input sequences to a given element than normally allowed by the `type` method. For example, you may hold modifier keys entering values. In this example, the `shift` key will be held while `taylor` is entered into the element matching the given selector. After `taylor` is typed, `otwell` will be typed without any modifier keys:

```
$browser->keys('selector', ['{shift}', 'taylor'], 'otwell');
```

You may even send a "hot key" to the primary CSS selector that contains your application:

```
$browser->keys('.app', [{command}, 'j']);
```

TIP All modifier keys are wrapped in `{}` characters, and match the constants defined in the `Facebook\WebDriver\WebDriverKeys` class, which can be [found on GitHub](#).

Using The Mouse

Clicking On Elements

The `click` method may be used to "click" on an element matching the given selector:

```
$browser->click('.selector');
```

The `clickAtXPath` method may be used to "click" on an element matching the given XPath expression:

```
$browser->clickAtXPath('//div[@class = "selector"]');
```

The `clickAtPoint` method may be used to "click" on the topmost element at a given pair of coordinates relative to the viewable area of the browser:

```
$browser->clickAtPoint(0, 0);
```

The `doubleClick` method may be used to simulate the double "click" of a mouse:

```
$browser->doubleClick();
```

The `rightClick` method may be used to simulate the right "click" of a mouse:

```
$browser->rightClick();
```

```
$browser->rightClick('.selector');
```

The `clickAndHold` method may be used to simulate a mouse button being clicked and held down. A subsequent call to the `releaseMouse` method will undo this behavior and release the mouse button:

```
$browser->clickAndHold()
```

```
->pause(1000)
->releaseMouse();
```

Mouseover

The `mouseover` method may be used when you need to move the mouse over an element matching the given selector:

```
$browser->mouseover('.selector');
```

Drag & Drop

The `drag` method may be used to drag an element matching the given selector to another element:

```
$browser->drag('from-selector', 'to-selector');
```

Or, you may drag an element in a single direction:

```
$browser->dragLeft('.selector', 10);
$browser->dragRight('.selector', 10);
$browser->dragUp('.selector', 10);
$browser->dragDown('.selector', 10);
```

Finally, you may drag an element by a given offset:

```
$browser->dragOffset('.selector', 10, 10);
```

JavaScript Dialogs

Dusk provides various methods to interact with JavaScript Dialogs:

```
// Wait for a dialog to appear:
$browser->waitForDialog($seconds = null);

// Assert that a dialog has been displayed and that its message matches the given value:
$browser->assertDialogOpened('value');

// Type the given value in an open JavaScript prompt dialog:
$browser->typeInDialog('Hello World');
```

To close an opened JavaScript Dialog, clicking the OK button:

```
$browser->acceptDialog();
```

To close an opened JavaScript Dialog, clicking the Cancel button (for a confirmation dialog only):

```
$browser->dismissDialog();
```

Scoping Selectors

Sometimes you may wish to perform several operations while scoping all of the operations within a given selector. For example, you may wish to assert that some text exists only within a table and then click a button within that table. You may use the `with` method to accomplish this. All operations performed within the callback given to the `with` method will be scoped to the original selector:

```
$browser->with('.table', function ($table) {
    $table->assertSee('Hello World')
    ->clickLink('Delete');
});
```

You may occasionally need to execute assertions outside of the current scope. You may use the `elsewhere` method to accomplish this:

```
$browser->with('.table', function ($table) {
    // Current scope is `body .table`...
    $browser->elsewhere('.page-title', function ($title) {
        // Current scope is `body .page-title`...
        $title->assertSee('Hello World');
    });
});
```

```
});  
});
```

Waiting For Elements

When testing applications that use JavaScript extensively, it often becomes necessary to "wait" for certain elements or data to be available before proceeding with a test. Dusk makes this a cinch. Using a variety of methods, you may wait for elements to be visible on the page or even wait until a given JavaScript expression evaluates to true.

Waiting

If you need to pause the test for a given number of milliseconds, use the `pause` method:

```
$browser->pause(1000);
```

Waiting For Selectors

The `waitFor` method may be used to pause the execution of the test until the element matching the given CSS selector is displayed on the page. By default, this will pause the test for a maximum of five seconds before throwing an exception. If necessary, you may pass a custom timeout threshold as the second argument to the method:

```
// Wait a maximum of five seconds for the selector...  
$browser->waitFor('.selector');  
  
// Wait a maximum of one second for the selector...  
$browser->waitFor('.selector', 1);
```

You may also wait until the given selector is missing from the page:

```
$browser->waitUntilMissing('.selector');  
  
$browser->waitUntilMissing('.selector', 1);
```

Scoping Selectors When Available

Occasionally, you may wish to wait for a given selector and then interact with the element matching the selector. For example, you may wish to wait until a modal window is available and then press the "OK" button within the modal. The `whenAvailable` method may be used in this case. All element operations performed within the given callback will be scoped to the original selector:

```
$browser->whenAvailable('.modal', function ($modal) {  
    $modal->assertSee('Hello World')  
        ->press('OK');  
});
```

Waiting For Text

The `waitForText` method may be used to wait until the given text is displayed on the page:

```
// Wait a maximum of five seconds for the text...  
$browser->waitForText('Hello World');  
  
// Wait a maximum of one second for the text...  
$browser->waitForText('Hello World', 1);
```

You may use the `waitUntilMissingText` method to wait until the displayed text has been removed from the page:

```
// Wait a maximum of five seconds for the text to be removed...  
$browser->waitUntilMissingText('Hello World');  
  
// Wait a maximum of one second for the text to be removed...  
$browser->waitUntilMissingText('Hello World', 1);
```

Waiting For Links

The `waitForLink` method may be used to wait until the given link text is displayed on the page:

```
// Wait a maximum of five seconds for the link...
$browser->waitForLink('Create');

// Wait a maximum of one second for the link...
$browser->waitForLink('Create', 1);
```

Waiting On The Page Location

When making a path assertion such as `$browser->assertPathIs('/home')`, the assertion can fail if `window.location.pathname` is being updated asynchronously. You may use the `waitForLocation` method to wait for the location to be a given value:

```
$browser->waitForLocation('/secret');
```

You may also wait for a named route's location:

```
$browser->waitForRoute($routeName, $parameters);
```

Waiting for Page Reloads

If you need to make assertions after a page has been reloaded, use the `waitForReload` method:

```
$browser->click('.some-action')
    ->waitForReload()
    ->assertSee('something');
```

Waiting On JavaScript Expressions

Sometimes you may wish to pause the execution of a test until a given JavaScript expression evaluates to `true`. You may easily accomplish this using the `waitUntil` method. When passing an expression to this method, you do not need to include the `return` keyword or an ending semi-colon:

```
// Wait a maximum of five seconds for the expression to be true...
$browser->waitUntil('App.dataLoaded');

$browser->waitUntil('App.data.servers.length > 0');

// Wait a maximum of one second for the expression to be true...
$browser->waitUntil('App.data.servers.length > 0', 1);
```

Waiting On Vue Expressions

The following methods may be used to wait until a given Vue component attribute has a given value:

```
// Wait until the component attribute contains the given value...
$browser->waitUntilVue('user.name', 'Taylor', '@user');

// Wait until the component attribute doesn't contain the given value...
$browser->waitUntilVueIsNot('user.name', null, '@user');
```

Waiting With A Callback

Many of the "wait" methods in Dusk rely on the underlying `waitUsing` method. You may use this method directly to wait for a given callback to return `true`. The `waitUsing` method accepts the maximum number of seconds to wait, the interval at which the Closure should be evaluated, the Closure, and an optional failure message:

```
$browser->waitUsing(10, 1, function () use ($something) {
    return $something->isReady();
}, "Something wasn't ready in time.");
```

Scrolling An Element Into View

Sometimes you may not be able to click on an element because it is outside of the viewable area of the browser.

The `scrollIntoView` method will scroll the browser window until the element at the given selector is within the view:

```
$browser->scrollIntoView('selector')
    ->click('selector');
```

Making Vue Assertions

Dusk even allows you to make assertions on the state of [Vue](#) component data. For example, imagine your application contains the following Vue component:

```
// HTML...

<profile dusk="profile-component"></profile>

// Component Definition...

Vue.component('profile', {
    template: '<div>{{ user.name }}</div>',

    data: function () {
        return {
            user: {
                name: 'Taylor'
            }
        };
    }
});
```

You may assert on the state of the Vue component like so:

```
/**
 * A basic Vue test example.
 *
 * @return void
 */
public function testVue()
{
    $this->browse(function (Browser $browser) {
        $browser->visit('/')
            ->assertVue('user.name', 'Taylor', '@profile-component');
    });
}
```

Available Assertions

Dusk provides a variety of assertions that you may make against your application. All of the available assertions are documented in the list below:

[assertTitle](#) [assertTitleContains](#)
[assertUrlIs](#) [assertSchemeIs](#)
[assertSchemeIsNot](#) [assertHostIs](#)
[assertHostIsNot](#) [assertPortIs](#)
[assertPortIsNot](#)
[assertPathBeginsWith](#)
[assertPathIs](#) [assertPathIsNot](#)
[assertRouteIs](#)
[assertQueryStringHas](#)
[assertQueryStringMissing](#)
[assertFragmentIs](#)
[assertFragmentBeginsWith](#)
[assertFragmentIsNot](#)
[assertHasCookie](#)
[assertHasPlainCookie](#)
[assertCookieMissing](#)

[assertPlainCookieMissing](#)
[assertCookieValue](#)
[assertPlainCookieValue](#) [assertSee](#)
[assertDontSee](#) [assertSeeIn](#)
[assertDontSeeIn](#) [assertSourceHas](#)
[assertSourceMissing](#)
[assertSeeLink](#) [assertDontSeeLink](#)
[assertInputValue](#)
[assertInputValueIsNot](#)
[assertChecked](#) [assertNotChecked](#)
[assertRadioSelected](#)
[assertRadioNotSelected](#)
[assertSelected](#) [assertNotSelected](#)
[assertSelectHasOptions](#)
[assertSelectMissingOption](#)
[assertSelectMissingOptions](#)

[assertSelectHasOption](#)
[assertValue](#) [assertAttribute](#)
[assertAriaAttribute](#)
[assertDataAttribute](#) [assertVisible](#)
[assertPresent](#) [assertMissing](#)
[assertDialogOpened](#)
[assertEnabled](#) [assertDisabled](#)
[assertButtonEnabled](#)
[assertButtonDisabled](#)
[assertFocused](#) [assertNotFocused](#)
[assertAuthenticated](#) [assertGuest](#)
[assertAuthenticatedAs](#) [assertVue](#)
[assertVueIsNot](#)
[assertVueContains](#)
[assertVueDoesNotContain](#)

assertTitle

Assert that the page title matches the given text:

```
$browser->assertTitle($title);
```

assertTitleContains

Assert that the page title contains the given text:

```
$browser->assertTitleContains($title);
```

assertUrlIs

Assert that the current URL (without the query string) matches the given string:

```
$browser->assertUrlIs($url);
```

assertSchemeIs

Assert that the current URL scheme matches the given scheme:

```
$browser->assertSchemeIs($scheme);
```

assertSchemeIsNot

Assert that the current URL scheme does not match the given scheme:

```
$browser->assertSchemeIsNot($scheme);
```

assertHostIs

Assert that the current URL host matches the given host:

```
$browser->assertHostIs($host);
```

assertHostIsNot

Assert that the current URL host does not match the given host:

```
$browser->assertHostIsNot($host);
```

assertPortIs

Assert that the current URL port matches the given port:

```
$browser->assertPortIs($port);
```

assertPortIsNot

Assert that the current URL port does not match the given port:

```
$browser->assertPortIsNot($port);
```

assertPathBeginsWith

Assert that the current URL path begins with the given path:

```
$browser->assertPathBeginsWith($path);
```

assertPathIs

Assert that the current path matches the given path:

```
$browser->assertPathIs('/home');
```

assertPathIsNot

Assert that the current path does not match the given path:

```
$browser->assertPathIsNot('/home');
```

assertRouteIs

Assert that the current URL matches the given named route's URL:

```
$browser->assertRouteIs($name, $parameters);
```

assertQueryStringHas

Assert that the given query string parameter is present:

```
$browser->assertQueryStringHas($name);
```

Assert that the given query string parameter is present and has a given value:

```
$browser->assertQueryStringHas($name, $value);
```

assertQueryStringMissing

Assert that the given query string parameter is missing:

```
$browser->assertQueryStringMissing($name);
```

assertFragmentIs

Assert that the current fragment matches the given fragment:

```
$browser->assertFragmentIs('anchor');
```

assertFragmentBeginsWith

Assert that the current fragment begins with the given fragment:

```
$browser->assertFragmentBeginsWith('anchor');
```

assertFragmentIsNot

Assert that the current fragment does not match the given fragment:

```
$browser->assertFragmentIsNot('anchor');
```

assertHasCookie

Assert that the given encrypted cookie is present:

```
$browser->assertHasCookie($name);
```

assertHasPlainCookie

Assert that the given unencrypted cookie is present:

```
$browser->assertHasPlainCookie($name);
```

assertCookieMissing

Assert that the given encrypted cookie is not present:

```
$browser->assertCookieMissing($name);
```

assertPlainCookieMissing

Assert that the given unencrypted cookie is not present:

```
$browser->assertPlainCookieMissing($name);
```

assertCookieValue

Assert that an encrypted cookie has a given value:

```
$browser->assertCookieValue($name, $value);
```

assertPlainCookieValue

Assert that an unencrypted cookie has a given value:

```
$browser->assertPlainCookieValue($name, $value);
```

assertSee

Assert that the given text is present on the page:

```
$browser->assertSee($text);
```

assertDontSee

Assert that the given text is not present on the page:

```
$browser->assertDontSee($text);
```

assertSeeIn

Assert that the given text is present within the selector:

```
$browser->assertSeeIn($selector, $text);
```

assertDontSeeIn

Assert that the given text is not present within the selector:

```
$browser->assertDontSeeIn($selector, $text);
```

assertSourceHas

Assert that the given source code is present on the page:

```
$browser->assertSourceHas($code);
```

assertSourceMissing

Assert that the given source code is not present on the page:

```
$browser->assertSourceMissing($code);
```

assertSeeLink

Assert that the given link is present on the page:


```
$browser->assertSeeLink($linkText);
```

assertDontSeeLink

Assert that the given link is not present on the page:

```
$browser->assertDontSeeLink($linkText);
```

assertInputValue

Assert that the given input field has the given value:

```
$browser->assertInputValue($field, $value);
```

assertInputValueIsNot

Assert that the given input field does not have the given value:

```
$browser->assertInputValueIsNot($field, $value);
```

assertChecked

Assert that the given checkbox is checked:

```
$browser->assertChecked($field);
```

assertNotChecked

Assert that the given checkbox is not checked:

```
$browser->assertNotChecked($field);
```

assertRadioSelected

Assert that the given radio field is selected:

```
$browser->assertRadioSelected($field, $value);
```

assertRadioNotSelected

Assert that the given radio field is not selected:

```
$browser->assertRadioNotSelected($field, $value);
```

assertSelected

Assert that the given dropdown has the given value selected:

```
$browser->assertSelected($field, $value);
```

assertNotSelected

Assert that the given dropdown does not have the given value selected:

```
$browser->assertNotSelected($field, $value);
```

assertSelectHasOptions

Assert that the given array of values are available to be selected:

```
$browser->assertSelectHasOptions($field, $values);
```

assertSelectMissingOption

Assert that the given value is not available to be selected:

```
$browser->assertSelectMissingOption($field, $value);
```

assertSelectMissingOptions

Assert that the given array of values are not available to be selected:

```
$browser->assertSelectMissingOptions($field, $values);
```

assertSelectHasOption

Assert that the given value is available to be selected on the given field:

```
$browser->assertSelectHasOption($field, $value);
```

assertValue

Assert that the element matching the given selector has the given value:

```
$browser->assertValue($selector, $value);
```

assertAttribute

Assert that the element matching the given selector has the given value in the provided attribute:

```
$browser->assertAttribute($selector, $attribute, $value);
```

assertAriaAttribute

Assert that the element matching the given selector has the given value in the provided aria attribute:

```
$browser->assertAriaAttribute($selector, $attribute, $value);
```

For example, given the markup `<button aria-label="Add"></button>`, you may assert against the `aria-label` attribute like so:

```
$browser->assertAriaAttribute('button', 'label', 'Add')
```

assertDataAttribute

Assert that the element matching the given selector has the given value in the provided data attribute:

```
$browser->assertDataAttribute($selector, $attribute, $value);
```

For example, given the markup `<tr id="row-1" data-content="attendees"></tr>`, you may assert against the `data-label` attribute like so:

```
$browser->assertDataAttribute('#row-1', 'content', 'attendees')
```

assertVisible

Assert that the element matching the given selector is visible:

```
$browser->assertVisible($selector);
```

assertPresent

Assert that the element matching the given selector is present:

```
$browser->assertPresent($selector);
```

assertMissing

Assert that the element matching the given selector is not visible:

```
$browser->assertMissing($selector);
```

assertDialogOpened

Assert that a JavaScript dialog with the given message has been opened:

```
$browser->assertDialogOpened($message);
```

assertEnabled

Assert that the given field is enabled:

```
$browser->assertEnabled($field);
```

assertDisabled

Assert that the given field is disabled:

```
$browser->assertDisabled($field);
```

assertButtonEnabled

Assert that the given button is enabled:

```
$browser->assertButtonEnabled($button);
```

assertButtonDisabled

Assert that the given button is disabled:

```
$browser->assertButtonDisabled($button);
```

assertFocused

Assert that the given field is focused:

```
$browser->assertFocused($field);
```

assertNotFocused

Assert that the given field is not focused:

```
$browser->assertNotFocused($field);
```

assertAuthenticated

Assert that the user is authenticated:

```
$browser->assertAuthenticated();
```

assertGuest

Assert that the user is not authenticated:

```
$browser->assertGuest();
```

assertAuthenticatedAs

Assert that the user is authenticated as the given user:

```
$browser->assertAuthenticatedAs($user);
```

assertVue

Assert that a given Vue component data property matches the given value:

```
$browser->assertVue($property, $value, $componentSelector = null);
```

assertVueIsNot

Assert that a given Vue component data property does not match the given value:

```
$browser->assertVueIsNot($property, $value, $componentSelector = null);
```

assertVueContains

Assert that a given Vue component data property is an array and contains the given value:

```
$browser->assertVueContains($property, $value, $componentSelector = null);
```

assertVueDoesNotContain

Assert that a given Vue component data property is an array and does not contain the given value:

```
$browser->assertVueDoesNotContain($property, $value, $componentSelector = null);
```

Pages

Sometimes, tests require several complicated actions to be performed in sequence. This can make your tests harder to read and understand. Pages allow you to define expressive actions that may then be performed on a given page using a single method. Pages also allow you to define short-cuts to common selectors for your application or a single page.

Generating Pages

To generate a page object, use the `dusk:page` Artisan command. All page objects will be placed in the `tests/Browser/Pages` directory:

```
php artisan dusk:page Login
```

Configuring Pages

By default, pages have three methods: `url`, `assert`, and `elements`. We will discuss the `url` and `assert` methods now. The `elements` method will be [discussed in more detail below](#).

The url Method

The `url` method should return the path of the URL that represents the page. Dusk will use this URL when navigating to the page in the browser:

```
/**
 * Get the URL for the page.
 *
 * @return string
 */
public function url()
{
    return '/login';
}
```

The assert Method

The `assert` method may make any assertions necessary to verify that the browser is actually on the given page. Completing this method is not necessary; however, you are free to make these assertions if you wish. These assertions will be run automatically when navigating to the page:

```
/**
 * Assert that the browser is on the page.
 *
 * @return void
 */
public function assert(Browser $browser)
{
    $browser->assertPathIs($this->url());
}
```

Navigating To Pages

Once a page has been configured, you may navigate to it using the `visit` method:

```
use Tests\Browser\Pages\Login;

$browser->visit(new Login);
```

You may use the `visitRoute` method to navigate to a named route:

```
$browser->visitRoute('login');
```

You may navigate "back" and "forward" using the `back` and `forward` methods:

```
$browser->back();

$browser->forward();
```

You may use the `refresh` method to refresh the page:

```
$browser->refresh();
```

Sometimes you may already be on a given page and need to "load" the page's selectors and methods into the current test context. This is common when pressing a button and being redirected to a given page without explicitly navigating to it. In this situation, you may use the `on` method to load the page:

```
use Tests\Browser\Pages\CreatePlaylist;

$browser->visit('/dashboard')
    ->clickLink('Create Playlist')
    ->on(new CreatePlaylist)
    ->assertSee('@create');
```

Shorthand Selectors

The `elements` method of pages allows you to define quick, easy-to-remember shortcuts for any CSS selector on your page. For example, let's define a shortcut for the "email" input field of the application's login page:

```
/**
 * Get the element shortcuts for the page.
 *
 * @return array
 */
public function elements()
{
    return [
        '@email' => 'input[name=email]',
    ];
}
```

Now, you may use this shorthand selector anywhere you would use a full CSS selector:

```
$browser->type('@email', 'taylor@laravel.com');
```

Global Shorthand Selectors

After installing Dusk, a base `Page` class will be placed in your `tests/Browser/Pages` directory. This class contains a `siteElements` method which may be used to define global shorthand selectors that should be available on every page throughout your application:

```
/**
 * Get the global element shortcuts for the site.
 *
 * @return array
 */
public static function siteElements()
{
    return [
        '@element' => '#selector',
    ];
}
```

Page Methods

In addition to the default methods defined on pages, you may define additional methods which may be used throughout your tests. For example, let's imagine we are building a music management application. A common action for one page of the application might be to create a playlist. Instead of re-writing the logic to create a playlist in each test, you may define a `createPlaylist` method on a page class:

```
<?php

namespace Tests\Browser\Pages;

use Laravel\Dusk\Browser;

class Dashboard extends Page
{
    // Other page methods...

    /**
     * Create a new playlist.
     *
     * @param \Laravel\Dusk\Browser $browser
     * @param string $name
     * @return void
     */
    public function createPlaylist(Browser $browser, $name)
    {
        $browser->type('name', $name)
            ->check('share')
            ->press('Create Playlist');
    }
}
```

Once the method has been defined, you may use it within any test that utilizes the page. The browser instance will automatically be passed to the page method:

```
use Tests\Browser\Pages\Dashboard;

$browser->visit(new Dashboard)
    ->createPlaylist('My Playlist')
    ->assertSee('My Playlist');
```

Components

Components are similar to Dusk's "page objects", but are intended for pieces of UI and functionality that are re-used throughout your application, such as a navigation bar or notification window. As such, components are not bound to specific URLs.

Generating Components

To generate a component, use the `dusk:component` Artisan command. New components are placed in the `tests/Browser/Components` directory:

```
php artisan dusk:component DatePicker
```

As shown above, a "date picker" is an example of a component that might exist throughout your application on a variety of pages. It can become cumbersome to manually write the browser automation logic to select a date in dozens of tests throughout your test suite. Instead, we can define a Dusk component to represent the date picker, allowing us to encapsulate that logic within the component:

```
<?php

namespace Tests\Browser\Components;

use Laravel\Dusk\Browser;
use Laravel\Dusk\Component as BaseComponent;

class DatePicker extends BaseComponent
{
    /**
     * Get the root selector for the component.
     *
     * @return string
     */
    public function selector()
    {
        return '.date-picker';
    }

    /**
     * Assert that the browser page contains the component.
     *
     * @param Browser $browser
     * @return void
     */
    public function assert(Browser $browser)
    {
        $browser->assertVisible($this->selector());
    }

    /**
     * Get the element shortcuts for the component.
     *
     * @return array
     */
    public function elements()
    {
        return [
            '@date-field' => 'input.datepicker-input',
            '@year-list' => 'div > div.datepicker-years',
            '@month-list' => 'div > div.datepicker-months',
            '@day-list' => 'div > div.datepicker-days',
        ];
    }

    /**
     * Select the given date.
     *
     * @param \Laravel\Dusk\Browser $browser
     * @param int $year
     * @param int $month
     * @param int $day
     * @return void
     */
    public function selectDate($browser, $year, $month, $day)
    {
        $browser->click('@date-field')
            ->within('@year-list', function ($browser) use ($year) {
                $browser->click($year);
            })
            ->within('@month-list', function ($browser) use ($month) {
                $browser->click($month);
            })
            ->within('@day-list', function ($browser) use ($day) {
                $browser->click($day);
            });
    }
}
```

Using Components

Once the component has been defined, we can easily select a date within the date picker from any test. And, if the logic necessary to select a date changes, we only need to update the component:

```
<?php

namespace Tests\Browser;

use Illuminate\Foundation\Testing\DatabaseMigrations;
use Laravel\Dusk\Browser;
use Tests\Browser\Components\DatePicker;
use Tests\DuskTestCase;

class ExampleTest extends DuskTestCase
{
    /**
     * A basic component test example.
     *
     * @return void
     */
    public function testBasicExample()
    {
        $this->browse(function (Browser $browser) {
            $browser->visit('/')
                ->within(new DatePicker, function ($browser) {
                    $browser->selectDate(2019, 1, 30);
                })
                ->assertSee('January');
        });
    }
}
```

Continuous Integration

NOTE Before adding a continuous integration configuration file, ensure that your `.env.testing` file contains an `APP_URL` entry with a value of `http://127.0.0.1:8000`.

CircleCI

If you are using CircleCI to run your Dusk tests, you may use this configuration file as a starting point. Like TravisCI, we will use the `php artisan serve` command to launch PHP's built-in web server:

```
version: 2
jobs:
  build:
    steps:
      - run: sudo apt-get install -y libsqlite3-dev
      - run: cp .env.testing .env
      - run: composer install -n --ignore-platform-reqs
      - run: php artisan key:generate
      - run: php artisan dusk:chrome-driver
      - run: npm install
      - run: npm run production
      - run: vendor/bin/phpunit

      - run:
          name: Start Chrome Driver
          command: ./vendor/laravel/dusk/bin/chromedriver-linux
          background: true

      - run:
          name: Run Laravel Server
          command: php artisan serve
          background: true

      - run:
          name: Run Laravel Dusk Tests
          command: php artisan dusk

      - store_artifacts:
          path: tests/Browser/screenshots

      - store_artifacts:
          path: tests/Browser/console

      - store_artifacts:
```



```
path: storage/logs
```

Codeship

To run Dusk tests on [Codeship](#), add the following commands to your Codeship project. These commands are just a starting point and you are free to add additional commands as needed:

```
phpenv local 7.2
cp .env.testing .env
mkdir -p ./bootstrap/cache
composer install --no-interaction --prefer-dist
php artisan key:generate
php artisan dusk:chrome-driver
nohup bash -c "php artisan serve 2>&1 &" && sleep 5
php artisan dusk
```

Heroku CI

To run Dusk tests on [Heroku CI](#), add the following Google Chrome buildpack and scripts to your Heroku app.json file:

```
{
  "environments": {
    "test": {
      "buildpacks": [
        { "url": "heroku/php" },
        { "url": "https://github.com/heroku/heroku-buildpack-google-chrome" }
      ],
      "scripts": {
        "test-setup": "cp .env.testing .env",
        "test": "nohup bash -c './vendor/laravel/dusk/bin/chromedriver-linux > /dev/null 2>&1 &' && nohup bash -c 'php artisan serve > /dev/null 2>&1 &' && php artisan dusk"
      }
    }
  }
}
```

Travis CI

To run your Dusk tests on [Travis CI](#), use the following .travis.yml configuration. Since Travis CI is not a graphical environment, we will need to take some extra steps in order to launch a Chrome browser. In addition, we will use php artisan serve to launch PHP's built-in web server:

```
language: php

php:
  - 7.3

addons:
  chrome: stable

install:
  - cp .env.testing .env
  - travis_retry composer install --no-interaction --prefer-dist --no-suggest
  - php artisan key:generate
  - php artisan dusk:chrome-driver

before_script:
  - google-chrome-stable --headless --disable-gpu --remote-debugging-port=9222 http://localhost &
  - php artisan serve &

script:
  - php artisan dusk
```

GitHub Actions

If you are using [Github Actions](#) to run your Dusk tests, you may use this configuration file as a starting point. Like TravisCI, we will use the php artisan serve command to launch PHP's built-in web server:

```
name: CI
on: [push]
```

jobs:

```
dusk-php:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v2
    - name: Prepare The Environment
      run: cp .env.example .env
    - name: Create Database
      run: |
        sudo systemctl start mysql
        mysql --user="root" --password="root" -e "CREATE DATABASE 'my-database' character set UTF8mb4
collate utf8mb4_bin;"
    - name: Install Composer Dependencies
      run: composer install --no-progress --no-suggest --prefer-dist --optimize-autoloader
    - name: Generate Application Key
      run: php artisan key:generate
    - name: Upgrade Chrome Driver
      run: php artisan dusk:chrome-driver ` /opt/google/chrome/chrome --version | cut -d " " -f3 | cut
-d "." -f1`
    - name: Start Chrome Driver
      run: ./vendor/laravel/dusk/bin/chromedriver-linux &
    - name: Run Laravel Server
      run: php artisan serve &
    - name: Run Dusk Tests
      env:
        APP_URL: "http://127.0.0.1:8000"
      run: php artisan dusk
```

Testing

Database

Database Testing

- [Introduction](#)
- [Generating Factories](#)
- [Resetting The Database After Each Test](#)
- [Writing Factories](#)
 - [Extending Factories](#)
 - [Factory States](#)
 - [Factory Callbacks](#)
- [Using Factories](#)
 - [Creating Models](#)
 - [Persisting Models](#)
 - [Relationships](#)
- [Using Seeds](#)
- [Available Assertions](#)

Introduction

Laravel provides a variety of helpful tools to make it easier to test your database driven applications. First, you may use the `assertDatabaseHas` helper to assert that data exists in the database matching a given set of criteria. For example, if you would like to verify that there is a record in the `users` table with the `email` value of `sally@example.com`, you can do the following:

```
public function testDatabase()
{
    // Make call to application...

    $this->assertDatabaseHas('users', [
        'email' => 'sally@example.com',
    ]);
}
```

You can also use the `assertDatabaseMissing` helper to assert that data does not exist in the database.

The `assertDatabaseHas` method and other helpers like it are for convenience. You are free to use any of PHPUnit's built-in assertion methods to supplement your feature tests.

Generating Factories

To create a factory, use the `make:factory` [Artisan command](#):

```
php artisan make:factory PostFactory
```

The new factory will be placed in your `database/factories` directory.

The `--model` option may be used to indicate the name of the model created by the factory. This option will pre-fill the generated factory file with the given model:

```
php artisan make:factory PostFactory --model=Post
```

Resetting The Database After Each Test

It is often useful to reset your database after each test so that data from a previous test does not interfere with subsequent tests. The `RefreshDatabase` trait takes the most optimal approach to migrating your test database depending on if you are using an in-memory database or a traditional database. Use the trait on your test class and everything will be handled for you:

```
<?php
```

```

namespace Tests\Feature;

use Illuminate\Foundation\Testing\RefreshDatabase;
use Illuminate\Foundation\Testing\WithoutMiddleware;
use Tests\TestCase;

class ExampleTest extends TestCase
{
    use RefreshDatabase;

    /**
     * A basic functional test example.
     *
     * @return void
     */
    public function testBasicExample()
    {
        $response = $this->get('/');

        // ...
    }
}

```

Writing Factories

When testing, you may need to insert a few records into your database before executing your test. Instead of manually specifying the value of each column when you create this test data, Laravel allows you to define a default set of attributes for each of your [Eloquent models](#) using model factories. To get started, take a look at the database/factories/UserFactory.php file in your application. Out of the box, this file contains one factory definition:

```

use Faker\Generator as Faker;
use Illuminate\Support\Str;

$factory->define(App\User::class, function (Faker $faker) {
    return [
        'name' => $faker->name,
        'email' => $faker->unique()->safeEmail,
        'email_verified_at' => now(),
        'password' => '$2y$10$92IXUNpkj00r0Q5byMi.Ye4oKoEa3Ro9llC/.og/at2.uhewG/igi', // password
        'remember_token' => Str::random(10),
    ];
});

```

Within the Closure, which serves as the factory definition, you may return the default test values of all attributes on the model. The Closure will receive an instance of the [Faker](#) PHP library, which allows you to conveniently generate various kinds of random data for testing.

You may also create additional factory files for each model for better organization. For example, you could create UserFactory.php and CommentFactory.php files within your database/factories directory. All of the files within the factories directory will automatically be loaded by Laravel.

TIP You can set the Faker locale by adding a `faker_locale` option to your `config/app.php` configuration file.

Extending Factories

If you have extended a model, you may wish to extend its factory as well in order to utilize the child model's factory attributes during testing and seeding. To accomplish this, you may call the factory builder's `raw` method to obtain the raw array of attributes from any given factory:

```

$factory->define(App\Admin::class, function (Faker\Generator $faker) {
    return factory(App\User::class)->raw([
        // ...
    ]);
});

```

Factory States

States allow you to define discrete modifications that can be applied to your model factories in any combination. For example, your `User` model might have a `delinquent` state that modifies one of its default attribute values. You may define your state transformations using the `state` method. For simple states, you may pass an array of attribute modifications:

```
$factory->state(App\User::class, 'delinquent', [
    'account_status' => 'delinquent',
]);
```

If your state requires calculation or a `$faker` instance, you may use a Closure to calculate the state's attribute modifications:

```
$factory->state(App\User::class, 'address', function ($faker) {
    return [
        'address' => $faker->address,
    ];
});
```

Factory Callbacks

Factory callbacks are registered using the `afterMaking` and `afterCreating` methods, and allow you to perform additional tasks after making or creating a model. For example, you may use callbacks to relate additional models to the created model:

```
$factory->afterMaking(App\User::class, function ($user, $faker) {
    // ...
});

$factory->afterCreating(App\User::class, function ($user, $faker) {
    $user->accounts()->save(factory(App\Account::class)->make());
});
```

You may also define callbacks for [factory states](#):

```
$factory->afterMakingState(App\User::class, 'delinquent', function ($user, $faker) {
    // ...
});

$factory->afterCreatingState(App\User::class, 'delinquent', function ($user, $faker) {
    // ...
});
```

Using Factories

Creating Models

Once you have defined your factories, you may use the global `factory` function in your feature tests or seed files to generate model instances. So, let's take a look at a few examples of creating models. First, we'll use the `make` method to create models but not save them to the database:

```
public function testDatabase()
{
    $user = factory(App\User::class)->make();

    // Use model in tests...
}
```

You may also create a Collection of many models or create models of a given type:

```
// Create three App\User instances...
$users = factory(App\User::class, 3)->make();
```

Applying States

You may also apply any of your [states](#) to the models. If you would like to apply multiple state transformations to the models, you should specify the name of each state you would like to apply:

```
$users = factory(App\User::class, 5)->states('delinquent')->make();
```

```
$users = factory(App\User::class, 5)->states('premium', 'delinquent')->make();
```

Overriding Attributes

If you would like to override some of the default values of your models, you may pass an array of values to the `make` method. Only the specified values will be replaced while the rest of the values remain set to their default values as specified by the factory:

```
$user = factory(App\User::class)->make([
    'name' => 'Abigail',
]);
```

TIP [Mass assignment protection](#) is automatically disabled when creating models using factories.

Persisting Models

The `create` method not only creates the model instances but also saves them to the database using Eloquent's `save` method:

```
public function testDatabase()
{
    // Create a single App\User instance...
    $user = factory(App\User::class)->create();

    // Create three App\User instances...
    $users = factory(App\User::class, 3)->create();

    // Use model in tests...
}
```

You may override attributes on the model by passing an array to the `create` method:

```
$user = factory(App\User::class)->create([
    'name' => 'Abigail',
]);
```

Relationships

In this example, we'll attach a relation to some created models. When using the `create` method to create multiple models, an Eloquent [collection instance](#) is returned, allowing you to use any of the convenient functions provided by the collection, such as `each`:

```
$users = factory(App\User::class, 3)
    ->create()
    ->each(function ($user) {
        $user->posts()->save(factory(App\Post::class)->make());
    });
```

You may use the `createMany` method to create multiple related models:

```
$user->posts()->createMany(
    factory(App\Post::class, 3)->make()->toArray()
);
```

Relations & Attribute Closures

You may also attach relationships to models in your factory definitions. For example, if you would like to create a new user instance when creating a `Post`, you may do the following:

```
$factory->define(App\Post::class, function ($faker) {
    return [
        'title' => $faker->title,
        'content' => $faker->paragraph,
        'user_id' => factory(App\User::class),
    ];
});
```

If the relationship depends on the factory that defines it you may provide a callback which accepts the

evaluated attribute array:

```
$factory->define(App\Post::class, function ($faker) {
    return [
        'title' => $faker->title,
        'content' => $faker->paragraph,
        'user_id' => factory(App\User::class),
        'user_type' => function (array $post) {
            return App\User::find($post['user_id']->type);
        },
    ];
});
```

Using Seeds

If you would like to use [database seeders](#) to populate your database during a feature test, you may use the `seed` method. By default, the `seed` method will return the `DatabaseSeeder`, which should execute all of your other seeders. Alternatively, you pass a specific seeder class name to the `seed` method:

```
<?php

namespace Tests\Feature;

use Illuminate\Foundation\Testing\RefreshDatabase;
use Illuminate\Foundation\Testing\WithoutMiddleware;
use OrderStatusSeeder;
use Tests\TestCase;

class ExampleTest extends TestCase
{
    use RefreshDatabase;

    /**
     * Test creating a new order.
     *
     * @return void
     */
    public function testCreatingANewOrder()
    {
        // Run the DatabaseSeeder...
        $this->seed();

        // Run a single seeder...
        $this->seed(OrderStatusSeeder::class);

        // ...
    }
}
```

Available Assertions

Laravel provides several database assertions for your [PHPUnit](#) feature tests:

Method	Description
<code>\$this->assertDatabaseCount(\$table, int \$count);</code>	Assert that a table in the database contains the given amount of entries.
<code>\$this->assertDatabaseHas(\$table, array \$data);</code>	Assert that a table in the database contains the given data.
<code>\$this->assertDatabaseMissing(\$table, array \$data);</code>	Assert that a table in the database does not contain the given data.
<code>\$this->assertDeleted(\$table, array \$data);</code>	Assert that the given record has been deleted.
<code>\$this->assertSoftDeleted(\$table, array \$data);</code>	Assert that the given record has been soft deleted.

For convenience, you may pass a model to the `assertDeleted` and `assertSoftDeleted` helpers to assert the record was deleted or soft deleted, respectively, from the database based on the model's primary key.

For example, if you are using a model factory in your test, you may pass this model to one of these helpers to test your application properly deleted the record from the database:

```
public function testDatabase()
{
    $user = factory(App\User::class)->create();
```

```
        // Make call to application...
        $this->assertDeleted($user);
    }
```


Testing

Mocking

Mocking

- [Introduction](#)
- [Mocking Objects](#)
- [Bus Fake](#)
- [Event Fake](#)
 - [Scoped Event Fakes](#)
- [HTTP Fake](#)
- [Mail Fake](#)
- [Notification Fake](#)
- [Queue Fake](#)
- [Storage Fake](#)
- [Facades](#)

Introduction

When testing Laravel applications, you may wish to "mock" certain aspects of your application so they are not actually executed during a given test. For example, when testing a controller that dispatches an event, you may wish to mock the event listeners so they are not actually executed during the test. This allows you to only test the controller's HTTP response without worrying about the execution of the event listeners, since the event listeners can be tested in their own test case.

Laravel provides helpers for mocking events, jobs, and facades out of the box. These helpers primarily provide a convenience layer over Mockery so you do not have to manually make complicated Mockery method calls. You can also use [Mockery](#) or PHPUnit to create your own mocks or spies.

Mocking Objects

When mocking an object that is going to be injected into your application via Laravel's service container, you will need to bind your mocked instance into the container as an `instance` binding. This will instruct the container to use your mocked instance of the object instead of constructing the object itself:

```
use App\Service;
use Mockery;

$this->instance(Service::class, Mockery::mock(Service::class, function ($mock) {
    $mock->shouldReceive('process')->once();
}));
```

In order to make this more convenient, you may use the `mock` method, which is provided by Laravel's base test case class:

```
use App\Service;

$this->mock(Service::class, function ($mock) {
    $mock->shouldReceive('process')->once();
});
```

You may use the `partialMock` method when you only need to mock a few methods of an object. The methods that are not mocked will be executed normally when called:

```
use App\Service;

$this->partialMock(Service::class, function ($mock) {
    $mock->shouldReceive('process')->once();
});
```

Similarly, if you want to spy on an object, Laravel's base test case class offers a `spy` method as a convenient wrapper around the `Mockery::spy` method:

```
use App\Service;

$this->spy(Service::class, function ($mock) {
    $mock->shouldReceive('process');
});
```

Bus Fake

As an alternative to mocking, you may use the `Bus` facade's `fake` method to prevent jobs from being dispatched. When using fakes, assertions are made after the code under test is executed:

```
<?php

namespace Tests\Feature;

use App\Jobs\ShipOrder;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Illuminate\Foundation\Testing\WithoutMiddleware;
use Illuminate\Support\Facades\Bus;
use Tests\TestCase;

class ExampleTest extends TestCase
{
    public function testOrderShipping()
    {
        Bus::fake();

        // Perform order shipping...

        // Assert a specific type of job was dispatched meeting the given truth test...
        Bus::assertDispatched(function (ShipOrder $job) use ($order) {
            return $job->order->id === $order->id;
        });

        // Assert a job was not dispatched...
        Bus::assertNotDispatched(AnotherJob::class);
    }
}
```

Event Fake

As an alternative to mocking, you may use the `Event` facade's `fake` method to prevent all event listeners from executing. You may then assert that events were dispatched and even inspect the data they received. When using fakes, assertions are made after the code under test is executed:

```
<?php

namespace Tests\Feature;

use App\Events\OrderFailedToShip;
use App\Events\OrderShipped;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Illuminate\Foundation\Testing\WithoutMiddleware;
use Illuminate\Support\Facades\Event;
use Tests\TestCase;

class ExampleTest extends TestCase
{
    /**
     * Test order shipping.
     */
    public function testOrderShipping()
    {
        Event::fake();

        // Perform order shipping...

        // Assert a specific type of event was dispatched meeting the given truth test...
        Event::assertDispatched(function (OrderShipped $event) use ($order) {
            return $event->order->id === $order->id;
        });

        // Assert an event was dispatched twice...
        Event::assertDispatched(OrderShipped::class, 2);
    }
}
```

```
        // Assert an event was not dispatched...
        Event::assertNotDispatched(OrderFailedToShip::class);
    }
}
```

NOTE After calling `Event::fake()`, no event listeners will be executed. So, if your tests use model factories that rely on events, such as creating a UUID during a model's creating event, you should call `Event::fake()` **after** using your factories.

Faking A Subset Of Events

If you only want to fake event listeners for a specific set of events, you may pass them to the `fake` or `fakeFor` method:

```
/**
 * Test order process.
 */
public function testOrderProcess()
{
    Event::fake([
        OrderCreated::class,
    ]);

    $order = factory(Order::class)->create();

    Event::assertDispatched(OrderCreated::class);

    // Other events are dispatched as normal...
    $order->update([...]);
}
```

Scoped Event Fakes

If you only want to fake event listeners for a portion of your test, you may use the `fakeFor` method:

```
<?php

namespace Tests\Feature;

use App\Events\OrderCreated;
use App\Order;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Illuminate\Support\Facades\Event;
use Illuminate\Foundation\Testing\WithoutMiddleware;
use Tests\TestCase;

class ExampleTest extends TestCase
{
    /**
     * Test order process.
     */
    public function testOrderProcess()
    {
        $order = Event::fakeFor(function () {
            $order = factory(Order::class)->create();

            Event::assertDispatched(OrderCreated::class);

            return $order;
        });

        // Events are dispatched as normal and observers will run ...
        $order->update([...]);
    }
}
```

HTTP Fake

The `Http` facade's `fake` method allows you to instruct the HTTP client to return stubbed / dummy responses when requests are made. For more information on faking outgoing HTTP requests, please consult the [HTTP Client testing documentation](#).

Mail Fake

You may use the `Mail` facade's `fake` method to prevent mail from being sent. You may then assert that [mailables](#) were sent to users and even inspect the data they received. When using fakes, assertions are made after the code under test is executed:

```
<?php

namespace Tests\Feature;

use App\Mail\OrderShipped;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Illuminate\Foundation\Testing\WithoutMiddleware;
use Illuminate\Support\Facades\Mail;
use Tests\TestCase;

class ExampleTest extends TestCase
{
    public function testOrderShipping()
    {
        Mail::fake();

        // Assert that no mailables were sent...
        Mail::assertNothingSent();

        // Perform order shipping...

        // Assert a specific type of mailable was dispatched meeting the given truth test...
        Mail::assertSent(function (OrderShipped $mail) use ($order) {
            return $mail->order->id === $order->id;
        });

        // Assert a message was sent to the given users...
        Mail::assertSent(OrderShipped::class, function ($mail) use ($user) {
            return $mail->hasTo($user->email) &&
                $mail->hasCc('...') &&
                $mail->hasBcc('...');
        });

        // Assert a mailable was sent twice...
        Mail::assertSent(OrderShipped::class, 2);

        // Assert a mailable was not sent...
        Mail::assertNotSent(AnotherMailable::class);
    }
}
```

If you are queueing mailables for delivery in the background, you should use the `assertQueued` method instead of `assertSent`:

```
Mail::assertQueued(...);
Mail::assertNotQueued(...);
```

Notification Fake

You may use the `Notification` facade's `fake` method to prevent notifications from being sent. You may then assert that [notifications](#) were sent to users and even inspect the data they received. When using fakes, assertions are made after the code under test is executed:

```
<?php

namespace Tests\Feature;

use App\Notifications\OrderShipped;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Illuminate\Foundation\Testing\WithoutMiddleware;
use Illuminate\Notifications\AnonymousNotifiable;
use Illuminate\Support\Facades\Notification;
use Tests\TestCase;

class ExampleTest extends TestCase
{
    public function testOrderShipping()
    {
```

```

Notification::fake();

// Assert that no notifications were sent...
Notification::assertNothingSent();

// Perform order shipping...

// Assert a specific type of notification was sent meeting the given truth test...
Notification::assertSentTo(
    $user,
    function (OrderShipped $notification, $channels) use ($order) {
        return $notification->order->id === $order->id;
    }
);

// Assert a notification was sent to the given users...
Notification::assertSentTo(
    [$user], OrderShipped::class
);

// Assert a notification was not sent...
Notification::assertNotSentTo(
    [$user], AnotherNotification::class
);

// Assert a notification was sent via Notification::route() method...
Notification::assertSentTo(
    new AnonymousNotifiable, OrderShipped::class
);

// Assert Notification::route() method sent notification to the correct user...
Notification::assertSentTo(
    new AnonymousNotifiable,
    OrderShipped::class,
    function ($notification, $channels, $notifiable) use ($user) {
        return $notifiable->routes['mail'] === $user->email;
    }
);
}
}
}

```

Queue Fake

As an alternative to mocking, you may use the `queue` facade's `fake` method to prevent jobs from being queued. You may then assert that jobs were pushed to the queue and even inspect the data they received. When using fakes, assertions are made after the code under test is executed:

```

<?php

namespace Tests\Feature;

use App\Jobs\AnotherJob;
use App\Jobs\FinalJob;
use App\Jobs\ShipOrder;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Illuminate\Foundation\Testing\WithoutMiddleware;
use Illuminate\Support\Facades\Queue;
use Tests\TestCase;

class ExampleTest extends TestCase
{
    public function testOrderShipping()
    {
        Queue::fake();

        // Assert that no jobs were pushed...
        Queue::assertNothingPushed();

        // Perform order shipping...

        // Assert a specific type of job was pushed meeting the given truth test...
        Queue::assertPushed(function (ShipOrder $job) use ($order) {
            return $job->order->id === $order->id;
        });

        // Assert a job was pushed to a given queue...
        Queue::assertPushedOn('queue-name', ShipOrder::class);
    }
}

```

```
// Assert a job was pushed twice...
Queue::assertPushed(ShipOrder::class, 2);

// Assert a job was not pushed...
Queue::assertNotPushed(AnotherJob::class);

// Assert a job was pushed with a given chain of jobs, matching by class...
Queue::assertPushedWithChain(ShipOrder::class, [
    AnotherJob::class,
    FinalJob::class
]);

// Assert a job was pushed with a given chain of jobs, matching by both class and properties...
Queue::assertPushedWithChain(ShipOrder::class, [
    new AnotherJob('foo'),
    new FinalJob('bar'),
]);

// Assert a job was pushed without a chain of jobs...
Queue::assertPushedWithoutChain(ShipOrder::class);
}
}
```

Storage Fake

The storage facade's `fake` method allows you to easily generate a fake disk that, combined with the file generation utilities of the `UploadedFile` class, greatly simplifies the testing of file uploads. For example:

```
<?php

namespace Tests\Feature;

use Illuminate\Foundation\Testing\RefreshDatabase;
use Illuminate\Foundation\Testing\WithoutMiddleware;
use Illuminate\Http\UploadedFile;
use Illuminate\Support\Facades\Storage;
use Tests\TestCase;

class ExampleTest extends TestCase
{
    public function testAlbumUpload()
    {
        Storage::fake('photos');

        $response = $this->json('POST', '/photos', [
            UploadedFile::fake()->image('photo1.jpg'),
            UploadedFile::fake()->image('photo2.jpg')
        ]);

        // Assert one or more files were stored...
        Storage::disk('photos')->assertExists('photo1.jpg');
        Storage::disk('photos')->assertExists(['photo1.jpg', 'photo2.jpg']);

        // Assert one or more files were not stored...
        Storage::disk('photos')->assertMissing('missing.jpg');
        Storage::disk('photos')->assertMissing(['missing.jpg', 'non-existing.jpg']);
    }
}
```

TIP By default, the `fake` method will delete all files in its temporary directory. If you would like to keep these files, you may use the `"persistentFake"` method instead.

Facades

Unlike traditional static method calls, [facades](#) may be mocked. This provides a great advantage over traditional static methods and grants you the same testability you would have if you were using dependency injection. When testing, you may often want to mock a call to a Laravel facade in one of your controllers. For example, consider the following controller action:

```
<?php

namespace App\Http\Controllers;

use Illuminate\Support\Facades\Cache;
```

```
class UserController extends Controller
{
    /**
     * Show a list of all users of the application.
     *
     * @return Response
     */
    public function index()
    {
        $value = Cache::get('key');

        //
    }
}
```

We can mock the call to the cache facade by using the `shouldReceive` method, which will return an instance of a [Mockery](#) mock. Since facades are actually resolved and managed by the Laravel [service container](#), they have much more testability than a typical static class. For example, let's mock our call to the cache facade's `get` method:

```
<?php

namespace Tests\Feature;

use Illuminate\Foundation\Testing\RefreshDatabase;
use Illuminate\Foundation\Testing\WithoutMiddleware;
use Illuminate\Support\Facades\Cache;
use Tests\TestCase;

class UserControllerTest extends TestCase
{
    public function testGetIndex()
    {
        Cache::shouldReceive('get')
            ->once()
            ->with('key')
            ->andReturn('value');

        $response = $this->get('/users');

        // ...
    }
}
```

NOTE You should not mock the `Request` facade. Instead, pass the input you desire into the HTTP helper methods such as `get` and `post` when running your test. Likewise, instead of mocking the `Config` facade, call the `Config::set` method in your tests.

Official Packages

Cashier (Stripe)

Laravel Cashier

- [Introduction](#)
- [Upgrading Cashier](#)
- [Installation](#)
- [Configuration](#)
 - [Billable Model](#)
 - [API Keys](#)
 - [Currency Configuration](#)
 - [Logging](#)
- [Customers](#)
 - [Retrieving Customers](#)
 - [Creating Customers](#)
 - [Updating Customers](#)
 - [Billing Portal](#)
- [Payment Methods](#)
 - [Storing Payment Methods](#)
 - [Retrieving Payment Methods](#)
 - [Determining If A User Has A Payment Method](#)
 - [Updating The Default Payment Method](#)
 - [Adding Payment Methods](#)
 - [Deleting Payment Methods](#)
- [Subscriptions](#)
 - [Creating Subscriptions](#)
 - [Checking Subscription Status](#)
 - [Changing Plans](#)
 - [Subscription Quantity](#)
 - [Multiplan Subscriptions](#)
 - [Subscription Taxes](#)
 - [Subscription Anchor Date](#)
 - [Cancelling Subscriptions](#)
 - [Resuming Subscriptions](#)
- [Subscription Trials](#)
 - [With Payment Method Up Front](#)
 - [Without Payment Method Up Front](#)
 - [Extending Trials](#)
- [Handling Stripe Webhooks](#)
 - [Defining Webhook Event Handlers](#)
 - [Failed Subscriptions](#)
 - [Verifying Webhook Signatures](#)
- [Single Charges](#)
 - [Simple Charge](#)
 - [Charge With Invoice](#)
 - [Refunding Charges](#)
- [Invoices](#)
 - [Retrieving Invoices](#)
 - [Generating Invoice PDFs](#)
- [Handling Failed Payments](#)
- [Strong Customer Authentication \(SCA\)](#)
 - [Payments Requiring Additional Confirmation](#)
 - [Off-session Payment Notifications](#)
- [Stripe SDK](#)
- [Testing](#)

Introduction

Laravel Cashier provides an expressive, fluent interface to [Stripe's](#) subscription billing services. It handles almost all of the boilerplate subscription billing code you are dreading writing. In addition to basic subscription management, Cashier can handle coupons, swapping subscription, subscription "quantities", cancellation grace periods, and even generate invoice PDFs.

Upgrading Cashier

When upgrading to a new version of Cashier, it's important that you carefully review [the upgrade guide](#).

NOTE To prevent breaking changes, Cashier uses a fixed Stripe API version. Cashier 12 utilizes Stripe API version 2020-03-02. The Stripe API version will be updated on minor releases in order to make use of new Stripe features and improvements.

Installation

First, require the Cashier package for Stripe with Composer:

```
composer require laravel/cashier
```

NOTE To ensure Cashier properly handles all Stripe events, remember to [set up Cashier's webhook handling](#).

Database Migrations

The Cashier service provider registers its own database migration directory, so remember to migrate your database after installing the package. The Cashier migrations will add several columns to your `users` table as well as create a new `subscriptions` table to hold all of your customer's subscriptions:

```
php artisan migrate
```

If you need to overwrite the migrations that ship with the Cashier package, you can publish them using the `vendor:publish` Artisan command:

```
php artisan vendor:publish --tag="cashier-migrations"
```

If you would like to prevent Cashier's migrations from running entirely, you may use the `ignoreMigrations` provided by Cashier. Typically, this method should be called in the `register` method of your `AppServiceProvider`:

```
use Laravel\Cashier\Cashier;

Cashier::ignoreMigrations();
```

NOTE Stripe recommends that any column used for storing Stripe identifiers should be case-sensitive. Therefore, you should ensure the column collation for the `stripe_id` column is set to, for example, `utf8_bin` in MySQL. More info can be found [in the Stripe documentation](#).

Configuration

Billable Model

Before using Cashier, add the `Billable` trait to your model definition. This trait provides various methods to allow you to perform common billing tasks, such as creating subscriptions, applying coupons, and updating payment method information:

```
use Laravel\Cashier\Billable;

class User extends Authenticatable
{
    use Billable;
}
```

Cashier assumes your Billable model will be the `App\User` class that ships with Laravel. If you wish to change

this you can specify a different model in your `.env` file:

```
CASHIER_MODEL=App\User
```

NOTE If you're using a model other than Laravel's supplied `App\User` model, you'll need to publish and alter the [migrations](#) provided to match your alternative model's table name.

API Keys

Next, you should configure your Stripe keys in your `.env` file. You can retrieve your Stripe API keys from the Stripe control panel.

```
STRIPE_KEY=your-stripe-key  
STRIPE_SECRET=your-stripe-secret
```

Currency Configuration

The default Cashier currency is United States Dollars (USD). You can change the default currency by setting the `CASHIER_CURRENCY` environment variable:

```
CASHIER_CURRENCY=eur
```

In addition to configuring Cashier's currency, you may also specify a locale to be used when formatting money values for display on invoices. Internally, Cashier utilizes [PHP's NumberFormatter class](#) to set the currency locale:

```
CASHIER_CURRENCY_LOCALE=n1_BE
```

NOTE In order to use locales other than `en`, ensure the `ext-intl` PHP extension is installed and configured on your server.

Logging

Cashier allows you to specify the log channel to be used when logging all Stripe related exceptions. You may specify the log channel using the `CASHIER_LOGGER` environment variable:

```
CASHIER_LOGGER=stack
```

Customers

Retrieving Customers

You can retrieve a customer by their Stripe ID using the `Cashier::findBillable` method. This will return an instance of the `Billable` model:

```
use Laravel\Cashier\Cashier;  
  
$user = Cashier::findBillable($stripeId);
```

Creating Customers

Occasionally, you may wish to create a Stripe customer without beginning a subscription. You may accomplish this using the `createAsStripeCustomer` method:

```
$stripeCustomer = $user->createAsStripeCustomer();
```

Once the customer has been created in Stripe, you may begin a subscription at a later date. You can also use an optional `$options` array to pass in any additional parameters which are supported by the Stripe API:

```
$stripeCustomer = $user->createAsStripeCustomer($options);
```

You may use the `asStripeCustomer` method if you want to return the customer object if the billable entity is already a customer within Stripe:

```
$stripeCustomer = $user->asStripeCustomer();
```

The `createOrGetStripeCustomer` method may be used if you want to return the customer object but are not sure whether the billable entity is already a customer within Stripe. This method will create a new customer in Stripe if one does not already exist:

```
$stripeCustomer = $user->createOrGetStripeCustomer();
```

Updating Customers

Occasionally, you may wish to update the Stripe customer directly with additional information. You may accomplish this using the `updateStripeCustomer` method:

```
$stripeCustomer = $user->updateStripeCustomer($options);
```

Billing Portal

Stripe offers [an easy way to set up a billing portal](#) so your customer can manage their subscription, payment methods, and view their billing history. You can redirect your users to the billing portal using the `redirectToBillingPortal` method from a controller or route:

```
use Illuminate\Http\Request;

public function billingPortal(Request $request)
{
    return $request->user()->redirectToBillingPortal();
}
```

By default, when the user is finished managing their subscription, they can return to the home route of your application. You may provide a custom URL the user should return to by passing the URL as an argument to the `redirectToBillingPortal` method:

```
use Illuminate\Http\Request;

public function billingPortal(Request $request)
{
    return $request->user()->redirectToBillingPortal(
        route('billing')
    );
}
```

If you would like to only generate the URL to the billing portal, you may use the `billingPortalUrl` method:

```
$url = $user->billingPortalUrl(route('billing'));
```

Payment Methods

Storing Payment Methods

In order to create subscriptions or perform "one off" charges with Stripe, you will need to store a payment method and retrieve its identifier from Stripe. The approach used to accomplish differs based on whether you plan to use the payment method for subscriptions or single charges, so we will examine both below.

Payment Methods For Subscriptions

When storing credit cards to a customer for future use, the Stripe Setup Intents API must be used to securely gather the customer's payment method details. A "Setup Intent" indicates to Stripe the intention to charge a customer's payment method. Cashier's `Billable` trait includes the `createSetupIntent` to easily create a new Setup Intent. You should call this method from the route or controller that will render the form which gathers your customer's payment method details:

```
return view('update-payment-method', [
    'intent' => $user->createSetupIntent()
]);
```

After you have created the Setup Intent and passed it to the view, you should attach its secret to the element that will gather the payment method. For example, consider this "update payment method" form:

```
<input id="card-holder-name" type="text">

<!-- Stripe Elements Placeholder -->
<div id="card-element"></div>

<button id="card-button" data-secret="{{ $intent->client_secret }}">
    Update Payment Method
</button>
```

Next, the Stripe.js library may be used to attach a Stripe Element to the form and securely gather the customer's payment details:

```
<script src="https://js.stripe.com/v3/"></script>

<script>
    const stripe = Stripe('stripe-public-key');

    const elements = stripe.elements();
    const cardElement = elements.create('card');

    cardElement.mount('#card-element');
</script>
```

Next, the card can be verified and a secure "payment method identifier" can be retrieved from Stripe using [Stripe's confirmCardSetup method](#):

```
const cardHolderName = document.getElementById('card-holder-name');
const cardButton = document.getElementById('card-button');
const clientSecret = cardButton.dataset.secret;

cardButton.addEventListener('click', async (e) => {
    const { setupIntent, error } = await stripe.confirmCardSetup(
        clientSecret, {
            payment_method: {
                card: cardElement,
                billing_details: { name: cardHolderName.value }
            }
        }
    );

    if (error) {
        // Display "error.message" to the user...
    } else {
        // The card has been verified successfully...
    }
});
```

After the card has been verified by Stripe, you may pass the resulting `setupIntent.payment_method` identifier to your Laravel application, where it can be attached to the customer. The payment method can either be [added as a new payment method](#) or [used to update the default payment method](#). You can also immediately use the payment method identifier to [create a new subscription](#).

TIP If you would like more information about Setup Intents and gathering customer payment details please [review this overview provided by Stripe](#).

Payment Methods For Single Charges

Of course, when making a single charge against a customer's payment method we'll only need to use a payment method identifier a single time. Due to Stripe limitations, you may not use the stored default payment method of a customer for single charges. You must allow the customer to enter their payment method details using the Stripe.js library. For example, consider the following form:

```
<input id="card-holder-name" type="text">

<!-- Stripe Elements Placeholder -->
<div id="card-element"></div>

<button id="card-button">
    Process Payment
</button>
```

Next, the Stripe.js library may be used to attach a Stripe Element to the form and securely gather the customer's payment details:

```
<script src="https://js.stripe.com/v3/"></script>

<script>
    const stripe = Stripe('stripe-public-key');

    const elements = stripe.elements();
    const cardElement = elements.create('card');

    cardElement.mount('#card-element');
</script>
```

Next, the card can be verified and a secure "payment method identifier" can be retrieved from Stripe using [Stripe's createPaymentMethod method](#):

```
const cardHolderName = document.getElementById('card-holder-name');
const cardButton = document.getElementById('card-button');

cardButton.addEventListener('click', async (e) => {
    const { paymentMethod, error } = await stripe.createPaymentMethod(
        'card', cardElement, {
            billing_details: { name: cardHolderName.value }
        }
    );

    if (error) {
        // Display "error.message" to the user...
    } else {
        // The card has been verified successfully...
    }
});
```

If the card is verified successfully, you may pass the `paymentMethod.id` to your Laravel application and process a [single charge](#).

Retrieving Payment Methods

The `paymentMethods` method on the `Billable` model instance returns a collection of `Laravel\Cashier\PaymentMethod` instances:

```
$paymentMethods = $user->paymentMethods();
```

To retrieve the default payment method, the `defaultPaymentMethod` method may be used:

```
$paymentMethod = $user->defaultPaymentMethod();
```

You can also retrieve a specific payment method that is owned by the `Billable` model using the `findPaymentMethod` method:

```
$paymentMethod = $user->findPaymentMethod($paymentMethodId);
```

Determining If A User Has A Payment Method

To determine if a `Billable` model has a default payment method attached to their account, use the `hasDefaultPaymentMethod` method:

```
if ($user->hasDefaultPaymentMethod()) {
    //
}
```

To determine if a `Billable` model has at least one payment method attached to their account, use the `hasPaymentMethod` method:

```
if ($user->hasPaymentMethod()) {
    //
}
```

Updating The Default Payment Method

The `updateDefaultPaymentMethod` method may be used to update a customer's default payment method information. This method accepts a Stripe payment method identifier and will assign the new payment method as the default billing payment method:

```
$user->updateDefaultPaymentMethod($paymentMethod);
```

To sync your default payment method information with the customer's default payment method information in Stripe, you may use the `updateDefaultPaymentMethodFromStripe` method:

```
$user->updateDefaultPaymentMethodFromStripe();
```

NOTE The default payment method on a customer can only be used for invoicing and creating new subscriptions. Due to limitations from Stripe, it may not be used for single charges.

Adding Payment Methods

To add a new payment method, you may call the `addPaymentMethod` method on the billable user, passing the payment method identifier:

```
$user->addPaymentMethod($paymentMethod);
```

TIP To learn how to retrieve payment method identifiers please review the [payment method storage documentation](#).

Deleting Payment Methods

To delete a payment method, you may call the `delete` method on the `Laravel\Cashier\PaymentMethod` instance you wish to delete:

```
$paymentMethod->delete();
```

The `deletePaymentMethods` method will delete all of the payment method information for the Billable model:

```
$user->deletePaymentMethods();
```

NOTE If a user has an active subscription, you should prevent them from deleting their default payment method.

Subscriptions

Creating Subscriptions

To create a subscription, first retrieve an instance of your billable model, which typically will be an instance of `App\User`. Once you have retrieved the model instance, you may use the `newSubscription` method to create the model's subscription:

```
$user = User::find(1);
```

```
$user->newSubscription('default', 'price_premium')->create($paymentMethod);
```

The first argument passed to the `newSubscription` method should be the name of the subscription. If your application only offers a single subscription, you might call this `default` or `primary`. The second argument is the specific plan the user is subscribing to. This value should correspond to the plan's price identifier in Stripe.

The `create` method, which accepts [a Stripe payment method identifier](#) or Stripe `PaymentMethod` object, will begin the subscription as well as update your database with the customer ID and other relevant billing information.

NOTE Passing a payment method identifier directly to the `create()` subscription method will also automatically add it to the user's stored payment methods.

Quantities

If you would like to set a specific quantity for the plan when creating the subscription, you may use the `quantity` method:

```
$user->newSubscription('default', 'price_monthly')
    ->quantity(5)
    ->create($paymentMethod);
```

Additional Details

If you would like to specify additional customer or subscription details, you may do so by passing them as the second and third arguments to the `create` method:

```
$user->newSubscription('default', 'price_monthly')->create($paymentMethod, [
    'email' => $email,
], [
    'metadata' => ['note' => 'Some extra information.'],
]);
```

To learn more about the additional fields supported by Stripe, check out Stripe's documentation on [customer creation](#) and [subscription creation](#).

Coupons

If you would like to apply a coupon when creating the subscription, you may use the `withCoupon` method:

```
$user->newSubscription('default', 'price_monthly')
    ->withCoupon('code')
    ->create($paymentMethod);
```

Adding Subscriptions

If you would like to add a subscription to a customer who already has a default payment method set you can use the `add` method when using the `newSubscription` method:

```
$user = User::find(1);

$user->newSubscription('default', 'price_premium')->add();
```

Checking Subscription Status

Once a user is subscribed to your application, you may easily check their subscription status using a variety of convenient methods. First, the `subscribed` method returns `true` if the user has an active subscription, even if the subscription is currently within its trial period:

```
if ($user->subscribed('default')) {
    //
}
```

The `subscribed` method also makes a great candidate for a [route middleware](#), allowing you to filter access to routes and controllers based on the user's subscription status:

```
public function handle($request, Closure $next)
{
    if ($request->user() && ! $request->user()->subscribed('default')) {
        // This user is not a paying customer...
        return redirect('billing');
    }

    return $next($request);
}
```

If you would like to determine if a user is still within their trial period, you may use the `onTrial` method. This method can be useful for displaying a warning to the user that they are still on their trial period:

```
if ($user->subscription('default')->onTrial()) {
    //
}
```

The `subscribedToPlan` method may be used to determine if the user is subscribed to a given plan based on a given Stripe Price ID. In this example, we will determine if the user's `default` subscription is actively subscribed to the `monthly` plan:

```
if ($user->subscribedToPlan('price_monthly', 'default')) {  
    //  
}
```

By passing an array to the `subscribedToPlan` method, you may determine if the user's `default` subscription is actively subscribed to the `monthly` or the `yearly` plan:

```
if ($user->subscribedToPlan(['price_monthly', 'price_yearly'], 'default')) {  
    //  
}
```

The `recurring` method may be used to determine if the user is currently subscribed and is no longer within their trial period:

```
if ($user->subscription('default')->recurring()) {  
    //  
}
```

Cancelled Subscription Status

To determine if the user was once an active subscriber, but has cancelled their subscription, you may use the `cancelled` method:

```
if ($user->subscription('default')->cancelled()) {  
    //  
}
```

You may also determine if a user has cancelled their subscription, but are still on their "grace period" until the subscription fully expires. For example, if a user cancels a subscription on March 5th that was originally scheduled to expire on March 10th, the user is on their "grace period" until March 10th. Note that the `subscribed` method still returns `true` during this time:

```
if ($user->subscription('default')->onGracePeriod()) {  
    //  
}
```

To determine if the user has cancelled their subscription and is no longer within their "grace period", you may use the `ended` method:

```
if ($user->subscription('default')->ended()) {  
    //  
}
```

Subscription Scopes

Most subscription states are also available as query scopes so that you may easily query your database for subscriptions that are in a given state:

```
// Get all active subscriptions...  
$subscriptions = Subscription::query()->active()->get();  
  
// Get all of the cancelled subscriptions for a user...  
$subscriptions = $user->subscriptions()->cancelled()->get();
```

A complete list of available scopes is available below:

```
Subscription::query()->active();  
Subscription::query()->cancelled();  
Subscription::query()->ended();  
Subscription::query()->incomplete();  
Subscription::query()->notCancelled();  
Subscription::query()->notOnGracePeriod();  
Subscription::query()->notOnTrial();  
Subscription::query()->onGracePeriod();  
Subscription::query()->onTrial();  
Subscription::query()->pastDue();
```



```
Subscription::query()->recurring();
```

Incomplete and Past Due Status

If a subscription requires a secondary payment action after creation the subscription will be marked as `incomplete`. Subscription statuses are stored in the `stripe_status` column of Cashier's `subscriptions` database table.

Similarly, if a secondary payment action is required when swapping plans the subscription will be marked as `past_due`. When your subscription is in either of these states it will not be active until the customer has confirmed their payment. Checking if a subscription has an incomplete payment can be done using the `hasIncompletePayment` method on the `Billable` model or a subscription instance:

```
if ($user->hasIncompletePayment('default')) {  
    //  
}  
  
if ($user->subscription('default')->hasIncompletePayment()) {  
    //  
}
```

When a subscription has an incomplete payment, you should direct the user to Cashier's payment confirmation page, passing the `latestPayment` identifier. You may use the `latestPayment` method available on subscription instance to retrieve this identifier:

```
<a href="{{ route('cashier.payment', $subscription->latestPayment()->id) }}">  
    Please confirm your payment.  
</a>
```

If you would like the subscription to still be considered active when it's in a `past_due` state, you may use the `keepPastDueSubscriptionsActive` method provided by Cashier. Typically, this method should be called in the `register` method of your `AppServiceProvider`:

```
use Laravel\Cashier\Cashier;  
  
/**  
 * Register any application services.  
 *  
 * @return void  
 */  
public function register()  
{  
    Cashier::keepPastDueSubscriptionsActive();  
}
```

NOTE When a subscription is in an `incomplete` state it cannot be changed until the payment is confirmed. Therefore, the `swap` and `updateQuantity` methods will throw an exception when the subscription is in an `incomplete` state.

Changing Plans

After a user is subscribed to your application, they may occasionally want to change to a new subscription plan. To swap a user to a new subscription, pass the plan's price identifier to the `swap` method:

```
$user = App\User::find(1);  
  
$user->subscription('default')->swap('provider-price-id');
```

If the user is on trial, the trial period will be maintained. Also, if a "quantity" exists for the subscription, that quantity will also be maintained.

If you would like to swap plans and cancel any trial period the user is currently on, you may use the `skipTrial` method:

```
$user->subscription('default')  
    ->skipTrial()  
    ->swap('provider-price-id');
```

If you would like to swap plans and immediately invoice the user instead of waiting for their next billing cycle, you may use the `swapAndInvoice` method:

```
$user = App\User::find(1);  
$user->subscription('default')->swapAndInvoice('provider-price-id');
```

Prorations

By default, Stripe prorates charges when swapping between plans. The `noProrate` method may be used to update the subscription's without prorating the charges:

```
$user->subscription('default')->noProrate()->swap('provider-price-id');
```

For more information on subscription proration, consult the [Stripe documentation](#).

NOTE Executing the `noProrate` method before the `swapAndInvoice` method will have no affect on proration. An invoice will always be issued.

Subscription Quantity

Sometimes subscriptions are affected by "quantity". For example, your application might charge \$10 per month **per user** on an account. To easily increment or decrement your subscription quantity, use the `incrementQuantity` and `decrementQuantity` methods:

```
$user = User::find(1);  
$user->subscription('default')->incrementQuantity();  
  
// Add five to the subscription's current quantity...  
$user->subscription('default')->incrementQuantity(5);  
  
$user->subscription('default')->decrementQuantity();  
  
// Subtract five to the subscription's current quantity...  
$user->subscription('default')->decrementQuantity(5);
```

Alternatively, you may set a specific quantity using the `updateQuantity` method:

```
$user->subscription('default')->updateQuantity(10);
```

The `noProrate` method may be used to update the subscription's quantity without prorating the charges:

```
$user->subscription('default')->noProrate()->updateQuantity(10);
```

For more information on subscription quantities, consult the [Stripe documentation](#).

NOTE Please note that when working with multiplan subscriptions, an extra "plan" parameter is required for the above quantity methods.

Multiplan Subscriptions

[Multiplan subscriptions](#) allow you to assign multiple billing plans to a single subscription. For example, imagine you are building a customer service "helpdesk" application that has a base subscription of \$10 per month, but offers a live chat add-on plan for an additional \$15 per month:

```
$user = User::find(1);  
$user->newSubscription('default', [  
    'price_monthly',  
    'chat-plan',  
)->create($paymentMethod);
```

Now the customer will have two plans on their `default` subscription. Both plans will be charged for on their respective billing intervals. You can also use the `quantity` method to indicate the specific quantity for each plan:

```
$user = User::find(1);
```

```
$user->newSubscription('default', ['price_monthly', 'chat-plan'])
    ->quantity(5, 'chat-plan')
    ->create($paymentMethod);
```

Or, you may dynamically add the extra plan and its quantity using the `plan` method:

```
$user = User::find(1);

$user->newSubscription('default', 'price_monthly')
    ->plan('chat-plan', 5)
    ->create($paymentMethod);
```

Alternatively, you may add a new plan to an existing subscription at a later time:

```
$user = User::find(1);

$user->subscription('default')->addPlan('chat-plan');
```

The example above will add the new plan and the customer will be billed for it on their next billing cycle. If you would like to bill the customer immediately you may use the `addPlanAndInvoice` method:

```
$user->subscription('default')->addPlanAndInvoice('chat-plan');
```

If you would like to add a plan with a specific quantity, you can pass the quantity as the second parameter of the `addPlan` OR `addPlanAndInvoice` methods:

```
$user = User::find(1);

$user->subscription('default')->addPlan('chat-plan', 5);
```

You may remove plans from subscriptions using the `removePlan` method:

```
$user->subscription('default')->removePlan('chat-plan');
```

NOTE You may not remove the last plan on a subscription. Instead, you should simply cancel the subscription.

Swapping

You may also change the plans attached to a multiplan subscription. For example, imagine you're on a `basic-plan` subscription with a `chat-plan` add-on and you want to upgrade to the `pro-plan` plan:

```
$user = User::find(1);

$user->subscription('default')->swap(['pro-plan', 'chat-plan']);
```

When executing the code above, the underlying subscription item with the `basic-plan` is deleted and the one with the `chat-plan` is preserved. Additionally, a new subscription item for the new `pro-plan` is created.

You can also specify subscription item options. For example, you may need to specify the subscription plan quantities:

```
$user = User::find(1);

$user->subscription('default')->swap([
    'pro-plan' => ['quantity' => 5],
    'chat-plan'
]);
```

If you want to swap a single plan on a subscription, you may do so using the `swap` method on the subscription item itself. This approach is useful if you, for example, want to preserve all of the existing metadata on the subscription item.

```
$user = User::find(1);

$user->subscription('default')
    ->findItemOrFail('basic-plan')
    ->swap('pro-plan');
```

Proration

By default, Stripe will prorate charges when adding or removing plans from a subscription. If you would like to make a plan adjustment without proration, you should chain the `noProrate` method onto your plan operation:

```
$user->subscription('default')->noProrate()->removePlan('chat-plan');
```

Quantities

If you would like to update quantities on individual subscription plans, you may do so using the [existing quantity methods](#) and passing the name of the plan as an additional argument to the method:

```
$user = User::find(1);

$user->subscription('default')->incrementQuantity(5, 'chat-plan');
$user->subscription('default')->decrementQuantity(3, 'chat-plan');
$user->subscription('default')->updateQuantity(10, 'chat-plan');
```

NOTE When you have multiple plans set on a subscription the `stripe_plan` and `quantity` attributes on the `Subscription` model will be `null`. To access the individual plans, you should use the `items` relationship available on the `Subscription` model.

Subscription Items

When a subscription has multiple plans, it will have multiple subscription "items" stored in your database's `subscription_items` table. You may access these via the `items` relationship on the subscription:

```
$user = User::find(1);

$subscriptionItem = $user->subscription('default')->items->first();

// Retrieve the Stripe plan and quantity for a specific item...
$stripePlan = $subscriptionItem->stripe_plan;
$quantity = $subscriptionItem->quantity;
```

You can also retrieve a specific plan using the `findItemOrFail` method:

```
$user = User::find(1);

$subscriptionItem = $user->subscription('default')->findItemOrFail('chat-plan');
```

Subscription Taxes

To specify the tax rates a user pays on a subscription, implement the `taxRates` method on your billable model, and return an array with the Tax Rate IDs. You can define these tax rates in [your Stripe dashboard](#):

```
public function taxRates()
{
    return ['tax-rate-id'];
}
```

The `taxRates` method enables you to apply a tax rate on a model-by-model basis, which may be helpful for a user base that spans multiple countries and tax rates. If you're working with multiplan subscriptions you can define different tax rates for each plan by implementing a `planTaxRates` method on your billable model:

```
public function planTaxRates()
{
    return [
        'plan-id' => ['tax-rate-id'],
    ];
}
```

NOTE The `taxRates` method only applies to subscription charges. If you use `Cashier` to make "one off" charges, you will need to manually specify the tax rate at that time.

Syncing Tax Rates

When changing the hard-coded Tax Rate IDs returned by the `taxRates` method, the tax settings on any existing

subscriptions for the user will remain the same. If you wish to update the tax value for existing subscriptions with the returned `taxTaxRates` values, you should call the `syncTaxRates` method on the user's subscription instance:

```
$user->subscription('default')->syncTaxRates();
```

This will also sync any subscription item tax rates so make sure you also properly change the `planTaxRates` method.

Tax Exemption

Cashier also offers methods to determine if the customer is tax exempt by calling the Stripe API. The `isNotTaxExempt`, `isTaxExempt`, and `reverseChargeApplies` methods are available on the billable model:

```
$user = User::find(1);

$user->isTaxExempt();
$user->isNotTaxExempt();
$user->reverseChargeApplies();
```

These methods are also available on any `Laravel\Cashier\Invoice` object. However, when calling these methods on an `Invoice` object the methods will determine the exemption status at the time the invoice was created.

Subscription Anchor Date

By default, the billing cycle anchor is the date the subscription was created, or if a trial period is used, the date that the trial ends. If you would like to modify the billing anchor date, you may use the `anchorBillingCycleOn` method:

```
use App\User;
use Carbon\Carbon;

$user = User::find(1);

$anchor = Carbon::parse('first day of next month');

$user->newSubscription('default', 'price_premium')
    ->anchorBillingCycleOn($anchor->startOfDay())
    ->create($paymentMethod);
```

For more information on managing subscription billing cycles, consult the [Stripe billing cycle documentation](#)

Cancelling Subscriptions

To cancel a subscription, call the `cancel` method on the user's subscription:

```
$user->subscription('default')->cancel();
```

When a subscription is cancelled, Cashier will automatically set the `ends_at` column in your database. This column is used to know when the `subscribed` method should begin returning `false`. For example, if a customer cancels a subscription on March 1st, but the subscription was not scheduled to end until March 5th, the `subscribed` method will continue to return `true` until March 5th.

You may determine if a user has cancelled their subscription but are still on their "grace period" using the `onGracePeriod` method:

```
if ($user->subscription('default')->onGracePeriod()) {
    //
}
```

If you wish to cancel a subscription immediately, call the `cancelNow` method on the user's subscription:

```
$user->subscription('default')->cancelNow();
```

Resuming Subscriptions

If a user has cancelled their subscription and you wish to resume it, use the `resume` method. The user **must** still be on their grace period in order to resume a subscription:

```
$user->subscription('default')->resume();
```

If the user cancels a subscription and then resumes that subscription before the subscription has fully expired, they will not be billed immediately. Instead, their subscription will be re-activated, and they will be billed on the original billing cycle.

Subscription Trials

With Payment Method Up Front

If you would like to offer trial periods to your customers while still collecting payment method information up front, you should use the `trialDays` method when creating your subscriptions:

```
$user = User::find(1);

$user->newSubscription('default', 'price_monthly')
    ->trialDays(10)
    ->create($paymentMethod);
```

This method will set the trial period ending date on the subscription record within the database, as well as instruct Stripe to not begin billing the customer until after this date. When using the `trialDays` method, Cashier will overwrite any default trial period configured for the plan in Stripe.

NOTE If the customer's subscription is not cancelled before the trial ending date they will be charged as soon as the trial expires, so you should be sure to notify your users of their trial ending date.

The `trialUntil` method allows you to provide a `DateTime` instance to specify when the trial period should end:

```
use Carbon\Carbon;

$user->newSubscription('default', 'price_monthly')
    ->trialUntil(Carbon::now()->addDays(10))
    ->create($paymentMethod);
```

You may determine if the user is within their trial period using either the `onTrial` method of the user instance, or the `onTrial` method of the subscription instance. The two examples below are identical:

```
if ($user->onTrial('default')) {
    //
}

if ($user->subscription('default')->onTrial()) {
    //
}
```

Defining Trial Days In Stripe / Cashier

You may choose to define how many trial days your plan's receive in the Stripe dashboard or always pass them explicitly using Cashier. If you choose to define your plan's trial days in Stripe you should be aware that new subscriptions, including new subscriptions for a customer that had a subscription in the past, will always receive a trial period unless you explicitly call the `trialDays(0)` method.

Without Payment Method Up Front

If you would like to offer trial periods without collecting the user's payment method information up front, you may set the `trial_ends_at` column on the user record to your desired trial ending date. This is typically done during user registration:

```
$user = User::create([
    // Populate other user properties...
    'trial_ends_at' => now()->addDays(10),
]);
```

NOTE Be sure to add a [date mutator](#) for `trial_ends_at` to your model definition.

Cashier refers to this type of trial as a "generic trial", since it is not attached to any existing subscription. The `onTrial` method on the `User` instance will return `true` if the current date is not past the value of `trial_ends_at`:

```
if ($user->onTrial()) {
    // User is within their trial period...
}
```

You may also use the `onGenericTrial` method if you wish to know specifically that the user is within their "generic" trial period and has not created an actual subscription yet:

```
if ($user->onGenericTrial()) {
    // User is within their "generic" trial period...
}
```

Once you are ready to create an actual subscription for the user, you may use the `newSubscription` method as usual:

```
$user = User::find(1);

$user->newSubscription('default', 'price_monthly')->create($paymentMethod);
```

Extending Trials

The `extendTrial` method allows you to extend the trial period of a subscription after it's been created:

```
// End the trial 7 days from now...
$subscription->extendTrial(
    now()->addDays(7)
);

// Add an additional 5 days to the trial...
$subscription->extendTrial(
    $subscription->trial_ends_at->addDays(5)
);
```

If the trial has already expired and the customer is already being billed for the subscription, you can still offer them an extended trial. The time spent within the trial period will be deducted from the customer's next invoice.

Handling Stripe Webhooks

TIP You may use [the Stripe CLI](#) to help test webhooks during local development.

Stripe can notify your application of a variety of events via webhooks. By default, a route that points to Cashier's webhook controller is configured through the Cashier service provider. This controller will handle all incoming webhook requests.

By default, this controller will automatically handle cancelling subscriptions that have too many failed charges (as defined by your Stripe settings), customer updates, customer deletions, subscription updates, and payment method changes; however, as we'll soon discover, you can extend this controller to handle any webhook event you like.

To ensure your application can handle Stripe webhooks, be sure to configure the webhook URL in the Stripe control panel. By default, Cashier's webhook controller listens to the `/stripe/webhook` URL path. The full list of all webhooks you should configure in the Stripe control panel are:

- `customer.subscription.updated`
- `customer.subscription.deleted`
- `customer.updated`
- `customer.deleted`
- `invoice.payment_action_required`

NOTE Make sure you protect incoming requests with Cashier's included [webhook signature verification](#) middleware.

Webhooks & CSRF Protection

Since Stripe webhooks need to bypass Laravel's [CSRF protection](#), be sure to list the URI as an exception in your `VerifyCsrfToken` middleware or list the route outside of the `web` middleware group:

```
protected $except = [
    'stripe/**',
];
```

Defining Webhook Event Handlers

Cashier automatically handles subscription cancellation on failed charges, but if you have additional webhook events you would like to handle, extend the Webhook controller. Your method names should correspond to Cashier's expected convention, specifically, methods should be prefixed with `handle` and the "camel case" name of the webhook you wish to handle. For example, if you wish to handle the `invoice.payment_succeeded` webhook, you should add a `handleInvoicePaymentSucceeded` method to the controller:

```
<?php

namespace App\Http\Controllers;

use Laravel\Cashier\Http\Controllers\WebhookController as CashierController;

class WebhookController extends CashierController
{
    /**
     * Handle invoice payment succeeded.
     *
     * @param array $payload
     * @return \Symfony\Component\HttpFoundation\Response
     */
    public function handleInvoicePaymentSucceeded($payload)
    {
        // Handle The Event
    }
}
```

Next, define a route to your Cashier controller within your `routes/web.php` file. This will overwrite the default shipped route:

```
Route::post(
    'stripe/webhook',
    '\App\Http\Controllers\WebhookController@handleWebhook'
);
```

Cashier emits a `Laravel\Cashier\Events\WebhookReceived` event when a webhook is received, and a `Laravel\Cashier\Events\WebhookHandled` event when a webhook was handled by Cashier. Both events contain the full payload of the Stripe webhook.

Failed Subscriptions

What if a customer's credit card expires? No worries - Cashier's Webhook controller will cancel the customer's subscription for you. Failed payments will automatically be captured and handled by the controller. The controller will cancel the customer's subscription when Stripe determines the subscription has failed (normally after three failed payment attempts).

Verifying Webhook Signatures

To secure your webhooks, you may use [Stripe's webhook signatures](#). For convenience, Cashier automatically includes a middleware which validates that the incoming Stripe webhook request is valid.

To enable webhook verification, ensure that the `STRIPE_WEBHOOK_SECRET` environment variable is set in your `.env` file. The webhook secret may be retrieved from your Stripe account dashboard.

Single Charges

Simple Charge

NOTE The charge method accepts the amount you would like to charge in the **lowest denominator of the currency used by your application**.

If you would like to make a "one off" charge against a subscribed customer's payment method, you may use the charge method on a billable model instance. You'll need to [provide a payment method identifier](#) as the second argument:

```
// Stripe Accepts Charges In Cents...
$stripeCharge = $user->charge(100, $paymentMethod);
```

The charge method accepts an array as its third argument, allowing you to pass any options you wish to the underlying Stripe charge creation. Consult the Stripe documentation regarding the options available to you when creating charges:

```
$user->charge(100, $paymentMethod, [
    'custom_option' => $value,
]);
```

You may also use the charge method without an underlying customer or user:

```
use App\User;

$stripeCharge = (new User)->charge(100, $paymentMethod);
```

The charge method will throw an exception if the charge fails. If the charge is successful, an instance of `Laravel\Cashier\Payment` will be returned from the method:

```
try {
    $payment = $user->charge(100, $paymentMethod);
} catch (Exception $e) {
    //
}
```

Charge With Invoice

Sometimes you may need to make a one-time charge but also generate an invoice for the charge so that you may offer a PDF receipt to your customer. The `invoiceFor` method lets you do just that. For example, let's invoice the customer \$5.00 for a "One Time Fee":

```
// Stripe Accepts Charges In Cents...
$user->invoiceFor('One Time Fee', 500);
```

The invoice will be charged immediately against the user's default payment method. The `invoiceFor` method also accepts an array as its third argument. This array contains the billing options for the invoice item. The fourth argument accepted by the method is also an array. This final argument accepts the billing options for the invoice itself:

```
$user->invoiceFor('Stickers', 500, [
    'quantity' => 50,
], [
    'default_tax_rates' => ['tax-rate-id'],
]);
```

NOTE The `invoiceFor` method will create a Stripe invoice which will retry failed billing attempts. If you do not want invoices to retry failed charges, you will need to close them using the Stripe API after the first failed charge.

Refunding Charges

If you need to refund a Stripe charge, you may use the `refund` method. This method accepts the Stripe Payment Intent ID as its first argument:

```
$payment = $user->charge(100, $paymentMethod);

$user->refund($payment->id);
```

Invoices

Retrieving Invoices

You may easily retrieve an array of a billable model's invoices using the `invoices` method:

```
$invoices = $user->invoices();

// Include pending invoices in the results...
$invoices = $user->invoicesIncludingPending();
```

You may use the `findInvoice` method to retrieve a specific invoice:

```
$invoice = $user->findInvoice($invoiceId);
```

Displaying Invoice Information

When listing the invoices for the customer, you may use the invoice's helper methods to display the relevant invoice information. For example, you may wish to list every invoice in a table, allowing the user to easily download any of them:

```
<table>
    @foreach ($invoices as $invoice)
        <tr>
            <td>{{ $invoice->date()->toFormattedDateString() }}</td>
            <td>{{ $invoice->total() }}</td>
            <td><a href="/user/invoice/{{ $invoice->id }}">Download</a></td>
        </tr>
    @endforeach
</table>
```

Generating Invoice PDFs

From within a route or controller, use the `downloadInvoice` method to generate a PDF download of the invoice. This method will automatically generate the proper HTTP response to send the download to the browser:

```
use Illuminate\Http\Request;

Route::get('user/invoice/{invoice}', function (Request $request, $invoiceId) {
    return $request->user()->downloadInvoice($invoiceId, [
        'vendor' => 'Your Company',
        'product' => 'Your Product',
    ]);
});
```

The `downloadInvoice` method also allows for an optional custom filename as the third parameter. This filename will automatically be suffixed with `.pdf` for you:

```
return $request->user()->downloadInvoice($invoiceId, [
    'vendor' => 'Your Company',
    'product' => 'Your Product',
], 'my-invoice');
```

Handling Failed Payments

Sometimes, payments for subscriptions or single charges can fail. When this happens, Cashier will throw an `IncompletePayment` exception that informs you that this happened. After catching this exception, you have two options on how to proceed.

First, you could redirect your customer to the dedicated payment confirmation page which is included with Cashier. This page already has an associated route that is registered via Cashier's service provider. So, you may catch the `IncompletePayment` exception and redirect to the payment confirmation page:

```
use Laravel\Cashier\Exceptions\IncompletePayment;

try {
    $subscription = $user->newSubscription('default', $planId)
```

```
        ->create($paymentMethod);
    } catch (IncompletePayment $exception) {
        return redirect()->route(
            'cashier.payment',
            [$exception->payment->id, 'redirect' => route('home')]
        );
    }
}
```

On the payment confirmation page, the customer will be prompted to enter their credit card info again and perform any additional actions required by Stripe, such as "3D Secure" confirmation. After confirming their payment, the user will be redirected to the URL provided by the `redirect` parameter specified above. Upon redirection, `message` (string) and `success` (integer) query string variables will be added to the URL.

Alternatively, you could allow Stripe to handle the payment confirmation for you. In this case, instead of redirecting to the payment confirmation page, you may [setup Stripe's automatic billing emails](#) in your Stripe dashboard. However, if a `IncompletePayment` exception is caught, you should still inform the user they will receive an email with further payment confirmation instructions.

Payment exceptions may be thrown for the following methods: `charge`, `invoiceFor`, and `invoice` on the `Billable` user. When handling subscriptions, the `create` method on the `SubscriptionBuilder`, and the `incrementAndInvoice` and `swapAndInvoice` methods on the `Subscription` model may throw exceptions.

There are currently two types of payment exceptions which extend `IncompletePayment`. You can catch these separately if needed so that you can customize the user experience:

- `PaymentActionRequired`: this indicates that Stripe requires extra verification in order to confirm and process a payment.
- `PaymentFailure`: this indicates that a payment failed for various other reasons, such as being out of available funds.

Strong Customer Authentication

If your business is based in Europe you will need to abide by the Strong Customer Authentication (SCA) regulations. These regulations were imposed in September 2019 by the European Union to prevent payment fraud. Luckily, Stripe and Cashier are prepared for building SCA compliant applications.

NOTE Before getting started, review [Stripe's guide on PSD2 and SCA](#) as well as their [documentation on the new SCA APIs](#).

Payments Requiring Additional Confirmation

SCA regulations often require extra verification in order to confirm and process a payment. When this happens, Cashier will throw a `PaymentActionRequired` exception that informs you that this extra verification is needed. More info on how to handle these exceptions be found [here](#).

Incomplete and Past Due State

When a payment needs additional confirmation, the subscription will remain in an `incomplete` or `past_due` state as indicated by its `stripe_status` database column. Cashier will automatically activate the customer's subscription via a webhook as soon as payment confirmation is complete.

For more information on `incomplete` and `past_due` states, please refer to [our additional documentation](#).

Off-Session Payment Notifications

Since SCA regulations require customers to occasionally verify their payment details even while their subscription is active, Cashier can send a payment notification to the customer when off-session payment confirmation is required. For example, this may occur when a subscription is renewing. Cashier's payment notification can be enabled by setting the `CASHIER_PAYMENT_NOTIFICATION` environment variable to a notification class. By default, this notification is disabled. Of course, Cashier includes a notification class you may use for this purpose, but you are free to provide your own notification class if desired:

CASHIER_PAYMENT_NOTIFICATION=Laravel\Cashier\Notifications\ConfirmPayment

To ensure that off-session payment confirmation notifications are delivered, verify that [Stripe webhooks are configured](#) for your application and the `invoice.payment_action_required` webhook is enabled in your Stripe dashboard. In addition, your `Billable` model should also use Laravel's `Illuminate\Notifications\Notifiable` trait.

NOTE Notifications will be sent even when customers are manually making a payment that requires additional confirmation. Unfortunately, there is no way for Stripe to know that the payment was done manually or "off-session". But, a customer will simply see a "Payment Successful" message if they visit the payment page after already confirming their payment. The customer will not be allowed to accidentally confirm the same payment twice and incur an accidental second charge.

Stripe SDK

Many of Cashier's objects are wrappers around Stripe SDK objects. If you would like to interact with the Stripe objects directly, you may conveniently retrieve them using the `asStripe` method:

```
$stripeSubscription = $subscription->asStripeSubscription();  
  
$stripeSubscription->application_fee_percent = 5;  
  
$stripeSubscription->save();
```

You may also use the `updateStripeSubscription` method to update the Stripe subscription directly:

```
$subscription->updateStripeSubscription(['application_fee_percent' => 5]);
```

Testing

When testing an application that uses Cashier, you may mock the actual HTTP requests to the Stripe API; however, this requires you to partially re-implement Cashier's own behavior. Therefore, we recommend allowing your tests to hit the actual Stripe API. While this is slower, it provides more confidence that your application is working as expected and any slow tests may be placed within their own PHPUnit testing group.

When testing, remember that that Cashier itself already has a great test suite, so you should only focus on testing the subscription and payment flow of your own application and not every underlying Cashier behavior.

To get started, add the **testing** version of your Stripe secret to your `phpunit.xml` file:

```
<env name="STRIPE_SECRET" value="sk_test_<your-key>"/>
```

Now, whenever you interact with Cashier while testing, it will send actual API requests to your Stripe testing environment. For convenience, you should pre-fill your Stripe testing account with subscriptions / plans that you may then use during testing.

TIP In order to test a variety of billing scenarios, such as credit card denials and failures, you may use the vast range of [testing card numbers and tokens](#) provided by Stripe.

Official Packages

Cashier (Paddle)

Laravel Cashier Paddle

- [Introduction](#)
- [Upgrading Cashier](#)
- [Installation](#)
- [Configuration](#)
 - [Billable Model](#)
 - [API Keys](#)
 - [Paddle JS](#)
 - [Currency Configuration](#)
- [Core Concepts](#)
 - [Pay Links](#)
 - [Inline Checkout](#)
 - [User Identification](#)
- [Prices](#)
- [Customers](#)
 - [Customer Defaults](#)
- [Subscriptions](#)
 - [Creating Subscriptions](#)
 - [Checking Subscription Status](#)
 - [Subscription Single Charges](#)
 - [Updating Payment Information](#)
 - [Changing Plans](#)
 - [Subscription Quantity](#)
 - [Pausing Subscriptions](#)
 - [Cancelling Subscriptions](#)
- [Subscription Trials](#)
 - [With Payment Method Up Front](#)
 - [Without Payment Method Up Front](#)
- [Handling Paddle Webhooks](#)
 - [Defining Webhook Event Handlers](#)
 - [Failed Subscriptions](#)
 - [Verifying Webhook Signatures](#)
- [Single Charges](#)
 - [Simple Charge](#)
 - [Charging Products](#)
 - [Refunding Orders](#)
- [Receipts](#)
 - [Past & Upcoming Payments](#)
- [Handling Failed Payments](#)
- [Testing](#)

Introduction

Laravel Cashier Paddle provides an expressive, fluent interface to [Paddle's](#) subscription billing services. It handles almost all of the boilerplate subscription billing code you are dreading. In addition to basic subscription management, Cashier can handle: coupons, swapping subscription, subscription "quantities", cancellation grace periods, and more.

While working with Cashier we recommend you also refer to Paddle's [user guides](#) and [API documentation](#).

Upgrading Cashier

When upgrading to a new version of Cashier, it's important that you carefully review [the upgrade guide](#).

Installation

First, require the Cashier package for Paddle with Composer:

```
composer require laravel/cashier-paddle
```

NOTE To ensure Cashier properly handles all Paddle events, remember to [set up Cashier's webhook handling](#).

Database Migrations

The Cashier service provider registers its own database migration directory, so remember to migrate your database after installing the package. The Cashier migrations will create a new `customers` table. In addition, a new `subscriptions` table will be created to store all of your customer's subscriptions. Finally, a new `receipts` table will be created to store all of your receipt information:

```
php artisan migrate
```

If you need to overwrite the migrations that ship with the Cashier package, you can publish them using the `vendor:publish` Artisan command:

```
php artisan vendor:publish --tag="cashier-migrations"
```

If you would like to prevent Cashier's migrations from running entirely, you may use the `ignoreMigrations` provided by Cashier. Typically, this method should be called in the `register` method of your `AppServiceProvider`:

```
use Laravel\Paddle\Cashier;

Cashier::ignoreMigrations();
```

Configuration

Billable Model

Before using Cashier, you must add the `Billable` trait to your user model definition. This trait provides various methods to allow you to perform common billing tasks, such as creating subscriptions, applying coupons and updating payment method information:

```
use Laravel\Paddle\Billable;

class User extends Authenticatable
{
    use Billable;
}
```

If you have billable entities that are not users, you may also add the trait to those classes:

```
use Laravel\Paddle\Billable;

class Team extends Model
{
    use Billable;
}
```

API Keys

Next, you should configure your Paddle keys in your `.env` file. You can retrieve your Paddle API keys from the Paddle control panel:

```
PADDLE_VENDOR_ID=your-paddle-vendor-id
PADDLE_VENDOR_AUTH_CODE=your-paddle-vendor-auth-code
PADDLE_PUBLIC_KEY="your-paddle-public-key"
```

Paddle JS

Paddle relies on its own JavaScript library to initiate the Paddle checkout widget. You can load the JavaScript library by placing the `@paddleJS` directive right before your application layout's closing `</head>` tag:

```
<head>
    ...

    @paddleJS
</head>
```

Currency Configuration

The default Cashier currency is United States Dollars (USD). You can change the default currency by setting the `CASHIER_CURRENCY` environment variable:

```
CASHIER_CURRENCY=EUR
```

In addition to configuring Cashier's currency, you may also specify a locale to be used when formatting money values for display on invoices. Internally, Cashier utilizes [PHP's NumberFormatter class](#) to set the currency locale:

```
CASHIER_CURRENCY_LOCALE=nl_BE
```

NOTE In order to use locales other than `en`, ensure the `ext-intl` PHP extension is installed and configured on your server.

Core Concepts

Pay Links

Paddle lacks an extensive CRUD API to perform state changes. Therefore, most interactions with Paddle are done through [its checkout widget](#). Before we can display the checkout widget, we will generate a "pay link" using Cashier:

```
$user = User::find(1);

$payLink = $user->newSubscription('default', $premium = 34567)
    ->returnTo(route('home'))
    ->create();

return view('billing', ['payLink' => $payLink]);
```

Cashier includes a `paddle-button` Blade component. We may pass the pay link URL to this component as a "prop". When this button is clicked, Paddle's checkout widget will be displayed:

```
<x-paddle-button :url="$payLink" class="px-8 py-4">
    Subscribe
</x-paddle-button>
```

By default, this will display a button with the standard Paddle styling. You can remove all Paddle styling by adding the `data-theme="none"` attribute to the component:

```
<x-paddle-button :url="$payLink" class="px-8 py-4" data-theme="none">
    Subscribe
</x-paddle-button>
```

The Paddle checkout widget is asynchronous. Once the user creates or updates a subscription within the widget, Paddle will send our application webhooks so that we may properly update the subscription state in our own database. Therefore, it's important that you properly [set up webhooks](#) to accommodate for state changes from Paddle.

After a subscription state change, the delay for receiving the corresponding webhook is typically minimal but you should account for this in your application by considering that your user's subscription might not be immediately available after completing the checkout.

For more information, you may review [the Paddle API documentation on pay link generation](#).

Inline Checkout

If you don't want to make use of the "overlay" style checkout widget, Paddle also has an option to display the widget inline. While this approach does not allow you to adjust any of the checkout's HTML fields, it allows you to embed the widget within your application.

To make it easy for you to get started with inline checkout, Cashier includes a `paddle-checkout` Blade component. To get started, you should [generate a pay link](#) and pass the pay link to the component's `override` attribute:

```
<x-paddle-checkout :override="$payLink" class="w-full" />
```

To adjust the height of the inline checkout component, you may pass the `height` attribute to the Blade component:

```
<x-paddle-checkout :override="$payLink" class="w-full" height="500" />
```

Inline Checkout Without Pay Links

Alternatively, you may customize the widget with custom options instead of using a pay link:

```
$options = [
    'product' => $productId,
    'title' => 'Product Title',
];

<x-paddle-checkout :options="$options" class="w-full" />
```

Please consult Paddle's [guide on Inline Checkout](#) as well as their [Parameter Reference](#) for further details on available options.

NOTE If you would like to also use the `passthrough` option when specifying custom options, you should provide a key / value array since Cashier will automatically handle converting the array to a JSON string. In addition, the `customer_id` passthrough option is reserved for internal Cashier usage.

User Identification

In contrast to Stripe, Paddle users are unique across the whole of Paddle, not unique per Paddle account. Because of this, Paddle's API's do not currently provide a method to update a user's details such as their email address. When generating pay links, Paddle identifies users using the `customer_email` parameter. When creating a subscription, Paddle will try to match the user provided email to an existing Paddle user.

In light of this behavior, there are some important things to keep in mind when using Cashier and Paddle. First, you should be aware that even though subscriptions in Cashier are tied to the same application user, **they could be tied to different users within Paddle's internal systems**. Secondly, each subscription has its own connected payment method information and could also have different email addresses within Paddle's internal systems (depending on which email was assigned to the user when the subscription was created).

Therefore, when displaying subscriptions you should always inform the user which email address or payment method information is connected to the subscription on a per-subscription basis. Retrieving this information can be done with the following methods on the `Subscription` model:

```
$subscription = $user->subscription('default');

$customerEmailAddress = $subscription->paddleEmail();
$paymentMethod = $subscription->paymentMethod();
$cardBrand = $subscription->cardBrand();
$cardLastFour = $subscription->cardLastFour();
$cardExpirationDate = $subscription->cardExpirationDate();
```

There is currently no way to modify a user's email address through the Paddle API. When a user wants to update their email address within Paddle, the only way for them to do so is to contact Paddle customer support. When communicating with Paddle, they need to provide the `paddleEmail` value of the subscription to assist Paddle in updating the correct user.

Prices

Paddle allows you to customize prices per currency, essentially allowing you to configure different prices for different countries. Cashier Paddle allows you to retrieve all of the prices for a given product using the `productPrices` method:

```
use Laravel\Paddle\Cashier;

// Retrieve prices for two products...
$prices = Cashier::productPrices([123, 456]);
```

The currency will be determined based on the IP address of the request; however, you may optionally provide a specific country to retrieve prices for:

```
use Laravel\Paddle\Cashier;

// Retrieve prices for two products...
$prices = Cashier::productPrices([123, 456], ['customer_country' => 'BE']);
```

After retrieving the prices you may display them however you wish:

```
<ul>
    @foreach ($prices as $price)
        <li>{{ $price->product_title }} - {{ $price->price()->gross() }}</li>
    @endforeach
</ul>
```

You may also display the net price (excludes tax) and display the tax amount separately:

```
<ul>
    @foreach ($prices as $price)
        <li>{{ $price->product_title }} - {{ $price->price()->net() }} (+ {{ $price->price()->tax() }}
tax)</li>
    @endforeach
</ul>
```

If you retrieved prices for subscription plans you can display their initial and recurring price separately:

```
<ul>
    @foreach ($prices as $price)
        <li>{{ $price->product_title }} - Initial: {{ $price->initialPrice()->gross() }} - Recurring: {{
$price->recurringPrice()->gross() }}</li>
    @endforeach
</ul>
```

For more information, [check Paddle's API documentation on prices](#).

Customers

If a user is already a customer and you would like to display the prices that apply to that customer, you may do so by retrieving the prices directly from the customer instance:

```
use App\User;

// Retrieve prices for two products...
$prices = User::find(1)->productPrices([123, 456]);
```

Internally, Cashier will use the user's [paddleCountry method](#) to retrieve the prices in their currency. So, for example, a user living in the United States will see prices in USD while a user in Belgium will see prices in EUR. If no matching currency can be found the default currency of the product will be used. You can customize all prices of a product or subscription plan in the Paddle control panel.

Coupons

You may also choose to display prices after a coupon reduction. When calling the `productPrices` method, coupons may be passed as a comma delimited string:

```
use Laravel\Paddle\Cashier;
```

```
$prices = Cashier::productPrices([123, 456], ['coupons' => 'SUMMERSALE,20PERCENTOFF']);
```

Then, display the calculated prices using the `price` method:

```
<ul>
    @foreach ($prices as $price)
        <li>{{ $price->product_title }} - {{ $price->price()->gross() }}</li>
    @endforeach
</ul>
```

You may display the original listed prices (without coupon discounts) using the `listPrice` method:

```
<ul>
    @foreach ($prices as $price)
        <li>{{ $price->product_title }} - {{ $price->listPrice()->gross() }}</li>
    @endforeach
</ul>
```

NOTE When using the prices API, Paddle only allows to apply coupons to one-time purchase products and not to subscription plans.

Customers

Customer Defaults

Cashier allows you to set some useful defaults for your customer when creating pay links. Setting these defaults allow you to pre-fill a customer's email address, country, and postcode so that they can immediately move on to the payment portion of the checkout widget. You can set these defaults by overriding the following methods on your billable user:

```
/**
 * Get the customer's email address to associate with Paddle.
 *
 * @return string|null
 */
public function paddleEmail()
{
    return $this->email;
}

/**
 * Get the customer's country to associate with Paddle.
 *
 * This needs to be a 2 letter code. See the link below for supported countries.
 *
 * @return string|null
 * @link https://developer.paddle.com/reference/platform-parameters/supported-countries
 */
public function paddleCountry()
{
    //
}

/**
 * Get the customer's postcode to associate with Paddle.
 *
 * See the link below for countries which require this.
 *
 * @return string|null
 * @link https://developer.paddle.com/reference/platform-parameters/supported-countries#countries-requiring-postcode
 */
public function paddlePostcode()
{
    //
}
```

These defaults will be used for every action in Cashier that generates a [pay link](#).

Subscriptions

Creating Subscriptions

To create a subscription, first retrieve an instance of your billable model, which typically will be an instance of `App\User`. Once you have retrieved the model instance, you may use the `newSubscription` method to create the model's subscription pay link:

```
$user = User::find(1);

$payLink = $user->newSubscription('default', $premium = 12345)
    ->returnTo(route('home'))
    ->create();

return view('billing', ['payLink' => $payLink]);
```

The first argument passed to the `newSubscription` method should be the name of the subscription. If your application only offers a single subscription, you might call this `default` or `primary`. The second argument is the specific plan the user is subscribing to. This value should correspond to the plan's identifier in Paddle. The `returnTo` method accepts a URL that your user will be redirected to after they successfully complete the checkout.

The `create` method will create a pay link which you can use to generate a payment button. The payment button can be generated using the `paddle-button` Blade component that ships with Cashier Paddle:

```
<x-paddle-button :url="$payLink" class="px-8 py-4">
    Subscribe
</x-paddle-button>
```

After the user has finished their checkout, a `subscription_created` webhook will be dispatched from Paddle. Cashier will receive this webhook and setup the subscription for your customer. In order to make sure all webhooks are properly received and handled by your application, ensure you have properly [setup webhook handling](#).

Additional Details

If you would like to specify additional customer or subscription details, you may do so by passing them as a key / value array to the `create` method:

```
$payLink = $user->newSubscription('default', $monthly = 12345)
    ->returnTo(route('home'))
    ->create([
        'vat_number' => $vatNumber,
    ]);
```

To learn more about the additional fields supported by Paddle, check out Paddle's documentation on [generating pay links](#).

Coupons

If you would like to apply a coupon when creating the subscription, you may use the `withCoupon` method:

```
$payLink = $user->newSubscription('default', $monthly = 12345)
    ->returnTo(route('home'))
    ->withCoupon('code')
    ->create();
```

Metadata

You can also pass an array of metadata using the `withMetadata` method:

```
$payLink = $user->newSubscription('default', $monthly = 12345)
    ->returnTo(route('home'))
    ->withMetadata(['key' => 'value'])
    ->create();
```

NOTE When providing metadata, please avoid using `subscription_name` as a metadata key. This key is reserved for internal use by Cashier.

Checking Subscription Status

Once a user is subscribed to your application, you may check their subscription status using a variety of convenient methods. First, the `subscribed` method returns `true` if the user has an active subscription, even if the subscription is currently within its trial period:

```
if ($user->subscribed('default')) {  
    //  
}
```

The `subscribed` method also makes a great candidate for a [route middleware](#), allowing you to filter access to routes and controllers based on the user's subscription status:

```
public function handle($request, Closure $next)  
{  
    if ($request->user() && ! $request->user()->subscribed('default')) {  
        // This user is not a paying customer...  
        return redirect('billing');  
    }  
  
    return $next($request);  
}
```

If you would like to determine if a user is still within their trial period, you may use the `onTrial` method. This method can be useful for displaying a warning to the user that they are still on their trial period:

```
if ($user->subscription('default')->onTrial()) {  
    //  
}
```

The `subscribedToPlan` method may be used to determine if the user is subscribed to a given plan based on a given Paddle plan ID. In this example, we will determine if the user's `default` subscription is actively subscribed to the monthly plan:

```
if ($user->subscribedToPlan($monthly = 12345, 'default')) {  
    //  
}
```

By passing an array to the `subscribedToPlan` method, you may determine if the user's `default` subscription is actively subscribed to the monthly or the yearly plan:

```
if ($user->subscribedToPlan([$monthly = 12345, $yearly = 54321], 'default')) {  
    //  
}
```

The `recurring` method may be used to determine if the user is currently subscribed and is no longer within their trial period:

```
if ($user->subscription('default')->recurring()) {  
    //  
}
```

Cancelled Subscription Status

To determine if the user was once an active subscriber, but has cancelled their subscription, you may use the `cancelled` method:

```
if ($user->subscription('default')->cancelled()) {  
    //  
}
```

You may also determine if a user has cancelled their subscription, but are still on their "grace period" until the subscription fully expires. For example, if a user cancels a subscription on March 5th that was originally scheduled to expire on March 10th, the user is on their "grace period" until March 10th. Note that the `subscribed` method still returns `true` during this time:

```
if ($user->subscription('default')->onGracePeriod()) {  
    //  
}
```

To determine if the user has cancelled their subscription and is no longer within their "grace period", you may use the `ended` method:

```
if ($user->subscription('default')->ended()) {  
    //  
}
```

Subscription Scopes

Most subscription states are also available as query scopes so that you may easily query your database for subscriptions that are in a given state:

```
// Get all active subscriptions...  
$subscriptions = Subscription::query()->active()->get();  
  
// Get all of the cancelled subscriptions for a user...  
$subscriptions = $user->subscriptions()->cancelled()->get();
```

A complete list of available scopes is available below:

```
Subscription::query()->active();  
Subscription::query()->onTrial();  
Subscription::query()->notOnTrial();  
Subscription::query()->pastDue();  
Subscription::query()->recurring();  
Subscription::query()->ended();  
Subscription::query()->paused();  
Subscription::query()->notPaused();  
Subscription::query()->onPausedGracePeriod();  
Subscription::query()->notOnPausedGracePeriod();  
Subscription::query()->cancelled();  
Subscription::query()->notCancelled();  
Subscription::query()->onGracePeriod();  
Subscription::query()->notOnGracePeriod();
```

Past Due Status

If a payment fails for a subscription, it will be marked as `past_due`. When your subscription is in this state it will not be active until the customer has updated their payment information. You may determine if a subscription is past due using the `pastDue` method on the subscription instance:

```
if ($user->subscription('default')->pastDue()) {  
    //  
}
```

When a subscription is past due, you should instruct the user to [update their payment information](#). You may configure how past due subscriptions are handled in your [Paddle subscription settings](#).

If you would like subscriptions to still be considered active when they are `past_due`, you may use the `keepPastDueSubscriptionsActive` method provided by Cashier. Typically, this method should be called in the `register` method of your `AppServiceProvider`:

```
use Laravel\Paddle\Cashier;  
  
/**  
 * Register any application services.  
 *  
 * @return void  
 */  
public function register()  
{  
    Cashier::keepPastDueSubscriptionsActive();  
}
```

NOTE When a subscription is in a `past_due` state it cannot be changed until payment information has been updated. Therefore, the `swap` and `updateQuantity` methods will throw an exception when the subscription is in a `past_due` state.

Subscription Single Charges

Subscription single charges allow you to charge subscribers with a one-time charge on top of their subscriptions:

```
$response = $user->subscription('default')->charge(12.99, 'Support Add-on');
```

In contrast to [single charges](#), this method will immediately charge the customer's stored payment method for the subscription. The charge amount is always in the currency of which the subscription currently is set to.

Updating Payment Information

Paddle always saves a payment method per subscription. If you want to update the default payment method for a subscription, you should first generate a subscription "update URL" using the `updateUrl` method on the subscription model:

```
$user = App\User::find(1);

$updateUrl = $user->subscription('default')->updateUrl();
```

Then, you may use the generated URL in combination with Cashier's provided `paddle-button` Blade component to allow the user to initiate the Paddle widget and update their payment information:

```
<x-paddle-button :url="$updateUrl" class="px-8 py-4">
    Update Card
</x-paddle-button>
```

When a user has finished updating their information, a `subscription_updated` webhook will be dispatched by Paddle and the subscription details will be updated in your application's database.

Changing Plans

After a user has subscribed to your application, they may occasionally want to change to a new subscription plan. To swap a user to a new subscription, you should pass the Paddle plan's identifier to the subscription's `swap` method:

```
$user = App\User::find(1);

$user->subscription('default')->swap($premium = 34567);
```

If the user is on trial, the trial period will be maintained. Also, if a "quantity" exists for the subscription, that quantity will also be maintained.

If you would like to swap plans and cancel any trial period the user is currently on, you may use the `skipTrial` method:

```
$user->subscription('default')
    ->skipTrial()
    ->swap($premium = 34567);
```

If you would like to swap plans and immediately invoice the user instead of waiting for their next billing cycle, you may use the `swapAndInvoice` method:

```
$user = App\User::find(1);

$user->subscription('default')->swapAndInvoice($premium = 34567);
```

Prorations

By default, Paddle prorates charges when swapping between plans. The `noProrate` method may be used to update the subscription's without prorating the charges:

```
$user->subscription('default')->noProrate()->swap($premium = 34567);
```

Subscription Quantity

Sometimes subscriptions are affected by "quantity". For example, your application might charge \$10 per month

per user on an account. To easily increment or decrement your subscription quantity, use the `incrementQuantity` and `decrementQuantity` methods:

```
$user = User::find(1);

$user->subscription('default')->incrementQuantity();

// Add five to the subscription's current quantity...
$user->subscription('default')->incrementQuantity(5);

$user->subscription('default')->decrementQuantity();

// Subtract five to the subscription's current quantity...
$user->subscription('default')->decrementQuantity(5);
```

Alternatively, you may set a specific quantity using the `updateQuantity` method:

```
$user->subscription('default')->updateQuantity(10);
```

The `noProrate` method may be used to update the subscription's quantity without prorating the charges:

```
$user->subscription('default')->noProrate()->updateQuantity(10);
```

Pausing Subscriptions

To pause a subscription, call the `pause` method on the user's subscription:

```
$user->subscription('default')->pause();
```

When a subscription is paused, Cashier will automatically set the `paused_from` column in your database. This column is used to know when the `paused` method should begin returning `true`. For example, if a customer pauses a subscription on March 1st, but the subscription was not scheduled to recur until March 5th, the `paused` method will continue to return `false` until March 5th.

You may determine if a user has paused their subscription but are still on their "grace period" using the `onPausedGracePeriod` method:

```
if ($user->subscription('default')->onPausedGracePeriod()) {
    //
}
```

To resume a paused a subscription, you may call the `unpause` method on the user's subscription:

```
$user->subscription('default')->unpause();
```

NOTE A subscription cannot be modified while it is paused. If you want to swap to a different plan or update quantities you must resume the subscription first.

Cancelling Subscriptions

To cancel a subscription, call the `cancel` method on the user's subscription:

```
$user->subscription('default')->cancel();
```

When a subscription is cancelled, Cashier will automatically set the `ends_at` column in your database. This column is used to know when the `subscribed` method should begin returning `false`. For example, if a customer cancels a subscription on March 1st, but the subscription was not scheduled to end until March 5th, the `subscribed` method will continue to return `true` until March 5th.

You may determine if a user has cancelled their subscription but are still on their "grace period" using the `onGracePeriod` method:

```
if ($user->subscription('default')->onGracePeriod()) {
    //
}
```

If you wish to cancel a subscription immediately, you may call the `cancelNow` method on the user's subscription:

```
$user->subscription('default')->cancelNow();
```

NOTE Paddle's subscriptions cannot be resumed after cancellation. If your customer wishes to resume their subscription, they will have to subscribe to a new subscription.

Subscription Trials

With Payment Method Up Front

NOTE While trialing and collecting payment method details up front, Paddle prevents any subscription changes such as swapping plans or updating quantities. If you want to allow a customer to swap plans during a trial the subscription must be cancelled and recreated.

If you would like to offer trial periods to your customers while still collecting payment method information up front, you should use the `trialDays` method when creating your subscription pay links:

```
$user = User::find(1);

$payLink = $user->newSubscription('default', $monthly = 12345)
    ->returnTo(route('home'))
    ->trialDays(10)
    ->create();

return view('billing', ['payLink' => $payLink]);
```

This method will set the trial period ending date on the subscription record within the database, as well as instruct Paddle to not begin billing the customer until after this date.

NOTE If the customer's subscription is not cancelled before the trial ending date they will be charged as soon as the trial expires, so you should be sure to notify your users of their trial ending date.

You may determine if the user is within their trial period using either the `onTrial` method of the user instance or the `onTrial` method of the subscription instance. The two examples below have identical behavior:

```
if ($user->onTrial('default')) {
    //
}

if ($user->subscription('default')->onTrial()) {
    //
}
```

Defining Trial Days In Paddle / Cashier

You may choose to define how many trial days your plan's receive in the Paddle dashboard or always pass them explicitly using Cashier. If you choose to define your plan's trial days in Paddle you should be aware that new subscriptions, including new subscriptions for a customer that had a subscription in the past, will always receive a trial period unless you explicitly call the `trialDays(0)` method.

Without Payment Method Up Front

If you would like to offer trial periods without collecting the user's payment method information up front, you may set the `trial_ends_at` column on the customer record attached to your user to your desired trial ending date. This is typically done during user registration:

```
$user = User::create([
    // Other user properties...
]);

$user->createAsCustomer([
    'trial_ends_at' => now()->addDays(10)
]);
```

Cashier refers to this type of trial as a "generic trial", since it is not attached to any existing subscription. The `onTrial` method on the user instance will return `true` if the current date is not past the value of `trial_ends_at`:


```
if ($user->onTrial()) {  
    // User is within their trial period...  
}
```

You may also use the `onGenericTrial` method if you wish to know specifically that the user is within their "generic" trial period and has not created an actual subscription yet:

```
if ($user->onGenericTrial()) {  
    // User is within their "generic" trial period...  
}
```

Once you are ready to create an actual subscription for the user, you may use the `newSubscription` method as usual:

```
$user = User::find(1);  
  
$payLink = $user->newSubscription('default', $monthly = 12345)  
    ->returnTo(route('home'))  
    ->create();
```

NOTE There is no way to extend or modify a trial period on a Paddle subscription after it has been created.

Handling Paddle Webhooks

TIP You may use [Valet's share command](#) to help test webhooks during local development.

Paddle can notify your application of a variety of events via webhooks. By default, a route that points to Cashier's webhook controller is configured through the Cashier service provider. This controller will handle all incoming webhook requests.

By default, this controller will automatically handle cancelling subscriptions that have too many failed charges ([as defined by your Paddle subscription settings](#)), subscription updates, and payment method changes; however, as we'll soon discover, you can extend this controller to handle any webhook event you like.

To ensure your application can handle Paddle webhooks, be sure to [configure the webhook URL in the Paddle control panel](#). By default, Cashier's webhook controller listens to the `/paddle/webhook` URL path. The full list of all webhooks you should configure in the Paddle control panel are:

- Subscription Created
- Subscription Updated
- Subscription Deleted
- Payment Succeeded
- Subscription Payment Succeeded

NOTE Make sure you protect incoming requests with Cashier's included [webhook signature verification](#) middleware.

Webhooks & CSRF Protection

Since Paddle webhooks need to bypass Laravel's [CSRF protection](#), be sure to list the URI as an exception in your `VerifyCsrfToken` middleware or list the route outside of the `web` middleware group:

```
protected $except = [  
    'paddle/*',  
];
```

Defining Webhook Event Handlers

Cashier automatically handles subscription cancellation on failed charges, but if you have additional webhook events you would like to handle, you should extend the `WebhookController`. Your method names should correspond to Cashier's expected convention, specifically, methods should be prefixed with `handle` and the "camel case" name of the webhook you wish to handle. For example, if you wish to handle the `payment_succeeded` webhook, you should add a `handlePaymentSucceeded` method to the controller:

```
<?php

namespace App\Http\Controllers;

use Laravel\Paddle\Http\Controllers\WebhookController as CashierController;

class WebhookController extends CashierController
{
    /**
     * Handle payment succeeded.
     *
     * @param array $payload
     * @return void
     */
    public function handlePaymentSucceeded($payload)
    {
        // Handle The Event
    }
}
```

Next, define a route to your Cashier controller within your `routes/web.php` file. This will overwrite the route included with Cashier:

```
Route::post(
    'paddle/webhook',
    '\App\Http\Controllers\WebhookController'
);
```

Cashier emits a `Laravel\Paddle\Events\WebhookReceived` event when a webhook is received and a `Laravel\Paddle\Events\WebhookHandled` event when a webhook was handled. Both events contain the full payload of the Paddle webhook.

Cashier also emit events dedicated to the type of the received webhook. In addition to the full payload from Paddle, they also contain the relevant models that were used to process the webhook such as the billable model, the subscription, or the receipt:

- `PaymentSucceeded`
- `SubscriptionPaymentSucceeded`
- `SubscriptionCreated`
- `SubscriptionUpdated`
- `SubscriptionCancelled`

You can optionally also override the default, built-in webhook route by setting the `CASHIER_WEBHOOK` env variable in your `.env` file. This value should be the full URL to your webhook route and needs to match the URL set in your Paddle control panel:

```
CASHIER_WEBHOOK=https://example.com/my-paddle-webhook-url
```

Failed Subscriptions

What if a customer's credit card expires? No worries - Cashier's Webhook controller will cancel the customer's subscription for you. Failed payments will automatically be captured and handled by the controller. The controller will cancel the customer's subscription when Paddle determines the subscription has failed (normally after three failed payment attempts).

Verifying Webhook Signatures

To secure your webhooks, you may use [Paddle's webhook signatures](#). For convenience, Cashier automatically includes a middleware which validates that the incoming Paddle webhook request is valid.

To enable webhook verification, ensure that the `PADDLE_PUBLIC_KEY` environment variable is set in your `.env` file. The public key may be retrieved from your Paddle account dashboard.

Single Charges

Simple Charge

If you would like to make a "one off" charge against a customer, you may use the `charge` method on a billable model instance to generate a pay link for the charge. The `charge` method accepts the charge amount (float) as its first argument and a charge description as its second argument:

```
$payLink = $user->charge(12.99, 'Product Title');  
return view('pay', ['payLink' => $payLink]);
```

After generating the pay link, you may use Cashier's provided `paddle-button` Blade component to allow the user to initiate the Paddle widget and complete the charge:

```
<x-paddle-button :url="$payLink" class="px-8 py-4">  
    Buy  
</x-paddle-button>
```

The `charge` method accepts an array as its third argument, allowing you to pass any options you wish to the underlying Paddle pay link creation. Please consult [the Paddle documentation](#) to learn more about the options available to you when creating charges:

```
$payLink = $user->charge(12.99, 'Product Title', [  
    'custom_option' => $value,  
]);
```

Charges happen in the currency specified in the `cashier.currency` configuration option. By default, this is set to USD. You may override the default currency by setting the `CASHIER_CURRENCY` in your `.env` file:

```
CASHIER_CURRENCY=EUR
```

You can also [override prices per currency](#) using Paddle's dynamic pricing matching system. To do so, pass an array of prices instead of a fixed amount:

```
$payLink = $user->charge([  
    'USD:19.99',  
    'EUR:15.99',  
], 'Product Title');
```

Charging Products

If you would like to make a "one off" charge against a specific product configured within Paddle, you may use the `chargeProduct` method on a billable model instance to generate a pay link:

```
$payLink = $user->chargeProduct($productId);  
return view('pay', ['payLink' => $payLink]);
```

Then, you may provide the pay link to the `paddle-button` component to allow the user to initialize the Paddle widget:

```
<x-paddle-button :url="$payLink" class="px-8 py-4">  
    Buy  
</x-paddle-button>
```

The `chargeProduct` method accepts an array as its second argument, allowing you to pass any options you wish to the underlying Paddle pay link creation. Please consult [the Paddle documentation](#) regarding the options that are available to you when creating charges:

```
$payLink = $user->chargeProduct($productId, [  
    'custom_option' => $value,  
]);
```

Refunding Orders

If you need to refund a Paddle order, you may use the `refund` method. This method accepts the Paddle Order ID as its first argument. You may retrieve receipts for a given billable entity using the `receipts` method:

```
$receipt = $user->receipts()->first();  
$refundRequestId = $user->refund($receipt->order_id);
```

You may also optionally specify a specific amount to refund as well as a reason for the refund:

```
$receipt = $user->receipts()->first();

$refundRequestId = $user->refund(
    $receipt->order_id, 5.00, 'Unused product time'
);
```

TIP You can use the `$refundRequestId` as a reference for the refund when contacting Paddle support.

Receipts

You may easily retrieve an array of a billable model's receipts using the `receipts` method:

```
$receipts = $user->receipts();
```

When listing the receipts for the customer, you may use the receipt's helper methods to display the relevant receipt information. For example, you may wish to list every receipt in a table, allowing the user to easily download any of the receipts:

```
<table>
    @foreach ($receipts as $receipt)
        <tr>
            <td>{{ $receipt->paid_at->toFormattedDateString() }}</td>
            <td>{{ $receipt->amount() }}</td>
            <td><a href="{{ $receipt->receipt_url }}" target="_blank">Download</a></td>
        </tr>
    @endforeach
</table>
```

Past & Upcoming Payments

You may use the `lastPayment` and `nextPayment` methods to display a customer's past or upcoming payments for recurring subscriptions:

```
$subscription = $user->subscription('default');

$lastPayment = $subscription->lastPayment();
$nextPayment = $subscription->nextPayment();
```

Both of these methods will return an instance of `Laravel\Paddle\Payment`; however, `nextPayment` will return `null` when the billing cycle has ended (such as when a subscription has been cancelled):

```
Next payment: {{ $nextPayment->amount() }} due on {{ $nextPayment->date()->format('d/m/Y') }}
```

Handling Failed Payments

Subscription payments fail for various reasons, such as expired cards or a card having insufficient funds. When this happens, we recommend that you let Paddle handle payment failures for you. Specifically, you may [setup Paddle's automatic billing emails](#) in your Paddle dashboard.

Alternatively, you can perform more precise customization by catching the [subscription payment failed](#) webhook and enabling the "Subscription Payment Failed" option in the Webhook settings of your Paddle dashboard:

```
<?php

namespace App\Http\Controllers;

use Laravel\Paddle\Http\Controllers\WebhookController as CashierController;

class WebhookController extends CashierController
{
    /**
     * Handle subscription payment failed.
     *
     * @param array $payload
     * @return void
     */
}
```

```
    public function handleSubscriptionPaymentFailed($payload)
    {
        // Handle the failed subscription payment...
    }
}
```

Testing

Paddle currently lacks a proper CRUD API so you will need to manually test your billing flow. Paddle also lacks a sandboxed developer environment so any card charges you make are live charges. In order to work around this, we recommend you use coupons with a 100% discount or free products during testing.

Official Packages

Dusk

Laravel Dusk

- [Introduction](#)
- [Installation](#)
 - [Managing ChromeDriver Installations](#)
 - [Using Other Browsers](#)
- [Getting Started](#)
 - [Generating Tests](#)
 - [Running Tests](#)
 - [Environment Handling](#)
 - [Creating Browsers](#)
 - [Browser Macros](#)
 - [Authentication](#)
 - [Database Migrations](#)
 - [Cookies](#)
 - [Taking A Screenshot](#)
 - [Storing Console Output To Disk](#)
 - [Storing Page Source To Disk](#)
- [Interacting With Elements](#)
 - [Dusk Selectors](#)
 - [Clicking Links](#)
 - [Text, Values, & Attributes](#)
 - [Using Forms](#)
 - [Attaching Files](#)
 - [Using The Keyboard](#)
 - [Using The Mouse](#)
 - [JavaScript Dialogs](#)
 - [Scoping Selectors](#)
 - [Waiting For Elements](#)
 - [Scrolling An Element Into View](#)
 - [Making Vue Assertions](#)
- [Available Assertions](#)
- [Pages](#)
 - [Generating Pages](#)
 - [Configuring Pages](#)
 - [Navigating To Pages](#)
 - [Shorthand Selectors](#)
 - [Page Methods](#)
- [Components](#)
 - [Generating Components](#)
 - [Using Components](#)
- [Continuous Integration](#)
 - [CircleCI](#)
 - [Codeship](#)
 - [Heroku CI](#)
 - [Travis CI](#)
 - [GitHub Actions](#)

Introduction

Laravel Dusk provides an expressive, easy-to-use browser automation and testing API. By default, Dusk does not require you to install JDK or Selenium on your machine. Instead, Dusk uses a standalone [ChromeDriver](#) installation. However, you are free to utilize any other Selenium compatible driver you wish.

Installation

To get started, you should add the `laravel/dusk` Composer dependency to your project:

```
composer require --dev laravel/dusk
```

NOTE If you are manually registering Dusk's service provider, you should **never** register it in your production environment, as doing so could lead to arbitrary users being able to authenticate with your application.

After installing the Dusk package, run the `dusk:install` Artisan command:

```
php artisan dusk:install
```

A `Browser` directory will be created within your `tests` directory and will contain an example test. Next, set the `APP_URL` environment variable in your `.env` file. This value should match the URL you use to access your application in a browser.

To run your tests, use the `dusk` Artisan command. The `dusk` command accepts any argument that is also accepted by the `phpunit` command:

```
php artisan dusk
```

If you had test failures the last time you ran the `dusk` command, you may save time by re-running the failing tests first using the `dusk:fails` command:

```
php artisan dusk:fails
```

Managing ChromeDriver Installations

If you would like to install a different version of ChromeDriver than what is included with Laravel Dusk, you may use the `dusk:chrome-driver` command:

```
# Install the latest version of ChromeDriver for your OS...
php artisan dusk:chrome-driver
```

```
# Install a given version of ChromeDriver for your OS...
php artisan dusk:chrome-driver 74
```

```
# Install a given version of ChromeDriver for all supported OSs...
php artisan dusk:chrome-driver --all
```

NOTE Dusk requires the `chromedriver` binaries to be executable. If you're having problems running Dusk, you should ensure the binaries are executable using the following command: `chmod -R 0755 vendor/laravel/dusk/bin/`.

Using Other Browsers

By default, Dusk uses Google Chrome and a standalone [ChromeDriver](#) installation to run your browser tests. However, you may start your own Selenium server and run your tests against any browser you wish.

To get started, open your `tests/DuskTestCase.php` file, which is the base Dusk test case for your application. Within this file, you can remove the call to the `startChromeDriver` method. This will stop Dusk from automatically starting the ChromeDriver:

```
/**
 * Prepare for Dusk test execution.
 *
 * @beforeClass
 * @return void
 */
public static function prepare()
{
    // static::startChromeDriver();
}
```

Next, you may modify the `driver` method to connect to the URL and port of your choice. In addition, you may

modify the "desired capabilities" that should be passed to the WebDriver:

```
/**
 * Create the RemoteWebDriver instance.
 *
 * @return \Facebook\WebDriver\Remote\RemoteWebDriver
 */
protected function driver()
{
    return RemoteWebDriver::create(
        'http://localhost:4444/wd/hub', DesiredCapabilities::phantomjs()
    );
}
```

Getting Started

Generating Tests

To generate a Dusk test, use the `dusk:make` Artisan command. The generated test will be placed in the `tests/Browser` directory:

```
php artisan dusk:make LoginTest
```

Running Tests

To run your browser tests, use the `dusk` Artisan command:

```
php artisan dusk
```

If you had test failures the last time you ran the `dusk` command, you may save time by re-running the failing tests first using the `dusk:fails` command:

```
php artisan dusk:fails
```

The `dusk` command accepts any argument that is normally accepted by the PHPUnit test runner, allowing you to only run the tests for a given [group](#), etc:

```
php artisan dusk --group=foo
```

Manually Starting ChromeDriver

By default, Dusk will automatically attempt to start ChromeDriver. If this does not work for your particular system, you may manually start ChromeDriver before running the `dusk` command. If you choose to start ChromeDriver manually, you should comment out the following line of your `tests/DuskTestCase.php` file:

```
/**
 * Prepare for Dusk test execution.
 *
 * @beforeClass
 * @return void
 */
public static function prepare()
{
    // static::startChromeDriver();
}
```

In addition, if you start ChromeDriver on a port other than 9515, you should modify the `driver` method of the same class:

```
/**
 * Create the RemoteWebDriver instance.
 *
 * @return \Facebook\WebDriver\Remote\RemoteWebDriver
 */
protected function driver()
{
    return RemoteWebDriver::create(
        'http://localhost:9515', DesiredCapabilities::chrome()
    );
}
```



```
}
```

Environment Handling

To force Dusk to use its own environment file when running tests, create a `.env.dusk.{environment}` file in the root of your project. For example, if you will be initiating the `dusk` command from your `local` environment, you should create a `.env.dusk.local` file.

When running tests, Dusk will back-up your `.env` file and rename your Dusk environment to `.env`. Once the tests have completed, your `.env` file will be restored.

Creating Browsers

To get started, let's write a test that verifies we can log into our application. After generating a test, we can modify it to navigate to the login page, enter some credentials, and click the "Login" button. To create a browser instance, call the `browse` method:

```
<?php

namespace Tests\Browser;

use App\User;
use Illuminate\Foundation\Testing\DatabaseMigrations;
use Laravel\Dusk\Chrome;
use Tests\DuskTestCase;

class ExampleTest extends DuskTestCase
{
    use DatabaseMigrations;

    /**
     * A basic browser test example.
     *
     * @return void
     */
    public function testBasicExample()
    {
        $user = factory(User::class)->create([
            'email' => 'taylor@laravel.com',
        ]);

        $this->browse(function ($browser) use ($user) {
            $browser->visit('/login')
                ->type('email', $user->email)
                ->type('password', 'password')
                ->press('Login')
                ->assertPathIs('/home');
        });
    }
}
```

As you can see in the example above, the `browse` method accepts a callback. A browser instance will automatically be passed to this callback by Dusk and is the main object used to interact with and make assertions against your application.

Creating Multiple Browsers

Sometimes you may need multiple browsers in order to properly carry out a test. For example, multiple browsers may be needed to test a chat screen that interacts with websockets. To create multiple browsers, "ask" for more than one browser in the signature of the callback given to the `browse` method:

```
$this->browse(function ($first, $second) {
    $first->loginAs(User::find(1))
        ->visit('/home')
        ->waitForText('Message');

    $second->loginAs(User::find(2))
        ->visit('/home')
        ->waitForText('Message')
        ->type('message', 'Hey Taylor')
        ->press('Send');
```

```
$first->waitForText('Hey Taylor')
    ->assertSee('Jeffrey Way');
});
```

Resizing Browser Windows

You may use the `resize` method to adjust the size of the browser window:

```
$browser->resize(1920, 1080);
```

The `maximize` method may be used to maximize the browser window:

```
$browser->maximize();
```

The `fitContent` method will resize the browser window to match the size of the content:

```
$browser->fitContent();
```

When a test fails, Dusk will automatically resize the browser to fit the content prior to taking a screenshot. You may disable this feature by calling the `disableFitOnFailure` method within your test:

```
$browser->disableFitOnFailure();
```

You may use the `move` method to move the browser window to a different position on your screen:

```
$browser->move(100, 100);
```

Browser Macros

If you would like to define a custom browser method that you can re-use in a variety of your tests, you may use the `macro` method on the `Browser` class. Typically, you should call this method from a [service provider's](#) `boot` method:

```
<?php

namespace App\Providers;

use Illuminate\Support\ServiceProvider;
use Laravel\Dusk\Browser;

class DuskServiceProvider extends ServiceProvider
{
    /**
     * Register the Dusk's browser macros.
     *
     * @return void
     */
    public function boot()
    {
        Browser::macro('scrollToElement', function ($element = null) {
            $this->script("$('#html, body').animate({ scrollTop: $('#$element').offset().top }, 0);");

            return $this;
        });
    }
}
```

The `macro` function accepts a name as its first argument, and a Closure as its second. The macro's Closure will be executed when calling the macro as a method on a `Browser` implementation:

```
$this->browse(function ($browser) use ($user) {
    $browser->visit('/pay')
        ->scrollToElement('#credit-card-details')
        ->assertSee('Enter Credit Card Details');
});
```

Authentication

Often, you will be testing pages that require authentication. You can use Dusk's `loginAs` method in order to

avoid interacting with the login screen during every test. The `loginAs` method accepts a user ID or user model instance:

```
$this->browse(function ($first, $second) {  
    $first->loginAs(User::find(1))  
        ->visit('/home');  
});
```

NOTE After using the `loginAs` method, the user session will be maintained for all tests within the file.

Database Migrations

When your test requires migrations, like the authentication example above, you should never use the `RefreshDatabase` trait. The `RefreshDatabase` trait leverages database transactions which will not be applicable across HTTP requests. Instead, use the `DatabaseMigrations` trait:

```
<?php  
  
namespace Tests\Browser;  
  
use App\User;  
use Illuminate\Foundation\Testing\DatabaseMigrations;  
use Laravel\Dusk\Chrome;  
use Tests\DuskTestCase;  
  
class ExampleTest extends DuskTestCase  
{  
    use DatabaseMigrations;  
}
```

Cookies

You may use the `cookie` method to get or set an encrypted cookie's value:

```
$browser->cookie('name');  
  
$browser->cookie('name', 'Taylor');
```

You may use the `plainCookie` method to get or set an unencrypted cookie's value:

```
$browser->plainCookie('name');  
  
$browser->plainCookie('name', 'Taylor');
```

You may use the `deleteCookie` method to delete the given cookie:

```
$browser->deleteCookie('name');
```

Taking A Screenshot

You may use the `screenshot` method to take a screenshot and store it with the given filename. All screenshots will be stored within the `tests/Browser/screenshots` directory:

```
$browser->screenshot('filename');
```

Storing Console Output To Disk

You may use the `storeConsoleLog` method to write the console output to disk with the given filename. Console output will be stored within the `tests/Browser/console` directory:

```
$browser->storeConsoleLog('filename');
```

Storing Page Source To Disk

You may use the `storeSource` method to write the page's current source to disk with the given filename. The page source will be stored within the `tests/Browser/source` directory:

```
$browser->storeSource('filename');
```

Interacting With Elements

Dusk Selectors

Choosing good CSS selectors for interacting with elements is one of the hardest parts of writing Dusk tests. Over time, frontend changes can cause CSS selectors like the following to break your tests:

```
// HTML...

<button>Login</button>

// Test...

$browser->click('.login-page .container div > button');
```

Dusk selectors allow you to focus on writing effective tests rather than remembering CSS selectors. To define a selector, add a dusk attribute to your HTML element. Then, prefix the selector with @ to manipulate the attached element within a Dusk test:

```
// HTML...

<button dusk="login-button">Login</button>

// Test...

$browser->click('@login-button');
```

Clicking Links

To click a link, you may use the `clickLink` method on the browser instance. The `clickLink` method will click the link that has the given display text:

```
$browser->clickLink($linkText);
```

You may use the `seeLink` method to determine if a link that has the given display text is visible on the page:

```
if ($browser->seeLink($linkText)) {
    // ...
}
```

NOTE These methods interact with jQuery. If jQuery is not available on the page, Dusk will automatically inject it into the page so it is available for the test's duration.

Text, Values, & Attributes

Retrieving & Setting Values

Dusk provides several methods for interacting with the current display text, value, and attributes of elements on the page. For example, to get the "value" of an element that matches a given selector, use the `value` method:

```
// Retrieve the value...
$value = $browser->value('selector');

// Set the value...
$browser->value('selector', 'value');
```

You may use the `inputValue` method to get the "value" of an input element that has a given field name:

```
// Retrieve the value of an input element...
$inputValue = $browser->inputValue('field');
```

Retrieving Text

The `text` method may be used to retrieve the display text of an element that matches the given selector:

```
$text = $browser->text('selector');
```

Retrieving Attributes

Finally, the `attribute` method may be used to retrieve an attribute of an element matching the given selector:

```
$attribute = $browser->attribute('selector', 'value');
```

Using Forms

Typing Values

Dusk provides a variety of methods for interacting with forms and input elements. First, let's take a look at an example of typing text into an input field:

```
$browser->type('email', 'taylor@laravel.com');
```

Note that, although the method accepts one if necessary, we are not required to pass a CSS selector into the `type` method. If a CSS selector is not provided, Dusk will search for an input field with the given `name` attribute. Finally, Dusk will attempt to find a `textarea` with the given `name` attribute.

To append text to a field without clearing its content, you may use the `append` method:

```
$browser->type('tags', 'foo')
    ->append('tags', ', bar, baz');
```

You may clear the value of an input using the `clear` method:

```
$browser->clear('email');
```

You can instruct Dusk to type slowly using the `typeSlowly` method. By default, Dusk will pause for 100 milliseconds between key presses. To customize the amount of time between key presses, you may pass the appropriate number of milliseconds as the third argument to the method:

```
$browser->typeSlowly('mobile', '+1 (202) 555-5555');
$browser->typeSlowly('mobile', '+1 (202) 555-5555', 300);
```

You may use the `appendSlowly` method to append text slowly:

```
$browser->type('tags', 'foo')
    ->appendSlowly('tags', ', bar, baz');
```

Dropdowns

To select a value in a dropdown selection box, you may use the `select` method. Like the `type` method, the `select` method does not require a full CSS selector. When passing a value to the `select` method, you should pass the underlying option value instead of the display text:

```
$browser->select('size', 'Large');
```

You may select a random option by omitting the second parameter:

```
$browser->select('size');
```

Checkboxes

To "check" a checkbox field, you may use the `check` method. Like many other input related methods, a full CSS selector is not required. If an exact selector match can't be found, Dusk will search for a checkbox with a matching `name` attribute:

```
$browser->check('terms');
$browser->unchecked('terms');
```

Radio Buttons

To "select" a radio button option, you may use the `radio` method. Like many other input related methods, a full CSS selector is not required. If an exact selector match can't be found, Dusk will search for a radio with matching `name` and `value` attributes:

```
$browser->radio('version', 'php7');
```

Attaching Files

The `attach` method may be used to attach a file to a `file` input element. Like many other input related methods, a full CSS selector is not required. If an exact selector match can't be found, Dusk will search for a file input with matching `name` attribute:

```
$browser->attach('photo', __DIR__.'/photos/me.png');
```

NOTE The `attach` function requires the `zip` PHP extension to be installed and enabled on your server.

Using The Keyboard

The `keys` method allows you to provide more complex input sequences to a given element than normally allowed by the `type` method. For example, you may hold modifier keys entering values. In this example, the `shift` key will be held while `taylor` is entered into the element matching the given selector. After `taylor` is typed, `otwell` will be typed without any modifier keys:

```
$browser->keys('selector', ['{shift}', 'taylor'], 'otwell');
```

You may even send a "hot key" to the primary CSS selector that contains your application:

```
$browser->keys('.app', [{command}, 'j']);
```

TIP All modifier keys are wrapped in `{}` characters, and match the constants defined in the `Facebook\WebDriver\WebDriverKeys` class, which can be [found on GitHub](#).

Using The Mouse

Clicking On Elements

The `click` method may be used to "click" on an element matching the given selector:

```
$browser->click('.selector');
```

The `clickAtXPath` method may be used to "click" on an element matching the given XPath expression:

```
$browser->clickAtXPath('//div[@class = "selector"]');
```

The `clickAtPoint` method may be used to "click" on the topmost element at a given pair of coordinates relative to the viewable area of the browser:

```
$browser->clickAtPoint(0, 0);
```

The `doubleClick` method may be used to simulate the double "click" of a mouse:

```
$browser->doubleClick();
```

The `rightClick` method may be used to simulate the right "click" of a mouse:

```
$browser->rightClick();
```

```
$browser->rightClick('.selector');
```

The `clickAndHold` method may be used to simulate a mouse button being clicked and held down. A subsequent call to the `releaseMouse` method will undo this behavior and release the mouse button:

```
$browser->clickAndHold()
```

```
->pause(1000)
->releaseMouse();
```

Mouseover

The `mouseover` method may be used when you need to move the mouse over an element matching the given selector:

```
$browser->mouseover('.selector');
```

Drag & Drop

The `drag` method may be used to drag an element matching the given selector to another element:

```
$browser->drag('from-selector', 'to-selector');
```

Or, you may drag an element in a single direction:

```
$browser->dragLeft('.selector', 10);
$browser->dragRight('.selector', 10);
$browser->dragUp('.selector', 10);
$browser->dragDown('.selector', 10);
```

Finally, you may drag an element by a given offset:

```
$browser->dragOffset('.selector', 10, 10);
```

JavaScript Dialogs

Dusk provides various methods to interact with JavaScript Dialogs:

```
// Wait for a dialog to appear:
$browser->waitForDialog($seconds = null);

// Assert that a dialog has been displayed and that its message matches the given value:
$browser->assertDialogOpened('value');

// Type the given value in an open JavaScript prompt dialog:
$browser->typeInDialog('Hello World');
```

To close an opened JavaScript Dialog, clicking the OK button:

```
$browser->acceptDialog();
```

To close an opened JavaScript Dialog, clicking the Cancel button (for a confirmation dialog only):

```
$browser->dismissDialog();
```

Scoping Selectors

Sometimes you may wish to perform several operations while scoping all of the operations within a given selector. For example, you may wish to assert that some text exists only within a table and then click a button within that table. You may use the `with` method to accomplish this. All operations performed within the callback given to the `with` method will be scoped to the original selector:

```
$browser->with('.table', function ($table) {
    $table->assertSee('Hello World')
    ->clickLink('Delete');
});
```

You may occasionally need to execute assertions outside of the current scope. You may use the `elsewhere` method to accomplish this:

```
$browser->with('.table', function ($table) {
    // Current scope is `body .table`...
    $browser->elsewhere('.page-title', function ($title) {
        // Current scope is `body .page-title`...
        $title->assertSee('Hello World');
    });
});
```

```
});  
});
```

Waiting For Elements

When testing applications that use JavaScript extensively, it often becomes necessary to "wait" for certain elements or data to be available before proceeding with a test. Dusk makes this a cinch. Using a variety of methods, you may wait for elements to be visible on the page or even wait until a given JavaScript expression evaluates to true.

Waiting

If you need to pause the test for a given number of milliseconds, use the `pause` method:

```
$browser->pause(1000);
```

Waiting For Selectors

The `waitFor` method may be used to pause the execution of the test until the element matching the given CSS selector is displayed on the page. By default, this will pause the test for a maximum of five seconds before throwing an exception. If necessary, you may pass a custom timeout threshold as the second argument to the method:

```
// Wait a maximum of five seconds for the selector...  
$browser->waitFor('.selector');  
  
// Wait a maximum of one second for the selector...  
$browser->waitFor('.selector', 1);
```

You may also wait until the given selector is missing from the page:

```
$browser->waitUntilMissing('.selector');  
  
$browser->waitUntilMissing('.selector', 1);
```

Scoping Selectors When Available

Occasionally, you may wish to wait for a given selector and then interact with the element matching the selector. For example, you may wish to wait until a modal window is available and then press the "OK" button within the modal. The `whenAvailable` method may be used in this case. All element operations performed within the given callback will be scoped to the original selector:

```
$browser->whenAvailable('.modal', function ($modal) {  
    $modal->assertSee('Hello World')  
        ->press('OK');  
});
```

Waiting For Text

The `waitForText` method may be used to wait until the given text is displayed on the page:

```
// Wait a maximum of five seconds for the text...  
$browser->waitForText('Hello World');  
  
// Wait a maximum of one second for the text...  
$browser->waitForText('Hello World', 1);
```

You may use the `waitUntilMissingText` method to wait until the displayed text has been removed from the page:

```
// Wait a maximum of five seconds for the text to be removed...  
$browser->waitUntilMissingText('Hello World');  
  
// Wait a maximum of one second for the text to be removed...  
$browser->waitUntilMissingText('Hello World', 1);
```

Waiting For Links

The `waitForLink` method may be used to wait until the given link text is displayed on the page:

```
// Wait a maximum of five seconds for the link...
$browser->waitForLink('Create');

// Wait a maximum of one second for the link...
$browser->waitForLink('Create', 1);
```

Waiting On The Page Location

When making a path assertion such as `$browser->assertPathIs('/home')`, the assertion can fail if `window.location.pathname` is being updated asynchronously. You may use the `waitForLocation` method to wait for the location to be a given value:

```
$browser->waitForLocation('/secret');
```

You may also wait for a named route's location:

```
$browser->waitForRoute($routeName, $parameters);
```

Waiting for Page Reloads

If you need to make assertions after a page has been reloaded, use the `waitForReload` method:

```
$browser->click('.some-action')
    ->waitForReload()
    ->assertSee('something');
```

Waiting On JavaScript Expressions

Sometimes you may wish to pause the execution of a test until a given JavaScript expression evaluates to `true`. You may easily accomplish this using the `waitUntil` method. When passing an expression to this method, you do not need to include the `return` keyword or an ending semi-colon:

```
// Wait a maximum of five seconds for the expression to be true...
$browser->waitUntil('App.dataLoaded');

$browser->waitUntil('App.data.servers.length > 0');

// Wait a maximum of one second for the expression to be true...
$browser->waitUntil('App.data.servers.length > 0', 1);
```

Waiting On Vue Expressions

The following methods may be used to wait until a given Vue component attribute has a given value:

```
// Wait until the component attribute contains the given value...
$browser->waitUntilVue('user.name', 'Taylor', '@user');

// Wait until the component attribute doesn't contain the given value...
$browser->waitUntilVueIsNot('user.name', null, '@user');
```

Waiting With A Callback

Many of the "wait" methods in Dusk rely on the underlying `waitUsing` method. You may use this method directly to wait for a given callback to return `true`. The `waitUsing` method accepts the maximum number of seconds to wait, the interval at which the Closure should be evaluated, the Closure, and an optional failure message:

```
$browser->waitUsing(10, 1, function () use ($something) {
    return $something->isReady();
}, "Something wasn't ready in time.");
```

Scrolling An Element Into View

Sometimes you may not be able to click on an element because it is outside of the viewable area of the browser.

The `scrollIntoView` method will scroll the browser window until the element at the given selector is within the view:

```
$browser->scrollIntoView('selector')
    ->click('selector');
```

Making Vue Assertions

Dusk even allows you to make assertions on the state of [Vue](#) component data. For example, imagine your application contains the following Vue component:

```
// HTML...

<profile dusk="profile-component"></profile>

// Component Definition...

Vue.component('profile', {
    template: '<div>{{ user.name }}</div>',

    data: function () {
        return {
            user: {
                name: 'Taylor'
            }
        };
    }
});
```

You may assert on the state of the Vue component like so:

```
/**
 * A basic Vue test example.
 *
 * @return void
 */
public function testVue()
{
    $this->browse(function (Browser $browser) {
        $browser->visit('/')
            ->assertVue('user.name', 'Taylor', '@profile-component');
    });
}
```

Available Assertions

Dusk provides a variety of assertions that you may make against your application. All of the available assertions are documented in the list below:

[assertTitle](#) [assertTitleContains](#)
[assertUrlIs](#) [assertSchemeIs](#)
[assertSchemeIsNot](#) [assertHostIs](#)
[assertHostIsNot](#) [assertPortIs](#)
[assertPortIsNot](#)
[assertPathBeginsWith](#)
[assertPathIs](#) [assertPathIsNot](#)
[assertRouteIs](#)
[assertQueryStringHas](#)
[assertQueryStringMissing](#)
[assertFragmentIs](#)
[assertFragmentBeginsWith](#)
[assertFragmentIsNot](#)
[assertHasCookie](#)
[assertHasPlainCookie](#)
[assertCookieMissing](#)

[assertPlainCookieMissing](#)
[assertCookieValue](#)
[assertPlainCookieValue](#) [assertSee](#)
[assertDontSee](#) [assertSeeIn](#)
[assertDontSeeIn](#) [assertSourceHas](#)
[assertSourceMissing](#)
[assertSeeLink](#) [assertDontSeeLink](#)
[assertInputValue](#)
[assertInputValueIsNot](#)
[assertChecked](#) [assertNotChecked](#)
[assertRadioSelected](#)
[assertRadioNotSelected](#)
[assertSelected](#) [assertNotSelected](#)
[assertSelectHasOptions](#)
[assertSelectMissingOption](#)
[assertSelectMissingOptions](#)

[assertSelectHasOption](#)
[assertValue](#) [assertAttribute](#)
[assertAriaAttribute](#)
[assertDataAttribute](#) [assertVisible](#)
[assertPresent](#) [assertMissing](#)
[assertDialogOpened](#)
[assertEnabled](#) [assertDisabled](#)
[assertButtonEnabled](#)
[assertButtonDisabled](#)
[assertFocused](#) [assertNotFocused](#)
[assertAuthenticated](#) [assertGuest](#)
[assertAuthenticatedAs](#) [assertVue](#)
[assertVueIsNot](#)
[assertVueContains](#)
[assertVueDoesNotContain](#)

assertTitle

Assert that the page title matches the given text:

```
$browser->assertTitle($title);
```

assertTitleContains

Assert that the page title contains the given text:

```
$browser->assertTitleContains($title);
```

assertUrlIs

Assert that the current URL (without the query string) matches the given string:

```
$browser->assertUrlIs($url);
```

assertSchemeIs

Assert that the current URL scheme matches the given scheme:

```
$browser->assertSchemeIs($scheme);
```

assertSchemeIsNot

Assert that the current URL scheme does not match the given scheme:

```
$browser->assertSchemeIsNot($scheme);
```

assertHostIs

Assert that the current URL host matches the given host:

```
$browser->assertHostIs($host);
```

assertHostIsNot

Assert that the current URL host does not match the given host:

```
$browser->assertHostIsNot($host);
```

assertPortIs

Assert that the current URL port matches the given port:

```
$browser->assertPortIs($port);
```

assertPortIsNot

Assert that the current URL port does not match the given port:

```
$browser->assertPortIsNot($port);
```

assertPathBeginsWith

Assert that the current URL path begins with the given path:

```
$browser->assertPathBeginsWith($path);
```

assertPathIs

Assert that the current path matches the given path:

```
$browser->assertPathIs('/home');
```

assertPathIsNot

Assert that the current path does not match the given path:

```
$browser->assertPathIsNot('/home');
```

assertRouteIs

Assert that the current URL matches the given named route's URL:

```
$browser->assertRouteIs($name, $parameters);
```

assertQueryStringHas

Assert that the given query string parameter is present:

```
$browser->assertQueryStringHas($name);
```

Assert that the given query string parameter is present and has a given value:

```
$browser->assertQueryStringHas($name, $value);
```

assertQueryStringMissing

Assert that the given query string parameter is missing:

```
$browser->assertQueryStringMissing($name);
```

assertFragmentIs

Assert that the current fragment matches the given fragment:

```
$browser->assertFragmentIs('anchor');
```

assertFragmentBeginsWith

Assert that the current fragment begins with the given fragment:

```
$browser->assertFragmentBeginsWith('anchor');
```

assertFragmentIsNot

Assert that the current fragment does not match the given fragment:

```
$browser->assertFragmentIsNot('anchor');
```

assertHasCookie

Assert that the given encrypted cookie is present:

```
$browser->assertHasCookie($name);
```

assertHasPlainCookie

Assert that the given unencrypted cookie is present:

```
$browser->assertHasPlainCookie($name);
```

assertCookieMissing

Assert that the given encrypted cookie is not present:

```
$browser->assertCookieMissing($name);
```

assertPlainCookieMissing

Assert that the given unencrypted cookie is not present:

```
$browser->assertPlainCookieMissing($name);
```

assertCookieValue

Assert that an encrypted cookie has a given value:

```
$browser->assertCookieValue($name, $value);
```

assertPlainCookieValue

Assert that an unencrypted cookie has a given value:

```
$browser->assertPlainCookieValue($name, $value);
```

assertSee

Assert that the given text is present on the page:

```
$browser->assertSee($text);
```

assertDontSee

Assert that the given text is not present on the page:

```
$browser->assertDontSee($text);
```

assertSeeIn

Assert that the given text is present within the selector:

```
$browser->assertSeeIn($selector, $text);
```

assertDontSeeIn

Assert that the given text is not present within the selector:

```
$browser->assertDontSeeIn($selector, $text);
```

assertSourceHas

Assert that the given source code is present on the page:

```
$browser->assertSourceHas($code);
```

assertSourceMissing

Assert that the given source code is not present on the page:

```
$browser->assertSourceMissing($code);
```

assertSeeLink

Assert that the given link is present on the page:

```
$browser->assertSeeLink($linkText);
```

assertDontSeeLink

Assert that the given link is not present on the page:

```
$browser->assertDontSeeLink($linkText);
```

assertInputValue

Assert that the given input field has the given value:

```
$browser->assertInputValue($field, $value);
```

assertInputValueIsNot

Assert that the given input field does not have the given value:

```
$browser->assertInputValueIsNot($field, $value);
```

assertChecked

Assert that the given checkbox is checked:

```
$browser->assertChecked($field);
```

assertNotChecked

Assert that the given checkbox is not checked:

```
$browser->assertNotChecked($field);
```

assertRadioSelected

Assert that the given radio field is selected:

```
$browser->assertRadioSelected($field, $value);
```

assertRadioNotSelected

Assert that the given radio field is not selected:

```
$browser->assertRadioNotSelected($field, $value);
```

assertSelected

Assert that the given dropdown has the given value selected:

```
$browser->assertSelected($field, $value);
```

assertNotSelected

Assert that the given dropdown does not have the given value selected:

```
$browser->assertNotSelected($field, $value);
```

assertSelectHasOptions

Assert that the given array of values are available to be selected:

```
$browser->assertSelectHasOptions($field, $values);
```

assertSelectMissingOption

Assert that the given value is not available to be selected:

```
$browser->assertSelectMissingOption($field, $value);
```

assertSelectMissingOptions

Assert that the given array of values are not available to be selected:

```
$browser->assertSelectMissingOptions($field, $values);
```

assertSelectHasOption

Assert that the given value is available to be selected on the given field:

```
$browser->assertSelectHasOption($field, $value);
```

assertValue

Assert that the element matching the given selector has the given value:

```
$browser->assertValue($selector, $value);
```

assertAttribute

Assert that the element matching the given selector has the given value in the provided attribute:

```
$browser->assertAttribute($selector, $attribute, $value);
```

assertAriaAttribute

Assert that the element matching the given selector has the given value in the provided aria attribute:

```
$browser->assertAriaAttribute($selector, $attribute, $value);
```

For example, given the markup `<button aria-label="Add"></button>`, you may assert against the `aria-label` attribute like so:

```
$browser->assertAriaAttribute('button', 'label', 'Add')
```

assertDataAttribute

Assert that the element matching the given selector has the given value in the provided data attribute:

```
$browser->assertDataAttribute($selector, $attribute, $value);
```

For example, given the markup `<tr id="row-1" data-content="attendees"></tr>`, you may assert against the `data-label` attribute like so:

```
$browser->assertDataAttribute('#row-1', 'content', 'attendees')
```

assertVisible

Assert that the element matching the given selector is visible:

```
$browser->assertVisible($selector);
```

assertPresent

Assert that the element matching the given selector is present:

```
$browser->assertPresent($selector);
```

assertMissing

Assert that the element matching the given selector is not visible:

```
$browser->assertMissing($selector);
```

assertDialogOpened

Assert that a JavaScript dialog with the given message has been opened:

```
$browser->assertDialogOpened($message);
```

assertEnabled

Assert that the given field is enabled:

```
$browser->assertEnabled($field);
```

assertDisabled

Assert that the given field is disabled:

```
$browser->assertDisabled($field);
```

assertButtonEnabled

Assert that the given button is enabled:

```
$browser->assertButtonEnabled($button);
```

assertButtonDisabled

Assert that the given button is disabled:

```
$browser->assertButtonDisabled($button);
```

assertFocused

Assert that the given field is focused:

```
$browser->assertFocused($field);
```

assertNotFocused

Assert that the given field is not focused:

```
$browser->assertNotFocused($field);
```

assertAuthenticated

Assert that the user is authenticated:

```
$browser->assertAuthenticated();
```

assertGuest

Assert that the user is not authenticated:

```
$browser->assertGuest();
```

assertAuthenticatedAs

Assert that the user is authenticated as the given user:

```
$browser->assertAuthenticatedAs($user);
```

assertVue

Assert that a given Vue component data property matches the given value:

```
$browser->assertVue($property, $value, $componentSelector = null);
```

assertVueIsNot

Assert that a given Vue component data property does not match the given value:

```
$browser->assertVueIsNot($property, $value, $componentSelector = null);
```

assertVueContains

Assert that a given Vue component data property is an array and contains the given value:

```
$browser->assertVueContains($property, $value, $componentSelector = null);
```

assertVueDoesNotContain

Assert that a given Vue component data property is an array and does not contain the given value:

```
$browser->assertVueDoesNotContain($property, $value, $componentSelector = null);
```

Pages

Sometimes, tests require several complicated actions to be performed in sequence. This can make your tests harder to read and understand. Pages allow you to define expressive actions that may then be performed on a given page using a single method. Pages also allow you to define short-cuts to common selectors for your application or a single page.

Generating Pages

To generate a page object, use the `dusk:page` Artisan command. All page objects will be placed in the `tests/Browser/Pages` directory:

```
php artisan dusk:page Login
```

Configuring Pages

By default, pages have three methods: `url`, `assert`, and `elements`. We will discuss the `url` and `assert` methods now. The `elements` method will be [discussed in more detail below](#).

The url Method

The `url` method should return the path of the URL that represents the page. Dusk will use this URL when navigating to the page in the browser:

```
/**
 * Get the URL for the page.
 *
 * @return string
 */
public function url()
{
    return '/login';
}
```

The assert Method

The `assert` method may make any assertions necessary to verify that the browser is actually on the given page. Completing this method is not necessary; however, you are free to make these assertions if you wish. These assertions will be run automatically when navigating to the page:

```
/**
 * Assert that the browser is on the page.
 *
 * @return void
 */
public function assert(Browser $browser)
{
    $browser->assertPathIs($this->url());
}
```

Navigating To Pages

Once a page has been configured, you may navigate to it using the `visit` method:

```
use Tests\Browser\Pages\Login;

$browser->visit(new Login);
```

You may use the `visitRoute` method to navigate to a named route:

```
$browser->visitRoute('login');
```

You may navigate "back" and "forward" using the `back` and `forward` methods:

```
$browser->back();

$browser->forward();
```

You may use the `refresh` method to refresh the page:

```
$browser->refresh();
```

Sometimes you may already be on a given page and need to "load" the page's selectors and methods into the current test context. This is common when pressing a button and being redirected to a given page without explicitly navigating to it. In this situation, you may use the `on` method to load the page:

```
use Tests\Browser\Pages\CreatePlaylist;

$browser->visit('/dashboard')
    ->clickLink('Create Playlist')
    ->on(new CreatePlaylist)
    ->assertSee('@create');
```

Shorthand Selectors

The `elements` method of pages allows you to define quick, easy-to-remember shortcuts for any CSS selector on your page. For example, let's define a shortcut for the "email" input field of the application's login page:

```
/**
 * Get the element shortcuts for the page.
 *
 * @return array
 */
public function elements()
{
    return [
        '@email' => 'input[name=email]',
    ];
}
```

Now, you may use this shorthand selector anywhere you would use a full CSS selector:

```
$browser->type('@email', 'taylor@laravel.com');
```

Global Shorthand Selectors

After installing Dusk, a base `Page` class will be placed in your `tests/Browser/Pages` directory. This class contains a `siteElements` method which may be used to define global shorthand selectors that should be available on every page throughout your application:

```
/**
 * Get the global element shortcuts for the site.
 *
 * @return array
 */
public static function siteElements()
{
    return [
        '@element' => '#selector',
    ];
}
```

Page Methods

In addition to the default methods defined on pages, you may define additional methods which may be used throughout your tests. For example, let's imagine we are building a music management application. A common action for one page of the application might be to create a playlist. Instead of re-writing the logic to create a playlist in each test, you may define a `createPlaylist` method on a page class:

```
<?php

namespace Tests\Browser\Pages;

use Laravel\Dusk\Browser;

class Dashboard extends Page
{
    // Other page methods...

    /**
     * Create a new playlist.
     *
     * @param \Laravel\Dusk\Browser $browser
     * @param string $name
     * @return void
     */
    public function createPlaylist(Browser $browser, $name)
    {
        $browser->type('name', $name)
            ->check('share')
            ->press('Create Playlist');
    }
}
```

Once the method has been defined, you may use it within any test that utilizes the page. The browser instance will automatically be passed to the page method:

```
use Tests\Browser\Pages\Dashboard;

$browser->visit(new Dashboard)
    ->createPlaylist('My Playlist')
    ->assertSee('My Playlist');
```

Components

Components are similar to Dusk's "page objects", but are intended for pieces of UI and functionality that are re-used throughout your application, such as a navigation bar or notification window. As such, components are not bound to specific URLs.

Generating Components

To generate a component, use the `dusk:component` Artisan command. New components are placed in the `tests/Browser/Components` directory:

```
php artisan dusk:component DatePicker
```

As shown above, a "date picker" is an example of a component that might exist throughout your application on a variety of pages. It can become cumbersome to manually write the browser automation logic to select a date in dozens of tests throughout your test suite. Instead, we can define a Dusk component to represent the date picker, allowing us to encapsulate that logic within the component:

```
<?php

namespace Tests\Browser\Components;

use Laravel\Dusk\Browser;
use Laravel\Dusk\Component as BaseComponent;

class DatePicker extends BaseComponent
{
    /**
     * Get the root selector for the component.
     *
     * @return string
     */
    public function selector()
    {
        return '.date-picker';
    }

    /**
     * Assert that the browser page contains the component.
     *
     * @param Browser $browser
     * @return void
     */
    public function assert(Browser $browser)
    {
        $browser->assertVisible($this->selector());
    }

    /**
     * Get the element shortcuts for the component.
     *
     * @return array
     */
    public function elements()
    {
        return [
            '@date-field' => 'input.datepicker-input',
            '@year-list' => 'div > div.datepicker-years',
            '@month-list' => 'div > div.datepicker-months',
            '@day-list' => 'div > div.datepicker-days',
        ];
    }

    /**
     * Select the given date.
     *
     * @param \Laravel\Dusk\Browser $browser
     * @param int $year
     * @param int $month
     * @param int $day
     * @return void
     */
    public function selectDate($browser, $year, $month, $day)
    {
        $browser->click('@date-field')
            ->within('@year-list', function ($browser) use ($year) {
                $browser->click($year);
            })
            ->within('@month-list', function ($browser) use ($month) {
                $browser->click($month);
            })
            ->within('@day-list', function ($browser) use ($day) {
                $browser->click($day);
            });
    }
}
```

Using Components

Once the component has been defined, we can easily select a date within the date picker from any test. And, if the logic necessary to select a date changes, we only need to update the component:

```
<?php

namespace Tests\Browser;

use Illuminate\Foundation\Testing\DatabaseMigrations;
use Laravel\Dusk\Browser;
use Tests\Browser\Components\DatePicker;
use Tests\DuskTestCase;

class ExampleTest extends DuskTestCase
{
    /**
     * A basic component test example.
     *
     * @return void
     */
    public function testBasicExample()
    {
        $this->browse(function (Browser $browser) {
            $browser->visit('/')
                ->within(new DatePicker, function ($browser) {
                    $browser->selectDate(2019, 1, 30);
                })
                ->assertSee('January');
        });
    }
}
```

Continuous Integration

NOTE Before adding a continuous integration configuration file, ensure that your `.env.testing` file contains an `APP_URL` entry with a value of `http://127.0.0.1:8000`.

CircleCI

If you are using CircleCI to run your Dusk tests, you may use this configuration file as a starting point. Like TravisCI, we will use the `php artisan serve` command to launch PHP's built-in web server:

```
version: 2
jobs:
  build:
    steps:
      - run: sudo apt-get install -y libsqlite3-dev
      - run: cp .env.testing .env
      - run: composer install -n --ignore-platform-reqs
      - run: php artisan key:generate
      - run: php artisan dusk:chrome-driver
      - run: npm install
      - run: npm run production
      - run: vendor/bin/phpunit

      - run:
          name: Start Chrome Driver
          command: ./vendor/laravel/dusk/bin/chromedriver-linux
          background: true

      - run:
          name: Run Laravel Server
          command: php artisan serve
          background: true

      - run:
          name: Run Laravel Dusk Tests
          command: php artisan dusk

      - store_artifacts:
          path: tests/Browser/screenshots

      - store_artifacts:
          path: tests/Browser/console

      - store_artifacts:
```

```
path: storage/logs
```

Codeship

To run Dusk tests on [Codeship](#), add the following commands to your Codeship project. These commands are just a starting point and you are free to add additional commands as needed:

```
phpenv local 7.2
cp .env.testing .env
mkdir -p ./bootstrap/cache
composer install --no-interaction --prefer-dist
php artisan key:generate
php artisan dusk:chrome-driver
nohup bash -c "php artisan serve 2>&1 &" && sleep 5
php artisan dusk
```

Heroku CI

To run Dusk tests on [Heroku CI](#), add the following Google Chrome buildpack and scripts to your Heroku app.json file:

```
{
  "environments": {
    "test": {
      "buildpacks": [
        { "url": "heroku/php" },
        { "url": "https://github.com/heroku/heroku-buildpack-google-chrome" }
      ],
      "scripts": {
        "test-setup": "cp .env.testing .env",
        "test": "nohup bash -c './vendor/laravel/dusk/bin/chromedriver-linux > /dev/null 2>&1 &' && nohup bash -c 'php artisan serve > /dev/null 2>&1 &' && php artisan dusk"
      }
    }
  }
}
```

Travis CI

To run your Dusk tests on [Travis CI](#), use the following .travis.yml configuration. Since Travis CI is not a graphical environment, we will need to take some extra steps in order to launch a Chrome browser. In addition, we will use php artisan serve to launch PHP's built-in web server:

```
language: php

php:
  - 7.3

addons:
  chrome: stable

install:
  - cp .env.testing .env
  - travis_retry composer install --no-interaction --prefer-dist --no-suggest
  - php artisan key:generate
  - php artisan dusk:chrome-driver

before_script:
  - google-chrome-stable --headless --disable-gpu --remote-debugging-port=9222 http://localhost &
  - php artisan serve &

script:
  - php artisan dusk
```

GitHub Actions

If you are using [Github Actions](#) to run your Dusk tests, you may use this configuration file as a starting point. Like TravisCI, we will use the php artisan serve command to launch PHP's built-in web server:

```
name: CI
on: [push]
```

jobs:

```
dusk-php:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v2
    - name: Prepare The Environment
      run: cp .env.example .env
    - name: Create Database
      run: |
        sudo systemctl start mysql
        mysql --user="root" --password="root" -e "CREATE DATABASE 'my-database' character set UTF8mb4
collate utf8mb4_bin;"
    - name: Install Composer Dependencies
      run: composer install --no-progress --no-suggest --prefer-dist --optimize-autoloader
    - name: Generate Application Key
      run: php artisan key:generate
    - name: Upgrade Chrome Driver
      run: php artisan dusk:chrome-driver ` /opt/google/chrome/chrome --version | cut -d " " -f3 | cut
-d "." -f1`
    - name: Start Chrome Driver
      run: ./vendor/laravel/dusk/bin/chromedriver-linux &
    - name: Run Laravel Server
      run: php artisan serve &
    - name: Run Dusk Tests
      env:
        APP_URL: "http://127.0.0.1:8000"
      run: php artisan dusk
```

Official Packages

Envoy

Laravel Envoy

- [Introduction](#)
 - [Installation](#)
- [Writing Tasks](#)
 - [Setup](#)
 - [Variables](#)
 - [Stories](#)
 - [Multiple Servers](#)
- [Running Tasks](#)
 - [Confirming Task Execution](#)
- [Notifications](#)
 - [Slack](#)
 - [Discord](#)
 - [Telegram](#)

Introduction

[Laravel Envoy](#) provides a clean, minimal syntax for defining common tasks you run on your remote servers. Using Blade style syntax, you can easily setup tasks for deployment, Artisan commands, and more. Currently, Envoy only supports the Mac and Linux operating systems.

Installation

First, install Envoy using the Composer `global require` command:

```
composer global require laravel/envoy
```

Since global Composer libraries can sometimes cause package version conflicts, you may wish to consider using `cgr`, which is a drop-in replacement for the `composer global require` command. The `cgr` library's installation instructions can be [found on GitHub](#).

NOTE Make sure to place the `$HOME/.config/composer/vendor/bin` OR `$HOME/.composer/vendor/bin` directory in your `PATH` so the `envoy` executable is found when running the `envoy` command in your terminal.

Updating Envoy

You may also use Composer to keep your Envoy installation up to date. Issuing the `composer global update` command will update all of your globally installed Composer packages:

```
composer global update
```

Writing Tasks

All of your Envoy tasks should be defined in an `Envoy.blade.php` file in the root of your project. Here's an example to get you started:

```
@servers(['web' => ['user@192.168.1.1']])

@task('foo', ['on' => 'web'])
    ls -la
@endtask
```

As you can see, an array of `@servers` is defined at the top of the file, allowing you to reference these servers in the `on` option of your task declarations. The `@servers` declaration should always be placed on a single line. Within your `@task` declarations, you should place the Bash code that should run on your server when the task is

executed.

You can force a script to run locally by specifying the server's IP address as `127.0.0.1`:

```
@servers(['localhost' => '127.0.0.1'])
```

Setup

Sometimes, you may need to execute some PHP code before executing your Envoy tasks. You may use the `@setup` directive to declare variables and do other general PHP work before any of your other tasks are executed:

```
@setup
    $now = new DateTime();

    $environment = isset($env) ? $env : "testing";
@endsetup
```

If you need to require other PHP files before your task is executed, you may use the `@include` directive at the top of your `Envoy.blade.php` file:

```
@include('vendor/autoload.php')

@task('foo')
    # ...
@endtask
```

You may also import other Envoy files so their stories and tasks are added to yours. After they have been imported, you may execute the tasks in those files as if they were defined in your own. You should use the `@import` directive at the top of your `Envoy.blade.php` file:

```
@import('package/Envoy.blade.php')
```

Variables

If needed, you may pass option values into Envoy tasks using the command line:

```
envoy run deploy --branch=master
```

You may access the options in your tasks via Blade's "echo" syntax. You may also use `if` statements and loops within your tasks. For example, let's verify the presence of the `$branch` variable before executing the `git pull` command:

```
@servers(['web' => '192.168.1.1'])

@task('deploy', ['on' => 'web'])
    cd site

    @if ($branch)
        git pull origin {{ $branch }}
    @endif

    php artisan migrate
@endtask
```

Stories

Stories group a set of tasks under a single, convenient name, allowing you to group small, focused tasks into large tasks. For instance, a `deploy` story may run the `git` and `composer` tasks by listing the task names within its definition:

```
@servers(['web' => '192.168.1.1'])

@story('deploy')
    git
    composer
@endstory

@task('git')
```

```
        git pull origin master
    @endtask

    @task('composer')
        composer install
    @endtask
```

Once the story has been written, you may run it just like a typical task:

```
envoy run deploy
```

Multiple Servers

Envoy allows you to easily run a task across multiple servers. First, add additional servers to your `@servers` declaration. Each server should be assigned a unique name. Once you have defined your additional servers, list each of the servers in the task's `on` array:

```
@servers(['web-1' => '192.168.1.1', 'web-2' => '192.168.1.2'])

@task('deploy', ['on' => ['web-1', 'web-2']])
    cd site
    git pull origin {{ $branch }}
    php artisan migrate
@endtask
```

Parallel Execution

By default, tasks will be executed on each server serially. In other words, a task will finish running on the first server before proceeding to execute on the second server. If you would like to run a task across multiple servers in parallel, add the `parallel` option to your task declaration:

```
@servers(['web-1' => '192.168.1.1', 'web-2' => '192.168.1.2'])

@task('deploy', ['on' => ['web-1', 'web-2'], 'parallel' => true])
    cd site
    git pull origin {{ $branch }}
    php artisan migrate
@endtask
```

Running Tasks

To run a task or story that is defined in your `Envoy.blade.php` file, execute Envoy's `run` command, passing the name of the task or story you would like to execute. Envoy will run the task and display the output from the servers as the task is running:

```
envoy run deploy
```

Confirming Task Execution

If you would like to be prompted for confirmation before running a given task on your servers, you should add the `confirm` directive to your task declaration. This option is particularly useful for destructive operations:

```
@task('deploy', ['on' => 'web', 'confirm' => true])
    cd site
    git pull origin {{ $branch }}
    php artisan migrate
@endtask
```

Notifications

Slack

Envoy also supports sending notifications to [Slack](#) after each task is executed. The `@slack` directive accepts a Slack hook URL and a channel name. You may retrieve your webhook URL by creating an "Incoming WebHooks" integration in your Slack control panel. You should pass the entire webhook URL into the `@slack`

directive:

```
@finished
    @slack('webhook-url', '#bots')
@endfinished
```

You may provide one of the following as the channel argument:

- To send the notification to a channel: #channel
- To send the notification to a user: @user

Discord

Envoy also supports sending notifications to [Discord](#) after each task is executed. The @discord directive accepts a Discord hook URL and a message. You may retrieve your webhook URL by creating a "Webhook" in your Server Settings and choosing which channel the webhook should post to. You should pass the entire Webhook URL into the @discord directive:

```
@finished
    @discord('discord-webhook-url')
@endfinished
```

Telegram

Envoy also supports sending notifications to [Telegram](#) after each task is executed. The @telegram directive accepts a Telegram Bot ID and a Chat ID. You may retrieve your Bot ID by creating a new bot using [BotFather](#). You can retrieve a valid Chat ID using [@username to id bot](#). You should pass the entire Bot ID and Chat ID into the @telegram directive:

```
@finished
    @telegram('<bot-id>', '<chat-id>')
@endfinished
```

Official Packages

Horizon

Laravel Horizon

- [Introduction](#)
- [Installation](#)
 - [Configuration](#)
 - [Dashboard Authorization](#)
- [Upgrading Horizon](#)
- [Running Horizon](#)
 - [Deploying Horizon](#)
- [Tags](#)
- [Notifications](#)
- [Metrics](#)

Introduction

Horizon provides a beautiful dashboard and code-driven configuration for your Laravel powered Redis queues. Horizon allows you to easily monitor key metrics of your queue system such as job throughput, runtime, and job failures.

All of your worker configuration is stored in a single, simple configuration file, allowing your configuration to stay in source control where your entire team can collaborate.

Installation

NOTE You should ensure that your queue connection is set to `redis` in your queue configuration file.

You may use Composer to install Horizon into your Laravel project:

```
composer require laravel/horizon
```

After installing Horizon, publish its assets using the `horizon:install` Artisan command:

```
php artisan horizon:install
```

Configuration

After publishing Horizon's assets, its primary configuration file will be located at `config/horizon.php`. This configuration file allows you to configure your worker options and each configuration option includes a description of its purpose, so be sure to thoroughly explore this file.

NOTE You should ensure that the `environments` portion of your `horizon` configuration file contains an entry for each environment on which you plan to run Horizon.

Balance Options

Horizon allows you to choose from three balancing strategies: `simple`, `auto`, and `false`. The `simple` strategy, which is the configuration file's default, splits incoming jobs evenly between processes:

```
'balance' => 'simple',
```

The `auto` strategy adjusts the number of worker processes per queue based on the current workload of the queue. For example, if your `notifications` queue has 1,000 waiting jobs while your `render` queue is empty, Horizon will allocate more workers to your `notifications` queue until it is empty. When the `balance` option is set to `false`, the default Laravel behavior will be used, which processes queues in the order they are listed in your configuration.

When using the `auto` strategy, you may define the `minProcesses` and `maxProcesses` configuration options to control the minimum and maximum number of processes Horizon should scale up and down to:

```
'environments' => [
    'production' => [
        'supervisor-1' => [
            'connection' => 'redis',
            'queue' => ['default'],
            'balance' => 'auto',
            'minProcesses' => 1,
            'maxProcesses' => 10,
            'tries' => 3,
        ],
    ],
],
```

Job Trimming

The horizon configuration file allows you to configure how long recent and failed jobs should be persisted (in minutes). By default, recent jobs are kept for one hour while failed jobs are kept for a week:

```
'trim' => [
    'recent' => 60,
    'failed' => 10080,
],
```

Dashboard Authorization

Horizon exposes a dashboard at `/horizon`. By default, you will only be able to access this dashboard in the `local` environment. Within your `app/Providers/HorizonServiceProvider.php` file, there is a `gate` method. This authorization gate controls access to Horizon in **non-local** environments. You are free to modify this gate as needed to restrict access to your Horizon installation:

```
/**
 * Register the Horizon gate.
 *
 * This gate determines who can access Horizon in non-local environments.
 *
 * @return void
 */
protected function gate()
{
    Gate::define('viewHorizon', function ($user) {
        return in_array($user->email, [
            'taylor@laravel.com',
        ]);
    });
}
```

NOTE Remember that Laravel injects the *authenticated* user to the Gate automatically. If your app is providing Horizon security via another method, such as IP restrictions, then your Horizon users may not need to "login". Therefore, you will need to change function (`$user`) above to function (`$user = null`) to force Laravel to not require authentication.

Upgrading Horizon

When upgrading to a new major version of Horizon, it's important that you carefully review [the upgrade guide](#).

In addition, when upgrading to any new Horizon version, you should re-publish Horizon's assets:

```
php artisan horizon:publish
```

To keep the assets up-to-date and avoid issues in future updates, you may add the command to the `post-update-cmd` scripts in your `composer.json` file:

```
{
    "scripts": {
        "post-update-cmd": [
            "@php artisan horizon:publish --ansi"
        ]
    }
}
```

```
}  
    }  
}
```

Running Horizon

Once you have configured your workers in the `config/horizon.php` configuration file, you may start Horizon using the `horizon` Artisan command. This single command will start all of your configured workers:

```
php artisan horizon
```

You may pause the Horizon process and instruct it to continue processing jobs using the `horizon:pause` and `horizon:continue` Artisan commands:

```
php artisan horizon:pause
```

```
php artisan horizon:continue
```

You may check the current status of the Horizon process using the `horizon:status` Artisan command:

```
php artisan horizon:status
```

You may gracefully terminate the master Horizon process on your machine using the `horizon:terminate` Artisan command. Any jobs that Horizon is currently processing will be completed and then Horizon will exit:

```
php artisan horizon:terminate
```

Deploying Horizon

If you are deploying Horizon to a live server, you should configure a process monitor to monitor the `php artisan horizon` command and restart it if it quits unexpectedly. When deploying fresh code to your server, you will need to instruct the master Horizon process to terminate so it can be restarted by your process monitor and receive your code changes.

Installing Supervisor

Supervisor is a process monitor for the Linux operating system, and will automatically restart your `horizon` process if it fails. To install Supervisor on Ubuntu, you may use the following command:

```
sudo apt-get install supervisor
```

TIP If configuring Supervisor yourself sounds overwhelming, consider using [Laravel Forge](#), which will automatically install and configure Supervisor for your Laravel projects.

Supervisor Configuration

Supervisor configuration files are typically stored in the `/etc/supervisor/conf.d` directory. Within this directory, you may create any number of configuration files that instruct supervisor how your processes should be monitored. For example, let's create a `horizon.conf` file that starts and monitors a `horizon` process:

```
[program:horizon]  
process_name=%(program_name)s  
command=php /home/forge/app.com/artisan horizon  
autostart=true  
autorestart=true  
user=forge  
redirect_stderr=true  
stdout_logfile=/home/forge/app.com/horizon.log  
stopwaitsecs=3600
```

NOTE You should ensure that the value of `stopwaitsecs` is greater than the number of seconds consumed by your longest running job. Otherwise, Supervisor may kill the job before it is finished processing.

Starting Supervisor

Once the configuration file has been created, you may update the Supervisor configuration and start the processes using the following commands:

```
sudo supervisorctl reread
sudo supervisorctl update
sudo supervisorctl start horizon
```

For more information on Supervisor, consult the [Supervisor documentation](#).

Tags

Horizon allows you to assign “tags” to jobs, including mailables, event broadcasts, notifications, and queued event listeners. In fact, Horizon will intelligently and automatically tag most jobs depending on the Eloquent models that are attached to the job. For example, take a look at the following job:

```
<?php

namespace App\Jobs;

use App\Video;
use Illuminate\Bus\Queueable;
use Illuminate\Contracts\Queue\ShouldQueue;
use Illuminate\Foundation\Bus\Dispatchable;
use Illuminate\Queue\InteractsWithQueue;
use Illuminate\Queue\SerializesModels;

class RenderVideo implements ShouldQueue
{
    use Dispatchable, InteractsWithQueue, Queueable, SerializesModels;

    /**
     * The video instance.
     *
     * @var \App\Video
     */
    public $video;

    /**
     * Create a new job instance.
     *
     * @param \App\Video $video
     * @return void
     */
    public function __construct(Video $video)
    {
        $this->video = $video;
    }

    /**
     * Execute the job.
     *
     * @return void
     */
    public function handle()
    {
        //
    }
}
```

If this job is queued with an `App\Video` instance that has an `id` of `1`, it will automatically receive the tag `App\Video:1`. This is because Horizon will examine the job's properties for any Eloquent models. If Eloquent models are found, Horizon will intelligently tag the job using the model's class name and primary key:

```
$video = App\Video::find(1);
App\Jobs\RenderVideo::dispatch($video);
```

Manually Tagging

If you would like to manually define the tags for one of your queueable objects, you may define a `tags` method on the class:

```
class RenderVideo implements ShouldQueue
{
    /**
     * Get the tags that should be assigned to the job.
     *
     * @return array
     */
    public function tags()
    {
        return ['render', 'video:'.$this->video->id];
    }
}
```

Notifications

Note: When configuring Horizon to send Slack or SMS notifications, you should review the [prerequisites for the relevant notification driver](#).

If you would like to be notified when one of your queues has a long wait time, you may use the `Horizon::routeMailNotificationsTo`, `Horizon::routeSlackNotificationsTo`, and `Horizon::routeSmsNotificationsTo` methods. You may call these methods from your application's `HorizonServiceProvider`:

```
Horizon::routeMailNotificationsTo('example@example.com');
Horizon::routeSlackNotificationsTo('slack-webhook-url', '#channel');
Horizon::routeSmsNotificationsTo('15556667777');
```

Configuring Notification Wait Time Thresholds

You may configure how many seconds are considered a "long wait" within your `config/horizon.php` configuration file. The `waits` configuration option within this file allows you to control the long wait threshold for each connection / queue combination:

```
'waits' => [
    'redis:default' => 60,
    'redis:critical,high' => 90,
],
```

Metrics

Horizon includes a metrics dashboard which provides information on your job and queue wait times and throughput. In order to populate this dashboard, you should configure Horizon's `snapshot` Artisan command to run every five minutes via your application's [scheduler](#):

```
/**
 * Define the application's command schedule.
 *
 * @param \Illuminate\Console\Scheduling\Schedule $schedule
 * @return void
 */
protected function schedule(Schedule $schedule)
{
    $schedule->command('horizon:snapshot')->everyFiveMinutes();
}
```


Official Packages

Passport

Laravel Passport

- [Introduction](#)
- [Upgrading Passport](#)
- [Installation](#)
 - [Frontend Quickstart](#)
 - [Deploying Passport](#)
 - [Migration Customization](#)
- [Configuration](#)
 - [Client Secret Hashing](#)
 - [Token Lifetimes](#)
 - [Overriding Default Models](#)
- [Issuing Access Tokens](#)
 - [Managing Clients](#)
 - [Requesting Tokens](#)
 - [Refreshing Tokens](#)
 - [Revoking Tokens](#)
 - [Purging Tokens](#)
- [Authorization Code Grant with PKCE](#)
 - [Creating The Client](#)
 - [Requesting Tokens](#)
- [Password Grant Tokens](#)
 - [Creating A Password Grant Client](#)
 - [Requesting Tokens](#)
 - [Requesting All Scopes](#)
 - [Customizing The User Provider](#)
 - [Customizing The Username Field](#)
 - [Customizing The Password Validation](#)
- [Implicit Grant Tokens](#)
- [Client Credentials Grant Tokens](#)
- [Personal Access Tokens](#)
 - [Creating A Personal Access Client](#)
 - [Managing Personal Access Tokens](#)
- [Protecting Routes](#)
 - [Via Middleware](#)
 - [Passing The Access Token](#)
- [Token Scopes](#)
 - [Defining Scopes](#)
 - [Default Scope](#)
 - [Assigning Scopes To Tokens](#)
 - [Checking Scopes](#)
- [Consuming Your API With JavaScript](#)
- [Events](#)
- [Testing](#)

Introduction

Laravel already makes it easy to perform authentication via traditional login forms, but what about APIs? APIs typically use tokens to authenticate users and do not maintain session state between requests. Laravel makes API authentication a breeze using Laravel Passport, which provides a full OAuth2 server implementation for your Laravel application in a matter of minutes. Passport is built on top of the [League OAuth2 server](#) that is maintained by Andy Millington and Simon Hamp.

NOTE This documentation assumes you are already familiar with OAuth2. If you do not know anything about OAuth2, consider familiarizing yourself with the general [terminology](#) and features of OAuth2 before

continuing.

Upgrading Passport

When upgrading to a new major version of Passport, it's important that you carefully review [the upgrade guide](#).

Installation

To get started, install Passport via the Composer package manager:

```
composer require laravel/passport "^9.0"
```

The Passport service provider registers its own database migration directory with the framework, so you should migrate your database after installing the package. The Passport migrations will create the tables your application needs to store clients and access tokens:

```
php artisan migrate
```

Next, you should run the `passport:install` command. This command will create the encryption keys needed to generate secure access tokens. In addition, the command will create "personal access" and "password grant" clients which will be used to generate access tokens:

```
php artisan passport:install
```

TIP If you would like to use UUIDs as the primary key value of the Passport `client` model instead of auto-incrementing integers, please install Passport using [the uuids option](#).

After running the `passport:install` command, add the `Laravel\Passport\HasApiTokens` trait to your `App\User` model. This trait will provide a few helper methods to your model which allow you to inspect the authenticated user's token and scopes:

```
<?php

namespace App;

use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;
use Laravel\Passport\HasApiTokens;

class User extends Authenticatable
{
    use HasApiTokens, Notifiable;
}
```

Next, you should call the `Passport::routes` method within the `boot` method of your `AuthServiceProvider`. This method will register the routes necessary to issue access tokens and revoke access tokens, clients, and personal access tokens:

```
<?php

namespace App\Providers;

use Illuminate\Foundation\Support\Providers\AuthServiceProvider as ServiceProvider;
use Illuminate\Support\Facades\Gate;
use Laravel\Passport\Passport;

class AuthServiceProvider extends ServiceProvider
{
    /**
     * The policy mappings for the application.
     *
     * @var array
     */
    protected $policies = [
        'App\Model' => 'App\Policies\ModelPolicy',
    ];

    /**
     * Register any authentication / authorization services.
     */
}
```

```
* @return void
*/
public function boot()
{
    $this->registerPolicies();

    Passport::routes();
}
}
```

Finally, in your `config/auth.php` configuration file, you should set the `driver` option of the `api` authentication guard to `passport`. This will instruct your application to use Passport's `TokenGuard` when authenticating incoming API requests:

```
'guards' => [
    'web' => [
        'driver' => 'session',
        'provider' => 'users',
    ],
    'api' => [
        'driver' => 'passport',
        'provider' => 'users',
    ],
],
```

Client UUIDs

You may run the `passport:install` command with the `--uuids` option present. This flag will instruct Passport that you would like to use UUIDs instead of auto-incrementing integers as the Passport `client` model's primary key values. After running the `passport:install` command with the `--uuids` option, you will be given additional instructions regarding disabling Passport's default migrations:

```
php artisan passport:install --uuids
```

Frontend Quickstart

NOTE In order to use the Passport Vue components, you must be using the [Vue](#) JavaScript framework. These components also use the Bootstrap CSS framework. However, even if you are not using these tools, the components serve as a valuable reference for your own frontend implementation.

Passport ships with a JSON API that you may use to allow your users to create clients and personal access tokens. However, it can be time consuming to code a frontend to interact with these APIs. So, Passport also includes pre-built [Vue](#) components you may use as an example implementation or starting point for your own implementation.

To publish the Passport Vue components, use the `vendor:publish` Artisan command:

```
php artisan vendor:publish --tag=passport-components
```

The published components will be placed in your `resources/js/components` directory. Once the components have been published, you should register them in your `resources/js/app.js` file:

```
Vue.component(
    'passport-clients',
    require('./components/passport/Clients.vue').default
);

Vue.component(
    'passport-authorized-clients',
    require('./components/passport/AuthorizedClients.vue').default
);

Vue.component(
    'passport-personal-access-tokens',
    require('./components/passport/PersonalAccessTokens.vue').default
);
```

NOTE Prior to Laravel v5.7.19, appending `.default` when registering components results in a console error. An explanation for this change can be found in the [Laravel Mix v4.0.0 release notes](#).

After registering the components, make sure to run `npm run dev` to recompile your assets. Once you have recompiled your assets, you may drop the components into one of your application's templates to get started creating clients and personal access tokens:

```
<passport-clients></passport-clients>
<passport-authorized-clients></passport-authorized-clients>
<passport-personal-access-tokens></passport-personal-access-tokens>
```

Deploying Passport

When deploying Passport to your production servers for the first time, you will likely need to run the `passport:keys` command. This command generates the encryption keys Passport needs in order to generate access token. The generated keys are not typically kept in source control:

```
php artisan passport:keys
```

If necessary, you may define the path where Passport's keys should be loaded from. You may use the `Passport::loadKeysFrom` method to accomplish this:

```
/**
 * Register any authentication / authorization services.
 *
 * @return void
 */
public function boot()
{
    $this->registerPolicies();

    Passport::routes();

    Passport::loadKeysFrom('/secret-keys/oauth');
}
```

Additionally, you may publish Passport's configuration file using `php artisan vendor:publish --tag=passport-config`, which will then provide the option to load the encryption keys from your environment variables:

```
PASSPORT_PRIVATE_KEY="-----BEGIN RSA PRIVATE KEY-----
<private key here>
-----END RSA PRIVATE KEY-----"

PASSPORT_PUBLIC_KEY="-----BEGIN PUBLIC KEY-----
<public key here>
-----END PUBLIC KEY-----"
```

Migration Customization

If you are not going to use Passport's default migrations, you should call the `Passport::ignoreMigrations` method in the `register` method of your `AppServiceProvider`. You may export the default migrations using `php artisan vendor:publish --tag=passport-migrations`.

Configuration

Client Secret Hashing

If you would like your client's secrets to be hashed when stored in your database, you should call the `Passport::hashClientSecrets` method in the `boot` method of your `AppServiceProvider`:

```
Passport::hashClientSecrets();
```

Once enabled, all of your client secrets will only be shown one time when your client is created. Since the plain-text client secret value is never stored in the database, it is not possible to recover if lost.

Token Lifetimes

By default, Passport issues long-lived access tokens that expire after one year. If you would like to configure a longer / shorter token lifetime, you may use the `tokensExpireIn`, `refreshTokensExpireIn`, and

`personalAccessTokensExpireIn` methods. These methods should be called from the `boot` method of your `AuthServiceProvider`:

```
/**
 * Register any authentication / authorization services.
 *
 * @return void
 */
public function boot()
{
    $this->registerPolicies();

    Passport::routes();

    Passport::tokensExpireIn(now()->addDays(15));

    Passport::refreshTokensExpireIn(now()->addDays(30));

    Passport::personalAccessTokensExpireIn(now()->addMonths(6));
}
```

NOTE The `expires_at` columns on the Passport database tables are read-only and for display purposes only. When issuing tokens, Passport stores the expiration information within the signed and encrypted tokens. If you need to invalidate a token you should revoke it.

Overriding Default Models

You are free to extend the models used internally by Passport:

```
use Laravel\Passport\Client as PassportClient;

class Client extends PassportClient
{
    // ...
}
```

Then, you may instruct Passport to use your custom models via the `Passport` class:

```
use App\Models\Passport\AuthCode;
use App\Models\Passport\Client;
use App\Models\Passport\PersonalAccessClient;
use App\Models\Passport\Token;

/**
 * Register any authentication / authorization services.
 *
 * @return void
 */
public function boot()
{
    $this->registerPolicies();

    Passport::routes();

    Passport::useTokenModel(Token::class);
    Passport::useClientModel(Client::class);
    Passport::useAuthCodeModel(AuthCode::class);
    Passport::usePersonalAccessClientModel(PersonalAccessClient::class);
}
```

Issuing Access Tokens

Using OAuth2 with authorization codes is how most developers are familiar with OAuth2. When using authorization codes, a client application will redirect a user to your server where they will either approve or deny the request to issue an access token to the client.

Managing Clients

First, developers building applications that need to interact with your application's API will need to register their application with yours by creating a "client". Typically, this consists of providing the name of their application and a URL that your application can redirect to after users approve their request for authorization.

The passport:client Command

The simplest way to create a client is using the `passport:client` Artisan command. This command may be used to create your own clients for testing your OAuth2 functionality. When you run the `client` command, Passport will prompt you for more information about your client and will provide you with a client ID and secret:

```
php artisan passport:client
```

Redirect URLs

If you would like to allow multiple redirect URLs for your client, you may specify them using a comma-delimited list when prompted for the URL by the `passport:client` command:

```
http://example.com/callback,http://examplefoo.com/callback
```

NOTE Any URL which contains commas must be encoded.

JSON API

Since your users will not be able to utilize the `client` command, Passport provides a JSON API that you may use to create clients. This saves you the trouble of having to manually code controllers for creating, updating, and deleting clients.

However, you will need to pair Passport's JSON API with your own frontend to provide a dashboard for your users to manage their clients. Below, we'll review all of the API endpoints for managing clients. For convenience, we'll use [Axios](#) to demonstrate making HTTP requests to the endpoints.

The JSON API is guarded by the `web` and `auth` middleware; therefore, it may only be called from your own application. It is not able to be called from an external source.

TIP If you don't want to implement the entire client management frontend yourself, you can use the [frontend quickstart](#) to have a fully functional frontend in a matter of minutes.

GET /oauth/clients

This route returns all of the clients for the authenticated user. This is primarily useful for listing all of the user's clients so that they may edit or delete them:

```
axios.get('/oauth/clients')
  .then(response => {
    console.log(response.data);
  });
```

POST /oauth/clients

This route is used to create new clients. It requires two pieces of data: the client's name and a redirect URL. The redirect URL is where the user will be redirected after approving or denying a request for authorization.

When a client is created, it will be issued a client ID and client secret. These values will be used when requesting access tokens from your application. The client creation route will return the new client instance:

```
const data = {
  name: 'Client Name',
  redirect: 'http://example.com/callback'
};

axios.post('/oauth/clients', data)
  .then(response => {
    console.log(response.data);
  })
  .catch (response => {
    // List errors on response...
  });
```

PUT /oauth/clients/{client-id}

This route is used to update clients. It requires two pieces of data: the client's name and a redirect URL. The redirect URL is where the user will be redirected after approving or denying a request for authorization. The route will return the updated client instance:

```
const data = {
  name: 'New Client Name',
  redirect: 'http://example.com/callback'
};

axios.put('/oauth/clients/' + clientId, data)
  .then(response => {
    console.log(response.data);
  })
  .catch (response => {
    // List errors on response...
  });
```

DELETE /oauth/clients/{client-id}

This route is used to delete clients:

```
axios.delete('/oauth/clients/' + clientId)
  .then(response => {
    //
  });
```

Requesting Tokens

Redirecting For Authorization

Once a client has been created, developers may use their client ID and secret to request an authorization code and access token from your application. First, the consuming application should make a redirect request to your application's `/oauth/authorize` route like so:

```
Route::get('/redirect', function (Request $request) {
  $request->session()->put('state', $state = Str::random(40));

  $query = http_build_query([
    'client_id' => 'client-id',
    'redirect_uri' => 'http://example.com/callback',
    'response_type' => 'code',
    'scope' => '',
    'state' => $state,
  ]);

  return redirect('http://your-app.com/oauth/authorize?'.$query);
});
```

TIP Remember, the `/oauth/authorize` route is already defined by the `Passport::routes` method. You do not need to manually define this route.

Approving The Request

When receiving authorization requests, Passport will automatically display a template to the user allowing them to approve or deny the authorization request. If they approve the request, they will be redirected back to the `redirect_uri` that was specified by the consuming application. The `redirect_uri` must match the redirect URL that was specified when the client was created.

If you would like to customize the authorization approval screen, you may publish Passport's views using the `vendor:publish` Artisan command. The published views will be placed in `resources/views/vendor/passport`:

```
php artisan vendor:publish --tag=passport-views
```

Sometimes you may wish to skip the authorization prompt, such as when authorizing a first-party client. You may accomplish this by [extending the client model](#) and defining a `skipsAuthorization` method. If `skipsAuthorization` returns `true` the client will be approved and the user will be redirected back to the `redirect_uri` immediately:

```
<?php

namespace App\Models\Passport;

use Laravel\Passport\Client as BaseClient;

class Client extends BaseClient
{
    /**
     * Determine if the client should skip the authorization prompt.
     *
     * @return bool
     */
    public function skipsAuthorization()
    {
        return $this->firstParty();
    }
}
```

Converting Authorization Codes To Access Tokens

If the user approves the authorization request, they will be redirected back to the consuming application. The consumer should first verify the `state` parameter against the value that was stored prior to the redirect. If the state parameter matches the consumer should issue a `POST` request to your application to request an access token. The request should include the authorization code that was issued by your application when the user approved the authorization request. In this example, we'll use the Guzzle HTTP library to make the `POST` request:

```
Route::get('/callback', function (Request $request) {
    $state = $request->session()->pull('state');

    throw_unless(
        strlen($state) > 0 && $state === $request->state,
        InvalidArgumentException::class
    );

    $http = new GuzzleHttp\Client;

    $response = $http->post('http://your-app.com/oauth/token', [
        'form_params' => [
            'grant_type' => 'authorization_code',
            'client_id' => 'client-id',
            'client_secret' => 'client-secret',
            'redirect_uri' => 'http://example.com/callback',
            'code' => $request->code,
        ],
    ]);

    return json_decode((string) $response->getBody(), true);
});
```

This `/oauth/token` route will return a JSON response containing `access_token`, `refresh_token`, and `expires_in` attributes. The `expires_in` attribute contains the number of seconds until the access token expires.

TIP Like the `/oauth/authorize` route, the `/oauth/token` route is defined for you by the `Passport::routes` method. There is no need to manually define this route. By default, this route is throttled using the settings of the `ThrottleRequests` middleware.

JSON API

Passport also includes a JSON API for managing authorized access tokens. You may pair this with your own frontend to offer your users a dashboard for managing access tokens. For convenience, we'll use [Axios](#) to demonstrate making HTTP requests to the endpoints. The JSON API is guarded by the `web` and `auth` middleware; therefore, it may only be called from your own application.

GET /oauth/tokens

This route returns all of the authorized access tokens that the authenticated user has created. This is primarily useful for listing all of the user's tokens so that they can revoke them:

```
axios.get('/oauth/tokens')
```



```
.then(response => {
    console.log(response.data);
});
```

DELETE /oauth/tokens/{token-id}

This route may be used to revoke authorized access tokens and their related refresh tokens:

```
axios.delete('/oauth/tokens/' + tokenId);
```

Refreshing Tokens

If your application issues short-lived access tokens, users will need to refresh their access tokens via the refresh token that was provided to them when the access token was issued. In this example, we'll use the Guzzle HTTP library to refresh the token:

```
$http = new GuzzleHttp\Client;

$response = $http->post('http://your-app.com/oauth/token', [
    'form_params' => [
        'grant_type' => 'refresh_token',
        'refresh_token' => 'the-refresh-token',
        'client_id' => 'client-id',
        'client_secret' => 'client-secret',
        'scope' => '',
    ],
]);

return json_decode((string) $response->getBody(), true);
```

This `/oauth/token` route will return a JSON response containing `access_token`, `refresh_token`, and `expires_in` attributes. The `expires_in` attribute contains the number of seconds until the access token expires.

Revoking Tokens

You may revoke a token by using the `revokeAccessToken` method on the `TokenRepository`. You may revoke a token's refresh tokens using the `revokeRefreshTokensByAccessTokenId` method on the `RefreshTokenRepository`:

```
$tokenRepository = app('Laravel\Passport\TokenRepository');
$refreshTokenRepository = app('Laravel\Passport\RefreshTokenRepository');

// Revoke an access token...
$tokenRepository->revokeAccessToken($tokenId);

// Revoke all of the token's refresh tokens...
$refreshTokenRepository->revokeRefreshTokensByAccessTokenId($tokenId);
```

Purging Tokens

When tokens have been revoked or expired, you might want to purge them from the database. Passport ships with a command that can do this for you:

```
# Purge revoked and expired tokens and auth codes...
php artisan passport:purge

# Only purge revoked tokens and auth codes...
php artisan passport:purge --revoked

# Only purge expired tokens and auth codes...
php artisan passport:purge --expired
```

You may also configure a [scheduled job](#) in your console `kernel` class to automatically prune your tokens on a schedule:

```
/**
 * Define the application's command schedule.
 *
 * @param \Illuminate\Console\Scheduling\Schedule $schedule
 * @return void
 */
```

```
protected function schedule(Schedule $schedule)
{
    $schedule->command('passport:purge')->hourly();
}
```

Authorization Code Grant with PKCE

The Authorization Code grant with "Proof Key for Code Exchange" (PKCE) is a secure way to authenticate single page applications or native applications to access your API. This grant should be used when you can't guarantee that the client secret will be stored confidentially or in order to mitigate the threat of having the authorization code intercepted by an attacker. A combination of a "code verifier" and a "code challenge" replaces the client secret when exchanging the authorization code for an access token.

Creating The Client

Before your application can issue tokens via the authorization code grant with PKCE, you will need to create a PKCE-enabled client. You may do this using the `passport:client` command with the `--public` option:

```
php artisan passport:client --public
```

Requesting Tokens

Code Verifier & Code Challenge

As this authorization grant does not provide a client secret, developers will need to generate a combination of a code verifier and a code challenge in order to request a token.

The code verifier should be a random string of between 43 and 128 characters containing letters, numbers and "-", ".", "_", "~", as defined in the [RFC 7636 specification](#).

The code challenge should be a Base64 encoded string with URL and filename-safe characters. The trailing '=' characters should be removed and no line breaks, whitespace, or other additional characters should be present.

```
$encoded = base64_encode(hash('sha256', $code_verifier, true));
$codeChallenge = strtr(rtrim($encoded, '='), '+/', '-_');
```

Redirecting For Authorization

Once a client has been created, you may use the client ID and the generated code verifier and code challenge to request an authorization code and access token from your application. First, the consuming application should make a redirect request to your application's `/oauth/authorize` route:

```
Route::get('/redirect', function (Request $request) {
    $request->session()->put('state', $state = Str::random(40));

    $request->session()->put('code_verifier', $code_verifier = Str::random(128));

    $codeChallenge = strtr(rtrim(
        base64_encode(hash('sha256', $code_verifier, true))
        , '='), '+/', '-_');

    $query = http_build_query([
        'client_id' => 'client-id',
        'redirect_uri' => 'http://example.com/callback',
        'response_type' => 'code',
        'scope' => '',
        'state' => $state,
        'code_challenge' => $codeChallenge,
        'code_challenge_method' => 'S256',
    ]);

    return redirect('http://your-app.com/oauth/authorize?'.$query);
});
```

Converting Authorization Codes To Access Tokens

If the user approves the authorization request, they will be redirected back to the consuming application. The consumer should verify the `state` parameter against the value that was stored prior to the redirect, as in the standard Authorization Code Grant.

If the `state` parameter matches, the consumer should issue a `POST` request to your application to request an access token. The request should include the authorization code that was issued by your application when the user approved the authorization request along with the originally generated code verifier:

```
Route::get('/callback', function (Request $request) {
    $state = $request->session()->pull('state');

    $codeVerifier = $request->session()->pull('code_verifier');

    throw_unless(
        strlen($state) > 0 && $state === $request->state,
        InvalidArgumentException::class
    );

    $response = (new GuzzleHttp\Client)->post('http://your-app.com/oauth/token', [
        'form_params' => [
            'grant_type' => 'authorization_code',
            'client_id' => 'client-id',
            'redirect_uri' => 'http://example.com/callback',
            'code_verifier' => $codeVerifier,
            'code' => $request->code,
        ],
    ]);

    return json_decode((string) $response->getBody(), true);
});
```

Password Grant Tokens

The OAuth2 password grant allows your other first-party clients, such as a mobile application, to obtain an access token using an e-mail address / username and password. This allows you to issue access tokens securely to your first-party clients without requiring your users to go through the entire OAuth2 authorization code redirect flow.

Creating A Password Grant Client

Before your application can issue tokens via the password grant, you will need to create a password grant client. You may do this using the `passport:client` command with the `--password` option. If you have already run the `passport:install` command, you do not need to run this command:

```
php artisan passport:client --password
```

Requesting Tokens

Once you have created a password grant client, you may request an access token by issuing a `POST` request to the `/oauth/token` route with the user's email address and password. Remember, this route is already registered by the `Passport::routes` method so there is no need to define it manually. If the request is successful, you will receive an `access_token` and `refresh_token` in the JSON response from the server:

```
$http = new GuzzleHttp\Client;

$response = $http->post('http://your-app.com/oauth/token', [
    'form_params' => [
        'grant_type' => 'password',
        'client_id' => 'client-id',
        'client_secret' => 'client-secret',
        'username' => 'taylor@laravel.com',
        'password' => 'my-password',
        'scope' => '',
    ],
]);

return json_decode((string) $response->getBody(), true);
```

TIP Remember, access tokens are long-lived by default. However, you are free to [configure your](#)

[maximum access token lifetime](#) if needed.

Requesting All Scopes

When using the password grant or client credentials grant, you may wish to authorize the token for all of the scopes supported by your application. You can do this by requesting the `*` scope. If you request the `*` scope, the `can` method on the token instance will always return `true`. This scope may only be assigned to a token that is issued using the password or `client_credentials` grant:

```
$response = $http->post('http://your-app.com/oauth/token', [
    'form_params' => [
        'grant_type' => 'password',
        'client_id' => 'client-id',
        'client_secret' => 'client-secret',
        'username' => 'taylor@laravel.com',
        'password' => 'my-password',
        'scope' => '*',
    ],
]);
```

Customizing The User Provider

If your application uses more than one [authentication user provider](#), you may specify which user provider the password grant client uses by providing a `--provider` option when creating the client via the `artisan passport:client --password` command. The given provider name should match a valid provider defined in your `config/auth.php` configuration file. You can then [protect your route using middleware](#) to ensure that only users from the guard's specified provider are authorized.

Customizing The Username Field

When authenticating using the password grant, Passport will use the `email` attribute of your model as the "username". However, you may customize this behavior by defining a `findForPassport` method on your model:

```
<?php

namespace App;

use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;
use Laravel\Passport\HasApiTokens;

class User extends Authenticatable
{
    use HasApiTokens, Notifiable;

    /**
     * Find the user instance for the given username.
     *
     * @param string $username
     * @return \App\User
     */
    public function findForPassport($username)
    {
        return $this->where('username', $username)->first();
    }
}
```

Customizing The Password Validation

When authenticating using the password grant, Passport will use the `password` attribute of your model to validate the given password. If your model does not have a `password` attribute or you wish to customize the password validation logic, you can define a `validateForPassportPasswordGrant` method on your model:

```
<?php

namespace App;

use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;
```

```
use Illuminate\Support\Facades\Hash;
use Laravel\Passport\HasApiTokens;

class User extends Authenticatable
{
    use HasApiTokens, Notifiable;

    /**
     * Validate the password of the user for the Passport password grant.
     *
     * @param string $password
     * @return bool
     */
    public function validateForPassportPasswordGrant($password)
    {
        return Hash::check($password, $this->password);
    }
}
```

Implicit Grant Tokens

The implicit grant is similar to the authorization code grant; however, the token is returned to the client without exchanging an authorization code. This grant is most commonly used for JavaScript or mobile applications where the client credentials can't be securely stored. To enable the grant, call the `enableImplicitGrant` method in your `AuthServiceProvider`:

```
/**
 * Register any authentication / authorization services.
 *
 * @return void
 */
public function boot()
{
    $this->registerPolicies();

    Passport::routes();

    Passport::enableImplicitGrant();
}
```

Once a grant has been enabled, developers may use their client ID to request an access token from your application. The consuming application should make a redirect request to your application's `/oauth/authorize` route like so:

```
Route::get('/redirect', function (Request $request) {
    $request->session()->put('state', $state = Str::random(40));

    $query = http_build_query([
        'client_id' => 'client-id',
        'redirect_uri' => 'http://example.com/callback',
        'response_type' => 'token',
        'scope' => '',
        'state' => $state,
    ]);

    return redirect('http://your-app.com/oauth/authorize?'.$query);
});
```

TIP Remember, the `/oauth/authorize` route is already defined by the `Passport::routes` method. You do not need to manually define this route.

Client Credentials Grant Tokens

The client credentials grant is suitable for machine-to-machine authentication. For example, you might use this grant in a scheduled job which is performing maintenance tasks over an API.

Before your application can issue tokens via the client credentials grant, you will need to create a client credentials grant client. You may do this using the `--client` option of the `passport:client` command:

```
php artisan passport:client --client
```

Next, to use this grant type, you need to add the `checkClientCredentials` middleware to the `$routeMiddleware` property of your `app/Http/Kernel.php` file:

```
use Laravel\Passport\Http\Middleware\CheckClientCredentials;

protected $routeMiddleware = [
    'client' => CheckClientCredentials::class,
];
```

Then, attach the middleware to a route:

```
Route::get('/orders', function (Request $request) {
    ...
})->middleware('client');
```

To restrict access to the route to specific scopes you may provide a comma-delimited list of the required scopes when attaching the `client` middleware to the route:

```
Route::get('/orders', function (Request $request) {
    ...
})->middleware('client:check-status,your-scope');
```

Retrieving Tokens

To retrieve a token using this grant type, make a request to the `oauth/token` endpoint:

```
$guzzle = new GuzzleHttpClient;

$response = $guzzle->post('http://your-app.com/oauth/token', [
    'form_params' => [
        'grant_type' => 'client_credentials',
        'client_id' => 'client-id',
        'client_secret' => 'client-secret',
        'scope' => 'your-scope',
    ],
]);

return json_decode((string) $response->getBody(), true)['access_token'];
```

Personal Access Tokens

Sometimes, your users may want to issue access tokens to themselves without going through the typical authorization code redirect flow. Allowing users to issue tokens to themselves via your application's UI can be useful for allowing users to experiment with your API or may serve as a simpler approach to issuing access tokens in general.

Creating A Personal Access Client

Before your application can issue personal access tokens, you will need to create a personal access client. You may do this using the `passport:client` command with the `--personal` option. If you have already run the `passport:install` command, you do not need to run this command:

```
php artisan passport:client --personal
```

After creating your personal access client, place the client's ID and plain-text secret value in your application's `.env` file:

```
PASSPORT_PERSONAL_ACCESS_CLIENT_ID=client-id-value
PASSPORT_PERSONAL_ACCESS_CLIENT_SECRET=unhashed-client-secret-value
```

Next, you should register these values by placing the following calls to `Passport::personalAccessClientId` and `Passport::personalAccessClientSecret` within the `boot` method of your `AuthServiceProvider`:

```
/**
 * Register any authentication / authorization services.
 *
 * @return void
 */
public function boot()
```

```
{
  $this->registerPolicies();

  Passport::routes();

  Passport::personalAccessClientId(
    config('passport.personal_access_client.id')
  );

  Passport::personalAccessClientSecret(
    config('passport.personal_access_client.secret')
  );
}
```

Managing Personal Access Tokens

Once you have created a personal access client, you may issue tokens for a given user using the `createToken` method on the `User` model instance. The `createToken` method accepts the name of the token as its first argument and an optional array of [scopes](#) as its second argument:

```
$user = App\User::find(1);

// Creating a token without scopes...
$token = $user->createToken('Token Name')->accessToken;

// Creating a token with scopes...
$token = $user->createToken('My Token', ['place-orders'])->accessToken;
```

JSON API

Passport also includes a JSON API for managing personal access tokens. You may pair this with your own frontend to offer your users a dashboard for managing personal access tokens. Below, we'll review all of the API endpoints for managing personal access tokens. For convenience, we'll use [Axios](#) to demonstrate making HTTP requests to the endpoints.

The JSON API is guarded by the `web` and `auth` middleware; therefore, it may only be called from your own application. It is not able to be called from an external source.

TIP If you don't want to implement the personal access token frontend yourself, you can use the [frontend quickstart](#) to have a fully functional frontend in a matter of minutes.

GET /oauth/scopes

This route returns all of the [scopes](#) defined for your application. You may use this route to list the scopes a user may assign to a personal access token:

```
axios.get('/oauth/scopes')
  .then(response => {
    console.log(response.data);
  });
```

GET /oauth/personal-access-tokens

This route returns all of the personal access tokens that the authenticated user has created. This is primarily useful for listing all of the user's tokens so that they may edit or revoke them:

```
axios.get('/oauth/personal-access-tokens')
  .then(response => {
    console.log(response.data);
  });
```

POST /oauth/personal-access-tokens

This route creates new personal access tokens. It requires two pieces of data: the token's name and the `scopes` that should be assigned to the token:

```
const data = {
  name: 'Token Name',
```

```
    scopes: []
  });

  axios.post('/oauth/personal-access-tokens', data)
    .then(response => {
      console.log(response.data.accessToken);
    })
    .catch (response => {
      // List errors on response...
    });
```

DELETE /oauth/personal-access-tokens/{token-id}

This route may be used to revoke personal access tokens:

```
axios.delete('/oauth/personal-access-tokens/' + tokenId);
```

Protecting Routes

Via Middleware

Passport includes an [authentication guard](#) that will validate access tokens on incoming requests. Once you have configured the api guard to use the passport driver, you only need to specify the `auth:api` middleware on any routes that require a valid access token:

```
Route::get('/user', function () {
    //
})->middleware('auth:api');
```

Multiple Authentication Guards

If your application authenticates different types of users that perhaps use entirely different Eloquent models, you will likely need to define a guard configuration for each user provider type in your application. This allows you to protect requests intended for specific user providers. For example, given the following guard configuration the `config/auth.php` configuration file:

```
'api' => [
    'driver' => 'passport',
    'provider' => 'users',
],

'api-customers' => [
    'driver' => 'passport',
    'provider' => 'customers',
],
```

The following route will utilize the `api-customers` guard, which uses the `customers` user provider, to authenticate incoming requests:

```
Route::get('/customer', function () {
    //
})->middleware('auth:api-customers');
```

TIP For more information on using multiple user providers with Passport, please consult the [password grant documentation](#).

Passing The Access Token

When calling routes that are protected by Passport, your application's API consumers should specify their access token as a Bearer token in the Authorization header of their request. For example, when using the Guzzle HTTP library:

```
$response = $client->request('GET', '/api/user', [
    'headers' => [
        'Accept' => 'application/json',
        'Authorization' => 'Bearer '.$accessToken,
    ],
]);
```


Token Scopes

Scopes allow your API clients to request a specific set of permissions when requesting authorization to access an account. For example, if you are building an e-commerce application, not all API consumers will need the ability to place orders. Instead, you may allow the consumers to only request authorization to access order shipment statuses. In other words, scopes allow your application's users to limit the actions a third-party application can perform on their behalf.

Defining Scopes

You may define your API's scopes using the `Passport::tokensCan` method in the `boot` method of your `AuthServiceProvider`. The `tokensCan` method accepts an array of scope names and scope descriptions. The scope description may be anything you wish and will be displayed to users on the authorization approval screen:

```
use Laravel\Passport\Passport;

Passport::tokensCan([
    'place-orders' => 'Place orders',
    'check-status' => 'Check order status',
]);
```

Default Scope

If a client does not request any specific scopes, you may configure your Passport server to attach a default scope to the token using the `setDefaultScope` method. Typically, you should call this method from the `boot` method of your `AuthServiceProvider`:

```
use Laravel\Passport\Passport;

Passport::setDefaultScope([
    'check-status',
    'place-orders',
]);
```

Assigning Scopes To Tokens

When Requesting Authorization Codes

When requesting an access token using the authorization code grant, consumers should specify their desired scopes as the `scope` query string parameter. The `scope` parameter should be a space-delimited list of scopes:

```
Route::get('/redirect', function () {
    $query = http_build_query([
        'client_id' => 'client-id',
        'redirect_uri' => 'http://example.com/callback',
        'response_type' => 'code',
        'scope' => 'place-orders check-status',
    ]);

    return redirect('http://your-app.com/oauth/authorize?'.$query);
});
```

When Issuing Personal Access Tokens

If you are issuing personal access tokens using the user model's `createToken` method, you may pass the array of desired scopes as the second argument to the method:

```
$token = $user->createToken('My Token', ['place-orders'])->accessToken;
```

Checking Scopes

Passport includes two middleware that may be used to verify that an incoming request is authenticated with a token that has been granted a given scope. To get started, add the following middleware to the `$routeMiddleware` property of your `app/Http/Kernel.php` file:

```
'scopes' => \Laravel\Passport\Http\Middleware\CheckScopes::class,  
'scope' => \Laravel\Passport\Http\Middleware\CheckForAnyScope::class,
```

Check For All Scopes

The scopes middleware may be assigned to a route to verify that the incoming request's access token has *all* of the listed scopes:

```
Route::get('/orders', function () {  
    // Access token has both "check-status" and "place-orders" scopes...  
})->middleware(['auth:api', 'scopes:check-status,place-orders']);
```

Check For Any Scopes

The scope middleware may be assigned to a route to verify that the incoming request's access token has *at least one* of the listed scopes:

```
Route::get('/orders', function () {  
    // Access token has either "check-status" or "place-orders" scope...  
})->middleware(['auth:api', 'scope:check-status,place-orders']);
```

Checking Scopes On A Token Instance

Once an access token authenticated request has entered your application, you may still check if the token has a given scope using the `tokenCan` method on the authenticated user instance:

```
use Illuminate\Http\Request;  
  
Route::get('/orders', function (Request $request) {  
    if ($request->user()->tokenCan('place-orders')) {  
        //  
    }  
});
```

Additional Scope Methods

The `scopeIds` method will return an array of all defined IDs / names:

```
Laravel\Passport\Passport::scopeIds();
```

The `scopes` method will return an array of all defined scopes as instances of `Laravel\Passport\Scope`:

```
Laravel\Passport\Passport::scopes();
```

The `scopesFor` method will return an array of `Laravel\Passport\Scope` instances matching the given IDs / names:

```
Laravel\Passport\Passport::scopesFor(['place-orders', 'check-status']);
```

You may determine if a given scope has been defined using the `hasScope` method:

```
Laravel\Passport\Passport::hasScope('place-orders');
```

Consuming Your API With JavaScript

When building an API, it can be extremely useful to be able to consume your own API from your JavaScript application. This approach to API development allows your own application to consume the same API that you are sharing with the world. The same API may be consumed by your web application, mobile applications, third-party applications, and any SDKs that you may publish on various package managers.

Typically, if you want to consume your API from your JavaScript application, you would need to manually send an access token to the application and pass it with each request to your application. However, Passport includes a middleware that can handle this for you. All you need to do is add the `CreateFreshApiToken` middleware to your web middleware group in your `app/Http/Kernel.php` file:

```
'web' => [  
    // Other middleware...
```

```
        \Laravel\Passport\Http\Middleware\CreateFreshApiToken::class,
    ],
```

NOTE You should ensure that the `CreateFreshApiToken` middleware is the last middleware listed in your middleware stack.

This Passport middleware will attach a `laravel_token` cookie to your outgoing responses. This cookie contains an encrypted JWT that Passport will use to authenticate API requests from your JavaScript application. The JWT has a lifetime equal to your `session.lifetime` configuration value. Now, you may make requests to your application's API without explicitly passing an access token:

```
axios.get('/api/user')
    .then(response => {
        console.log(response.data);
    });
```

Customizing The Cookie Name

If needed, you can customize the `laravel_token` cookie's name using the `Passport::cookie` method. Typically, this method should be called from the `boot` method of your `AuthServiceProvider`:

```
/**
 * Register any authentication / authorization services.
 *
 * @return void
 */
public function boot()
{
    $this->registerPolicies();

    Passport::routes();

    Passport::cookie('custom_name');
}
```

CSRF Protection

When using this method of authentication, you will need to ensure a valid CSRF token header is included in your requests. The default Laravel JavaScript scaffolding includes an Axios instance, which will automatically use the encrypted `XSRF-TOKEN` cookie value to send a `X-XSRF-TOKEN` header on same-origin requests.

TIP If you choose to send the `X-CSRF-TOKEN` header instead of `X-XSRF-TOKEN`, you will need to use the unencrypted token provided by `csrf_token()`.

Events

Passport raises events when issuing access tokens and refresh tokens. You may use these events to prune or revoke other access tokens in your database. You may attach listeners to these events in your application's `EventServiceProvider`:

```
/**
 * The event listener mappings for the application.
 *
 * @var array
 */
protected $listen = [
    'Laravel\Passport\Events\AccessTokenCreated' => [
        'App\Listeners\RevokeOldTokens',
    ],

    'Laravel\Passport\Events\RefreshTokenCreated' => [
        'App\Listeners\PruneOldTokens',
    ],
];
```

Testing

Passport's `actingAs` method may be used to specify the currently authenticated user as well as its scopes. The

first argument given to the `actingAs` method is the user instance and the second is an array of scopes that should be granted to the user's token:

```
use App\User;
use Laravel\Passport\Passport;

public function testServerCreation()
{
    Passport::actingAs(
        factory(User::class)->create(),
        ['create-servers']
    );

    $response = $this->post('/api/create-server');

    $response->assertStatus(201);
}
```

Passport's `actingAsClient` method may be used to specify the currently authenticated client as well as its scopes. The first argument given to the `actingAsClient` method is the client instance and the second is an array of scopes that should be granted to the client's token:

```
use Laravel\Passport\Client;
use Laravel\Passport\Passport;

public function testGetOrders()
{
    Passport::actingAsClient(
        factory(Client::class)->create(),
        ['check-status']
    );

    $response = $this->get('/api/orders');

    $response->assertStatus(200);
}
```

Official Packages

Sanctum

Laravel Sanctum

- [Introduction](#)
 - [How It Works](#)
- [Installation](#)
- [API Token Authentication](#)
 - [Issuing API Tokens](#)
 - [Token Abilities](#)
 - [Protecting Routes](#)
 - [Revoking Tokens](#)
- [SPA Authentication](#)
 - [Configuration](#)
 - [Authenticating](#)
 - [Protecting Routes](#)
 - [Authorizing Private Broadcast Channels](#)
- [Mobile Application Authentication](#)
 - [Issuing API Tokens](#)
 - [Protecting Routes](#)
 - [Revoking Tokens](#)
- [Testing](#)

Introduction

Laravel Sanctum provides a featherweight authentication system for SPAs (single page applications), mobile applications, and simple, token based APIs. Sanctum allows each user of your application to generate multiple API tokens for their account. These tokens may be granted abilities / scopes which specify which actions the tokens are allowed to perform.

How It Works

Laravel Sanctum exists to solve two separate problems.

API Tokens

First, it is a simple package to issue API tokens to your users without the complication of OAuth. This feature is inspired by GitHub "access tokens". For example, imagine the "account settings" of your application has a screen where a user may generate an API token for their account. You may use Sanctum to generate and manage those tokens. These tokens typically have a very long expiration time (years), but may be manually revoked by the user at anytime.

Laravel Sanctum offers this feature by storing user API tokens in a single database table and authenticating incoming requests via the `Authorization` header which should contain a valid API token.

SPA Authentication

Second, Sanctum exists to offer a simple way to authenticate single page applications (SPAs) that need to communicate with a Laravel powered API. These SPAs might exist in the same repository as your Laravel application or might be an entirely separate repository, such as a SPA created using Vue CLI.

For this feature, Sanctum does not use tokens of any kind. Instead, Sanctum uses Laravel's built-in cookie based session authentication services. This provides the benefits of CSRF protection, session authentication, as well as protects against leakage of the authentication credentials via XSS. Sanctum will only attempt to authenticate using cookies when the incoming request originates from your own SPA frontend.

TIP It is perfectly fine to use Sanctum only for API token authentication or only for SPA authentication. Just because you use Sanctum does not mean you are required to use both features it offers.

Installation

You may install Laravel Sanctum via Composer:

```
composer require laravel/sanctum
```

Next, you should publish the Sanctum configuration and migration files using the `vendor:publish` Artisan command. The `sanctum` configuration file will be placed in your `config` directory:

```
php artisan vendor:publish --provider="Laravel\Sanctum\SanctumServiceProvider"
```

Finally, you should run your database migrations. Sanctum will create one database table in which to store API tokens:

```
php artisan migrate
```

Next, if you plan to utilize Sanctum to authenticate an SPA, you should add Sanctum's middleware to your `api` middleware group within your `app/Http/Kernel.php` file:

```
use Laravel\Sanctum\Http\Middleware\EnsureFrontendRequestsAreStateful;

'api' => [
    EnsureFrontendRequestsAreStateful::class,
    'throttle:60,1',
    \Illuminate\Routing\Middleware\SubstituteBindings::class,
],
```

Migration Customization

If you are not going to use Sanctum's default migrations, you should call the `sanctum::ignoreMigrations` method in the `register` method of your `AppServiceProvider`. You may export the default migrations using `php artisan vendor:publish --tag=sanctum-migrations`.

API Token Authentication

TIP You should not use API tokens to authenticate your own first-party SPA. Instead, use Sanctum's built-in [SPA authentication](#).

Issuing API Tokens

Sanctum allows you to issue API tokens / personal access tokens that may be used to authenticate API requests. When making requests using API tokens, the token should be included in the `Authorization` header as a `Bearer` token.

To begin issuing tokens for users, your `User` model should use the `HasApiTokens` trait:

```
use Laravel\Sanctum\HasApiTokens;

class User extends Authenticatable
{
    use HasApiTokens, Notifiable;
}
```

To issue a token, you may use the `createToken` method. The `createToken` method returns a `Laravel\Sanctum\NewAccessToken` instance. API tokens are hashed using SHA-256 hashing before being stored in your database, but you may access the plain-text value of the token using the `plainTextToken` property of the `NewAccessToken` instance. You should display this value to the user immediately after the token has been created:

```
$token = $user->createToken('token-name');

return $token->plainTextToken;
```

You may access all of the user's tokens using the `tokens` Eloquent relationship provided by the `HasApiTokens` trait:

```
foreach ($user->tokens as $token) {  
    //  
}
```

Token Abilities

Sanctum allows you to assign "abilities" to tokens, similar to OAuth "scopes". You may pass an array of string abilities as the second argument to the `createToken` method:

```
return $user->createToken('token-name', ['server:update'])->plainTextToken;
```

When handling an incoming request authenticated by Sanctum, you may determine if the token has a given ability using the `tokenCan` method:

```
if ($user->tokenCan('server:update')) {  
    //  
}
```

TIP For convenience, the `tokenCan` method will always return `true` if the incoming authenticated request was from your first-party SPA and you are using Sanctum's built-in [SPA authentication](#).

Protecting Routes

To protect routes so that all incoming requests must be authenticated, you should attach the `sanctum` authentication guard to your API routes within your `routes/api.php` file. This guard will ensure that incoming requests are authenticated as either a stateful authenticated requests from your SPA or contain a valid API token header if the request is from a third party:

```
Route::middleware('auth:sanctum')->get('/user', function (Request $request) {  
    return $request->user();  
});
```

Revoking Tokens

You may "revoke" tokens by deleting them from your database using the `tokens` relationship that is provided by the `HasApiTokens` trait:

```
// Revoke all tokens...  
$user->tokens()->delete();  
  
// Revoke the user's current token...  
$request->user()->currentAccessToken()->delete();  
  
// Revoke a specific token...  
$user->tokens()->where('id', $id)->delete();
```

SPA Authentication

Sanctum exists to offer a simple way to authenticate single page applications (SPAs) that need to communicate with a Laravel powered API. These SPAs might exist in the same repository as your Laravel application or might be an entirely separate repository, such as a SPA created using Vue CLI.

For this feature, Sanctum does not use tokens of any kind. Instead, Sanctum uses Laravel's built-in cookie based session authentication services. This provides the benefits of CSRF protection, session authentication, as well as protects against leakage of the authentication credentials via XSS. Sanctum will only attempt to authenticate using cookies when the incoming request originates from your own SPA frontend.

NOTE In order to authenticate, your SPA and API must share the same top-level domain. However, they may be placed on different subdomains.

Configuration

Configuring Your First-Party Domains

First, you should configure which domains your SPA will be making requests from. You may configure these domains using the `stateful` configuration option in your `sanctum` configuration file. This configuration setting determines which domains will maintain "stateful" authentication using Laravel session cookies when making requests to your API.

NOTE If you are accessing your application via a URL that includes the port (`127.0.0.1:8000`), you should ensure that you include the port number with the domain.

Sanctum Middleware

Next, you should add Sanctum's middleware to your `api` middleware group within your `app/Http/Kernel.php` file. This middleware is responsible for ensuring that incoming requests from your SPA can authenticate using Laravel's session cookies, while still allowing requests from third parties or mobile applications to authenticate using API tokens:

```
use Laravel\Sanctum\Http\Middleware\EnsureFrontendRequestsAreStateful;

'api' => [
    EnsureFrontendRequestsAreStateful::class,
    'throttle:60,1',
    \Illuminate\Routing\Middleware\SubstituteBindings::class,
],
```

CORS & Cookies

If you are having trouble authenticating with your application from an SPA that executes on a separate subdomain, you have likely misconfigured your CORS (Cross-Origin Resource Sharing) or session cookie settings.

You should ensure that your application's CORS configuration is returning the `Access-Control-Allow-Credentials` header with a value of `True` by setting the `supports_credentials` option within your application's `cors` configuration file to `true`.

In addition, you should enable the `withCredentials` option on your global `axios` instance. Typically, this should be performed in your `resources/js/bootstrap.js` file:

```
axios.defaults.withCredentials = true;
```

Finally, you should ensure your application's session cookie domain configuration supports any subdomain of your root domain. You may do this by prefixing the domain with a leading `.` within your session configuration file:

```
'domain' => '.domain.com',
```

Authenticating

To authenticate your SPA, your SPA's login page should first make a request to the `/sanctum/csrf-cookie` route to initialize CSRF protection for the application:

```
axios.get('/sanctum/csrf-cookie').then(response => {
    // Login...
});
```

During this request Laravel will set an `XSRF-TOKEN` cookie containing the current CSRF token. This token should then be passed in an `X-XSRF-TOKEN` header on subsequent requests, which libraries like `Axios` and the `Angular HttpClient` will do automatically for you.

Once CSRF protection has been initialized, you should make a `POST` request to the typical Laravel `/login` route. This `/login` route may be provided by the `laravel/ui` [authentication scaffolding](#) package.

If the login request is successful, you will be authenticated and subsequent requests to your API routes will automatically be authenticated via the session cookie that the Laravel backend issued to your client.

TIP You are free to write your own `/login` endpoint; however, you should ensure that it authenticates the user using the standard, [session based authentication services that Laravel provides](#).

Protecting Routes

To protect routes so that all incoming requests must be authenticated, you should attach the `sanctum` authentication guard to your API routes within your `routes/api.php` file. This guard will ensure that incoming requests are authenticated as either a stateful authenticated requests from your SPA or contain a valid API token header if the request is from a third party:

```
Route::middleware('auth:sanctum')->get('/user', function (Request $request) {
    return $request->user();
});
```

Authorizing Private Broadcast Channels

If your SPA needs to authenticate with [private / presence broadcast channels](#), you should place the `Broadcast::routes` method call within your `routes/api.php` file:

```
Broadcast::routes(['middleware' => ['auth:sanctum']]);
```

Next, in order for Pusher's authorization requests to succeed, you will need to provide a custom Pusher authorizer when initializing [Laravel Echo](#). This allows your application to configure Pusher to use the `axios` instance that is [properly configured for cross-domain requests](#):

```
window.Echo = new Echo({
    broadcaster: "pusher",
    cluster: process.env.MIX_PUSHER_APP_CLUSTER,
    encrypted: true,
    key: process.env.MIX_PUSHER_APP_KEY,
    authorizer: (channel, options) => {
        return {
            authorize: (socketId, callback) => {
                axios.post('/api/broadcasting/auth', {
                    socket_id: socketId,
                    channel_name: channel.name
                })
                .then(response => {
                    callback(false, response.data);
                })
                .catch(error => {
                    callback(true, error);
                });
            }
        };
    },
});
```

Mobile Application Authentication

You may use Sanctum tokens to authenticate your mobile application's requests to your API. The process for authenticating mobile application requests is similar to authenticating third-party API requests; however, there are small differences in how you will issue the API tokens.

Issuing API Tokens

To get started, create a route that accepts the user's email / username, password, and device name, then exchanges those credentials for a new Sanctum token. The endpoint will return the plain-text Sanctum token which may then be stored on the mobile device and used to make additional API requests:

```
use App\User;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Hash;
use Illuminate\Validation\ValidationException;

Route::post('/sanctum/token', function (Request $request) {
    $request->validate([
        'email' => 'required|email',
```

```
        'password' => 'required',
        'device_name' => 'required',
    });

    $user = User::where('email', $request->email)->first();

    if (! $user || ! Hash::check($request->password, $user->password)) {
        throw ValidationException::withMessages([
            'email' => ['The provided credentials are incorrect.'],
        ]);
    }

    return $user->createToken($request->device_name)->plainTextToken;
});
```

When the mobile device uses the token to make an API request to your application, it should pass the token in the `Authorization` header as a Bearer token.

TIP When issuing tokens for a mobile application, you are also free to specify [token abilities](#)

Protecting Routes

As previously documented, you may protect routes so that all incoming requests must be authenticated by attaching the `sanctum` authentication guard to the routes. Typically, you will attach this guard to the routes defined within your `routes/api.php` file:

```
Route::middleware('auth:sanctum')->get('/user', function (Request $request) {
    return $request->user();
});
```

Revoking Tokens

To allow users to revoke API tokens issued to mobile devices, you may list them by name, along with a "Revoke" button, within an "account settings" portion of your web application's UI. When the user clicks the "Revoke" button, you can delete the token from the database. Remember, you can access a user's API tokens via the `tokens` relationship provided by the `HasApiTokens` trait:

```
// Revoke all tokens...
$user->tokens()->delete();

// Revoke a specific token...
$user->tokens()->where('id', $id)->delete();
```

Testing

While testing, the `Sanctum::actingAs` method may be used to authenticate a user and specify which abilities are granted to their token:

```
use App\User;
use Laravel\Sanctum\Sanctum;

public function test_task_list_can_be_retrieved()
{
    Sanctum::actingAs(
        factory(User::class)->create(),
        ['view-tasks']
    );

    $response = $this->get('/api/task');

    $response->assertOk();
}
```

If you would like to grant all abilities to the token, you should include `*` in the ability list provided to the `actingAs` method:

```
Sanctum::actingAs(
    factory(User::class)->create(),
    ['*']
);
```

Official Packages

Scout

Laravel Scout

- [Introduction](#)
- [Installation](#)
 - [Queueing](#)
 - [Driver Prerequisites](#)
- [Configuration](#)
 - [Configuring Model Indexes](#)
 - [Configuring Searchable Data](#)
 - [Configuring The Model ID](#)
 - [Identifying Users](#)
- [Indexing](#)
 - [Batch Import](#)
 - [Adding Records](#)
 - [Updating Records](#)
 - [Removing Records](#)
 - [Pausing Indexing](#)
 - [Conditionally Searchable Model Instances](#)
- [Searching](#)
 - [Where Clauses](#)
 - [Pagination](#)
 - [Soft Deleting](#)
 - [Customizing Engine Searches](#)
- [Custom Engines](#)
- [Builder Macros](#)

Introduction

Laravel Scout provides a simple, driver based solution for adding full-text search to your [Eloquent models](#). Using model observers, Scout will automatically keep your search indexes in sync with your Eloquent records.

Currently, Scout ships with an [Algolia](#) driver; however, writing custom drivers is simple and you are free to extend Scout with your own search implementations.

Installation

First, install Scout via the Composer package manager:

```
composer require laravel/scout:^8.0
```

After installing Scout, you should publish the Scout configuration using the `vendor:publish` Artisan command. This command will publish the `scout.php` configuration file to your `config` directory:

```
php artisan vendor:publish --provider="Laravel\Scout\ScoutServiceProvider"
```

Finally, add the `Laravel\Scout\Searchable` trait to the model you would like to make searchable. This trait will register a model observer to keep the model in sync with your search driver:

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;
use Laravel\Scout\Searchable;

class Post extends Model
{
    use Searchable;
```

```
}
```

Queueing

While not strictly required to use Scout, you should strongly consider configuring a [queue driver](#) before using the library. Running a queue worker will allow Scout to queue all operations that sync your model information to your search indexes, providing much better response times for your application's web interface.

Once you have configured a queue driver, set the value of the `queue` option in your `config/scout.php` configuration file to `true`:

```
'queue' => true,
```

Driver Prerequisites

Algolia

When using the Algolia driver, you should configure your Algolia `id` and `secret` credentials in your `config/scout.php` configuration file. Once your credentials have been configured, you will also need to install the Algolia PHP SDK via the Composer package manager:

```
composer require algolia/algoliasearch-client-php:^2.2
```

Configuration

Configuring Model Indexes

Each Eloquent model is synced with a given search "index", which contains all of the searchable records for that model. In other words, you can think of each index like a MySQL table. By default, each model will be persisted to an index matching the model's typical "table" name. Typically, this is the plural form of the model name; however, you are free to customize the model's index by overriding the `searchableAs` method on the model:

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;
use Laravel\Scout\Searchable;

class Post extends Model
{
    use Searchable;

    /**
     * Get the index name for the model.
     *
     * @return string
     */
    public function searchableAs()
    {
        return 'posts_index';
    }
}
```

Configuring Searchable Data

By default, the entire `toArray` form of a given model will be persisted to its search index. If you would like to customize the data that is synchronized to the search index, you may override the `toSearchableArray` method on the model:

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;
```

```
use Laravel\Scout\Searchable;

class Post extends Model
{
    use Searchable;

    /**
     * Get the indexable data array for the model.
     *
     * @return array
     */
    public function toSearchableArray()
    {
        $array = $this->toArray();

        // Customize array...

        return $array;
    }
}
```

Configuring The Model ID

By default, Scout will use the primary key of the model as the unique ID stored in the search index. If you need to customize this behavior, you may override the `getScoutKey` and the `getScoutKeyName` methods on the model:

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;
use Laravel\Scout\Searchable;

class User extends Model
{
    use Searchable;

    /**
     * Get the value used to index the model.
     *
     * @return mixed
     */
    public function getScoutKey()
    {
        return $this->email;
    }

    /**
     * Get the key name used to index the model.
     *
     * @return mixed
     */
    public function getScoutKeyName()
    {
        return 'email';
    }
}
```

Identifying Users

Scout also allows you to auto identify users when using Algolia. Associating the authenticated user with search operations may be helpful when viewing your search analytics within Algolia's dashboard. You can enable user identification by setting `SCOUT_IDENTIFY` to `true` in your `.env` file:

```
SCOUT_IDENTIFY=true
```

Enabling this feature this will also pass the request's IP address and your authenticated user's primary identifier to Algolia so this data is associated with any search request that is made by the user.

Indexing

Batch Import

If you are installing Scout into an existing project, you may already have database records you need to import into your search driver. Scout provides an `import` Artisan command that you may use to import all of your existing records into your search indexes:

```
php artisan scout:import "App\Post"
```

The `flush` command may be used to remove all of a model's records from your search indexes:

```
php artisan scout:flush "App\Post"
```

Adding Records

Once you have added the `Laravel\Scout\Searchable` trait to a model, all you need to do is save a model instance and it will automatically be added to your search index. If you have configured Scout to [use queues](#) this operation will be performed in the background by your queue worker:

```
$order = new App\Order;

// ...

$order->save();
```

Adding Via Query

If you would like to add a collection of models to your search index via an Eloquent query, you may chain the `searchable` method onto an Eloquent query. The `searchable` method will [chunk the results](#) of the query and add the records to your search index. Again, if you have configured Scout to use queues, all of the chunks will be added in the background by your queue workers:

```
// Adding via Eloquent query...
App\Order::where('price', '>', 100)->searchable();

// You may also add records via relationships...
$user->orders()->searchable();

// You may also add records via collections...
$orders->searchable();
```

The `searchable` method can be considered an "upsert" operation. In other words, if the model record is already in your index, it will be updated. If it does not exist in the search index, it will be added to the index.

Updating Records

To update a searchable model, you only need to update the model instance's properties and save the model to your database. Scout will automatically persist the changes to your search index:

```
$order = App\Order::find(1);

// Update the order...

$order->save();
```

You may also use the `searchable` method on an Eloquent query to update a collection of models. If the models do not exist in your search index, they will be created:

```
// Updating via Eloquent query...
App\Order::where('price', '>', 100)->searchable();

// You may also update via relationships...
$user->orders()->searchable();

// You may also update via collections...
$orders->searchable();
```

Removing Records

To remove a record from your index, `delete` the model from the database. This form of removal is even

compatible with [soft deleted](#) models:

```
$order = App\Order::find(1);  
$order->delete();
```

If you do not want to retrieve the model before deleting the record, you may use the `unsearchable` method on an Eloquent query instance or collection:

```
// Removing via Eloquent query...  
App\Order::where('price', '>', 100)->unsearchable();  
  
// You may also remove via relationships...  
$user->orders()->unsearchable();  
  
// You may also remove via collections...  
$orders->unsearchable();
```

Pausing Indexing

Sometimes you may need to perform a batch of Eloquent operations on a model without syncing the model data to your search index. You may do this using the `withoutSyncingToSearch` method. This method accepts a single callback which will be immediately executed. Any model operations that occur within the callback will not be synced to the model's index:

```
App\Order::withoutSyncingToSearch(function () {  
    // Perform model actions...  
});
```

Conditionally Searchable Model Instances

Sometimes you may need to only make a model searchable under certain conditions. For example, imagine you have `App\Post` model that may be in one of two states: "draft" and "published". You may only want to allow "published" posts to be searchable. To accomplish this, you may define a `shouldBeSearchable` method on your model:

```
public function shouldBeSearchable()  
{  
    return $this->isPublished();  
}
```

The `shouldBeSearchable` method is only applied when manipulating models through the `save` method, queries, or relationships. Directly making models or collections searchable using the `searchable` method will override the result of the `shouldBeSearchable` method:

```
// Will respect "shouldBeSearchable"...  
App\Order::where('price', '>', 100)->searchable();  
  
$user->orders()->searchable();  
  
$order->save();  
  
// Will override "shouldBeSearchable"...  
$orders->searchable();  
  
$order->searchable();
```

Searching

You may begin searching a model using the `search` method. The `search` method accepts a single string that will be used to search your models. You should then chain the `get` method onto the search query to retrieve the Eloquent models that match the given search query:

```
$orders = App\Order::search('Star Trek')->get();
```

Since Scout searches return a collection of Eloquent models, you may even return the results directly from a route or controller and they will automatically be converted to JSON:

```
use Illuminate\Http\Request;

Route::get('/search', function (Request $request) {
    return App\Order::search($request->search)->get();
});
```

If you would like to get the raw results before they are converted to Eloquent models, you should use the `raw` method:

```
$orders = App\Order::search('Star Trek')->raw();
```

Search queries will typically be performed on the index specified by the model's [searchableAs](#) method. However, you may use the `within` method to specify a custom index that should be searched instead:

```
$orders = App\Order::search('Star Trek')
    ->within('tv_shows_popularity_desc')
    ->get();
```

Where Clauses

Scout allows you to add simple "where" clauses to your search queries. Currently, these clauses only support basic numeric equality checks, and are primarily useful for scoping search queries by a tenant ID. Since a search index is not a relational database, more advanced "where" clauses are not currently supported:

```
$orders = App\Order::search('Star Trek')->where('user_id', 1)->get();
```

Pagination

In addition to retrieving a collection of models, you may paginate your search results using the `paginate` method. This method will return a `Paginator` instance just as if you had [paginated a traditional Eloquent query](#):

```
$orders = App\Order::search('Star Trek')->paginate();
```

You may specify how many models to retrieve per page by passing the amount as the first argument to the `paginate` method:

```
$orders = App\Order::search('Star Trek')->paginate(15);
```

Once you have retrieved the results, you may display the results and render the page links using [Blade](#) just as if you had paginated a traditional Eloquent query:

```
<div class="container">
    @foreach ($orders as $order)
        {{ $order->price }}
    @endforeach
</div>

{{ $orders->links() }}
```

Soft Deleting

If your indexed models are [soft deleting](#) and you need to search your soft deleted models, set the `soft_delete` option of the `config/scout.php` configuration file to `true`:

```
'soft_delete' => true,
```

When this configuration option is `true`, Scout will not remove soft deleted models from the search index. Instead, it will set a hidden `__soft_deleted` attribute on the indexed record. Then, you may use the `withTrashed` or `onlyTrashed` methods to retrieve the soft deleted records when searching:

```
// Include trashed records when retrieving results...
$orders = App\Order::search('Star Trek')->withTrashed()->get();

// Only include trashed records when retrieving results...
$orders = App\Order::search('Star Trek')->onlyTrashed()->get();
```

TIP When a soft deleted model is permanently deleted using `forceDelete`, Scout will remove it from the search index automatically.

Customizing Engine Searches

If you need to customize the search behavior of an engine you may pass a callback as the second argument to the search method. For example, you could use this callback to add geo-location data to your search options before the search query is passed to Algolia:

```
use Algolia\AlgoliaSearch\SearchIndex;

App\Order::search('Star Trek', function (SearchIndex $algolia, string $query, array $options) {
    $options['body']['query']['bool']['filter']['geo_distance'] = [
        'distance' => '1000km',
        'location' => ['lat' => 36, 'lon' => 111],
    ];

    return $algolia->search($query, $options);
})->get();
```

Custom Engines

Writing The Engine

If one of the built-in Scout search engines doesn't fit your needs, you may write your own custom engine and register it with Scout. Your engine should extend the `Laravel\Scout\Engines\Engine` abstract class. This abstract class contains eight methods your custom engine must implement:

```
use Laravel\Scout\Builder;

abstract public function update($models);
abstract public function delete($models);
abstract public function search(Builder $builder);
abstract public function paginate(Builder $builder, $perPage, $page);
abstract public function mapIds($results);
abstract public function map(Builder $builder, $results, $model);
abstract public function getTotalCount($results);
abstract public function flush($model);
```

You may find it helpful to review the implementations of these methods on the `Laravel\Scout\Engines\AlgoliaEngine` class. This class will provide you with a good starting point for learning how to implement each of these methods in your own engine.

Registering The Engine

Once you have written your custom engine, you may register it with Scout using the `extend` method of the Scout engine manager. You should call the `extend` method from the `boot` method of your `AppServiceProvider` or any other service provider used by your application. For example, if you have written a `MySQLSearchEngine`, you may register it like so:

```
use Laravel\Scout\EngineManager;

/**
 * Bootstrap any application services.
 *
 * @return void
 */
public function boot()
{
    resolve(EngineManager::class)->extend('mysql', function () {
        return new MySQLSearchEngine;
    });
}
```

Once your engine has been registered, you may specify it as your default Scout driver in your `config/scout.php` configuration file:

```
'driver' => 'mysql',
```

Builder Macros

If you would like to define a custom builder method, you may use the `macro` method on the `Laravel\Scout\Builder` class. Typically, "macros" should be defined within a [service provider's](#) `boot` method:

```
<?php

namespace App\Providers;

use Illuminate\Support\Facades\Response;
use Illuminate\Support\ServiceProvider;
use Laravel\Scout\Builder;

class ScoutMacroServiceProvider extends ServiceProvider
{
    /**
     * Register the application's scout macros.
     *
     * @return void
     */
    public function boot()
    {
        Builder::macro('count', function () {
            return $this->engine->getTotalCount(
                $this->engine()->search($this)
            );
        });
    }
}
```

The macro function accepts a name as its first argument, and a Closure as its second. The macro's Closure will be executed when calling the macro name from a `Laravel\Scout\Builder` implementation:

```
App\Order::search('Star Trek')->count();
```

Official Packages

Socialite

Laravel Socialite

- [Introduction](#)
- [Upgrading Socialite](#)
- [Installation](#)
- [Configuration](#)
- [Routing](#)
- [Optional Parameters](#)
- [Access Scopes](#)
- [Stateless Authentication](#)
- [Retrieving User Details](#)

Introduction

In addition to typical, form based authentication, Laravel also provides a simple, convenient way to authenticate with OAuth providers using [Laravel Socialite](#). Socialite currently supports authentication with Facebook, Twitter, LinkedIn, Google, GitHub, GitLab and Bitbucket.

TIP Adapters for other platforms are listed at the community driven [Socialite Providers](#) website.

Upgrading Socialite

When upgrading to a new major version of Socialite, it's important that you carefully review [the upgrade guide](#).

Installation

To get started with Socialite, use Composer to add the package to your project's dependencies:

```
composer require laravel/socialite
```

Configuration

Before using Socialite, you will also need to add credentials for the OAuth services your application utilizes. These credentials should be placed in your `config/services.php` configuration file, and should use the key `facebook`, `twitter`, `linkedin`, `google`, `github`, `gitlab` or `bitbucket`, depending on the providers your application requires. For example:

```
'github' => [
    'client_id' => env('GITHUB_CLIENT_ID'),
    'client_secret' => env('GITHUB_CLIENT_SECRET'),
    'redirect' => 'http://your-callback-url',
],
```

TIP If the `redirect` option contains a relative path, it will automatically be resolved to a fully qualified URL.

Routing

Next, you are ready to authenticate users! You will need two routes: one for redirecting the user to the OAuth provider, and another for receiving the callback from the provider after authentication. We will access Socialite using the `Socialite` facade:

```
<?php

namespace App\Http\Controllers\Auth;
```

```
use App\Http\Controllers\Controller;
use Laravel\Socialite\Facades\Socialite;

class LoginController extends Controller
{
    /**
     * Redirect the user to the GitHub authentication page.
     *
     * @return \Illuminate\Http\Response
     */
    public function redirectToProvider()
    {
        return Socialite::driver('github')->redirect();
    }

    /**
     * Obtain the user information from GitHub.
     *
     * @return \Illuminate\Http\Response
     */
    public function handleProviderCallback()
    {
        $user = Socialite::driver('github')->user();

        // $user->token;
    }
}
```

The `redirect` method takes care of sending the user to the OAuth provider, while the `user` method will read the incoming request and retrieve the user's information from the provider.

You will need to define routes to your controller methods:

```
Route::get('login/github', 'Auth\LoginController@redirectToProvider');
Route::get('login/github/callback', 'Auth\LoginController@handleProviderCallback');
```

Optional Parameters

A number of OAuth providers support optional parameters in the redirect request. To include any optional parameters in the request, call the `with` method with an associative array:

```
return Socialite::driver('google')
    ->with(['hd' => 'example.com'])
    ->redirect();
```

NOTE When using the `with` method, be careful not to pass any reserved keywords such as `state` or `response_type`.

Access Scopes

Before redirecting the user, you may also add additional "scopes" on the request using the `scopes` method. This method will merge all existing scopes with the ones you supply:

```
return Socialite::driver('github')
    ->scopes(['read:user', 'public_repo'])
    ->redirect();
```

You can overwrite all existing scopes using the `setScopes` method:

```
return Socialite::driver('github')
    ->setScopes(['read:user', 'public_repo'])
    ->redirect();
```

Stateless Authentication

The `stateless` method may be used to disable session state verification. This is useful when adding social authentication to an API:

```
return Socialite::driver('google')->stateless()->user();
```

NOTE Stateless authentication is not available for the Twitter driver, which uses OAuth 1.0 for authentication.

Retrieving User Details

Once you have a user instance, you can grab a few more details about the user:

```
$user = Socialite::driver('github')->user();

// OAuth Two Providers
$token = $user->token;
$refreshToken = $user->refreshToken; // not always provided
$expiresIn = $user->expiresIn;

// OAuth One Providers
$token = $user->token;
$tokenSecret = $user->tokenSecret;

// All Providers
$user->getId();
$user->getNickname();
$user->getName();
$user->getEmail();
$user->getAvatar();
```

Retrieving User Details From A Token (OAuth2)

If you already have a valid access token for a user, you can retrieve their details using the `userFromToken` method:

```
$user = Socialite::driver('github')->userFromToken($token);
```

Retrieving User Details From A Token And Secret (OAuth1)

If you already have a valid pair of token / secret for a user, you can retrieve their details using the `userFromTokenAndSecret` method:

```
$user = Socialite::driver('twitter')->userFromTokenAndSecret($token, $secret);
```

Official Packages

Telescope

Laravel Telescope

- [Introduction](#)
- [Installation](#)
 - [Configuration](#)
 - [Data Pruning](#)
 - [Migration Customization](#)
 - [Dashboard Authorization](#)
- [Upgrading Telescope](#)
- [Filtering](#)
 - [Entries](#)
 - [Batches](#)
- [Tagging](#)
- [Available Watchers](#)
 - [Cache Watcher](#)
 - [Command Watcher](#)
 - [Dump Watcher](#)
 - [Event Watcher](#)
 - [Exception Watcher](#)
 - [Gate Watcher](#)
 - [Job Watcher](#)
 - [Log Watcher](#)
 - [Mail Watcher](#)
 - [Model Watcher](#)
 - [Notification Watcher](#)
 - [Query Watcher](#)
 - [Redis Watcher](#)
 - [Request Watcher](#)
 - [Schedule Watcher](#)
- [Displaying User Avatars](#)

Introduction

Laravel Telescope is an elegant debug assistant for the Laravel framework. Telescope provides insight into the requests coming into your application, exceptions, log entries, database queries, queued jobs, mail, notifications, cache operations, scheduled tasks, variable dumps and more. Telescope makes a wonderful companion to your local Laravel development environment.

Installation

You may use Composer to install Telescope into your Laravel project:

```
composer require laravel/telescope "^3.0"
```

After installing Telescope, publish its assets using the `telescope:install` Artisan command. After installing Telescope, you should also run the `migrate` command:

```
php artisan telescope:install
```

```
php artisan migrate
```

Installing Only In Specific Environments

If you plan to only use Telescope to assist your local development, you may install Telescope using the `--dev` flag:

```
composer require laravel/telescope "^3.0" --dev
```

After running `telescope:install`, you should remove the `TelescopeServiceProvider` service provider registration from your app configuration file. Instead, manually register the service provider in the `register` method of your `AppServiceProvider`:

```
/**
 * Register any application services.
 *
 * @return void
 */
public function register()
{
    if ($this->app->isLocal()) {
        $this->app->register(\Laravel\Telescope\TelescopeServiceProvider::class);
        $this->app->register(TelescopeServiceProvider::class);
    }
}
```

You should also prevent the Telescope package from being [auto-discovered](#) by adding the following to your `composer.json` file:

```
"extra": {
    "laravel": {
        "dont-discover": [
            "laravel/telescope"
        ]
    }
},
```

Migration Customization

If you are not going to use Telescope's default migrations, you should call the `Telescope::ignoreMigrations` method in the `register` method of your `AppServiceProvider`. You may export the default migrations using the `php artisan vendor:publish --tag=telescope-migrations` command.

Configuration

After publishing Telescope's assets, its primary configuration file will be located at `config/telescope.php`. This configuration file allows you to configure your watcher options and each configuration option includes a description of its purpose, so be sure to thoroughly explore this file.

If desired, you may disable Telescope's data collection entirely using the `enabled` configuration option:

```
'enabled' => env('TELESCOPE_ENABLED', true),
```

Data Pruning

Without pruning, the `telescope_entries` table can accumulate records very quickly. To mitigate this, you should schedule the `telescope:prune` Artisan command to run daily:

```
$schedule->command('telescope:prune')->daily();
```

By default, all entries older than 24 hours will be pruned. You may use the `hours` option when calling the command to determine how long to retain Telescope data. For example, the following command will delete all records created over 48 hours ago:

```
$schedule->command('telescope:prune --hours=48')->daily();
```

Dashboard Authorization

Telescope exposes a dashboard at `/telescope`. By default, you will only be able to access this dashboard in the local environment. Within your `app/Providers/TelescopeServiceProvider.php` file, there is a `gate` method. This authorization gate controls access to Telescope in **non-local** environments. You are free to modify this gate as needed to restrict access to your Telescope installation:

```
/**
 * Register the Telescope gate.
 *
 * This gate determines who can access Telescope in non-local environments.
 *
 * @return void
 */
protected function gate()
{
    Gate::define('viewTelescope', function ($user) {
        return in_array($user->email, [
            'taylor@laravel.com',
        ]);
    });
}
```

NOTE You should ensure you change your `APP_ENV` environment variable to `production` in your production environment. Otherwise, your Telescope installation will be publicly available.

Upgrading Telescope

When upgrading to a new major version of Telescope, it's important that you carefully review [the upgrade guide](#).

In addition, when upgrading to any new Telescope version, you should re-publish Telescope's assets:

```
php artisan telescope:publish
```

To keep the assets up-to-date and avoid issues in future updates, you may add the `telescope:publish` command to the `post-update-cmd` scripts in your application's `composer.json` file:

```
{
    "scripts": {
        "post-update-cmd": [
            "@php artisan telescope:publish --ansi"
        ]
    }
}
```

Filtering

Entries

You may filter the data that is recorded by Telescope via the `filter` callback that is registered in your `TelescopeServiceProvider`. By default, this callback records all data in the `local` environment and exceptions, failed jobs, scheduled tasks, and data with monitored tags in all other environments:

```
/**
 * Register any application services.
 *
 * @return void
 */
public function register()
{
    $this->hideSensitiveRequestDetails();

    Telescope::filter(function (IncomingEntry $entry) {
        if ($this->app->isLocal()) {
            return true;
        }

        return $entry->isReportableException() ||
            $entry->isFailedJob() ||
            $entry->isScheduledTask() ||
            $entry->hasMonitoredTag();
    });
}
```

Batches

While the `filter` callback filters data for individual entries, you may use the `filterBatch` method to register a callback that filters all data for a given request or console command. If the callback returns `true`, all of the entries are recorded by Telescope:

```
use Illuminate\Support\Collection;

/**
 * Register any application services.
 *
 * @return void
 */
public function register()
{
    $this->hideSensitiveRequestDetails();

    Telescope::filterBatch(function (Collection $entries) {
        if ($this->app->isLocal()) {
            return true;
        }

        return $entries->contains(function ($entry) {
            return $entry->isReportableException() ||
                $entry->isFailedJob() ||
                $entry->isScheduledTask() ||
                $entry->hasMonitoredTag();
        });
    });
}
```

Tagging

Telescope allows you to search entries by "tag". Often, tags are Eloquent model class names or authenticated user IDs which Telescope automatically adds to entries. Occasionally, you may want to attach your own custom tags to entries. To accomplish this, you may use the `Telescope::tag` method. The `tag` method accepts a callback which should return an array of tags. The tags returned by the callback will be merged with any tags Telescope would automatically attach to the entry. You should call the `tag` method within your `TelescopeServiceProvider`:

```
use Laravel\Telescope\Telescope;

/**
 * Register any application services.
 *
 * @return void
 */
public function register()
{
    $this->hideSensitiveRequestDetails();

    Telescope::tag(function (IncomingEntry $entry) {
        if ($entry->type === 'request') {
            return ['status:'.$entry->content['response_status']];
        }

        return [];
    });
}
```

Available Watchers

Telescope watchers gather application data when a request or console command is executed. You may customize the list of watchers that you would like to enable within your `config/telescope.php` configuration file:

```
'watchers' => [
    Watchers\CacheWatcher::class => true,
    Watchers\CommandWatcher::class => true,
    ...
],
```

Some watchers also allow you to provide additional customization options:

```
'watchers' => [
```

```
        Watchers\QueryWatcher::class => [
            'enabled' => env('TELESCOPE_QUERY_WATCHER', true),
            'slow' => 100,
        ],
        ...
    ],
],
```

Cache Watcher

The cache watcher records data when a cache key is hit, missed, updated and forgotten.

Command Watcher

The command watcher records the arguments, options, exit code, and output whenever an Artisan command is executed. If you would like to exclude certain commands from being recorded by the watcher, you may specify the command in the `ignore` option in your `config/telescope.php` file:

```
'watchers' => [
    Watchers\CommandWatcher::class => [
        'enabled' => env('TELESCOPE_COMMAND_WATCHER', true),
        'ignore' => ['key:generate'],
    ],
    ...
],
```

Dump Watcher

The dump watcher records and displays your variable dumps in Telescope. When using Laravel, variables may be dumped using the global `dump` function. The dump watcher tab must be open in a browser for the recording to occur, otherwise the dumps will be ignored by the watcher.

Event Watcher

The event watcher records the payload, listeners, and broadcast data for any events dispatched by your application. The Laravel framework's internal events are ignored by the Event watcher.

Exception Watcher

The exception watcher records the data and stack trace for any reportable Exceptions that are thrown by your application.

Gate Watcher

The gate watcher records the data and result of gate and policy checks by your application. If you would like to exclude certain abilities from being recorded by the watcher, you may specify those in the `ignore_abilities` option in your `config/telescope.php` file:

```
'watchers' => [
    Watchers\GateWatcher::class => [
        'enabled' => env('TELESCOPE_GATE_WATCHER', true),
        'ignore_abilities' => ['viewNova'],
    ],
    ...
],
```

Job Watcher

The job watcher records the data and status of any jobs dispatched by your application.

Log Watcher

The log watcher records the log data for any logs written by your application.

Mail Watcher

The mail watcher allows you to view an in-browser preview of the emails along with their associated data. You may also download the email as an .eml file.

Model Watcher

The model watcher records model changes whenever an Eloquent created, updated, restored, or deleted event is dispatched. You may specify which model events should be recorded via the watcher's `events` option:

```
'watchers' => [
    Watchers\ModelWatcher::class => [
        'enabled' => env('TELESCOPE_MODEL_WATCHER', true),
        'events' => ['eloquent.created*', 'eloquent.updated*'],
    ],
    ...
],
```

Notification Watcher

The notification watcher records all notifications sent by your application. If the notification triggers an email and you have the mail watcher enabled, the email will also be available for preview on the mail watcher screen.

Query Watcher

The query watcher records the raw SQL, bindings, and execution time for all queries that are executed by your application. The watcher also tags any queries slower than 100ms as `slow`. You may customize the slow query threshold using the watcher's `slow` option:

```
'watchers' => [
    Watchers\QueryWatcher::class => [
        'enabled' => env('TELESCOPE_QUERY_WATCHER', true),
        'slow' => 50,
    ],
    ...
],
```

Redis Watcher

The Redis watcher records all Redis commands executed by your application. If you are using Redis for caching, cache commands will also be recorded by the Redis Watcher.

Request Watcher

The request watcher records the request, headers, session, and response data associated with any requests handled by the application. You may limit your response data via the `size_limit` (in KB) option:

```
'watchers' => [
    Watchers\RequestWatcher::class => [
        'enabled' => env('TELESCOPE_REQUEST_WATCHER', true),
        'size_limit' => env('TELESCOPE_RESPONSE_SIZE_LIMIT', 64),
    ],
    ...
],
```

Schedule Watcher

The schedule watcher records the command and output of any scheduled tasks run by your application.

Displaying User Avatars

The Telescope dashboard displays the user avatar for the user that was logged in when a given entry was saved. By default, Telescope will retrieve avatars using the Gravatar web service. However, you may customize the

avatar URL by registering a callback in your `TelescopeServiceProvider`. The callback will receive the user's ID and email address and should return the user's avatar image URL:

```
use App\User;
use Laravel\Telescope\Telescope;

/**
 * Register any application services.
 *
 * @return void
 */
public function register()
{
    Telescope::avatar(function ($id, $email) {
        return '/avatars/'.User::find($id)->avatar_path;
    });
}
```